

Официальное учебное пособие для самоподготовки

Учебный курс

Microsoft®



Microsoft®

SQL Server® 2008

Разработка баз данных

Тобиаш Тернстрем
Энн Вебер
Майк Хотек

совместно с компанией GrandMasters

Экзамен 70-433

MCTS

РУССКАЯ РЕДАКЦИЯ

Microsoft®



MCTS Self-Paced Training Kit (Exam 70-433): Microsoft® SQL Server® 2008—Database Development

Tobias Thernström
Ann Weber
Mike Hotek
and Grand Masters

*Тобиаш Тернстрем,
Энн Вебер, Майк Хотек
совместно с компанией GrandMasters*

Microsoft®

SQL Server® 2008

Разработка баз данных

Москва 2010

 РУССКАЯ РЕДАКЦИЯ

УДК 681.3.06
ББК 32.973.26-018.2
Т35

Тернстрем, Т.

Т35 Microsoft® SQL Server® 2008. Разработка баз данных. Учебный курс Microsoft®: Пер. с англ. / Т. Тернстрем, Э. Вебер, М. Хотек совместно с компанией GrandMasters. — М.: Издательство «Русская Редакция», 2010. — 496 с.: ил. + CD-ROM
ISBN 978-5-7502-0394-9

Данное официальное пособие Microsoft посвящено разработке баз данных Microsoft SQL Server 2008. Обсуждаются стратегии доступа и извлечение данных, изменение данных с помощью инструкций; таблицы, типы и декларативная целостность данных; сложные запросы, программирование в Microsoft SQL Server на языке T-SQL пользовательских хранимых процедур, функций, триггеров и представлений; методы повышения производительности запросов, расширение функциональных возможностей Microsoft SQL Server за счет использования XML, SQLCLR и Filestream, а также с помощью пространственных типов, полнотекстового поиска и Service Broker. Книга не только является ценным руководством для администраторов баз данных и специалистов по внедрению и поддержке Microsoft SQL Server 2008, но и позволяет самостоятельно подготовиться к сдаче сертификационного экзамена Microsoft 70-433. Прилагаемый компакт-диск содержит электронную версию книги на английском языке, тренировочные тесты, файлы программного кода, базы данных и дополнительные материалы.

Для программистов и администраторов баз данных

УДК 681.3.06
ББК 32.973.26-018.2

© 2009-2012, Translation Russian Edition Publishers.

Authorized Russian translation of the English edition of MCTS Self-Paced Training Kit (Exam 70-433): Microsoft® SQL Server® 2008—Database Development, ISBN 978-0-7356-2639-3 © GrandMasters and Mike Hotek.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

© 2009-2012, перевод ООО «Издательство «Русская редакция».

Авторизованный перевод с английского на русский язык произведения MCTS Self-Paced Training Kit (Exam 70-433): Microsoft® SQL Server® 2008—Database Development, ISBN 978-0-7356-2639-3 © GrandMasters and Mike Hotek.

Этот перевод оригинального издания публикуется и продается с разрешения O'Reilly Media, Inc., которая владеет или распоряжается всеми правами на его публикацию и продажу.

© 2010-2012, оформление и подготовка к изданию, ООО «Издательство «Русская редакция».

Microsoft, а также товарные знаки, перечисленные в списке, расположенном по адресу: <http://www.microsoft.com/about/legal/en/us/IntellectualProperty/Trademarks/EN-US.aspx> являются товарными знаками или охраняемыми товарными знаками корпорации Microsoft в США и/или других странах. Все другие товарные знаки являются собственностью соответствующих фирм. Все названия компаний, организаций и продуктов, а также имена лиц, используемые в примерах, вымышлены и не имеют никакого отношения к реальным компаниям, организациям, продуктам и лицам.

Оглавление

Об авторах	3
Благодарности	5
Введение	7
Использование компакт-диска	7
Как устанавливать учебные базы данных	8
Как установить тренировочные тесты	9
Как пользоваться тренировочными тестами	9
Варианты тренировочных тестов	10
Как деинсталлировать тренировочные тесты	10
Программа Microsoft Certified Professional	11
Техническая поддержка	11
Ознакомительная версия программного обеспечения	12
Глава 1. Извлечение данных	13
Прежде чем начать	13
УРОК 1. Запросы данных	14
Синтаксическая запись инструкции <i>SELECT</i>	14
Манипулирование результирующими наборами	20
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Запросы данных	22
Резюме урока	23
УРОК 2. Соединение связанных таблиц	24
Применение оператора <i>JOIN</i>	24
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Соединение связанных таблиц	29
Резюме урока	31
УРОК 3. Реализация группирующих запросов	31
Работа со статистическими функциями	31
Применение предложения <i>GROUP BY</i>	33
Применение операторов <i>WITH ROLLUP</i> и <i>WITH CUBE</i>	34
Применение статистической функции <i>GROUPING</i>	36

Применение <i>GROUPING SETS</i>	37
Примеры применения <i>GROUPING SETS</i>	37
Применение предложения <i>HAVING</i>	40
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Реализация группирующих запросов	41
Резюме урока	42
УРОК 4. Объединение наборов данных	43
Применение оператора <i>UNION</i>	44
Применение команд <i>EXCEPT</i> и <i>INTERSECT</i>	45
Применение оператора <i>APPLY</i>	46
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Объединение наборов данных	47
Резюме урока	48
УРОК 5. Применение встроенных скалярных функций	49
Применение встроенных скалярных функций.....	49
Примеры встроенных функций.....	50
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Применение встроенных скалярных функций	54
Резюме урока	55
Обзор главы	56
Резюме.....	56
Ключевые термины	57
Сценарии.....	57
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	58
Тренировочный тест	59
Глава 2. Изменение данных — инструкции <i>INSERT</i>, <i>UPDATE</i>, <i>DELETE</i> и <i>MERGE</i>	60
Прежде чем начать.....	61
УРОК 1. Изменение данных с помощью инструкций <i>INSERT</i> , <i>UPDATE</i> и <i>DELETE</i> ...	62
Вставка данных	63
Изменение данных	65
Удаление данных.....	67
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Изменение данных	69
Резюме урока	73
УРОК 2. Расширение функциональных возможностей DML с помощью предложения <i>OUTPUT</i> и инструкции <i>MERGE</i>	73
Применение предложения <i>OUTPUT</i>	74
Применение инструкции <i>MERGE</i>	76
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Реализация расширенных средств DML	79
Резюме урока	82
УРОК 3. Управление транзакциями	83
Общие сведения о транзакциях.....	83
Определение явных транзакций.....	84
Особые сценарии <i>ROLLBACK</i>	84
Сбор данных о транзакциях	85
Общие сведения о блокировках	87
Установка уровней изоляции транзакции	93

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Задание явных транзакций	94
Резюме урока	96
Обзор главы	96
Резюме	96
Ключевые термины	97
Сценарии	97
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	98
Тренировочный тест	99

Глава 3. Таблицы, типы данных и декларативная целостность

данных	100
Прежде чем начать	100
УРОК 1. Работа с таблицами и типами данных	101
Типы данных	101
Основные сведения о таблицах	108
Сжатие	115
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание таблиц и типов данных	115
Резюме урока	120
УРОК 2. Декларативная целостность данных	120
Проверка данных	121
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Реализация ограничений	134
Обзор главы	136
Резюме	137
Ключевые термины	137
Сценарий	137
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	138
Тренировочный тест	139

Глава 4. Сложные запросы

Прежде чем начать	140
УРОК 1. Построение рекурсивных запросов с ОТВ	141
Обобщенные табличные выражения	142
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание рекурсивного ОТВ	144
Резюме урока	145
УРОК 2. Реализация вложенных запросов	145
Несвязанные вложенные запросы	145
Итоги с накоплением	147
Связанные вложенные запросы	148
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Поиск разрывов в последовательности	149
Резюме урока	150
УРОК 3. Применение ранжирующих функций	151
Ранжирование данных	151
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Поиск разрывов в последовательности	153
Резюме урока	155

Обзор главы	155
Резюме.....	156
Ключевые термины	156
Сценарий.....	156
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	157
Тренировочный тест	157

Глава 5. Программирование в Microsoft SQL Server на языке T-SQL пользовательских хранимых процедур, функций, триггеров и представлений158

Прежде чем начать.....	159
УРОК 1. Хранимые процедуры	160
Создание хранимых процедур.....	161
Комментарии к программному коду	162
Переменные, параметры и коды возврата.....	163
Конструкции, управляющие ходом выполнения	166
Сообщения об ошибках	169
Обработка ошибок	170
Выполнение хранимых процедур	174
Среда выполнения модулей.....	176
Курсоры	177
Компиляция и перекомпиляция	180
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание хранимой процедуры	183
Резюме урока	184
УРОК 2. Функции, определенные пользователем.....	185
Системные функции.....	185
Функции, определенные пользователем	186
Получение данных из функции.....	189
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание функций	190
Резюме урока	193
УРОК 3. Триггеры.....	193
Триггеры DML	194
Триггеры DDL	195
Триггеры входа.....	197
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание триггеров	198
Резюме урока	201
УРОК 4. Представления	201
Создание представления	202
Модификация данных с помощью представления	202
Секционированные представления	203
Создание индексируемого представления.....	205
Детерминированность.....	206
Подстановка в запросах.....	206
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание представлений	208
Резюме урока	209

Обзор главы	210
Резюме.....	210
Ключевые термины	210
Сценарий.....	211
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	212
Тренировочный тест	213
Глава 6. Методы повышения производительности запроса	214
Прежде чем начать.....	215
УРОК 1. Настройка запросов.....	215
Оценка производительности запроса	215
Настройка производительности запроса	220
Вложенные запросы без корреляции с внешним запросом.....	223
Табличные функции, определенные пользователем	228
Курсоры	229
Определение запросов, нуждающихся в настройке	230
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Настройка производительности запроса.....	230
Резюме урока	236
УРОК 2. Создание индексов	236
Повышение производительности с помощью индексов покрытия.....	236
Применение кластеризованных индексов	245
Производительности считывания по сравнению с производительностью операции записи	246
Применение вычисляемых столбцов	250
Применение индексированных представлений	254
Анализ применения индексов	256
Секционирование	258
Автоматическая настройка индексов	265
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Индексирование для поддержки запросов.....	265
Резюме урока	274
Обзор главы	274
Резюме.....	274
Ключевые термины	275
Сценарий.....	275
КОНТРОЛЬНОЕ УПРАЖНЕНИЕ.....	275
Тренировочный тест	276
Глава 7. Расширение функциональных возможностей Microsoft SQL Server за счет XML, SQLCLR и Filestream	277
Прежде чем начать.....	278
УРОК 1. Работа с XML.....	278
Извлечение табличных данных в виде XML.....	280
Применение типа данных XML	297
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Применение XML	302
Резюме урока	304

УРОК 2. Применение SQLCLR и <i>Filestream</i>	304
Основы применения SQLCLR.....	305
Объекты, которые можно создать с помощью SQLCLR	310
Что разрешено делать моему коду CLR?	332
Применение <i>Filestream</i>	332
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. SQLCLR.....	334
Резюме урока.....	338
Обзор главы.....	338
Резюме.....	339
Ключевые термины.....	339
Сценарий.....	339
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	340
Тренировочный тест	341
 Глава 8. Расширение функциональных возможностей	
Microsoft SQL Server с помощью пространственных типов,	
полнотекстового поиска и Service Broker	342
Прежде чем начать.....	343
УРОК 1. Реализация пространственных типов данных	345
Общие представления о пространственных типах данных	345
Создание экземпляров пространственных типов данных.....	348
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание экземпляров пространственных	
типов данных	352
Резюме урока.....	354
УРОК 2. Выполнение полнотекстового поиска	354
Обзор полнотекстового поиска.....	355
Настройка полнотекстового поиска	358
Создание полнотекстовых запросов	361
Выявление ошибок полнотекстового поиска.....	361
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Написание полнотекстовых запросов	371
Резюме урока.....	373
УРОК 3. Разработка решений Service Broker	374
Обзор Service Broker.....	374
Создание приложений Service Broker.....	377
Включение компонента Service Broker	379
Настройка компонентов Service Broker.....	381
Отправка и получение сообщений.....	386
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Реализация простого решения Service Broker ...	390
Резюме урока.....	392
Обзор главы.....	392
Резюме.....	392
Ключевые термины.....	393
Сценарии.....	394
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ	395
Тренировочный тест	395

Глава 9. Знакомство со средствами, облегчающими управление Microsoft SQL Server.....	397
Прежде чем начать.....	398
УРОК 1. Интеграция Database Mail.....	398
Обзор компонента Database Mail.....	399
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.....	409
Резюме урока.....	413
УРОК 2. Реализация сценариев средствами Windows PowerShell.....	413
Что такое Windows PowerShell?.....	413
Перемещение по иерархии SQL Server PowerShell.....	414
Применение SQL Server PowerShell для перечисления объектов.....	416
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Применение поставщика SQL Server PowerShell.....	421
Резюме урока.....	422
УРОК 3. Отслеживание изменений данных.....	423
Сравнение отслеживания изменений и CDC.....	423
Настройка отслеживания изменений.....	425
Настройка CDC.....	436
SQL Server Audit.....	444
ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Настройка SQL Server для отслеживания изменений.....	447
Резюме урока.....	452
Обзор главы.....	453
Резюме.....	453
Ключевые термины.....	453
Сценарии.....	454
КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ.....	454
Тренировочный тест.....	455
 Ответы к сценариям.....	 457
Глава 1.....	457
Сценарий 1. Извлечение данных.....	457
Сценарий 2. Группировка данных.....	457
Глава 2.....	458
Сценарий 1. Изменение данных.....	458
Сценарий 2. Применение транзакций.....	458
Глава 3.....	458
Сценарий. Ограничения и типы данных.....	458
Глава 4.....	459
Сценарий. Повышение производительности запроса.....	459
Глава 5.....	459
Сценарий. Повышение производительности приложения.....	459
Глава 6.....	460
Сценарий. Настройка производительности запроса.....	460

Глава 7.....	460
Сценарий. Как следует хранить данные?	460
Глава 8.....	461
Сценарий 1. Создание экземпляров пространственных данных	461
Сценарий 2. Запросы полнотекстового индекса.....	461
Глава 9.....	461
Сценарий 1. Интеграция Windows PowerShell и Database Mail.....	461
Сценарий 2. Отслеживание изменений	462
Словарь терминов.....	463
Системные требования	472
Аппаратные требования	472
Программные требования	472
Приложение. Описание компакт-диска.....	474
Как использовать этот компакт-диск	474
Меню AutoRun CD	474
Электронные обозреватели файлов	474
Что на этом компакт-диске?.....	474
Электронная версия книги "MCTS Self-Paced Training Kit (Exam 70-433): Microsoft SQL Server 2008 Database Development".....	475
Тренировочные тесты	475
Файлы программного кода	475
Базы данных.....	475
Дополнительные главы	475
Информация о технической поддержке	476
Microsoft Learning Technical Support	476
Заявление об отсутствии гарантий: стороннее ПО или ссылки на сторонние Web-сайты.....	476
Предметный указатель	477

Моему мужу Рону и детям, Мелиссе и Скотту, за все терпение и помощь, особенно на завершающем этапе.

—Анна

Марисса, пусть преподавание приносит тебе такое же полное удовлетворение и радость, как и мне.

—Майк

Моей жене Фриде, моей матери Вивеке и памяти моего отца Бертила.

—Тобиас

Об авторах



Тобиас Тернстрем (Tobias Thernström). Более 13 лет занимается Microsoft SQL Server. В настоящее время он принимает участие в разработке компонента SQL Server Engine в отделении Microsoft в Редмонде. Тобиас принимал участие в подготовке нескольких программ сертификаций по SQL Server, предлагаемых корпорацией Microsoft. Он сертифицированный пре-

подаватель (Microsoft Certified Trainer, MCT) и соучредитель Шведской группы пользователей SQL Server (<http://www.sqlug.se>).



Анна Вебер (Ann Weber) — независимый инструктор, консультант и автор из Дублина, штат Огайо, она сертифицированный преподаватель (MCT), сертифицированный Microsoft IT-специалист (Microsoft Certified IT Professional, MCITP) по SQL Server и сертифицированный Microsoft системный инженер (Microsoft Certified Systems Engineer, MCSE). Обладая более

чем 12-летним опытом работы с различными версиями SQL Server, она специализируется на классах и проектах, ориентированных на этот программный продукт. В нескольких проектах Анна тесно сотрудничала с отделом сертификации (Microsoft Certification) и отделом обучения (Microsoft Learning) корпорации Microsoft. Несмотря на то, что она любит преподавательскую работу в классе, ей также нравится делиться знаниями, создавая различные отчеты и разрабатывая курсы для пользователей.



Майк Хотек (Mike Hotek) — вице-президент американской корпорации MHS Enterprises, Inc. и президент филиппинской корпорации FilAm Software Technology. Разработчик приложений в течение почти трех десятилетий и специалист по SQL Server на протяжении почти двух десятилетий, он за эти годы провел консультации для почти 1000 проектов по SQL Server и разработал программные продукты и решения, охватывающие все свойства и функции в SQL Server: реляционные отношения, процессы управления ETL (Extract, Transform, and Load), формирование отчетов, интерактивную аналитическую обработку (Online Analytical Processing, OLAP) и интеллектуальный анализ данных (data mining). Он знаток более 40 языков разработки или платформ, начиная с Cobol, RPG, Fortran и LISP и заканчивая Powerbuilder, Delphi, .NET и PHP. Майк, как автор или соавтор, написал восемь книг, семь из которых посвящены SQL Server, наряду с десятками статей для разных профессиональных журналов. Если он не дает консультации по проектам SQL Server, не выступает на конференциях, не проводит семинары, не разрабатывает программное обеспечение или не читает курсы лекций, его можно найти за станком в его столярной мастерской.

Благодарности

Прежде всего, я хочу поблагодарить мою жену Фриду (Frida), поддерживавшую меня во время работы над книгой. Я также благодарен сотрудникам издательства Microsoft Press за превращение моего шведского языка в английский; я знаю, что это было нелегко. Я получил удовольствие, работая с Кеном (Ken), Мелиссой (Melissa), Лаурой (Laura), ДиЭнн (DeAnn) и вашим замечательным техническим рецензентом, Куртом (Kurt)! Я хотел бы принести благодарности сотрудникам компании GrandMasters за помощь в создании мощной команды, которая сделала возможным появление этой книги. И, наконец, моим замечательным соавторам, Анне и Майку — без вас книги не было бы никогда. Спасибо вам, вы — надежная опора!

Тобиас Тернстрем (Tobias Thernström)

Я хотела бы поблагодарить мою семью и друзей за их поддержку во время работы над этим проектом. Я также благодарна Ричарду Кобылке (Richard Kobylka) и Лайзе Крейсслер (Lisa Kreissler) за веру в меня и предоставление благоприятной возможности. Я была бы несправедлива, если бы не поблагодарила Майка Хотекка (Mike Hotek) за то, что он делился со мной своей мудростью и знаниями, приобретенными во время написания множества книг. Его помощь и чувство юмора помогали мне на протяжении всего проекта. И в заключение мне хотелось бы поблагодарить Лауру Сакерман (Laura Sackerman) за то, что она превратила мой первый проект в издательстве Microsoft Press в логически стройную композицию.

Анна Вебер (Ann Weber)

Я хочу поблагодарить всех наших читателей за многолетнюю поддержку и привязанность. Благодарю вас, мои соавторы, Анна и Тобиас — мой малый вклад не смог бы привести к выходу книги без труда, который вы вложили в нее. Большое спасибо Ричарду (Richard), ДиЭнн (DeAnn) и Курту (Kurt) — ваш многочасовой напряженный труд превратил нашу технотарабарщину в понятный и связный английский язык. Настоящим удовольствием было работать с Мелиссой (Melissa), проводившей бесконечные часы, координируя наши трудно управляемые действия и помогая рождению книги. Лаура (Laura), это наша третья совместная книга, и как всегда работа была наслаждением.

Я глубоко признателен "человеку за сценой", Кену Джонсу (Ken Jones), — даже когда поезд без машиниста мчится по неверному пути, он всегда умудряется прибыть на нужную станцию, и это свидетельство неоценимых усилий и времени, которые Вы, Кен, затрачиваете на каждую книгу Microsoft Press.

Майк Хотек (Mike Hotek)

Введение

Обучающий курс разработан для IT-профессионалов, планирующих сдавать экзамен Exam 70-433 на получение сертификата технического специалиста Microsoft (Microsoft Certified Technology Specialist, MCTS), а также разработчиков, которым нужно знать, как реализовывать, запрашивать и оптимизировать базы данных с помощью Microsoft SQL Server 2008. Предполагается, что к моменту освоения этого обучающего курса вы уже имеете опыт работы в ОС Microsoft Windows и программе SQL Server 2008 или знакомы с предыдущими версиями SQL Server или другой платформой баз данных.

Используя этот обучающий курс, вы узнаете, как:

- ◆ создавать объекты баз данных и управлять ими;
- ◆ запрашивать и изменять данные;
- ◆ повышать производительность запросов;
- ◆ расширять функциональные возможности баз данных с помощью полнотекстового поиска, компонента Service Broker и оболочки SQL Server PowerShell;
- ◆ встраивать компонент Database Mail.

Использование компакт-диска

К данному обучающему курсу прилагается сопроводительный компакт-диск. Он содержит следующие компоненты.

- ◆ *Тренировочные тесты.* Вы можете готовиться к сдаче экзамена 70-433 на получение сертификата, применяя тесты, созданные из набора, содержащего 200 реальных экзаменационных вопросов. Предлагаемые разнообразные тренировочные тесты помогут вам как следует подготовиться к экзамену.
- ◆ *Файлы примеров.* Не все упражнения включают программный код, но для всех упражнений его содержащих, на сопроводительном компакт-диске предлагается один или несколько файлов в папке для соответствующей главы. Вы

можете либо ввести программный код, предложенный в книге, либо открыть в окне запроса соответствующий файл с программным кодом.

- ◆ *eBook*. Электронная версия (eBook) этого обучающего курса (на английском языке) включена для применения в тех случаях, когда вы не хотите носить с собой обычную книгу. eBook представлена в PDF-формате, и ее можно просматривать с помощью программы Adobe Acrobat или Adobe Reader. Вы можете во время чтения электронной книги или работы над упражнениями вырезать из нее и вставлять программный код.
- ◆ *Дополнительные главы*. На компакт-диск помещены дополнительные главы из других книг издательства Microsoft Press, посвященные SQL Server 2008. Эти главы представлены в PDF-формате.

Электронное содержание для читателей электронных книг

Если вы купили только электронное издание данной книги, то можете получить содержание сопроводительного компакт-диска, прилагаемого к печатному изданию книги.

Загружаемый файл содержимого компакт-диска можно найти на Web-сайте <http://go.microsoft.com/fwlink/?LinkId=139187>. Всегда предлагается новейшая версия файла, доступная всем читателям.

Как устанавливать учебные базы данных

ПРЕДУПРЕЖДЕНИЕ

Имена баз данных уникальны в пределах экземпляра SQL Server. Учебные базы данных названы AdventureWorks2008, AdventureWorksDW2008 и Northwind. Если у вас уже есть базы данных с такими именами, необходимо изменить имена либо предлагаемых учебных баз, либо тех баз данных, которые уже есть в вашем экземпляре сервера. Если вы изменили имена учебных баз, необходимо также внести соответствующие изменения всюду, где встречается имя базы данных в этой книге.

Для установки учебных баз данных с сопроводительного компакт-диска на ваш жесткий диск выполните следующие действия.

1. Следуйте инструкциям в разд. "How to: Enable FILESTREAM" ("Как включить FILESTREAM") электронной документации по SQL Server на Web-странице <http://msdn.microsoft.com/ru-ru/library/cc645923.aspx>.
2. Вставьте в ваш дисковод CD-ROM сопроводительный компакт-диск.
3. Перейдите в папку \Databases и затем в выбранный вами каталог.
4. Скопируйте файлы Northwind.mdf и Northwind.ldf в папку C:\Program Files\Microsoft SQL Server\MSSQL10.MSSQLSERVER\MSSQL\DATA.
5. Скопируйте файлы AdventureWorksDW2008_Data.mdf и AdventureWorksDW2008_Log.ldf в папку C:\Program Files\Microsoft SQL Server\MSSQL10.MSSQLSERVER\MSSQL\DATA.

6. Скопируйте файлы AdventureWorks2008_Data.mdf и AdventureWorks2008_Log.ldf с папкой Documents целиком в папку C:\Program Files\Microsoft SQL Server\MSSQL10.MSSQLSERVER\MSSQL\DATA.
7. В среде SQL Server Management Studio подключитесь к вашему экземпляру SQL Server в окне **Object Explorer** (Обозреватель объектов).
8. Щелкните правой кнопкой мыши узел **Databases** (Базы данных) и выберите команду **Attach** (Присоединить).
9. Щелкните мышью кнопку **Add** (Добавить), выберите файл AdventureWorks2008_Data.mdf и щелкните мышью кнопку **OK**.
10. Щелкните мышью кнопку **Add** (Добавить), выберите файл AdventureWorksDW2008_Data.mdf и щелкните мышью кнопку **OK**.
11. Щелкните мышью кнопку **Add** (Добавить), выберите файл Northwind.mdf и щелкните мышью кнопку **OK**.
12. Щелкните мышью кнопку **OK**. Убедитесь в том, что базы данных AdventureWorks2008, AdventureWorksDW2008 и Northwind присоединены к вашему экземпляру.

Как установить тренировочные тесты

Для установки программного обеспечения тренировочных тестов с сопроводительного компакт-диска на ваш жесткий диск выполните следующие действия.

1. Вставьте сопроводительный компакт-диск в ваш дисковод CD-ROM и примите условия лицензионного соглашения, появляющиеся на экране. Вы увидите меню компакт-диска.

Другой вариант установочных инструкций при отключенном автозапуске

Если на экране не появляется меню компакт-диска и условия лицензионного соглашения, возможно, на вашем компьютере отключен автозапуск. Альтернативные инструкции по установке можно найти в файле Readme.txt на сопроводительном компакт-диске.

2. Щелкните кнопкой мыши строку **Practice Tests** (Тренировочные тесты) и следуйте появляющимся на экране инструкциям.

Как пользоваться тренировочными тестами

Для запуска программного обеспечения тренировочных тестов выполните следующие действия.

1. Щелкните мышью кнопку **Start** (Пуск) и выберите последовательность команд **All Programs | Microsoft Press Training Kit Exam Prep**. На экране

появится окно, отображающее все установленные на вашем компьютере комплекты обучающего курса по подготовке к экзамену, предлагаемые издательством Microsoft Press.

- Щелкните дважды кнопкой мыши тест, которым хотите воспользоваться.

Варианты тренировочных тестов

Запустив тест, можно выбрать его выполнение в одном из следующих режимов: **Certification Mode** (Режим сертификации), **Study Mode** (Обучающий режим) или **Custom Mode** (Настраиваемый режим).

- ◆ **Certification Mode** воспроизводит с большой долей подобия сдачу экзамена на получение сертификата. В тесте задано установленное количество вопросов, он хронометрируется, и вы не можете приостановить тест или перезапустить таймер.
- ◆ **Study Mode** формирует тест без учета времени выполнения. В этом режиме вы сможете просматривать правильные ответы и пояснения после ответа на каждый вопрос.
- ◆ **Custom Mode** предоставляет вам полное управление параметрами теста, поэтому вы сможете настроить их, как вам захочется.

Во всех режимах интерфейс, который вы видите, в основном один и тот же, но разные параметры включены или отключены в зависимости от выбранного режима. Когда вы просматриваете ответ на отдельный вопрос тренировочного теста, на экране появляется раздел **References** (Ссылки). В этом разделе указано, где в обучающем курсе вы можете найти информацию, относящуюся к данному вопросу, а также приведены ссылки на другие источники. После щелчка мышью по кнопке **Test Results** (Результаты теста) для подсчета общего количества набранных в тесте баллов можно щелкнуть кнопкой мыши вкладку **Learning Plan** (План обучения), чтобы увидеть список ссылок по каждой теме.

Как деинсталлировать тренировочные тесты

Для деинсталляции программного обеспечения тестов для обучающего курса откройте Control Panel (Панель управления) и воспользуйтесь апплетом **Add Or Remove Programs** (Установка и удаление программ) (в ОС Windows XP и Windows Server 2003) или **Programs And Features** (Программы) (в ОС Windows Vista и Windows Server 2008).

Программа Microsoft Certified Professional

Сертификации Microsoft — это лучший способ проверить ваших специалистов в области современных продуктов и технологий корпорации Microsoft. Экзамены и сопутствующие сертификации разработаны для подтверждения вашей высокой квалификации и мастерства при проектировании и разработке или реализации и технической поддержке решений на основе программных продуктов и технологий Microsoft. Компьютерные профессионалы, прошедшие сертификацию Microsoft, считаются высококвалифицированными специалистами и популярны в промышленных кругах. Сертификация дает множество преимуществ специалисту и работодателям и организациям.

Дополнительная информация о перечне сертификаций Microsoft

Полный перечень сертификаций корпорации Microsoft см. на Web-странице <http://www.microsoft.com/learning/mcp/default.mspx> (или <http://msdn.microsoft.com/ru-ru/bb188199.aspx>).

Техническая поддержка

Были предприняты все возможные усилия для того, чтобы избавить книгу и сопроводительный компакт-диск от неточностей и ошибок. Если у вас есть замечания, вопросы или предложения, касающиеся этой книги или сопроводительного компакт-диска, отправляйте их в издательство Microsoft Press одним из следующих способов:

- ◆ электронной почтой по адресу tkinput@microsoft.com;
- ◆ обычной почтой по адресу

Microsoft Press

Attn: MCTS Self-Paced Training Kit (Exam 70-433): Microsoft SQL Server 2008 — Database

Development, Editor

One Microsoft Way

Redmond, WA 98052-6399

Дополнительные сведения о технической поддержке данной книги и сопроводительного компакт-диска (включая ответы на часто задаваемые вопросы об установке и использовании) см. на Web-сайте <http://www.microsoft.com/learning/support/books>.

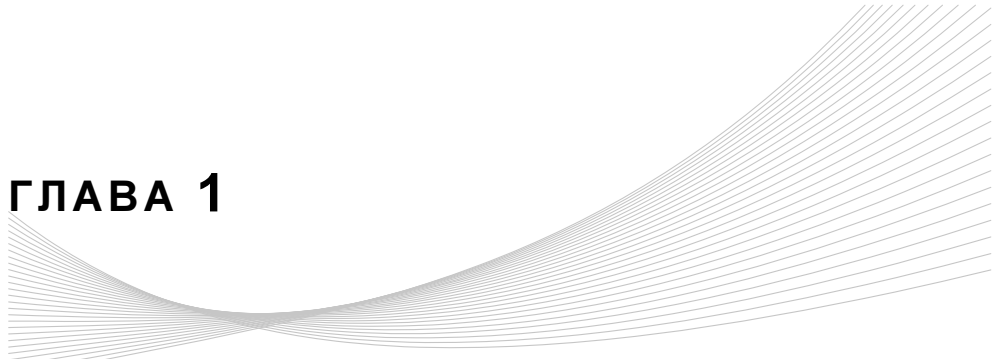
Для прямого подключения к базе знаний Microsoft Knowledge Base и ввода запроса посетите Web-сайт <http://support.microsoft.com/search>. Сведения о техни-

ческой поддержке программного обеспечения Microsoft ищите на Web-сайте <http://support.microsoft.com>.

Ознакомительная версия программного обеспечения

180-дневную ознакомительную версию SQL Server 2008 можно загрузить со страницы <http://www.microsoft.com/sqlserver/2008/ru/ru/trial-software.aspx> Web-сайта Microsoft. Корпорация Microsoft и отдел технической поддержки Microsoft Technical Support не осуществляют техподдержку этой ознакомительной версии.

Сведения о проблемах, относящихся к использованию ознакомительной версии совместно с предлагаемым обучающим курсом, опубликованы в разделе **Support** Web-сайта Microsoft Press (<http://www.microsoft.com/learning/support/books>). За информацией о заказе полной версии программного обеспечения Microsoft обращайтесь в местный отдел продаж корпорации Microsoft или посетите Web-сайт <http://www.microsoft.com>.



ГЛАВА 1

Извлечение данных

Одна из главных задач, которую вам придется решать при работе с базами данных в Microsoft SQL Server, — извлечение данных. Поскольку запросы данных — основной способ обработки баз данных, книга начинается с обсуждения этой важной темы. Извлечение данных выполняется с помощью инструкции `SELECT` с множеством операторов и предложений, расширяющих базовые функциональные возможности простой инструкции `SELECT`.

Экзаменационные темы в этой главе

- ◆ Запросы данных с помощью инструкций `SELECT`.
- ◆ Реализация агрегатных запросов.
- ◆ Объединение наборов данных.
- ◆ Применение встроенных скалярных функций.

Уроки в этой главе

- ◆ Урок 1. Запросы данных.
- ◆ Урок 2. Соединение связанных таблиц.
- ◆ Урок 3. Реализация группирующих запросов.
- ◆ Урок 4. Объединение наборов данных.
- ◆ Урок 5. Применение встроенных скалярных функций.

Прежде чем начать

Для усвоения уроков этой главы необходимо:

- ◆ иметь базовое представление о типах данных SQL Server;
- ◆ знать основные концепции реляционных баз данных;

- ◆ иметь базовые представления о правилах именования схем и объектов, включая составные или многочастные имена, такие как `Schema.Table`;
- ◆ быть знакомым с основами программирования на языке SQL, такими как пакеты, сценарии (или скрипты), циклы и комментарии;
- ◆ знать о том, как открывать и выполнять запросы в среде SQL Server Management Studio (SSMS);
- ◆ должны быть установлены: программа SQL Server 2008 версии Developer, Enterprise или Enterprise Evaluation, учебные базы данных AdventureWorks2008, AdventureWorksDW2008 и Northwind. Базы данных AdventureWorks2008 и AdventureWorksDW2008 можно загрузить с Web-сайта Codeplex

<http://www.codeplex.com/MSFTDBProdSamples/Release/ProjectReleases.aspx?ReleaseId=18407>.

Базу данных Northwind в настоящее время можно найти по адресу

<http://www.microsoft.com/downloads/details.aspx?FamilyID=06616212-0356-46A0-8DA2-EEBC53A68034&displaylang=en>.

В случае базы данных Northwind файл `msi` просто извлечет файлы на ваш жесткий диск. Для присоединения базы данных Northwind используйте SSMS.

Тестовая или учебная база данных

Если не оговаривается особо, в примерах и практических заданиях этой главы применяется база данных AdventureWorks2008.

УРОК 1. Запросы данных

Основная задача базы данных — хранение данных и предоставление доступа к ним. В языке программирования SQL для извлечения данных применяется инструкция `SELECT`.

ПОСЛЕ УСВОЕНИЯ ЭТОГО УРОКА ВЫ СМОЖЕТЕ

- Писать базовые инструкции `SELECT`.
- Манипулировать результирующими наборами данных запроса.

Расчетное время урока — 30 мин.

Синтаксическая запись инструкции *SELECT*

Самая простая инструкция `SELECT` должна включать как минимум предложения `SELECT` и `FROM`. Простейшая инструкция `SELECT` извлекает из таблицы все столбцы и все строки. В следующем примере программного кода извлекаются все строки и столбцы из таблицы `Employee` (Сотрудник) схемы `HumanResources` (Сотрудники компании).

```
SELECT * FROM HumanResources.Employee;
```

Для сокращения результирующего набора и извлечения только относящихся к делу столбцов замените звездочку (*) необходимыми именами столбцов, как показано в приведенном далее примере.

```
SELECT LoginID, JobTitle, BirthDate  
       , MaritalStatus, Gender  
FROM HumanResources.Employee;
```

ПРАКТИЧЕСКИЕ СОВЕТЫ.

Удачные приемы записи программного кода

Если строки разрываются для большей наглядности программного кода, полезно, как показано в приведенном примере, помещать запятую в начало следующей строки, а не оставлять ее в конце предыдущей. В этом случае при тестировании или поиске неполадок будет легче превратить длинные строки в комментарии. Кроме того, этот прием снижает вероятность ошибок синтаксического анализа, вызванных пропущенной или лишней запятой.

Стандартное поведение инструкции `SELECT` — возврат всех строк таблицы или всех строк, соответствующих условию `WHERE`. Если вы хотите исключить строки, имеющие дубликаты, можно применить вариант `SELECT DISTINCT`. На рис. 1.1 приведены все цвета, включенные в таблицу `Production.Product` (Товар). Без ключевого слова `DISTINCT` был бы выведен цвет, связанный с каждой строкой таблицы, что увеличило бы результирующий набор и затруднило его чтение.

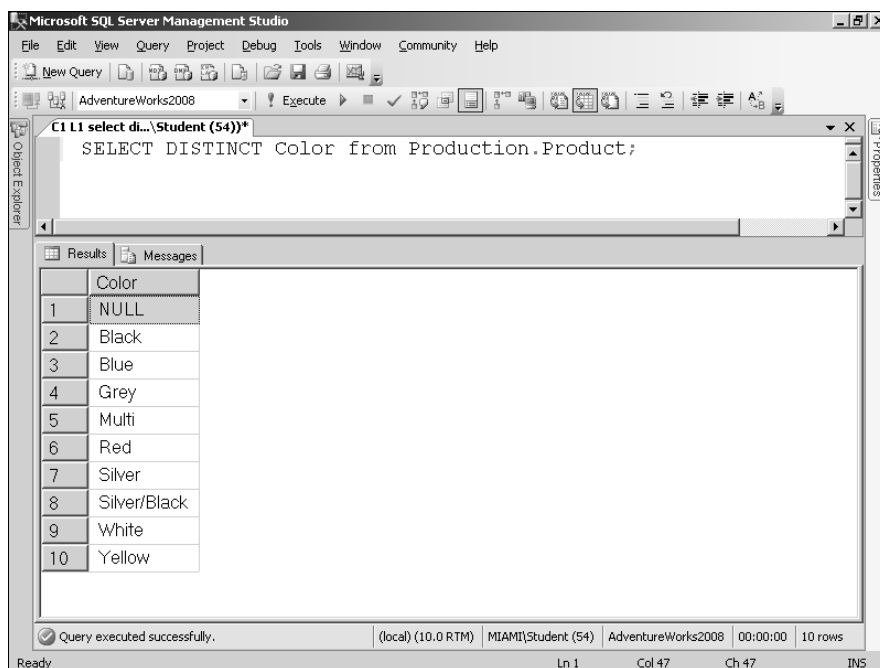


Рис. 1.1. Вариант `SELECT DISTINCT`

Для ограничения количества строк, возвращаемых инструкцией `SELECT`, можно также добавить предложение `WHERE`. Например, если вы хотите просмотреть только цвета и не хотите знать, что в таблице есть строки со значением `NULL`, показанные в предыдущем примере, можно использовать следующий программный код.

```
SELECT DISTINCT Color FROM Production.Product
WHERE Color IS NOT NULL;
```

Работа со значениями `NULL`

Значение `NULL` — это отсутствие данных в столбце. Если у параметра `ANSI_NULLS` установлено значение `ON`, это вариант настройки, принятый по умолчанию, вы не сможете проверить значения на равенство значению `NULL`. Если попытаться это сделать, сервер вернет "unknown" (неизвестно). Например, если бы в предыдущем программном коде вы заменили `IS NOT` на `<>`, то в случае настройки сервера, принятой по умолчанию, запрос не вернул бы ни одной строки. Если выполнить все приведенные далее команды, то будет получен тот же результат, что и в предыдущем примере программного кода.

```
SET ANSI_NULLS OFF;
SELECT DISTINCT Color FROM Production.Product
WHERE Color <> NULL;
```

Задание предложения `WHERE`

Как показано в предыдущем примере, предложение `WHERE` чаще всего используется для ограничения количества строк, возвращаемых в результирующем наборе данных. Включение грамотно написанных предложений `WHERE`, как правило, повышает производительность запроса благодаря ограничению объема данных, которые следует отправить в клиентское приложение.

Дополнительная информация об обработке запроса и его производительности

Более подробную информацию о том, как SQL Server обрабатывает запросы, и о более сложных средствах извлечения данных см. в книге: Itzik Ben-Gan, Lubor Kollar, Steve Kass, Dejan Sarka. Inside Microsoft SQL Server 2008: T-SQL Querying. — Microsoft Press, 2009 (Бен-Ган И. Microsoft SQL Server 2008. Основы T-SQL. — СПб.: БХВ-Петербург, Русская редакция, 2009).

Предложение `WHERE` может включать множество условий поиска, которые могут содержать логические операторы и предикаты, такие как `LIKE`, `BETWEEN`, `EXISTS`, `IS NULL`, `IS NOT NULL` и `CONTAINS`. Как упоминалось ранее, оператор `IS (IS NOT) NULL` возвращает строки, содержащие значение `NULL` в названном столбце. Ключевое слово `CONTAINS` можно применять только тогда, когда создан полнотекстовый индекс для названного столбца. Дополнительную информацию о полнотекстовом поиске см. в главе 8.

Логические операторы, включая `AND`, `OR` и `NOT`, можно использовать для задания нескольких условий в предложении `WHERE`. Следующий пример программного

кода возвращает только товары с атрибутом цвета Silver (серебристый) и отпускной ценой больше 200 долл.:

```
SELECT * FROM Production.Product
WHERE Color = 'Silver' AND ListPrice > 200
```

В следующем примере предполагается, что таблица `Product` (Товар) включает вновь добавленные товары, у которых еще нет отпускной цены. Приведенный далее программный код отбирает товары серебристого цвета с ценой более 200 долл., а также новые товары серебристого цвета с отпускной ценой 0 долл. (цена по умолчанию для вновь добавленных товаров). Если бы не были вставлены круглые скобки, из-за порядка вычисления условий, основанного на приоритете логических операторов, запрос вернул бы товары серебристого цвета с ценой более 200 долл. и товары всех цветов с заявленной ценой 0 долл.

```
SELECT * FROM Production.Product
WHERE Color = 'Silver'
      AND (ListPrice > 200 OR ListPrice = 0)
```

ВАЖНАЯ ИНФОРМАЦИЯ.

Порядок выполнения операторов

Когда механизм управления базой данных проводит синтаксический анализ и компилирует запрос, условия, включающие логические операторы, выполняются в следующем порядке: NOT, AND, OR. Очень важно помнить об этом порядке, в противном случае запрос может вернуть неожиданный результирующий набор. Для управления порядком выполнения операций можно использовать круглые скобки.

РЕАЛЬНАЯ ИСТОРИЯ.

Анна Вебер (Ann Weber)

Много лет назад я обучала студентов программе Microsoft System Management Server (SMS). Один из моих слушателей попытался запросить базу данных SQL, созданную SMS и содержащую всю информацию о клиентских компьютерах в сети. Он попробовал создать отчет, включающий компьютеры с определенными аппаратными характеристиками, на которых еще не была установлена ОС Microsoft Windows 98 (я предупредила, что это было очень давно). Поскольку он применял комбинацию условий с операторами AND, NOT и OR и не понимал, в каком порядке выполняются логические операторы, результаты оказались неправильными, и студент винил SMS, потому что установки ОС по разным причинам (Windows 98 уже установлена, неподходящие аппаратные конфигурации и т. д.) заканчивались аварийно. Я попросила его на следующий день принести свой запрос в класс. Мы добавили несколько скобок в его запрос, и студент забрал его, чтобы проверить в работе. Все установки завершились успешно, т. к. новый запрос возвращал из базы данных корректный набор рабочих станций. Для того чтобы избежать подобных проблем, применяйте следующее правило: "Если сомневаетесь, используйте круглые скобки".

Применение оператора AND обычно приводит к сокращению результирующего набора, тем самым повышая производительность. Оператор NOT обычно снижает производительность, т. к. оптимизатор запроса не может использовать индексы при обработке предложения WHERE, если задан оператор NOT. Что касается индек-

сов в случае применения оператора `OR`, в индекс должны быть включены все столбцы, на которые ссылается оператор `OR`, в противном случае не применяется ни один из индексов.

Логический оператор `LIKE` позволяет вам сопоставлять символьную строку, найденную в столбце, с образцом, заданным в предложении `WHERE`. В операторе `LIKE` используются следующие подстановочные символы.

- ◆ Процент (%) обозначает любое количество символов (включая 0 символов) в строке. Например, `%at` соответствует строкам `at`, `cat`, `hat` и `that`.
- ◆ Знак подчеркивания (_) обозначает точно один символ в строке. Например, `_at` соответствует строкам `cat` и `hat`, но не строкам `that` или `at`.
- ◆ Квадратные скобки ([]) обозначают один символ в множестве или диапазоне символов. Множество часто отображается как явный список символов, например, `[abcd]`, но для большей ясности символы могут быть разделены запятыми, например, `[a,b,c,d]`. Диапазон разделяется дефисом, например `[a-d]`. Каждый из приведенных вариантов включает все строки, в которых задан символ `a`, `b`, `c` или `d`.
- ◆ Знак вставки (^) обозначает любой символ вне множества или диапазона символов. Например, диапазон `[^a-d]` эквивалентен диапазону `[e-z]`.

Производительность при наличии начальных подстановочных знаков или логического оператора NOT

Ни начальные подстановочные знаки, ни логический оператор `NOT` не позволяют оптимизатору запроса использовать индексы для оптимизации поиска. Для достижения оптимальной производительности следует избегать применения ключевого слова `NOT` и начальных подстановочных знаков.

Следующий пример программного кода возвращает всех сотрудников, чья должность начинается со слова `market` (сбыт, торговля). Поскольку мы используем базу данных, не зависящую от регистра символов, ввод прописной буквы не рассматривается как компонент критерия поиска.

```
SELECT BusinessEntityID, JobTitle
FROM HumanResources.Employee
WHERE JobTitle LIKE 'Market%';
```

Для поиска всех сотрудников, в любой части названия должности которых встречается слово `market`, следует заменить условие в операторе `LIKE` на `'%Market%'`.

Результирующий набор (рис. 1.2) содержит всех сотрудников, должности которых начинаются с буквы `c` или `e`.

На рис. 1.3 показан тот же запрос, но в нем вместо множества задан диапазон и возвращаются все сотрудники, название должности которых начинается с букв `c`, `d` или `e`.

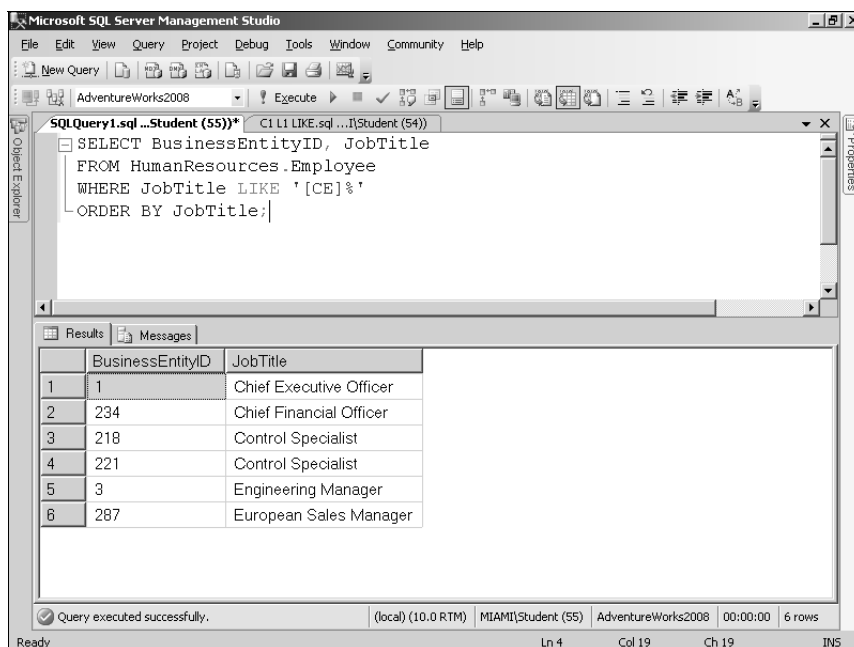


Рис. 1.2. Оператор LIKE, представляющий сопоставление с множеством

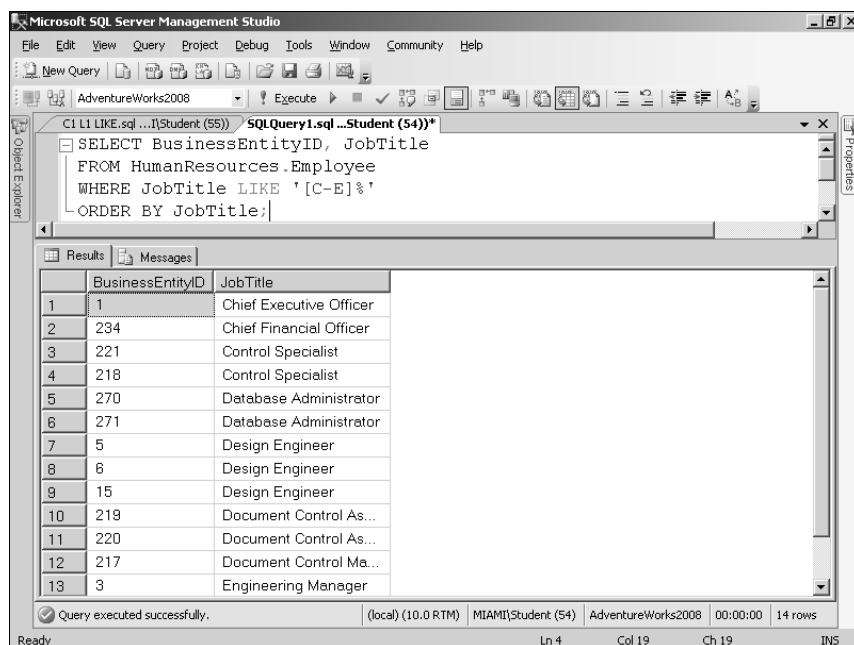


Рис. 1.3. Оператор LIKE, содержащий сопоставление с диапазоном

Оператор `BETWEEN` возвращает все строки, попадающие в диапазон значений. Следующий пример программного кода вернет все строки из таблицы `Production.Product` (Товар), в которых отпускная цена находится между 50 и 80 долл. Обратите внимание на то, что отпускную цену (`ListPrice`) можно применять в предложении `WHERE` даже несмотря на то, что она не упомянута в списке выбора предложения `SELECT`.

```
SELECT ProductNumber, Name, Color
FROM Production.Product
WHERE ListPrice BETWEEN 50 AND 80
```

ВАЖНАЯ ИНФОРМАЦИЯ.

Включающий оператор `BETWEEN`

Важно помнить о том, что оператор `BETWEEN` включает в сопоставление внешние границы заданного диапазона. Например, `BETWEEN 1 AND 5` включает числа, начиная с 1, 1.001, 1.11 и т. д. и заканчивая 4.9, 4.99 и 5.0. Иногда при работе с буквенно-цифровыми полями это может создать путаницу. Если вы запрашиваете столбец с названиями книг и ищите `BETWEEN 'S' AND 'Z'`, будут возвращены все заголовки книг, начинающиеся с S, даже если это единственная буква в заголовке, но на другом конце диапазона возвращаются книги с заголовком просто Z, а заголовок `Zebras 101` не возвращается.

Логический оператор `EXISTS` задает вложенный запрос, который следует применять для определения наличия строк в результирующем наборе подзапроса.

Дополнительная информация о вложенных запросах

Вложенные запросы или подзапросы подробно обсуждаются в *главе 4*.

Манипулирование результирующими наборами

Несмотря на то, что для предоставления отчетов пользователям ваша организация по всей вероятности использует службу `Reporting Services` или другое программное обеспечение, генерирующее отчеты, вам все же приходится обрабатывать результирующие наборы для улучшения их наглядности и программируемости. Добиться этого можно с помощью включения в программный код псевдонимов, строковых констант и операции сцепления или конкатенации.

Вы можете задать псевдоним таблицы, который будет применяться, если нежелательно многократно повторять в запросе ввод имени таблицы. Кроме того, псевдоним таблицы может потребоваться в определенных командах, таких как самосоединения. *Самосоединения* рассматриваются в *уроке 2*. Столбцу тоже можно задать псевдоним для того, чтобы в результирующем наборе отображалось новое имя столбца.

В следующем запросе применяются и псевдонимы столбцов, и псевдоним таблицы.

```
SELECT PP.FirstName AS 'First Name', PP.LastName AS 'Last Name'  
      , PP.PersonType AS 'Person Category'  
FROM Person.Person AS PP  
ORDER BY PersonType
```

В этом запросе используется дополнительное предложение, влияющее на способ отображения результирующего набора. Предложение `ORDER BY` указывает один или несколько столбцов, которые должны применяться для сортировки результирующего набора. В предыдущем запросе используется вариант сортировки по умолчанию `ORDER BY ASC`, который упорядочивает результирующий набор от наименьших значений к наибольшим. Иногда, особенно в случае числовых значений, возможно, вы захотите начать с наибольших значений. Если вы хотите увидеть первыми наибольшие значения, следует применить предложение `ORDER BY DESC`. На сортировку результатов влияют заданные для столбца порядок сортировки и набор параметров символьной обработки (collation).

Дополнительная информация о наборах параметров символьной обработки и порядке сортировки

Дополнительную информацию о наборах параметров символьной обработки и вариантах сортировки см. в главе 3.

При задании списка выбора можно включить в результирующий набор выражения. Для этих выражений можно с помощью псевдонимов задать имена столбцов, содержащих эти выражения. Сцепления (конкатенации) позволяют объединять множественные столбцы и строковые константы в единый столбец. В следующем примере создается один столбец из столбцов таблицы `Person` (Частное лицо) с именем и фамилией. Кроме того, между именем и фамилией вставляется пробел.

```
SELECT PP.FirstName + ' ' + PP.LastName AS 'Name'  
      , PP.PersonType AS 'Category'  
FROM Person.Person AS PP  
ORDER BY PersonType
```

Выражение

В интерактивном справочном руководстве (SQL Server Books Online) и других справочных ресурсах по синтаксису языка Transact-SQL (T-SQL) часто встречается термин *"выражение"*. В SQL Server Books Online выражение определяется как "комбинация символов и операторов, которые вычисляют единственное значение данных".

В таблице `Person` (Частное лицо) столбец `PersonType` (Категория частного лица) содержит значение "ЕМ" для сотрудников. На рис. 1.4 показана инструкция, перестраивающая предыдущую инструкцию и добавляющая столбец с заголовком `Description` (Описание), в котором в каждой строке отображается строка `Employee` (Сотрудник). Запрос также ограничивает результирующий набор, оставляя только строки, содержащие поле `PersonType` (Категория персоны), равное "ЕМ".

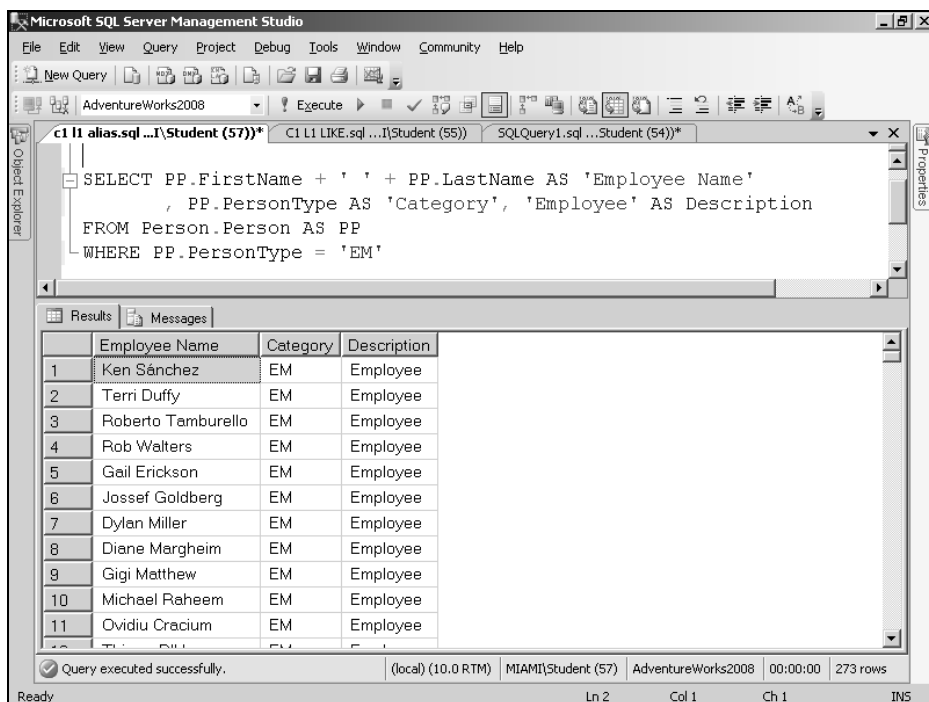


Рис. 1.4. Применение строковой константы

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Запросы данных

На этом практическом занятии вы извлечете данные из таблицы `ProductSubcategory` (Подкатегория товара). Для манипулирования результирующим набором вы примените предложение `WHERE`, оператор `LIKE` и предложение `ORDER BY`.

Упражнение. Написание инструкции `SELECT`

В этом упражнении вы напишите базовую инструкцию `SELECT`, которая извлекает все строки и столбцы из таблицы `ProductSubcategory`. Затем вы модифицируете инструкцию для изменения возвращаемого результирующего набора.

1. Запустите, если необходимо, среду SSMS в вашем экземпляре SQL Server и откройте окно создания запроса.
2. В имеющемся окне запроса введите и выполните следующий программный код для того, чтобы задать базу данных `AdventureWorks2008` и затем вернуть все строки и столбцы таблицы `ProductSubcategory`.

```
USE AdventureWorks2008;
SELECT * FROM Production.ProductSubcategory;
```

3. В существующем окне запроса ниже имеющегося программного кода введите, выделите и выполните следующий код, чтобы вернуть только столбцы ProductSubcategoryID (ID подкатегории товара), ProductCategoryID (ID категории товара), Name (Название) и ModifiedDate (Дата изменения).

```
SELECT ProductSubcategoryID, ProductCategoryID, Name, ModifiedDate
FROM Production.ProductSubcategory;
```

4. В существующем окне запроса ниже имеющегося программного кода введите, выделите и выполните следующий код для возврата строк, у которых в столбце Name (Название) встречается слово bike.

```
SELECT ProductSubcategoryID, ProductCategoryID, Name, ModifiedDate
FROM Production.ProductSubcategory
WHERE Name LIKE '%Bike%';
```

5. В существующем окне запроса ниже имеющегося программного кода введите, выделите и выполните следующий код для того, чтобы ввести псевдоним для столбца Name (Название), поясняющий, что он содержит название подкатегории. Убедитесь в изменении заголовка столбца в результирующем наборе.

```
SELECT ProductSubcategoryID, ProductCategoryID
       , Name AS 'Subcategory Name', ModifiedDate
FROM Production.ProductSubcategory
WHERE Name LIKE '%Bike%';
```

6. В существующем окне запроса ниже имеющегося программного кода введите, выделите и выполните следующий код для сортировки результирующего набора по именам подкатегорий.

```
SELECT ProductSubcategoryID, ProductCategoryID
       , Name AS 'Subcategory Name', ModifiedDate
FROM Production.ProductSubcategory
WHERE Name LIKE '%Bike%'
ORDER BY [Subcategory Name];
```

7. Сохраните сценарий и закройте окно запроса.

8. Оставьте среду SSMS открытой для следующего практического занятия.

Резюме урока

- ◆ Инструкция `SELECT` может применяться для извлечения данных из таблиц или представлений.
- ◆ Результирующий набор инструкции `SELECT` можно отфильтровать, добавив предложение `WHERE`.
- ◆ Результирующий набор инструкции `SELECT` можно отсортировать, применив предложение `ORDER BY`.

- ◆ Сцепление строк, псевдонимы и строковые константы можно использовать для обработки и форматирования результирующего набора.

УРОК 2. Соединение связанных таблиц

В нормализованных базах данных информация, необходимая для одного результирующего набора, может находиться в двух или нескольких таблицах базы данных.

После усвоения этого урока вы сможете

- Писать запросы, использующие операторы `INNER`, `OUTER`, `FULL` и `CROSS JOIN`.
- Объяснить разницу между различными операторами `JOIN`.

Расчетное время урока — 45 мин.

Применение оператора `JOIN`

Оператор `JOIN` позволяет возвращать данные из столбцов, хранящихся в нескольких связанных таблицах. Несмотря на то, что фактические связи, реализованные с помощью создания ограничений `PRIMARY KEY` и `FOREIGN KEY`, не обязательны, для того чтобы результаты были значимыми, необходимо, чтобы по крайней мере один столбец в каждой из таблиц имел одно и то же значение.

Если применяемый в запросе столбец с одним и тем же именем встречается в нескольких таблицах, в списке выбора или при перечислении столбцов в предложении `WHERE` и других предложениях инструкции `SELECT`, следует указывать имя столбца вместе с именем таблицы. Например, столбец `Name` (Название) есть в таблицах `Production.Product`, `Production.Subcategory` и `Production.Category`. Если вы пишете запрос, соединяющий эти таблицы, и хотите включить столбцы `Name` в список выбора или в другую часть запроса, необходимо задавать столбцы как `Production.Product.Name` и т. д. Для того чтобы избавиться от длинного программного кода из-за длинных имен схемы и объектов, можно применять псевдонимы имен таблиц. В следующем примере показано использование псевдонимов.

```
SELECT FirstName, LastName, JobTitle, VacationHours, SickLeaveHours
FROM HumanResources.Employee E INNER JOIN Person.Person P
ON E.BusinessEntityID = P.BusinessEntityID;
```

При определении условия оператора `JOIN` необходимо задать таблицы, которые будут соединяться, тип соединения и условие соединения, формируемого из столбцов, по которым таблицы соединяются, и логического оператора. `INNER JOIN` — это тип соединения, применяемый по умолчанию, когда в запросе задано только ключевое слово `JOIN`.

Характеристика внутренних соединений

Внутренние соединения возвращают только те строки, которые удовлетворяют условию соединения. Несмотря на то, что внутреннее соединение может быть приведено как в предложении `FROM`, так и в предложении `WHERE`, рекомендуется задавать оператор `JOIN` в предложении `FROM`. Все примеры и практические занятия, приведенные в этой книге, следуют данной рекомендации.

Следующий пример возвращает имена и фамилии сотрудников из таблицы `Person.Person` (Частное лицо) и их должности, время, пропущенное по болезни, и время отпуска из таблицы `HumanResources.Employee` (Сотрудник). В этом примере не применяются псевдонимы. Обратите внимание на то, что столбцам в списке выбора не требуются полные имена, поскольку их имена уникальны в обеих таблицах. Имя столбца `BusinessEntityID` (ID делового партнера) используется в обеих таблицах, поэтому его при каждой ссылке следует уточнять с помощью имени таблицы.

```
SELECT FirstName, LastName, JobTitle, VacationHours, SickLeaveHours
FROM HumanResources.Employee INNER JOIN Person.Person
ON HumanResources.Employee.BusinessEntityID =
    Person.Person.BusinessEntityID;
```

Поскольку задано внутреннее соединение, строки из таблицы `HumanResources.Employee`, не имеющие соответствующей строки в таблице `Person.Person`, не возвращаются. Для преодоления такого поведения следует применить оператор `OUTER JOIN`.

Характеристика внешних соединений

Внешнее соединение может применяться для возврата всех строк из одной таблицы и данных из строк второй таблицы, имеющих соответствие строкам первой, или для возврата всех строк из всех таблиц, заданных в предложении `JOIN`. Слово `OUTER` может быть пропущено в синтаксической записи, но обязательно следует указать `LEFT`, `RIGHT` или `FULL`.

Ключевые слова `LEFT` и `RIGHT` можно использовать для задания таблицы, из которой будут возвращены все строки. Когда вы задаете `LEFT OUTER JOIN`, возвращаются все строки из таблицы, указанной слева от ключевого слова `JOIN`. Эта таблица называется *внешней*. Вы можете добиться одного и того же результата с помощью как оператора `LEFT`, так и оператора `RIGHT`, изменив порядок указания имен таблиц. Например, в следующем примере программного кода возвращаются все строки из таблицы `Person.Person` (Частное лицо) наряду с соответствующей информацией из отобранных строк таблицы `HumanResources.Employee` (Сотрудник). На рис. 1.5 приведен результирующий набор этого запроса. Обратите внимание на значения `NULL` в столбцах `JobTitle` (Должность), `VacationHours` (Количество часов отпуска) и `SickLeaveHours` (Количество часов пропусков по болезни) из таблицы `HumanResources.Employee`. Этих значений нет в наличии для строк

таблицы `Person.Person`, не имеющих соответствующей информации в таблице `HumanResources.Employee`. Поскольку в таблице `HumanResources.Employee` для названных столбцов не допустимы значения `NULL`, становится понятно, какие строки были добавлены в результирующий набор при изменении типа соединения с `INNER JOIN` на `RIGHT OUTER JOIN`. Если бы значения `NULL` были разрешены, вы не смогли бы только по наличию значений `NULL` выяснить, какие строки были добавлены при изменении типа соединения.

```
SELECT FirstName, LastName, JobTitle, VacationHours, SickLeaveHours
FROM HumanResources.Employee E RIGHT OUTER JOIN Person.Person P
ON E.BusinessEntityID = P.BusinessEntityID;
```

Тот же результат можно получить с помощью типа соединения `LEFT OUTER JOIN`, если изменить порядок указания имен таблиц, как показано в следующем запросе.

```
SELECT FirstName, LastName, JobTitle, VacationHours, SickLeaveHours
FROM Person.Person P LEFT OUTER JOIN HumanResources.Employee E
ON E.BusinessEntityID = P.BusinessEntityID;
```

	FirstName	LastName	JobTitle	VacationHo...	SickLeaveHo...
1	Syed	Abbas	Pacific Sales Manager	20	30
2	Catherine	Abel	NULL	NULL	NULL
3	Kim	Abercrombie	NULL	NULL	NULL
4	Kim	Abercrombie	NULL	NULL	NULL
5	Kim	Abercrombie	Production Technician - WC60	24	32
6	Hazem	Abolrous	Quality Assurance Manager	80	60
7	Sam	Abolrous	NULL	NULL	NULL
8	Humberto	Acevedo	NULL	NULL	NULL
9	Gustavo	Achong	NULL	NULL	NULL
10	Pilar	Ackerman	NULL	NULL	NULL
11	Pilar	Ackerman	Shipping and Receiving Supervisor	93	66
12	Aaron	Adams	NULL	NULL	NULL
13	Adam	Adams	NULL	NULL	NULL

Query executed successfully. (local) (10.0 RTM) | MIAMI\Student (54) | AdventureWorks2008 | 00:00:00 | 19972 rows

Рис. 1.5. Результирующий набор `RIGHT OUTER`

Оператор `FULL OUTER JOIN` отображает каждую строку обеих таблиц из предложения `JOIN`. Это может быть полезно для поиска не имеющих соответствия строк, если в таблицах не реализована реляционная (ссылочная) целостность. Если в столбце соединения есть ограничения в виде внешнего ключа, оператор `FULL OUTER JOIN` обеспечивает тот же результат, что и оператор `LEFT OUTER JOIN` с таблицей, в которой внешний ключ определен слева от ключевого слова `JOIN`. Это объясняется тем, что для соблюдения ограничения у столбца внешнего ключа должна быть соответствующая строка для столбца первичного ключа. Кроме того, когда заданы ограничения внешнего ключа, оператор `OUTER JOIN`, опреде-

ленный для таблицы с первичным ключом, заданной как внешняя таблица, обеспечивает те же результаты, что и оператор `INNER JOIN`.

Дополнительная информация об ограничениях

Дополнительную информацию об ограничениях см. в главе 3.

Характеристика перекрестных соединений

Перекрестное соединение формирует результат, называемый декартовым произведением двух таблиц. В предложении `JOIN` каждая строка из первой таблицы соединяется с каждой строкой из второй таблицы. Этот тип соединения применяется лишь в нескольких ситуациях. Синтаксическая запись оператора `CROSS JOIN` такая же, как у других типов соединений.

Работа с несколькими таблицами

Для доступа к столбцам, необходимым вашему запросу, вы можете соединить более двух таблиц. Главная практическая рекомендация — стараться избегать в операциях `JOIN` использования более четырех или пяти таблиц. Во избежание проблем, связанных с долгим выполнением и ресурсоемкими запросами, новые инструкции с оператором `JOIN`, особенно содержащие большое число таблиц или строк, всегда следует тестировать на непроизводственном (неосновном) сервере.

Реальная история.

Анна Вебер (Ann Weber)

Я работала с небольшой группой в крупном университете, который предлагал специальные курсы для преподавателей, вспомогательного персонала и иногда студентов университета. Было куплено приложение с базой данных для управления расписанием учебных курсов, включающим зачисления студентов, преподавателей, аудитории и технические средства. База данных, созданная приложением, была полностью нормализована и защищена от внесения в схему изменений. Координатор обучения захотела написать запрос, который возвращал бы номер учебного курса, его название, назначенную аудиторию, назначенного преподавателя и количество оставшихся свободных мест. Для этого ей понадобилось соединить 11 таблиц, такова была структура базы данных. Поскольку у координатора были необходимые права, она разработала Web-приложение, содержащее инструкцию `SELECT`, соединяющую 11 таблиц. По многим причинам, включая ограниченные ресурсы сервера, дополнительные накладные расходы, вызванные применением Web-приложения, и блокирующую конкуренцию, вызванную ее запросом, во время его выполнения было создано множество проблем с несколькими базами данных, размещенными на этом сервере. После этой ситуации она временно потеряла права на написание новых запросов, обращенных непосредственно к рабочему серверу. К счастью, вскоре после этого университет обновил свою базу данных с помощью SQL Server 2005, и компания, у которой они купили базу данных, добавила им права на создание индексированных представлений, которые помогли решить проблему 11-табличного соединения.

Самый распространенный тип операции `JOIN`, включающей более двух таблиц, — `INNER JOIN`. В следующем примере дополняется приведенный ранее в

этом уроке пример с INNER JOIN, в него включается адрес электронной почты из таблицы Person.EmailAddress (Адрес электронной почты).

```
SELECT FirstName, LastName, JobTitle, VacationHours
       , SickLeaveHours, EmailAddress
FROM HumanResources.Employee INNER JOIN Person.Person
    ON HumanResources.Employee.BusinessEntityID =
       Person.Person.BusinessEntityID
INNER JOIN Person.EmailAddress
    ON Person.Person.BusinessEntityID =
       Person.EmailAddress.BusinessEntityID
```

ПРЕДУПРЕЖДЕНИЕ.

Работа с несколькими таблицами

Работая с оператором OUTER JOIN, включающим более двух таблиц, нужно с особой тщательностью проверять результирующие наборы. Компонент Database Engine создает временный результирующий набор в первой операции JOIN и затем использует его в следующей операции JOIN. В зависимости от указанного порядка и заданных операторов вы можете получить разные результаты.

Характеристика самосоединения

Самосоединение (self-join) происходит, когда в предложении JOIN несколько раз указывается одна и та же таблица, т. к. в этом случае для каждого упоминания таблицы используются разные псевдонимы. Самосоединения полезны, например, когда ваша база данных содержит таблицу, ссылающуюся на саму себя.

В следующем примере из базы данных AdventureWorksDW2008 отображаются имя и фамилия каждого сотрудника с именем и фамилией его или ее непосредственного начальника. Для облегчения понимания запроса применяются псевдонимы ("E" для ссылки на данные сотрудника и "DS" для ссылки на сведения о непосредственном начальнике). Поле ParentEmployeeKey в записи каждого сотрудника указывает на поле непосредственного начальника EmployeeKey.

```
SELECT E.FirstName + ' ' + E.LastName AS 'Employee Name'
       , DS.FirstName + ' ' + DS.LastName AS 'Direct Supervisor'
FROM DimEmployee E INNER JOIN DimEmployee DS
    ON E.ParentEmployeeKey = DS.EmployeeKey;
```

Поскольку эта таблица включает начальников разных уровней, вы также можете применить *обобщенное табличное выражение* (OTB) (common table expression, CTE) для формирования рекурсивного результата.

Дополнительная информация об обобщенных табличных выражениях

Дополнительную информацию об OTB см. в главе 4.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Соединение связанных таблиц

На этом практическом занятии вы напишите запросы, расширяющие запрос, разработанный на практическом занятии *урока 1*. Результирующий запрос включает в себя названия подкатегорий и категорий товаров, входящих в подкатегорию, содержащую в своем названии слово *bike* (велосипед).

Кроме того, вы напишите запрос, соединяющий таблицу `PurchaseOrderHeader` (Общие сведения о заказе на покупку) с самой собой для получения списка пар заказов на покупку. Каждая строка будет содержать два заказа на покупку с одним и тем же поставщиком и одинаковыми способами доставки.

Упражнение 1. Соединение таблиц

В этом упражнении вы будете работать с таблицами `Product` (Товар), `ProductCategory` (Категория товара) и `ProductSubcategory` (Подкатегория товара). Вы улучшите этот запрос так, чтобы он включал названия товара, подкатегории и категории тех товаров в названии подкатегории которых содержится слово *bike*.

1. Откройте окно создания запроса в среде SSMS.
2. В открытом окне введите и затем выполните следующую команду для возврата данных из таблицы `ProductSubcategory`, в которых название подкатегории содержит слово *bike*.

```
USE AdventureWorks2008;
SELECT ProductSubcategoryID, ProductCategoryID
       , Name AS 'Subcategory Name'
FROM Production.ProductSubcategory
WHERE Name LIKE '%Bike%'
ORDER BY [Subcategory Name];
```

3. Обратите внимание на то, что результирующий набор включает в себя номер категории товара, но не название категории. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий программный код для соединения таблиц `ProductCategory` и `ProductSubcategory` и извлечения столбца `Name` (Название) из таблицы `ProductCategory`. Применяется внутреннее соединение, т. к. вы ищете только сопоставимые строки в обеих таблицах.

```
SELECT ProductSubcategoryID
       , Production.ProductCategory.ProductCategoryID
       , Production.ProductSubcategory.Name AS 'Subcategory Name'
       , Production.ProductCategory.Name AS 'Category Name'
FROM Production.ProductSubcategory
INNER JOIN Production.ProductCategory
```



```
ON Production.ProductCategory.ProductCategoryID =
    Production.ProductSubcategory.ProductCategoryID
WHERE Production.ProductSubcategory.Name LIKE '%Bike%'
ORDER BY [Subcategory Name];
```

4. Теперь добавим таблицу `Product` для того, чтобы вы могли видеть, какие товары имеются в каждой из этих подкатегорий. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните приведенный далее программный код для соединения таблицы товаров с имеющимся запросом.

```
SELECT P.ProductID AS 'Product ID'
    , PSC.ProductSubcategoryID AS 'Subcategory ID'
    , PC.ProductCategoryID AS 'Category ID'
    , P.Name AS 'Product Name'
    , PSC.Name AS 'Subcategory Name'
    , PC.Name AS 'Category Name'
FROM Production.ProductSubcategory AS PSC
INNER JOIN Production.ProductCategory AS PC
ON PC.ProductCategoryID = PSC.ProductCategoryID
INNER JOIN Production.Product AS P
ON P.ProductSubcategoryID = PSC.ProductSubcategoryID
WHERE PSC.Name LIKE '%Bike%'
ORDER BY [Subcategory Name];
```

Упражнение 2. Выполнение самосоединения

В этом упражнении вы сформируете запрос, который возвращает уникальные пары заказов на покупку, содержащие одного и того же поставщика и один и тот же способ доставки. Одинаковые пары идентификаторов заказов на покупку не должны включаться в отчет несколько раз.

1. Откройте окно создания запроса.
2. В открытом окне запроса введите и выполните следующую команду для возврата пар заказов на покупку с одним и тем же поставщиком и одинаковыми способами доставки.

```
SELECT a.PurchaseOrderID, b.PurchaseOrderID,
    , a.VendorID, a.ShipMethodID
FROM Purchasing.PurchaseOrderHeader AS a
JOIN Purchasing.PurchaseOrderHeader AS b
ON a.VendorID = b.VendorID AND a.ShipMethodID = b.ShipMethodID
WHERE a.PurchaseOrderID < b.PurchaseOrderID;
```

3. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий программный код, заменив для облегчения чтения результирующего набора идентификаторы поставщиков и методов доставки их именами и названиями.

```
SELECT a.PurchaseOrderID, b.PurchaseOrderID, v.Name, s.Name
FROM Purchasing.PurchaseOrderHeader AS a
JOIN Purchasing.PurchaseOrderHeader AS b
  ON a.VendorID = b.VendorID AND a.ShipMethodID = b.ShipMethodID
JOIN Purchasing.Vendor AS v
  ON a.VendorID = v.BusinessEntityID
JOIN Purchasing.ShipMethod AS s
  ON a.ShipMethodID = s.ShipMethodID
WHERE a.PurchaseOrderID < b.PurchaseOrderID;
```

4. Сохраните сценарий и закройте окно запроса. Оставьте открытой среду SSMS до следующего практического занятия.

Резюме урока

- ◆ Предложение `JOIN` позволяет извлекать столбцы из связанных таблиц и группировать их в единый результирующий набор.
- ◆ К типам соединений относятся следующие: `INNER`, `LEFT OUTER`, `RIGHT OUTER`, `FULL OUTER` и `CROSS`.
- ◆ Операторы `JOIN` могут соединять более двух таблиц.
- ◆ Таблицу можно соединить саму с собой, задавая для каждой ссылки на таблицу разные псевдонимы.

УРОК 3. Реализация группирующих запросов

Статистические функции позволяют выполнять вычисления значений в столбце. Вставка предложения `GROUP BY` дает возможность группировать наборы данных.

После усвоения этого урока вы сможете

- Рассказать о назначении имеющихся статистических функций.
- Формировать сводные данные с помощью предложения `GROUP BY`.

Расчетное время урока — 45 мин.

Работа со статистическими функциями

Статистические функции выполняют обработку наборов данных и возвращают скалярное (единственное) значение. В SQL Server 2008 есть следующие статистические функции.

- ◆ `AVG` возвращает среднее арифметическое всех значений набора данных.
- ◆ `CHECKSUM_AGG` возвращает контрольную сумму всех значений в наборе данных.

- ◆ `COUNT` возвращает количество значений, содержащихся в наборе данных. `COUNT(*)` возвращает число строк в наборе. Если задан столбец, например `COUNT(FaxNo)`, возвращаемое значение отражает количество строк, содержащее данные в указанном столбце. Значения `NULL` игнорируются. Кроме того, `COUNT DISTINCT` возвращает количество уникальных, отличных от `NULL` значений в наборе данных.
- ◆ `COUNT_BIG` действует так же, как `COUNT`, но возвращает тип данных `bigint`, в то время как `COUNT` возвращает только тип данных `int`.
- ◆ `GROUPING` возвращает 1 или 0 и идентифицирует сводные и несводные строки в случае применения предложения `GROUP BY`. Значение 1 означает сводную строку, а 0 — строку с подробными данными.
- ◆ `MAX` возвращает наибольшее значение в наборе для полей числового типа, дат и полей на основе символов.
- ◆ `MIN` возвращает наименьшее значение в наборе для полей числового типа, дат и полей на основе символов.
- ◆ `SUM` возвращает сумму значений в наборе данных. Вы можете задать `ALL` или `DISTINCT` для получения суммы всех значений или только отличающихся значений в наборе данных.
- ◆ `STDEV` возвращает статистическое среднееквадратичное отклонение значений в наборе данных.
- ◆ `STDEVP` возвращает статистическое среднееквадратичное отклонение для генеральной совокупности из значений набора данных.
- ◆ `VAR` возвращает статистическую дисперсию значений в наборе данных.
- ◆ `VARP` возвращает статистическую дисперсию для генеральной совокупности из значений набора данных.

Значения `NULL` игнорируются всеми статистическими функциями. Это важно знать при проверке корректности интерпретации данных. Например, в таблице с результатами теста, когда применяются статистические функции `AVG` или `MIN`, между `NULL` и 0 существует большая разница. Если тест пропущен и не должен включаться в вычисление, значения `NULL` не оказывают негативного влияния на точность расчетов, но если тест должен включаться в подсчет среднего значения, средства проверки целостности базы данных должны убедиться в том, что вместо значения `NULL` введен 0.

Следующий пример возвращает среднюю, максимальную и минимальную отпускные цены всех товаров в таблице `Production.Product` (Товар). У новых товаров, которым еще не назначена цена, или товаров, которые не продаются потребителям, отпускная цена равна 0. Для получения более точных итоговых значений такие товары удаляются из результирующего набора.

```
SELECT AVG(Listprice) AS 'Average', MIN(Listprice) AS 'Minimum'
      , MAX(Listprice) AS 'Maximum'
FROM Production.Product
WHERE ListPrice <> 0;
```

Применение предложения **GROUP BY**

В запросы, содержащие статистические функции, часто включается предложение **GROUP BY**. Если в предложение **SELECT** включена статистическая функция, то и все остальные выражения в предложении **SELECT** также должны быть статистическими функциями или включаться в предложение **GROUP BY**.

Предложение **GROUP BY** позволяет определять промежуточные итоги для сгруппированных данных. Например, следующая команда возвращает среднюю, минимальную и максимальную отпускные цены для товаров, принадлежащих к каждой подкатегории товаров.

```
SELECT Production.Product.ProductSubcategoryID
      , AVG(Listprice) AS 'Average', MIN(Listprice) AS 'Minimum'
      , MAX(Listprice) AS 'Maximum'
FROM Production.Product
WHERE ListPrice <> 0
GROUP BY Product.ProductSubcategoryID;
```

Результирующий набор приведенного запроса показан на рис. 1.6. Верхняя строка, в которой поле **ProductSubcategoryID** (ID подкатегории товара) отображено

	ProductSubcategoryID	Average	Minimum	Maximum
1	NULL	159.1333	133.34	196.92
2	1	1683.365	539.99	3399.99
3	2	1597.45	539.99	3578.27
4	3	1425.2481	742.35	2384.07
5	4	73.89	44.54	120.27
6	5	92.24	53.99	121.49
7	6	106.50	106.50	106.50
8	7	20.24	20.24	20.24
9	8	278.99	175.49	404.99
10	9	106.475	91.49	121.46
11	10	184.40	148.22	229.49
12	11	87.0733	34.20	124.73
13	12	678.2535	249.79	1364.50
14	13	64.0185	40.49	80.99
15	14	780.0436	337.22	1431.50
16	15	39.6333	27.12	52.64

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (58) AdventureWorks2008 00:00:00 38 rows

Рис. 1.6. Результирующий набор базового примера с предложением **GROUP BY**

как NULL, — это итоговая строка, содержащая среднюю, минимальную и максимальную отпускные цены товаров из всех подкатегорий.

Применение операторов *WITH ROLLUP* и *WITH CUBE*

Для того чтобы иметь возможность выводить промежуточные итоги по нескольким столбцам, можно добавить оператор `WITH ROLLUP` или `WITH CUBE`. Эти операторы предоставляют общий итог наряду с промежуточными итогами, базирующимися на столбцах, включенных в предложение `GROUP BY`. Порядок, в котором задаются столбцы, изменяет итоговые данные, возвращаемые в результирующем наборе с оператором `WITH ROLLUP`.

Например, показанный здесь запрос возвращает минимальные и максимальные отпускные цены для каждой подкатегории, входящей в каждую категорию. Поскольку столбцы с идентификаторами подкатегории и категории перечислены в предложении `SELECT`, они должны быть приведены и в предложении `GROUP BY`. Столбец `Production.ProductCategory.ProductCategoryID` (ID категории товара) указан первым в предложении `GROUP BY`, поэтому в результирующий набор включена итоговая строка, отображающая заданные сводные значения для всех подкатегорий, входящих в данную категорию. Если бы первым был приведен столбец `Product.ProductSubcategoryID` (ID подкатегории товара), итоговые строки возвращали бы для данной подкатегории сводные значения по всем категориям. В данных примера каждая подкатегория относится только к одной категории, что снижает полезность полученного результирующего набора.

```
SELECT Production.ProductCategory.ProductCategoryID
       , Production.Product.ProductSubcategoryID
       , AVG(Listprice) AS 'Average', MIN(Listprice) AS 'Minimum'
       , MAX(Listprice) AS 'Maximum'
FROM Production.Product
JOIN Production.ProductSubcategory
  ON Production.ProductSubcategory.ProductSubcategoryID =
     Production.Product.ProductSubcategoryID
JOIN Production.ProductCategory
  ON Production.ProductSubcategory.ProductCategoryID =
     Production.ProductCategory.ProductCategoryID
WHERE ListPrice <> 0
GROUP BY Production.ProductCategory.ProductCategoryID
       , Product.ProductSubcategoryID
WITH ROLLUP
```

Часть результирующего набора, показанная на рис. 1.7, в первой приведенной строке (строка 27), отображает среднюю, минимальную и максимальную отпускные цены для подкатегории товаров 25. Средняя, минимальная и максимальная цены для категории 3 приведены в строке 28 и помечены значением NULL в столбце `ProductSubcategoryID`. Средняя, минимальная и максимальная цены по

всем категориям и подкатегориям отображены в строке 42 и помечены значениями NULL в обоих столбцах ProductCategoryID и ProductSubcategoryID. Если бы оператор WITH ROLLUP не был задан, итоговые строки 28 и 41 не были бы включены в результирующий набор.

	ProductCategoryID	ProductSubcategoryID	Average	Minimum	Maximum
27	3	25	63.50	63.50	63.50
28	3	NULL	50.9914	8.99	89.99
29	4	26	120.00	120.00	120.00
30	4	27	159.00	159.00	159.00
31	4	28	7.99	4.99	9.99
32	4	29	7.95	7.95	7.95
33	4	30	21.98	21.98	21.98
34	4	31	34.99	34.99	34.99
35	4	32	54.99	54.99	54.99
36	4	33	31.3233	13.99	44.99
37	4	34	25.00	25.00	25.00
38	4	35	125.00	125.00	125.00
39	4	36	22.49	19.99	24.99
40	4	37	19.4827	2.29	35.00
41	4	NULL	34.3489	2.29	159.00
42	NULL	NULL	744.5952	2.29	3578.27

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (59) AdventureWorks2008 00:00:00 42 rows

Рис. 1.7. Результаты действия предложения GROUP BY с оператором WITH ROLLUP

Совместимость CHECKSUM_AGG

Статистическая функция CHECKSUM_AGG не совместима с операторами ROLLUP, CUBE или GROUPING SETS.

Оператор WITH CUBE можно применять, если нужно включить в результат итоговую информацию по нескольким столбцам. Поскольку в данном примере каждая подкатегория входит только в одну категорию, оператор WITH CUBE не имеет смысла. Но если бы я перечислял среднюю цену каждого товара в конкретном заказе и из каждой конкретной подкатегории, я мог бы использовать оператор WITH CUBE для возврата итоговых данных, основанных на идентификаторе каждого заказа и идентификаторе каждой подкатегории. Я также получил бы в результирующем наборе строку с общим итогом.

ПРАКТИЧЕСКИЙ СОВЕТ.

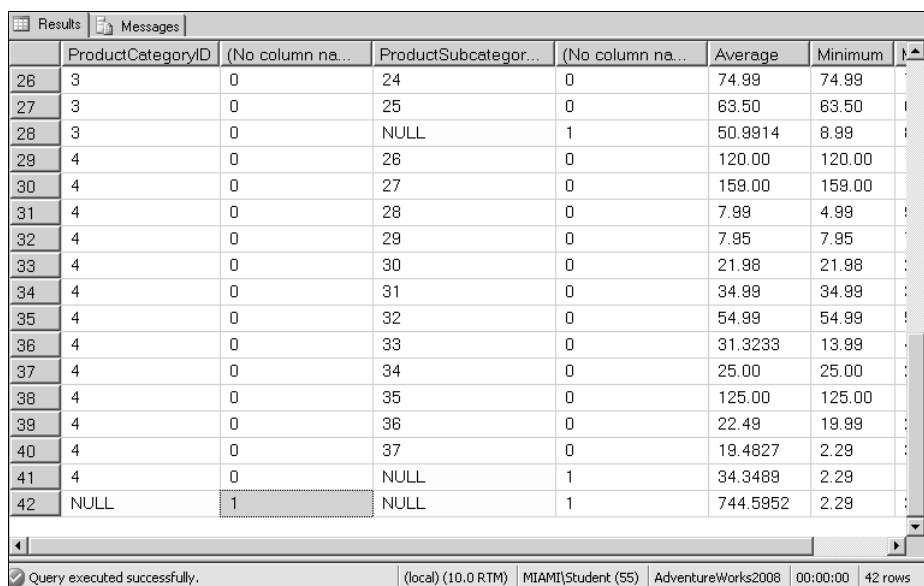
Повышение производительности GROUP BY

Избегайте применения оператора WITH CUBE в больших таблицах, если в предложение GROUP BY включено более трех столбцов. Оператор WITH CUBE возвращает итоговые данные по каждому столбцу, входящему в предложение GROUP BY. Такие результирующие наборы при вставке дополнительных столбцов в предложение GROUP BY могут очень быстро разрастаться и резко снижать производительность.

Применение статистической функции **GROUPING**

Когда в столбце, возвращаемом предложением `GROUP BY`, содержатся значения `NULL`, поиск и интерпретация итоговых строк могут оказаться трудным делом. В предложение `SELECT` можно включить статистическую функцию `GROUPING`, чтобы показать, какие строки содержат итоговую информацию, а какие — просто данные со значениями `NULL`. Статистическая функция `GROUPING` возвращает 1 для сводной или итоговой строки и 0 для обычной строки с данными.

Несмотря на то, что такие результирующие наборы становятся громоздкими и трудными для чтения, если в дальнейшем результаты будут обрабатываться предложением, то для определения сводных строк оно легко сможет воспользоваться возвращенными нулями и единицами и затем выполнить необходимые операции.



The screenshot shows a SQL query result grid with 7 columns: ProductCategoryID, (No column na..., ProductSubcategor..., (No column na..., Average, Minimum, and a scroll bar. The data is grouped by ProductCategoryID. The last row (row 42) shows a NULL value for ProductCategoryID and a 1 for the grouping indicator, indicating a summary row for the NULL category.

	ProductCategoryID	(No column na...	ProductSubcategor...	(No column na...	Average	Minimum
26	3	0	24	0	74.99	74.99
27	3	0	25	0	63.50	63.50
28	3	0	NULL	1	50.9914	8.99
29	4	0	26	0	120.00	120.00
30	4	0	27	0	159.00	159.00
31	4	0	28	0	7.99	4.99
32	4	0	29	0	7.95	7.95
33	4	0	30	0	21.98	21.98
34	4	0	31	0	34.99	34.99
35	4	0	32	0	54.99	54.99
36	4	0	33	0	31.3233	13.99
37	4	0	34	0	25.00	25.00
38	4	0	35	0	125.00	125.00
39	4	0	36	0	22.49	19.99
40	4	0	37	0	19.4827	2.29
41	4	0	NULL	1	34.3489	2.29
42	NULL	1	NULL	1	744.5952	2.29

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (55) AdventureWorks2008 00:00:00 42 rows

Рис. 1.8. Результаты после добавления функции `GROUPING`

В следующий пример включена функция `GROUPING` со столбцами из запроса с оператором `ROLLUP` предыдущего примера. Обратите внимание на то, что в части результирующего набора, приведенной на рис. 1.8, в подкатегорию не включены итоговые данные по всем категориям товаров. Для того чтобы в этом столбце появились сводные данные, нужно было бы в предложении `GROUP BY` изменить порядок указания столбцов на обратный, либо задать оператор `WITH CUBE`.

```
SELECT Production.ProductCategory.ProductCategoryID
       , GROUPING (Production.ProductCategory.ProductCategoryID)
       , Production.Product.ProductSubcategoryID
       , GROUPING (Production.Product.ProductSubcategoryID)
```

```
        , AVG(Listprice) AS 'Average', MIN(Listprice) AS 'Minimum'  
        , MAX(Listprice) AS 'Maximum'  
FROM Production.Product  
JOIN Production.ProductSubcategory  
    ON Production.ProductSubcategory.ProductSubcategoryID =  
        Production.Product.ProductSubcategoryID  
JOIN Production.ProductCategory  
    ON Production.ProductSubcategory.ProductCategoryID =  
        Production.ProductCategory.ProductCategoryID  
WHERE ListPrice <> 0  
GROUP BY Production.ProductCategory.ProductCategoryID  
        , Product.ProductSubcategoryID  
WITH ROLLUP;
```

Применение **GROUPING SETS**

Оператор **GROUPING SETS** был введен в версии SQL Server 2008 для достижения большей гибкости в задании инструкций **SELECT**, содержащих статистические функции. В зависимости от способа задания оператор **GROUPING SETS** может быть эквивалентен действиям стандартных операторов **ROLLUP** или **CUBE**, нескольким операциям **GROUP BY** в сочетании с оператором **UNION ALL** или подмножеству данных, обычно возвращаемому оператором **ROLLUP** или **CUBE**.

Дополнительная информация об эквивалентах **GROUPING SETS**

Более полный перечень команд, эквивалентных разным комбинациям **GROUPING SETS**, см. в разд. "GROUPING SETS Equivalents" (Эквиваленты **GROUPING SETS**) электронной документации по SQL Server¹.

Примеры применения **GROUPING SETS**

Примеры этого раздела знакомят вас с несколькими вариантами возможного применения оператора **GROUPING SETS**.

В следующий пример включены два отдельных запроса, формирующих идентичные результирующие наборы, но запрос с оператором **GROUPING SETS**, приведенный первым, гораздо понятнее и легче читается. Комментарий между запросами выделен жирным шрифтом для обозначения начала второго запроса (помеченного как **Equivalent code** (эквивалентный код)).

```
SELECT Production.ProductCategory.ProductCategoryID  
        , Production.Product.ProductSubcategoryID  
        , AVG(Listprice) AS 'Average'
```

¹ Электронную документацию по SQL Server 2008 можно найти по адресу <http://technet.microsoft.com/ru-ru/library/ms130214.aspx>. — *Ред.*


```

FROM Production.Product
JOIN Production.ProductSubcategory
    ON Production.ProductSubcategory.ProductSubcategoryID =
        Production.Product.ProductSubcategoryID
JOIN Production.ProductCategory
    ON Production.ProductSubcategory.ProductCategoryID =
        Production.ProductCategory.ProductCategoryID
WHERE ListPrice <> 0
GROUP BY GROUPING SETS( (Production.ProductCategory.ProductCategoryID)
                        , (Product.ProductSubcategoryID) )

```

--Equivalent code

```

SELECT NULL AS 'ProductCategoryID'
      , Production.Product.ProductSubcategoryID
      , AVG(Listprice) AS 'Average'
FROM Production.Product
JOIN Production.ProductSubcategory
    ON Production.ProductSubcategory.ProductSubcategoryID =
        Production.Product.ProductSubcategoryID
JOIN Production.ProductCategory
    ON Production.ProductSubcategory.ProductCategoryID =
        Production.ProductCategory.ProductCategoryID
WHERE ListPrice <> 0
GROUP BY (Production.Product.ProductSubcategoryID)
UNION ALL
SELECT Production.ProductCategory.ProductCategoryID, NULL
      , AVG(Listprice) AS 'Average'
FROM Production.Product
JOIN Production.ProductSubcategory
    ON Production.ProductSubcategory.ProductSubcategoryID =
        Production.Product.ProductSubcategoryID
JOIN Production.ProductCategory
    ON Production.ProductSubcategory.ProductCategoryID =
        Production.ProductCategory.ProductCategoryID
WHERE ListPrice <> 0
GROUP BY (Production.ProductCategory.ProductCategoryID)

```

Часть результирующего набора, создаваемого любой из этих команд, отображена на рис. 1.9.

Оператор UNION ALL

Полное описание оператора UNION ALL см. в главе 4.

Следующий пример формирует тот же результат, что и предыдущий, но он добавляет сводную строку, в которой содержится итог по двум столбцам.

```

SELECT Production.ProductCategory.ProductCategoryID
      , Production.Product.ProductSubcategoryID
      , AVG(Listprice) AS 'Average'

```

```

FROM Production.Product
JOIN Production.ProductSubcategory
    ON Production.ProductSubcategory.ProductSubcategoryID =
        Production.Product.ProductSubcategoryID
JOIN Production.ProductCategory
    ON Production.ProductSubcategory.ProductCategoryID =
        Production.ProductCategory.ProductCategoryID
WHERE ListPrice <> 0
GROUP BY GROUPING SETS
    (ROLLUP (Production.ProductCategory.ProductCategoryID)
      , (Product.ProductSubcategoryID) );

```

	ProductCategoryID	ProductSubcategoryID	Average
26	NULL	26	120.00
27	NULL	27	159.00
28	NULL	28	7.99
29	NULL	29	7.95
30	NULL	30	21.98
31	NULL	31	34.99
32	NULL	32	54.99
33	NULL	33	31.3233
34	NULL	34	25.00
35	NULL	35	125.00
36	NULL	36	22.49
37	NULL	37	19.4827
38	1	NULL	1588.737
39	2	NULL	469.8602
40	3	NULL	50.9914
41	4	NULL	34.3489

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (54) AdventureWorks2008 00:00:00 41 rows

Рис. 1.9. Результаты примера с оператором GROUPING SETS

	ProductCategoryID	ProductSubcategoryID	Average
27	NULL	27	159.00
28	NULL	28	7.99
29	NULL	29	7.95
30	NULL	30	21.98
31	NULL	31	34.99
32	NULL	32	54.99
33	NULL	33	31.3233
34	NULL	34	25.00
35	NULL	35	125.00
36	NULL	36	22.49
37	NULL	37	19.4827
38	NULL	NULL	744.5952
39	1	NULL	1588.737
40	2	NULL	469.8602
41	3	NULL	50.9914
42	4	NULL	34.3489

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (55) AdventureWorks2008 00:00:00 42 rows

Рис. 1.10. Пример результирующего набора GROUPING SETS с оператором ROLLUP

В части результирующего набора, показанной на рис. 1.10, в строке 38 отображена новая сводная строка.

Применение предложения *HAVING*

Предложения *WHERE* можно применять для ограничения результирующего набора на основе значений в столбцах, заданных в предложении *FROM*, а предложение *HAVING* позволяет отбирать строки на основе результатов вычислений, выполненных статистическими функциями.

В следующем примере используется предложение *HAVING*, которое возвращает только те подкатегории товаров, в которых минимальная цена больше 200 долл.

```
SELECT Production.ProductCategory.ProductCategoryID
       , Production.Product.ProductSubcategoryID
       , AVG(Listprice) AS 'Average', MIN(Listprice) AS 'Minimum'
       , MAX(Listprice) AS 'Maximum'
FROM Production.Product
JOIN Production.ProductSubcategory
  ON Production.ProductSubcategory.ProductSubcategoryID =
     Production.Product.ProductSubcategoryID
JOIN Production.ProductCategory
  ON Production.ProductSubcategory.ProductCategoryID =
     Production.ProductCategory.ProductCategoryID
WHERE ListPrice <> 0
GROUP BY Production.ProductCategory.ProductCategoryID
       , Product.ProductSubcategoryID
WITH ROLLUP
HAVING MIN(ListPrice) > 200;
```

Результирующий набор этого запроса показан на рис. 1.11.

	ProductCategoryID	ProductSubcategor...	Average	Minimum	Maximum
1	1	1	1683.365	539.99	3399.99
2	1	2	1597.45	539.99	3578.27
3	1	3	1425.2481	742.35	2384.07
4	1	NULL	1586.737	539.99	3578.27
5	2	12	678.2535	249.79	1364.50
6	2	14	780.0436	337.22	1431.50
7	2	16	631.4155	333.42	1003.91

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (55) AdventureWorks2008 00:00:00 7 rows

Рис. 1.11. Результаты запроса с предложением *HAVING*

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Реализация группирующих запросов

На этом практическом занятии вы создадите запросы разной степени сложности, начиная с возврата результатов системной статистической функции и заканчивая возвратом сводных значений, сгруппированных по нескольким столбцам и включающих оператор `GROUPING SETS`.

Упражнение. Применение предложения *GROUP BY*

В этом упражнении вы примените статистические функции и предложение `GROUP BY` с разными операторами, такими как `ROLLUP` и `HAVING`, которые предоставляют как сводные, так и обычные данные в результирующем наборе.

1. Откройте в среде SSMS окно создания запроса.
2. В открывшемся окне запроса введите и выполните следующий программный код, возвращающий общий итог всех товаров, встречающихся во всех строках всех заказов на продажу.

```
USE AdventureWorks2008;  
SELECT SUM(LineTotal) FROM Sales.SalesOrderDetail;
```

3. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий программный код, чтобы получить итог отпускной цены для всех комбинаций идентификаторов заказов на продажу и идентификаторов товаров. Затем просмотрите результирующий набор.

```
SELECT SalesOrderID, ProductID, SUM(Linetotal) AS 'Total'  
FROM Sales.SalesOrderDetail  
GROUP BY SalesOrderID, ProductID  
ORDER BY SalesOrderID;
```

4. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код, чтобы получить общий итог, а также промежуточный итог по всем товарам для каждого заказа на продажу. Далее просмотрите результирующий набор.

```
SELECT SalesOrderID, ProductID, SUM(Linetotal) AS 'Total'  
FROM Sales.SalesOrderDetail  
GROUP BY SalesOrderID, ProductID  
WITH ROLLUP  
ORDER BY SalesOrderID;
```

5. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код, чтобы изменить порядок перечисления столбцов в предложении `GROUP BY` на обратный и получить промежуточный итог по каждому товару для всех заказов на продажу, а также общий итог. Порядок сортировки изменен для облегчения просмотра результатов. После выполнения запроса просмотрите результирующий набор.

```
SELECT SalesOrderID, ProductID, SUM(Linetotal) AS 'Total'
FROM Sales.SalesOrderDetail
GROUP BY ProductID, SalesOrderID
WITH ROLLUP
ORDER BY ProductID;
```

6. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код, чтобы получить промежуточные итоги по идентификаторам заказов на продажу и по идентификаторам товаров. Просмотрите результирующий набор.

```
SELECT SalesOrderID, ProductID, SUM(Linetotal) AS 'Total'
FROM Sales.SalesOrderDetail
GROUP BY ProductID, SalesOrderID
WITH CUBE
ORDER BY SalesOrderID, ProductID;
```

7. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код, чтобы оставить в результирующем наборе только строки с итогами, превышающими 10 000 долл. Просмотрите результаты.

```
SELECT SalesOrderID, ProductID, SUM(Linetotal) AS 'Total'
FROM Sales.SalesOrderDetail
GROUP BY ProductID, SalesOrderID
WITH CUBE
HAVING SUM(Linetotal) > 10000
ORDER BY SalesOrderID, ProductID;
```

8. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код, чтобы создать без применения каких-либо операторов сворачивания группирующий набор по столбцам SalesOrderID (ID заказа на продажу) и ProductID (ID товара). Просмотрите результирующий набор.

```
SELECT SalesOrderID, ProductID, SUM(Linetotal) AS 'Total'
FROM Sales.SalesOrderDetail
GROUP BY GROUPING SETS (ProductID, SalesOrderID)
ORDER BY SalesOrderID, ProductID
```

9. Сохраните сценарий и закройте окно запроса, но оставьте открытой среду SSMS для последующих практических занятий.

Резюме урока

- ◆ Статистические функции выполняют вычисления выражений, которые предоставляются как входные параметры функций.
- ◆ Применяйте предложение `GROUP BY`, когда итоги должны вычисляться на основе данных из заданных строк, а не из всей таблицы.

- ◆ Включайте в предложение `GROUP BY` все столбцы, перечисленные в предложениях `SELECT`, `WHERE` или `ORDER BY`.
- ◆ Применяйте операторы `ROLLUP` и `CUBE` для предоставления дополнительных сводных данных.
- ◆ Используйте функцию `GROUPING` для показа строк, содержащих итоговые данные, предоставленные операторами `ROLLUP` и `CUBE`.
- ◆ Применяйте оператор `GROUPING SETS` для обеспечения большей гибкости и читабельности ваших запросов с предложением `GROUP BY`.

УРОК 4. Объединение наборов данных

Программа SQL Server 2008 предоставляет несколько операторов, дающих возможность объединять или сравнивать результаты множественных инструкций `SELECT`. Оператор `UNION` был доступен, начиная с первой версии SQL Server, и давал возможность объединять результирующие наборы множественных запросов. С другой стороны, в SQL Server 2005 были введены операторы `EXCEPT` и `INTERSECT` для сравнения результатов двух запросов и формирования нового результата на основе наличия или отсутствия подходящих строк в обоих результирующих наборах.

Кроме того, наборы данных могут обрабатываться оператором `APPLY`, применяющим табличную функцию к каждой строке результатов запроса, которые определяются как внешняя таблица.

Функции, определенные пользователем

SQL Server 2008 предоставляет возможность создавать функции, определенные пользователем (user-defined functions, UDF). Скалярные функции возвращают единственное значение заданного типа, а встраиваемые табличные функции и многооператорные табличные функции возвращают результат табличного типа. Дополнительную информацию см. в главе 5.

После усвоения этого урока вы сможете

- Писать запросы, использующие оператор `UNION` для объединения результирующих наборов.
- Писать запросы, использующие операторы `EXCEPT` и `INTERSECT` для сравнения результатов нескольких запросов.

Расчетное время урока — 30 мин.

Операторы `UNION`, `EXCEPT` и `INTERSECT` могут задаваться между двумя или несколькими запросами для формирования единого результирующего набора. В то время как оператор `UNION` объединяет результирующие наборы из множественных запросов в единый результирующий набор, операторы `EXCEPT` и `INTERSECT` сравнивают результирующие наборы двух запросов для того, чтобы определить, ка-

кое подмножество строк должно быть включено в окончательный результирующий набор.

Применение оператора *UNION*

Оператор *UNION* позволяет объединять в набор результирующие наборы, созданные в множественных инструкциях *SELECT*. Несмотря на то, что синтаксис оператора *UNION* прост и понятен (вы добавляете ключевое слово *UNION* или *UNION ALL* между каждыми двумя инструкциями *SELECT*), вы должны следовать нескольким основным правилам, если хотите получить корректный запрос. При задании оператора *UNION* оба запроса должны возвращать одинаковое количество столбцов. Кроме того, типы данных в соответствующих столбцах должны быть совместимы. Например, если у первого столбца в одной инструкции *SELECT* тип данных *integer*, а у первого столбца второй инструкции *SELECT* тип данных *character*, для успешного выполнения команды поле целочисленного типа должно быть преобразовано.

В базе данных Northwind у клиентов алфавитно-цифровое поле идентификатора клиента, а у сотрудников цифровое поле идентификатора. Если вы хотите извлечь эти столбцы как единый результирующий набор, поле *EmployeeID* необходимо преобразовать, как показано в следующем примере.

```
SELECT CONVERT(Char(8), Employees.EmployeeID) FROM Employees
UNION
SELECT Customers.CustomerID FROM Customers
```

Помимо перечисленных правил существуют и другие правила, о которых вам следует знать. Во-первых, заголовки столбцов берутся из первого из приведенных запросов. Следовательно, вы должны задавать псевдонимы в первой инструкции *SELECT*. Кроме того, предложения *ORDER BY* следует задавать в конце последней инструкции *SELECT*. Предложение *ORDER BY* может ссылаться на псевдонимы, определенные в первой инструкции *SELECT*. И наконец, если строки нужно фильтровать, каждая инструкция *SELECT* должна включать собственное предложение *WHERE*.

Ключевое слово *ALL* указывает на то, что отбираться должны все строки, включая дубликаты. По умолчанию дублирующиеся строки не возвращаются.

В следующем примере также используются таблицы *Employees* (Сотрудники) и *Customers* (Клиенты) из базы данных Northwind для создания единого результирующего набора, который объединяет клиентов и сотрудников в одном наборе, содержащем фамилию, имя, название компании и номер телефона.

```
SELECT FirstName + ' ' + LastName AS 'Contact Name'
      , 'Northwind Traders' AS 'Company', Employees.HomePhone AS 'Phone'
FROM Employees
UNION
```

```
SELECT ContactName, CompanyName, Phone
FROM Customers
ORDER BY 'Contact Name';
```

Применение команд *EXCEPT* и *INTERSECT*

В отличие от оператора *UNION*, который возвращает комбинацию строк из разных инструкций *SELECT*, операторы *EXCEPT* и *INTERSECT* сравнивают результирующие наборы двух отдельных запросов и предоставляют подмножество данных.

Оператор *EXCEPT* возвращает все строки, имеющиеся в таблице, находящейся слева от оператора, и не имеющие сопоставимых строк в таблице, расположенной справа. В следующем примере из базы данных AdventureWorksDW2008 в результирующем наборе перечислены все сотрудники, никогда не помещавшие заказы на продажу через торговых посредников.

```
SELECT EmployeeKey
FROM DimEmployee
EXCEPT
SELECT EmployeeKey
FROM FactResellerSales
```

Как и в случае оператора *UNION*, в обоих запросах должно быть определено одинаковое количество столбцов с совместимыми типами данных. Для этого вы можете соединить таблицы и применить оператор *EXCEPT*. В следующем примере к запросу, приведенному в предыдущем примере, добавляются фамилия, имя сотрудника и его должность.

```
SELECT EmployeeKey, FirstName, LastName, Title
FROM DimEmployee
EXCEPT
SELECT FRS.EmployeeKey, DE.FirstName, DE.LastName, DE.Title
FROM FactResellerSales FRS
JOIN DimEmployee DE
ON DE.EmployeeKey = FRS.EmployeeKey
ORDER BY Title;
```

С другой стороны, если мы хотим увидеть только тех сотрудников, кто поместил заказы на продажу посредникам, мы можем применить оператор *INTERSECT*, как показано в приведенном далее примере.

```
SELECT EmployeeKey, FirstName, LastName, Title
FROM DimEmployee
INTERSECT
SELECT FRS.EmployeeKey, DE.FirstName, DE.LastName, DE.Title
FROM FactResellerSales FRS
JOIN DimEmployee DE
ON DE.EmployeeKey = FRS.EmployeeKey
ORDER BY Title;
```


Применение оператора *APPLY*

Оператор *APPLY* отличается от других операторов, обсуждавшихся до сих пор, т. к. он использует результаты запроса к таблице или представлению, именуемому *левым входом* (left input), и результаты табличной функции, именуемой *правым входом* (right input).

У оператора *APPLY* есть две формы: *CROSS APPLY* и *OUTER APPLY*. *CROSS APPLY* возвращает только те строки из левого входа, которые извлекают данные из табличной функции. *OUTER APPLY* возвращает все строки из левой внешней таблицы. Как и в операторе *OUTER JOIN*, в столбцы, для которых табличная функция не предоставляет данных, вставляются значения *NULL*. В примере, приведенном далее, объединяются результаты из таблицы *Person.EmailAddress* (Адрес электронной почты) с результатами функции *ufnGetContactInformation*, определенной пользователем, и возвращаются имя, фамилия, должность, тип контактной информации и адрес электронной почты для всех контактов, хранящихся в базе данных. Значение *BusinessEntityID* (ID делового партнера) из каждой строки результирующего набора таблицы — это входной параметр, необходимый функции *ufnGetContactInformation*.

```
SELECT GCI.FirstName, GCI.LastName, GCI.JobTitle
      , GCI.BusinessEntityType, PE.EmailAddress
FROM Person.EmailAddress AS PE
CROSS APPLY
dbo.ufnGetContactInformation(PE.BusinessEntityID) AS GCI;
```

Долго выполняющийся запрос

Имейте в виду, что для выполнения этого запроса потребуется много времени.

В предыдущем примере запрос возвращает 19 683 строки. Если оператор *CROSS APPLY* заменить оператором *OUTER APPLY*, из таблицы *EmailAddress* будут возвра-

	FirstName	LastName	JobTitle	BusinessEntityTy...	EmailAddress
1	A	Leonetti	Purchasing Agent	Store Contact	a0@adventure-works.com
2	NULL	NULL	NULL	NULL	a1@adventure-works.com
3	Aaron	Con	Purchasing Manager	Store Contact	aaron0@adventure-works.com
4	NULL	NULL	NULL	NULL	aaron1@adventure-works.com
5	Aaron	Hughes	NULL	Consumer	aaron10@adventure-works.com
6	Aaron	Flores	NULL	Consumer	aaron11@adventure-works.com
7	Aaron	Washington	NULL	Consumer	aaron12@adventure-works.com
8	Aaron	Butler	NULL	Consumer	aaron13@adventure-works.com
9	Aaron	Simmons	NULL	Consumer	aaron14@adventure-works.com
10	Aaron	Foster	NULL	Consumer	aaron15@adventure-works.com

Query executed successfully. (local) (10.0 RTM) MIAMI\Student (57) AdventureWorks2008 00:02:37 19972 rows

Рис. 1.12. Результаты применения оператора *OUTER APPLY*

щены все 19 972 строки, несмотря на то, что для некоторых из них нет данных, возвращаемых функцией. На рис. 1.12 показан результирующий набор после применения OUTER APPLY, отображающий NULL во всех столбцах, возвращенных функцией.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Объединение наборов данных

На этом практическом занятии вы примените операторы UNION, EXCEPT и INTERSECT в базах данных AdventureWorks2008 и AdventureWorksDW2008.

Упражнение 1. Применение оператора UNION

В этом упражнении вы объедините результирующие наборы из таблиц FactInternetSales (Действительные интернет-продажи) и FactResellerSales (Действительные продажи посредникам) базы данных AdventureWorksDW2008.

1. Откройте окно создания запроса в SSMS.
2. В открытом окне запроса введите и выполните следующую команду для отображения всех заказов из таблиц действительных заказов посредников и интернет-продаж. Поскольку в таблице с фактическими интернет-продажами нет столбца, связанного с именем посредника из таблицы фактических продаж посредникам, в список выбора включается строковая константа 'N/A' для того, чтобы выполнялось требование команды о наличии одинакового количества столбцов в обеих инструкциях SELECT. Помимо этого имя посредника извлекается из таблицы DimReseller (Полный список посредников).

```
USE AdventureWorksDW2008;
SELECT SalesOrderNumber, SalesOrderLineNumber
      , SalesAmount, TaxAmt, 'N/A' FROM FactInternetSales
UNION ALL
SELECT SalesOrderNumber, SalesOrderLineNumber
      , SalesAmount, TaxAmt, ResellerName FROM FactResellerSales
JOIN DimReseller
      ON DimReseller.ResellerKey = FactResellerSales.ResellerKey
ORDER BY SalesOrderNumber;
```

3. Сохраните сценарий и закройте окно запроса.

Упражнение 2. Применение операторов EXCEPT и INTERSECT

В этом упражнении вы примените операторы EXCEPT и INTERSECT для сравнения результатов из таблиц Production.Product (Товар) и Sales.SalesOrderDetail (Подробные данные о заказе) базы данных AdventureWorks2008.

1. Откройте окно создания запроса.
2. В открывшемся окне запроса введите и выполните следующую команду для отображения ID и названия всех действительных товаров, ни разу не продававшихся. Товары с отпускной ценой 0 долл. не включаются, т. к. они пока не доступны для продаж.

```
USE AdventureWorks2008;
```

```
SELECT ProductID, Name, ListPrice FROM Production.Product
WHERE ListPrice <> 0
EXCEPT
SELECT SOD.ProductID, Name
      , p.listprice FROM Sales.SalesOrderDetail SOD
JOIN Production.Product P
      ON P.ProductID = SOD.ProductID
ORDER BY ProductID;
```

3. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий код для отображения тех товаров, которые были включены в заказы на продажу.

```
SELECT ProductID, Name, ListPrice FROM Production.Product
INTERSECT
SELECT SOD.ProductID, Name
      , p.listprice FROM Sales.SalesOrderDetail SOD
JOIN Production.Product P
      ON P.ProductID = SOD.ProductID
ORDER BY ProductID;
```

4. Из-за ограничений целостности в этой базе данных команда `SELECT` из пункта 3 возвращает тот же результирующий набор, что и команда `SELECT DISTINCT`, обращающаяся к таблице `SalesOrderDetail`, т. к. любой проданный товар включается в таблицу `Product`. Для того чтобы убедиться в этом, вы можете выполнить приведенную далее команду и сравнить ее результат с результирующим набором из пункта 3.

```
SELECT DISTINCT SOD.ProductID, Name, p.listprice
FROM Sales.SalesOrderDetail SOD
JOIN Production.Product P
      ON P.ProductID = SOD.ProductID
ORDER BY ProductID;
```

5. Сохраните сценарий и закройте окно запроса.

Резюме урока

- ◆ Оператор `UNION` объединяет результирующие наборы двух или нескольких инструкций `SELECT`.

- ◆ Оператор `EXCEPT` возвращает строки, имеющиеся в левой инструкции `SELECT` и не сопоставимые со строками из правой инструкции `SELECT`.
- ◆ Оператор `INTERSECT` возвращает только строки, общие для обеих инструкций `SELECT`.
- ◆ Оператор `APPLY` использует результаты запроса как входной параметр и применяет табличную функцию к каждой строке результирующего набора.
- ◆ `OUTER APPLY` возвращает все строки внешней таблицы наряду с результатами, возвращенными функцией, в случае соответствия строк, в то время как `CROSS APPLY` возвращает только те строки внешней таблицы, для которых найдено соответствие в результатах, полученных функцией.

УРОК 5. Применение встроенных скалярных функций

SQL Server 2008 предлагает большое количество встроенных функций разных категорий. Вы можете применять многие из них для усовершенствования ваших запросов. Вы также можете использовать многие из этих функций для выполнения таких действий, как вставка данных, создание таблиц и формирование ограничений. Эти действия будут обсуждаться далее в книге.

Дополнительная информация о функциях

Полный список категорий функций, сопровождаемый описанием каждой из них, см. в разд. "Functions (Database Engine)" (Функции (компонент Database Engine)) электронной документации по SQL Server.

После усвоения этого урока вы сможете

- Объяснить назначение встроенных функций.
- Описать разнообразие встроенных скалярных функций и задачи, выполняемые ими.

Расчетное время урока — 45 мин.

Применение встроенных скалярных функций

Встроенные функции могут применяться в предложениях `SELECT` или `WHERE` инструкции `SELECT`, но для достижения оптимальной производительности следует избегать использования функций в предложении `WHERE`. Когда вы используете встроенную функцию, часто после имени функции указываются круглые скобки, даже если у функции нет входных параметров. Во многих случаях пустые скобки заставляют процессор базы данных применять текущее значение. Например, функция `db_id` возвращает идентификатор базы данных, активной в системе в данный момент. По этой причине многие из системных функций называют *не-*

детерминированными (nondeterministic), т. е. результат может отличаться, и у системы нет перечня заранее установленных значений для результатов функции. С другой стороны, функция ISNULL детерминированная, т. к. при вызове функции предлагаемое системой значение содержится в определении функции.

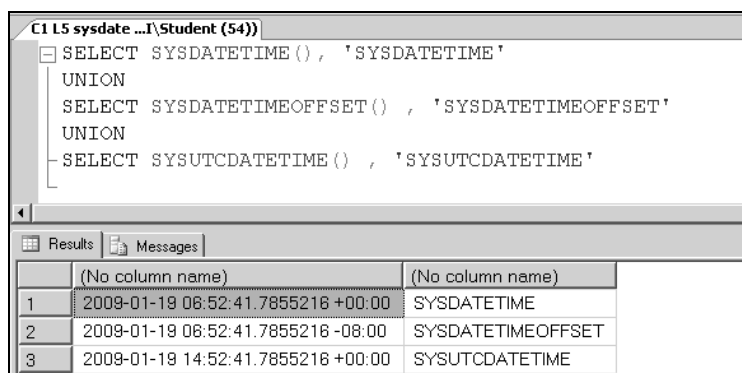
Несмотря на то, что обсуждение всех имеющихся встроенных функций выходит за рамки данной главы, вы познакомитесь с рядом примеров и описаниями функционирования некоторых самых популярных функций.

Примеры встроенных функций

Существует множество встроенных функций. Корпорация Microsoft разделила их все на группы в соответствии с определениями и назначением функций.

Функции для работы с датами и временем суток

Есть много функций для работы с датами и временем суток. Все, появившиеся в версии SQL Server 2008, новые функции SYSDATETIME, SYSDATETIMEOFFSET и SYSUTCDATETIME извлекают с сервера текущее системное время с точностью до 100 нс. Примеры результатов показаны на рис. 1.13.



SQL Query Window: C:\15 sysdate ...I\Student (54)

```

SELECT SYSDATETIME() , 'SYSDATETIME'
UNION
SELECT SYSDATETIMEOFFSET() , 'SYSDATETIMEOFFSET'
UNION
SELECT SYSUTCDATETIME() , 'SYSUTCDATETIME'

```

	(No column name)	(No column name)
1	2009-01-19 08:52:41.7855218 +00:00	SYSDATETIME
2	2009-01-19 08:52:41.7855218 -08:00	SYSDATETIMEOFFSET
3	2009-01-19 14:52:41.7855218 +00:00	SYSUTCDATETIME

Рис. 1.13. Примеры результатов системных функций для извлечения даты

Если вам не нужна такая точность, следующие функции возвращают результат типа datetime вместо типов данных datetime2(7) и datetimeoffset(7), возвращаемых приведенными функциями. Функции извлечения даты GETDATE и GETUTCDATE используют одинаковый формат представления, требующий наличия скобок после имени функции, а функция CURRENT_TIMESTAMP — нет. Функции CURRENT_TIMESTAMP и GETDATE формируют одинаковые результаты.

Применяя эти менее точные функции обработки дат и времени, можно повысить производительность, т. к. они возвращают более короткий результат. На рис. 1.14 показаны результаты применения перечисленных менее точных функций.

SQL Query Window: C1 L5 older dat...T\Student (54))

```

SELECT GETDATE(), 'GETDATE'
UNION
SELECT CURRENT_TIMESTAMP, 'CURRENT_TIMESTAMP'
UNION
SELECT GETUTCDATE(), 'GETUTCDATE'

```

Results Tab:

	(No column name)	(No column name)
1	2009-01-19 08:58:29.300	CURRENT_TIMESTAMP
2	2009-01-19 08:58:29.300	GETDATE
3	2009-01-19 14:58:29.300	GETUTCDATE

Рис. 1.14. Результаты дополнительных менее точных функций извлечения даты и времени

В дополнение к функциям, возвращающим текущие дату и время, есть функции, которые возвращают части даты и времени суток, например, `DATEPART` и `DATENAME`. Функция `DATENAME` возвращает название заданной части даты, например "January" (Январь) для месяца. Функция `DATEPART`, напротив, возвращает числовое обозначение заданной части даты. Например, "1" для января. Есть и функции, выполняющие арифметические операции над датами, например, `DATEADD` и `DATEDIFF`.

Функции `DATEADD` и `DATEDIFF` требуют задания части даты. Для указания части даты существует множество ключевых слов и символов. Например, год можно задать как `YEAR`, `YY` или `YYYY`. День может быть представлен как `DAY` или `DD` и т. д. На рис. 1.15 дан результат нескольких запросов, использующих функции обработки дат.

Далее приведена синтаксическая запись функции `DATEDIFF`

```
DATEDIFF(datepart, startdate, enddate)
```

Параметр `startdate` (начальная дата) вычитается из параметра `enddate` (конечная дата). Например, если бы следующий программный код был выполнен 19 января 2009 г., результат первого запроса был бы `-8`, указывающий на то, что 8 полных лет прошло между предложенными начальной и конечной датами. Результат второго запроса был бы `-18`, представляя 18-кратную смену дня между начальной и конечной датами. Если поменять местами начальную и конечную даты, результирующий набор вернет положительные числа.

```
SELECT DATEDIFF(YEAR, GETDATE(), '1/1/2001')
```

```
SELECT DATEDIFF(DD, GETDATE(), '1/1/2009')
```

Дополнительная информация о функциях обработки дат и времени

Полный список типов данных и функций, которые применяются для работы с полями дат, см. в разд. "Date and Time Data Types and Functions (Transact SQL)" (Типы данных и функции даты и времени (Transact SQL)) электронной документации по SQL Server.

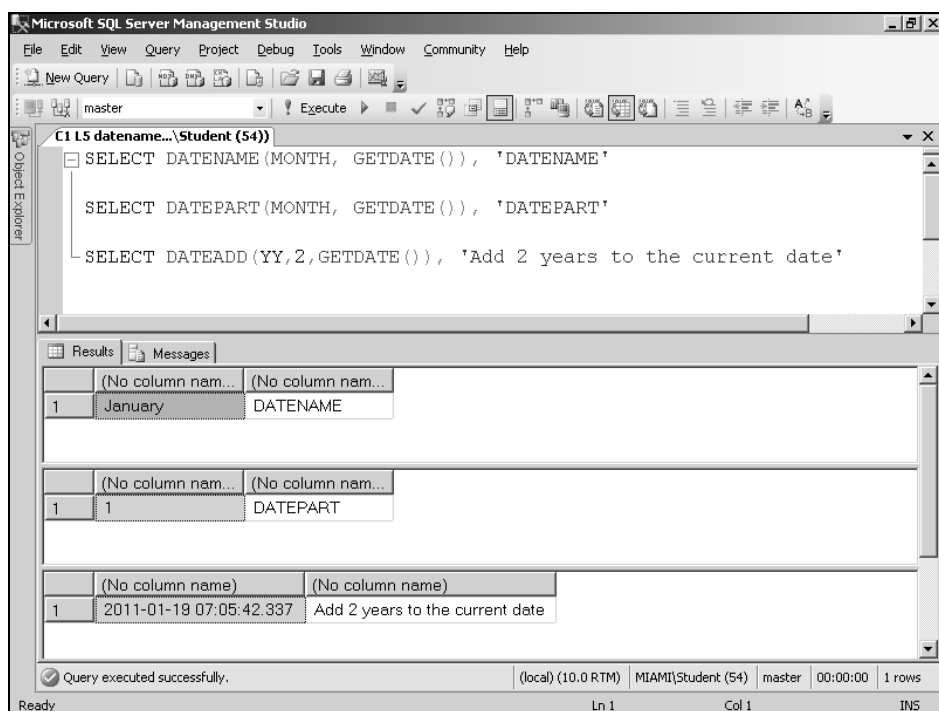


Рис. 1.15. Запросы, возвращающие части даты и времени суток

Системные функции

Помимо функций для работы с датами и временем суток часто используются системные функции, которые обрабатывают формат данных. Функции `CAST` и `CONVERT` применяются для преобразования выражения из одного типа данных в другой. Функция `CONVERT` может также использоваться для форматирования дат. На рис. 1.16 показан ряд преобразований данных. Очень важно, особенно в интернациональных организациях, понимать, как пользователи интерпретируют даты. Например, в Соединенных Штатах большинство пользователей истолкует дату "01/05/07" как 5 января 2007 г., во Франции пользователи будут интерпретировать ее как 1 мая 2007 г., а в Японии воспримут ее как 7 мая 2001 г. Если вы не зададите конкретный формат даты, формат будет определяться в зависимости от локализации установленной версии SQL Server. Если установлена локализованная версия, форматирование определяется языком по умолчанию, установленным на компьютере, который выполняет программу SQL Server.

Дополнительная информация об интернациональных настройках

Дополнительную информацию об управлении интернациональными настройками см. в главе 3.

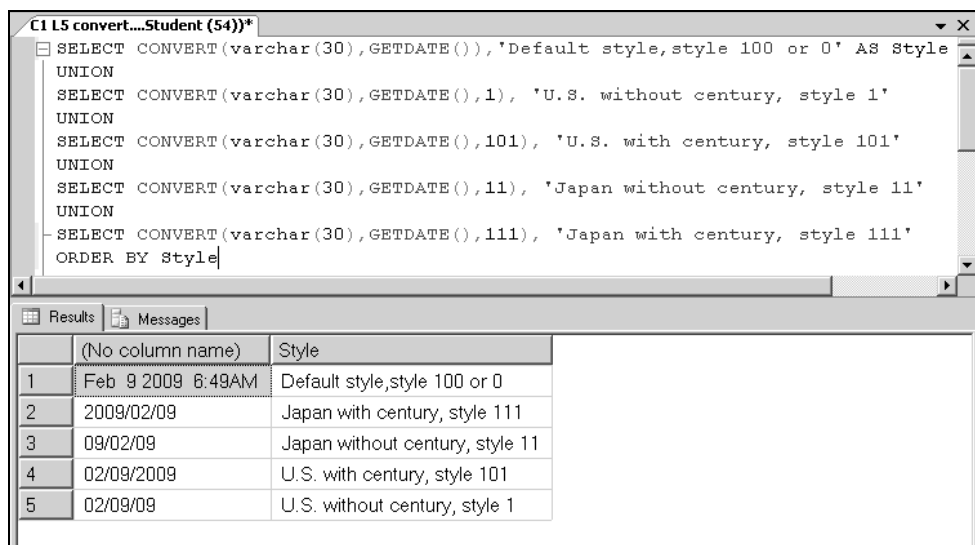


Рис. 1.16. Разные стили представления дат, извлеченных функцией CONVERT

В процессе программирования и поиска неисправностей полезны дополнительные системные функции. Функция @@ERROR возвращает номер последней ошибки, обнаруженной в текущем сеансе, а функция @@TRANCOUNT возвращает для текущего сеанса количество активных в данный момент транзакций. В более ранних версиях SQL Server эти функции были известны как глобальные переменные. В отличие от предыдущих примеров в синтаксическую запись этих функций не включены круглые скобки. Инструкция @@ERROR должна выполняться сразу за командой, вызвавшей ошибку.

Строковые функции

Строковые функции выполняют обработку строковых выражений. Далее перечислены наиболее часто применяемые строковые функции.

- ◆ LEFT, RIGHT возвращают конкретное количество символов с заданной стороны (слева или справа) от выражения.
- ◆ UPPER, LOWER возвращают заданный регистр (прописные или строчные буквы) для всех символов выражения.
- ◆ SUBSTRING получает выражение как входной параметр и возвращает заданное число символов, начиная с указанной позиции. Например, SUBSTRING(FirstName,1,3) возвращает, начиная с первой позиции, три символа из столбца FirstName (Имя).
- ◆ REPLACE замещает в заданной строке одну подстроку другой.

- ◆ `LEN` возвращает количество символов в заданной строке. Если вы хотите получить количество байтов, а не символов, следует применять функцию `DATALENGTH`.

Дополнительные функции

Следующие функции также часто востребованы.

- ◆ `DATALENGTH` возвращает количество байтов, необходимых для представления выражения. Может использоваться с любым типом данных, но особенно полезна для типов данных `varchar`, `varbinary` и `nvarchar`, `binary` и `image`.
- ◆ `PATINDEX` возвращает начальную позицию первого вхождения заданного образца в указанную строку.
- ◆ `CHARINDEX` возвращает начальную позицию заданной строки в другой заданной строке. Функция также принимает необязательный входной параметр `start_location`, который определяет позицию во втором выражении, с которой следует начать сравнение.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Применение встроенных скалярных функций

На этом практическом занятии вы примените множество встроенных скалярных функций, вносящих дополнительный смысл и форматирование в ваши результирующие наборы.

1. Откройте окно создания запроса в SSMS.
2. В имеющемся окне запроса введите и выполните следующий программный код для извлечения дня недели даты оформления каждого заказа и упорядочивания результата по дням недели.

```
USE AdventureWorks2008;
SELECT SalesOrderID, DATENAME(DW, orderdate) AS 'Day of Week'
FROM Sales.SalesOrderHeader
ORDER BY DATEPart (DW, orderdate);
```

3. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий код для получения количества дней, прошедших между помещением заказа и его доставкой.

```
SELECT SalesOrderID
      , DATEDIFF(DD, Orderdate, ShipDate) AS 'Days lapsed'
FROM Sales.SalesOrderHeader;
```

4. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий код для возврата столбца, названного

Employee Code и формируемого из первых трех символов имени и первых трех символов фамилии каждого сотрудника.

```
SELECT LEFT(FirstName, 3) + LEFT(LastName, 3) AS 'Employee Code'  
FROM Person.Person;
```

5. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий код для замены во всех описаниях товаров слова `bike` на слово `bicycle`.

```
SELECT REPLACE(Description, 'bike', 'bicycle')  
FROM Production.ProductDescription;
```

6. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий код для возврата в одном столбце имени, фамилии и номера сотрудника. Для этого вы должны преобразовать данные столбца `BusinessEntityID` (идентификатор делового партнера) (который содержит номер сотрудника) в символьный тип данных.

```
SELECT FirstName + ' ' + Lastname + ' Employee Number: '  
       + CONVERT(char, E.BusinessEntityID)  
FROM HumanResources.Employee AS E  
JOIN Person.Person AS P  
    ON P.BusinessEntityID = E.BusinessEntityID;
```

7. В имеющемся окне запроса под набранным программным кодом введите, выделите и выполните следующий код для возврата номера и даты заказа. Отформатируйте дату так, чтобы время суток в нее не включалось.

```
SELECT SalesOrderNumber, CONVERT(varchar(30), OrderDate, 107)  
FROM Sales.SalesOrderHeader;
```

8. Измените стиль оформления даты на дополнительные форматы, например, 11, 03, 111, 114 и 103. Обратите внимание на то, что формат 114 не предоставляет необходимую информацию, а 03 и 11 будут сбивать с толку пользователей, не знакомых с этими форматами.
9. Сохраните сценарий и закройте SSMS.

Резюме урока

- ◆ Для получения более информативных результирующих наборов применяйте встроженные функции.
- ◆ Для обработки и возврата в результирующем наборе дат и времени суток используйте функции обработки дат и времени суток.
- ◆ Для форматирования или извлечения данных в строковых выражениях применяйте строковые функции.

Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

Резюме

- ◆ Инструкция `SELECT` позволяет извлекать данные из базы данных. Простейшая инструкция `SELECT` включает в себя предложения `SELECT` и `FROM`. Предложение `WHERE` может сократить количество строк, возвращаемых в результирующем наборе.
- ◆ Для объединения столбцов из нескольких связанных таблиц, позволяющего внести большую ясность в ваши инструкции `SELECT`, и расширения функциональности этих инструкций применяйте операторы `JOIN`. Для извлечения только тех строк, у которых есть сопоставимая на основе предиката `ON` строка в каждой таблице, используйте оператор `INNER JOIN`. Для возврата всех строк внешней таблицы и сопоставимых с ними строк из внутренней таблицы применяйте оператор `LEFT` или `RIGHT OUTER JOIN`. Для задания обеих таблиц как внешних, возвращающих все строки, применяйте оператор `FULL OUTER JOIN`.
- ◆ Для формирования сводных данных применяйте статистические функции и предложение `GROUP BY`. Для получения дополнительной итоговой информации используйте операторы `ROLLUP` и `CUBE`. Для того чтобы показать, какие строки содержат сводные данные, полученные с помощью операторов `ROLLUP` и `CUBE`, используйте функцию `GROUPING`. Применяйте оператор `GROUPING SETS` для обеспечения большей гибкости и наглядности ваших запросов, содержащих предложение `GROUP BY`.
- ◆ Для объединения и сравнения результирующих наборов используйте операторы `UNION`, `INTERSECT` и `EXCEPT`.
- ◆ Для обеспечения дополнительного форматирования смыслового содержимого данных, включенных в ваши результирующие наборы, применяйте встроенные функции. К ним относятся различные группы функций, определенных в SQL Server 2008, но самые популярные из них — строковые функции и функции обработки дат и времени суток.

Ключевые термины

Знаете ли вы, что означают следующие ключевые термины? Вы можете проверить свои ответы, заглянув в словарь терминов, приведенный в конце книги.

- ◆ Функция.
- ◆ Скалярная функция.
- ◆ Пользовательская или определенная пользователем функция.
- ◆ Статистическая функция.
- ◆ Табличная функция.
- ◆ UNION.
- ◆ JOIN.
- ◆ INNER JOIN.
- ◆ OUTER JOIN.

Сценарии

В следующих конкретных сценариях вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в разд. *"Ответы к сценариям"* в конце книги.

Сценарий 1. Извлечение данных

Вы — разработчик базы данных для компании Wide World Importers. Вы отвечаете за базу данных продаж компании. Эта база приведена к 3-й нормальной форме.

Прикладной программист, недавно познакомившийся с SQL Server, разрабатывает Web-приложение, извлекающее информацию из базы данных. Несмотря на то, что он уверен в возможности разработки необходимых запросов с помощью справочной системы, программист просит предоставить набор эталонных запросов, которые он мог бы экстраполировать и создать требуемые запросы.

Вам необходимо снабдить Web-разработчика образцами запросов, демонстрирующими решение необходимых задач. Приложение должно включать запросы, которые возвращают информацию об импортированных товарах (в том числе название и описание товара), основываясь на датах помещения и выполнения заказа. Отделу продаж хотелось бы в приложении иметь возможность отображать даты в разных форматах. Кроме того, продавцам хотелось бы при необходимости сортировать данные, исходя из количества дней, прошедших между датой помещения заказа и датой его исполнения. Наконец, пользователи хотели бы иметь возможность извлекать данные за разные периоды, например за прошлый месяц, квартал или год.

Ответьте на следующий вопрос вашего менеджера: "Какие функции и синтаксические элементы (предложения) вам следует включить в образцы запросов для того, чтобы своими примерами помочь программисту разработать его приложение?"

Сценарий 2. Группировка данных

Прикладной программист из сценария 1 благодаря примерам запросов, предоставленным вами, успешно реализовал первый вариант Web-приложения. Теперь клиент попросил, чтобы программист включил сводные данные, указав, сколько всего товаров было продано и сколько всего было продано товаров конкретной категории. Каждая строка таблицы `SalesOrderDetail` (Подробные сведения о заказе на продажу) содержит идентификатор одного товара, а столбец `Qty` (Количество) включает объем этого товара, приведенный в заказе с указанным номером.

Ответьте на следующий вопрос вашего менеджера: "Какие функции и синтаксические элементы (предложения) вам следует включить в образцы запросов для того, чтобы своими примерами помочь программисту разработать его приложение?"

КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

Запросы данных с помощью инструкций *SELECT*

Упражнение 1. Напишите и выполните различные инструкции `SELECT` с предложением `WHERE`, в которых используются логические операторы и предложения `LIKE` и `ORDER BY`. Просмотрите и сравните результаты, полученные с помощью разных инструкций.

Объединение наборов данных

Упражнение 2. Напишите и выполните различные инструкции `SELECT`, содержащие оператор `JOIN`. Тщательно сравните результаты соединений типов `INNER`, `LEFT` или `RIGHT OUTER` и `FULL OUTER`.

Упражнение 3. Напишите и выполните различные инструкции `SELECT`, в которых применяются операторы `UNION`, `EXCEPT`, `INTERSECT`, `CROSS APPLY` и `OUTER APPLY`.

Реализация группирующих запросов

Упражнение 4. Напишите и выполните различные инструкции `SELECT`, в которых используются статистические функции вместе с предложением `GROUP BY` и функцией `GROUPING`.

Применение встроенных скалярных функций

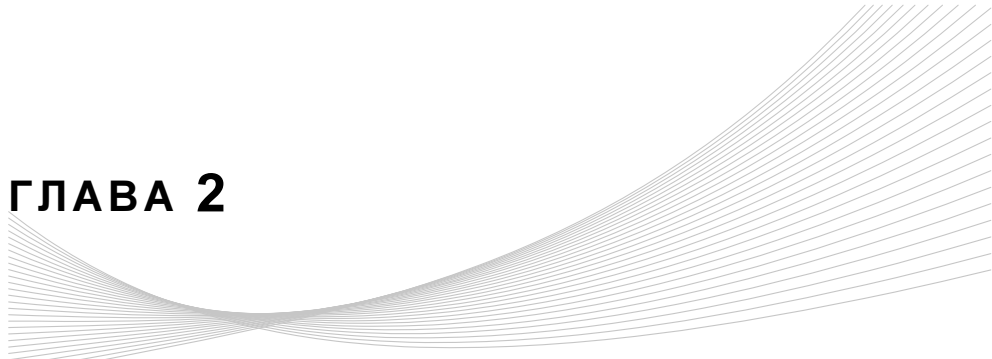
Упражнение 5. Напишите и выполните различные запросы, возвращающие значения, полученные с помощью встроенных функций, включая разные функции обработки дат и времени суток, функции CAST и CONVERT, REPLACE, LEN и DATALENGTH, PATINDEX и CHARINDEX.

Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей правдоподобия будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

Дополнительная информация о тренировочных тестах

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.



ГЛАВА 2

Изменение данных — инструкции *INSERT*, *UPDATE*, *DELETE* и *MERGE*

Одна из основных функций, необходимых любой базе данных, — возможность добавлять, удалять и изменять данные. В языке программирования SQL для этого предназначен язык манипулирования данными (data manipulation language, DML). Команды *INSERT*, *UPDATE* и *DELETE* позволяют манипулировать данными на основе ввода информации или поиска ее в базе данных.

В версии Microsoft SQL Server 2005 было введено предложение *OUTPUT*, позволяющее возвращать информацию из каждой строки, обработанной инструкциями *INSERT*, *UPDATE* или *DELETE*. В версии SQL Server 2008 была введена инструкция *MERGE* для того, чтобы усовершенствовать выполнение инструкций *INSERT*, *UPDATE* и *DELETE*, обращенных к таблице, основанной на результатах запроса к соединенной таблице. Эти два средства повышают гибкость ваших инструкций DML.

Для обеспечения согласованности и надежности данных SQL Server предоставляет возможность выполнения инструкций в виде составной части транзакции. *Транзакция* — это группа команд, которая выполняется успешно или терпит неудачу как единое целое. Один из наиболее часто приводимых примеров транзакций — перевод денежных средств со сберегательного счета на текущий счет. Должно быть пройдено как минимум два шага: списание со сберегательного счета и занесение на текущий счет. Транзакция должна гарантировать, что если после завершения части транзакции система вышла из строя, вся транзакция будет возвращена к исходному состоянию.

Дополнительная информация об обработке ошибок

Сведения о применении транзакций в хранимых процедурах и обработке ошибок см. в главе 5.

Экзаменационные темы в этой главе

- ◆ Изменение данных с помощью инструкций INSERT, UPDATE и DELETE.
- ◆ Возврат данных с помощью предложения OUTPUT.
- ◆ Модификация данных с помощью инструкций MERGE.
- ◆ Управление транзакциями.

Уроки в этой главе

- ◆ Урок 1. Изменение данных с помощью инструкций INSERT, UPDATE и DELETE.
- ◆ Урок 2. Расширение функциональных возможностей DML с помощью предложения OUTPUT и инструкции MERGE.
- ◆ Урок 3. Управление транзакциями.

Прежде чем начать

Для усвоения уроков этой главы необходимо:

- ◆ иметь представление о концепциях разработки реляционных баз данных и применяемой терминологии;
- ◆ знать базовые типы данных SQL Server;
- ◆ иметь общее представление о синтаксической записи инструкции SELECT;
- ◆ знать о том, как работает среда SQL Server Management Studio (SSMS), и уметь в среде SSMS открывать окна для создания запроса и файлы существующих запросов;
- ◆ должны быть установлены программа SQL Server 2008 версии Developer, Enterprise или Enterprise Evaluation и учебные базы данных AdventureWorks2008 и Northwind.

РЕАЛЬНАЯ ИСТОРИЯ.

Анна Вебер (Ann Weber)

Транзакции — это ключ для обеспечения всех пользователей в любое время согласованным и достоверным представлением данных. Как консультант и преподаватель я прошла через множество трудностей, вызванных плохо написанными транзакциями. Одна из наиболее досадных ошибок, с которыми я сталкивалась, была связана с возникающими в приложении время от времени взаимоблокировками или тупиковыми ситуациями, вызванными длительными транзакциями, имеющимися в компании. Сокращение транзакции и изменение порядка обращения к таблицам в пределах транзакций минимизировало количество возникавших взаимоблокировок. Но еще более досадным был тот факт, что программный код не обрабатывал ошибки. Пользователи теряли все данные, которые они вводили в этот момент (а полный ввод занимал у них до 15 минут), и получали следующее сообщение об ошибке: "Transaction (Process ID xx) was deadlocked on lock resources with another process and has been chosen as the deadlock victim. Rerun the transaction". (Транзакция (Процесс ID xx) при обращении к ресурсам, заблокированным другим процессом, оказалась

в тупиковой ситуации и была выбрана жертвой или потерпевшей в этой взаимоблокировке. Выполните транзакцию еще раз.). Даже если вы не ждете возникновения взаимоблокировок, необходимо предусмотреть такую возможность в программе. Пользователи никогда не должны видеть сообщения подобного рода. Клиент должен был перехватывать пользовательский ввод до открытия транзакции, затем перехватить ошибку взаимоблокировки и попытаться повторно послать пользовательский ввод перед окончательной отправкой дружелюбного по отношению к пользователю сообщения, констатирующего аварийное завершение вставки и советующего повторить попытку позже.

УРОК 1. Изменение данных с помощью инструкций *INSERT*, *UPDATE* и *DELETE*

Инструкции *INSERT*, *UPDATE* и *DELETE* позволяют добавлять, изменять и удалять данные в ваших базах данных. Эти инструкции могут применяться разными способами, обладающими возможностью задавать конкретные значения. На значения, которые можно или нельзя определять, изменять либо удалять в инструкциях *INSERT*, *UPDATE* и *DELETE* влияет *схема таблицы* (также называемая *определением таблицы* (table definition)). Например, для создания идентифицирующего столбца, которому при каждой вставке новой строки автоматически присваиваются возрастающие значения, используется свойство *IDENTITY*. По этой причине в нормальных обстоятельствах при выполнении инструкции *INSERT* или *UPDATE* вы не можете ввести значение идентифицирующего столбца. Преодолеть это ограничение можно с помощью применения в текущем соединении команды *SET IDENTITY_INSERT ON*. Определения *NULL* и ограничения также оказывают влияние на инструкции языка DML.

Дополнительная информация об ограничениях и идентифицирующих столбцах

Дополнительную информацию о табличных схемах, ограничениях и свойстве *IDENTITY* см. в главе 3.

После усвоения этого урока вы сможете

- Применять инструкцию *INSERT* для добавления новых строк в ваши таблицы.
- Использовать инструкцию *UPDATE* для изменения строк в ваших таблицах.
- Применять инструкцию *DELETE* для удаления строк из ваших таблиц.

Расчетное время урока — 45 мин.

РЕАЛЬНАЯ ИСТОРИЯ. Анна Вебер (Ann Weber)

Инструкции DML — основа функциональных средств в SQL Server. Как инструктор и консультант я много раз сталкивалась с потерями данных, вызванными неверно разработанными инструкциями *UPDATE* или *DELETE*. Для минимизации риска потери

данных при написании команд DML стоит соблюдать некоторые предосторожности. Во-первых, разрабатывайте и тестируйте программный код на неосновном сервере. Во-вторых, для проверки корректности возвращаемого результирующего набора выполняйте логическое построение команды UPDATE, INSERT или DELETE в виде инструкции SELECT. Кроме того, либо выполняйте команду как часть транзакции в явном виде и не фиксируйте транзакцию, пока не убедитесь в ее правильности, либо включите неявные транзакции на время построения незапланированных произвольных запросов на изменение данных. Помните, что в случае неявных транзакций инструкция DML запускает транзакцию автоматически, но инструкцию COMMIT или ROLLBACK вы должны выполнять вручную. Тот факт, что неявные транзакции на сервере под управлением SQL Server по умолчанию отключены, застал врасплох многих разработчиков Oracle. Если не изменить установочные параметры SQL Server, принятые по умолчанию, или не запустить транзакцию в явном виде, нет способа выполнить откат команды после ее выполнения. На серверах Oracle поведение неявных транзакций всегда активизировано. Если вы хотите добиться того же результата на SQL Server, для каждого создаваемого соединения сделайте активным свойство соединения SET_IMPLICIT_TRANSACTIONS.

Вставка данных

Инструкция INSERT позволяет вставлять в таблицу новые строки. В зависимости от схемы таблицы вам может понадобиться предоставить данные для всех или части столбцов таблицы. На требования к данным в ваших инструкциях INSERT могут оказывать влияние ограничения DEFAULT, свойства IDENTITY и значения NULL.

Синтаксическая запись инструкции INSERT

У команды INSERT следующая синтаксическая запись:

```
[ WITH <common_table_expression> [ ,...n ] ]
INSERT
    [ TOP ( expression ) [ PERCENT ] ]
    [ INTO ]
    { <object> | rowset_function_limited
      [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
    }
{
    [ ( column_list ) ]
    [ <OUTPUT Clause> ]
    { VALUES ( { DEFAULT | NULL | expression } [ ,...n ] ) [ ,...n ]
      | derived_table
      | execute_statement
      | <dml_table_source>
      | DEFAULT VALUES
    }
}
[; ]
```

Дополнительная информация о предложениях WITH <COMMON TABLE EXPRESSION> и OUTPUT

Более подробную информацию о предложении WITH и обобщенных табличных выражениях см. в главе 4. Дополнительную информацию о предложении OUTPUT см. в уроке 2 этой главы.

Применение инструкции INSERT

При вставке можно добавлять данные во все столбцы или в часть из них. При добавлении данных во все столбцы вам не нужно задавать в инструкции INSERT имена столбцов, как показано в приведенном далее примере.

```
INSERT INTO Sales.SalesReason
VALUES ('Item Closeout', 'Other', getdate());
```

В этой команде данные столбца SalesReasonID не вводятся, т. к. это идентифицирующий столбец, и он заполняется автоматически с помощью механизма управления SQL Server.

У столбца ModifiedDate (Дата последнего обновления строки) таблицы SalesReason (Причина покупки клиентом товара) есть связанное с ним ограничение DEFAULT, которое задается в виде функции getdate(). По этой причине следующая команда приведет к тем же результатам, что и предыдущая.

```
INSERT INTO Sales.SalesReason
VALUES ('Item Closeout', 'Other', DEFAULT);
```

Необязательное ключевое слово INTO

Ключевое слово INTO необязательно во всех инструкциях INSERT.

Если в столбце допускаются значения NULL, для вставки строки с данными, пропущенными в одном или нескольких столбцах, можно использовать ключевое слово NULL, как показано полужирным шрифтом в следующем примере.

```
INSERT INTO Sales.SpecialOffer
VALUES ('Temporary Holiday Discount', .10, 'Seasonal Discount'
, 'Reseller', GETDATE(), DATEADD(dd, 30, getdate())
, 1, NULL, DEFAULT, DEFAULT);
```

Если вы хотите задать конкретные столбцы, при написании инструкции INSERT можно сослаться на их имена, как показано в следующем примере.

```
INSERT INTO Sales.SpecialOffer
(Description, DiscountPct, Type, Category, StartDate, EndDate, MinQty)
VALUES ('Temporary Holiday Discount', .10, 'Seasonal Discount'
, 'Reseller', GETDATE(), DATEADD(dd, 30, getdate()), 1);
```

Применение инструкции INSERT...SELECT позволяет добавлять в существующую таблицу строки, основанные на данных, извлеченных из другой таблицы. В следующей команде берутся строки из таблицы Employees (Сотрудники) и добавля-

ются в таблицу Customers (Клиенты). Поскольку количество и тип данных столбцов в результирующем наборе должны соответствовать целевой таблице, для получения корректного числа столбцов применяется конкатенация (сцепление). В следующей команде с функцией SUBSTRING извлекаются первые пять букв фамилии сотрудника для формирования ID клиента, подобного идентификатору в таблице Customers (Клиенты).

```
INSERT INTO Customers
SELECT SUBSTRING(lastname,1,5), 'Northwind Traders'
      , FirstName + ' ' + LastName, 'Employee', Address, City, Region
      , PostalCode, Country, HomePhone, NULL
FROM Employees
```

База данных

В только что обсуждавшемся примере используется база данных Northwind.

И наконец, инструкция SELECT INTO позволяет создать новую временную или постоянную таблицу, заполненную результатами заданной инструкции SELECT. Следующая команда создает временную таблицу с именем #EmployeeDepartment (Отдел сотрудника), включающую имя и фамилию каждого сотрудника, его должность и сведения об отделе.

```
SELECT IDENTITY (int, 1,1) AS EmpID, FirstName, LastName, JobTitle
      , Name AS 'Department', GroupName as 'Division'
INTO #EmployeeDepartment
FROM HumanResources.Employee
JOIN Person.Person
  ON HumanResources.Employee.BusinessEntityID =
     Person.Person.BusinessEntityID
JOIN HumanResources.EmployeeDepartmentHistory
  ON HumanResources.Employee.BusinessEntityID =
     HumanResources.EmployeeDepartmentHistory.BusinessEntityID
JOIN HumanResources.Department
  ON HumanResources.Department.DepartmentID =
     HumanResources.EmployeeDepartmentHistory.DepartmentID
WHERE HumanResources.EmployeeDepartmentHistory.EndDate IS NULL
ORDER BY Department, Division
```

Изменение данных

Со временем возникает необходимость модифицировать имеющуюся в базе данных информацию. Команда UPDATE позволяет изменять значение одного или нескольких столбцов в одной или нескольких строках таблицы. Добавление значения в столбец, равный в данный момент значению NULL, и удаление значения из столбца выполняются инструкциями UPDATE, а не инструкциями INSERT или DELETE. Важно помнить о том, что инструкция INSERT добавляет в таблицу новую строку целиком, а инструкция DELETE удаляет из таблицы полную строку.

Синтаксическая запись инструкции **UPDATE**

У инструкции **UPDATE** следующая синтаксическая запись:

```
[ WITH <common_table_expression> [...n] ]
UPDATE
    [ TOP ( expression ) [ PERCENT ] ]
    { <object> | rowset_function_limited
      [ WITH ( <Table_Hint_Limited> [ ...n ] ) ]
    }
SET
    { column_name = { expression | DEFAULT | NULL }
      | { udt_column_name. { property_name = expression
                           | field_name = expression }
        | method_name ( argument [ ,...n ] )
      }
    }
    | column_name { .WRITE ( expression, @Offset, @Length ) }
    | @variable = expression
    | @variable = column = expression
    | column_name { += | -= | *= | /= | %= | &= | ^= | |= }
      expression
    | @variable { += | -= | *= | /= | %= | &= | ^= | |= }
      expression
    | @variable = column { += | -= | *= | /= | %= | &= | ^= | |= }
      expression
    } [ ,...n ]
[ <OUTPUT Clause> ]
    [ FROM { <table_source> } [ ,...n ] ]
    [ WHERE { <search_condition>
      | { [ CURRENT OF
          { { [ GLOBAL ] cursor_name }
            | cursor_variable_name
          }
        }
    ]
  }
]
[ OPTION ( <query_hint> [ ,...n ] ) ]
[ ; ]
```

Применение инструкции **UPDATE**

В простейшей форме инструкции **UPDATE** требуется только имя изменяемого объекта (таблицы или представления) и устанавливаемое значение. Предложение **WHERE** определяет, какие строки обновляются командой. Поскольку единственная инструкция **UPDATE** обладает способностью обновлять все строки таблицы одной

командой, для поиска и обработки многострочных обновлений необходимо разрабатывать программные средства, такие как хранимые процедуры и триггеры.

Следующая команда обновляет каждую строку в таблице `CurrencyRate` (Валютный курс) и добавляет 0.005 к валютному курсу на конец текущего дня.

```
UPDATE Sales.CurrencyRate
SET EndOfDayRate = EndOfDayRate + .005;
```

Если вам нужно изменить только конкретные строки, следует задать предложение `WHERE` для ограничения количества строк. Следующая команда в таблице `SalesReason` (Причина покупки товара клиентом) для `SalesReasonID` со значением 10 задает значение столбца `Name`, равное 'N/A'.

```
UPDATE Sales.SalesReason
SET Name = 'N/A'
WHERE SalesReasonID = 10;
```

С помощью предложения `FROM` вы также можете обновить информацию в целевой таблице, основываясь на информации из таблицы, присоединенной к ней. Например, следующая команда увеличивает на 5% цену за единицу для всех товаров красного цвета.

```
UPDATE Sales.SalesOrderDetail
SET UnitPrice = UnitPrice * 1.05
FROM SALES.SalesOrderDetail JOIN Production.Product
ON SALES.SalesOrderDetail.ProductID = Production.Product.ProductID
WHERE Production.Product.Color = 'Red'
```

ПРАКТИЧЕСКИЙ СОВЕТ.

Проверка внесенных изменений данных

Опишите логическую структуру инструкции `UPDATE` или `DELETE` в виде инструкции `SELECT` и проверьте логику, прежде чем выполнять вместо `SELECT` инструкцию `UPDATE` или `DELETE`. Кроме того, выполняйте изменения данных как часть явной транзакции, чтобы иметь возможность отката в случае непредусмотренных изменений. Транзакции будут обсуждаться в *уроке 3 данной главы*.

Удаление данных

Инструкция `DELETE` позволяет удалять из базы данных строки, которые больше не нужны. С ее помощью строки из таблицы удаляются целиком. Если нужно удалить только значение в столбце заданной строки, следует применять инструкцию `UPDATE`.

Синтаксическая запись инструкции *DELETE*

Полная синтаксическая запись инструкции `DELETE` выглядит следующим образом:

```
[ WITH <common_table_expression> [ ,...n ] ]
DELETE
[ TOP ( expression ) [ PERCENT ] ]
```

```

[ FROM ]
{ <object> | rowset_function_limited
  [ WITH ( <table_hint_limited> [ ...n ] ) ]
}
[ <OUTPUT Clause> ]
[ FROM <table_source> [ ,...n ] ]
[ WHERE { <search_condition>
  | { [ CURRENT OF
      { { [ GLOBAL ] cursor_name }
        | cursor_variable_name
      }
    ]
  }
}
]
[ OPTION ( <Query Hint> [ ,...n ] ) ]
[; ]

<object> ::=
{
  [ server_name.database_name.schema_name.
    | database_name. [ schema_name ] .
    | schema_name.
  ]
  table_or_view_name
}

```

Курсоры

Параметры CURRENT OF, GLOBAL, *cursor_name* и *cursor_variable_name* применимы только в случае использования курсоров. Обсуждение курсоров выходит за рамки книги. Дополнительную информацию о курсорах см. в разд. "Transact-SQL Cursors" (Курсоры языка Transact-SQL) электронной документации.

Применение инструкции DELETE

У инструкции DELETE самая простая запись из трех синтаксических записей инструкций DML. Как и в случае инструкции UPDATE, при написании инструкции DELETE важно не забывать о вставке предложения WHERE. Если выполнить инструкцию DELETE без предложения WHERE, из таблицы будут удалены все строки.

Синтаксическая запись удаления из таблицы всех строк очень проста.

```
DELETE FROM <tablename>
```

Необязательное ключевое слово FROM

В синтаксической записи инструкции DELETE присутствие ключевого слова FROM не обязательно.

Для ограничения количества удаляемых строк добавляйте в инструкцию `DELETE` предложение `WHERE`. В следующем примере из таблицы `Employees` (Сотрудники) удаляются все строки, относящиеся к сотруднику с идентификатором 7.

```
DELETE FROM EMPLOYEES  
WHERE EmployeeID = 7
```

База данных

В этом примере используется база данных Northwind, хотя из-за ограничения внешнего ключа для таблицы `Orders` (Заказы) она не очень подходит в качестве демонстрационной базы данных.

Как и команды `INSERT` и `UPDATE`, инструкция `DELETE` может удалить строки в одной таблице, основываясь на информации, возвращенной из присоединенной таблицы. Для этого определяется второе предложение `FROM`. Следующая команда удаляет из таблицы `Order Details` (Подробные сведения о заказе) строки с заказами, которые были помещены до 10 июля 1996 г. и были доставлены.

```
DELETE FROM [Order Details]  
FROM ORDERS  
JOIN [Order Details] ON Orders.OrderID = [Order Details].OrderID  
WHERE OrderDate < '07-10-1996' AND ShippedDate IS NOT NULL
```

Применение инструкции TRUNCATE TABLE

Инструкцию `TRUNCATE TABLE`, как и инструкцию `DELETE` без предложения `WHERE`, можно использовать для удаления всех данных из таблицы. У инструкции `TRUNCATE TABLE` есть несколько отличий от инструкции `DELETE`. Во-первых, инструкция `DELETE` регистрирует сведения о каждой удаленной строке, а инструкция `TRUNCATE TABLE` создает только элементы, необходимые для освобождения страниц данных. Во-вторых, инструкция `TRUNCATE TABLE` выполняется быстрее и требует меньше ресурсов сервера благодаря минимальному протоколированию наряду со способом, которым Database Engine удаляет данные из таблицы. Наконец, если в таблице есть идентифицирующий столбец, команда `TRUNCATE TABLE` переустанавливает начальное значение, применяемое для идентификации.

У инструкции `TRUNCATE TABLE` простая синтаксическая запись.

```
TRUNCATE TABLE <tablename>
```

Ни инструкция `DELETE` без предложения `WHERE`, ни инструкция `TRUNCATE TABLE` не влияют на структуру схемы таблицы или связанных объектов.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Изменение данных

На этом практическом занятии вы с помощью разных методов добавите новые данные в таблицу `Credit` (Кредит) из базы данных `Sales` (Продажи). Вы также обновите и удалите данные из таблицы `Credit`.

Упражнение 1. Добавление данных

В этом упражнении вы вставите данные в таблицу `Credit` и проверите результаты.

1. Если необходимо, запустите среду SSMS и подключитесь к вашему экземпляру SQL Server.
2. Для создания базы данных `Sales` (Продажи) и таблицы `Credit` (Кредит), используемых в данных упражнениях, откройте, просмотрите и выполните файл `Lesson01 PracticeSetup.sql`, который можно найти среди других файлов примеров в папке `Practice`.
3. Закройте текущее окно запроса.
4. Откройте окно создания запроса, введите и выполните следующую команду, чтобы просмотреть столбцы таблицы `Credit`. В этот момент в таблице нет строк.

```
USE Sales;  
GO  
SELECT * FROM Credit;
```

5. В текущем окне запроса под имеющимся программным кодом введите, выделите и выполните следующую команду, чтобы вставить в таблицу `Credit` новую строку, задавая конкретные значения для каждого столбца в соответствии с его относительным местоположением в схеме таблицы.

```
INSERT INTO Credit  
VALUES ('Izak', 'Cohen', 5000, 'izak@adatum.com');
```

Дополнительная информация об идентифицирующих столбцах

Для столбца `CustomerID` (ID клиента) нельзя задать значение, поскольку он описан как идентифицирующий (ключевой) столбец. Более подробную информацию о свойстве `IDENTITY` см. в главе 3.

6. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду, чтобы, задавая значения для каждого столбца в соответствии с его относительным местоположением, вставить в таблицу `Credit` новую строку, в которой данные предоставлены не для всех столбцов. В качестве ограничения величины кредита вводится "0" как значение по умолчанию, заданное во время создания таблицы.

```
INSERT INTO Credit  
VALUES ('David', 'Hamilton', DEFAULT, NULL);
```

7. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду, чтобы, задавая значения для каждого столбца в соответствии с его относительным местоположением, вставить в таблицу `Credit` новую строку, в которой данные предоставлены не для всех столбцов.

Эта команда завершится аварийно, т. к. количество заданных столбцов не соответствует определению таблицы.

```
INSERT INTO Credit  
VALUES ('Don', 'Hall', 5000);
```

8. Для того чтобы исправить эту ошибку, удалите предыдущую команду и введите, выделите и выполните следующую команду. В этой команде в предложении VALUES указано нужное количество столбцов в должном порядке и в таблицу вставляются две строки.

```
INSERT INTO Credit (firstname, lastname, creditlimit)  
VALUES ('Don', 'Hall', 5000);  
INSERT INTO Credit (firstname, lastname, creditlimit)  
VALUES ('Punya', 'Palit', 10000);
```

9. В текущем окне запроса введите, выделите и выполните следующую команду для просмотра строк, добавленных в таблицу Credit:

```
SELECT * FROM CREDIT;
```

10. Сохраните сценарий и закройте окно запроса. Оставьте открытой среду SSMS для следующего упражнения.

Упражнение 2. Обновление данных

В этом упражнении вы обновите данные, введенные в таблицу Credit в упражнении 1. Затем вы проверите результаты каждой инструкции UPDATE.

1. Откройте окно создания запроса, введите и выполните следующую команду для добавления адреса электронной почты клиента Punya Palit (CustomerID 4).

```
USE Sales;  
GO  
UPDATE Credit  
SET Email = 'punya@adatum.com'  
WHERE CustomerID = 4;
```

2. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для просмотра строки, измененной в пункте 1.

```
SELECT * FROM Credit WHERE CustomerID = 4;
```

3. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для вставки значения 500 в каждую строку, в столбец CreditLimit.

```
UPDATE Credit  
SET CreditLimit = CreditLimit + 500;
```

4. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для просмотра строк, измененных в пункте 3.

Обратите внимание на то, что предельные значения кредита увеличились на 500.

```
SELECT * FROM Credit;
```

5. Сохраните сценарий и закройте окно запроса. Оставьте открытой среду SSMS для следующего упражнения.

Упражнение 3. Удаление данных

В этом упражнении вы удалите данные, которые ввели в таблицу `Credit` в упражнении 1. Вы также проверите действие инструкции `TRUNCATE TABLE`.

1. Откройте окно создания запроса, введите и выполните следующую команду для удаления строки о клиенте `Punya Palit` (`CustomerID = 4`).

```
USE SALES;  
GO  
DELETE FROM Credit  
WHERE CustomerID = 4;
```

2. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для просмотра оставшихся строк.

```
SELECT * FROM Credit;
```

3. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для удаления всех строк из таблицы `Credit` и проверки результатов.

```
DELETE FROM Credit;  
GO  
SELECT * FROM Credit;
```

4. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для того, чтобы вставить новую строку в таблицу кредитов и проверить результаты. Обратите внимание на то, что для поля `CustomerID` выбрано следующее увеличенное значение, несмотря на то, что все предыдущие строки были удалены.

```
INSERT INTO Credit (firstname, lastname, creditlimit)  
VALUES ('Punya', 'Palit', 10000);
```

```
SELECT * FROM Credit;
```

5. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду для усечения таблицы и переустановки значения свойства `Identity` столбца `CustomerID` в его исходное начальное значение.

```
TRUNCATE TABLE Credit;
```

6. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующую команду, чтобы добавить новую строку в таблицу кредитов

и проверить результаты. Обратите внимание на то, что значение столбца `CustomerID` равно исходному начальному значению.

```
INSERT INTO Credit (firstname, lastname, creditlimit)
VALUES ('Punya', 'Palit', 10000);
```

```
SELECT * FROM Credit;
```

7. Сохраните сценарий и закройте окно запроса.
8. Убедитесь в том, что вы закрыли все окна запросов, открытые во время этого практического занятия, и затем откройте, просмотрите и выполните файл `Lesson01 PracticeCleanup.sql`, который можно найти среди сопроводительных файлов примеров в папке `Chapter 2\Lesson 1`.

Резюме урока

- ◆ Инструкция `INSERT` позволяет вставлять в таблицу новые строки.
- ◆ Инструкция `UPDATE` позволяет изменять имеющиеся в таблице данные. Она дает возможность не только изменять значение в столбце, но и добавлять значение или удалять его из отдельного столбца таблицы, не затрагивая остальные данные корректируемой строки.
- ◆ Инструкция `DELETE` позволяет удалять из таблицы одну или несколько строк.

УРОК 2. Расширение функциональных возможностей DML с помощью предложения *OUTPUT* и инструкции *MERGE*

Предложение `OUTPUT` и инструкцию `MERGE` можно применять для расширения функциональных возможностей, предоставляемых инструкциями языка DML. Предложение `OUTPUT` позволяет возвращать информацию из строк, обработанных инструкциями `INSERT`, `UPDATE` или `DELETE`. Благодаря этому средству вы сможете с большей уверенностью решать дополнительные задачи, полагаясь на предоставленную информацию. К ним могут относиться подтверждение адресов электронной почты, контроль данных и аналогичные задачи.

Инструкция `MERGE` предоставляет возможность выполнять действие `INSERT`, `UPDATE` или `DELETE` над целевой таблицей, основываясь на наборе правил, которые определяются сравнением строк целевой и исходной таблиц.

После усвоения этого урока вы сможете

- Применять предложение `OUTPUT` для возврата информации из вставленных, удаленных и измененных строк.

- Использовать инструкцию `MERGE` для выполнения инструкций `INSERT`, `UPDATE` или `DELETE`, основываясь на сравнении двух таблиц.

Расчетное время урока — 30 мин.

Применение предложения *OUTPUT*

Предложение `OUTPUT` позволяет получить доступ к вставленным или удаленным таблицам, которые в версиях, предшествующих SQL Server 2005, были доступны только благодаря триггерам. По этой причине некоторые действия, выполнявшиеся раньше с помощью триггеров, теперь могут выполняться хранимыми процедурами, и потребность в определенных триггерах отпадает.

Дополнительная информация о триггерах и хранимых процедурах

Принимая решение о реализации функциональной возможности с помощью хранимых процедур и триггеров, следует рассматривать как преимущества, так и недостатки такого решения. Дополнительную информацию о триггерах и хранимых процедурах см. в главе 5.

Синтаксическая запись предложения *OUTPUT*

Синтаксическая запись предложения `OUTPUT` может быть включена в любую инструкцию `INSERT`, `UPDATE` или `DELETE`. Для предложения `OUTPUT` характерна следующая синтаксическая запись:

```
<OUTPUT_CLAUSE> ::=
{
    [ OUTPUT <dml_select_list>
        INTO { @table_variable | output_table } [ ( column_list ) ] ]
    [ OUTPUT <dml_select_list> ]
}
<dml_select_list> ::=
{ <column_name> | scalar_expression } [ [AS] column_alias_identifier ]
    [ ,...n ]

<column_name> ::=
{ DELETED | INSERTED | from_table_name } . { * | column_name } | $action
```

Примеры использования предложения *OUTPUT*

С помощью предложения `OUTPUT` можно реализовать множество операций. Далее приведено несколько примеров.

База данных и сценарии примеров

В примерах запросов с предложением `OUTPUT` используется база данных Northwind. Сценарии для создания примеров данных и запросы, представленные в виде текста, можно найти среди сопроводительных файлов примеров в папке Chapter2\TextSamples.

Во многих рабочих средах необходимо записывать вставляемые данные в контрольную таблицу. В следующем примере компания хочет иметь возможность формировать отчеты, показывающие, сколько строк было вставлено за разные периоды времени. В требованиях предприятия указано, что в специальную контрольную таблицу, применяемую для формирования отчетов, следует включать дату, когда строка была добавлена, и идентификатор первичного ключа из исходной таблицы. Один из способов сделать это — применить предложение OUTPUT. В данном примере таблица `Audit` (Контрольная) включает столбец `AuditID`, являющийся идентифицирующим, столбец `InsertedDate`, содержащий дату добавления строки, и столбец `InsertedID` со значением первичного ключа из строки, добавленной в таблицу, изменения которой отслеживаются. Следующая команда добавляет новую строку в таблицу `Employees` (Сотрудники) и соответствующую строку в таблицу `Audit` с текущей датой и временем суток и значением `EmployeeID` для нового сотрудника.

```
INSERT INTO Employees (LastName, FirstName, Title)
OUTPUT getdate(), inserted.EmployeeID INTO Audit
VALUES ('Ralls', 'Kim', 'Support Rep');
```

Другой сценарий включает в себя архивирование данных. Предположим, что компания хочет переместить все записи о заказах, помещенных до December 1, 1997 (1 декабря 1997 г.), из таблицы `Order Details` (Подробные сведения о заказе) в таблицу `OrderDetailsArchive` (Архив подробных сведений о заказах). Вы можете удалить строки из таблицы `Order Details` и перенести их в таблицу `OrderDetailsArchive` за один шаг, применив следующую инструкцию DELETE с предложением OUTPUT.

```
DELETE FROM [Order Details]
OUTPUT deleted.* INTO OrderDetailsArchive
FROM Orders join [Order Details]
ON Orders.OrderID = [Order Details].OrderID
WHERE OrderDate < '12-01-1997';
```

И последний сценарий предназначен для компании, желающей видеть исходное и окончательное состояние столбца `CategoryName` (Название категории) при каждом его изменении. Эта информация вместе с датой изменения и регистрационным ID каждого сотрудника, внесившего изменения, собирается в таблице `CategoryChanges` (Изменения категорий). Приведенная далее команда добавляет информацию в таблицу `CategoryChanges`, которая содержит следующие столбцы: `ChangeID` (ID изменения), `CategoryID` (ID категории), `OldCategoryName` (Старое название категории), `NewCategoryName` (Новое название категории), `ModifiedDate` (Дата изменения) и `LoginID` (Регистрационный ID). Команда также изменяет название категории и добавляет строку в таблицу `CategoryChanges`.

```
UPDATE Categories
SET CategoryName = 'Dried Produce'
```

```

OUTPUT inserted.CategoryID, deleted.CategoryName
      , inserted.CategoryName, getdate(), SUSER_SNAME()
      INTO CategoryChanges
WHERE CategoryID = 7;

```

Применение инструкции *MERGE*

Инструкция *MERGE* наряду с системой отслеживания измененных данных (change data capture, CDC), средства, появившиеся в версии SQL Server 2008, значительно расширили функциональные возможности хранилищ данных (warehouses) и вспомогательных, промежуточных баз данных (staging databases). Инструкция *MERGE* позволяет сравнить строки в исходной и целевой таблицах. Затем, на основе результатов сравнения, можно задать подходящую команду *INSERT*, *UPDATE* или *DELETE*, которая должна быть выполнена.

Дополнительная информация о системе отслеживания измененных данных (CDC)

Более подробные сведения о CDC см. в главе 9.

Синтаксическая запись инструкции *MERGE*

У инструкции *MERGE* следующая синтаксическая запись:

```

[ WITH <common_table_expression> [,...n] ]
MERGE
    [ TOP ( expression ) [ PERCENT ] ]
    [ INTO ] target_table [ WITH ( <merge_hint> ) ]
    [ [ AS ] table_alias ]
    USING <table_source>
    ON <merge_search_condition>
    [ WHEN MATCHED [ AND <clause_search_condition> ]
      THEN <merge_matched> ]
    [ WHEN NOT MATCHED [ BY TARGET ] [ AND <clause_search_condition> ]
      THEN <merge_not_matched> ]
    [ WHEN NOT MATCHED BY SOURCE [ AND <clause_search_condition> ]
      THEN <merge_matched> ]
    [ <output_clause> ]
    [ OPTION ( <query_hint> [ ,...n ] ) ];

```

В синтаксической записи *MERGE* как ее составные части могут быть заданы следующие варианты.

- ◆ **[INTO] <target_table>.** Определяет таблицу или представление, в которые будут вставляться, в которых будут обновляться или из которых будут удаляться строки, возвращенные предложениями *WHEN*. Эти таблица или представление также используются для сопоставления данных со строками исход-

ной таблицы `<table_source>` на основе условия `<clause_search_condition>`. Если `<target_table>` — это представление, то для успешного выполнения все условия, касающиеся обновления представления, должны быть подходящими для инструкции MERGE.

- ◆ `[AS] table_alias`. Задаёт псевдонимы, которые могут применяться для сокращения набора или большей наглядности за счёт сокращения имен таблиц, на которые делаются многочисленные ссылки в одной команде.
- ◆ `USING <table_source>`. Определяет таблицу, представление или выражение, из которых берутся строки, сопоставляемые с целевой таблицей.
- ◆ `ON <merge_search_condition>`. Задаёт условия, которые должны применяться для определения того, совпадают ли строки в двух таблицах. Аналогична предложению `ON` в операторе `JOIN`, это может быть просто `<table1_id> = <table2_id>`.
- ◆ `WHEN MATCHED THEN <merge_matched>`. Определяет действие, выполняемое со строками в целевой таблице, для которых существует соответствие между строками исходной и целевой таблиц, основанное на предложении `ON <merge_search_condition>` и любых дополнительных условиях, заданных как часть предложения `WHEN MATCHED THEN <merge_matched>`. Для успешности операции `UPDATE` исходная строка должна соответствовать только одной целевой строке. Одна инструкция `MERGE` может включать до двух предложений `WHEN MATCHED`, соединённых оператором `AND`. Если определены два предложения `WHEN MATCHED`, одно должно выполнять операцию `UPDATE`, а другое — `DELETE`. Кроме того, второе предложение `WHEN MATCHED` выполняется только если не выполняется первое. Например, предположим, что целевая таблица включает только строки о товарах, которые есть на складе. Она не содержит строк о товарах с нулевым или отрицательным запасом. Исходная таблица включает изменения складских запасов, и добавления и изъятия (представленные как положительные и отрицательные целые числа). Первое предложение `WHEN MATCHED` содержит условие о добавлении объёмов сопоставимых строк и последующей проверке того, что они больше 0. Если это так, объём в целевой таблице задаётся равным сумме исходного и целевого объёмов товара. Второе предложение `WHEN MATCHED` содержит условие проверки того, что сумма меньше или равна 0, и если это так, строка удаляется из целевой таблицы.
- ◆ `WHEN NOT MATCHED [BY TARGET] THEN <merge_not_matched>`. Указывает на то, что строка будет добавлена в целевую таблицу, если сопоставимая строка не найдена и если в предложении `WHEN NOT MATCHED [BY TARGET]` определены любые дополнительные условия. В инструкции `MERGE` может присутствовать только одно предложение `WHEN NOT MATCHED [BY TARGET]`.
- ◆ `WHEN NOT MATCHED BY SOURCE THEN <merge_matched>`. Определяет операцию `UPDATE` или `DELETE` над строками, которые присутствуют в целевой таблице и

отсутствуют в исходной. Как и в случае предложения `WHEN MATCHED`, инструкция `MERGE` может содержать до двух предложений `WHEN NOT MATCHED BY SOURCE` и, если присутствуют оба, одно должно быть определено как операция `UPDATE`, а другое — как `DELETE`. Если строки из исходной таблицы не возвращаются, нельзя ссылаться на ее столбцы в предложении `<merge_matched>`, иначе будет возвращена ошибка 207 (недопустимое имя столбца).

Примеры инструкций *MERGE*

Распространенный сценарий слияния — перемещение данных из одной таблицы в другую. Предположим, что компании необходимо скопировать информацию из таблицы `SalesOrderDetail` (Подробные сведения о заказе) в базе данных `AdventureWorks2008` в таблицу `SalesOrderDetailHistory` (Архив подробных сведений о заказах). В дополнение к столбцам, определенным в таблице `SalesOrderDetail`, таблица `SalesOrderDetailHistory` содержит столбец `Cancelled` (Аннулированная). Вместо применения временных меток (timestamp) или другого метода идентификации строк, измененных с момента последнего перемещения информации, инструкция `MERGE` может сравнить две таблицы и добавить только новые строки без необходимости вставки всех строк или поддержания столбцов `Timestamp` (Дата и время модификации). Использование системы отслеживания измененных данных, CDC, делает этот процесс еще эффективнее. В этом примере инструкция `MERGE` вставляет любые новые строки в таблицу `SalesOrderDetailHistory` и добавляет значение `True` в столбец `Cancelled` для любой строки, которой уже нет в исходной таблице. Поскольку бизнес-правила и ограничения данных запрещают обновления в таблице `SalesOrderDetails`, обновления строк не регистрируются и не передаются.

```
MERGE INTO Sales.SalesOrderDetailHistory AS SODH
    USING Sales.SalesOrderDetail AS SOD
    ON SODH.salesorderid = SOD.salesorderid
    AND SODH.SalesOrderDetailID = SOD.SalesOrderDetailID
WHEN NOT MATCHED BY TARGET THEN
    INSERT (Linetotal, SalesOrderID, SalesOrderDetailID
        , CarrierTrackingNumber, OrderQty, ProductID, SpecialOfferID
        , UnitPrice, UnitPriceDiscount
        , rowguid, ModifiedDate, Cancelled)
VALUES (Linetotal, SalesOrderID, SalesOrderDetailID
    , CarrierTrackingNumber, OrderQty, ProductID
    , SpecialOfferID, UnitPrice, UnitPriceDiscount
    , rowguid, ModifiedDate, DEFAULT)
WHEN NOT MATCHED BY SOURCE THEN
    UPDATE SET SODH.Cancelled = 'True';
```

Как показано на рис. 2.1, для вывода сведений о том, делались ли вставки, обновления или удаления во время выполнения инструкции `MERGE`, можно применить предложение `OUTPUT` в сочетании с переменной `$action`.

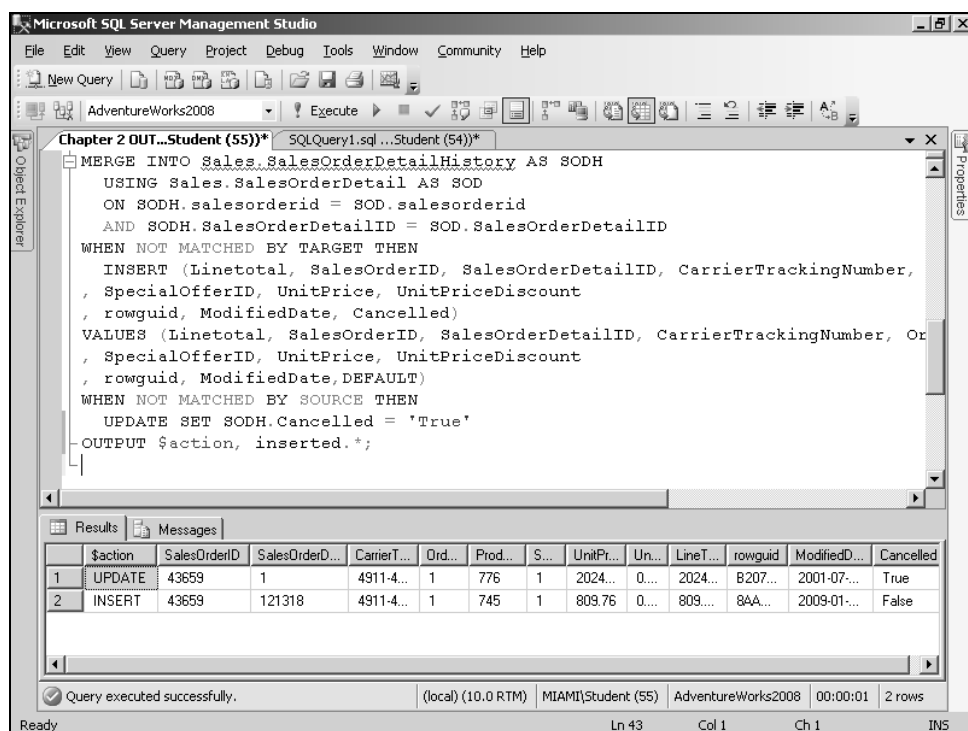


Рис. 2.1. Результаты выполнения предложения OUTPUT \$action

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Реализация расширенных средств DML

На этом практическом занятии вы примените предложение OUTPUT для вывода исходного и окончательного значений столбца с объемом товара в случае модификации строки таблицы SalesOrderDetail (Подробные сведения о заказе). Вы также воспользуетесь инструкцией MERGE в сочетании с предложением OUTPUT для сохранения информации в таблице SalesOrderDetailHistory (Архив подробных сведений о заказах) и отслеживания изменений, выполненных инструкцией MERGE, в таблице MergeAudit (Контроль Merge).

Упражнение 1. Применение предложения OUTPUT

В этом упражнении вы проверите тест, применяющий предложение OUTPUT для формирования табличной переменной с информацией, определенной по результатам команды UPDATE. Вы можете выполнить данное обновление как часть транзакции, получая возможность отказаться от изменений после успешного завершения теста. Транзакции подробно рассматриваются в уроке 3 данной главы.

1. Если необходимо, запустите среду SSMS, установите соединение с вашим экземпляром SQL Server и откройте окно создания запроса.
2. В окне создания запроса введите и выполните следующую команду, объявляющую переменную табличного типа, обновляющую строку в таблице SalesOrderDetail и выводящую результаты в табличную переменную.

```
USE AdventureWorks2008;

BEGIN TRANSACTION;

DECLARE @testoutput TABLE
    (SalesOrderID int, SalesOrderDetailID int, QtyBefore int
    , QtyAfter int, ModifiedDate datetime2, UserNmae varchar(30))
UPDATE Sales.SalesOrderDetail
SET OrderQty = 2
OUTPUT inserted.SalesOrderID, inserted.SalesOrderDetailID
    , deleted.OrderQty, inserted.OrderQty
    , GETDATE(), SUSER_SNAME()
INTO @testoutput
WHERE SalesOrderID = 43659 and SalesOrderDetailID = 1;

SELECT * FROM @testoutput;
```

3. Убедитесь, что строка с требуемыми данными возвращена.
 4. В открытом окне запроса под имеющимся программным кодом введите, выделите и выполните следующую команду для отмены изменения, внесенного в строку таблицы SalesOrderDetail.
- ```
ROLLBACK TRANSACTION;
```
5. Сохраните сценарий и закройте окно запроса. Оставьте открытой среду SSMS для следующего упражнения.

## Упражнение 2. Применение инструкции *MERGE* и предложения *OUTPUT*

В этом упражнении вы выполните сценарий, который создает в базе данных AdventureWorks2008 таблицу SalesOrderDetailHistory. Затем вы примените инструкцию MERGE и предложение OUTPUT для сохранения информации в таблице SalesOrderDetailHistory и предоставления контрольной информации об изменениях, выполненных в таблице SalesOrderDetailHistory.

1. В среде SSMS откройте окно создания запроса.
2. Для создания таблицы SalesOrderDetailHistory из таблицы SalesOrderDetail откройте, просмотрите и выполните файл Lesson02 PracticeSetup.sql, который можно найти среди сопроводительных файлов примеров в папке Practice. Сценарий также добавляет столбец Cancelled (Аннулированная) и устанавли-

вает начальные значения столбца, равные False. В заключение сценарий добавляет контрольную таблицу, хранящую информацию об изменениях, выполненных инструкцией MERGE.

3. Просмотрите результирующий набор и отметьте, что все строки из таблицы SalesOrder Detail были скопированы в новую таблицу SalesOrderDetailHistory. Это упражнение имеет дело с полем SalesOrderID (ID заказа на продажу), равным 43659, поэтому просмотрите строки, относящиеся к идентификатору заказа 43659.
4. Откройте окно создания запроса.
5. В открытое окно введите и выполните следующую команду для запуска новой транзакции и вставки строки в таблицу SalesOrder Detail, изменения в строке поля OrderQty (Объем заказа) и удаления строки из таблицы SalesOrder Detail.

```
USE AdventureWorks2008;
```

```
BEGIN TRANSACTION;
```

```
DELETE FROM Sales.SalesOrderDetail
WHERE SalesOrderID = 43659 AND SalesOrderDetailID = 1
INSERT INTO Sales.SalesOrderDetail
 (SalesOrderID, CarrierTrackingNumber, OrderQty, ProductID
 , SpecialOfferID, UnitPrice, UnitPriceDiscount
 , rowguid, ModifiedDate)
VALUES (43659, '4911-403-C-98', 1, 745, 1, 809.76, 0.00,
 DEFAULT, DEFAULT)
UPDATE Sales.SalesOrderDetail
SET OrderQty = 2
WHERE SalesOrderID = 43659 AND SalesOrderDetailID = 2;
```

6. В открытом окне запроса под имеющимся программным кодом введите, выделите и выполните следующий код для внесения изменений в таблицу Sales.SalesOrderDetailHistory и добавления необходимой информации о внесении изменений в таблицу MergeAudit.

```
SET IDENTITY_INSERT Sales.SalesOrderDetailHistory ON;
```

```
MERGE INTO Sales.SalesOrderDetailHistory AS SODH
 USING Sales.SalesOrderDetail AS SOD
 ON SODH.SalesOrderID = SOD.SalesOrderId
 AND SODH.SalesOrderDetailID = SOD.SalesOrderDetailID
WHEN NOT MATCHED BY TARGET THEN
 INSERT (Linetotal, SalesOrderID, SalesOrderDetailID
 , CarrierTrackingNumber, OrderQty, ProductID
 , SpecialOfferID, UnitPrice, UnitPriceDiscount
 , rowguid, ModifiedDate, Cancelled)
```

```

VALUES (Linetotal, SalesOrderID, SalesOrderDetailID
 , CarrierTrackingNumber, OrderQty, ProductID
 , SpecialOfferID, UnitPrice, UnitPriceDiscount
 , rowguid, ModifiedDate, DEFAULT)
WHEN NOT MATCHED BY SOURCE THEN
 UPDATE SET SODH.Cancelled = 'True'
WHEN MATCHED AND
 (SODH.OrderQty <> SOD.OrderQty
 OR SODH.SpecialOfferID <> SOD.SpecialOfferID
 OR SODH.UnitPrice <> SOD.Unitprice
 OR SODH.UnitPriceDiscount <> SOD.UnitPriceDiscount)
THEN
 UPDATE SET SODH.OrderQty = SOD.OrderQty
 , SODH.SpecialOfferID = SOD.SpecialOfferID
 , SODH.UnitPrice = SOD.Unitprice
 , SODH.UnitPriceDiscount = SOD.UnitPriceDiscount
 , SODH.Linetotal = SOD.Linetotal
OUTPUT inserted.salesorderid, inserted.salesorderdetailid
 , getdate(), $action
INTO MergeAudit;

SELECT * from Sales.SalesOrderDetailHistory
WHERE Salesorderid = 43659;

SELECT * FROM MergeAudit;
```

7. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующую команду для отмены выполненных вами изменений.

```
ROLLBACK TRANSACTION;
```

8. Сохраните сценарий и закройте окно запроса.
9. Убедитесь в том, что вы закрыли все окна запросов, открытые на этом практическом занятии, и затем откройте, просмотрите и выполните файл Lesson02 PracticeCleanup.sql, который можно найти среди сопроводительных файлов примеров в папке Chapter 2\Lesson 2.

## Резюме урока

- ◆ Предложение `OUTPUT` позволяет перенаправлять вызывающему приложению или объекту, такому как таблица или табличная переменная, информацию о выполненной инструкции `INSERT`, `UPDATE` или `DELETE`.
- ◆ Инструкция `MERGE` предоставляет возможность выполнять над целевой таблицей операции DML в зависимости от того, найдены или нет сведения о соответствии строк в исходной таблице.

## УРОК 3. Управление транзакциями

Поскольку обработка данных — важнейшая функция в базах данных оперативной обработки транзакций (Online Transaction Processing, OLTP), в процессе управления и сохранения непротиворечивых данных транзакции играют важную роль.

---

### ***После усвоения этого урока вы сможете***

---

- Объяснить, почему в SQL Server 2008 применяются транзакции.
- Определить неявные транзакции и установить сеанс поддержки неявных транзакций.
- Определить явные транзакции.
- Описать, как действует команда `ROLLBACK` в разных ситуациях, например при наличии точек сохранения информации (savepoint) или вложенных транзакций.
- Описать разные режимы блокировок в SQL Server, назначаемые ресурсам.
- Описать уровни изоляции транзакции.
- Задать уровни изоляции транзакций.

Расчетное время урока — 60 мин.

## Общие сведения о транзакциях

*Транзакции* часто определяются как набор действий или операций, которые выполняются успешно или завершаются аварийно как единое целое. Более строго, транзакции могут обеспечивать четыре основные характеристики процессов обработки данных, обращенных к базе данных.

- ◆ *Атомарность* — если в транзакцию вовлечены две или несколько порций данных, либо фиксируются все, либо ни одна.
- ◆ *Согласованность* — в конце транзакции либо существует новое адекватное представление данных, либо данные возвращаются в исходное состояние. Возвращение данных в исходное состояние — часть механизма отката, обеспечиваемого транзакциями SQL Server.
- ◆ *Изоляция* — во время выполнения транзакции (до того, как она будет зафиксирована или произойдет ее откат) данные должны оставаться изолированными и недоступными для других транзакций. Уровень изоляции может контролироваться в каждой транзакции, как будет описано позже в этом уроке.
- ◆ *Длительность* или долговечность — после фиксации транзакции окончательное состояние данных сохраняется, даже если сервер выходит из строя или перезагружается. Это свойство транзакции обеспечивается контрольными точками и процессом восстановления базы данных, выполняемым при запуске SQL Server.

Для обозначения этих четырех свойств используется сокращение ACID (Atomicity, Consistency, Isolation, Durability).

По умолчанию в SQL Server каждая инструкция `INSERT`, `UPDATE` или `DELETE` — это отдельная транзакция, которая фиксируется автоматически и не предоставляет возможности отката.

Вы можете активизировать неявные транзакции в установочных параметрах вашего соединения и механизм управления Database Engine автоматически начнет транзакцию при выполнении следующих команд: `ALTER TABLE`, `CREATE`, `DELETE`, `DENY`, `DROP`, `FETCH`, `GRANT`, `INSERT`, `OPEN`, `REVOKE`, `SELECT`, `TRUNCATE TABLE` или `UPDATE`. Транзакция активна до тех пор, пока вы вручную не введете инструкцию `COMMIT` или `ROLLBACK`. Активизировать неявные транзакции можно, применив инструкции `SET IMPLICIT_TRANSACTIONS ON` через прикладные программные интерфейсы (API) Object Linking and Embedding Database (OLE DB) или Open Database Connectivity (ODBC) на странице ANSI окна **Query Options** (Параметры запроса) в среде SSMS, или модифицировать свойства сервера, изменив его поведение по умолчанию на активизацию неявных транзакций во всех соединениях, если в конкретном соединении они явно не установлены в положение `OFF`.

## Определение явных транзакций

*Явные транзакции* обычно определяются в хранимых процедурах. Явная транзакция запускается, когда выполняется инструкция `BEGIN TRANSACTION`. Транзакция завершается при вводе инструкции `COMMIT TRANSACTION` или инструкции `ROLLBACK TRANSACTION`. После того как транзакция зафиксирована, SQL Server даже в аварийных ситуациях гарантирует, что данные записаны в базу данных. Инструкция `ROLLBACK` возвращает данные в состояние, предшествующее запуску транзакции.

### ***Механизм отката***

Несмотря на то, что инструкция `ROLLBACK` возвращает данные в прежнее состояние, некоторые компоненты, например, такие, как начальные значения для идентифицирующих столбцов, не восстанавливаются.

Когда транзакция активна, исходя из уровня изоляции транзакции, поддерживаются блокировки ресурсов, к которым получен доступ. Когда транзакция завершается (с помощью `COMMIT` или `ROLLBACK`), все блокировки снимаются. Знание того, как эти блокировки и уровни изоляции функционируют, очень важно для понимания и оптимизации программного кода, включающего транзакции.

## Особые сценарии ***ROLLBACK***

Если транзакции вложены благодаря наличию в сеансе множественных инструкций `BEGIN TRANSACTION`, инструкция `ROLLBACK` выполняет откат для самой внешней

из вложенных транзакций. Это справедливо даже если перед командой ROLLBACK, относящейся к внешней транзакции, встречаются инструкции COMMIT для транзакций на более низких уровнях вложенности. В следующем примере выполняется откат данных вплоть до транзакции, запущенной в строке 1, и вставляемая строка полностью удаляется из таблицы (номера строк приведены только для удобства ссылок на них).

```
1. BEGIN TRANSACTION
2. INSERT INTO TestTable
3. VALUES (1, 'a', 'b');

4. BEGIN TRANSACTION
5. UPDATE TestTable
6. SET Col2 = 'c' WHERE TestID = 1;
7. COMMIT TRANSACTION;

8. ROLLBACK;
```

Если вы хотите выполнить откат только части транзакции, можно задать точки сохранения с помощью инструкции `SAVE TRANSACTION savepoint_name` и затем сослаться на имя точки сохранения в инструкции ROLLBACK. Делая это, вы сообщаете Database Engine об отмене изменений данных только до точки, в которой вы привели инструкцию SAVE TRANSACTION с тем же именем.

Если задается несколько точек сохранения с одним и тем же именем, инструкция ROLLBACK выполняет откат до самой последней точки сохранения с именем, указанным в инструкции ROLLBACK. Если вы хотите отменить транзакцию целиком, введите инструкцию ROLLBACK TRANSACTION с именем транзакции или вообще без имени. Помните о том, что использование инструкции ROLLBACK TRANSACTION без имени приводит к откату и всех вложенных транзакций.

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### **Дополнительный программный код и ограничения**

Вы должны вставлять инструкцию COMMIT для сохраняемой порции транзакции и любого дополнительного программного кода, который может следовать за инструкцией ROLLBACK TRANSACTION *savepoint name*. Кроме того, точки сохранения нельзя применять с распределенными транзакциями.

## **Сбор данных о транзакциях**

Важно следить за активными транзакциями, особенно при работе с вложенными транзакциями и точками сохранения. В хранимых процедурах завершение транзакции (COMMIT или ROLLBACK) до закрытия соединения должно проверяться в процедурах обработки ошибок.

Для просмотра числа активных в текущем сеансе транзакций можно применять глобальную переменную @@trancount.



Для получения более подробной информации можно использовать следующие объекты, предназначенные для динамического управления транзакциями:

- ◆ sys.dm\_tran\_active\_snapshot\_database\_transactions;
- ◆ sys.dm\_tran\_current\_snapshot;
- ◆ sys.dm\_tran\_database\_transactions;
- ◆ sys.dm\_tran\_session\_transactions;
- ◆ sys.dm\_tran\_transactions\_snapshot;
- ◆ sys.dm\_tran\_active\_transactions;
- ◆ sys.dm\_tran\_current\_transaction;
- ◆ sys.dm\_tran\_top\_version\_generators;
- ◆ sys.dm\_tran\_version\_store;
- ◆ sys.dm\_tran\_locks.

Объект sys.dm\_tran\_current\_transaction, как и переменная @@trancount, предоставляет сведения о текущей транзакции в текущем сеансе.

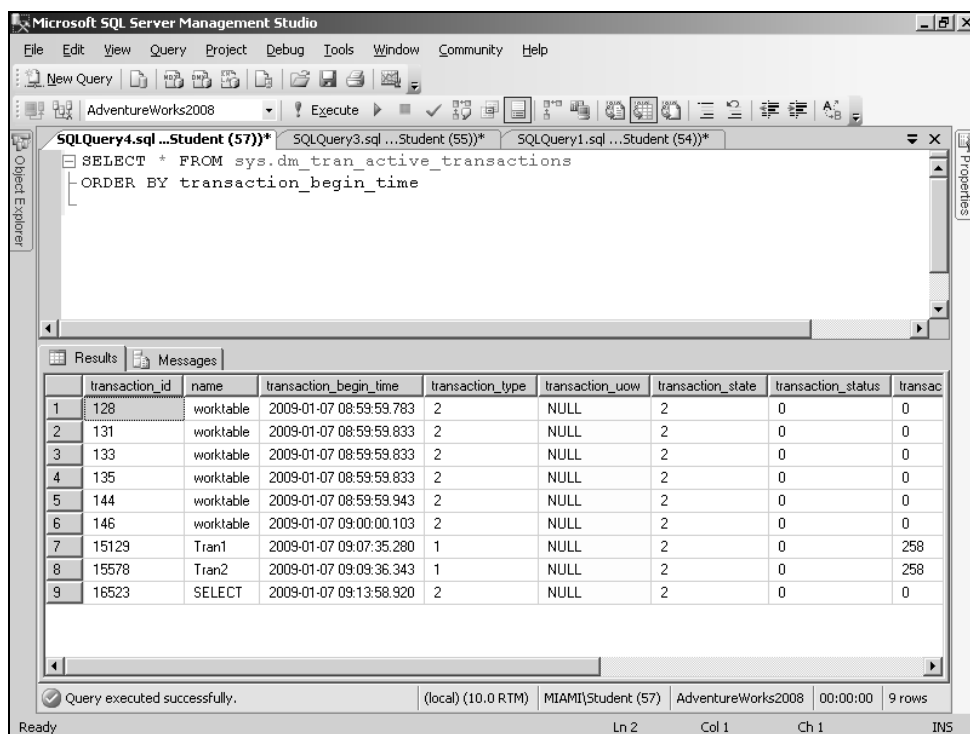


Рис. 2.2. Объект sys.dm\_tran\_active\_transactions

Объект `sys.dm_tran_active_transactions` возвращает информацию обо всех активных транзакциях в экземпляре сервера. Выводимый идентификатор транзакции уникален в пределах всех баз данных, но не в пределах всех экземпляров сервера. На рис. 2.2 показан пример вывода из объекта `sys.dm_tran_active_transactions`, в котором в текущий момент две транзакции с именами `Tran1` и `Tran2` активны в двух сеансах, отличных от текущего сеанса запроса.

## Общие сведения о блокировках

Для того чтобы понять, как транзакции взаимодействуют друг с другом на сервере баз данных, прежде всего, нужно разобраться с уровнями изоляции и механизмом блокировок. В среде баз данных существуют два основных подхода к управлению блокировками. При пессимистическом способе управления предполагается, что пользователи одновременно могут пытаться прочесть или обновить одни и те же данные, и для устранения проблем, вызванных одновременными обращениями многих пользователей к одним и тем же данным, применяются блокировки. При оптимистическом способе управления предполагается, либо что пользователи не будут одновременно обращаться к одним и тем же данным, либо что определенный уровень временной несогласованности приемлем для параллельных считываний во время обновления, и никаких блокировок чтения не устанавливается, что повышает степень параллелизма и производительность сервера.

Для реализации нужного вам способа управления используется сочетание *подсказок блокировок* (lock hint) и *уровней изоляции*. Несмотря на то, что блокировки и уровни изоляции очень тесно связаны, мы сначала обсудим блокировки и способы отображения на сервере информации о блокировках, а затем последует обсуждение уровней изоляции.

Как правило, блокировки обрабатываются динамически не в приложении, а с помощью диспетчера блокировок (lock manager), компонента Database Engine.

SQL Server присваивает блокировкам разные уровни для оптимизации производительности, управления ресурсами и обеспечения параллельности. Блокировки могут назначаться таким ресурсам, как строки, страницы, индексы, таблицы и базы данных. В зависимости от требований блокировкам может быть присвоено несколько уровней, формирующих иерархию связанных блокировок.

В SQL Server 2008 включены следующие режимы блокировок.

- ◆ *Совмещаемая блокировка* (Shared, S) присваивается ресурсам для операций считывания (SELECT). Совмещаемые блокировки совместимы с другими совмещаемыми блокировками. Совмещаемые блокировки не совместимы с монопольными блокировками. Если установлен уровень изоляции `REPEATABLE READ` или более высокий или применяется подсказка блокировки, совмещаемые блокировки удерживаются на все время выполнения транзакции. В про-

тивном случае совмещаемые блокировки освобождаются, как только завершается операция считывания.

- ◆ *Блокировка обновления (Update, U)* устанавливается для ресурсов, нуждающихся в совмещаемой блокировке, но при этом ожидается необходимость повышения режима блокировки до монопольной. В каждый момент времени только одна транзакция может получить для конкретного ресурса блокировку обновления. Когда возникает необходимость модификации ресурса, блокировка обновления изменяется на монопольную блокировку.
- ◆ *Монопольная блокировка (Exclusive, X)* устанавливается для модификации данных ресурса. Монопольная блокировка не совместима с любым другим типом блокировки. Только подсказка `NOLOCK` или уровень изоляции `READ UNCOMMITTED` переопределяют функциональные возможности монопольной блокировки.
- ◆ *Блокировка с намерением (Intent (IS, IX, SIX))* устанавливается для повышения производительности и эффективности блокировок ресурсов за счет помещения блокировок с намерением на объект высокого уровня (например, таблицу) перед установкой совмещаемых или монопольных блокировок на более низких уровнях (например, на странице).
- ◆ *Блокировка схемы (Schema (Sch-M, Sch-S)). Блокировки модификации схемы (Sch-M)* устанавливаются на объекты во время операций изменения схемы, таких как добавление в таблицу нового столбца. *Блокировки стабильности схемы (Sch-S)* устанавливаются на объекты во время компиляции или выполнения запросов. Блокировки Sch-M блокируют все другие операции до тех пор, пока блокировка не будет снята. Блокировки Sch-S не совместимы с блокировками Sch-M.
- ◆ *Блокировка массового обновления (Bulk Update (BU))* устанавливается на таблицы для массовых вставок. Эти блокировки допускают множественные потоки массовых вставок, но запрещают доступ к таблице другим процессам. Такие блокировки устанавливаются либо с помощью подсказки `TABLOCK`, либо с помощью хранимой процедуры `sp_tableoption`, активизирующей параметр таблицы `Table lock on bulk load`.
- ◆ *Блокировка диапазона ключей (Key-range)* устанавливается на диапазон строк для защиты от фантомных вставок и удалений в наборе записей, к которому обращается транзакция. Эти блокировки используются транзакциями с уровнем изоляции `SERIALIZABLE`.

## Взаимоблокировка и сценарии блокирования

Поскольку транзакции с конкретными уровнями изоляции удерживают блокировки до своего завершения, транзакции могут блокировать друг друга, мешая успешному завершению. В SQL Server 2008 по умолчанию транзакции ждут по-

лучения доступа к ресурсу неопределенный промежуток времени, если SQL Server не обнаружит, что возникла взаимоблокировка (deadlock). При взаимоблокировке две транзакции удерживают ресурсы, которые требуются каждой из них перед завершением транзакции. По этой причине ни одна из транзакций не может завершиться успешно. Основываясь на величине накладных расходов SQL Server, необходимых для отката каждой из транзакций, диспетчер блокировок выбирает "жертву" взаимоблокировки и выполняет ее откат, генерируя ошибку 1205. Все ошибки 1205 нужно отслеживать и соответствующим образом обрабатывать, т. к. эта ошибка не пытается перезапустить транзакцию или предоставить пользователям информационное сообщение.

Вы можете применять следующие практические приемы для снижения вероятности возникновения взаимоблокировок и проблем, вызванных блокировками:

- ◆ создавайте короткие транзакции;
- ◆ перед открытием транзакции соберите и проверьте входные данные, полученные от пользователей;
- ◆ сохраняйте в транзакциях, когда это возможно, одинаковый порядок доступа к ресурсам;
- ◆ храните транзакции в едином пакете;
- ◆ если это допустимо, применяйте уровень изоляции, самый низкий из возможных или основанный на контроле версий строк;
- ◆ обращайтесь в транзакции к минимально возможному объему данных.

Для дальнейшего администрирования проблем блокировок администраторы баз данных на основе анализа производительности могут отрегулировать время ожидания запросов. Этого можно добиться с помощью дополнительных свойств сервера. Для определения транзакций, на которые повлияли взаимоблокировки, можно применить приложение SQL Server Profiler и создать на языке Extensible Markup Language (XML) отображение цепочки событий взаимоблокировки, включая системный ID процесса (system process ID, SPID) транзакций, вовлеченных во взаимоблокировку.

## Отчеты о состоянии блокировок

Существует множество возможностей для отображения состояния блокировок на вашем компьютере с работающим SQL Server. Для фиксации блокировки и информации о блокировании можно применить SQL Server Profiler. Для фиксации статистических данных о времени ожидания в блокировке, о количестве блокировок в секунду и т. д. можно использовать системный монитор (System Monitor), а именно консоль производительности (perfmon). Для сбора информации о блокировках, удерживаемых транзакциями, вы можете применить представление динамического администрирования (DMV) `sys.dm_tran_locks`. И, на-

конец, для просмотра информации о блокирующих процессах можно использовать программу Activity Monitor (Монитор активности) в среде SSMS.

Вид Activity Monitor, показывающего сеанс, заблокированный другим сеансом, приведен на рис. 2.3.

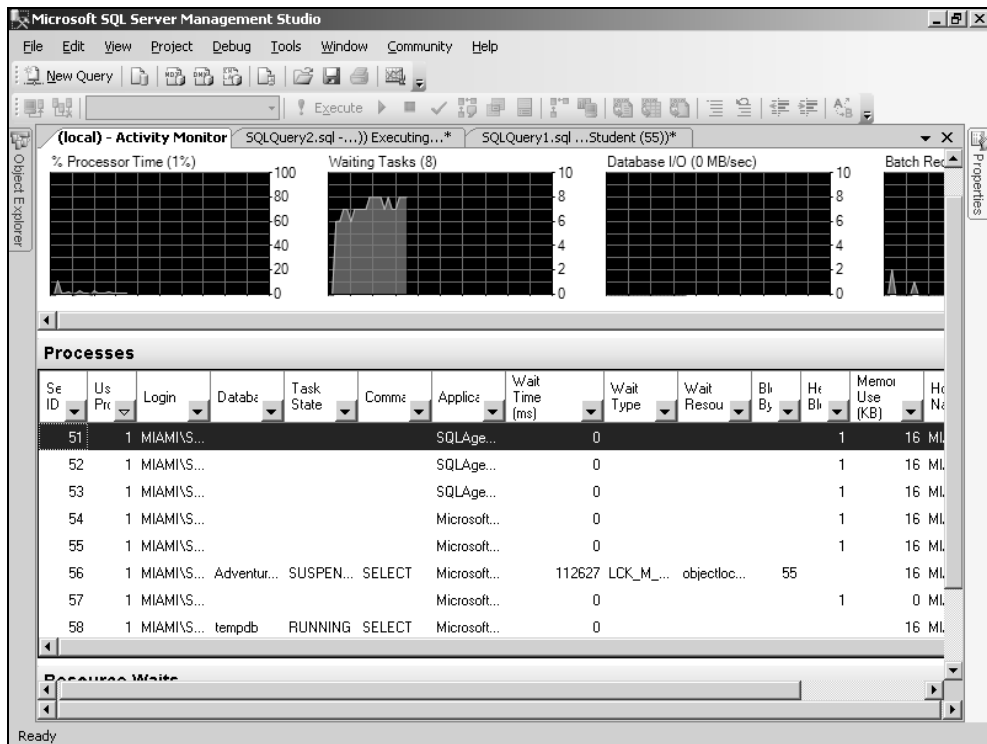


Рис. 2.3. Activity Monitor

DMV-представление `sys.dm_tran_locks` предоставляет подробную информацию обо всех блокировках, которые в данный момент удерживаются в экземпляре сервера. Затем вы можете использовать DMV `sys.dm_tran_active_transactions` для получения дополнительной информации о заблокированной и блокирующей транзакциях. На рис. 2.4 и 2.5 показаны результаты запросов к представлениям `sys.dm_tran_locks` и `sys.dm_tran_active_transactions`.

Во многих из этих отчетов вы видите метод блокировки и заблокированный ресурс. Вы можете увидеть блокировки ресурсов следующих типов:

- ◆ идентификатор строки (Row Identifier, RID) — идентификатор строки применяется для задания блокировки одной строки, размещенной в куче;

The screenshot shows the Microsoft SQL Server Management Studio interface. The query window displays the following SQL query:

```
SELECT resource_type, DB_NAME(resource_database_id)
, resource_description, resource_associated_entity_id
, request_mode AS 'Mode', request_status AS 'Status'
, request_session_id AS SPID
, request_owner_id AS 'Tran ID'
FROM sys.dm_tran_locks
WHERE request_owner_type = 'TRANSACTION'
```

The Results pane shows the following data:

|    | resource_type | (No column name)   | resource_description | resource_associated_entity_id | Mode | Status | SPID | Tran ID |
|----|---------------|--------------------|----------------------|-------------------------------|------|--------|------|---------|
| 1  | RID           | AdventureWorks2008 | 1:23065.5            | 72057594057523200             | X    | GRANT  | 55   | 51251   |
| 2  | OBJECT        | AdventureWorks2008 |                      | 583673127                     | IS   | GRANT  | 54   | 15129   |
| 3  | OBJECT        | AdventureWorks2008 |                      | 583673127                     | IX   | GRANT  | 55   | 51251   |
| 4  | RID           | AdventureWorks2008 | 1:23065.4            | 72057594057523200             | X    | GRANT  | 55   | 51251   |
| 5  | PAGE          | AdventureWorks2008 | 1:742                | 72057594057457664             | IX   | GRANT  | 54   | 15129   |
| 6  | RID           | AdventureWorks2008 | 1:742.0              | 72057594057457664             | X    | GRANT  | 54   | 15129   |
| 7  | OBJECT        | AdventureWorks2008 |                      | 567673070                     | IX   | GRANT  | 54   | 15129   |
| 8  | PAGE          | AdventureWorks2008 | 1:23065              | 72057594057523200             | IS   | GRANT  | 54   | 15129   |
| 9  | PAGE          | AdventureWorks2008 | 1:23065              | 72057594057523200             | IX   | GRANT  | 55   | 51251   |
| 10 | RID           | AdventureWorks2008 | 1:23065.3            | 72057594057523200             | X    | GRANT  | 55   | 51251   |
| 11 | RID           | AdventureWorks2008 | 1:23065.3            | 72057594057523200             | S    | WAIT   | 54   | 15129   |

The status bar indicates: Query executed successfully. (local) (10.0 RTM) MIAMI\Student (59) master 00:00:00 11 rows

Рис. 2.4. Представление динамического администрирования sys.dm\_tran\_locks

The screenshot shows the Microsoft SQL Server Management Studio interface. The query window displays the following SQL query:

```
SELECT * FROM sys.dm_tran_active_transactions
ORDER BY transaction_begin_time
```

The Results pane shows the following data:

|    | transaction_id | name        | transaction_begin_time  | trans... | transa... | tra... | tra... | transaction_status2 | dbc_state | dbc_status | dbc_i |
|----|----------------|-------------|-------------------------|----------|-----------|--------|--------|---------------------|-----------|------------|-------|
| 1  | 128            | worktable   | 2009-01-07 08:59:59.783 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 2  | 131            | worktable   | 2009-01-07 08:59:59.833 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 3  | 133            | worktable   | 2009-01-07 08:59:59.833 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 4  | 135            | worktable   | 2009-01-07 08:59:59.833 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 5  | 144            | worktable   | 2009-01-07 08:59:59.943 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 6  | 146            | worktable   | 2009-01-07 09:00:00.103 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 7  | 15129          | Tran1       | 2009-01-07 09:07:35.280 | 1        | NULL      | 2      | 0      | 258                 | 0         | 0          | -1    |
| 8  | 51251          | Tran2       | 2009-01-07 12:14:02.093 | 1        | NULL      | 2      | 0      | 258                 | 0         | 0          | -1    |
| 9  | 81817          | SELECT      | 2009-01-07 12:47:22.070 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 10 | 81818          | sort_fak... | 2009-01-07 12:47:22.070 | 2        | NULL      | 2      | 0      | 0                   | 0         | 0          | 0     |
| 11 | 81819          | sort_init   | 2009-01-07 12:47:22.070 | 1        | NULL      | 2      | 0      | 514                 | 0         | 0          | -1    |

The status bar indicates: Query executed successfully. (local) (10.0 RTM) MIAMI\Student (61) AdventureWorks2008 00:00:00 11 rows

Рис. 2.5. Представление динамического администрирования sys.dm\_tran\_active\_transactions

### ***Кучи***

Куча (heap) — это метод хранения таблицы без кластеризованного индекса (clustered index). Дополнительную информацию об индексировании см. в главе 6.

- ◆ ключ (KEY) — диапазон ключей в индексе, применяемый для определения блокировки диапазона ключей;
- ◆ страница (PAGE) — восьмиклобайтовая страница из таблиц или индексов;
- ◆ экстенд (EXTENT) — группа из восьми последовательных страниц в таблице или индексе;
- ◆ НоВТ — куча (heap) или индекс сбалансированного дерева (B-tree);
- ◆ таблица (TABLE) — вся таблица, состоящая из данных и страниц индекса;
- ◆ файл (FILE) — файл базы данных целиком;
- ◆ приложение (APPLICATION) — специализированный прикладной ресурс;
- ◆ метаданные (METADATA) — используются для блокировок метаданных;
- ◆ единица распределения (ALLOCATION\_UNIT) — одна единица распределения;
- ◆ база данных (DATABASE) — файл базы данных целиком.

## **Применение SQL Server Extended Events**

В версию SQL Server 2008 включена система обработки событий для серверных систем SQL Server Extended Events, таких как FindBlocker и lock\_count, которые могут использоваться в сочетании с журналами Windows Event, приложениями SQL Profiler или System Monitor.

### ***Дополнительная информация о расширенных событиях***

Работа с SQL Server Extended Events выходит за рамки тем, обсуждаемых в этой книге. Дополнительную информацию см. в статье "Advanced Troubleshooting with Extended Events" (Усовершенствованное выявление неисправностей с помощью расширенных событий) на странице <http://technet.microsoft.com/en-us/magazine/dd314391.aspx>. Для того чтобы узнать больше о загрузке копии Extended Events Manager приложения C# Microsoft.NET WinForms, которое помогает создавать сеансы Extended Events и работать с ними, см. "SQL Server 2008 Extended Events Manager" (Диспетчер расширенных событий SQL Server 2008) на странице <http://www.codeplex.com/ExtendedEventManager>.

## **Применение DBCC LOG**

В SQL Server включена инструкция DBCC LOG, которая представляет собой недокументированное средство, возвращающее информацию о сведениях, содержащихся в текущем журнале транзакций. У этой инструкции следующая синтаксическая запись:

```
DBCC LOG (<databasename>, <output identifier>)
```

Параметру *output identifier* можно задать любое из следующих значений:

- ◆ 0 — возвращается минимальная информация, включающая текущий регистрационный номер (Log Sequence Number, LSN), операцию, контекст, ID транзакции и версию регистрационного блока (log block generation);
- ◆ 1 — возвращается вся информация предыдущего уровня, а также флаги и данные о длине записи;
- ◆ 2 — возвращается вся информация предыдущего уровня, а также имя объекта, имя индекса, ID страницы и ID области памяти (slot ID);
- ◆ 3 — возвращается полный набор сведений об операции;
- ◆ 4 — возвращается полный набор сведений об операции и шестнадцатеричный дамп строки текущей транзакции в журнале.

## Установка уровней изоляции транзакции

С помощью инструкции SET TRANSACTION ISOLATION LEVEL можно установить следующие уровни изоляции транзакции.

- ◆ READ UNCOMMITTED — разрешает инструкциям читать строки, обновленные любой транзакцией и еще не зафиксированные в базе данных. Этот уровень изоляции минимизирует вероятность конфликтов, но допускает "грязные" считывания (dirty reads) и неповторимые (фантомные) считывания (nonrepeatable (phantom) reads).
- ◆ READ COMMITTED — разрешает инструкциям в текущих соединении и транзакции выполнять неповторимые (фантомные) считывания, но запрещает "грязные" считывания (данные, обновленные транзакцией, открытой в другом соединении). Это уровень изоляции, принятый по умолчанию в SQL Server 2008.
- ◆ REPEATABLE READ — не разрешает транзакциям считывать незафиксированные измененные данные ("грязные" считывания) и гарантирует сохранение совмещаемых блокировок до завершения текущей транзакции.
- ◆ SNAPSHOT — требует, чтобы параметр базы данных ALLOW\_SNAPSHOT\_ISOLATION был равен значению ON. Этот уровень изоляции формирует моментальный снимок данных в момент их считывания в транзакцию, но не поддерживает блокировку данных. Могут выполняться обновления данных из другой транзакции, но текущая транзакция не видит этих изменений, отраженных в последующих считываниях исходных данных. Если текущая транзакция модифицирует данные, эти модификации видны только в текущей транзакции.
- ◆ SERIALIZABLE — не разрешает считывать данные, которые были обновлены, но не зафиксированы другими транзакциями. Кроме того, никакая другая транзакция не может до завершения текущей транзакции обновлять данные, кото-



рые были считаны текущей транзакцией. Уровень изоляции `SERIALIZABLE` защищает от фантомных считываний, но создает наивысший уровень конкуренции и блокирования.

### ***Дополнительная информация об уровнях изоляции транзакции и блокировании***

Всестороннее изучение уровней изоляции и их влияния на блокировки и блокирование выходит за рамки книги. Подробное описание уровней изоляции приведено в книге: Kalen Delaney et al. *Microsoft SQL Server 2008 Internals*". — Microsoft Press, 2009 (Кэлен Делани и др. *Microsoft SQL Server 2008 изнутри*).

## **Синтаксическая запись команды**

После того как вы определили подходящий уровень изоляции транзакции, синтаксическая запись команды очевидна, как показано далее.

```
SET TRANSACTION ISOLATION LEVEL
{ READ UNCOMMITTED
| READ COMMITTED
| REPEATABLE READ
| SNAPSHOT
| SERIALIZABLE
}
[;]
```

После того как в сеансе выполнена инструкция `SET TRANSACTION ISOLATION LEVEL`, все транзакции в пределах этого соединения используют заданный уровень изоляции.

## **ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Задание явных транзакций**

На этом практическом занятии вы проконтролируете выполнение отката вложенных транзакций.

### **Упражнение. Определение явных транзакций**

В этом упражнении вы создадите очень простую таблицу и проверите, как откаты влияют на зафиксированные вложенные транзакции.

1. Если нужно, запустите среду SSMS, установите соединение с вашим экземпляром SQL Server и откройте окно создания запроса.
2. В окне запроса для создания простой таблицы введите и выполните следующий программный код.

```
USE AdventureWorks2008;
CREATE TABLE testtran (col1 int, col2 int);
```

3. Откройте окно создания запроса, введите, выделите и выполните следующий программный код для запуска транзакции, проверьте данные, находящиеся в данный момент в таблице `testtran`, добавьте строку в эту таблицу и затем проверьте данные, включенные в таблицу `testtran`, и количество открытых транзакций.

```
BEGIN TRAN
 SELECT * FROM testtran;
 INSERT INTO testtran VALUES (1,1);
 SELECT * FROM testtran;
 SELECT @@TRANCOUNT;
```

4. В текущем окне запроса под имеющимся программным кодом для запуска вложенной транзакции, вставки второй строки и проверки строк в таблице и количества открытых транзакций введите, выделите и выполните следующий код.

```
BEGIN TRAN
 INSERT INTO testtran VALUES (2,2);
 SELECT * FROM testtran;
 SELECT @@TRANCOUNT;
```

5. В текущем окне запроса для фиксации внутренней транзакции и проверки данных в таблице и уменьшения числа транзакций на 1 под имеющимся программным кодом введите, выделите и выполните следующий код.

```
COMMIT TRAN
 SELECT * FROM testtran;
 SELECT @@TRANCOUNT;
```

6. В текущем окне запроса под имеющимся программным кодом введите, выделите и выполните следующий код, чтобы выполнить откат транзакций и убедиться, что обе строки удалены из таблицы и количество транзакций снизилось до 0.

```
ROLLBACK TRAN

 SELECT * FROM testtran;
 SELECT @@TRANCOUNT;
```

7. Сохраните сценарий и закройте окно запроса.
8. Убедитесь в том, что вы закрыли все окна запросов, открытые на этом практическом занятии, и затем откройте, просмотрите и выполните файл `Lesson03 PracticeCleanup.sql`, который можно найти среди файлов примеров в папке `Chapter 2\Lesson 3`.
9. Закройте среду SSMS.

## Резюме урока

- ◆ Транзакция — это набор операций, формирующих элементарный блок работы, который должен выполняться успешно или завершаться аварийно как единое целое.
- ◆ По умолчанию неявные транзакции не активизированы. Если неявные транзакции включены, ряд инструкций открывает транзакцию автоматически. Для завершения транзакции разработчик должен выполнить инструкцию `COMMIT` или `ROLLBACK`.
- ◆ Явные транзакции открываются инструкцией `BEGIN TRANSACTION` и завершаются инструкцией `ROLLBACK TRANSACTION` или `COMMIT TRANSACTION`.
- ◆ Выполнение команды `ROLLBACK` при наличии вложенных транзакций приводит к откату всех транзакций вплоть до самой внешней инструкции `BEGIN TRANSACTION` независимо от ранее выполненных для вложенных транзакций инструкций `COMMIT`.
- ◆ В SQL Server для обеспечения согласованности данных в случае одновременной обработки нескольких транзакций применяются разные режимы блокировок, включая совмещаемую (S), монопольную (X) блокировки и блокировку с намерением (IS, IX, SIX).
- ◆ В версии SQL Server 2008 поддерживаются следующие уровни изоляции: `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SNAPSHOT` и `SERIALIZABLE`.

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить знания и навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Просмотреть резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ Инструкции языка DML, такие как `INSERT`, `UPDATE` и `DELETE`, позволяют управлять условиями хранения и поиска данных вашей организации.

- ◆ Инструкция MERGE и предложение OUTPUT позволяют расширить функциональные возможности управления вашими базами данных оперативной обработки транзакций (OLTP), хранилищами данных и средой формирования отчетов. Эти компоненты наряду с системой отслеживания измененных данных (CDC) предоставляют средства для сравнения строк и формирования логики применения операций UPDATE, INSERT или DELETE на основе этих сравнений.
- ◆ Транзакции и блокировки обеспечивают средства, благодаря которым несколько пользователей одновременно могут обращаться к данным и обновлять их на сервере с запущенной программой SQL Server, получая при этом согласованное отображение данных.

## Ключевые термины

Знаете ли вы, что означают следующие ключевые термины? Вы можете проверить свои ответы, заглянув в словарь терминов, приведенный в конце книги.

- ◆ Системный идентификатор процесса (System process ID, SPID).
- ◆ Идентификатор строки (Row identifier, RID).
- ◆ Атомарность.
- ◆ Согласованность.
- ◆ Изоляция.
- ◆ Длительность или долговечность.

## Сценарии

В следующих конкретных сценариях вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в разд. "Ответы к сценариям" в конце книги.

### Сценарий 1. Изменение данных

Вы — разработчик базы данных для компании Wide World Importers. Пять компаний, из которых ваша компания импортирует товары, решили отказаться от факсов и получать информацию только по Интернету или электронной почте. Корпоративные стандарты данных требуют, чтобы столбец с номером факса содержал либо числовое значение, либо значение NULL. Кроме того, несмотря на то, что столбец для адресов электронной почты существует, раньше у этих компаний в их записях не было адресов электронной почты. Вы должны добавить эту информацию. И наконец, у всех обновлений факсов, адресов и адресов электронной почты должны храниться значения до и после изменения наряду с именем лица, внесшего изменение, и датой модификации, записанные в таблицу `ImporterPropertiesAudit` (Контроль свойств импортеров).

Ответьте на следующий вопрос вашего менеджера: "Какую инструкцию (инструкций) и/или предложения вам следует применить для решения этой задачи?"

## Сценарий 2. Применение транзакций

Вы — разработчик базы данных для компании Litware, Inc. У Litware было много проблем, связанных с долгим временем ожидания и многочисленными взаимоблокировками. Администратор базы данных собрал сведения о производительности с помощью консоли `perfmon` и приложения `SQL Server Profiler` и установил, что основная часть проблем, как кажется, вызвана необходимостью выполнения пяти длительных хранимых процедур, содержащих транзакции. Вас попросили посмотреть эти хранимые процедуры и дать рекомендации по устранению блокировок и заторов.

Ответьте на следующий вопрос вашего менеджера: "Какие данные вам следует принять во внимание при выработке рекомендаций?"

## КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы полностью освоить экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

### Изменение данных

#### с помощью инструкций *INSERT*, *UPDATE* и *DELETE*

**Упражнение 1.** Напишите несколько инструкций `INSERT` с различными вариантами.

**Упражнение 2.** Напишите инструкции `UPDATE` и `DELETE`, включая основанные на результатах из таблицы, полученной с помощью оператора `JOIN`.

### Получение данных с помощью предложения *OUTPUT*

**Упражнение 3.** Напишите различные инструкции, использующие с помощью предложения `OUTPUT`, как таблицы `INSERTED`, так и таблицы `DELETED`.

### Модификация данных с помощью инструкций *MERGE*

**Упражнение 4.** Напишите различные инструкции `MERGE`, включая использующие по два определения предложений `WHEN NOT MATCHED BY SOURCE THEN <merge_matched>` и `WHEN MATCHED THEN <merge_matched>`.

### Управление транзакциями

**Упражнение 5.** Попрактикуйтесь в работе с транзакциями, включая вложенные транзакции, точки сохранения и откаты. Проверяйте пошагово переменную @@trancount и полученные результаты.

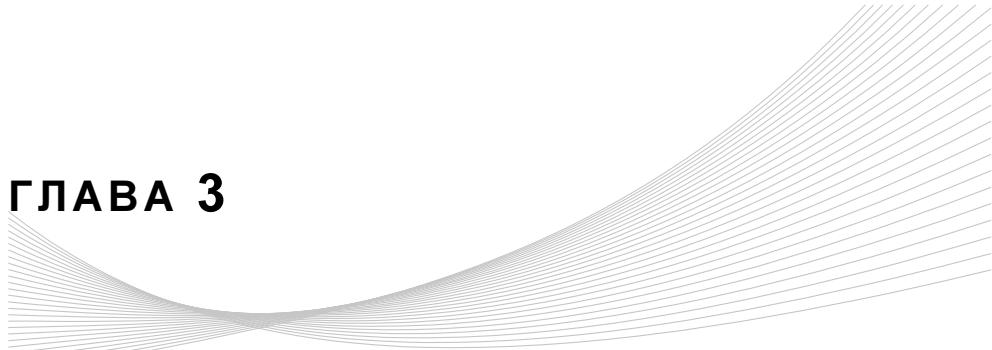
## Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобия будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### ***Дополнительная информация о тренировочных тестах***

---

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.



## ГЛАВА 3

# Таблицы, типы данных и декларативная целостность данных

Главное звено любой системы управления реляционной базой данных (СУРБД) — таблица. Однако со времени появления первых версий СУРБД таблицы сильно эволюционировали. В этой главе вы познакомитесь с возможностями и ограничениями, с которыми сталкиваются при разработке таблиц. В этой главе также обсуждаются способы оптимизации структур ваших таблиц и целостность данных.

### Экзаменационные темы в этой главе

- ◆ Создание и модификация таблиц.
- ◆ Реализация типов данных.
- ◆ Управление международными настройками.
- ◆ Создание и модификация ограничений.

### Уроки в этой главе

- ◆ Урок 1. Работа с таблицами и типами данных.
- ◆ Урок 2. Декларативная целостность данных.

## Прежде чем начать

Для усвоения уроков этой главы необходимо:

- ◆ знать базовые элементы языка Transact-SQL (T-SQL);
- ◆ иметь четкое представление о типах данных в любом языке программирования;

- ◆ иметь установленные на компьютере: версию Microsoft SQL Server 2008 Developer Edition, Enterprise Edition или Enterprise Evaluation Edition и учебную базу данных AdventureWorks.

### **РЕАЛЬНАЯ ИСТОРИЯ.**

#### ***Тобиас Тернстрем (Tobias Thernström)***

Без декларативной целостности данных ваша база данных может прийти в ужасное состояние. Я сталкивался с подобным много раз, когда работал с клиентами как консультант. Как правило, все начинается с запросов, которые ведут себя странно. Например, однажды я посетил клиента, жаловавшегося на заказ, который точно должен был быть в базе данных, но по утверждению клиента потерялся. Я инстинктивно сразу предположил, что заказ потерялся из-за того, что клиент его случайно удалил, но запросив базу данных, я убедился, что заказ существует. Тогда почему же он не отображался в приложении? Просто запрос, который извлекал заказ, соединялся (с помощью оператора `INNER JOIN`) с клиентом из таблицы, содержащей клиентов, а клиент в этой таблице отсутствовал! Все дело в том, что в базе данных не было объявлено никакого ограничения внешнего ключа между таблицами клиентов и заказов, клиент был случайно удален, несмотря на то, что имел реальные (и не доставленные) заказы.

## **УРОК 1. Работа с таблицами и типами данных**

Разработка, создание и сопровождение таблиц — одна из самых важных задач разработчика. В данном уроке мы обсудим эту и другие задачи, связанные с таблицами. Вы также познакомитесь с типами данных, имеющимися в SQL Server 2008.

### ***После усвоения этого урока вы сможете***

- Составить полное представление о базовых типах данных, входящих в SQL Server 2008.
- Корректно применять типы данных.
- Реализовывать декларативную целостность данных в ваших таблицах.

Расчетное время урока — 60 мин.

## **Типы данных**

Прежде чем вы сможете начать создавать таблицы, следует разобраться с разными типами данных, которые можно применять для определения доменов столбцов в таблицах, т. е. какие данные можно вводить в столбец. В SQL Server есть два разных вида типов данных:

- ◆ системные типы данных SQL Server;
- ◆ определенные пользователем типы (User-defined types, UDT) или типы общей языковой среды исполнения (SQL Common Language Runtime, SQLCLR).



Вы начнете с обзора доступных системных типов данных и затем рассмотрите возможность упрощения использования типов данных с помощью типов, определенных пользователем. Последние реализованы средствами среды SQLCLR, которая рассматривается в *главе 7*.

Имеющиеся системные типы данных обычно делят на несколько групп, включая точные числовые, приближенные числовые, символьные, даты и время суток и двоичные.

## Символьные типы

В SQL Server включено довольно много строковых типов данных: `char`, `varchar`, `nchar`, `nvarchar`, `text` и `ntext`. Типы `text` и `ntext` не одобряются, поэтому старайтесь их не применять. Они были заменены типами `varchar(max)` и `nvarchar(max)`. Все `char`-типы принимают один параметр — количество символов для хранения данных. Разница между `char` и `varchar` (как и между `nchar` и `nvarchar`) состоит в том, что `char` — тип данных фиксированной длины, а `varchar` — переменной длины. Это означает, что для `char` всегда выделяется столько памяти, сколько необходимо для хранения данных объявленной длины, а `varchar` хранит только реальные введенные данные. Преимущество применения `char` по сравнению с `varchar` заключается в том, что для обновлений столбца типа `char` никогда не требуется перемещения строки, т. к. вводимые данные всегда соответствуют отведенному объему памяти. Учтите, что это преимущество почти всегда перевешивает, несмотря на то, что тип `varchar` использует гораздо меньше памяти, чем `char` (например, `varchar(100)` по сравнению с `char(100)`).

С типами `char` и `varchar` для задания кодовой страницы (набора символов), используемой при хранении и интерпретации содержимого столбца, применяется набор параметров символьной обработки (*collation*). Набор параметров символьной обработки также используется при выборе сортировки и сравнения данных, хранящихся в этих столбцах. В SQL Server 2008 есть 2397 вариантов наборов параметров символьной обработки, к ним относятся `Japanese_CI_AI`, `Finnish_Swedish_CI_AI` и `Latin1_General_CI_AI`. Остальные варианты можно найти, запросив табличную функцию `fn_helpcollations`. Набор параметров символьной обработки `Japanese_CI_AI` для хранения символов японского языка использует кодовую страницу 932. Наборы `Finnish_Swedish_CI_AI` и `Latin1_General_CI_AI` используют кодовую страницу 1252. Часть `_CI_AI` в имени набора параметров задает символьную обработку без учета регистра (CI) или с учетом (CS), а также без учета диакритических знаков (AI) или с их учетом (AS). Важно знать, что диакритические знаки различны в разных языках, а значит, и в наборах параметров символьной обработки. Например, символ "ö" в наборе `Latin1_General_CI_AI` считается символом "o" с диакритическим знаком, следовательно, выражение "o" = "ö" вернет `True`; с другой стороны, в наборе параметров `Finnish_Swedish_CI_AI` "ö" считается отдельным символом и "o" = "ö" вернет `False`.

И в заключение о типах `nchar` и `nvarchar`. Оба эти типа данных хранят символы, используя страницу универсального кода Unicode (UCS-2). Это значит, что если применять тип `nchar` или `nvarchar`, можно хранить символы любого типа, независимо от выбранного вами набора параметров символьной обработки, потому что для хранения каждого символа всегда используются два байта. Сравните это с типами `varchar` и `char`, которые хранят символы в одном или двух байтах в зависимости от набора параметров символьной обработки. Помните о том, что все равно следует задавать набор параметров символьной обработки, т. к. именно он определяет способ сортировки и сравнения данных, содержащихся в вашем столбце.

Если при сравнении вы хотите применить набор параметров символьной обработки, отличающийся от заданного для столбца, можно задать его в выражении. Далее приведен пример с предложением `WHERE`, в котором задается набор параметров символьной обработки.

```
... WHERE Name = 'Oqvist' COLLATE Finnish_Swedish_CI_AS
```

Имейте в виду, что, изменив в выражении набор параметров символьной обработки, для выполнения операции поиска, SQL Server не сможет использовать определенный для столбца индекс, т. к. он отсортирован в соответствии с другим набором параметров символьной обработки.

## Точные числовые типы

Точные числовые типы состоят из типов `integer` (или целое число) и типов с фиксированной десятичной точкой. Все точные числовые типы всегда формируют один и тот же результат независимо от применяемой архитектуры процессора или величины чисел (т. е. от того, насколько велики числа). В табл. 3.1 перечислены имеющиеся точные числовые типы данных.

**Таблица 3.1.** Точные числовые типы данных

| Тип данных            | Объем памяти | Допустимые значения                          | Примечания                                                                                                                |
|-----------------------|--------------|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <code>tinyint</code>  | 1 байт       | 0—255                                        | Эквивалентен типу данных <code>byte</code> в большинстве языков программирования, не может хранить отрицательные значения |
| <code>smallint</code> | 2 байта      | –32 768—32 767                               | Знаковое 16-разрядное целое                                                                                               |
| <code>int</code>      | 4 байта      | –2 147 483 648—<br>2 147 483 647             | Знаковое 32-разрядное целое                                                                                               |
| <code>bigint</code>   | 8 байтов     | $-2 \times 10^{63}$ — $2 \times 10^{63} - 1$ | Знаковое 64-разрядное целое                                                                                               |

Таблица 3.1 (окончание)

| Тип данных                                               | Объем памяти                                   | Допустимые значения                                | Примечания                              |
|----------------------------------------------------------|------------------------------------------------|----------------------------------------------------|-----------------------------------------|
| decimal (разрядность, количество знаков в дробной части) | От 5 до 17 байтов в зависимости от точности    | $-10 \times 10^{38} + 1$ — $10 \times 10^{38} - 1$ | Десятичное число, содержащее до 38 цифр |
| numeric (разрядность, количество знаков в дробной части) | Функционально эквивалентен типу данных decimal |                                                    |                                         |

У целочисленных типов данных (tinyint, smallint, int и bigint) нет параметров, а у decimal (и numeric) есть. Применяя тип данных decimal, можно задать разрядность (precision) и количество десятичных знаков в дробной части (scale) для хранимого значения данного типа. Разрядность определяет общее количество цифр, поддерживаемое типом данных, максимальная разрядность равна 38, а количество знаков в дробной части определяет, сколько цифр, заданных в разрядности, отводится под дробную часть. Тип, определенный как decimal(38,0), позволяет хранить только целые числа, а тип decimal(38, 38) — только десятичные дроби. Если вы определите столбец просто как decimal, без указания разрядности и количества десятичных знаков после запятой, он получит по умолчанию разрядность, равную 18, и 0 для знаков в дробной части. В зависимости от заданной вами разрядности типу данных decimal требуется от 5 до 17 байтов памяти. Для экономии области хранения и ресурсов памяти важно выбирать минимально допустимую разрядность. В табл. 3.2 перечислены необходимые объемы памяти для разных разрядностей. Имейте в виду, что количество знаков в дробной части не влияет на требуемый объем памяти.

Таблица 3.2. Требуемый объем памяти для типа данных decimal

| Разрядность | Память, байт |
|-------------|--------------|
| От 1 до 9   | 5            |
| От 10 до 19 | 9            |
| От 20 до 28 | 13           |
| От 29 до 38 | 17           |

Приближенные числовые типы

SQL Server поддерживает два типа данных с плавающей точкой или приближенными числовыми значениями: float и real. Тип данных float, как и тип decimal, описанный ранее, принимает параметр. Параметр, предоставляемый типу данных float, задает количество битов, используемых для хранения мантиссы числа с плавающей точкой (табл. 3.3). Любое значение, меньшее или равное 24, интер-

претируется как 24, все значения бóльшие 24 — как 53. Это означает, что мантисса занимает 24 или 53 бита в зависимости от величины параметра, заданного для типа данных `float`.

**Таблица 3.3.** Приближенные числовые типы данных

| Тип данных                                 | Объем памяти                                      | Допустимые значения                                                                                           |
|--------------------------------------------|---------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>float</code><br>( $n \leq 24$ )      | 4 байта                                           | От $-3.40 \times 10^{38}$ до $-1.18 \times 10^{-38}$ и от $1.18 \times 10^{-38}$ до $3.40 \times 10^{38}$     |
| <code>float</code><br>( $24 < n \leq 53$ ) | 8 байтов                                          | От $-1.79 \times 10^{308}$ до $-2.23 \times 10^{-308}$ и от $2.23 \times 10^{-308}$ до $1.79 \times 10^{308}$ |
| <code>real</code>                          | Функционально эквивалентен <code>float(24)</code> |                                                                                                               |

## Обработка дат и времени суток

В табл. 3.4 перечислены типы данных в SQL Server 2008, которые могут хранить значения дат и времени суток.

**Таблица 3.4.** Типы данных для хранения дат и времени суток

| Тип данных                                                | Объем памяти      | Допустимые значения                                                                                                            | Примечания                                                                                                                                                                                    |
|-----------------------------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>datetime</code>                                     | 8 байтов          | С 1 января 1753 г. по 31 декабря 9999 г. с точностью до каждой третьей миллисекунды                                            | Поддерживается в основном для обратной совместимости. Везде, где это возможно, применяйте типы <code>datetime2</code> , <code>date</code> , <code>time</code> или <code>datetimeoffset</code> |
| <code>smalldatetime</code>                                | 4 байта           | С 1 января 1900 г. по 5 июня 2079 г. с точностью до минуты                                                                     | Поддерживается в основном для обратной совместимости. Везде, где это возможно, применяйте типы <code>datetime2</code> , <code>date</code> , <code>time</code> или <code>datetimeoffset</code> |
| <code>datetime2</code><br>(точность до долей секунд)      | От 6 до 8 байтов  | С 1 января 0001 г. по 31 декабря 9999 г. с точностью до заданной доли секунды                                                  | Применяйте, если нужны дата и время суток, но без учета сдвига часового пояса                                                                                                                 |
| <code>datetimeoffset</code><br>(точность до долей секунд) | От 8 до 10 байтов | С 1 января 0001 г. по 31 декабря 9999 г. с точностью до заданной доли секунды и сдвигом часового пояса от $-14:00$ до $+14:00$ | Применяйте, если нужны дата, время суток и сдвиг или смещение часового пояса                                                                                                                  |
| <code>date</code>                                         | 3 байта           | С 1 января 0001 по 31 декабря 9999                                                                                             | Применяется, если нужна только дата                                                                                                                                                           |
| <code>time</code><br>(доли секунд)                        | От 3 до 5 байтов  | С 00:00:00 до 23:59:59 с точностью до заданной доли секунды                                                                    | Используется, если необходимо только время суток                                                                                                                                              |

Одним из самых долгожданных было появление в SQL Server 2008 новых типов данных `date` и `time`. До версии SQL Server 2008 для обработки дат и времени суток применялись только два типа данных: `datetime` и `smalldatetime`. Поскольку и сегодня оба эти типа все еще используются в SQL Server 2008, и потребуется много времени, прежде чем все базы данных, которые обновляются в соответствии с SQL Server 2008, будут преобразованы с применением новых типов данных `date` и `time`, очень важно понять, как использовать типы данных `smalldatetime` и `datetime`. С типом данных `datetime` связаны некоторые проблемы. Одна из них состоит в том, что дата и время суток хранятся вместе, что не всегда желательно. Рассмотрим следующий запрос, в котором вы хотите извлечь все заказы, сделанные August 18, 2008 (18 августа 2008 г.).

```
SELECT SalesOrderID
 , CustomerId
 , OrderDate
FROM Sales.SalesOrderHeader
WHERE OrderDate = '20080818';
```

Этот запрос вернет только заказы, сделанные 18 августа 2008 г. точно в 00:00:00. Для решения этой проблемы вы должны корректно обработать в запросе часть типа данных `datetime`, содержащую время суток. Выполнение этой задачи приведет нас ко второй проблеме, связанной с типом `datetime` и касающейся точности представления времени в типе данных. Минимальная поддерживаемая единица времени — каждая третья миллисекунда (или каждая минута для типа данных `smalldatetime`). Это означает, что последняя цифра в значении типа `datetime` (т. е. `yyyy-MM-dd hh:mm:ss.xxx`) может быть только 0, 4 или 7. Это в свою очередь значит, что последнее поддерживаемое время суток в типе `datetime` — 23:59:59.997. Время суток 23:59:59.998 округляется в меньшую сторону до 23:59:59.997, а время суток 23:59:59.999 округляется в большую сторону до времени следующего дня 00:00:00.000. Об этом крайне важно помнить при работе с типом данных `datetime`. Продолжив работу с примером запроса к таблице `Sales.SalesOrderHeader` (Общие сведения о заказе на продажу), чтобы получить желаемый результат, вы должны применить один из двух приведенных далее запросов для извлечения всех заказов на August 18, 2008 (18 августа 2008 г.).

```
-- Запрос #1
SELECT SalesOrderID
 , CustomerId
 , OrderDate
FROM Sales.SalesOrderHeader
WHERE OrderDate
 BETWEEN '2008-08-18T00:00:00' AND '2008-08-18T23:59:59.997';

-- Запрос #2
SELECT SalesOrderID
 , CustomerId
 , OrderDate
```

```
FROM Sales.SalesOrderHeader
WHERE OrderDate >= '20080818' AND OrderDate < '20080819';
```

Несмотря на то, что оба запроса возвращают одинаковый результат, лучше применять второй запрос, т. к. он также корректно обрабатывает новый тип данных `datetime2`. Происходит это потому, что тип данных `datetime2` может сохранять доли секунды вплоть до 100 наносекунд (или 0.0000001 с), т. е. 0.997 перестает быть последней миллисекундой суток. Если вы запрашиваете данные о времени суток, следует попытаться применить предикат "less than" (меньше чем), не полагаясь на конкретную величину доли секунды. В заключение при попытке преобразования строковой константы в тип данных `datetime` или `smalldatetime` следует всегда применять формат `YYYYMMDD` для дат без времени суток и формат `YYYY-MM-DDTHH:MI:SS.XXX` для дат со временем суток. Оба эти формата действуют независимо от языка и форматов дат, установленных в вашем соединении.

### **Дополнительная информация о форматах дат**

Подробную информацию о том, как повлиять на форматы дат, принимаемые SQL Server см. в разд. "SET DATEFORMAT" ("SET DATEFORMAT (Transact-SQL)") и "SET LANGUAGE" ("SET LANGUAGE (Transact-SQL)") электронной документации по SQL Server.

Новые долгожданные типы данных для дат и времени суток в SQL Server 2008 — `datetime2`, `date`, `time` и `datetimeoffset`. Начнем с простейшего их них — `date`.

Тип данных `date` порой очень прост и весьма полезен. Это тип данных, способный хранить только даты. По этой причине он самый подходящий в тех случаях, когда время суток не требуется. Данный тип упрощает запросы, поскольку нет проблем, связанных со временем суток, и кроме того, этот тип данных экономит память, выделяя только 3 байта. При преобразовании строковой константы в тип данных `date` всегда следует применять формат `YYYY-MM-DD` (обязательно включая столетие).

Тип данных `time` подходит для хранения времени суток без даты. Для этого типа данных можно предоставлять необязательный параметр, задающий точность как количество десятичных знаков в дробной части секунды, которое вы хотите обеспечивать при представлении конкретного времени суток. Значения, которые вы можете задать, лежат между 0 и 7, т. е. между секундой как наименьшей величиной (0) и 100 наносекундами в той же роли (7). Если этот параметр не задан при объявлении типа данных, по умолчанию используется значение 7. При преобразовании строковой константы в тип данных `time` всегда следует применять формат `HH:MI:SS.NNNNNNN`. Объем памяти, занимаемый данными типа `time`, зависит от заданной точности. В табл. 3.5 приведены объемы памяти в зависимости от задаваемой точности.

**Таблица 3.5.** Требуемый объем памяти для типа данных *time*

| Точность | Объем памяти, байт |
|----------|--------------------|
| 0—2      | 3                  |
| 3—4      | 4                  |
| 5—7      | 5                  |

Тип данных `datetime2` — это комбинация типов данных `date` и `time` с тем же самым параметром точности для долей секунды, что и у типа `time`. При преобразовании строковой константы в тип данных `datetime2` всегда следует использовать формат `YYYY-MM-DD HH:MI:SS.NNNNNNN`. В табл. 3.6 перечислены объемы памяти, используемые типом данных `datetime2` в зависимости от заданной точности.

**Таблица 3.6.** Требуемый объем памяти для типа данных *datetime2*

| Точность | Объем памяти, байт |
|----------|--------------------|
| 0—2      | 6                  |
| 4—5      | 7                  |
| 6—7      | 8                  |

Последний доступный в SQL Server 2008 тип данных — `datetimeoffset`. Помимо данных, сохраняемых типом данных `datetime`, этот тип содержит сдвиг или смещение часового пояса. Эта информация может быть очень полезна, если важно знать не только время произошедшего, но и в каком часовом поясе нечто произошло. При преобразовании строковой константы в тип данных `datetimeoffset` всегда следует использовать формат `YYYY-MM-DD HH:MI:SS.NNNNNNN[+|-HH:MI]`. В табл. 3.7 перечислены объемы памяти, используемые типом данных `datetimeoffset` в зависимости от заданной точности.

**Таблица 3.7.** Требуемый объем памяти для типа данных *datetimeoffset*

| Точность | Объем памяти, байт |
|----------|--------------------|
| 0—2      | 6                  |
| 4—5      | 7                  |
| 6—7      | 8                  |

## Основные сведения о таблицах

Таблица — центральный объект любой СУРБД. В SQL Server существует несколько разновидностей таблиц: постоянные таблицы (или просто таблицы), ло-

кальные временные таблицы, глобальные временные таблицы и табличные переменные. Помимо этих разновидностей есть также табличные типы и табличные параметры, которые будут подробно обсуждаться в *главе 5*.

## Создание таблицы

Для создания таблицы необходима схема, в которой будет создаваться таблица. Схема аналогична пространству имен во многих языках программирования, но у схем может быть только один уровень (т. е. схемы не могут размещаться в других схемах). Во вновь создаваемых базах данных уже существует несколько схем: `dbo`, `sys` и `information_schema`. Схема `dbo` — это схема по умолчанию для новых объектов, в то время как схемы `sys` и `information_schema` используются разными системными объектами. До версии SQL Server 2005 схем не существовало. Вместо размещения в схеме объект принадлежал пользователю базы данных (однако синтаксическая запись была такой же `<владелец>.<объект>`). В этих версиях рекомендовалось, чтобы все объекты принадлежали схеме `dbo`, но это уже не верно. Начиная с версии SQL Server 2005, все объекты должны принадлежать схеме, определенной пользователем. Схемы создаются с помощью инструкции `CREATE SCHEMA`, как показано в следующем примере создания схемы и таблицы в ней.

```
CREATE SCHEMA Sales;
GO
```

```
CREATE TABLE Sales.Customers (CustomerId INT NOT NULL
 , Name NVARCHAR(50) NOT NULL);
```

Таблицы создаются инструкцией `CREATE TABLE` или `SELECT...INTO` (инструкция `SELECT...INTO` создает новую таблицу, основываясь на запросе). Далее приведена базовая синтаксическая запись инструкции `CREATE TABLE`.

```
CREATE TABLE
 [database_name . [schema_name] . | schema_name .] table_name
 ({ <column_definition> | <computed_column_definition>
 | <column_set_definition> }
 [<table_constraint>] [,...n])
 [ON { partition_scheme_name (partition_column_name) | filegroup
 | "default" }]
 [{ TEXTIMAGE_ON { filegroup | "default" } }]
 [WITH (<table_option> [,...n])]
[;]
```

Прежде чем мы углубимся в тонкости синтаксической записи, рассмотрим правила, применяемые для именования таблиц и столбцов.



## Имена таблиц и столбцов (идентификаторы)

Имена таблиц и столбцов — это идентификаторы и они должны придерживаться определенных правил. Идентификаторы бывают стандартными или с ограничителями. Далее перечислены требования к каждому из этих типов.

**Стандартные идентификаторы.** Далее приведены требования к стандартным идентификаторам.

- ◆ Первым символом должна быть буква или знак подчеркивания (`_`), но не цифра.

### ***Исключение***

Первым символом также может быть знак "эт" (`@`) или знак номера (`#`), но у них обоих следующие специальные назначения:

- `@` определяет переменную или параметр;
  - `@@` означают то же, что и `@`, и их не следует применять, поскольку многие системные функции начинаются со знаков `@@`;
  - `#` обозначает временный объект (т. е. объект, доступен только в текущем соединении);
  - `##` обозначает глобальный временный объект (т. е. объект, доступен в любом соединении с одним и тем же экземпляром).
- ◆ Последующие символы могут быть буквами, цифрами, знаком `@`, знаком доллара (`$`), знаком номера (`#`) и знаком подчеркивания (`_`).
  - ◆ Идентификатор не должен быть зарезервированным словом языка T-SQL.
  - ◆ Пробелы и специальные символы внутри идентификаторов не допускаются.

**Идентификаторы с ограничителями.** Любой идентификатор, не удовлетворяющий правилам именования для стандартных идентификаторов, должен иметь ограничители в виде знаков кавычек (`"`) или квадратных скобок (`[]`). Применение кавычек соответствует стандарту ANSI языка SQL, но вы должны знать, что для того чтобы применять кавычки в идентификаторах с ограничителями, установочный параметр сеанса `SET QUOTED_IDENTIFIER` должен иметь значение `ON`. (Квадратные скобки могут использоваться всегда в идентификаторах с ограничителями.) По умолчанию параметр `SET QUOTED_IDENTIFIER` имеет значение `ON`, но более старым версиям программного кода на языке T-SQL может потребоваться установить его в положение `OFF`, заставив SQL Server интерпретировать знаки кавычек не как ограничители идентификаторов, а как ограничители строк.

Далее приведены примеры идентификаторов.

```
-- Стандартные идентификаторы
CREATE TABLE HR.Employees (EmployeeId INT NOT NULL);
```

```
-- Идентификаторы с ограничителями:
SET QUOTED_IDENTIFIER ON;
```

```
CREATE TABLE HR."Organisation Employees" ("Employee Id" INT NOT NULL);
-- или
CREATE TABLE HR.[Organisation Employees] ([Employee Id] INT NOT NULL);
```

## Создание конкретной таблицы

Теперь перейдем к созданию простой таблицы. Рассмотрим пример.

```
CREATE TABLE HR.Employees (EmployeeId INT NOT NULL
 , FirstName NVARCHAR(50) NOT NULL
 , LastName NVARCHAR(50) NOT NULL
 , PhoneNumber VARCHAR(15) NULL
 , BirthDate DATE NOT NULL);
```

Этот программный код создает таблицу с именем `Employees` (Сотрудники), содержащую пять столбцов. Инструкция `CREATE TABLE` начинается с указания схемы, в которую следует поместить таблицу (в данном случае `HR`), имени таблицы (`Employees`) и столбцов таблицы. Столбцы задаются тремя базовыми свойствами: именем столбца, типом данных и допустимостью пропусков данных (*nullability*) (может ли столбец содержать значения `NULL`).

## Рекомендации по именованию

При выборе имен таблиц и столбцов важно следовать правилам именования, принятым в организации или выработанным при разработке проекта. Далее приведено несколько типичных рекомендаций по именованию.

- ◆ Применяйте `PascalCasing` (также известный как `upper camel casing` (ВерблюжийРегистр, также ГорбатыйРегистр, СтильВерблюда<sup>1</sup>).
- ◆ Избегайте аббревиатур.
- ◆ Длинное, но понятное пользователю имя предпочтительнее короткого, но непонятного.

## Выбор типов данных

Типы данных, применяемые в каждом столбце, также очень важны. Мы уже обсудили большинство типов данных, доступных в `SQL Server 2008`, но в этом разделе приводятся некоторые рекомендации, которым вы должны пытаться следовать при выборе используемых типов данных.

- ◆ Всегда применяйте тип данных, которому требуется минимальный объем дискового пространства для реализации нужных вам функций.

---

<sup>1</sup> Стиль написания составных слов, при котором несколько слов пишутся слитно без пробелов, при этом каждое слово пишется с прописной буквы. Стиль получил название `CamelCase`, поскольку заглавные буквы внутри слова напоминают горбы верблюда (англ. `Camel`). [www.wikipedia.org](http://www.wikipedia.org). — Пер.

- ◆ Изменение типа данных впоследствии — всегда очень дорогостоящий процесс (принимая во внимание как время разработки, так и ресурсы сервера). Не применяйте тип данных, если есть вероятность того, что он может не удовлетворить нужды вашего приложения в будущем.
- ◆ В большинстве случаев используйте тип данных переменной длины, такой как `nvarchar`, вместо типа данных фиксированной длины, например, `nchar`.
- ◆ Один из возможных случаев, в которых применение типа данных фиксированной длины предпочтительнее типа данных переменной длины, — частое изменение значения столбца. Если значение часто обновляется, затраты на перемещение строки в другое место, подходящее для нового значения, могут превысить расходы, связанные с дополнительным объемом памяти, необходимым для фиксированного типа данных.
- ◆ Избегайте применения типов данных `datetime` и `smalldatetime`, т. к. они занимают больше дискового пространства и обеспечивают меньшую точность, чем новые типы данных `date`, `time` и `datetime2`.
- ◆ Используйте типы данных `varchar(max)`, `nvarchar(max)` и `varbinary(max)` вместо типов данных `text`, `ntext` и `image`, которые могут быть удалены из будущих версий SQL Server.
- ◆ Применяйте вместо типа данных `timestamp` тип данных `rowversion`, поскольку тип данных `timestamp` может отсутствовать в новых версиях SQL Server.
- ◆ Используйте типы данных `varchar(max)`, `nvarchar(max)`, `varbinary(max)` и `xml`, только если нельзя использовать тип данных с заданным размером. Применение этих типов данных лишает вас возможности оперативно перестраивать индексы, и эти типы не могут применяться в ключе индекса.

### ***Дополнительная информация об индексах***

Индексы подробно обсуждаются в главе 6.

- ◆ Применяйте типы данных `float` или `real`, только если разрядность, предоставляемая типом `decimal`, недостаточна.

### ***NULL или NOT NULL?***

Решение о допустимости или недопустимости в столбце значений `NULL`, может стать проблемой. У многих людей очень строгие взгляды на значения `NULL` — они либо принимают их, либо настроены к ним крайне враждебно.

На самом деле, принять подобное решение очень легко. Как правило, не разрешайте их использовать, поскольку это простейший способ создания таблицы. Если разрешить применение значений `NULL` там, где в них нет нужды, существенно возрастает вероятность возникновения проблем при создании запросов к вашим таблицам.

Если значения в вашем столбце не обязательны (т. е. не во всех строках есть значения), в вашем столбце должны быть допустимы значения `NULL`. Никогда не следует использовать другие значения вместо `NULL` (например `-1` для целых), т. к. это создаст множество проблем в ваших запросах. Причина в том, что `-1` означает "минус один", а не "unknown" (неизвестное значение), которое является определением значения `NULL`. Например, если вы используете функцию `AVG`, значения `-1` она включает в вычисление, а значения `NULL` пропускает.

Другой вариант — добавить новую таблицу со связью "один-к-одному" с таблицей, которую вы разрабатываете, и хранить потенциально неизвестное значение в другой таблице. Если в строке не должно быть значения, вы просто не включаете строку в другую таблицу. Рассмотрим пример.

```
CREATE TABLE HR.Employees (EmployeeId INT NOT NULL
 , FirstName NVARCHAR(50) NOT NULL
 , LastName NVARCHAR(50) NOT NULL
 , BirthDate DATE NOT NULL);
CREATE TABLE HR.EmployeePhoneNumbers (EmployeeId INT NOT NULL
 , PhoneNumber VARCHAR(15) NOT NULL);
-- Сотрудник с номером телефона:
INSERT HR.Employees (EmployeeId, FirstName, LastName, BirthDate)
VALUES (1, N'John', N'Kane', '1970-02-20');

INSERT HR.EmployeePhoneNumbers (EmployeeId, PhoneNumber)
VALUES (1, N'+1-425-555-1234');

-- Сотрудник без номера телефона:
INSERT HR.Employees (EmployeeId, FirstName, LastName, BirthDate)
VALUES (2, N'Jane', N'Dow', '1965-05-30');
```

Эта реализация применяется не слишком часто, потому что она увеличивает потребность в запросах с операторами `OUTER JOIN` или вложенными запросами для извлечения из другой таблицы столбцов с допустимыми значениями `NULL`. Это в свою очередь повышает риск снижения производительности и усложняет запросы по сравнению с вариантом, допускающим значения `NULL` в исходной таблице.

## Идентификация

У всех таблиц должен быть один или несколько столбцов, однозначно определяющих строки в таблице. Такие столбцы называются *первичным ключом*, который обсуждается в *уроке 2 данной главы*. Чаще всего трудно выбрать столбец данных, значения которого могут служить хорошим первичным ключом, потому что обычно они не гарантируют уникальности строк или потому что значения могут часто меняться. Вместо применения такого столбца, именуемого *естественным ключом*, можно использовать технический, автоматически генерируемый ключ. В SQL Server для указания одного столбца в таблице, значения которого

должны автоматически увеличиваться или уменьшаться по мере добавления строк, применяется свойство `IDENTITY`. Синтаксическая запись идентифицирующего столбца:

```
<column name> <data type> IDENTITY(<seed>, <increment>) NOT NULL
```

Здесь *seed* — начальное значение для генерации чисел, а *increment* — это значение, на которое ключ увеличивается (или уменьшается, если оно отрицательное). В идентифицирующем столбце значения `NULL` не допускаются.

В следующем примере создается таблица `HR.Employees` (Сотрудники) и определяется столбец `EmployeeId` (Идентификатор сотрудника) с техническим ключом, начинающимся со значения 1000 и увеличивающимся на 2 для каждой строки.

```
CREATE TABLE HR.Employees (EmployeeId INT IDENTITY(1000, 2) NOT NULL
 , FirstName NVARCHAR(50) NOT NULL
 , LastName NVARCHAR(50) NOT NULL
 , BirthDate DATE NOT NULL);
```

Имейте в виду, что свойство `IDENTITY` можно задать только при создании нового столбца. Существующий столбец нельзя преобразовать в идентифицирующий. Для изменения существующего столбца его нужно удалить, а на его место вставить идентифицирующий столбец.

Идентифицирующий столбец можно использовать только с типами данных, хранящими целые числа, такими как типы данных `integer` и `decimal` с количеством знаков в дробной части, равным 0.

Другое важное замечание, касающееся идентифицирующих столбцов, состоит в том, что они не формируют полных последовательностей. Если вставка завершилась неудачно, она все равно использует идентифицирующее значение, создавая дыру в последовательности. Рассмотрим следующий пример.

```
INSERT HR.Employees (FirstName, LastName, BirthDate)
VALUES ('John', 'Kane', '1970-01-30');
-- Генерируется EmployeeID 1000
GO
```

```
INSERT HR.Employees (FirstName, LastName, BirthDate)
VALUES ('John', 'Kane', '1970-01-32');
-- Завершается аварийно из-за неверной даты
GO
```

```
INSERT HR.Employees (FirstName, LastName, BirthDate)
VALUES ('Jane', 'Dow', '1972-03-30');
-- Генерируется EmployeeID 1004 (1002 пропущен)
GO
```

В данном примере вторая инструкция `INSERT` завершается аварийно и ключ 1002 пропускается, поэтому Jane Dow вставляется с ключом 1004.

## Сжатие

В SQL Server 2008 введена возможность сжатия данных в таблицах и индексах, если используется версия SQL Server 2008 Enterprise Edition. Сжатие данных реализовано на двух уровнях: строки и страницы. В следующей инструкции таблица настраивается на сжатие на уровне страницы.

```
ALTER TABLE HR.Employees
 REBUILD
 WITH (DATA_COMPRESSION = PAGE);
```

Если настроить сжатие на уровне строки, SQL Server изменяет формат, применяемый для хранения строк. Проще говоря, этот строковый формат преобразует все типы данных в типы переменной длины. Он также не использует память для хранения значений NULL. Чем больше типов данных фиксированной длины (таких как datetime2, int, decimal и nchar) вы используете в таблице, тем больше будет ваш выигрыш от сжатия на уровне строки.

Сжатие на уровне страницы включает сжатие на уровне строки и добавляет сжатие на уровне страницы с помощью словаря страницы (page dictionary) и префиксов в именах столбцов (column prefixing). Словарь страницы просто вводит указатели между строками одной и той же страницы, чтобы избежать хранения избыточных данных. Рассмотрим упрощенный механизм хранения имен на уровне страницы.

```
Row 01: John Kane
Row 02: John Woods
Row 03: John Kane
```

Если бы эта страница применяла словарь страницы, она выглядела бы так.

```
Row 01: John Kane
Row 02: John Woods
Row 03: 01
```

В данном случае значение в строке 03 (Row 03) указывает на значение в строке 01 (Row 01), экономя несколько байтов памяти. Сжатие на уровне страницы включает также задание префикса столбца, которое аналогично словарю страницы, но может повторно использовать части значений.

Решая, применять или не применять сжатие на уровне строки или страницы, очень важно проверить, сколько на самом деле памяти будет сэкономлено при включении сжатия.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

### Создание таблиц и типов данных

На этом практическом занятии вы создадите таблицы и типы данных и измените имеющиеся таблицы. Поскольку упражнения образуют некую логическую последовательность, важно выполнять их в указанном порядке.

## Упражнение 1. Создание новой таблицы

В этом упражнении вы создадите таблицу, которая сможет хранить информацию о клиентах.

1. Откройте среду Microsoft SQL Server Management Studio (SSMS) и установите соединение с экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных TestDB и схемы Test.

```
CREATE DATABASE TestDB;
GO
```

```
USE TestDB;
GO
```

```
CREATE SCHEMA Test;
GO
```

3. В окне запроса создайте новую таблицу со следующими свойствами.

- Ее имя должно быть Customers (Клиенты).
- Она должна находиться в схеме Test.
- У нее должны быть следующие столбцы.
  - CustomerId (Id клиента) — целое число в диапазоне от 1 до 100 000. У этого столбца значения должны увеличиваться автоматически. Первое значение должно быть равно 1000, а затем каждая последующая строка должна получать значение, увеличенное на 1. Следует применить тип данных, использующий минимальный необходимый объем памяти.
  - Name (Имя) — строка, содержащая символы Unicode, длиной до 70 символов.
  - CreatedDateTime (Дата и время создания) — дата и время суток, когда клиент был добавлен в базу данных.
  - CreditLimit (Кредитный лимит) — точное числовое значение, допускающее до пяти знаков в дробной части и числа, меньшие 10 000 000.

Во всех столбцах, кроме CustomerId, допускаются значения NULL.

Далее приведены правильные инструкции, но не смотрите на них, пока не попробуете создать таблицу самостоятельно.

Введите, выделите и выполните следующую инструкцию.

```
CREATE TABLE Test.Customers (CustomerId INT IDENTITY(1000, 1) NOT NULL
 , Name NVARCHAR(70) NULL
 , CreatedDateTime DATETIME2 NULL
 , CreditLimit DECIMAL(12,5) NULL);
```

## Упражнение 2. Создание новых типов данных

В этом упражнении вы создадите два новых типа данных, которые могут применяться в приложении базы данных для минимизации ошибок в случае использования разных типов для данных одного и того же вида.

1. Если необходимо, откройте SSMS и установите соединение с нужным экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующую инструкцию SQL для использования базы данных TestDB.

```
USE TestDB;
```

3. В окне запроса создайте новый тип данных со следующими свойствами.
  - Он должен называться NAME.
  - Он должен находиться в схеме Test.
  - Это должна быть строка до 70 символов длиной, содержащая символы Unicode.

Правильная инструкция следует далее, но не смотрите на нее, а попытайтесь сначала создать тип данных самостоятельно.

4. Введите, выделите и выполните следующую инструкцию.

```
CREATE TYPE Test.NAME FROM NVARCHAR(70);
```

5. В окне запроса создайте новый тип данных со следующими свойствами.
  - Он должен называться CURRENCYVALUE (Валютная стоимость).
  - Он должен размещаться в схеме Test.
  - Это должно быть точное число, которое может содержать до пяти знаков в дробной части и принимать значения до 999 999 999.99999.

Правильная инструкция приведена далее, но сначала не смотрите на нее, а попытайтесь создать тип данных самостоятельно.

6. Введите, выделите и выполните следующую инструкцию.

```
CREATE TYPE Test.CURRENCYVALUE FROM DECIMAL(14,5);
```

## Упражнение 3. Модификация существующей таблицы

В этом упражнении вы измените таблицу, созданную вами в упражнении 1, для того чтобы воспользоваться типами данных, созданными в упражнении 2.

1. Если необходимо, откройте SSMS и установите соединение с нужным экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующую инструкцию SQL для использования базы данных TestDB.

```
USE TestDB;
```



3. В окне запроса модифицируйте таблицу `Test.Customers`, внося в нее следующие изменения:

- замените тип данных столбца `Name` на тип `Test.NAME` и запретите значения `NULL`;
- замените тип данных столбца `CreditLimit` на тип `Test.CURRENCYVALUE`;
- измените столбец `CreatedDateTime`, запретив в нем значения `NULL`.

Корректный набор инструкций приведен далее, но прежде чем просмотреть его, попробуйте модифицировать таблицу самостоятельно.

Введите и выполните следующие инструкции.

```
ALTER TABLE Test.Customers
 ALTER COLUMN Name Test.NAME NOT NULL;

ALTER TABLE Test.Customers
 ALTER COLUMN CreditLimit Test.CURRENCYVALUE NULL;

ALTER TABLE Test.Customers
 ALTER COLUMN CreatedDateTime DATETIME2 NOT NULL;
```

#### Упражнение 4. Реализация сжатия данных

В этом упражнении вы пополните таблицу `Test.Customers` 100 000 строк и затем сравните занимаемый таблицей объем дискового пространства в зависимости от примененного уровня сжатия данных. Учтите, что сжатие данных доступно только в версиях Developer, Enterprise и Enterprise Evaluation SQL Server 2008.

1. Если необходимо, откройте SSMS и установите соединение с нужным экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующую инструкцию SQL для использования базы данных `TestDB`.

```
USE TestDB;
```

3. В окне запроса введите, выделите и выполните следующий запрос для заполнения таблицы `Test.Customers` 100 000 строк.

```
INSERT Test.Customers (Name, CreatedDateTime, CreditLimit)
 SELECT TOP(100000)
 sol.name,
 SYSDATETIME()
 CASE
 WHEN ABS(sol.object_id) > 100000000 THEN NULL
 ELSE ABS(sol.object_id)
 END
 FROM sys.all_objects AS sol
 CROSS JOIN sys.all_objects AS so2;
```

4. В окне запроса введите, выделите и выполните следующие инструкции для перестройки таблицы в режиме без сжатия и формирования отчета о занимаемом таблицей дисковом пространстве.

```
ALTER TABLE Test.Customers
 REBUILD WITH (DATA_COMPRESSION = NONE);

EXEC sp_spaceused
 @objname = 'Test.Customers',
 @updateusage = 'true';
```

Обратите внимание на общий объем дискового пространства, зарезервированный для таблицы и отображенный хранимой процедурой `sp_spaceused`.

5. В окне запроса введите, выделите и выполните следующий запрос для перестройки таблицы с использованием сжатия на уровне строк и вывода сообщения о дисковом пространстве, занимаемом таблицей.

```
ALTER TABLE Test.Customers
 REBUILD WITH (DATA_COMPRESSION = ROW);

EXEC sp_spaceused
 @objname = 'Test.Customers',
 @updateusage = 'true';
```

Запомните общий объем дискового пространства, зарезервированный для таблицы.

6. В окне запроса введите, выделите и выполните следующий запрос для формирования таблицы с применением режима сжатия на уровне страницы и отображения сведений о дисковом пространстве, занимаемом таблицей.

```
ALTER TABLE Test.Customers
 REBUILD WITH (DATA_COMPRESSION = PAGE);

EXEC sp_spaceused
 @objname = 'Test.Customers',
 @updateusage = 'true';
```

Запомните объем дискового пространства, зарезервированный для таблицы.

7. Для очистки диска после этого практического занятия закройте все открытые в SSMS окна запросов, откройте окно создания запроса и выполните следующую инструкцию SQL.

```
USE master;
GO

DROP DATABASE TestDB;
```

## Резюме урока

- ♦ Создание таблиц — нечто большее, чем просто описание столбцов. Очень важно выбрать правильный тип данных и обеспечить целостность данных.
- ♦ Для того чтобы корректно применять различные типы данных, следует прежде всего познакомиться с особенностями их реализации.
- ♦ Целостность данных должна быть частью определения вашей таблицы с самого начала, давая уверенность в том, что вы защитили свои данные от ошибок.

## УРОК 2. Декларативная целостность данных

Проверка допустимости данных — одна из наиболее общих задач в разработке программного обеспечения. И как результат, подпрограммы проверки данных приобретают тенденцию распространения по разным приложениям. Вы с большой долей вероятности найдете проверку данных в следующих технологиях:

- ♦ приложения Microsoft Windows Forms или Windows Presentation Foundation (WPF);
- ♦ страницы ASP.NET и приложения Silverlight;
- ♦ язык JavaScript, встроенный в язык гипертекстовой разметки (HyperText Markup Language, HTML);
- ♦ бизнес-компоненты (например, сборки библиотеки .NET или компоненты COM);
- ♦ базы данных.

К сожалению, очень часто обнаруживается, что в базах данных создано слишком мало подпрограмм проверки данных. Происходит это потому, что многие разработчики склонны верить, что проверка выполнена до того, как данные добрались до базы данных. Предлагаемый урок посвящен разновидности проверки в базе данных, которую вы можете и, вероятно, должны выполнять, и способу ее реализации.

---

### ***После усвоения этого урока вы сможете***

- Обеспечивать декларативную целостность в ваших таблицах.
- Задавать ограничения первичного ключа.
- Задавать ограничения внешнего ключа.
- Задавать ограничения UNIQUE.
- Задавать ограничения CHECK.

Расчетное время урока — 60 мин.

## Проверка данных

В SQL Server проверить целостность данных можно двумя способами: с помощью декларативной целостности данных или с помощью процедурной целостности данных (procedural data integrity).

*Декларативная целостность данных* — это набор правил, которые применяются к таблице и ее столбцам в инструкциях `CREATE TABLE` или `ALTER TABLE`. Эти правила называют *ограничениями*.

---

### **Не используйте правила!**

---

Декларативную целостность данных можно обеспечить и с помощью правил (используя инструкцию `CREATE RULE`); но правила применять не следует, т. к. они будут удалены в будущих выпусках SQL Server.

*Процедурная целостность данных* реализуется либо с помощью хранимой процедуры, которой предоставляется возможность проверки допустимости данных, либо с помощью создания триггеров, проверяющих данные до или после задания инструкции языка манипулирования данными (DML) (такой как `INSERT`, `UPDATE` или `DELETE`). Хранимые процедуры и триггеры обсуждаются в *главе 5*.

Как правило, декларативная целостность данных — это простейшая встраиваемая проверка целостности, поскольку она требует минимума усилий от разработчика. Вероятность того, что она породит ошибки, крайне мала, поскольку этот вид целостности содержит меньше программного кода, чем процедурная целостность данных. С другой стороны, процедурная целостность предусматривает гораздо более развитые варианты проверки целостности. Типичное приложение базы данных должно реализовывать как декларативную, так и процедурную целостность данных. Этот урок посвящен декларативной целостности данных.

## Реализация декларативной целостности данных

Декларативная целостность данных реализуется с помощью ограничений. Есть пять типов ограничений: `PRIMARY KEY`, `UNIQUE`, `FOREIGN KEY`, `CHECK` и `DEFAULT`.

**Ограничения `PRIMARY KEY` и `UNIQUE`.** Первичные ключи и ограничения `UNIQUE` определяют столбец или комбинацию столбцов, которые однозначно идентифицируют строку в таблице. Реализуется это созданием уникального индекса, т. е. индекса, для которого не допустимы дублирующиеся значения. По этой причине первичный ключ и ограничения `UNIQUE` имеют одни и те же ограничения размера, что и ключ индекса, они не могут включать в себя более 15 столбцов или 900 байтов данных.

Если больше ничего не задано, для первичного ключа создается *кластеризованный* индекс (clustered index), а для ограничений `UNIQUE` — *некластеризованный* индекс (non-clustered index). Но это поведение можно изменить, задав тип создаваемого индекса в инструкции `ALTER TABLE` или `CREATE TABLE` следующим образом.

```
-- Первичный ключ как кластеризованный индекс.
ALTER TABLE MyTable
 ADD PRIMARY KEY (MyTableID);

-- Первичный ключ как некластеризованный индекс.
ALTER TABLE MyTable
 ADD PRIMARY KEY NONCLUSTERED (MyTableID);
```

Поскольку первичные ключи и ограничения `UNIQUE` — одновременно и ограничения и индексы, информацию о них можно найти в представлениях каталога `sys.key_constraints` и `sys.indexes`.

### **Вычисляемые столбцы**

Первичные ключи и ограничения уникальности можно создавать на вычисляемых столбцах.

**Ограничения `FOREIGN KEY`.** Ограничения внешнего ключа (`FOREIGN KEY`) определяют столбец или комбинацию столбцов, чьи значения должны существовать в другом столбце или комбинации столбцов в той же или другой таблице одной и той же базы данных. Ограничения внешнего ключа управляют ссылочной целостностью между таблицами или в одной таблице. Для реализации ограничения внешнего ключа вы должны следовать приведенным далее правилам.

- ◆ Столбцы, на которые ссылаются, должны иметь точно такой же тип данных (и набор параметров символьной обработки для строковых столбцов), что и местные столбцы.
- ◆ Столбцы, на которые ссылаются, должны иметь созданный для них уникальный индекс. Обычно он формируется с помощью первичного ключа или ограничения `UNIQUE`.
- ◆ Поскольку внешний ключ должен указывать на уникальный индекс, у столбцов внешнего ключа те же ограничения размера, что и у ограничения для первичного ключа и ограничения `UNIQUE`.

Вы также можете создавать внешние ключи на базе вычисляемых столбцов. Информацию об имеющихся в вашей базе данных ограничениях внешнего ключа можно найти, запросив представления каталога `sys.foreign_keys` и `sys.foreign_key_columns`.

Внешние ключи, как правило, часто запрашиваются в пользовательских запросах и в соединениях, а также когда SQL Server должен проверить ссылочную целостность при удалении или обновлении строк первичного ключа. Это означает, что внешние ключи обычно сильно выигрывают от индексирования. Более подробно индексирование будет обсуждаться в *главе 6*.

Когда ограничение внешнего ключа замечает нарушение ссылочной целостности из-за операции `DELETE` или `UPDATE` со строкой, на которую внешний ключ ссыла-

ется, стандартная реакция — генерация сообщения об ошибке и откат инструкции, нарушившей ограничение. Если для вас это нежелательный результат, можно изменить действие по умолчанию для внешнего ключа при удалении строки, на которую он ссылается, или обновлении столбца, на который дается ссылка, или оба действия. Есть четыре варианта действий, из которых можно выбрать:

- ◆ NO ACTION (по умолчанию);
- ◆ SET NULL;
- ◆ SET DEFAULT;
- ◆ CASCADE.

Далее приведен пример реализации.

```
CREATE TABLE Test.Customers(CustomerID INT PRIMARY KEY);
CREATE TABLE Test.Orders(OrderID INT PRIMARY KEY
 , CustomerID INT NULL
 REFERENCES Test.Customers
 ON DELETE SET NULL
 ON UPDATE CASCADE);
```

Стандартное поведение для ограничения внешнего ключа — NO ACTION. Если внешний ключ обнаруживает нарушение и задано NO ACTION, SQL Server отменяет действие инструкции, нарушившей ограничение, и генерирует сообщение об ошибке.

SET NULL и SET DEFAULT вместо генерации сообщения об ошибке и отката инструкции присваивают всем значениям, на которые делается ссылка, либо NULL (для SET NULL), либо DEFAULT (для SET DEFAULT, если конечно для столбца значение по умолчанию задано).

В связанных таблицах Orders (Заказы) и Customers (Клиенты), показанных в примере программного кода, если удаляется клиент, столбцу CustomerID (ID клиента) присваивается значение NULL во всех заказах, принадлежащих этому клиенту, и вызвавшему приложению не посылается никакого сообщения об ошибке.

Действие CASCADE заставляет SQL Server для инструкции DELETE удалить все строки, на которые делается ссылка (ON DELETE), и для инструкции UPDATE обновить все значения, на которые делается ссылка (ON UPDATE). Если в том же примере программного кода изменяется столбец CustomerID в строке из таблицы Customers, все соответствующие строки в таблице Orders для отражения изменения обновляются тем же самым значением CustomerID. Если для ограничения внешнего ключа задано действие ON DELETE CASCADE и строка из таблицы Customers удаляется, все связанные с ней строки из таблицы Orders тоже удаляются. Это может показаться разумным, но иногда бывает невозможно реализовать CASCADE для всех ограничений внешнего ключа, поскольку не поддерживаются циклические ссылки. Например, в следующем сценарии будет сгенериро-

вано сообщение об ошибке, когда вы попытаетесь добавить внешний ключ FKCustomersLastOrder, потому что он создает циклическую ссылку. Если клиент удаляется, все связанные с ним заказы должны быть удалены и все клиенты, ссылающиеся на эти заказы через столбец LastOrderID, также должны быть удалены.

```
CREATE TABLE Test.Customers (CustomerID INT PRIMARY KEY
 , LastOrderID INT NULL);
```

```
CREATE TABLE Test.Orders (OrderID INT PRIMARY KEY
 , CustomerID INT NOT NULL
 REFERENCES Test.Customers
 ON DELETE CASCADE
 ON UPDATE NO ACTION);
```

```
ALTER TABLE Test.Customers ADD
 CONSTRAINT FKCustomersLastOrder
 FOREIGN KEY (LastOrderID)
 REFERENCES Test.Orders (OrderID)
 ON DELETE CASCADE
 ON UPDATE NO ACTION;
```

Рассмотрите, что произойдет в предыдущем примере, если удаляется клиент — все заказы клиента также будут удалены. Это могло бы быть удачным решением, но рассмотрим следующий программный код.

```
CREATE TABLE Test.Countries (CountryID INT PRIMARY KEY);
```

```
CREATE TABLE Test.Cities (CityID INT PRIMARY KEY
 , CountryID INT NOT NULL
 REFERENCES Test.Countries
 ON DELETE CASCADE);
```

```
CREATE TABLE Test.Customers (CustomerID INT PRIMARY KEY
 , CityID INT NOT NULL
 REFERENCES Test.Cities
 ON DELETE CASCADE);
```

```
CREATE TABLE Test.Orders (OrderID INT PRIMARY KEY
 , CustomerID INT NOT NULL
 REFERENCES Test.Customers
 ON DELETE CASCADE);
```

В этом примере, если вы удалите страну, все города в этой стране, все клиенты в этих городах и все заказы, принадлежащие этим клиентам, также будут удалены. Будьте осторожны — вы можете удалить больше, чем рассчитываете. Например, некто в среде SSMS выполняет запрос

```
DELETE Test.Countries WHERE CountryID = 1;
```

Исполнитель думает, что удаляет только одну строку в таблице `Countries` (Страны), в то время как он (или она) может на самом деле удалить миллионы строк. Время, необходимое для выполнения этой инструкции `DELETE`, зависит от количества удаляемых строк. Когда инструкция завершится, SSMS вернет следующее сообщение.

```
(1 row(s) affected)
```

Такое сообщение выводится, даже если были удалены миллионы строк, потому что сообщение извещает нас только о том, сколько строк непосредственно было удалено выполняемой инструкцией. В таком поведении нет ничего неправильно-го, но его обязательно нужно принимать во внимание.

### Триггеры

Если вы задали внешние ключи с каскадными действиями, любые триггеры `AFTER`, применяемые к обрабатываемым таблицам, все равно выполняются, но происходит это после того, как вся цепочка каскадных действий завершена. Если во время ее выполнения возникает ошибка, выполняется откат всей цепочки каскадных действий, и для этой цепочки никакие триггеры `AFTER` не выполняются.

**Ограничения `CHECK`.** Ограничения `CHECK` — это набор правил, которые должны проверяться до того, как будет разрешено помещать данные в таблицу. К достоинствам применения этого типа ограничений относятся следующие.

- ◆ их просто реализовать (они очень похожи на предложение `WHERE`);
- ◆ они проверяются автоматически;
- ◆ они могут повышать производительность.

Далее показан пример проверочного ограничения, которое убеждает в том, что у товара неотрицательная цена.

```
ALTER TABLE Products
ADD CHECK(Price >= 0.0);
```

Простота проверочных ограничений — огромное преимущество по сравнению с использованием триггеров. Но существуют и некоторые недостатки, к ним относятся следующие:

- ◆ сообщения об ошибках, поступающие от проверочных ограничений, генерируются системой, и их нельзя заменить более понятными пользователям;
- ◆ проверочное ограничение не может "просмотреть" предыдущее значение столбца. Это означает, что оно не может применяться для некоторых типов данных в правилах целостности данных, таких как "Обновления столбца с ценой не могут увеличить или уменьшить цену на более чем 10%."

Важная особенность проверочных ограничений заключается в том, что они отбрасывают значения, равные `False`, скорее, чем принимают значения, равные `True`. Кажется, что это одно и то же, но в SQL Server это не так из-за проблемы,



связанной со значениями `NULL`, что очень важно знать. Например, если у вас есть проверочное ограничение, которое утверждает, что `Price > 10.0`, вы все равно можете вставить значение `NULL` в столбец `Price` (Цена). Это значение допустимо, потому что любое сравнение со значением `NULL` возвращает `NULL`, что не равно ни `True`, ни `False`. Если вы не хотите, чтобы проверочное ограничение разрешало значения `NULL`, вы можете либо запретить значения `NULL` в столбце `Price`, задав ограничение для столбца `NOT NULL`, либо изменить проверочное ограничение на следующее: `Price > 10.0 AND Price IS NOT NULL`.

**Усовершенствование проверочных ограничений с помощью функций, определенных пользователем.** Функции, определенные пользователем, или пользовательские функции (User-defined function, UDF) создаются на языке T-SQL и в виде управляемого кода (managed code, также называемого пользовательскими функциями .NET или CLR) и могут быть составной частью проверочных ограничений, следовательно, кратко обсуждаются в данном разделе. Более подробное описание пользовательских функций см. в главе 5.

Выражение в проверочном ограничении может содержать большую часть логических операторов, которые применяются в предложении `WHERE` (включая `NOT`, `AND` и `OR`). В нем можно вызывать скалярные пользовательские функции и ссылаться на другие столбцы в той же таблице, но ограничение не может непосредственно содержать вложенные запросы. Поскольку вы можете писать собственные скалярные функции на языке T-SQL или в виде управляемого кода, через них в проверочных ограничениях можно применять сложную логику и даже использовать вложенные запросы.

В следующем примере создается в виде управляемого кода пользовательская функция `fnIsPhoneNumber` (приведенная на языках Microsoft Visual Basic и C#) для проверки с помощью регулярного выражения того, что строка содержит корректный номер телефона в США.

```
' VB
<Microsoft.SqlServer.Server.SqlFunction(IsDeterministic:=True, _
 DataAccess:=DataAccessKind.None)> _
Public Shared Function fnIsPhoneNumber(ByVal phoneNumber As SqlString) _
 As SqlBoolean
 If (phoneNumber.IsNull) Then
 Return SqlBoolean.Null
 End If
 Return System.Text.RegularExpressions.Regex.IsMatch _
 (phoneNumber.Value, "^([1-9]\d{2})\s?\d{3}-\d{4}$")
End Function

// C#
[SqlFunction(IsDeterministic = true, DataAccess=DataAccessKind.None)]
static public SqlBoolean fnIsPhoneNumber(SqlString phoneNumber)
```

```
{
 if (phoneNumber.IsNull){
 return SqlBoolean.Null;
 }
 return System.Text.RegularExpressions.Regex
 .IsMatch(phoneNumber.Value, @"^\([1-9]\d{2}\)\s?\d{3}\-\d{4}$");
}
```

### **Дополнительная информация о пользовательских функциях**

Пользовательские функции подробно обсуждаются в *главе 7*.

В следующем программном коде создается таблица и проверочное ограничение, ссылающееся на пользовательскую функцию.

```
CREATE TABLE Test.Contacts (ContactID INT IDENTITY PRIMARY KEY
 , Name NVARCHAR(50) NOT NULL
 , PhoneNumber VARCHAR(20) NULL
 , CONSTRAINT CKContactsPhoneNumber
 CHECK(dbo.fnIsPhoneNumber(PhoneNumber) = CAST(1 AS BIT))
);

-- Допустима:
INSERT Test.Contacts (Name, PhoneNumber)
VALUES ('Tobias', '(425)555-1111');
INSERT Test.Contacts (Name, PhoneNumber)
VALUES ('Chris', NULL);

-- Не допустима, порождает сообщение об ошибке:
INSERT Test.Contacts (Name, PhoneNumber)
VALUES ('Ann', '(42)555-2222');
```

Когда это проверочное ограничение выполняется? Только при необходимости. Оптимизатор выполняет проверочное ограничение, только если столбцы, на которые ссылаются в проверочных ограничениях, упоминаются в выполняемой инструкции DML. Для инструкций `INSERT` это всегда справедливо, т. к. `INSERT` воздействует на все столбцы (даже если вставляется значение `NULL`). Для инструкций `UPDATE` проверочное ограничение выполняется, только если на столбец, содержащийся в проверочном ограничении, есть ссылка в инструкции `UPDATE`.

### **Производительность**

Вставка большого количества логических операторов в ваши проверочные ограничения может нанести ущерб производительности. Полезно добавить нужные ограничения и затем выполнить тест производительности, чтобы убедиться в том, что она удовлетворительна.

**Применение пользовательской функции с вложенным запросом.** В проверочных ограничениях можно использовать вложенные запросы, если поместить их внутрь пользовательских функций. Но этот подход может привести к сниже-

нию производительности, т. к. вложенный запрос выполняется для каждой строки, на которую воздействует инструкция UPDATE или INSERT, обращенная к таблице. Предположим, что вы хотите расширить предыдущий пример, с помощью вложенного запроса проверяя телефонные коды регионов. Поддерживаемые коды регионов хранятся в отдельной таблице, названной Test.AreaCodes. Далее приведена расширенная версия пользовательской функции.

```
' VB
<Microsoft.SqlServer.Server.SqlFunction(IsDeterministic:=True, _
 DataAccess:=DataAccessKind.Read)> _
Public Shared Function fnIsPhoneNumber2(ByVal phoneNumber As SqlString) _
 As SqlBoolean

 If (phoneNumber.IsNull) Then
 Return SqlBoolean.Null
 End If
 If Not System.Text.RegularExpressions.Regex.IsMatch _
 (phoneNumber.Value, "^([1-9]\d{2})\s?\d{3}\-\d{4}$") Then
 Return False
 Else
 Dim areaCode As String = phoneNumber.Value.Substring(1, 3)

 Using conn As SqlConnection = _
 New SqlConnection("context connection=true;")
 Using cmd As SqlCommand = conn.CreateCommand()
 cmd.CommandText = _
 "IF EXISTS(SELECT * FROM Test.AreaCodes " & _
 " WHERE AreaCode = @AreaCode) " & _
 " SELECT CAST(1 AS BIT) AS Found " & _
 "ELSE " & _
 " SELECT CAST(0 AS BIT) AS Found"
 cmd.Parameters.Add("@AreaCode", SqlDbType.Char, 3). _
 Value = areaCode

 conn.Open()
 Return CType(cmd.ExecuteScalar(), Boolean)
 End Using
 End Using
 End If
End Function

// C#
[SqlFunction(IsDeterministic = true, DataAccess=DataAccessKind.Read)]
static public SqlBoolean fnIsPhoneNumber2(SqlString phoneNumber)
{
 if (phoneNumber.IsNull)
 return SqlBoolean.Null;
```

```

if (!System.Text.RegularExpressions.Regex
 .IsMatch(phoneNumber.Value, @"^\([1-9]\d{2}\)\s?\d{3}\-\d{4}$")){
 return false;
}else{
 string areaCode = phoneNumber.Value.Substring(1,3);
 using(SqlConnection conn = new SqlConnection(
 @"context connection=true;"))
 {
 using(SqlCommand cmd = conn.CreateCommand())
 {
 cmd.CommandText = @"IF EXISTS(SELECT * FROM Test.AreaCodes
 WHERE AreaCode = @AreaCode)
 SELECT CAST(1 AS BIT) AS Found
 ELSE
 SELECT CAST(0 AS BIT) AS Found";
 cmd.Parameters.Add("@AreaCode", SqlDbType.Char, 3)
 .Value = areaCode;
 conn.Open();
 return (bool)cmd.ExecuteScalar();
 }
 }
}
}

```

В следующем программном коде создается таблица и проверочное ограничение, ссылающееся на пользовательскую функцию.

```

CREATE TABLE Test.AreaCodes (
 AreaCode CHAR(3) NOT NULL PRIMARY KEY
);
-- Единственный допустимый код региона.
INSERT Test.AreaCodes (AreaCode) VALUES ('425');
CREATE TABLE Test.Contacts (ContactID INT IDENTITY PRIMARY KEY
 , Name NVARCHAR(50) NOT NULL
 , PhoneNumber VARCHAR(20) NULL
 , CONSTRAINT CKContactsPhoneNumber
 CHECK(dbo.fnIsPhoneNumber2(PhoneNumber) = CAST(1 AS BIT))
);
-- Допустима:
INSERT Test.Contacts (Name, PhoneNumber)
 VALUES ('Ann', '(425)555-1111');
INSERT Test.Contacts (Name, PhoneNumber)
 VALUES ('Chris', NULL);

-- Не допустима из-за неверного кода региона:
INSERT Test.Contacts (Name, PhoneNumber)
 VALUES ('Tobias', '(111)555-2222');

```

Очень важное замечание, касающееся использования вложенных запросов в проверочных ограничениях — проверочное ограничение проверяется для инструкций `UPDATE` и `INSERT` и не проверяется при удалении строк в таблице, на которую ссылается вложенный запрос. Данные, проверенные ограничениями для инструкций `INSERT` и `UPDATE`, могут быть удалены без генерации сообщения об ошибке. Например, следующая инструкция `DELETE` не сообщает об ошибке.

```
DELETE Test.AreaCodes WHERE AreaCode = '425';
```

Но после выполнения инструкции `DELETE` следующая инструкция `UPDATE` порождает сообщение об ошибке.

```
UPDATE Test.Contacts SET PhoneNumber = PhoneNumber;
```

Подобное поведение крайне нежелательно, т. к. вы можете подумать, что у вас есть такая же защита, как и в случае внешних ключей, защищающих вас также и от инструкции `DELETE`. В SQL Server 2008 (и 2005) вы зачастую можете заменить эту логику, применяя внешний ключ, как описано далее.

**Использование внешнего ключа с вложенным запросом.** Давайте реализуем проверку телефонного номера как комбинацию проверочного ограничения и ограничения внешнего ключа. Вы примените первую версию функции, определенной пользователем (без вложенного запроса), и внешний ключ. Как вы можете использовать внешний ключ? Вы хотите, чтобы он проверял только код региона из таблицы `Test.AreaCodes`, а не весь телефонный номер. Вы сделаете это, применив вычисляемый столбец, который возвращает только код региона из телефонного номера. Вам потребуется сделать пару вещей, чтобы стало возможным создание внешнего ключа, показанного в примере.

Результат выражения в столбце `AreaCode` (Код региона) должен иметь тот же тип данных, что и столбец, на который ссылается внешний ключ, `CHAR(3)`. Вы обеспечите это, вызвав функцию `CAST` в выражении `AreaCode`.

Столбец также должен быть помечен как `PERSISTED`, что означает физическое сохранение результата вычисляемого столбца в строке данных вместо его вычисления в запросе при каждой ссылке на столбец. Результат пересчитывается при каждом обновлении столбца. Одна из причин такого требования состоит в том, что оно влияет на производительность; вы не хотите, чтобы SQL Server выполнял функцию `SUBSTRING` каждый раз, когда нужно проверить внешний ключ.

В следующем сценарии создается новая версия таблицы `Test.Contacts`, включая добавленное ограничение внешнего ключа.

```
CREATE TABLE Test.Contacts (ContactID INT IDENTITY PRIMARY KEY
, Name NVARCHAR(50) NOT NULL
, PhoneNumber VARCHAR(20) NULL
, CONSTRAINT CKContactsPhoneNumber
CHECK(dbo.fnIsPhoneNumber(PhoneNumber) = 1)
```

```
, AreaCode AS CAST(SUBSTRING(PhoneNumber, 2, 3) AS CHAR(3)) PERSISTED
, CONSTRAINT FKContactsAreaCodes
 FOREIGN KEY (AreaCode)
 REFERENCES Test.AreaCodes);
```

Как видите, столбец `AreaCode` (Код региона) в таблице `Contacts` (Контакты) — просто подмножество столбца `PhoneNumber` (Номер телефона).

Что произойдет, если вы вставите в столбец `PhoneNumber` значение `NULL`? Функция `SUBSTRING` вернет `NULL`, и `NULL` принимается внешним ключом и интерпретируется как значение, которое не ссылается на таблицу `AreaCodes`.

**Влияние проверочных ограничений и ограничений внешнего ключа на производительность запроса.** Могут ли проверочные ограничения и ограничения внешнего ключа повысить производительность запроса? Может быть, они просто защищают нас от неверных данных, но в процессе слегка снижают производительность? Ответы на эти вопросы: "Да, могут" и "Нет, не снижают".

Поскольку внешние ключи и проверочные ограничения — это объявляемые правила, оптимизатор может использовать их для создания более эффективных планов запросов. Они, как правило, включают в себя пропуск некоторой части плана запроса, т. к. оптимизатор, например, может увидеть, что благодаря ограничению внешнего ключа нет необходимости выполнять определенную часть плана. В следующем примере программного кода приведен простой образец такого поведения при наличии ограничения внешнего ключа. Рассмотрим две следующие таблицы и внешний ключ `FKOrdersCustomers`.

```
CREATE TABLE Test.Customers (CustomerID INT PRIMARY KEY);

CREATE TABLE Test.Orders (OrderID INT PRIMARY KEY
, CustomerID INT NOT NULL
 CONSTRAINT FKOrdersCustomers
 REFERENCES Test.Customers (CustomerID));
```

Теперь давайте посмотрим, что на самом деле делает SQL Server, когда вы запрашиваете эти таблицы с внешним ключом. Для этого в среде SSMS из меню **Query** (Запрос) выберите **Include Actual Execution Plan** (Включить действующий план выполнения) или нажмите комбинацию клавиш <Ctrl>+<M>.

Следующий запрос вернет все заказы, у которых допустимые ссылки на клиента.

```
SELECT o.* FROM Test.Orders AS o
WHERE EXISTS (SELECT * FROM Test.Customers AS c
 WHERE c.CustomerID = o.CustomerID);
```

План выполнения (execution plan), который SQL Server использует для обработки этого запроса, показан на рис. 3.1. В плане выполнения вы можете увидеть, что к таблице `Test.Customers` нет обращений, единственная таблица, к которой происходит обращение, — таблица `Test.Orders`. Дело в том, что оптимизатор знает,

что в этом запросе нет необходимости выполнять оператор `EXISTS`, потому что ограничение внешнего ключа требует, чтобы все заказы ссылались на существующего клиента, что как раз и проверяется в предложении `WHERE`.

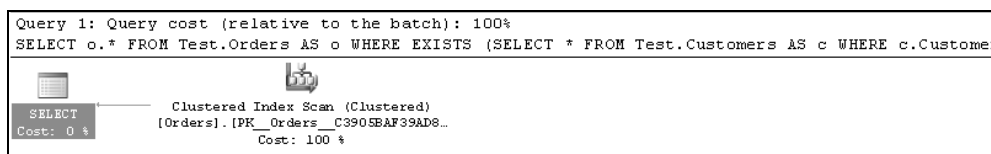


Рис. 3.1. Действующий план выполнения в SSMS

Теперь отключите внешний ключ, выполнив следующую инструкцию.

```
ALTER TABLE Test.Orders NOCHECK CONSTRAINT FKOrdersCustomers;
```

После повторного выполнения того же запроса вы получите новый план выполнения, показанный на рис. 3.2. Оптимизатор выполняет оператор `EXISTS` (в данном случае пиктограмма **Nested Loops** (Вложенные циклы) в плане выполнения) для возврата только тех заказов, у которых действительно есть допустимая ссылка на таблицу `Test.Customers`. Поскольку вы отключили ограничение внешнего ключа, SQL Server не может быть уверен в том, что у всех заказов действительно допустимые ссылки. Следовательно, он вынужден выполнять оператор `EXISTS`. В случае большой таблицы это может привести к громадной разнице в производительности.

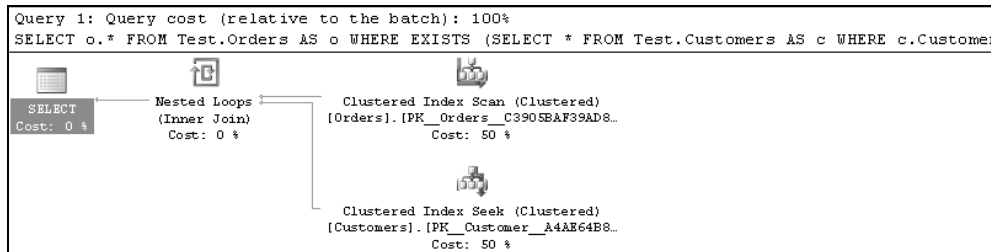


Рис. 3.2. Действующий план выполнения в SSMS при отключенном ограничении внешнего ключа

Теперь сделайте снова активным внешний ключ, выполнив следующую инструкцию, а затем еще раз выполните запрос.

```
ALTER TABLE Test.Orders
 CHECK CONSTRAINT FKOrdersCustomers;
```

После выполнения запроса на этот раз вы получите снова тот же план выполнения, показанный на рис. 3.2. Как же так? Вы вернули ограничение назад, теперь SQL Server должен быть уверен в том, что у всех заказов допустимые ссылки на клиентов. Но на самом деле это не так. Все потому, что внешний ключ не явля-

ется достоверным ("not trusted"). Оптимизатор не принимает во внимание ограничение, которое не является достоверным (это понятие применимо только к внешнему ключу и проверочным ограничениям). Вашему внешнему ключу больше нет доверия, потому что пока он был отключен, кто-то мог вставить или обновить строку заказа с недопустимым CustomerID (ID клиента). Повторная активизация ограничения не проверяет имеющиеся данные. Убедиться в том, что внешний ключ действительно не считается надежным, можно, выполнив следующий запрос.

```
SELECT name, is_not_trusted FROM sys.foreign_keys
WHERE name = 'FKOrdersCustomers';
```

Вы обнаружите, что столбец `is_not_trusted` содержит значение 1, обозначающее, что ограничение не является достоверным. Для того чтобы сделать его достоверным, вам необходимо изменить приведенную ранее инструкцию, активизирующую ограничение, добавив параметр `WITH CHECK`, как показано далее.

```
ALTER TABLE Test.Orders
 WITH CHECK
 CHECK CONSTRAINT FKOrdersCustomers;
```

Этот параметр заставляет SQL Server проверить все строки в таблице на соответствие ограничению перед тем, как включить его снова. Если какие-либо строки нарушают ограничение, выводится сообщение об ошибке и выполняется откат инструкции `ALTER TABLE`.

Если вы выполните запрос еще раз, то обнаружите, что вернулись к первому плану выполнения (см. рис. 3.1), и если вы снова выполните запрос к представлению каталога `sys.foreign_keys`, то увидите, что столбец `is_not_trusted` возвращает значение 0. Ограничение снова стало достоверным.

Еще одно, последнее замечание по поводу данной реализации: вы можете, как показано в следующем примере, изменить сценарий для таблиц `Test.Customers` (Клиенты) и `Test.Orders` (Заказы) так, чтобы в столбце `CustomerID` (ID клиента) таблицы `Test.Orders` были разрешены значения `NULL`, т. е. не объявлять этот столбец с ограничением `NOT NULL`.

```
CREATE TABLE Test.Customers (CustomerID INT PRIMARY KEY);
```

```
CREATE TABLE Test.Orders (OrderID INT PRIMARY KEY
 , CustomerID INT NULL
 CONSTRAINT FKOrdersCustomers
 REFERENCES Test.Customers (CustomerID));
```

Если вы выполните тот же запрос к данной структуре таблиц, то получите план выполнения, показанный на рис. 3.2. Это означает, что оператор `EXISTS` будет выполняться, даже если достоверное ограничение внешнего ключа на месте. Для того чтобы заставить оптимизатор вернуться к плану выполнения, в котором не



выполняется оператор EXISTS, необходимо изменить запрос, как показано в следующем примере.

```
SELECT o.* FROM Test.Orders AS o
WHERE EXISTS (SELECT * FROM Test.Customers AS c
 WHERE c.CustomerID = o.CustomerID)
AND o.CustomerID IS NOT NULL;
```

Этот запрос сообщает SQL Server о том, что не следует извлекать заказы с полем CustomerID, равным NULL, что вернет вас к плану, показанному на рис. 3.1.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Реализация ограничений

На этом практическом занятии вы добавите ограничения во время создания новой таблицы. Вы также модифицируете существующую таблицу, вставив в нее ограничения. Поскольку упражнения образуют логическую последовательность, важно выполнять их в указанном порядке.

### Упражнение 1. Создание новой таблицы с ограничениями

В этом упражнении вы создадите две таблицы, которые будут использоваться для хранения списка клиентов и их соответствующих заказов. Вы также определите для обеих таблиц базовые ограничения.

1. Если необходимо, откройте среду SSMS и установите соединение с экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных TestDB со схемой Test.

```
CREATE DATABASE TestDB;
GO
```

```
USE TestDB;
GO
```

```
CREATE SCHEMA Test;
GO
```

3. В окне запроса создайте новую таблицу со следующими свойствами.

- Она должна называться Customers (Клиенты).
- Она должна располагаться в схеме Test.
- У нее должны быть следующие столбцы.
  - CustomerId (Id клиента) — целое число в диапазоне от 1 до 100 000. У этого столбца значения должны увеличиваться автоматически. Следует применить тип данных, использующий минимальный необходимый объем памяти. Первое значение должно быть равно 1000, а затем

каждая последующая строка должна получать новое значение, увеличенное на 1. Это значение должно быть первичным ключом таблицы.

- Name (Имя) — строка, длиной до 50 символов.

- Ни в одном из столбцов не допустимы значения NULL.

Корректная инструкция приведена далее, но не смотрите на нее, пока не попробуете создать таблицу самостоятельно.

4. Введите, выделите и выполните следующую инструкцию.

```
CREATE TABLE Test.Customers (
 CustomerId INT IDENTITY(1000, 1) NOT NULL
 CONSTRAINT PKCustomers
 PRIMARY KEY
 , Name NVARCHAR(50) NOT NULL);
```

5. В окне запроса создайте новую таблицу со следующими свойствами.

- Она должна называться Orders (Заказы).
- Она должна находиться в схеме Test.
- У нее должны быть следующие столбцы.
  - OrderId (Id заказа) — целое число в диапазоне от 1 до 100 000. У этого столбца значения должны увеличиваться автоматически. Следует применить тип данных, использующий минимальный необходимый объем памяти. Первое значение должно быть равно 1000, а затем каждая последующая строка должна получать новое значение, увеличенное на 1. Это значение должно быть первичным ключом таблицы.
  - OrderDate (Дата заказа) — дата. Если значение не задано, автоматически должна быть добавлена текущая дата.
  - CustomerId должен ссылаться на строку с тем же значением в столбце CustomerId таблицы Test.Customers (Клиенты).
- Ни в одном из столбцов не допустимы значения NULL.

Корректная инструкция приведена далее, но не смотрите на нее, пока не попробуете создать таблицу самостоятельно.

6. Введите, выделите и выполните следующую инструкцию.

```
CREATE TABLE Test.Orders (OrderId INT IDENTITY(1000, 1) NOT NULL
 , OrderDate DATE NOT NULL DEFAULT SYSDATETIME()
 , CustomerId INT NOT NULL
 CONSTRAINT FKOrdersCustomerId
 REFERENCES Test.Customers (CustomerId));
```

## Упражнение 2. Вставка дополнительных ограничений

В этом упражнении вы вставите дополнительные ограничения в таблицы, созданные вами в упражнении 1.

1. Если необходимо, откройте среду SSMS и установите соединение с соответствующим экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для использования базы данных TestDB.

```
USE TestDB;
GO
```

3. В окне запроса добавьте ограничение, которое проверяет, начинается ли с буквы столбец Name в таблице Test.Customers и равна ли его длина как минимум трем символам.

Корректная инструкция приведена далее, но не смотрите на нее, пока не попробуете создать ограничение самостоятельно.

Введите, выделите и выполните следующую инструкцию.

```
ALTER TABLE Test.Customers
 ADD CONSTRAINT CKCustomerName
 CHECK(Name LIKE N'[A-Z]__%');
```

4. В окне запроса добавьте ограничение, которое проверяет, содержит ли текущую дату столбец OrderDate в таблице Test.Orders.

Корректная инструкция приведена далее, но не смотрите на нее, пока не попробуете создать ограничение самостоятельно.

Введите, выделите и выполните следующую инструкцию.

```
ALTER TABLE Test.Orders
 ADD CONSTRAINT CKOrdersOrderDate
 CHECK(OrderDate = CAST(SYSDATETIME() AS DATE));
```

5. Для очистки после этого занятия закройте все открытые в среде SSMS окна запросов, откройте окно создания запроса и выполните следующие инструкции SQL.

```
USE master;
GO
```

```
DROP DATABASE TestDB;
```

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить знания и навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Просмотреть резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.

- ◆ Найти способ решения проблемы в предлагаемом конкретном сценарии. Этот сценарий описывает реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ Всегда продумывайте, какие типы данных применять, поскольку изменение вашего решения в дальнейшем может оказаться гораздо более трудной задачей, чем вам кажется.
- ◆ Учитывайте возможность использования типов данных, определенных пользователем, для упрощения выбора корректного типа данных при создании таблиц и исключения несоответствий типов данных в вашей базе данных.
- ◆ Наличие подходящих имен, определенных в соответствии с документом, содержащим рекомендации по именованию объектов и столбцов, очень важно, т. к. гарантирует согласованность имен в вашей базе данных.
- ◆ Рассматривайте возможность сжатия больших таблиц для сохранения дискового пространства и памяти, а также возможного повышения производительности.
- ◆ Вставляйте ограничения для проверки целостности данных.
- ◆ Реализуйте ограничения для поддержки оптимизатора запросов.
- ◆ Для обеспечения расширенной целостности данных учитывайте возможность применения в ограничениях функций, определенных пользователем.

## Ключевые термины

- ◆ Ограничение (Constraint).
- ◆ Первичный ключ (Primary key).
- ◆ Ограничение `UNIQUE`.
- ◆ Ограничение внешнего ключа (Foreign key constraint).
- ◆ Проверочное ограничение (Check constraint).
- ◆ Идентификация (Identity).

## Сценарий

В следующем конкретном сценарии вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в *разд. "Ответы к сценариям"* в конце книги.

## Сценарий. Ограничения и типы данных

Вы — разработчик базы данных для компании Contoso Corporation. Вам даны полномочия на добавление проверочных ограничений и ограничений внешнего ключа в таблицу `Products` (Товары), которая содержит большой объем некорректных данных. Вам нужно добавить ограничения в таблицу, чтобы помешать появлению новых некорректных данных в результате вставок и обновлений.

Вам также необходимо создать новую таблицу с именем `ProductLog` (Регистрация товаров), в которой будет использоваться в качестве первичного ключа идентифицирующий столбец и будет содержаться огромное количество строк. По этой причине вы должны выбрать для идентифицирующего столбца самый большой тип данных, который поддерживается свойством `IDENTITY`.

Ответьте на следующие вопросы вашего менеджера:

1. "Как следует вставить ограничения в таблицу `Products`?"
2. "Какой тип данных вам следует применить в таблице `ProductLog`?"

## КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

### Создание и изменение таблиц

**Упражнение 1.** Создайте таблицы, которые должны хранить информацию о вашей коллекции DVD-дисков.

### Реализация типов данных

**Упражнение 2.** Создайте все альтернативные типы (типы данных, определенные пользователем), необходимые столбцам, которые вы определили в упражнении 1.

### Управление интернациональными настройками

**Упражнение 3.** Измените таблицы, определенные в упражнении 1, для того, чтобы использовать в них альтернативные типы, которые вы создали в упражнении 2.

### Создание и изменение ограничений

**Упражнение 4.** Обеспечьте ссылочную целостность таблиц, определенных вами в упражнении 1, применяя ограничения первичного и внешнего ключей.

**Упражнение 5.** Реализуйте проверочные ограничения и ограничения уникальности, которые по вашему мнению подходят для проверки целостности данных в таблицах, определенных в упражнении 1.

## Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобия будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### ***Дополнительная информация о тренировочных тестах***

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.

## ГЛАВА 4

# Сложные запросы

Во многих приложениях вам необходимо конструировать запросы, которые не исчерпываются базовой инструкцией `SELECT`, например, создание итогов с накоплением, поиск пропусков в последовательностях, отслеживание рекурсивной иерархии или ранжирование данных в наборе. В этой главе вы научитесь дополнительным приемам формирования запросов с использованием вложенных запросов, обобщенных табличных выражений (ОТВ) и ранжирующих функций.

### Экзаменационные темы в этой главе

- ◆ Выполнение вложенных запросов.
- ◆ Реализация запросов с обобщенными табличными выражениями (ОТВ).
- ◆ Применение ранжирующих функций.

### Уроки в этой главе

- ◆ Урок 1. Построение рекурсивных запросов с ОТВ.
- ◆ Урок 2. Реализация вложенных запросов.
- ◆ Урок 3. Применение ранжирующих функций.

## Прежде чем начать

Для усвоения уроков этой главы необходимо иметь:

- ◆ установленный SQL Server 2008;
- ◆ установленную базу данных AdventureWorks.

### **РЕАЛЬНАЯ ИСТОРИЯ.**

**Майк Хотек (Mike Hotek)**

---

Одна из самых распространенных потребностей, с которыми я сталкивался, особенно в хранилищах данных, — поиск места потери данных. Возможно, вы ждете из ма-

газина данные о ежедневных продажах, ежечасные сведения о дефектах на производственной линии или информацию об уровнях запасов в серии последовательно пронумерованных складских секций хранения. Пытаясь решать проблемы брешей в данных, многие разработчики обращаются к операциям с курсором. Для отслеживания предыдущих и последующих значений и поиска всех потерянных значений либо применяется курсор базы данных, либо данные возвращаются в приложение, которое их сортирует и затем перемещается по всем записям, отслеживая предыдущие и последующие значения для поиска пропущенных значений.

Заслуживающая особого внимания, знакомая мне проблема — обнаружение пропусков в последовательности. Несколько лет назад мне пришлось решать такую проблему для клиента, пытавшегося автоматически помещать рекламные объявления во временной последовательности, многократно находя пропуски или разрывы, соответствующие набору критериев. Существующее решение допускало множество проб и ошибок, связанных с вложенными курсорами. Применение вложенных запросов и соединений при условии неравенства решило проблему с 90-процентным сокращением времени и используемых ресурсов.

Мой коллега прислал мне сообщение по электронной почте с решением задачи вычисления корреляции между установкой рекламных щитов и интерактивной популяризацией товаров и услуг в социальных сетях, в котором использовались множественные циклы `WHILE` и табличные переменные. Несмотря на то, что программный код работал и формировал требуемый результат, его можно было сжать, превратив в набор несвязанных вложенных запросов с соединениями при условии неравенства, действующих гораздо эффективнее в случае большого результирующего набора. (Клинт, я надеюсь, что ты счел мое решение полезным.)

Даже когда я писал эту главу, другому клиенту понадобилась проверка корректности наборов данных в хранилище данных для поиска информации, исчезающей при загрузках.

Несмотря на то, что решения с последовательным перебором строк работают, поиск пропусков или разрывов в большом наборе данных может потребовать значительно времени. Вместо применения курсоров или множественных циклов `WHILE` вы можете очень эффективно решить проблему, либо применив вложенные запросы с соединениями при условии неравенства, либо скомбинировав ранжирующие функции с `OTB`.

## УРОК 1. Построение рекурсивных запросов с `OTB`

`OTB` (обобщенные табличные выражения) по функциональным возможностям аналогичны производным таблицам (производные таблицы обсуждаются в *уроке 2 данной главы*). Но `OTB` позволяют многократно обрабатывать результирующий набор, эффективно решая одну из наиболее трудных проблем в языке Transact-SQL (T-SQL) — выполнение рекурсивного запроса. Из этого урока вы узнаете, как формировать `OTB` для решения задач с рекурсивными запросами.

---

### ***После усвоения этого урока вы сможете***

Применять `OTB` в задачах с рекурсивными запросами.

Расчетное время урока — 20 мин.



## Обобщенные табличные выражения

ОТВ определяется с помощью двух компонентов:

- ◆ предложения WITH, содержащего инструкцию SELECT, которая генерирует корректную таблицу;
- ◆ внешней инструкции SELECT, которая ссылается на табличное выражение.

Следующее ОТВ вернет количество сотрудников, имеющих заданную должность.

```
WITH EmpTitle AS
(SELECT JobTitle, count(*) numtitles
 FROM HumanResources.Employee
 GROUP BY JobTitle)
SELECT b.BusinessEntityID, b.JobTitle, a.numtitles
FROM EmpTitle a INNER JOIN HumanResources.Employee b ON
 a.JobTitle = b.JobTitle;

GO
```

*Рекурсивное ОТВ* расширяет определение табличного выражения и состоит из двух компонентов:

- ◆ *опорного запроса* (anchor query), который задает начало рекурсии, наряду с инструкцией UNION ALL и вторым запросом, выполняющим рекурсию в опорном запросе;
- ◆ *внешнего запроса*, ссылающегося на подпрограмму и задающего максимальный уровень или глубину рекурсии.

Например, следующий запрос позволяет задать сотрудника и возвращает все уровни руководства сотрудником в организации вплоть до 25 уровней руководства организации.

```
DECLARE @EmployeeToGetOrgFor INT = 126;

WITH EMP_cte(BusinessEntityID, OrganizationNode, FirstName, LastName
 , JobTitle, RecursionLevel)
AS (SELECT e.BusinessEntityID, e.OrganizationNode, p.FirstName
 , p.LastName, e.JobTitle, 0
 FROM HumanResources.Employee e INNER JOIN Person.Person as p
 ON p.BusinessEntityID = e.BusinessEntityID
 WHERE e.BusinessEntityID = @EmployeeToGetOrgFor
 UNION ALL
 SELECT e.BusinessEntityID, e.OrganizationNode, p.FirstName
 , p.LastName, e.JobTitle, RecursionLevel + 1
 FROM HumanResources.Employee e INNER JOIN EMP_cte
 ON e.OrganizationNode = EMP_cte.OrganizationNode.GetAncestor(1)
 INNER JOIN Person.Person p ON
 p.BusinessEntityID = e.BusinessEntityID)
```

```
SELECT EMP_cte.RecursionLevel, EMP_cte.BusinessEntityID
 , EMP_cte.FirstName, EMP_cte.LastName
 , EMP_cte.OrganizationNode.ToString() AS OrganizationNode
 , p.FirstName AS 'ManagerFirstName', p.LastName AS 'ManagerLastName'
FROM EMP_cte INNER JOIN HumanResources.Employee e
 ON EMP_cte.OrganizationNode.GetAncestor(1) = e.OrganizationNode
 INNER JOIN Person.Person p ON p.BusinessEntityID = e.BusinessEntityID
ORDER BY RecursionLevel, EMP_cte.OrganizationNode.ToString()
OPTION (MAXRECURSION 25);
```

Первый запрос в предложении `WITH` определяет опорный результирующий набор. Второй запрос выполняется рекурсивно по отношению к опорному запросу вплоть до максимального уровня рекурсии. Рекурсия завершается внутренним соединением с ОТВ следующим образом:

```
INNER JOIN EMP_cte
```

Затем для извлечения рекурсивной операции вместе с любыми необходимыми дополнительными данными применяется внешний запрос. Предложение `OPTION` во внешнем запросе задает максимальное число допустимых уровней рекурсии.

---

### **ПРЕДУПРЕЖДЕНИЕ.**

#### **Уровни рекурсии**

Если итеративный запрос не достигает глубины рекурсии в течение определенного времени, значение `MAXRECURSION` исчерпывается, и вы получаете сообщение об ошибке.

---

### **Подсказка к экзамену**

Поскольку ключевое слово `WITH` в языке T-SQL используется по-разному, любые инструкции, начинающиеся с ключевого слова `WITH`, должны завершаться точкой с запятой.

---

### **Быстрая проверка**

#### **ВОПРОСЫ:**

1. Какие два компонента есть в ОТВ?
2. Из каких двух частей состоит рекурсивное ОТВ?

#### **ОТВЕТЫ:**

1. У ОТВ есть предложение `WITH`, содержащее инструкцию `SELECT` с определением таблицы, и внешняя инструкция `SELECT`, ссылающаяся на ОТВ.
2. Рекурсивное ОТВ включает в себя опорный запрос, задающий начальное состояние рекурсии и содержащий также оператор `UNION ALL`, второй запрос, который выполняет рекурсию в первом запросе, и внешний запрос, ссылающийся на ОТВ и задающий глубину рекурсии или максимальное количество уровней рекурсии.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

### Создание рекурсивного ОТВ

На этом практическом занятии вы примените рекурсивное ОТВ для пополнения спецификации материалов для компонента в базе данных AdventureWorks.

#### Упражнение. Создание рекурсивного ОТВ

В этом упражнении вы воспользуетесь рекурсивным ОТВ для пополнения спецификации материалов для компонента в базе данных AdventureWorks.

1. Откройте окно создания запроса, введите и выполните следующий запрос.

```
DECLARE @date date = '4/18/2000',
 @productassembly int = 749;

WITH BOM(ProductAssemblyID, ComponentID, AssemblyDescription
 , PerAssemblyQty, ComponentCost, ListPrice, BOMLevel
 , RecursionLevel)
AS (SELECT b.ProductAssemblyID, b.ComponentID, p.Name
 , b.PerAssemblyQty, p.StandardCost, p.ListPrice
 , b.BOMLevel, 0
 FROM Production.BillofMaterials b INNER JOIN Production.Product p
 ON b.ComponentID = p.ProductID
 WHERE b.ProductAssemblyID = @productassembly
 AND @date >= b.StartDate
 AND @date <= ISNULL(b.EndDate, @date)
 UNION ALL
 SELECT b.ProductAssemblyID, b.ComponentID, p.Name
 , b.PerAssemblyQty, p.StandardCost, p.ListPrice
 , b.BOMLevel, RecursionLevel + 1
 FROM BOM cte INNER JOIN Production.BillofMaterials b
 ON b.ProductAssemblyID = cte.ComponentID
 INNER JOIN Production.Product p ON b.ComponentID = p.ProductID
 WHERE @date >= b.StartDate AND @date <= ISNULL(b.EndDate, @date))
SELECT b.ProductAssemblyID, b.ComponentID, b.AssemblyDescription
 , SUM(b.PerAssemblyQty) AS ComponentQty , b.ComponentCost
 , b.ListPrice, b.BOMLevel, b.RecursionLevel
FROM BOM b
GROUP BY b.ComponentID, b.AssemblyDescription, b.ProductAssemblyID
 , b.BOMLevel, b.RecursionLevel, b.ComponentCost, b.ListPrice
ORDER BY b.BOMLevel, b.ProductAssemblyID, b.ComponentID
OPTION (MAXRECURSION 25)
GO
```

2. Повторно выполните запрос, изменив дату на 10/16/2000 (16 октября 2000 г.), и просмотрите результаты.

3. Изучите содержимое таблицы `Production.BillOfMaterials` (Спецификация материалов) и, чтобы понять, почему два запроса возвращают такие разные результаты, проследите, как изменятся компоненты со временем.

## Резюме урока

- ◆ Рекурсивное ОТВ содержит в предложении `WITH` две инструкции `SELECT`, разделенные оператором `UNION ALL`. Первый запрос определяет начальное состояние для рекурсии, а второй задает набор данных, который обрабатывается итеративно.
- ◆ Если ОТВ содержится в пакете, все инструкции, начинающиеся с предложения `WITH`, должны заканчиваться точкой с запятой.
- ◆ Внешний запрос обращается к ОТВ и задает глубину рекурсии.

## УРОК 2. Реализация вложенных запросов

Вложенные запросы позволяют вкладывать один запрос в другой для построения сложных подпрограмм, а также извлекать наборы данных, которые было бы невозможно сформировать, не прибегая к многошаговому процессу записи промежуточных результатов во временные внешние объекты.

Вы можете создавать запросы с вложенными запросами двух типов: *связанными* и *несвязанными*. Несвязанный вложенный запрос не зависит от внешнего запроса, в котором он содержится. Связанный вложенный запрос зависит от столбцов внешнего запроса и ссылается на них.

Вложенные запросы любого типа могут возвращать скалярные и множественные значения. Вложенные запросы со скалярными значениями можно вставлять в любое место инструкции `SELECT`, допускающее одно значение или его отсутствие. Вложенные запросы с множественными значениями могут применяться там, где ожидается множество значений.

---

### ***После усвоения этого урока вы сможете***

- Создавать связанные вложенные запросы.
- Формировать несвязанные вложенные запросы.

Расчетное время урока — 20 мин.

## Несвязанные вложенные запросы

Основное назначение несвязанного вложенного запроса — дать вам возможность написать более динамичный программный код, не требующий от пользователя знания всех промежуточных значений, существующих в базе данных в текущий момент. Например, если вы хотите получить список клиентов, припи-

санных к конкретному региону, вам сначала нужно узнать список городов или штатов, из которых следует извлекать требуемый список. Несмотря на то, что в вашей базе данных существует таблица, определяющая, какие города и штаты приписаны к заданному региону, можно применить несвязанный вложенный запрос, чтобы сделать ваш запрос гибким в отношении изменений способа организации региона. Далее приведен пример того, как он может действовать.

```
SELECT a.CustomerID, a.FirstName, a.LastName, b.Address, b.City
 , b.StateProvince
FROM Customer.Customer a INNER JOIN Customer.CustomerAddress b
 ON a.CustomerID = b.CustomerID
WHERE b.City IN (SELECT c.City FROM Customer.CityRegion c INNER JOIN
 Customer.Region d
 ON c.RegionID = d.RegionID
 WHERE d.Region = 'RegionX')
```

В качестве еще одного примера предположим, что вы хотите узнать все товары с заявленной ценой большей, чем средняя заявленная цена всех товаров. Вместо отдельного извлечения средней заявленной цены, сохранения ее значения в переменной и затем использования переменной во второй инструкции `SELECT`, вы можете применить следующий запрос.

```
SELECT a.ProductID, a.Name, a.ListPrice
FROM Production.Product a
WHERE a.ListPrice > (SELECT AVG(b.ListPrice) FROM Production.Product b)
```

### ***Производные таблицы***

Если вы хотите получить список сотрудников и количество сотрудников с одинаковой занимаемой должностью, можно попытаться выполнить следующий запрос.

```
SELECT BusinessEntityID, JobTitle, count(*)
FROM HumanResources.Employee
GROUP BY BusinessEntityID, JobTitle
```

Но у вас возникнет проблема. Вам нужно сосчитать количество сотрудников с заданной должностью, а затем вернуть список сотрудников с информацией о том, сколько еще сотрудников имеют ту же должность. Однако для соответствия требованиям предложения `GROUP BY` в него следует включить все столбцы без итогов. Вы сталкиваетесь с дилеммой, потому что, как оказывается, ваш запрос не может быть удовлетворен.

Проблема в вашем запросе — порядок операций. Сначала нужно вычислить количество людей с заданной должностью. Далее, присоединить полученный результат к таблице `Employee` (Сотрудник) и получить список сотрудников вместе с количеством людей, занимающих одинаковые должности.

Вместо использования временных таблиц для хранения промежуточного результирующего набора можно решить проблему средствами языка T-SQL, воспользовавшись интересным свойством предложения `FROM`, оно может принимать в качестве источника данных таблицу. Таблица конструируется из строк и столбцов. Когда вы выполняете инструкцию `SELECT`, то получаете результирующий набор, состоящий из строк и столбцов. Следовательно, представляется возможным поместить целиком

инструкцию `SELECT` в предложение `FROM`, т. к. к источнику данных предъявляется единственное требование — наличие табличной структуры.

Когда вы встраиваете инструкцию `SELECT` в предложение `FROM`, то применяете средство, называемое *производными* (derived) или *виртуальными* (virtual) *таблицами*. Инструкция `SELECT` возвращает результирующий набор, но у него нет имени, на которое можно было бы сослаться в запросе. С отсутствием имени можно справиться, заключив всю инструкцию `SELECT` в круглые скобки и указав для нее псевдоним. Решение вашей первоначальной задачи будет выглядеть следующим образом.

```
SELECT b.BusinessEntityID, b.JobTitle, a.numtitles
FROM (SELECT JobTitle, count(*) numtitles
 FROM HumanResources.Employee
 GROUP BY JobTitle) a
INNER JOIN HumanResources.Employee b ON a.JobTitle = b.JobTitle
```

SQL Server сначала выполнит инструкцию `SELECT...GROUP BY`, загрузит результаты в память и присвоит результатам заданный псевдоним. После этого вы сможете ссылаться на любой столбец производной таблицы в оставшейся части инструкции `SELECT`, как если бы вы работали с обычной таблицей. Помните о том, что любому итогу, сцеплению или вычислению в производной таблице следует присваивать псевдоним, потому что невозможно создать таблицу из столбцов, не имеющих имен.

Главное достоинство производных таблиц — размещение в памяти всего результирующего набора, что ускоряет доступ к данным, по сравнению с вариантом хранения результата на внешнем запоминающем устройстве. В большинстве случаев в любой подпрограмме можно применить производную таблицу вместо использования временной таблицы для хранения промежуточного результирующего набора, обрабатываемого в последующем запросе.

## Итоги с накоплением

Комбинируя производные таблицы с соединениями при условии неравенства, вы можете вычислять разнообразные итоги с накоплением. Следующий запрос возвращает итог с накоплением объемов заказов у каждого продавца.

```
SELECT SH3.SalesPersonID, SH3.OrderDate, SH3.DailyTotal, SUM(SH4.DailyTotal)
RunningTotal
FROM (SELECT SH1.SalesPersonID, SH1.OrderDate, SUM(SH1.TotalDue)
 DailyTotal
 FROM Sales.SalesOrderHeader SH1
 WHERE SH1.SalesPersonID IS NOT NULL
 GROUP BY SH1.SalesPersonID, SH1.OrderDate) SH3
INNER JOIN (SELECT SH2.SalesPersonID, SH2.OrderDate, SUM(SH2.TotalDue)
 DailyTotal
 FROM Sales.SalesOrderHeader SH2
 WHERE SH2.SalesPersonID IS NOT NULL
 GROUP BY SH2.SalesPersonID, SH2.OrderDate) SH4
ON SH3.SalesPersonID = SH4.SalesPersonID
AND SH3.OrderDate >= SH4.OrderDate
GROUP BY SH3.SalesPersonID, SH3.OrderDate, SH3.DailyTotal
ORDER BY SH3.SalesPersonID, SH3.OrderDate
```

Производные таблицы применяются для объединения всех заказов для продавцов, у которых продано несколько заказов за один день. Соединение по полю `SalesPersonID` (ID продавца) гарантирует накопление строк для одного продавца. Соединение при условии неравенства позволяет при подведении итога учитывать только строки, относящиеся к продавцу и имеющие более раннюю дату заказа, чем дата заказа, рассматриваемая в данный момент в результирующем наборе.

Добавив предложение `HAVING`, можно вычислять итоги с накоплением для множества различных ситуаций, например, отобразить итоги с накоплением по продажам у продавцов, выполнивших установленную квоту, или подводить итоги по продажам методом скользящего окна (sliding window).

## Связанные вложенные запросы

В связанном вложенном запросе внутренний запрос зависит от значений из внешнего запроса. Это заставляет внутренний запрос выполняться повторно, основываясь на входных значениях из внешнего запроса. Следующий запрос возвращает для всех проданных товаров их названия и заявленные цены.

```
SELECT a.ProductID, a.ListPrice
FROM Production.Product a
WHERE EXISTS (SELECT 1 FROM Sales.SalesOrderDetail b
 WHERE b.ProductID = a.ProductID)
```

Применение соединения или любого другого предложения `WHERE` будет возвращать товар при каждой продаже; при этом будет уточняться, сколько раз продавался товар вместо выделения товаров, проданных хотя бы однажды. Аргумент `EXISTS` также повышает производительность запроса, потому что SQL Server должен найти в таблице только одно вхождение, для которого предложение `WHERE` равно истине. Как только такое значение найдено, SQL Server завершает просмотр оставшихся строк, т. к., начиная с этой точки и далее, возвращаемое значение уже не будет меняться.

---

### Подсказка к экзамену

На экзамене вам нужно будет прочесть запрос и, основываясь только на инструкции `SELECT`, определить, можно ли использовать запрос для решения представленной производственной проблемы.

---

### Быстрая проверка

#### ВОПРОСЫ:

1. Какова разница между связанным и несвязанным вложенными запросами?
2. Что такое производная таблица?

#### ОТВЕТЫ:

1. Несвязанный вложенный запрос — это запрос, который встроен внутрь другого запроса, но не ссылается на какие-либо столбцы из внешнего запроса. Связан-

ный вложенный запрос встроен в другой запрос и ссылается на столбцы из внешнего запроса.

2. Производная таблица — это инструкция `SELECT`, встроенная в предложение `FROM`.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Поиск разрывов в последовательности

На этом практическом занятии вы используете вложенные запросы для поиска разрывов или пропусков в последовательности.

### Упражнение. Поиск разрывов последовательности

В следующем упражнении вы воспользуетесь вложенными запросами для поиска разрывов в последовательности.

1. Откройте окно создания запроса, введите и выполните следующий программный код для загрузки таблицы с тестовыми данными.

```
CREATE TABLE #orderdates
(CustomerID INT NOT NULL,
 OrderDate DATETIME NOT NULL);

DECLARE @startDate datetime,
 @endDate datetime,
 @CustomerID int

SET @CustomerID = 1
WHILE @CustomerID < 100
BEGIN
 SELECT @startDate = 'Jan 01, 2007'
 , @endDate = DATEADD(yy, 1, @startDate);

 WITH dates AS
 (SELECT @startDate AS begindate
 UNION ALL
 SELECT DATEADD(dd,1,begindate) FROM dates
 WHERE begindate < dateadd(dd,-1,@endDate))

 INSERT INTO #orderdates
 SELECT @CustomerID, begindate FROM dates
 OPTION (MAXRECURSION 0);

 SET @CustomerID = @CustomerID + 1
END

SELECT * FROM #orderdates;
```



2. В открытом окне запроса введите, выделите и выполните следующий программный код, создающий разрывы в последовательности.

```
--Теперь удалите несколько строк для создания разрывов
DELETE #orderdates
WHERE DATEDIFF(dd, 0, OrderDate)%11 = 0 ;
```

```
--Создайте пару многодневных разрывов
DELETE FROM #orderdates
WHERE OrderDate IN ('1/4/2007','2/17/2007')
 AND CustomerID%3 = 0
```

```
SELECT * FROM #orderdates;
```

3. В имеющемся окне запроса для получения списка всех разрывов в последовательности введите, выделите и выполните следующий программный код.

```
SELECT CustomerID, StartGap, EndGap
 , DATEDIFF(dd,StartGap,EndGap) + 1 AS NumberMissingDays
FROM (SELECT t1.CustomerID, t1.OrderDate AS StartGap
 , MIN(t2.OrderDate) AS EndGap
 FROM (SELECT CustomerID, DATEADD(dd, 1, OrderDate) AS OrderDate
 FROM #orderdates tbl1
 WHERE NOT EXISTS(SELECT * FROM #orderdates tbl2
 WHERE DATEDIFF(dd, tbl1.OrderDate
 , tbl2.OrderDate) = 1
 AND tbl1.CustomerID = tbl2.CustomerID)
 AND OrderDate <> (SELECT MAX(OrderDate)
 FROM #orderdates)) t1
 INNER JOIN (SELECT CustomerID
 , DATEADD(dd, -1, OrderDate) AS OrderDate
 FROM #orderdates tbl1
 WHERE NOT EXISTS(SELECT * FROM #orderdates tbl2
 WHERE DATEDIFF(dd, tbl2.OrderDate, tbl1.OrderDate) = 1
 AND tbl1.CustomerID = tbl2.CustomerID)
 AND OrderDate <> (SELECT MIN(OrderDate) FROM #orderdates)) t2
 ON t1.OrderDate <= t2.OrderDate AND t1.CustomerID = t2.CustomerID
GROUP BY t1.CustomerID, t1.OrderDate) a
ORDER BY CustomerID, StartGap;
```

## Резюме урока

- ◆ Несвязанные вложенные запросы — это независимые запросы, встроенные во внешний запрос и применяемые для извлечения скалярного значения или списка значений, которые могут использоваться внешним запросом для того, чтобы сделать программный код более динамичным?
- ◆ Связанные вложенные запросы — это запросы, которые встроены во внешний запрос, но ссылаются на значения во внешнем запросе.

## УРОК 3. Применение ранжирующих функций

Ранжирующие функции применяются для проведения простого анализа, например, статистического упорядочивания или разбиения на сегменты. Из этого урока вы узнаете, как использовать четыре ранжирующие функции, включенные в состав версии SQL Server 2008.

---

### ***После усвоения этого урока вы сможете***

Использовать в ваших запросах ранжирующие функции.

Расчетное время урока — 20 мин.

## Ранжирование данных

В языке T-SQL есть четыре функции, которые можно применять для ранжирования данных: ROW\_NUMBER, RANK, DENSE\_RANK и NTILE.

Функция ROW\_NUMBER присваивает строкам номера от 1 до  $n$  на основе заданного пользователем порядка сортировки. ROW\_NUMBER не учитывает совпадающие или дублирующиеся записи (ties) в результирующем наборе, поэтому, если у вас есть строки с одинаковыми значениями в столбцах, по которым сортируются данные, повторные обращения к базе данных для получения одного и того же результирующего набора могут формировать разную нумерацию строк.

Следующий пример возвращает данные о продавце с его или ее продажами с начала года до текущей даты, пронумерованные в порядке убывания в соответствии с объемом продаж с начала года до текущей даты.

```
SELECT p.FirstName, p.LastName, ROW_NUMBER()
 OVER(ORDER BY s.SalesYTD DESC) AS 'RowNumber', s.SalesYTD, a.PostalCode
FROM Sales.SalesPerson s INNER JOIN Person.Person p
 ON s.BusinessEntityID = p.BusinessEntityID
INNER JOIN Person.BusinessEntityAddress ba
 ON p.BusinessEntityID = ba.BusinessEntityID
INNER JOIN Person.Address a
 ON a.AddressID = ba.AddressID
WHERE s.TerritoryID IS NOT NULL
```

Функцию ROW\_NUMBER можно использовать с итогами, предоставляя последовательные номера в каждой группе строк. Номер в группе формируется с помощью необязательного предложения PARTITION BY следующим образом.

```
SELECT p.FirstName, p.LastName, ROW_NUMBER()
 OVER (PARTITION BY s.TerritoryID ORDER BY SalesYTD DESC)
 AS 'RowNumber', s.SalesYTD, s.TerritoryID
FROM Sales.SalesPerson s
INNER JOIN Person.Person p ON s.BusinessEntityID = p.BusinessEntityID
```

```
INNER JOIN Person.BusinessEntityAddress ba
 ON p.BusinessEntityID = ba.BusinessEntityID
INNER JOIN Person.Address a ON a.AddressID = ba.AddressID
WHERE s.TerritoryID IS NOT NULL
```

Если вам нужно пронумеровать результирующий набор и учесть дублирующиеся или совпадающие записи, можно применить функцию `RANK`. Если в результирующем наборе нет совпадающих записей, функция `RANK` формирует такой же результат, как и функция `ROW_NUMBER`. Если же совпадения есть, `RANK` присваивает одно и то же значение каждой записи с совпадающими значениями и затем пропускает следующее значение, оставляя разрыв в последовательности, соответствующий количеству строк с совпадающими значениями. В следующем примере показано, как применяется функция `RANK` к дубликатам в каждой группе с подведением итога.

```
SELECT a.ProductID, b.Name, a.LocationID, a.Quantity, RANK()
 OVER (PARTITION BY a.LocationID ORDER BY a.Quantity DESC) AS 'Rank'
FROM Production.ProductInventory a
INNER JOIN Production.Product b ON a.ProductID = b.ProductID
ORDER BY b.Name
```

```
SELECT a.ProductID, b.Name, a.LocationID, a.Quantity, RANK()
 OVER (PARTITION BY a.LocationID ORDER BY a.Quantity DESC) AS 'Rank'
FROM Production.ProductInventory a
INNER JOIN Production.Product b ON a.ProductID = b.ProductID
ORDER BY 'Rank'
```

Если вы не хотите иметь разрывы в последовательности, можно воспользоваться функцией `DENSE_RANK`. Она присваивает одно и то же значение всем дубликатам, но не создает пропусков в последовательности. В следующих двух примерах показан один и тот же результирующий набор в случае применения функции `DENSE_RANK`.

```
SELECT a.ProductID, b.Name, a.LocationID, a.Quantity, DENSE_RANK()
 OVER (PARTITION BY a.LocationID ORDER BY a.Quantity DESC) AS 'DenseRank'
FROM Production.ProductInventory a
INNER JOIN Production.Product b ON a.ProductID = b.ProductID
ORDER BY b.Name
```

```
SELECT a.ProductID, b.Name, a.LocationID, a.Quantity, DENSE_RANK()
 OVER (PARTITION BY a.LocationID ORDER BY a.Quantity DESC) AS DenseRank
FROM Production.ProductInventory a
INNER JOIN Production.Product b ON a.ProductID = b.ProductID
ORDER BY DenseRank
```

Функция `NTILE` применяется для разбиения результирующего набора на приблизительно равные группы. Например, если вы хотите разделить результирующий набор на шесть групп с приблизительно равным количеством строк в каждой

группе, можно задать NTILE(6). В следующих двух примерах показано, как можно воспользоваться NTILE для сегментирования результирующего набора.

```
SELECT p.FirstName, p.LastName,
 NTILE(4) OVER(ORDER BY s.SalesYTD DESC) AS QuarterGroup,
 s.SalesYTD, a.PostalCode
FROM Sales.SalesPerson s INNER JOIN Person.Person p
 ON s.BusinessEntityID = p.BusinessEntityID
INNER JOIN Person.BusinessEntityAddress ba
 ON p.BusinessEntityID = ba.BusinessEntityID
INNER JOIN Person.Address a ON a.AddressID = ba.AddressID
WHERE s.TerritoryID IS NOT NULL
```

```
SELECT p.FirstName, p.LastName,
 NTILE(2) OVER(PARTITION BY s.TerritoryID ORDER BY s.SalesYTD DESC) AS
 QuarterGroup, s.SalesYTD, s.TerritoryID
FROM Sales.SalesPerson s INNER JOIN Person.Person p
 ON s.BusinessEntityID = p.BusinessEntityID
INNER JOIN Person.BusinessEntityAddress ba
 ON p.BusinessEntityID = ba.BusinessEntityID
INNER JOIN Person.Address a ON a.AddressID = ba.AddressID
WHERE s.TerritoryID IS NOT NULL
```

---

### Подсказка к экзамену

Кроме понимания особенностей применения четырех ранжирующих функций необходимо знать, когда RANK/DENSE\_RANK формируют те же результаты, что и функция ROW\_NUMBER, а также какова разница между функциями RANK и DENSE\_RANK.

---

### Быстрая проверка

#### ВОПРОСЫ:

1. Какова разница между функциями RANK и DENSE\_RANK?
2. Когда функции ROW\_NUMBER, RANK и DENSE\_RANK формируют одинаковые результаты?

#### ОТВЕТЫ:

1. RANK присваивает одно и то же значение дубликатам, но оставляет разрыв в последовательности, соответствующий количеству совпадающих строк. DENSE\_RANK присваивает один и тот же номер дубликатам, но не создает разрыва в последовательности номеров.
2. ROW\_NUMBER, RANK и DENSE\_RANK формируют одинаковые результаты, если в столбцах результирующего набора, по которым проводится сортировка, нет дублирующихся значений.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Поиск разрывов в последовательности

На этом практическом занятии вы улучшите практическое решение *урока 2*, применив для повышения производительности ранжирующие функции с ОТВ.

## Упражнение

В этом упражнении вы улучшите практическое решение *урока 2*, применив для повышения производительности ранжирующие функции с ОТВ.

1. Откройте окно создания запроса, введите и выполните следующий программный код, загружающий таблицу с тестовыми данными.

```
CREATE TABLE #orderdates
(CustomerID INT NOT NULL,
 OrderDate DATETIME NOT NULL);

DECLARE @startDate datetime,
 @endDate datetime,
 @CustomerID int

SET @CustomerID = 1

WHILE @CustomerID < 100
BEGIN
 SELECT @startDate = 'Jan 01, 2007',
 @endDate = DATEADD(yy, 1, @startDate);

 WITH dates AS
 (SELECT @startDate AS begindate
 UNION ALL
 SELECT DATEADD(dd,1,begindate) FROM dates
 WHERE begindate < dateadd(dd,-1,@endDate))

 INSERT INTO #orderdates
 SELECT @CustomerID, begindate FROM dates
 OPTION (MAXRECURSION 0);

 SET @CustomerID = @CustomerID + 1
END

SELECT * FROM #orderdates;
```

2. В открытом окне запроса введите, выделите и выполните следующий программный код для создания разрывов в последовательности.

```
--Теперь удалите несколько строк для создания разрывов
DELETE #orderdates
WHERE DATEDIFF(dd, 0, OrderDate)%11 = 0 ;

--Создайте пару многодневных пропусков
DELETE FROM #orderdates
WHERE OrderDate IN ('1/4/2007','2/17/2007')
AND CustomerID%3 = 0

SELECT * FROM #orderdates;
```

3. В имеющемся окне запроса введите, выделите и выполните следующий программный код для получения списка всех пропусков в последовательности.

```
WITH OrderDatesCTE(CustomerID, RowNum, OrderDate) AS
(SELECT CustomerID, ROW_NUMBER()
 OVER(PARTITION BY CustomerID ORDER BY OrderDate) AS
 RowNum, OrderDate
 FROM #orderdates)
SELECT a.CustomerID, DATEADD(dd, 1, a.OrderDate) AS StartGap
 , DATEADD(dd, -1, b.OrderDate) AS EndGap
 , DATEDIFF(dd, DATEADD(dd, 1, a.OrderDate)
 , DATEADD(dd, -1, b.OrderDate)) + 1
 AS NumberMissingDays
FROM OrderDatesCTE a INNER JOIN OrderDatesCTE b
ON a.CustomerID = b.CustomerID and a.RowNum = b.RowNum - 1
WHERE DATEDIFF(dd, a.OrderDate, DATEADD(dd, -1, b.OrderDate)) <> 0
ORDER BY a.CustomerID, a.OrderDate
```

## Резюме урока

- ◆ Функция `ROW_NUMBER` применяется для последовательной нумерации строк в результирующем наборе, но может формировать разные результаты, если в столбцах, используемых для сортировки, встречаются совпадающие значения.
- ◆ Функция `RANK` присваивает дубликатам одинаковые номера и может сформировать разрывы в последовательной нумерации.
- ◆ Функция `DENSE_RANK` присваивает дубликатам одинаковые номера, но не создает разрывов в последовательной нумерации.
- ◆ Функция `NTILE` позволяет разбить результирующий набор на приблизительно равные по размеру группы.

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить знания и навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемом для тренировки конкретном сценарии. Сценарий описывает реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решение.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ Рекурсивные ОТВ могут применяться для решения разнообразных проблем, требующих обхода иерархии, более эффективного, чем предлагаемый методом, основанном на операциях с курсором.
- ◆ Вложенные запросы позволяют встраивать один запрос в другой. Несвязанный вложенный запрос не зависит от внешнего запроса, а связанный вложенный запрос ссылается на столбцы из внешнего запроса.
- ◆ Ранжирующие функции могут применяться для решения множества задач, требующих упорядочивания результирующего набора, например, нумерация страниц и поиск разрывов в последовательности.

## Ключевые термины

Знаете ли вы, что означают следующие ключевые термины? Вы можете проверить свои ответы, заглянув в словарь терминов, приведенный в конце книги.

- ◆ Связанный вложенный запрос.
- ◆ Несвязанный вложенный запрос.
- ◆ Рекурсивное ОТВ.

## Сценарий

В следующем конкретном сценарии вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в разд. *"Ответы к сценариям"* в конце книги.

### Сценарий. Повышение производительности запроса

Компания Blue Yonder Airlines реализует новое хранилище данных, получающее данные о ежедневных загрузках самолетов. Каждый день должен формироваться отчет для уведомления руководства о том, какая из дочерних компаний не передала свои данные. В то же время в производственной среде возникли серьезные проблемы с производительностью. Вас попросили помочь в разработке средств загрузки данных и формирования отчетов, а также в решении проблем с производительностью, возникающих в транзакционных базах данных. В процессе анализа вы обнаружили, что все хранимые процедуры основаны на курсорах и временных таблицах. Курсоры применяются для итераций в результирующем наборе, позволяющих найти пропущенные данные, заполнить пустые места в самолете или построить диаграммы продаж. Временные таблицы используются главным образом для хранения промежуточных результирующих наборов до того, как будет получен окончательный результат.

Ответьте на следующий вопрос вашего менеджера: "Что вы сможете сделать для решения проблем компании Blue Yonder Airlines?"

## КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

### Построение рекурсивных запросов с ОТВ

**Упражнение 1.** Замените в ваших приложениях решения для обхода иерархии на основе курсора рекурсивными ОТВ.

### Реализация вложенных запросов

**Упражнение 2.** Замените вложенными запросами решения на базе курсора, которые используются в ваших приложениях для вычисления итогов с накоплением.

**Упражнение 3.** Найдите в ваших приложениях программный код, выполняющий для решения задачи множественные вызовы. Измените код так, чтобы все значения извлекались в набор, который затем можно подставить в предложение WHERE как несвязанный вложенный запрос.

**Упражнение 4.** Удалите временные таблицы, которые применяются в ваших приложениях для хранения промежуточных результирующих наборов, и замените их производными таблицами.

### Применение ранжирующих функций

**Упражнение 5.** Замените решение на основе курсора, которое вы используете для поиска разрывов в последовательности в одном из ваших приложений, комбинацией ОТВ и функции ROW\_NUMBER.

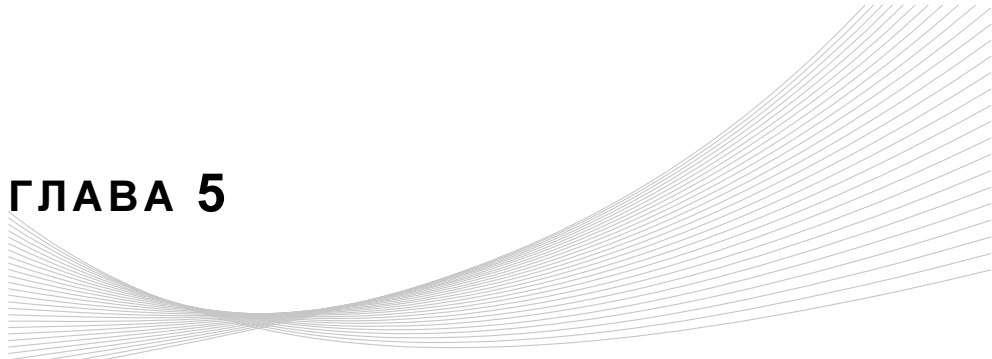
## Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобию будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### ***Дополнительная информация о тренировочных тестах***

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.





## ГЛАВА 5

# Программирование в Microsoft SQL Server на языке T-SQL пользовательских хранимых процедур, функций, триггеров и представлений

Microsoft SQL Server 2008 позволяет создавать программируемые объекты трех типов: хранимые процедуры, функции и триггеры. Вместо поочередного выполнения отдельных инструкций или команд вы можете с помощью мощного языка программирования, содержащего переменные, параметры, потоки управления и обработку ошибок, создавать объекты, включающие блоки программного кода. SQL Server поставляется с сотнями функций и позволяет вам создавать собственные функции, включающие в себя часто выполняемые операции извлечения данных. Так же как разработчики приложений применяют общедоступные (public) свойства и методы, предоставляя прикладные интерфейсы (API) для построения объектно-ориентированных, клиент-серверных, Web- и сервис-ориентированных и распределенных приложений, применяемых в пределах организации, хранимые процедуры формируют абстрактный прикладной интерфейс для таблиц и данных, содержащихся в ваших базах данных, что позволяет разработчикам вместо разбора и анализа хитростей структуры базы данных сосредоточиться на написании программного кода. Триггеры — это особая форма хранимой процедуры, которая запускается автоматически при выполнении команд языка манипулирования данными (DML) или языка определения данных (DDL).

В дополнение к трем типам программируемых объектов вы можете хранить в базе данных инструкции `SELECT`, применяя представления.

В этой главе вы узнаете, как создавать в ваших приложениях хранимые процедуры, функции, триггеры и представления. Вы также научитесь управлять транзак-

циями и реализовывать подпрограммы обработки ошибок. Несмотря на то, что программируемые объекты могут создаваться как на языке Transact-SQL (T-SQL), так и на языках среды Common Language Runtime (CLR), в этой главе вы будете применять T-SQL.

### **Экзаменационные темы в этой главе**

- ◆ Создание и изменение хранимых процедур.
- ◆ Создание и изменение функций, определенных пользователем.
- ◆ Создание и изменение триггеров DML.
- ◆ Создание и изменение триггеров DDL.
- ◆ Создание и распространение объектов на базе CLR.
- ◆ Реализация обработки ошибок.
- ◆ Управление транзакциями.
- ◆ Создание и изменение представлений.

### **Уроки в этой главе**

- ◆ Урок 1. Хранимые процедуры.
- ◆ Урок 2. Функции, определенные пользователем.
- ◆ Урок 3. Триггеры.
- ◆ Урок 4. Представления.

## **Прежде чем начать**

Для усвоения уроков этой главы необходимо иметь:

- ◆ установленную версию SQL Server 2008;
- ◆ установленную базу данных AdventureWorks.

### **Дополнительная информация об объектах CLR**

Вы можете создавать триггеры, функции и хранимые процедуры на языке, поддерживаемом средой CLR. Создание объектов CLR и управление ими обсуждаются в *главе 7*.

### **РЕАЛЬНАЯ ИСТОРИЯ.**

#### **Майк Хотек (Mike Hotek)**

Десятилетиями компании разрабатывали рабочие среды и средства генерации программного кода, которые, как предполагалось, должны были избавить разработчиков от необходимости разбираться в базах данных или языке структурированных запросов (Structured Query Language, SQL). В то время как многие из этих рабочих сред и средств генерации кода позволяли разработчикам быстро создавать боль-

шие объемы программного кода, для доступа к базе данных сам сгенерированный код, не будучи подвергнут серьезной модификации, очень слабо соответствовал требованиям бизнес-приложений.

В одной компании, где я работал над проектом, один из разработчиков не мог дожидаться появления нового генератора программного кода, который был разработан для того, чтобы избавить разработчиков от необходимости понимания применяемых баз данных. В приложении разработчик написал примерно четыре строки программного кода, который извлекал единственную строку из таблицы, основываясь на входном значении одного из ее столбцов. Это можно было бы сделать с помощью простой инструкции `SELECT`, которую можно было бы вставить в хранимую процедуру. Тем не менее, приложение, предъявленное SQL Server, создало функцию, содержащую инструкцию `SELECT`, вторую инструкцию `SELECT` для обращения к функции, программный код, уничтожающий функцию в конце процесса, и от 30 до 40 строк на языке T-SQL для проверки корректности обработки ошибок и гарантии того, что функция была создана и удалена корректно.

Если бы разработчик написал просто хранимую процедуру и затем вызвал ее в программном коде, ему пришлось бы написать в целом 8 строк кода, и каждый запрос пользователя выполнял бы всего три инструкции. Применение же автоматически генерируемого программного кода потребует создания и уничтожения объекта с помощью выполнения 70 строк кода на каждый пользовательский запрос.

Несмотря на то, что генератор программного кода делает всю работу, так что разработчику не о чем беспокоиться, сравнение производительностей показало, что автоматически генерируемому коду требуется в шесть раз больше ресурсов и гораздо более длительное время по сравнению с написанной простой хранимой процедурой.

Язык T-SQL очень легко выучить, он популярен, а синтаксис его почти не меняется более 50 лет. Каждый раз, когда я слышу о еще одном средстве генерации программного кода, которое кто-то создал для того, чтобы разработчикам не нужно было понимать, как обращаться к базе данных, мне на ум приходит — новое изобретение колеса.

Язык T-SQL гораздо проще, чем любые применяемые платформы для разработки. Точно так же как колесо, популярное уже 5000 лет и претерпевшее минимум усовершенствований, SQL надолго переживет любой язык разработки приложений, который вы когда-либо использовали, предоставляя при этом самый эффективный способ доступа к реляционной базе данных.

## УРОК 1. Хранимые процедуры

Хранимые процедуры предоставляют уровень абстракции, ограждающий приложения от низкоуровневой структуры базы данных. Будучи главной опорой любого приложения SQL Server, хранимые процедуры позволяют изменять структуру базы данных и управлять производительностью без переписывания приложений и распространения их обновлений. Из этого урока вы узнаете, как создавать хранимые процедуры, предоставляющие программный интерфейс, необходимый для создания легко сопровождаемых и эффективных приложений для работы с базами данных.

---

**После усвоения этого урока вы сможете**

---

- Создавать и изменять хранимые процедуры.
- Реализовывать обработку ошибок.
- Управлять транзакциями.

Расчетное время урока — 20 мин.

## Создание хранимых процедур

*Хранимая процедура* — это одна или несколько инструкций, которым дано имя и которые хранятся в базе данных. Почти любая команда языка T-SQL может быть включена в хранимую процедуру, что делает процедуры подходящими для приложений и для решения myriad административных задач. Не могут применяться в хранимой процедуре только перечисленные далее команды:

- ◆ `USE <database name>;`
- ◆ `SET SHOWPLAN_TEXT;`
- ◆ `SET SHOWPLAN_ALL;`
- ◆ `SET PARSEONLY;`
- ◆ `SET SHOWPLAN_XML;`
- ◆ `CREATE AGGREGATE;`
- ◆ `CREATE RULE;`
- ◆ `CREATE DEFAULT;`
- ◆ `CREATE SCHEMA;`
- ◆ `CREATE FUNCTION` **или** `ALTER FUNCTION;`
- ◆ `CREATE TRIGGER` **или** `ALTER TRIGGER;`
- ◆ `CREATE PROCEDURE` **или** `ALTER PROCEDURE;`
- ◆ `CREATE VIEW` **или** `ALTER VIEW.`

При первом обращении к хранимой процедуре SQL Server генерирует планы компиляции и выполнения, которые хранятся в кэше запроса и повторно используются последующими выполнениями. Следовательно, применяя хранимые процедуры, вы можете получить небольшой выигрыш в производительности, т. к. при последующих выполнениях хранимой процедуры исчезает необходимость в синтаксическом анализе, компиляции и генерации плана запроса. Но главное назначение хранимой процедуры — создание защитного слоя и прикладного интерфейса для ваших баз данных, которые отделяют приложения от изменений в структуре базы данных.

Далее приведена синтаксическая запись общего вида для создания хранимой процедуры.

```

CREATE { PROC | PROCEDURE } [schema_name.] procedure_name [; number]
 [{ @parameter [type_schema_name.] data_type }
 [VARYING] [= default] [OUT | OUTPUT] [READONLY]
] [,...n]
[WITH <procedure_option> [,...n]]
[FOR REPLICATION]
AS { <sql_statement> [;][...n] | <method_specifier> } [;]
<procedure_option> ::=
 [ENCRYPTION] [RECOMPILE] [EXECUTE AS Clause]

```

Когда вы задаете параметр `ENCRYPTION`, доступный для триггеров, функций, процедур и представлений, SQL Server применяет побитный оператор `OR` к программному коду объекта. Параметр `ENCRYPTION` перенесен из ранних версий SQL Server и создает некоторую путаницу. Если вы задаете параметр `ENCRYPTION`, это не означает применения процедуры шифрования для скрытия вашего программного кода. SQL Server применяет алгоритм простого побитного `OR`, который просто делает непонятным код в объекте. Если посмотреть на определение объекта, вы увидите непонятный текст. Но очень простая, общедоступная подпрограмма устраняет эту невнятность кода. SQL Server не разрешает скрывать программный код в триггерах, функциях, представлениях и хранимых процедурах, и любой человек, имеющий разрешение для объекта `VIEW DEFINITION`, может извлечь написанный вами программный код.

### **ПРЕДУПРЕЖДЕНИЕ.**

#### **Шифрование модулей**

SQL Server не является системой цифрового управления авторскими правами (digital rights management, DRM). Текст модуля не шифруется; скорее, он делается непонятным. Любой пользователь с доступом к метаданным базы данных легко может преобразовать обратно непонятный текст. Параметр `ENCRYPTION` не означает защиту от чтения другим пользователем программного кода в ваших модулях.

Если вы хотите изменить содержимое хранимой процедуры или ее параметры, можно применить инструкцию `ALTER PROCEDURE`.

От простого пакета на языке T-SQL хранимую процедуру отличают используемые структуры программного кода, такие как переменные, параметры, обработка ошибок и структуры, управляющие ходом выполнения.

## **Комментарии к программному коду**

Один из признаков хорошо разработанного программного кода — соответствующие комментарии, которые упрощают дальнейшее сопровождение.

### **ПРАКТИЧЕСКИЙ СОВЕТ**

В ваш программный код следует вставлять достаточно комментариев, позволяющих будущим разработчикам понять, что делает ваш код и какие искусственные приемы

необходимо применять. Триггер, функция или хранимая процедура не должны содержать комментариев в единственном случае — если программный код тривиален, например, хранимая процедура, содержащая единственную инструкцию `SELECT`.

В языке T-SQL есть две разные конструкции для вставки комментариев.

```
--Это одна строка комментариев
/*
Это
многострочный комментарий
*/
```

### **ПРЕДУПРЕЖДЕНИЕ.**

#### **Ограничители пакета**

В языке T-SQL ограничитель пакета — `GO`. В процессе синтаксического анализа, если SQL Server обнаруживает ключевое слово `GO`, пакет считается завершенным, и весь последующий программный код считается принадлежащим другому пакету. Даже если ключевое слово `GO` находится в блоке комментариев, SQL Server все равно интерпретирует его как ограничитель пакета.

## **Переменные, параметры и коды возврата**

SQL Server предоставляет объекты трех типов для передачи значений в ваш программный код, а также возврата скалярного значения вызывающей подпрограмме.

### **Переменные**

*Переменные* обеспечивают манипулирование данными, их хранение и передачу как внутри процедуры, так и между хранимыми процедурами и функциями. В SQL Server есть два вида переменных: локальные и глобальные. *Локальная переменная* обозначается одним символом "эт" (@), а *глобальная переменная* — двумя (@@). Кроме того, локальные переменные вы можете создавать, читать и писать в них, а глобальные переменные можно только читать. В табл. 5.1 перечислены некоторые самые популярные глобальные переменные.

**Таблица 5.1. Глобальные переменные**

| Глобальная переменная | Описание                                                              |
|-----------------------|-----------------------------------------------------------------------|
| @@ERROR               | Код ошибки последней выполненной инструкции                           |
| @@IDENTITY            | Последнее идентифицирующее значение, вставленное в текущем соединении |
| @@ROWCOUNT            | Количество строк, обработанных последней инструкцией                  |
| @@TRANCOUNT           | Количество открытых транзакций в текущем соединении                   |
| @@VERSION             | Версия SQL Server                                                     |

### **ПРАКТИЧЕСКИЙ СОВЕТ.** **Идентифицирующие значения**

Переменная @@IDENTITY содержит последнее идентифицирующее значение, вставленное в текущем соединении. Если вы вызываете несколько модулей, вставляющих идентифицирующие значения, последнее вставленное значение всегда содержится в переменной @@IDENTITY. Например, если вызывается процедура, вставляющая строку в таблицу TableA, которая содержит триггер, вставляющий строки в таблицы TableB, TableC и TableD, причем каждая вставка генерирует новое значение идентификации, значение @@IDENTITY, которое извлекается следом за вставкой в таблицу TableA, соответствует значению идентификации, вставленному в TableD. Из-за этого ограничения применять @@IDENTITY не рекомендуется. Используйте вместо нее функцию SCOPE\_IDENTITY(), которая вернет последнее идентифицирующее значение, вставленное в пределах текущей инструкции.

Вы задаете конкретную переменную с помощью инструкции DECLARE, в которой указываете имя и тип данных переменной. У переменной может быть любой тип данных, кроме типов text, ntext и image. Например,

```
DECLARE @intvariable INT,
 @datevariable DATE,
 @spatialvar GEOGRAPHY,
 @levelvar HIERARCHYID

DECLARE @tablevar TABLE
(ID INT NOT NULL,
Customer VARCHAR(50) NOT NULL)
```

### **Нерекомендуемые типы данных**

text, ntext и image — устаревшие типы данных, и применять их не следует.

Одна инструкция DECLARE может использоваться для задания нескольких переменных, но табличную переменную следует задавать в отдельной инструкции DECLARE.

Переменной можно присвоить статическое значение или значение, возвращаемое инструкцией SELECT. Для присваивания значения можно применять инструкцию SET или инструкцию SELECT, но если вы выполняете запрос, для присвоения значения вы должны использовать инструкцию SELECT. Инструкция SELECT также применяется для извлечения значения переменной.

Помимо присваивания значения с помощью инструкций SET или SELECT вы можете также присвоить значение во время объявления переменной. Далее приведено несколько примеров задания значений переменных разными способами.

```
DECLARE @intvariable INT = 2,
 @datevariable DATE = GETDATE(),
 @maxorderdate DATE = (SELECT MAX(OrderDate) FROM
 Orders.OrderHeader),
```

```
@counter1 INT,
@counter2 INT

SET @counter1 = 1
SELECT @counter2 = -1

SELECT @intvariable, @datevariable, @maxorderdate, @counter1, @counter2
```

### ***ПРЕДУПРЕЖДЕНИЕ.***

#### ***Присваивание значения переменной***

Чаще всего забывают о том, что переменные, за исключением табличной переменной, содержат скалярные значения. Если вы присваиваете переменной результаты инструкции `SELECT`, то должны быть уверены, что эта инструкция вернет единственную строку. Если инструкция `SELECT` возвращает несколько строк, переменной будет присвоено значение, соответствующее последней строке результирующего набора, а все другие значения из результирующего набора будут потеряны.

Переменная может применяться для выполнения вычислений, управления обработкой или как аргумент поиска (search argument, SARG) в запросе. С помощью переменных можно выполнять вычисления, используя инструкцию `SET` или `SELECT`. В SQL Server 2008 введен более компактный способ присваивания значений переменным с помощью вычисления.

--SQL Server 2005 и более ранние версии  
`DECLARE @var INT`

```
SET @var = 1
SET @var = @var + 1
SELECT @var
SET @var = @var * 2
SELECT @var
SET @var = @var / 4
SELECT @var
GO
```

--SQL Server 2008

```
DECLARE @var INT
SET @var = 1
SET @var += 1
SELECT @var
SET @var *= 2
SELECT @var
SET @var /= 4
SELECT @var
GO
```

## **Параметры**

*Параметры* — это локальные переменные, применяемые для передачи значений в хранимую процедуру во время ее выполнения. В процессе выполнения любые



параметры используются точно так же, как переменные и могут читаться и записываться. Объявляются параметры, как показано в следующем примере.

```
CREATE PROCEDURE <procedure name> @parm1 INT
 , @parm2 VARCHAR(20) = 'Default value'
AS
--Блок кода
```

Можно задавать параметры двух типов: входные и выходные. Выходной параметр обозначается ключевым словом OUTPUT.

```
CREATE PROCEDURE <procedure name> @parm1 INT
 , @parm2 VARCHAR(20) = 'Default value'
 , @orderid INT OUTPUT
AS
--Блок кода
```

Выходные параметры применяются, когда нужно вернуть единственное значение в приложение. Если нужно вернуть целый результирующий набор, в хранимую процедуру включается инструкция SELECT, которая формирует результаты и возвращает результирующий набор в приложение, как показано далее.

```
CREATE PROCEDURE <procedure name> @parm1 INT
 , @parm2 VARCHAR(20) = 'Default value'
AS
--Эта инструкция вернет результаты запроса в приложение
SELECT SalesOrderID, CustomerID, OrderDate
 , SubTotal, TaxAmt, Freight, TotalDue
FROM Sales.SalesOrderHeader
```

## Конструкции, управляющие ходом выполнения

В хранимых процедурах может применяться несколько конструкций, управляющих ходом выполнения процедуры:

- ◆ RETURN;
- ◆ IF...ELSE;
- ◆ BEGIN...END;
- ◆ WHILE;
- ◆ BREAK/CONTINUE;
- ◆ WAITFOR;
- ◆ GOTO.

Назад в приложение можно передать код возврата, указывающий на состояние выполнения процедуры. Коды возврата не предназначены для пересылки данных, они применяются для сообщений о состоянии выполнения. Инструкция RETURN завершает выполнение процедуры и возвращает управление вызывающе-

му приложению. После инструкции `RETURN` никакие инструкции не выполняются, как показано далее.

```
CREATE PROCEDURE <procedure name> @parm1 INT
 , @parm2 VARCHAR(20) = 'Default value'
AS
--Эта инструкция вернет 1 назад приложению, вызвавшему хранимую процедуру
RETURN 1
--Любой код, начиная с этого места, выполняться не будет
```

`IF...ELSE` позволяет выполнять программный код в соответствии с условием. Инструкция `IF` проверяет заданное условие и выполняет следующую инструкцию, если условие равно `True`. Необязательная инструкция `ELSE` позволяет выполнить заданный программный код, если проверка условия вернула значение `False`. Далее приведен пример.

```
DECLARE @var INT

SET @var = 1

IF @var = 1
 PRINT 'This is the code executed when true.'
ELSE
 PRINT 'This is the code executed when false.'
```

Независимо от ветви программного кода, выбранной конструкцией `IF...ELSE` в соответствии с условием, выполняется только одна инструкция, как показано в следующем примере.

```
DECLARE @var INT

SET @var = 1

IF @var = 2
 PRINT 'This is the code executed when true.'
 PRINT 'This will always execute.'
```

Поскольку инструкция `IF` выполняет в соответствии с условием только одну следующую строку кода, возникает проблема, если нужно в зависимости от заданного условия выполнить целый блок кода. Инструкция `BEGIN...END`, как показано далее, позволяет обозначить блоки программного кода, которые следует выполнить.

```
DECLARE @var INT

SET @var = 1

IF @var = 2
BEGIN
 PRINT 'This is the code executed when true.'
 PRINT 'This code is also executed only when the condition is true.'
END
```

Инструкция `WHILE` применяется для многократного выполнения блока кода до тех пор, пока заданное условие истинно. Далее приведен пример.

```
DECLARE @var1 INT,
 @var2 VARCHAR(30)

SET @var1 = 1

WHILE @var1 <= 10
BEGIN
 SET @var2 = 'Iteration #' + CAST(@var1 AS VARCHAR(2))
 PRINT @var2
 SET @var1 += 1
END
```

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### ***Выполнение инструкции***

Одна из самых распространенных ошибок, которую вы можете допустить при написании блоков программного кода, использующих `IF` или `WHILE`, — забыть о том, что SQL Server выполняет только следующую инструкцию в соответствии с условием. Во избежание подобных ошибок настоятельно рекомендуется всегда применять конструкцию `BEGIN...END` с инструкциями `IF` или `WHILE`, даже если вы намерены выполнить только одну строку кода, зависящую от условия. Это не только сделает код более понятным, но и поможет исключить ошибки при изменении программного кода в дальнейшем.

Инструкция `BREAK` применяется вместе с циклом `WHILE`. Если необходимо прервать выполнение в теле цикла `WHILE`, с помощью этой инструкции можно прервать повторное выполнение цикла. После выполнения инструкции `BREAK` выполняется строка программного кода, следующая за циклом `WHILE`. Инструкция `CONTINUE` применяется в цикле `WHILE` для того, чтобы заставить код продолжить выполнение с первой строки цикла.

#### ***Инструкции `BREAK` и `CONTINUE`***

Инструкции `BREAK` и `CONTINUE` почти никогда не применяются. Цикл `WHILE` завершается, как только условие цикла `WHILE` перестает быть истинным. Вместо встраивания в цикл дополнительной проверки вместе с инструкцией `BREAK` обычно циклами `WHILE` управляют с помощью подходящих условий циклов. Кроме того, цикл продолжает выполняться, пока условие `WHILE` равно `TRUE`. Следовательно, не возникает необходимости в применении инструкции `CONTINUE`.

Инструкция `WAITFOR` позволяет сделать паузу в выполнении программного кода. У этой инструкции есть три варианта: `WAITFOR DELAY`, `WAITFOR TIME` и `WAITFOR RECEIVE`. Вариант `WAITFOR RECEIVE` используется в сочетании с Service Broker, о котором вы узнаете в главе 8. `WAITFOR TIME` прерывает выполнение программного кода до наступления заданного момента времени. Вариант `WAITFOR DELAY` приостанавливает выполнение на заданный промежуток времени.

```
DECLARE @var1 INT,
 @var2 VARCHAR(30)

SET @var1 = 1

--Пауза на 2 секунды
WAITFOR DELAY '00:00:02'

WHILE @var1 <= 10
BEGIN
 SET @var2 = 'Iteration #' + CAST(@var1 AS VARCHAR(2))
 PRINT @var2
 SET @var1 += 1
END
```

Инструкция `GOTO` позволяет передать выполнение на метку, встроенную в процедуру. Применение конструкций, подобных `GOTO`, не одобряется во всех языках программирования.

## Сообщения об ошибках

Сообщения об ошибках в SQL Server состоят из трех компонентов:

- ◆ номера ошибки;
- ◆ уровня серьезности ошибки;
- ◆ непосредственно сообщения об ошибке.

Номер ошибки — это целое число. Сообщения об ошибках, поставляемые с SQL Server, имеют номера от 1 до 49 999.

В SQL Server определено 26 уровней серьезности ошибок, пронумерованных от 0 до 25. Любая ошибка с уровнем серьезности 16 или более высоким автоматически регистрируется в журнале ошибок SQL Server и в журнале регистрации событий Windows Application Event Log. Ошибки с уровнем серьезности 19—25 могут быть заданы только пользователями, обладающими постоянной ролью сервера sysadmin. Ошибки с уровнями серьезности 20—25 считаются фатальными и вызывают разрыв соединения и откат всех открытых транзакций.

Сообщение об ошибке может быть длиной до 255 символов Unicode и может принимать до двух параметров.

Вы можете создать собственные пользовательские сообщения об ошибках, которые должны нумероваться с номера 50 001 (номер 50 000 резервируется для обозначения сообщения без указанного номера). Сообщения об ошибках могут быть переведены на все языки, поддерживаемые SQL Server, но прежде чем создавать сообщение на другом языке, вы должны создать английскую версию этого сообщения.

Пользовательское сообщение об ошибке создается выполнением процедуры `sp_addmessage`, как показано далее.

```
sp_addmessage [@msgnum =] msg_id, [@severity =] severity,
 [@msgtext =] 'msg'
 [, [@lang =] 'language']
 [, [@with_log =] 'with_log']
 [, [@replace =] 'replace']
```

В следующем примере создается пользовательское сообщение на английском языке с номером ошибки 50 001 и уровнем серьезности 16.

```
EXEC sp_addmessage 50001, 16,
 N'The approved credit must be between 100 and 10,000';
GO
```

В приведенном далее примере создается принимающее два параметра пользовательское сообщение на английском и французском языках с номером ошибки 50 002 и уровнем серьезности 16.

```
EXEC sp_addmessage @msgnum = 50002, @severity = 16,
 @msgtext = N'The product named %s already exists in %d.',
 @lang = 'us_english';

EXEC sp_addmessage @msgnum = 50002, @severity = 16,
 @msgtext = N' %1! de produit existent deja dans %2!',
 @lang = 'French';
GO
```

Первый параметр сообщения обозначается %s, а второй — %d. Эти обозначения применяются только в сообщениях на американском английском. Для всех остальных языков используются обозначения %1! и %2! для первого и второго параметров соответственно.

Отправляются сообщения с помощью команды RAISERROR, имеющей следующую синтаксическую запись:

```
RAISERROR ({ msg_id | msg_str | @local_variable }
 { ,severity ,state }
 [,argument [,...n]])
 [WITH option [,...n]]
```

Можно просмотреть сообщения экземпляра SQL Server, запросив представление каталога sys.messages. Пользовательские сообщения об ошибках можно изменять, выполнив системную хранимую процедуру sp\_altermessage. Пользовательское сообщение об ошибке удаляется с помощью системной хранимой процедуры sp\_dropmessage.

## Обработка ошибок

Если бы вы всегда писали не содержащий недочетов хорошо определенный программный код, который всегда выполнялся успешно, предсказуемым образом, у вас никогда бы не было ошибок. Однако весь программный код — источник

сбоев и аварийных ситуаций. Приложение может попытаться передать некорректные параметры, бизнес-правила могут нарушаться или ваш код может быть не предназначен для обработки вызовов, выполненных из ряда новых приложений. Следовательно, в ваши хранимые процедуры необходимо включать обработку ошибок, которая позволит найти источник проблемы и исправить ее самым безболезненным способом.

До версии SQL Server 2005 единственным способом обработки ошибок была проверка значения глобальной переменной @@ERROR. При выполнении каждой инструкции SQL Server записывает состояние завершения в @@ERROR. Если возникла ошибка, ее номер содержится в @@ERROR. Если инструкция выполнялась успешно, переменная @@ERROR содержит 0. Вам необходимо запросить переменную, чтобы узнать успешно или аварийно завершилась инструкция. К сожалению, простое выполнение инструкции SELECT для извлечения значения переменной @@ERROR устанавливает новое значение этой переменной, переопределяя ее предыдущее значение. Применение @@ERROR для обработки ошибок очень обременительно, т. к. требует встроенных проверок после каждой инструкции и процедуры обработки ошибок для каждой инструкции.

Теперь для более структурированного подхода к обработке ошибок, подобного процедурам обработки ошибок в других языках программирования, можно применять блок TRY...CATCH.

У блока TRY...CATCH есть два компонента. Блок TRY применяется для охвата любого фрагмента кода, в котором может возникнуть ошибка, требующая отслеживания и обработки. Блок CATCH используется для обработки ошибки.

В следующем программном коде создается ошибка из-за нарушения ограничения первичного ключа. Можно предположить, что этот программный код оставит после себя пустую таблицу из-за ошибки в транзакции, но оказывается, что первая и третья инструкции INSERT завершаются успешно и вставляют в таблицу две строки.

```
--Ошибки транзакции
CREATE TABLE dbo.mytable
(ID INT NOT NULL PRIMARY KEY)

BEGIN TRAN
 INSERT INTO dbo.mytable VALUES(1)
 INSERT INTO dbo.mytable VALUES(1)
 INSERT INTO dbo.mytable VALUES(2)
COMMIT TRAN

SELECT * FROM dbo.mytable
```

Наличие в таблице двух вставленных в нее строк объясняется тем, что по умолчанию SQL Server не выполняет откат транзакции, содержащей ошибку. Если вы

хотите, чтобы транзакция завершалась полностью или целиком отбрасывалась, можно выполнить команду `SET` для изменения параметра вашего соединения `XACT_ABORT` следующим образом.

```
TRUNCATE TABLE dbo.mytable

SET XACT_ABORT ON;
BEGIN TRAN
 INSERT INTO dbo.mytable VALUES(1)
 INSERT INTO dbo.mytable VALUES(1)
 INSERT INTO dbo.mytable VALUES(2)
COMMIT TRAN
SET XACT_ABORT OFF;

SELECT * FROM dbo.mytable
```

Несмотря на то, что с помощью инструкции `SET XACT_ABORT ON` вы достигли цели, при изменении установочных параметров соединения можно получить непредсказуемые результаты в приложении, если в вашем программном коде параметры переустановлены некорректно. Лучшее решение — применение структурированного обработчика ошибок для отслеживания ошибки и выбора способа ее обработки.

В SQL Server 2008 блок `TRY...CATCH` реализован следующим образом.

- ◆ Если в блоке `TRY` обнаружена ошибка с уровнем серьезности менее 20, управление передается в соответствующий блок `CATCH`.
- ◆ Если ошибка встретится в блоке `CATCH`, транзакция отменяется и ошибка возвращается в вызывающее приложение, если блок `CATCH` не вложен в другой блок `TRY`.
- ◆ Блок `CATCH` должен следовать сразу за блоком `TRY`.
- ◆ В блоке `CATCH` вы можете зафиксировать текущую транзакцию или выполнить ее откат, если она не находится в состоянии, не допускающем фиксации.
- ◆ Инструкция `RAISERROR`, выполненная в блоке `TRY`, немедленно передает управление в блок `CATCH`, не возвращая в приложение сообщение об ошибке.
- ◆ Инструкция `RAISERROR`, выполненная в блоке `CATCH`, закрывает транзакцию, и управление передается вызывающему приложению с заданным сообщением об ошибке.
- ◆ Если в блоке `CATCH` не выполняется инструкция `RAISERROR`, вызывающее приложение никогда не получит сообщение об ошибке.

---

### **Отслеживание ошибок**

Блок `TRY...CATCH` не отслеживает ошибки, вызывающие разрыв соединения, например фатальную ошибку или выполнение системным администратором (sysadmin)

команды KILL. Вы также не можете отследить сбои, вызванные ошибками компиляции, синтаксическими ошибками или отсутствием объектов. Следовательно, для проверки наличия объектов нельзя использовать блок TRY...CATCH.

В следующем примере выполнена структурированная обработка ошибок блока программного кода из предыдущего примера.

```
-- TRY...CATCH
TRUNCATE TABLE dbo.mytable

BEGIN TRY
 BEGIN TRAN
 INSERT INTO dbo.mytable VALUES (1)
 INSERT INTO dbo.mytable VALUES (1)
 INSERT INTO dbo.mytable VALUES (2)
 COMMIT TRAN
END TRY

BEGIN CATCH
 ROLLBACK TRAN
 PRINT 'Catch'
END CATCH

SELECT * FROM dbo.mytable
```

Одна из наиболее важных особенностей блока TRY...CATCH заключается в том, что пока в блоке CATCH не будет выполнена инструкция RAISERROR, никакие сообщения об ошибках не посылаются в вызывающее приложение. В блоке CATCH у вас есть доступ к следующим функциям:

- ◆ ERROR\_NUMBER() — возвращает номер возникшей ошибки;
- ◆ ERROR\_MESSAGE() — возвращает текст сообщения об ошибке;
- ◆ ERROR\_SEVERITY() — возвращает уровень серьезности сообщения об ошибке;
- ◆ ERROR\_STATE() — возвращает статус ошибки;
- ◆ ERROR\_PROCEDURE() — возвращает имя функции, триггера или процедуры, выполнявшихся в момент возникновения ошибки;
- ◆ ERROR\_LINE() — возвращает номер строки в функции, триггере или процедуре, в которой возникла ошибка.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### **Возврат сообщений о системных ошибках**

Если вы применяете блок TRY...CATCH, никакие ошибки, включая системные, не возвращаются в вызывающее приложение. Единственный способ вернуть сообщение об ошибке в вызывающее приложение — выполнить инструкцию RAISERROR. Но с помощью RAISERROR вы можете задать только пользовательское сообщение об ошибке или динамически сформировать сообщение об ошибке. Таким образом, воз-



никает небольшая проблема, если нужно вернуть в вызывающее приложение сообщение об ошибке, генерируемой системой. Если вы хотите это сделать, следует динамически сформировать сообщение, включающее данные сообщения о системной ошибке, возвращаемые инструкцией `RAISERROR`, которая не предоставляет идентификатора сообщения. Таким образом, любые сообщения, генерируемые системой, всегда возвращаются с номером ошибки 50 000.

Внутри блока `CATCH` можно определить уровень вложенности текущей транзакции с помощью глобальной переменной `@@TRANCOUNT`. С помощью функции `XACT_STATE` вы можете также узнать состояние самой внутренней транзакции. Функция `XACT_STATE` возвращает следующие значения:

- ◆ 1 — открытая транзакция существует, и она может быть зафиксирована или выполнен ее откат;
- ◆ 0 — открытой транзакции нет;
- ◆ -1 — открытая транзакция существует, но она в обреченном состоянии. Из-за типа возникшей ошибки можно выполнить только откат транзакции.

Параметр `XACT_ABORT` ведет себя иначе, если применяется с блоком `TRY`. Вместо завершения транзакции управление передается в блок `CATCH`. Но если параметр `XACT_ABORT` в состоянии `ON`, любая ошибка становится фатальной. Транзакция остается в обреченном состоянии и функция `XACT_STATE` возвращает -1. Следовательно, вы не сможете зафиксировать транзакцию внутри блока `CATCH`, если установлен параметр `XACT_ABORT`.

### ***Подсказка к экзамену***

Разберитесь, как блоки `TRY...CATCH` обрабатывают ошибки, а также как ведет себя параметр `XACT_ABORT` в блоке `TRY...CATCH`.

## **Выполнение хранимых процедур**

Вы обращаетесь к хранимой процедуре, используя инструкцию `EXEC`. Если у хранимой процедуры нет входных параметров, достаточно следующего программного кода.

```
EXEC <stored procedure>
```

Если у хранимой процедуры есть входные параметры, их можно передать по именам или указав значения в нужном порядке.

```
-- Указание по имени
```

```
EXEC <stored procedure> @parm1=<value>, @parm2=<value>, ...
```

```
-- Указание по позиции
```

```
EXEC <stored procedure> <value>, <value>, ...
```

Позиционная передача параметров в хранимую процедуру приводит к более компактному программному коду, но более подвержена ошибкам. Когда пара-

метры передаются в процедуру по именам, изменения порядка следования параметров в процедуре не потребует никаких изменений в вашем приложении. Независимо от способа передачи параметров в процедуру вы должны задать значения для каждого параметра, не имеющего значения по умолчанию.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### **Выполнение хранимых процедур**

Если хранимая процедура выполняется в первой строке пакета программного кода, ключевое слово `EXEC` не обязательно. Но в любом другом месте пакета для вызова хранимой процедуры требуется `EXEC`. Даже если перед вызовом процедуры расположены только комментарии, ключевое слово `EXEC` все равно необходимо. Для того чтобы избежать путаницы и быть уверенным в том, что ваш программный код выполнится всегда, независимо от структуры пакета, для вызова хранимой процедуры используйте ключевое слово `EXEC`.

Для применения выходных параметров следом за каждым из них необходимо указывать ключевое слово `OUT` или `OUTPUT`.

```
-- Применение выходных параметров
DECLARE @variable1 <data type>,
 @variable2 <data type>
...
```

```
EXEC <stored procedure> @parameter1, @variable1 OUTPUT, @variable2 OUT
```

Если необходимо перехватить из хранимой процедуры код возврата, его следует сохранить в переменной следующим образом:

```
-- Перехват кода возврата
DECLARE @variable1 <data type>,
 @variable2 <data type>,
 @returncode INT
```

```
EXEC @returncode = <stored procedure> @parameter1,
 @variable1 OUTPUT, @variable2 OUT
```

### **Динамическое выполнение**

Несмотря на то, что в хранимых процедурах приложения очень редко используют динамическое выполнение команды, многие административные процедуры нуждаются в построении команд и их динамическом выполнении. В языке T-SQL есть два способа выполнения динамически сформированной инструкции: `EXEC(<command>)` и `sp_executesql <command>`. Далее показано, как применять каждый из методов.

```
EXEC('SELECT OrderID, CustomerID FROM Sales.SalesOrderHeader
 WHERE OrderID = 1')
GO
```

```
DECLARE @var VARCHAR(MAX)
SET @var = 'SELECT OrderID, CustomerID FROM Sales.SalesOrderHeader
 WHERE OrderID = 1'
EXEC(@var)
GO

EXEC sp_executesql N'SELECT OrderID, CustomerID
 FROM Sales.SalesOrderHeader WHERE OrderID = 1'
GO

DECLARE @var NVARCHAR(MAX)
SET @var = 'SELECT OrderID, CustomerID FROM Sales.SalesOrderHeader
 WHERE OrderID = 1'
EXEC sp_executesql @var
GO
```

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### ***Безопасность приложения и атаки типа SQL-инъекции***

При каждом построении строки для динамического выполнения появляется возможность атаки типа SQL-инъекции. Следует всегда проверять корректность параметров, передаваемых в хранимую процедуру для создания динамической команды. Во избежание множества проблем, связанных с внедрением в код на языке SQL, необходимо применять системную хранимую процедуру `sp_executesql` с подстановкой параметров. Атаки типа SQL-инъекции не обсуждаются в книге, но прежде чем писать программный код, использующий преимущества динамического выполнения, обязательно прочтите многочисленные опубликованные статьи о внедрении в код на SQL и оцените риск выбранного вами решения.

## **Среда выполнения модулей**

Функции и хранимые процедуры позволяют с помощью предложения `EXECUTE AS` изменять параметры безопасности, с которыми выполняется объект. У предложения `EXECUTE AS` есть три возможных аргумента:

- ◆ `LOGIN` — объект выполняется с настройками заданного регистрационного имени;
- ◆ `USER` — объект выполняется с параметрами безопасности заданного пользователя базы данных. Учетная запись не может быть ролью, группой или асимметричным ключом (asymmetric key);
- ◆ `CALLER` — объект выполняется с параметрами безопасности подпрограммы, которая вызвала модуль.

У предложения `EXECUTE AS` есть два дополнительных аргумента: `NO REVERT` и `COOKIE INTO`. Параметр `NO REVERT` указывает на то, что после того, как настройки были изменены, вернуть их обратно нельзя. Параметр `COOKIE INTO` задает файл cookies, который позволяет вернуться в среду (контекст), содержащую предыдущие параметры безопасности.

## Курсоры

SQL Server создан для обработки наборов данных. Однако бывают ситуации, когда необходимо обрабатывать данные построчно. Результат инструкции `SELECT` возвращается в серверный объект, называемый *курсором*, который позволяет обращаться к каждой строке результирующего набора поочередно и даже перемещаться вперед и назад по строкам набора.

### ***Производительность курсора***

SQL Server разработан и оптимизирован для операций над наборами данных или множествами. Курсор заставляет механизм управления базой данных выполнять построчную обработку. Курсор никогда не будет столь же эффективен, как эквивалентная обработка на основе множеств.

У курсоров есть пять компонентов. Инструкция `DECLARE` применяется для задания инструкции `SELECT`, которая служит основой для формирования строк курсора. Инструкция `OPEN` вызывает выполнение инструкции `SELECT` и загрузку строк в структуру памяти. Инструкция `FETCH` применяется для извлечения одной строки из курсора. Инструкция `CLOSE` завершает обработку курсора. Инструкция `DEALLOCATE` удаляет курсор и освобождает структуры памяти, содержащие результирующий набор курсора.

### ***ВАЖНАЯ ИНФОРМАЦИЯ.***

#### ***Удаление курсоров***

Если курсор используется в хранимой процедуре, нет необходимости закрывать и удалять его. Когда хранимая процедура завершается, SQL Server автоматически закрывает и удаляет курсоры, созданные в процедуре, освобождая память для последующего использования.

### ***Использование курсора***

Если вы пишете курсор, выполняющий одну и ту же операцию над каждой строкой, извлеченной курсором, следует изменить обработку и воспользоваться более эффективной операцией на основе наборов данных.

Далее приведена общая синтаксическая запись объявления курсора.

```
DECLARE cursor_name CURSOR [LOCAL | GLOBAL]
 [FORWARD_ONLY | SCROLL]
 [STATIC | KEYSET | DYNAMIC | FAST_FORWARD]
 [READ_ONLY | SCROLL_LOCKS | OPTIMISTIC]
 [TYPE_WARNING]
FOR select_statement
 [FOR UPDATE [OF column_name [,...n]]]
```

В следующих трех инструкциях показаны три разных способа объявления одного и того же курсора.

```
DECLARE curproducts CURSOR FAST_FORWARD FOR
 SELECT ProductID, ProductName, ListPrice FROM Products.Product
GO
```

```
DECLARE curproducts CURSOR READ_ONLY FOR
 SELECT ProductID, ProductName, ListPrice FROM Products.Product
GO
```

```
DECLARE curproducts CURSOR FOR
 SELECT ProductID, ProductName, ListPrice FROM Products.Product
FOR READ ONLY
GO
```

После объявления курсора вводится команда `OPEN` для выполнения инструкции `SELECT`.

```
OPEN curproducts
```

Далее необходимо извлечь данные из строки курсора с помощью инструкции `FETCH`. Когда инструкция `FETCH` выполняется первый раз, указатель помещен на первую строку результирующего набора курсора. После каждого выполнения инструкции `FETCH` курсор перемещается на следующую строку в результирующем наборе, пока не иссякнут все строки. Каждое выполнение инструкции `FETCH` также устанавливает значение глобальной переменной `@@FETCH_STATUS`. Обычно для многократной обработки курсора используется цикл `WHILE`, извлекающий одну строку в каждом проходе тела цикла. Цикл `WHILE` выполняется до тех пор, пока `@@FETCH_STATUS = 0`. Далее приведен пример.

```
DECLARE @ProductID INT,
 @ProductName VARCHAR(50),
 @ListPrice MONEY

DECLARE curproducts CURSOR FOR
 SELECT ProductID, ProductName, ListPrice FROM Products.Product
FOR READ ONLY

OPEN curproducts

FETCH curproducts INTO @ProductID, @ProductName, @ListPrice

WHILE @@FETCH_STATUS = 0
BEGIN
 SELECT @ProductID, @ProductName, @ListPrice
 FETCH curproducts INTO @ProductID, @ProductName, @ListPrice
END

CLOSE curproducts
DEALLOCATE curproducts
```

### **Обработка на базе наборов или множеств**

Если вы пишете хранимые процедуры, содержащие курсоры (особенно многоуровневые курсоры), следует еще раз оценить алгоритм, который вы описываете. Возможно, вы сумеете заменить курсоры обработкой на основе наборов или множеств, которые более эффективны.

Вы можете объявлять курсоры четырех разных типов.

- ◆ **FAST\_FORWARD** — самый быстродействующий тип курсора, поскольку позволяет перемещаться только вперед от одной строки к другой. Прокрутка (будет обсуждаться в этом разделе чуть позже) не поддерживается. Курсор **FAST\_FORWARD** эквивалентен курсору, объявленному как **FORWARD\_ONLY, READ\_ONLY**. **FAST\_FORWARD** — тип курсора, принятый по умолчанию.
- ◆ **STATIC** — результирующий набор курсора этого типа извлекается и сохраняется во временной таблице в базе данных **tempdb**. Все инструкции **FETCH** выполняются во временной таблице, и изменения таблиц, служащих основой для курсора, не видны. Курсор типа **STATIC** поддерживает прокрутку, но изменения для него запрещены.
- ◆ **KEYSET** — у этого типа курсора набор ключей, однозначно идентифицирующих каждую строку результирующего набора, сохраняется во временной таблице в базе данных **tempdb**. При прокрутке курсора неключевые столбцы извлекаются из таблиц, служащих основой курсора. Следовательно, любые изменения строк отражаются в курсоре по мере его прокрутки. Любые вставки в базовую таблицу не доступны для курсора. Если вы попытаетесь обратиться к строке, которая была удалена, переменная @@FETCH\_STATUS вернет значение -2.
- ◆ **DYNAMIC** — самый дорогостоящий тип курсора. В нем отражаются все изменения, сделанные в базовом результирующем наборе, включая вновь вставленные строки, появляющиеся по мере прокрутки курсора. Позицию строки и порядок строк в курсоре можно изменять при каждом выполнении инструкции **FETCH**. Для динамических курсоров недоступен параметр **FETCH ABSOLUTE**.

По умолчанию все курсоры обновляемы. Для внесения изменений в строки базовой таблицы можно выполнить инструкцию **UPDATE** или **DELETE** с предложением **WHERE CURRENT OF <cursor name>**, изменяющую или удаляющую строку лежащей в основе таблицы, на которую в данный момент указывает указатель курсора.

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### **Параметры курсора**

Если вы собираетесь всего лишь прочесть данные курсора, убедитесь в том, что вы объявили курсор, как доступный только для чтения. Курсоры только для чтения требуют меньше накладных расходов, чем обновляемые курсоры.

Доступ к строкам в курсоре можно ограничить с помощью параметров **FORWARD\_ONLY** и **SCROLL**. Если курсор объявлен как **FORWARD\_ONLY**, каждая строка

может быть прочитана один раз по мере продвижения указателя курсора в результирующем наборе. Если вы объявляете курсор с параметром `SCROLL`, у инструкции `FETCH` появляются следующие варианты:

- ◆ `FETCH FIRST` — извлечение первой строки в результирующем наборе;
- ◆ `FETCH LAST` — извлечение последней строки в результирующем наборе;
- ◆ `FETCH NEXT` — извлечение из результирующего набора следующей строки по отношению к текущей позиции указателя. `FETCH NEXT` эквивалентна выполнению просто инструкции `FETCH`, которая также перемещается в результирующем наборе каждый раз на одну строку вперед;
- ◆ `FETCH PRIOR` — извлечение из результирующего набора строки, непосредственно предшествующей текущей позиции указателя курсора;
- ◆ `FETCH ABSOLUTE n` — извлечение  $n$ -й строки, считая от начала результирующего набора;
- ◆ `FETCH RELATIVE n` — извлечение  $n$ -й строки, считая от текущей позиции указателя в результирующем наборе курсора.

В языке T-SQL для курсоров есть три параметра, обеспечивающих параллелизм:

- ◆ `READ_ONLY` — SQL Server не запрашивает блокировку на базовую строку в таблице, т. к. курсор помечен как `READ_ONLY` и не может быть изменен;
- ◆ `SCROLL LOCKS` — запрашивается блокировка при считывании каждой строки в курсор, гарантируя успешное выполнение транзакции, работающей с курсором;
- ◆ `OPTIMISTIC` — блокировка не запрашивается. Вместо этого SQL Server использует либо метку времени (timestamp), либо в случае отсутствия столбца с меткой времени вычисляемую контрольную сумму для того, чтобы определить, изменялись ли данные после того, как были считаны в курсор. Если данные были изменены, модификация завершается аварийно.

## Компиляция и перекомпиляция

Когда хранимая процедура создана, SQL Server проверяет синтаксис, но не объекты, на которые есть ссылки в процедуре. Когда хранимая процедура выполняется в первый раз, SQL Server выполняет синтаксический анализ программного кода и компилирует его. Во время первой компиляции процедуры выполняется проверка, гарантирующая, что все объекты, на которые есть ссылки, существуют или будут созданы в процедуре прежде, чем к ним произойдет обращение.

Во время компиляции генерируется план запроса и сохраняется в кэше запроса. Этот план, полученный в результате компиляции, реентерабельный и используется повторно при каждом выполнении процедуры. Каждое одновременно про-

исходящее выполнение хранимой процедуры также генерирует план для выполнения запроса, называемый *планом выполнения* (execution plan). Планы выполнения также хранятся в кэше запроса, но они не реентерабельные. Как только соединение завершило выполнение процедуры, план запроса, хранящийся в кэше запроса, может быть назначен другому соединению, которое выполняет эту процедуру. Следовательно, одна хранимая процедура, которая активно используется, может иметь в кэше запроса единственный план компиляции и много планов выполнения, каждый из которых занимает место в памяти.

В большинстве случаев сохранение плана компиляции (плана запроса) для повторного использования отнимает ресурсы, которые требуются для генерации плана выполнения при каждом выполнении процедуры. Однако если в хранимой процедуре есть несколько ветвей программного кода, зависящих от проверочных условий, для нее могут быть сгенерированы сильно отличающиеся друг от друга планы запроса. При применении одного и того же плана запроса для каждого выполнения в некоторых случаях может пострадать производительность.

Если хранимая процедура большую часть времени во время выполнения генерирует разные планы, следует создавать процедуру, используя параметр `RECOMPILE`. Если для хранимой процедуры применяется параметр `RECOMPILE`, SQL Server не сохраняет в кэше план компиляции запроса и не использует его повторно.

В дополнение к параметру `RECOMPILE` SQL Server может обнаружить во время выполнения процедуры устаревшие статистические данные. Если во время выполнения процедуры SQL Server определит, что план запроса хуже оптимального, выполнение приостанавливается до тех пор, пока не будет сгенерирован новый план запроса для следующей выполняемой в процедуре инструкции. До версии SQL Server 2005 заново генерировался план запроса для всей процедуры целиком. Но, начиная с SQL Server 2005, перекомпиляция проводится на уровне отдельной инструкции.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### **Перекомпиляция**

Одно из преимуществ процедур и функций — способность SQL Server хранить в кэше планы запросов для последующих выполнений программного кода. Несмотря на то, что вам хочется максимально использовать кэш, существуют ситуации, в которых последующим выполнениям для достижения оптимального выполнения процедуры могут потребоваться разные планы выполнения. Если сгенерированный план запроса зависит от значений, используемых в каждом выполнении, следует применять параметр `RECOMPILE`, чтобы заставить SQL Server оптимизировать процедуру или функцию при каждом их выполнении.

Более эффективный метод — разделение хранимой процедуры на несколько процедур. Например, у вас могла бы быть следующая процедура, в которой выполнение разных ветвей в инструкции `IF` генерирует резко отличающиеся планы запроса.



```
CREATE PROCEDURE PROC1
AS
 IF <некоторое условие>
 BEGIN
 <блок кода А>
 END
 ELSE
 BEGIN
 <блок кода В>
 END
GO
```

Вы можете выиграть от использования кэша и отказа от неоптимальных планов запроса, если создадите отдельную хранимую процедуру для каждой ветви условной проверки следующим образом.

```
CREATE PROCEDURE PROCA
AS
 <блок кода А>
GO
CREATE PROCEDURE PROCB
AS
 <блок кода В>
GO
CREATE PROCEDURE PROC1
AS
 IF <некоторое условие>
 BEGIN
 EXEC PROCA
 END
 ELSE
 BEGIN
 EXEC PROCB
 END
GO
```

Когда выполняется `PROC1`, генерируется очень простой план компиляции, который может повторно применяться независимо от выбранной ветви программного кода. Когда в первый раз выполняется `PROCA`, SQL Server проводит синтаксический анализ, компилирует и сохраняет в кэше план компиляции, который может повторно использоваться каждый раз, когда выполняется ветвь А программного кода. Аналогично, когда первый раз выполняется процедура `PROCB`, в кэше сохраняется план компиляции, который также в дальнейшем может повторно применяться.

---

### ***32-разрядный SQL Server по сравнению с 64-разрядным***

---

Когда возникает вопрос: 32- или 64-разрядную версию SQL Server устанавливать, многие люди ошибочно считают, что 64-разрядная версия автоматически повысит

производительность их приложений. Максимальный выигрыш получают те приложения, которые ограничивают размер кэша запроса, а не кэш данных.

Кэш запроса содержит исполняемый программный код, который в 32-разрядной версии не может занимать более 4 Гбайт, и одновременно SQL Server ограничивает максимальный размер кэша запроса примерно 20% памяти, отведенной экземпляру SQL Server. Приложения с большим количеством параллельно работающих пользователей могут быстро заполнить планами компиляции и выполнения всю память, отведенную для кэша запроса, заставляя любые дополнительные запросы ждать, пока не освободится память в кэше запросов.

Установив 64-разрядную версию SQL Server, вы избавляетесь от 4-гигабайтного ограничения для исполняемого кода и увеличиваете объем памяти, доступной для кэша запроса. Благодаря увеличению доступной памяти экземпляр сервера может обрабатывать большее количество параллельных выполнений, тем самым повышая производительность.

---

### **Быстрая проверка**

---

#### **ВОПРОСЫ:**

1. Курсоры каких четырех типов вы можете создать?
2. Как ведет себя параметр `XACT_ABORT` в блоке `TRY`?

#### **ОТВЕТЫ:**

1. Можно создать курсоры следующих четырех типов: `FAST_FORWARD`, `STATIC`, `KEYSET` и `DYNAMIC`.
2. Если в блоке `TRY` установлен параметр `XACT_ABORT`, то когда возникает ошибка, управление передается в блок `CATCH`. Но транзакция остается в обреченном состоянии и не может быть зафиксирована в блоке `CATCH`.

## **ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.**

### **Создание хранимой процедуры**

На этом практическом занятии вы создадите хранимые процедуры.

#### **Упражнение. Создаем хранимую процедуру**

В этом упражнении вы создадите две хранимые процедуры, чтобы сравнить эффективность обработки в подпрограмме, ориентированной на наборы данных, и подпрограмме, основанной на курсоре.

1. Откройте окно создания запроса и измените настройки на использование базы данных AdventureWorks.
2. В существующем окне запроса введите, выделите и выполните следующий программный код, создающий основанную на курсоре хранимую процедуру, которая изменяет дату приема на работу у всех сотрудников.

```
CREATE PROCEDURE HumanResources.UpdateAllEmployeeHireDateInefficiently
AS
BEGIN TRY
 SET XACT_ABORT ON
```

```

DECLARE curemployee CURSOR FOR SELECT EmployeeID FROM
 HumanResources.Employee
OPEN curemployee
FETCH FROM curemployee

WHILE @@FETCH_STATUS = 0
BEGIN
 UPDATE HumanResources.Employee
 SET HireDate = GETDATE()
 WHERE CURRENT OF curemployee

 FETCH FROM curemployee
END
END TRY
BEGIN CATCH
 ROLLBACK TRANSACTION
 PRINT 'An error occured, transaction rolled back'
END CATCH
GO

```

3. В имеющемся окне запроса введите, выделите и выполните следующий программный код для создания хранимой процедуры, изменяющей дату приема на работу у всех сотрудников, используя подход, основанный на множествах.

```

CREATE PROCEDURE HumanResources.UpdateAllEmployeeHireDateEfficiently
AS
DECLARE @now DATETIME = GETDATE()

BEGIN TRY
 SET XACT_ABORT ON

 UPDATE HumanResources.Employee
 SET HireDate = @now
END TRY
BEGIN CATCH
 ROLLBACK TRANSACTION
 PRINT 'An error occured, transaction rolled back'
END CATCH
GO

```

4. Сравните обе процедуры, выполнив следующий программный код.

```

EXEC HumanResources.UpdateAllEmployeeHireDateInefficiently
GO
EXEC HumanResources.UpdateAllEmployeeHireDateEfficiently
GO

```

## Резюме урока

- ♦ Хранимая процедура — это пакет кода на языке T-SQL, имеющий имя и хранящийся в базе данных.

- ◆ Вы можете передавать в хранимую процедуру параметры, как по имени, так в соответствии с порядком их следования. Вы также можете возвращать данные из процедуры с помощью выходных параметров.
- ◆ Для выполнения хранимой процедуры с особыми параметрами безопасности можно применять предложение `EXECUTE AS`.
- ◆ Курсоры позволяют обрабатывать данные построчно; но если вы вносите одно и то же изменение в каждую строку курсора, подход на основе наборов или множеств эффективнее.
- ◆ Блок `TRY...CATCH` обеспечивает вашим хранимым процедурам структурированную обработку ошибок.

## УРОК 2. Функции, определенные пользователем

*Функции* — это программируемые объекты, применяемые для выполнения вычислений, которые могут возвращаться в вызывающее приложение или вставляться в результирующий набор. Функции могут обращаться к данным и возвращать результаты, но они не могут вносить какие бы то ни было изменения. Из этого урока вы узнаете, как создавать функции, определенные пользователем, или пользовательские функции на языке T-SQL.

### ***После усвоения этого урока вы сможете***

Создавать и изменять функции, определенные пользователем (ФОП).

Расчетное время урока — 20 мин.

## Системные функции

SQL Server поставляется с обширным набором функций, которые вы можете использовать для выполнения множества операций. Встроенные функции можно разбить на 15 различных категорий (табл. 5.2).

**Таблица 5.2.** Системные функции

| Категория функций     | Назначение                                                                                                                                    |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Статистические        | Объединяют множество значений. Например, <code>SUM</code> , <code>AVG</code> , <code>COUNT_BIG</code> и <code>VAR</code>                      |
| Конфигурационные      | Возвращают сведения о настройках и параметрах системы. Например, <code>@@VERSION</code> , <code>@@SERVERNAME</code> и <code>@@LANGUAGE</code> |
| Криптографические     | Поддерживают шифрование и дешифрование                                                                                                        |
| Для обработки курсора | Возвращают информацию о состоянии курсора. Например, <code>@@FETCH_STATUS</code> и <code>@@CURSOR_ROWS</code>                                 |

Таблица 5.2 (окончание)

| Категория функций                       | Назначение                                                                                                                                                                                          |
|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Для обработки дат и времени суток       | Возвращают порции даты или времени суток или вычисляют даты и время суток. Примерами могут служить DATEADD, DATEPART, DATEDIFF и GETDATE                                                            |
| Управляющие или административные        | Возвращают данные для управления компонентами SQL Server. Примерами могут быть sys.dm_db_index_physical_stats, sys.dm_db_index_operational_stats и fn_trace_gettable                                |
| Математические                          | Выполняют математические операции. Примерами могут служить SIN, COS, TAN, LOG, PI и ROUND                                                                                                           |
| Для обработки метаданных                | Возвращают данные об объектах базы данных. Примерами могут быть OBJECT_NAME, OBJECT_ID, DATABASEPROPERTYEX и DB_NAME                                                                                |
| Ранжирующие                             | Возвращают значения, применяемые для ранжирования результирующих наборов. Ранжирующие функции описаны в главах 4 и 9                                                                                |
| Для формирования набора строк           | Возвращают результирующий набор, который можно соединить с другими таблицами. Примерами могут служить CONTAINS и FREETEXT                                                                           |
| Связанные с безопасностью               | Возвращают сведения о параметрах безопасности, относящиеся к пользователям и ролям. Например, SUSER_SNAME, Has_perms_by_name и USER_NAME                                                            |
| Строковые                               | Обрабатывают данные типов CHAR и VARCHAR. К примерам можно отнести POS, CHARINDEX, SOUNDEX, REPLACE, STUFF и RTRIM                                                                                  |
| Системные                               | Возвращают сведения о разных параметрах системы, базы данных и объекта, а также данные. Примерами могут служить DATALENGTH, HOST_NAME, ISDATE, ISNULL, SCOPE_IDENTITY, CAST и CONVERT               |
| Системной статистики                    | Возвращают оперативную информацию об экземпляре SQL Server. Например, fn_virtualfilestats и @@CONNECTIONS                                                                                           |
| Для обработки данных типов text и image | Обрабатывают данные, содержащие текст или изображение. Примерами могут служить TEXTPTR и TEXTVALID. Типы text и image устаревшие, поэтому функции этой категории не следует применять в приложениях |

Большинство системных функций хранится в базе данных mssqlsystemresource или master, или же они включены в библиотеки программного кода, поддерживаемые языком T-SQL.

## Функции, определенные пользователем

Вы можете создавать собственные функции, называемые пользовательскими или определенными пользователем (ФОП), и хранить их в любой базе данных, для которой у вас есть разрешение CREATE FUNCTION.

Несмотря на то, что функции обычно применяются для выполнения вычислений, им запрещено изменять состояние базы данных или экземпляра SQL Server.

Функции не могут выполнять следующие действия:

- ◆ изменять состояние экземпляра сервера или базы данных;
- ◆ модифицировать данные в таблице;
- ◆ вызывать функцию с внешним эффектом, такую как функция RAND;
- ◆ создавать временные таблицы и обращаться к ним;
- ◆ выполнять программный код динамически.

Функции могут возвращать скалярное или табличное значение. Функции с табличным значением (табличные) могут быть двух разных типов: встраиваемые или подставляемые (inline) и многооператорные (multi-statement).

Далее приведен общий вид синтаксической записи скалярной функции.

```
CREATE FUNCTION [schema_name.] function_name
([{ @parameter_name [AS] [type_schema_name.] parameter_data_type
 [= default] [READONLY] } [,...n]])
RETURNS return_data_type
 [WITH <function_option> [,...n]]
 [AS]
BEGIN
 <function_body>
 RETURN scalar_expression
END
```

Встраиваемая функция с табличным значением содержит единственную инструкцию SELECT, которая возвращает таблицу. Поскольку встраиваемая функция не выполняет никаких других операций, оптимизатор воспринимает ее как простое представление.

Далее приведен общий вид синтаксической записи встраиваемой функции с табличным значением.

```
CREATE FUNCTION [schema_name.] function_name
([{ @parameter_name [AS] [type_schema_name.] parameter_data_type
 [= default] [READONLY] } [,...n]])
RETURNS TABLE
 [WITH <function_option> [,...n]]
 [AS]
RETURN [(] select_stmt [)]
```

Общий вид синтаксической записи многооператорной функции с табличным значением выглядит следующим образом.

```
CREATE FUNCTION [schema_name.] function_name
([{ @parameter_name [AS] [type_schema_name.] parameter_data_type
 [= default] [READONLY] } [,...n]])
```

```
RETURNS @return_variable TABLE <table_type_definition>
[WITH <function_option> [,...n]]
[AS]
BEGIN
 <function_body>
RETURN
END
```

Программный код в теле любой функции следует завершать инструкцией `RETURN`. Из скалярной функции вы возвращаете единственное значение. В случае встраиваемой функции с табличным значением вы возвращаете инструкцию `SELECT`. В многооператорной табличной функции вы только вставляете в конце функции ключевое слово `RETURN`. За исключением встраиваемой табличной функции весь программный код в теле функции следует заключать в блок `BEGIN...END`.

Независимо от типа функции можно задавать четыре параметра: `ENCRYPTION`, `SCHEMABINDING`, `RETURNS NULL ON NULL INPUT/CALLED ON NULL INPUT` и `EXECUTE AS`. Параметры `ENCRYPTION`, `SCHEMABINDING` и `EXECUTE AS` можно задавать и для хранимых процедур. Кроме того, параметр `SCHEMABINDING` можно задавать для триггеров и представлений.

---

#### **Параметр `ENCRYPTION`**

Параметр `ENCRYPTION` ведет себя одинаково независимо от того, что вы создаете — хранимую процедуру, функцию, триггер или представление.

Применение параметра `SCHEMABINDING` гарантирует невозможность удаления связанных объектов. Например, если вы должны создать функцию, выполняющую инструкцию `SELECT` для таблицы `Sales.SalesOrderHeader`, обычно есть вероятность удаления таблицы без получения сообщения об ошибке. Во время следующего выполнения функции вы получите сообщение об ошибке, извещающее о том, что таблицы `Sales.SalesOrderHeader` не существует. Для того чтобы помешать удалению или изменению объектов, на которые полагаются программируемые объекты, вы задаете параметр `SCHEMABINDING`. Если вы попытаетесь удалить или изменить связанный объект, SQL Server помешает этому. Для того чтобы удалить или изменить связанный объект, сначала нужно удалить программируемый объект, зависящий от объекта, который вы хотите удалить или изменить.

---

#### **Параметр `SCHEMABINDING`**

Параметр `SCHEMABINDING` допускается в функциях и представлениях.

Для функции уникален параметр `RETURNS NULL ON NULL INPUT/CALLED ON NULL INPUT`, значение по умолчанию — `CALLED ON NULL INPUT`. Если вы задаете параметр функции со значением `NULL`, функция все равно вызывается и любой программный код в теле функции выполняется. Если задать параметр `RETURNS NULL`

ON NULL INPUT, при указании входного параметра со значением NULL функция не выполняется, и NULL немедленно передается в вызывающую подпрограмму. Если у вас есть функция, которая должна выполняться, только если вы передали ей параметры, не равные NULL, следует задать параметр RETURNS NULL ON NULL INPUT, и вы сможете избежать выполнения ненужного программного кода.

## Получение данных из функции

Получают данные из функции с помощью инструкции SELECT. Функции могут применяться в следующих компонентах:

- ◆ список инструкции SELECT;
- ◆ предложение WHERE;
- ◆ выражение;
- ◆ ограничение CHECK или DEFAULT;
- ◆ предложение FROM с оператором CROSS/OUTER APPLY.

Способ использования функции может оказывать существенное влияние на производительность выполняемых вами запросов.

Функция в списке SELECT применяется для вычисления итогов или выполнения вычислений на базе одного или нескольких столбцов из таблиц, указанных в предложении FROM. Функция в предложении WHERE используется для ограничения результирующего набора на основе результатов, полученных функцией.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### ***Применение функций в предложении WHERE***

Не следует создавать запросы, использующие функцию в предложении WHERE, потому что функция должна будет выполняться для каждой строки, которая может быть возвращена в результатах предложения FROM. Если вы написали инструкцию SELECT с оператором соединения для таблиц TableA и TableB, возвращающую 100 строк, соответствующих условию соединения, функция в предложении WHERE будет выполняться 100 раз. Если результат соединения порождает 10 000 строк, функция будет выполняться 10 000 раз. Даже если вы и не собираетесь повторно использовать программный код, вставив программный код из функции в вашу инструкцию SELECT, вы исключите всякую необходимость выполнения повторяющихся запросов.

Функции можно вкладывать друг в друга до тех пор, пока возвращаемое значение внутренней функции соответствует входному параметру внешней функции. Например, обычная подпрограмма грамматического анализа может содержать программный код, подобный следующему: DATALENGTH (POS (CHARINDEX (REPLACE (. . . )

В ограничениях CHECK и DEFAULT функции применяются для расширения возможностей статических расчетов. Например, если вы хотите проверить корректность кода региона в телефонном номере, сравнивая его со списком кодов региона, хранящемся в таблице, можно использовать функцию для выполнения проверки,



которая невозможна обычными средствами, т. к. ограничение CHECK не принимает инструкцию SELECT.

---

**Подсказка к экзамену**

Встраиваемая табличная функция ведет себя как представление и взаимозаменяема с ним.

---

**Быстрая проверка****ВОПРОСЫ:**

1. Функции каких трех типов вы можете создать?
2. Каковы обязательные элементы функции?

**ОТВЕТЫ:**

1. Вы можете создать скалярную функцию, которая возвращает единственное значение. Встраиваемую табличную функцию, которая содержит единственную инструкцию SELECT и интерпретируется так же, как представление, и многооператорную табличную функцию, возвращающую таблицу.
2. Все функции заканчиваются инструкцией RETURN. Скалярные функции вставляют возвращаемое значение сразу за инструкцией RETURN. Встраиваемая табличная функция включает инструкцию SELECT для возврата результирующего набора сразу следом за инструкцией RETURN. Многооператорные табличные функции просто завершаются ключевым словом RETURN. Тело функции, за исключением встраиваемых табличных функций, должно быть полностью заключено в блок BEGIN...END.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание функций

На этом практическом занятии вы создадите и примените функции трех разных типов.

### Упражнение 1. Создание скалярной функции

В этом упражнении вы создадите и примените функцию, которая возвращает текущий запас товара.

1. Откройте окно создания запроса, введите и выполните следующий программный код, создающий функцию.

```
CREATE FUNCTION Production.udf_GetProductInventory (@ProductID INT)
RETURNS INT
AS
-- На каждой стадии производства у товара есть свое место хранения.
-- Нас интересуют только законченные товары.
-- Товар также может храниться на нескольких полках и в нескольких
-- секциях, поэтому нам придется суммировать количества.
BEGIN
 DECLARE @Inventory INT
```

```
SELECT @Inventory = SUM(a.Quantity)
FROM Production.ProductInventory a
INNER JOIN Production.Location b ON a.LocationID = b.LocationID
WHERE a.ProductID = @ProductID
 AND b.Name IN ('Miscellaneous Storage','Finished Goods Storage')
IF (@Inventory IS NULL)
BEGIN
 SET @Inventory = 0
END
RETURN @Inventory
END
GO
```

2. В имеющемся окне запроса введите, выделите и выполните следующий код для возврата результатов из функции.

```
SELECT Production.udf_GetProductInventory (325)
GO
```

3. В имеющемся окне запроса введите, выделите и выполните следующий код для проверки результатов.

```
SELECT SUM(a.Quantity)
FROM Production.ProductInventory a
INNER JOIN Production.Location b ON a.LocationID = b.LocationID
WHERE a.ProductID = 325
 AND b.Name IN ('Miscellaneous Storage','Finished Goods Storage')
GO
```

## Упражнение 2. Создание встраиваемой табличной функции

В этом упражнении вы создадите и примените встраиваемую табличную функцию для возврата всех еще не доставленных заказов.

1. Откройте окно создания запроса, введите и выполните следующий программный код для создания недоставленного заказа.

```
UPDATE Sales.SalesOrderHeader
SET ShipDate = NULL
WHERE SalesOrderID = 75123
GO
```

2. В открытом окне запроса введите, выделите и выполните следующий код для создания функции.

```
CREATE FUNCTION Sales.GetUnshippedOrders()
RETURNS TABLE
AS
RETURN SELECT a.SalesOrderID, a.CustomerID, a.OrderDate, a.DueDate,
 c.Name, b.OrderQty
```

```
FROM Sales.SalesOrderHeader a
INNER JOIN Sales.SalesOrderDetail b
 ON a.SalesOrderID = b.SalesOrderID
INNER JOIN Production.Product c ON b.ProductID = c.ProductID
WHERE a.ShipDate IS NULL
```

GO

3. Протестируйте функцию. Включите отображение действующего плана выполнения, чтобы проверить, что SQL Server не выполнил функцию, а вместо этого подставил тело функции в инструкцию SELECT. В имеющемся окне запроса введите, выделите и выполните следующий программный код.

```
SELECT c.FirstName, c.MiddleName, c.LastName
FROM Sales.GetUnshippedOrders() a
INNER JOIN Sales.Customer b ON a.CustomerID = b.CustomerID
INNER JOIN Person.Person c ON b.PersonID = c.BusinessEntityID
GO
```

4. В имеющемся окне запроса введите, выделите и выполните следующий код, возвращающий исходные значения данным.

```
UPDATE Sales.SalesOrderHeader
SET ShipDate = '08/07/2004'
WHERE SalesOrderID = 75123
GO
```

### Упражнение 3. Создание многооператорной табличной функции

В этом упражнении вы создадите и примените многооператорную табличную функцию для возврата последнего заказа каждого клиента.

1. Откройте окно создания запроса, введите, выделите и выполните следующий программный код, создающий функцию.

```
CREATE FUNCTION Sales.GetLastShippedCustomerOrder (@CustomerID INT)
RETURNS @CustomerOrder TABLE
(SalesOrderID INT NOT NULL,
 CustomerID INT NOT NULL,
 OrderDate DATETIME NOT NULL,
 DueDate DATETIME NOT NULL,
 Name NVARCHAR(50) NOT NULL,
 OrderQty INT NOT NULL)
AS
BEGIN
 DECLARE @MaxOrderDate DATETIME

 SELECT @MaxOrderDate = MAX(OrderDate)
 FROM Sales.SalesOrderHeader
 WHERE CustomerID = @CustomerID
```

```
INSERT @CustomerOrder
SELECT a.SalesOrderID, a.CustomerID, a.OrderDate, a.DueDate,
 c.Name, b.OrderQty
FROM Sales.SalesOrderHeader a INNER JOIN Sales.SalesOrderDetail b
 ON a.SalesOrderID = b.SalesOrderID
INNER JOIN Production.Product c ON b.ProductID = c.ProductID
WHERE a.OrderDate = @MaxOrderDate
 AND a.CustomerID = @CustomerID
RETURN
END
GO
```

2. В имеющемся окне запроса введите, выделите и выполните следующий код для тестирования функции.

```
SELECT * FROM Sales.GetLastShippedCustomerOrder(11000)
GO
SELECT * FROM Sales.SalesOrderHeader WHERE CustomerID = 11000
GO
```

## Резюме урока

- ♦ Вы можете создавать скалярные функции, встраиваемые функции и многооператорные функции, возвращающие табличные значения.
- ♦ Тело всех функций, за исключением встраиваемых табличных функций, должно быть заключено в блок `BEGIN...END`.
- ♦ Все функции должны завершаться инструкцией `RETURN`.
- ♦ Функциям запрещено изменять состояние базы данных или экземпляра SQL Server.

## УРОК 3. Триггеры

*Триггер* — это хранимая процедура специального типа, запускаемая автоматически, когда выполняется связанная с ним инструкция языка DML или DDL. Из этого урока вы узнаете, как создать триггеры DML, которые выполняются, когда вы добавляете, изменяете или удаляете строки в таблице. Вы также научитесь создавать триггеры DDL, которые выполняются, когда выполняются команды DDL или пользователи регистрируются в экземпляре SQL Server.

---

### ***После усвоения этого урока вы сможете***

- Создавать триггеры DML.
- Создавать триггеры DDL.

Расчетное время урока — 20 мин.

## Триггеры DML

Несмотря на то, что триггер — программируемый объект, создаваемый вами, выполнить его напрямую вы не сможете. Триггеры DML обрабатывают таблицу или представление и задаются для определенного события: INSERT, UPDATE или DELETE. Когда вы инициируете событие, для которого задан триггер, SQL Server автоматически выполняет программный код триггера или, как говорят, "запускает" триггер.

Общий вид синтаксической записи создания триггера выглядит следующим образом.

```
CREATE TRIGGER [schema_name .] trigger_name
ON { table | view }
[WITH <dml_trigger_option> [,...n]]
{ FOR | AFTER | INSTEAD OF }
{ [INSERT] [,] [UPDATE] [,] [DELETE] }
[WITH APPEND]
[NOT FOR REPLICATION]
AS { sql_statement [;] [,...n] |
EXTERNAL NAME <method_specifier[;] > }
```

Если триггер определен как AFTER, он запускается после того, как модификация преодолевает все ограничения. Если модификация не удовлетворяет ограничению, такому как проверочное, первичного или внешнего ключа, триггер не запускается.

### ***Множественные триггеры для одного и того же действия***

Триггеры AFTER определяются только для таблиц, и для одного и того же действия может быть определено несколько триггеров AFTER. Если у вас есть несколько триггеров для одного и того же действия, можно задать с помощью системной хранимой процедуры `sp_settriggerorder` триггеры, запускаемые в первую и последнюю очередь. Все остальные триггеры для одного и того же действия будут выполняться в случайном порядке.

Триггер, определенный с предложением INSTEAD OF, вызывает выполнение программного кода триггера вместо инструкций INSERT, UPDATE или DELETE. Вы можете определить один триггер INSTEAD OF для заданного действия. Хотя триггеры INSTEAD OF можно создавать и для таблиц, и для представлений, почти всегда они создаются для представлений.

Триггер запускается только один раз для каждого действия независимо от количества обрабатываемых строк.

Параметр NOT FOR REPLICATION управляет поведением триггера при внесении изменений механизмом репликации (replication engine). По умолчанию любые действия INSERT, UPDATE или DELETE, выполняемые механизмом репликации в подписчике (subscriber), вызывают запуск соответствующего триггера. Если вы не

хотите, чтобы триггер запускался в подписчике, когда механизм репликации вносит изменения, можно задать параметр `NOT FOR REPLICATION`. Если изменение будет вноситься любым другим процессом, отличным от механизма репликации, триггер будет запускаться.

Во время выполнения триггера доступны две специальные таблицы, именуемые `inserted` и `deleted`. Это те же специальные таблицы `inserted` и `deleted`, которые обсуждались в *главе 2*.

## Триггеры DDL

Триггеры DDL могут запускаться, когда выполняется инструкция DDL или когда пользователь регистрируется в экземпляре SQL Server.

У триггера DDL следующий общий вид синтаксической записи.

```
CREATE TRIGGER trigger_name
ON { ALL SERVER | DATABASE
}
[WITH <ddl_trigger_option> [,...n]]
{ FOR | AFTER } { event_type | event_group } [,...n]
AS { sql_statement [;] [,...n] | EXTERNAL NAME <method specifier>
 [;] }
```

`<ddl_trigger_option> ::= [ ENCRYPTION ] [ EXECUTE AS Clause ]`

`<method specifier> ::= assembly_name.class_name.method_name`

Область действия триггеров DDL может быть ограничена базой данных или экземпляром сервера. Для задания области действия триггера DDL на уровне экземпляра применяется параметр `ON ALL SERVER`. Для ограничения области действия триггера базой данных используют параметр `ON DATABASE`.

Далее приведен пример триггера DDL.

```
CREATE TRIGGER tddl_tabledropalterprevent
ON DATABASE
FOR DROP_TABLE, ALTER_TABLE
AS
 PRINT 'You are attempting to drop or alter tables in production!'
 ROLLBACK;
```

### ***Рабочая среда транзакции***

Почти все команды DDL выполняются в среде транзакции. Поскольку триггер DDL действует в той же среде транзакции, можно выполнить откат любой инструкции DDL, выполняющейся в среде транзакции. `ALTER DATABASE` — одна из команд, выполняющаяся не в среде транзакции, т. к. эта команда воздействует на объекты за пределами SQL Server, не подчиняющиеся транзакционной семантике. Следовательно, нельзя выполнить откат команды `ALTER DATABASE`.

Значение типа события наследуется из выполняемой в данный момент инструкции DDL. В табл. 5.3 приведено несколько примеров.

**Таблица 5.3. Примеры команд DDL**

| Команда DDL       | Тип события       |
|-------------------|-------------------|
| CREATE DATABASE   | CREATE_DATABASE   |
| DROP LOGIN        | DROP_LOGIN        |
| UPDATE STATISTICS | UPDATE_STATISTICS |
| DROP TRIGGER      | DROP_TRIGGER      |
| ALTER TABLE       | ALTER_TABLE       |

### **События, связанные с безопасностью**

Вы можете запускать триггер DDL, когда предоставляете, отменяете или отвергаете разрешения, как на уровне сервера, так и на уровне базы данных. Для выполнения триггера DDL с инструкцией, касающейся разрешений на уровне базы данных, применяйте события типов GRANT\_DATABASE, REVOKE\_DATABASE или DENY\_DATABASE. Для выполнения триггера DDL, касающегося разрешений на уровне сервера, используйте события следующих типов: GRANT\_SERVER, REVOKE\_SERVER и DENY\_SERVER.

События разных типов свернуты в командную иерархию, именуемую *группами событий*. Например, события типов CREATE\_TABLE, ALTER\_TABLE и DROP\_TABLE входят в группу событий DDL\_TABLE\_EVENTS. Типы событий и группы событий позволяют создавать гибкие и компактные триггеры DDL.

### **Дополнительная информация о группах событий**

События и связанные с ними группы, допустимые в триггерах DDL, можно найти в разд. "Event Groups for Use with DDL Triggers" (Группы DDL-событий) электронной документации по SQL Server.

В то время как у триггеров DML есть доступ к таблицам inserted и deleted, триггеры DDL могут обращаться к функции EVENTDATA. Функция EVENTDATA возвращает документ на языке Extensible Markup Language (XML, расширяемый язык разметки). Этот документ можно запрашивать с помощью метода value(), доступного через язык запросов XQUERY.

```
<EVENT_INSTANCE>
 <EventType>type</EventType>
 <PostTime>date-time</PostTime>
 <SPID>spid</SPID>
 <ServerName>name</ServerName>
 <LoginName>name</LoginName>
 <UserName>name</UserName>
 <DatabaseName>name</DatabaseName>
 <SchemaName>name</SchemaName>
```

```

 <ObjectName>name</ObjectName>
 <ObjectType>type</ObjectType>
 <TSQLCommand>command</TSQLCommand>
</EVENT_INSTANCE>

```

Содержимое документа XML зависит от типа события, вызвавшего запуск триггера DDL. Схемы XML для всех событий DDL зафиксированы по адресу <http://schemas.microsoft.com/sqlserver>. Например, для событий типа CREATE\_TABLE, ALTER\_TABLE или DROP\_TABLE вы можете применить следующий запрос, извлекающий базу данных, схему, объект и выполняемую команду.

```

SELECT EVENTDATA().value
 (' (/EVENT_INSTANCE/DatabaseName) [1]', 'nvarchar(max)'),
EVENTDATA().value
 (' (/EVENT_INSTANCE/SchemaName) [1]', 'nvarchar(max)'),
EVENTDATA().value
 (' (/EVENT_INSTANCE/ObjectName) [1]', 'nvarchar(max)'),
EVENTDATA().value
 (' (/EVENT_INSTANCE/TSQLCommand) [1]', 'nvarchar(max)')

```

## Триггеры входа

Помимо отклика на события DDL, можно создать триггер, запускающийся при регистрации в экземпляре SQL Server. Триггеры входа (logon trigger) запускаются после успешной аутентификации, но до того, как будет действительно установлен сеанс пользователя. Из триггера входа невозможно вернуть пользователю любые сообщения.

Далее приведен общий вид синтаксической записи триггера входа.

```

CREATE TRIGGER trigger_name
ON ALL SERVER
[WITH <logon_trigger_option> [,...n]]
{ FOR | AFTER } LOGON
AS { sql_statement [;] [,...n] | EXTERNAL NAME <method specifier>
 [;] }
<logon_trigger_option> ::= [ENCRYPTION] [EXECUTE AS Clause]

```

Триггеры входа применяются для контроля и ограничения доступа. Например, вы можете ограничить количество соединений, которые пользователям разрешено устанавливать с экземпляром сервера. Если в триггере входа выполняется инструкция ROLLBACK, соединение с экземпляром сервера разрывается.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### **Обслуживание базы данных**

Одна из самых трудных задач, с которыми сталкиваются администраторы баз данных, особенно в случае Web-приложений, — вывод всех пользователей из базы данных для выполнения ее технического обслуживания. Независимо от того, на-



сколько тщательно вы все спланировали, неизбежно будут установлены неожиданные соединения. Как только вы "убьете" все соединения с базой данных, возникнет новое соединение, вовлекая вас в нескончаемый цикл. Один из способов заблокировать доступ — запретить любые регистрации пользователей, но изменение состояния регистрации и последующее снятие блокировки регистраций может привести к аварийному завершению проверок политики (policy checks), создав неожиданный сбой. Гораздо лучше создать триггер входа, отвергающий попытки регистрации во время работ по техническому обслуживанию. После проведения технических работ вам нужно будет только отключить триггер входа, чтобы восстановить доступ к базе данных.

### ***Подсказка к экзамену***

При подготовке к экзамену убедитесь в том, что вы знаете, как с помощью функции EVENTDATA извлекать информацию из триггера DDL.

### ***Быстрая проверка***

#### **ВОПРОСЫ:**

1. Триггеры каких трех типов вы можете создавать?
2. Как получить доступ к информации о причине события в триггере DDL?

#### **ОТВЕТЫ:**

1. Вы можете создавать триггеры DML, DDL и входа.
2. Вам следует запросить документ XML, возвращаемый функцией EVENTDATA, включенной в тело триггеров DDL и входа, для получения информации о событии, вызвавшем запуск триггера. У каждого события своя схема XML. Описание всех схем SQL Server можно найти по адресу <http://schemas.microsoft.com/sqlserver>.

## **ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание триггеров**

На этом практическом занятии вы создадите триггеры DML, DDL и входа.

### **Упражнение 1. Создание триггера DML**

В этом упражнении вы создадите триггер DML для сохранения и контрольного отслеживания изменений.

1. Откройте окно создания запроса, введите и выполните следующий программный код для создания базовой таблицы контрольного отслеживания (audit trail).

```
CREATE TABLE Production.ProductAuditTrail
(AuditID INT IDENTITY(1,1),
AuditDate DATETIME NOT NULL,
ChangeUser SYSNAME NOT NULL,
ProductID INT NOT NULL,
BeforeListPrice MONEY NOT NULL,
AfterListPrice MONEY NOT NULL)
GO
```

2. В имеющемся окне запроса введите, выделите и выполните следующий код для создания триггера контроля.

```
CREATE TRIGGER tu_ProductAuditTrail
ON Production.Product
FOR UPDATE
AS
INSERT INTO Production.ProductAuditTrail
(AuditDate, ChangeUser, ProductID, BeforeListPrice, AfterListPrice)
SELECT GETDATE(), SUSER_SNAME(), i.ProductID, d.ListPrice, i.ListPrice
FROM inserted i INNER JOIN deleted d ON i.ProductID = d.ProductID
GO
```

3. В имеющемся окне запроса введите, выделите и выполните следующий код для тестирования триггера контроля.

```
SELECT * FROM Production.ProductAuditTrail
GO
UPDATE Production.Product
SET ListPrice = ListPrice + 1
WHERE ProductID = 514
GO
SELECT * FROM Production.ProductAuditTrail
GO
```

## Упражнение 2. Создание триггера DDL

В этом упражнении вы создадите триггер DDL, препятствующий удалению таблиц.

1. Откройте окно создания запроса, введите и выполните следующий программный код для создания триггера DDL.

```
CREATE TRIGGER ddl_preventtabledrop
ON DATABASE
FOR DROP_TABLE
AS
PRINT 'Prevention of an accidental table drop.'
PRINT 'You are attempting to drop a table in a production database.'
PRINT 'If you really want to drop this table, please disable the DDL
 trigger.'
PRINT 'Once the table has been dropped, please re-enable the DDL
 trigger.'
ROLLBACK TRANSACTION
GO
```

2. В имеющемся окне запроса введите, выделите и выполните следующий код для тестирования триггера DDL.

```
CREATE TABLE dbo.Test (ID INT NOT NULL)
GO
```

```
SELECT * FROM dbo.Test
GO
```

```
DROP TABLE dbo.Test
GO
```

```
SELECT * FROM dbo.Test
GO
```

3. В имеющемся окне запроса введите, выделите и выполните следующий код для отключения триггера и проверки того, что удаление таблицы завершилось успешно.

```
DISABLE TRIGGER ddl_preventtabledrop ON DATABASE
GO
```

```
DROP TABLE dbo.Test
GO
```

```
SELECT * FROM dbo.Test
GO
```

```
ENABLE TRIGGER ddl_preventtabledrop ON DATABASE
GO
```

### Упражнение 3. Создание триггера входа

В этом упражнении вы создадите триггер, который может применяться для блокировки пользователей при попытке соединения с экземпляром SQL Server.

1. Откройте окно создания запроса, введите и выполните следующий программный код для создания триггера входа.

```
CREATE TRIGGER ddl_preventlogon
ON ALL SERVER
FOR LOGON
AS
IF IS_SRVROLEMEMBER('sysadmin',ORIGINAL_LOGIN()) = 0
BEGIN
 ROLLBACK
END
GO
```

2. В имеющемся окне запроса введите, выделите и выполните следующий код для создания тестовой учетной записи.

```
CREATE LOGIN TriggerTest WITH PASSWORD = '<InsertStrongPasswordHere>'
GO
```

3. Попробуйте зарегистрироваться с помощью только что созданной вами учетной записи.
4. Отключите триггер и снова сделайте попытку зарегистрироваться.

```
DISABLE TRIGGER ddl_preventlogon ON ALL SERVER
GO
```

## Резюме урока

- ◆ Триггеры — это особые хранимые процедуры, которые запускаются автоматически в ответ на события DDL или DML.
- ◆ Вы можете создавать триггеры трех типов: DML, DDL и входа.
- ◆ Триггер DML запускается, когда встречается инструкция `INSERT`, `UPDATE` или `DELETE`, для которой написан триггер.
- ◆ Триггер DDL запускается, когда встречается инструкция DDL, для которой написан триггер.
- ◆ Триггер входа выполняется, когда делается попытка регистрации.
- ◆ В триггере DML у вас есть доступ к специальным таблицам `inserted` и `deleted`.
- ◆ В триггере DDL или входа у вас есть доступ к документу на языке XML, предоставляемому функцией `EVENTDATA`.

## УРОК 4. Представления

В *главах 1 и 4* вы узнали о разных способах формирования инструкций `SELECT`, применяемых для извлечения данных. Некоторые из создаваемых вами инструкций `SELECT` используются для решения конкретной одноразовой задачи, а другие инструкции `SELECT` применяются в вашей рабочей среде снова и снова. Некоторые запросы, многократно используемые в вашей рабочей среде, содержат сложную бизнес-логику и сложный программный код на языке T-SQL, которые вы не хотите воспроизводить каждый раз, когда понадобится запрос. SQL Server позволяет сохранять инструкцию `SELECT` в базе данных с помощью объекта, называемого *представлением*. Из этого урока вы узнаете, как создавать представление, изменять данные с помощью представления и индексировать представление для повышения производительности.

---

### ***После усвоения этого урока вы сможете***

Создавать и изменять представления.

Расчетное время урока — 20 мин.

## Создание представления

*Представление* — это просто инструкция `SELECT` с заданным именем, хранящаяся в базе данных. Главное преимущество представления состоит в том, что после создания оно действует как таблица в отношении всех остальных инструкций `SELECT`, которые вы захотите написать.

У синтаксической записи создания представления следующий общий вид.

```
CREATE VIEW [schema_name .] view_name [(column [,...n])]
[WITH <view_attribute> [,...n]]
AS select_statement
[WITH CHECK OPTION] [;]
```

Определяемая для представления инструкция `SELECT` может ссылаться на таблицы, другие представления и функции, но не может делать следующего:

- ◆ содержать предложение `COMPUTE` или `COMPUTE BY`;
- ◆ создавать постоянную или временную таблицу с помощью ключевого слова `INTO`;
- ◆ применять предложение `OPTION`;
- ◆ ссылаться на временную таблицу;
- ◆ ссылаться на переменную любого типа;
- ◆ содержать предложение `ORDER BY`, если не задан параметр `TOP`.

Представление может содержать множественные инструкции `SELECT`, пока вы применяете операторы `UNION` или `UNION ALL`.

Для представления можно задавать атрибуты `ENCRYPTION`, `SCHEMABINDING` и `VIEW_METADATA`. Атрибуты `ENCRYPTION` и `SCHEMABINDING` применяются так же, как описывалось ранее в этой главе. Параметр `VIEW_METADATA` используется при создании обновляемого представления и заставляет SQL Server возвращать в клиентские приложения метаданные о представлении вместо таблиц, на базе которых оно создано.

## Модификация данных с помощью представления

Вы можете через представления изменять данные, если соблюдаются следующие требования:

- ◆ модификация данных должна ссылаться только на одну таблицу;
- ◆ столбцы в представлении должны непосредственно ссылаться на столбцы в таблице;
- ◆ столбец не должен быть производным итогового столбца;
- ◆ столбец не может вычисляться как результат действия операторов `UNION/UNION ALL`, `CROSSJOIN`, `EXCEPT` или `INTERSECT`;

- ◆ модифицируемый столбец не должен затрагиваться предложениями `DISTINCT`, `GROUP BY` или `HAVING`;
- ◆ не должен применяться оператор `TOP`.

### **Обновляемые представления**

---

Если представление не удовлетворяет перечисленным требованиям, можно создать для представления триггер `INSTEAD OF`. Этот триггер выполняется для операции DML, которую вы выполняете, вместо передачи этой операции DML через представление.

Поскольку определение представления может содержать предложение `WHERE`, через представление можно внести изменения, которые не видимы во время извлечения данных из представления. Предложение `WITH CHECK OPTION` требует лишь, чтобы модификацию данных, которую можно выполнить через представление, можно было также извлечь из представления с помощью инструкции `SELECT`.

### **Подсказка к экзамену**

---

На экзамене вы должны разбираться в требованиях к обновлению данных через представление.

## **Секционированные представления**

Вы можете выполнять секционирование (см. главу 6) для деления большой таблицы на множественные структуры хранения. Вместо применения встроенного в SQL Server средства секционирования можно секционировать большую таблицу вручную, разбив ее на несколько таблиц с одинаковой структурой. Если таблица секционирована вручную, все данные можно собрать вместе с помощью представления.

Представления, объединяющие множественные таблицы одинаковой структуры, называют *секционированными представлениями*. Секционированное представление (partitioned view) применяет оператор `UNION ALL` ко всем таблицам с одной и той же структурой. Представление данного типа должно удовлетворять следующим условиям.

- ◆ В списке выбора представления должны содержаться все столбцы секционированной таблицы.
- ◆ Столбцы с одним и тем же порядковым номером во всех инструкциях `SELECT` должны быть одного и того же типа и с одним и тем же набором параметров символьной обработки (collation).
- ◆ Как минимум, один столбец, соответствующий ограничению `CHECK`, уникальный для каждой таблицы-секции, должен иметь один и тот же порядковый номер во всех инструкциях `SELECT`.

- ◆ Ограничения должны формировать уникальные, неперекрывающиеся наборы данных во всех таблицах-секциях.
- ◆ Один и тот же столбец не должен применяться многократно в списках выбора.
- ◆ Столбец секционирования, определенный ограничением CHECK, должен быть частью первичного ключа.
- ◆ Столбец секционирования не может вычисляться, иметь тип данных `timestamp`, ограничение DEFAULT и не должен быть идентифицирующим столбцом.
- ◆ В определении представления одна и та же таблица-секция не может появляться дважды.
- ◆ У таблиц-секций не должно быть индексов для вычисляемых столбцов.
- ◆ У всех таблиц-секций первичный ключ должен иметь одно и то же число столбцов.
- ◆ У всех таблиц-секций должен быть одинаковый установочный параметр `ANSI_PADDING`.

Приведенный перечень может показаться довольно строгим, но его цель — гарантировать уникальный набор данных в каждой таблице-секции, включенной в секционированное представление. Если разрешить помещать одно и то же значение первичного ключа в нескольких таблицах-секциях, секционированное представление будет отображать дубликаты и создаст путаницу в приложениях, и вы не сможете определить единственную строку, предназначенную для обновления.

Одно из преимуществ ручного разбиения таблицы состоит в возможности использования дополнительных аппаратных ресурсов, недоступных для встроенного средства секционирования. Поскольку секционирование вручную создает множественные таблицы-секции, вы можете решить, в какой базе данных и в каком экземпляре SQL Server поместить каждую из них. Распределяя таблицы-секции секционированного представления по разным экземплярам SQL Server, вы создаете специальный случай, именуемый *распределенным секционированным представлением*.

У распределенного секционированного представления во многом та же структура, что и у секционированного представления, за исключением того, что у таблицы-секции четырехчастное имя и для объединения таблиц-секций используются связанные серверы.

### ***Дополнительная информация об обновлении секционированных представлений***

---

Несмотря на то, что обе формы секционированного представления используют инструкцию `UNION ALL`, вы можете выполнять обновления секционированного представления. К определению секционированных представлений и к любым транзакци-

ям предъявляются дополнительные требования, описанные в разд. "CREATE VIEW (Transact-SQL)" электронной документации по SQL Server на Web-странице <http://technet.microsoft.com/ru-ru/library/ms187956.aspx>.

## Создание индексируемого представления

Помимо выполнения изменений данных через представление вы также можете создавать индекс для представления. Исключением является секционированное представление, т. к. таблицы-секции могут охватывать базы данных и экземпляры SQL Server.

Когда создается обычное представление, SQL Server сохраняет только определение представления, которое затем оптимизатор подставляет в инструкции `SELECT`, адресованные представлению.

Индекс можно построить для ряда значений в столбце. Когда вы создаете индекс для представления, SQL Server выполняет инструкцию `SELECT`, определенную в представлении, сохраняет результирующий набор и затем строит индекс. Любая последующая инструкция DML, обращающаяся к какой-либо таблице, на которой определено представление, заставляет SQL Server обновлять сохраненный результирующий набор поэтапно (*incrementally*), а также поддерживать индекс, если это необходимо. Поскольку SQL Server физически хранит и поддерживает результирующий набор, или "материализует" данные, индексируемое представление иногда называют материализованным представлением (*materialized view*).

У индексируемых представлений очень длинный перечень ограничений. Ограничения индексируемого представления порождены тем фактом, что данные должны быть материализованы на диске в неизменном виде и данные в индексе также должны быть фиксированными.

Далее перечислены некоторые из ограничений для создания индексируемого представления.

- ◆ Инструкция `SELECT` не может ссылаться на другие представления.
- ◆ Все функции должны быть детерминированными.
- ◆ Функции `AVG`, `MIN`, `MAX`, `STDEV`, `STDEVP`, `VAR` и `VARP` запрещены.
- ◆ Создаваемый индекс должен быть кластеризованным и уникальным.
- ◆ Когда создаются представление и любые таблицы, на которые оно ссылается, параметру `ANSI_NULLS` следует задать значение `ON`.
- ◆ Представление должно создаваться с параметром `SCHEMABINDING`.
- ◆ Инструкция `SELECT` не должна содержать вложенные запросы, внешние соединения, операторы: `EXCEPT`, `INTERSECT`, `TOP`, `UNION`, `ORDER BY`, `DISTINCT`, `COMPUTE/COMPUTE BY`, `CROSS/OUTER APPLY`, `PIVOT` или `UNPIVOT`.



### **Дополнительная информация об ограничениях индексированного представления**

Полный список ограничений для создания индексированного представления можно найти в разд. "Creating Indexed Views" (Создание индексированных представлений) электронной документации по SQL Server на Web-странице <http://technet.microsoft.com/ru-ru/library/ms191432.aspx>.

Удовлетворение всех требований к созданию индексированного представления может показаться очень непростой задачей. Но главное преимущество индексированного представления заключается в действительной материализации данных и отсутствии необходимости вычислений на лету, как в случае обычного представления. Индексированные представления способны обеспечить значительный рост производительности, если у вас есть запросы, объединяющие большие объемы данных, например с итогами. Индексированные представления нуждаются в технической поддержке в случае внесения изменений в таблицы, лежащие в основе представления, поэтому индексированное представление не стоит создавать для таблиц, получающих большие порции изменений.

### **Подсказка к экзамену**

К экзамену нужно разобраться в ограничениях для создания индексированного представления и в способах повышения производительности ваших приложений с помощью индексированного представления.

## **Детерминированность**

Функция, возвращающая при каждом вызове одно и то же значение при одинаковых входных параметрах, называется *детерминированной*. Функция, которая, имея одни и те же входные параметры, при каждом вызове может возвращать разные значения, называется *недетерминированной*. Пример детерминированной функции — функция `SUBSTRING`, т. к. она при одних и тех же входных параметрах всегда возвращает одно и то же значение. Примерами недетерминированных функций могут быть функции `RAND` и `GETDATE`, потому что каждая из них может возвращать разные значения при каждом вызове.

Функции могут применяться в вычисляемых столбцах и внутри представлений. Вы также можете создать индекс для вычисляемого столбца или индексировать представление. Для создания индекса следует применять детерминированные функции, поскольку набор значений для индекса неизменен. Кроме того, поскольку создание индекса представления вызывает материализацию и сохранение результатов представления, для индексирования представления следует использовать в представлении только детерминированные функции.

## **Подстановка в запросах**

Если встречается ссылка на нематериализованное представление, SQL Server замещает имя представления реальной инструкцией `SELECT`, определенной для

представления, переписывает запрос, как будто ссылки на представление не было вовсе, и затем передает переписанный запрос оптимизатору.

У вас может быть представление со следующим определением.

```
CREATE VIEW Customers.CustomerOrders
AS
SELECT CASE WHEN a.CompanyName IS NOT NULL THEN a.CompanyName
 ELSE a.FirstName + ' ' + a.LastName END CustomerName,
 b.AddressLine1, b.AddressLine2, b.AddressLine3, b.City,
 d.StateProvinceAbbrev, e.CountryName, c.OrderDate, c.GrandTotal,
 c.FinalShipDate
FROM Customers.Customer a
INNER JOIN Customers.CustomerAddress b ON a.CustomerID = b.CustomerID
INNER JOIN Orders.OrderHeader c ON a.CustomerID = c.CustomerID
INNER JOIN LookupTables.StateProvince d
 ON b.StateProvinceID = d.StateProvinceID
INNER JOIN LookupTables.Country e ON b.CountryID = e.CountryID
GO
```

Далее вы можете ввести следующую инструкцию SELECT.

```
SELECT CustomerName, AddressLine1, AddressLine2, AddressLine3,
 City, StateProvinceAbbrev,
 CountryName, OrderDate, GrandTotal, FinalShipDate
FROM Customers.CustomerOrders
GO
```

Но на самом деле SQL Server предлагает оптимизатору следующий запрос.

```
SELECT CASE WHEN a.CompanyName IS NOT NULL THEN a.CompanyName
 ELSE a.FirstName + ' ' + a.LastName END CustomerName,
 b.AddressLine1, b.AddressLine2, b.AddressLine3, b.City,
 d.StateProvinceAbbrev, e.CountryName, c.OrderDate, c.GrandTotal,
 c.FinalShipDate
FROM Customers.Customer a
INNER JOIN Customers.CustomerAddress b ON a.CustomerID = b.CustomerID
INNER JOIN Orders.OrderHeader c ON a.CustomerID = c.CustomerID
INNER JOIN LookupTables.StateProvince d
 ON b.StateProvinceID = d.StateProvinceID
INNER JOIN LookupTables.Country e ON b.CountryID = e.CountryID
GO
```

Когда для представления создается индекс, данные материализуются. Запросы, которые ссылаются на индексированное представление, не замещаются определением представления, а возвращают результаты непосредственно из индексированного представления. Результаты могут возвращаться напрямую, потому что с точки зрения хранения индексированное представление — это в действительности таблица, которую обслуживает механизм хранения (storage engine).

В версии SQL Server Enterprise Edition при наличии индексированного представления подстановка в запросе идет на шаг дальше. Обычно оптимизатор выбирает индексы, созданные для таблиц, на которые есть ссылки в запросе, если он определяет, что данный индекс повышает производительность запроса. В SQL Server Enterprise Edition, если оптимизатор определяет, что данные можно получить с большей эффективностью из индексированного представления, он формирует план запроса, игнорирующий базовые таблицы, на которые есть ссылки в запросе, и извлекает данные из индексированного представления вместо таблиц.

---

**Подсказка к экзамену**

---

Для сдачи экзамена вам следует понять, как в каждой версии SQL Server трактуются представления и индексированные представления.

---

**Быстрая проверка**

---

**ВОПРОСЫ:**

1. Представления каких типов могут создаваться?
2. Индексы каких типов могут быть созданы для представления?

**ОТВЕТЫ:**

1. Вы можете создавать обычные представления, являющиеся просто сохраняемой инструкцией `SELECT`. Вы также можете создавать секционированные представления, использующие ключевые слова `UNION ALL` для объединения таблиц-секций.
2. Вы можете индексировать представление с помощью уникального кластеризованного индекса.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание представлений

На этом практическом занятии вы создадите представление для возврата еще недоставленных заказов.

### Упражнение. Создание представления

В этом упражнении вы создадите представление для возврата заказов, которые еще не доставлены.

1. В окне создания запроса измените заказ, пометив его с помощью следующего программного кода как недоставленный.

```
UPDATE Sales.SalesOrderHeader
SET ShipDate = NULL
WHERE SalesOrderID = 75123
GO
```

2. В имеющемся окне запроса для создания представления введите, выделите и выполните следующий код.

```
CREATE VIEW Sales.v_UnshippedOrders
AS
SELECT SalesOrderID, RevisionNumber, OrderDate, DueDate, ShipDate,
 Status, OnlineOrderFlag, SalesOrderNumber, PurchaseOrderNumber,
 AccountNumber, CustomerID, SalesPersonID, TerritoryID,
 BillToAddressID, ShipToAddressID, ShipMethodID, CreditCardID,
 CreditCardApprovalCode, CurrencyRateID, SubTotal, TaxAmt, Freight,
 TotalDue, Comment, rowguid, ModifiedDate
FROM Sales.SalesOrderHeader
WHERE ShipDate IS NULL
GO
```

3. В имеющемся окне запроса для тестирования представления и затем восстановления состояния измененного вами заказа введите, выделите и выполните следующий код.

```
SELECT * FROM Sales.v_UnshippedOrders
GO

UPDATE Sales.SalesOrderHeader
SET ShipDate = '08/07/2004'
WHERE SalesOrderID = 75123
GO
```

## Резюме урока

- ◆ Представление — это имя инструкции SELECT, сохраняемой в базе данных.
- ◆ Представление должно возвращать единственный результирующий набор и не может ссылаться на переменные или временные таблицы.
- ◆ Вы можете обновлять данные через представление до тех пор, пока модификация данных может быть задана для конкретного набора строк таблицы, лежащей в основе представления.
- ◆ Если представление не соответствует требованиям, разрешающим изменение данных, для внесения нужных изменений можно применить триггер INSTEAD OF.
- ◆ Создавая секционированное представление, можно с помощью инструкции UNION ALL объединять множественные таблицы, которые были физически секционированы.
- ◆ Распределенное секционированное представление использует связанные серверы для объединения множественных таблиц-секций из разных экземпляров SQL Server.
- ◆ Для повышения производительности можно создать уникальный кластеризованный индекс для представления, чтобы материализовать результирующий набор.

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ SQL Server позволяет создавать программируемые объекты четырех типов: функции, хранимые процедуры, триггеры и представления.
- ◆ Функции могут возвращать скалярное значение или результирующий набор и не могут изменять состояние базы данных или экземпляра SQL Server.
- ◆ Хранимые процедуры предоставляют программный интерфейс (API), позволяющий отделить структуру базы данных от приложений.
- ◆ Хранимая процедура может включать в себя практически любую команду языка T-SQL.
- ◆ Триггеры создаются для таблиц и представлений и запускаются автоматически в ответ на команды `INSERT`, `UPDATE` или `DELETE`.
- ◆ Представление позволяет присвоить имя инструкции `SELECT`, которая формирует единственный результирующий набор и сохраняется в базе данных.

## Ключевые термины

Знаете ли вы, что означают следующие ключевые термины? Вы можете проверить свои ответы, заглянув в словарь терминов, приведенный в конце книги.

- ◆ Курсор.
- ◆ Триггер DDL.
- ◆ Детерминированная функция.
- ◆ Триггер DML.
- ◆ Распределенное секционированное представление.
- ◆ Группа событий.

- ◆ Среда (контекст) выполнения.
- ◆ Заимствование роли и разрешений (Impersonation).
- ◆ Индексированное представление.
- ◆ Материализованное представление.
- ◆ Модуль.
- ◆ Недетерминированная функция.
- ◆ Параметр.
- ◆ Секционированное представление.
- ◆ Связь на уровне схемы (Schema binding).
- ◆ Хранимая процедура.
- ◆ Переменная.

## Сценарий

В следующих конкретных сценариях вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в *разд. "Ответы к сценариям"* в конце книги.

### Сценарий. Повышение производительности приложения

Fabrikam, крупный производитель и оптовый продавец тысяч потребительских и производственных товаров из пластмасс, нанял вас для устранения многочисленных проблем в его базовых приложениях.

Менеджер по разработке приложений идентифицировал 22 основных приложения, управляющих производственной деятельностью Fabrikam. Пользователи жаловались на то, что у всех этих приложений низкая производительность. Группа по разработке приложений попыталась решить проблемы производительности, но т. к. все запросы генерировались во время выполнения и передавались SQL Server как запросы, написанные специально для конкретного случая, внести изменения было крайне сложно. Несколько попыток использования нового программного кода закончились аварийным завершением приложений, заставившим отказаться от нового программного кода.

Производственные серверы баз данных настроены на разрешение административного доступа для всей группы разработчиков. В последние четыре месяца возникли многочисленные инциденты, связанные с удалением объектов с производственного сервера вместо их удаления с сервера разработки.

Есть три приложения, за модификацию которых все разработчики боятся браться, потому что используемая ими база данных содержит очень сложный про-

граммный код, написанный программистом, уже не работающим в компании. У приложений нет никакой сопроводительной документации.

Ответьте на следующие вопросы вашего менеджера:

1. Как решить проблему настройки производительности?
2. Как решить проблему непредумышленного удаления объектов из производственной базы данных?
3. Как решить проблему, связанную со сложным программным кодом, препятствующим внесению изменений в основные приложения?

## КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

### Создание хранимой процедуры

**Упражнение 1.** Найдите в ваших приложениях весь программный код на языке SQL, написанный специально для конкретного случая, и перенесите его в хранимые процедуры. Перепишите ваше приложение с применением хранимых процедур.

### Создание функции

**Упражнение 2.** Найдите блок нетривиального программного кода, выполняющего вычисление или возвращающего результирующий набор и при этом не изменяющего состояние базы данных или экземпляра SQL Server. Перенесите этот блок кода в функцию, чтобы этот код повторно не создавался при каждой ссылке на данное вычисление.

### Создание триггера

**Упражнение 3.** Создайте триггер DDL, не позволяющий кому бы то ни было удалять объекты в вашей производственной среде.

### Создание представления

**Упражнение 4.** Найдите в приложении сложную инструкцию `SELECT`. Создайте представление, чтобы включить в него эту сложную инструкцию. Внесите в приложение ссылку на представление.

**Упражнение 5.** Найдите детерминированное представление, которое часто выполняется, и превратите его в индексированное представление. Проследите за тем, как изменилась производительность индексированного представления.

## Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобия будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### **Дополнительная информация о тренировочных тестах**

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.



## ГЛАВА 6

# Методы повышения производительности запроса

После того как вы спроектировали вашу базу данных и приложение, как правило, хочется убедиться в том, что обеспечена производительность, соответствующая потребностям пользователей приложения. В этой главе описано, что делать для выявления в вашей базе данных проблем, связанных с производительностью, и как решать найденные проблемы. Microsoft SQL Server 2008 включает в себя обладающий богатыми возможностями и очень эффективный механизм управления базами данных, но ему все-таки требуется некоторая настройка производительности, если вы имеете дело с большой базой данных, сложными запросами, большим количеством обслуживаемых запросов или комбинацией перечисленных факторов.

### Экзаменационные темы в этой главе

- ◆ Создание и изменение индексов.
- ◆ Реализация решений с применением секционирования.
- ◆ Управление планами выполнения.
- ◆ Перехват планов выполнения.
- ◆ Сбор отслеживающей информации с помощью SQL Server Profiler.
- ◆ Получение выходных данных от Database Engine Tuning Advisor.
- ◆ Получение сведений из системных метаданных.

### Уроки в этой главе

- ◆ Урок 1. Настройка запросов.
- ◆ Урок 2. Создание индексов.

## Прежде чем начать

Для усвоения уроков этой главы необходимо:

- ◆ знать основы языка Transact-SQL (T-SQL);
- ◆ установить на компьютере версию SQL Server 2008 Developer Edition, Enterprise Edition или Enterprise Evaluation Edition с учебной базой данных AdventureWorks.

### **РЕАЛЬНАЯ ИСТОРИЯ.**

#### ***Тобиас Тернстрем (Tobias Thernström)***

Настройка запросов включает в себя не только понимание того, как работают запросы, индексы и связанные с ними элементы. Вы должны представлять, как все компоненты SQL Server действуют сообща. Пару лет назад я консультировал крупную издательскую компанию, в которой возникали длительные задержки при приеме заказов. Компания уже исследовала применяемое индексирование и не смогла найти какие-либо явные проблемы. Я начал с запуска утилиты SQL Trace, чтобы выяснить, что же на самом деле отнимает так много времени, и обнаружил, что задержки были вызваны простой инструкцией `INSERT`. Далее я убедился в том, что эта инструкция не была заблокирована никакими блокировками. В завершение я проверил, нет ли триггеров, определенных для таблицы. Выяснилось, что у таблицы действительно есть определенный для нее триггер `AFTER INSERT`. После просмотра триггера стало ясно, что именно он вызывал существенные задержки; триггер выполнял подсчет сводных данных, на основе всей таблицы заказов. Убедившись вместе с заказчиком в том, что новый подход позволит решать бизнес-задачу, мы пошли дальше и удалили триггер, а вместо него создали программу-агент, которая выполнялась каждые 10 минут и для каждой инструкции `INSERT` выполняла ту же функцию, для которой был разработан триггер. После удаления триггера инструкции `INSERT` стали выполняться практически без задержки. Я рассказал эту историю для того, чтобы вы поняли, что на производительность влияют не только сами запросы или индексные структуры, применяемые в них.

## УРОК 1. Настройка запросов

Настройка запросов — крайне важная задача для разработчика базы данных. Из этого урока вы узнаете о нескольких способах оптимизации запросов.

### ***После усвоения этого урока вы узнаете больше о том***

- Что влияет на производительность запроса.
- Как измерять производительность запроса.
- Как писать более эффективные запросы.

Расчетное время урока — 60 мин.

## Оценка производительности запроса

Один из наиболее важных аспектов настройки запроса — измерение или количественная оценка производительности. При измерении производительности нуж-

но знать, что на самом деле измерять, т. е. какую систему показателей применять. В SQL Server есть три основных учитываемых показателя: затраты на запрос или стоимость запроса (query cost), число считываний страниц (page reads) и время выполнения запроса (query execution time).

## Стоимость запроса

*Стоимость выполнения запроса* — обычно (но не всегда, как вы скоро увидите) подходящий показатель при сравнении производительностей запросов. Это внутренняя характеристика, применяемая в SQL Server и учитывающая ресурсы ЦП и ввода/вывода, потребляемые запросом. Теоретически, чем меньше стоимость запроса, тем выше его производительность. На стоимость не влияют такие проблемы, как конфликты ресурсов или ожидания снятия блокировок. В большинстве случаев стоимость запроса — хорошая мера производительности, но если в запросе применяются особые элементы, например, скалярные пользовательские функции или подпрограммы общезыковой среды выполнения (Common Language Runtime, CLR), затраты на них не учитываются, что делает стоимость запроса ниже реальной. Поэтому стоимость выполнения запроса называют *ориентировочной* или *приблизительной стоимостью запроса* (estimated query cost).

## Считывания страниц

*Количество считываний страниц* представляет количество 8-килобайтовых страниц данных, к которым обращался механизм запоминания SQL Server во время выполнения запроса. Извлечь эту характеристику можно с помощью команды SET STATISTICS IO ON. Эта команда приводит к выводу на вкладке окна запроса **Messages** (Сообщения) сведений, подобных приведенным далее.

```
Table 'Customer'. Scan count 2, logical reads 136, physical reads 0,
read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob
read-ahead reads 0.
```

```
Table 'SalesOrderHeader'. Scan count 121, logical reads 822, physical reads
5, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob
read-ahead reads 0.
```

Общее количество считанных страниц в этом примере равно  $136 + 822$ , т. е. сумме значений, обозначенных как "logical reads" (логические считывания). *Логические считывания* — это количество считанных из памяти страниц. Логические считывания предоставляют количество страниц данных, прочитанных из любого индекса таблицы SalesOrderHeader (общие сведения о заказах на продажу). Другие показатели сообщают о том, сколько логических считываний произведено с жесткого диска (physical и read-ahead reads), количество выполненных сканирований индекса или кучи (scan count) в ответ на запрос и количество считываний, потребовавшихся для извлечения данных типа Large Object (LOB). Данные типа LOB хранятся вне строки с типами данных varchar(max), nvarchar(max),

varbinary(max), text, ntext, image и XML. Показатель считываний страниц не учитывает ресурсы ЦП, потребляемые во время выполнения запроса. Поэтому количество считываний страниц — обычно не столь подходящая мера производительности, как стоимость запроса. У этого показателя существует та же проблема, что и у стоимости запроса, связанная со скалярными пользовательскими функциями и подпрограммами CLR и заключающаяся в том, что считывания страниц, вызванные этими объектами, не включаются в вывод параметра `STATISTICS IO`.

## Время выполнения запроса

*Время выполнения запроса* — самая изменчивая характеристика. На нее влияют блокировки и конфликты ресурсов на сервере. С учетом этого особенно важно всегда включать в сравнение производительностей время выполнения запроса, потому что эта мера может помочь обнаружить проблемы, пропущенные другими показателями производительности (считываниями страниц и стоимостью запроса). Если выполнить команду `SET STATISTICS TIME ON`, SQL Server вернет время выполнения всех запросов в миллисекундах.

## Теоретический порядок выполнения запроса

При настройке запроса жизненно важно иметь общее представление о теоретическом порядке выполнения инструкции `SELECT`. Это поможет вам понять, что на самом деле должен сделать SQL Server для формирования результатов запроса.

Теоретический порядок выполнения называют теоретическим, потому что оптимизатор может изменить действительный порядок выполнения для повышения производительности. Альтернативный порядок выполнения применяется, только если результаты запроса будут такими же, как в случае использования теоретического порядка выполнения.

Порядок выполнения следует разделить на два разных варианта, зависящих от наличия или отсутствия в запросе предложения `UNION`. Первый более простой вариант без оператора `UNION` приведен в табл. 6.1, описывающей порядок обработки предложений инструкции.

**Таблица 6.1.** Теоретический порядок выполнения при отсутствии предложения `UNION`

| Предложения                                                                                  | Результаты                                                                  |
|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 1. FROM, JOIN, APPLY и ON                                                                    | Выполняется соединение и применяется первый фильтр запроса (предложение ON) |
| 2. WHERE                                                                                     | Применяется второй фильтр запроса                                           |
| 3. GROUP BY и статистические функции (такие как SUM, AVG и т. д.), которые включены в запрос | Выполняются группирующие и сводные вычисления                               |

Таблица 6.1 (окончание)

| Предложения | Результаты                                                                                                                                      |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| 4. HAVING   | Применяется третий фильтр запроса (фильтрация результатов статистических функций)                                                               |
| 5. SELECT   | Выбираются столбцы, которые должны быть возвращены запросом                                                                                     |
| 6. ORDER BY | Результаты сортируются                                                                                                                          |
| 7. TOP      | Применяется четвертый (и последний) фильтр запроса; он заставляет запрос возвращать только первые X строк из результатов, полученных до сих пор |
| 8. FOR XML  | Табличный результат инструкции SELECT преобразуется в результат на языке расширяемой разметки (Extensible Markup Language, XML)                 |

В запросах с предложением UNION используется теоретический порядок выполнения, приведенный в табл. 6.2.

Таблица 6.2. Теоретический порядок выполнения при наличии предложения UNION

| Предложения                                                                                  | Результаты                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. FROM, JOIN, APPLY и ON                                                                    | Выполняется соединение и применяется первый фильтр запроса (предложение ON)                                                                                                                                                      |
| 2. WHERE                                                                                     | Применяется второй фильтр запроса                                                                                                                                                                                                |
| 3. GROUP BY и статистические функции (такие как SUM, AVG и т. д.), которые включены в запрос | Выполняются группирующие и сводные вычисления                                                                                                                                                                                    |
| 4. HAVING                                                                                    | Применяется третий фильтр запроса (фильтрация результатов статистических функций)                                                                                                                                                |
| 5. TOP                                                                                       | Применяется четвертый (и последний) фильтр запроса; он заставляет запрос возвращать только первые X строк из результатов, полученных до сих пор. (Учтите, что в этом случае предложение TOP выполняется до предложения ORDER BY) |
| 6. UNION и SELECT                                                                            | Результаты всех инструкций SELECT, включенных в запрос, объединяются; выбираются столбцы, которые должны быть возвращены запросом                                                                                                |
| 7. ORDER BY                                                                                  | Результаты сортируются                                                                                                                                                                                                           |
| 8. FOR XML                                                                                   | Табличный результат инструкции SELECT с предложением UNION преобразуется в результат на языке XML                                                                                                                                |

Разница в порядке выполнения объясняется введением предложения TOP (в версии SQL Server 7.0), которое не является частью стандарта ANSI/ISO языка SQL.

В соответствии со стандартом предложение `UNION` допускает присутствие только одного предложения `ORDER BY`, которое должно располагаться в последней инструкции `SELECT` в запросе и сортировать результат всего запроса. Это означает, что предложение `TOP` в любой части запроса с предложением `UNION`, за исключением последней инструкции `SELECT`, вернет первые *X* элементов прежде, чем они будут отсортированы. Это можно считать недостатком, но о нем следует знать. Например, сравните результаты двух запросов, каждый из которых должен вернуть два самых дорогих товара красного цвета и два самых дорогих товара черного цвета. Первый из приведенных далее запросов возвращает неверный результат.

```
USE AdventureWorks;
```

```
SELECT TOP(2) ProductID, Name, Color, ListPrice
FROM Production.Product
WHERE Color = 'Black'
UNION
SELECT TOP(2) ProductID, Name, Color, ListPrice
FROM Production.Product
WHERE Color = 'Red'
ORDER BY ListPrice DESC;
```

Далее приведен результат (неверный) этого запроса.

| ProductID | Name                    | Color | ListPrice |
|-----------|-------------------------|-------|-----------|
| 706       | HL Road Frame — Red, 58 | Red   | 1431,50   |
| 707       | Sport-100 Helmet, Red   | Red   | 34,99     |
| 317       | LL Crankarm             | Black | 0,00      |
| 318       | ML Crankarm             | Black | 0,00      |

Второй запрос, приведенный далее, генерирует ожидаемый результат.

```
USE AdventureWorks;
```

```
WITH a AS (
 SELECT TOP(2) ProductID, Name, Color, ListPrice
 FROM Production.Product
 WHERE Color = 'Black'
 ORDER BY ListPrice DESC
), b AS (
 SELECT TOP(2) ProductID, Name, Color, ListPrice
 FROM Production.Product
 WHERE Color = 'Red'
 ORDER BY ListPrice DESC
)
SELECT * FROM a
UNION ALL
SELECT * FROM b;
```

Второй запрос формирует следующий (корректный) результат.

| ProductID | Name                   | Color | ListPrice |
|-----------|------------------------|-------|-----------|
| 775       | Mountain-100 Black, 38 | Black | 3374,99   |
| 776       | Mountain-100 Black, 42 | Black | 3374,99   |
| 749       | Road-150 Red, 62       | Red   | 3578,27   |
| 750       | Road-150 Red, 44       | Red   | 3578,27   |

Как видите, первый запрос возвращает некорректные значения, потому что предложение `ORDER BY` выполняется после предложения `TOP`.

## Настройка производительности запроса

Существует несколько способов оптимизации запросов. Оптимизация включает в себя следующие задачи: переписывание запроса, денормализацию или нормализацию таблиц, добавление индексов, удаление индексов или комбинацию этих задач.

## Графический план выполнения

Графический план выполнения — полезное средство оптимизации запросов. В этой главе обсуждается ряд планов выполнения. В табл. 6.3 приведено несколько элементов, которые вам придется находить в плане выполнения.

Таблица 6.3. Элементы графического плана выполнения

| Ожидаемый элемент                                                                                                                                       | Возможные назначения                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Широкие стрелки<br>                                                  | Широкая стрелка обозначает большое количество строк, перемещаемых из одной операции плана выполнения в другую. Чем больше строк передается из одной операции в другую, тем шире стрелка                                             |
| Операции хэширования<br><br>Hash Match<br>(Inner Join)<br>Cost: 73 % | Если для обработки таких предложений, как <code>GROUP BY</code> и <code>JOIN</code> , применяется операция хэширования, часто это означает отсутствие подходящего индекса для оптимизации запроса                                   |
| Сортировки<br><br>Sort<br>Cost: 69 %                                 | Сортировка не обязательно плоха, но если она составляет в процентном отношении существенную часть стоимости выполнения запроса, следует рассмотреть возможность построения индекса для устранения необходимости операции сортировки |
| Длинные планы                                                                                                                                           | Чем меньше число операций, формирующих план выполнения, тем оптимальнее план                                                                                                                                                        |

Таблица 6.3 (окончание)

| Ожидаемый элемент                                                                                                                                                                                                                                                                                                                                                              | Возможные назначения                                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <p>Сканирования таблицы<br/>или кластеризованного индекса</p>  <p>Table Scan<br/>[Customers] [c1]<br/>Cost: 100 %</p>  <p>Clustered Index Scan (Clustered)<br/>[Customers].[Index1] [c1]<br/>Cost: 100 %</p> | <p>Сканирование кластеризованного индекса или таблицы означает, что нет индекса, подходящего для оптимизации запроса</p> |

## Применение аргументов поиска

*Аргумент поиска* (search argument, SARG) — это выражение фильтрации, используемое для ограничения количества строк, возвращаемых запросом, и способное применять операцию поиска в индексе, которая существенно повышает производительность запроса. Обычно если в выражении применяется столбец из таблицы (например, `LEFT(Name, 1) = 'A'`), выражение фильтрации не является SARG. Если заданный фильтр — не SARG и в запросе нет других элементов SARG, это приводит к построчному просмотру индекса или таблицы, который многократно сканирует всю таблицу или индекс. Вместо многократного перебора лучше применять операцию поиска. Поиск означает применение сбалансированного дерева индекса для обнаружения значений, которые ищутся в запросе. Использование сбалансированного дерева значительно снижает объем работы, которую должен проделать SQL Server для поиска строки. Разница между поиском и сканированием может составлять несколько порядков. Например, в следующем запросе SQL Server последовательно просматривает индекс `OrderDateIndex` (вместо проведения поиска в нем). На рис. 6.1 показан план выполнения этого запроса, который можно отобразить в окне среды SQL Server Management Studio (SSMS) с помощью нажатия комбинации клавиш `<Ctrl>+<M>`.

```
USE AdventureWorks;
```

```
CREATE NONCLUSTERED INDEX OrderDateIndex
ON Sales.SalesOrderHeader (OrderDate);
```

```
SELECT COUNT(*) FROM Sales.SalesOrderHeader
WHERE YEAR(OrderDate) = 2004;
```



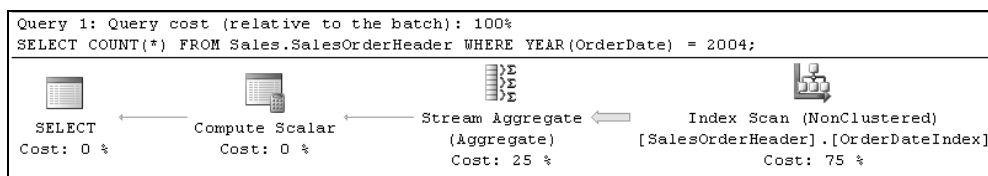


Рис. 6.1. Действующий план выполнения в среде SSMS, отображающий операцию сканирования индекса

Если вместо этого переписать запрос так, чтобы столбец `OrderDate` не использовался в выражении, можно вместо сканирования применить операцию поиска в индексе. На рис. 6.2 показан план выполнения для следующего запроса.

```
SELECT COUNT(*) FROM Sales.SalesOrderHeader
WHERE OrderDate >= '20040101' AND OrderDate < '20050101';
```

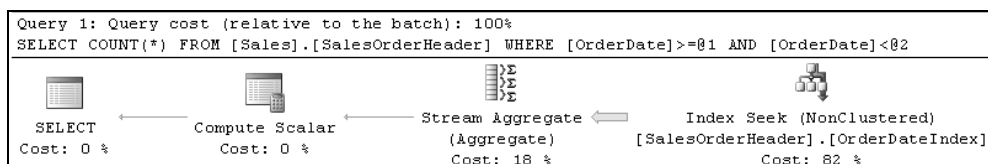


Рис. 6.2. Действующий план выполнения в среде SSMS, отображающий операцию поиска в индексе

Имейте в виду, что оператор `COLLATE` в выражении фильтрации также делает некорректным использование индекса для этого столбца. Вы узнаете об этом больше из *урока 2 данной главы*.

## Применение соединений

Одна из основных стратегий при оптимизации запросов — минимизация числа используемых операторов соединения. Еще одна особенность состоит в том, что внешние соединения сопряжены с большими затратами, чем внутренние соединения из-за дополнительной работы по поиску строк, не имеющих соответствия. Если в запросе применяются только внутренние соединения, поведение предложений `ON` и `WHERE` одинаково; независимо от того, что вы помещаете в предложение `ON` или `WHERE` выражение. Сравните два следующих запроса, в которых отличия выделены жирным шрифтом; оба они возвращают одинаковые результаты и используют идентичные планы выполнения.

```
-- Запрос 1
SELECT p.ProductID, p.Name, sod.SalesOrderID
FROM Production.Product AS p
INNER JOIN Sales.SalesOrderDetail AS sod
 ON sod.ProductID = p.ProductID
WHERE p.Color = 'Black';
```

```
-- Запрос 2
SELECT p.ProductID, p.Name, sod.SalesOrderID
FROM Production.Product AS p
INNER JOIN Sales.SalesOrderDetail AS sod
 ON sod.ProductID = p.ProductID
 AND p.Color = 'Black';
```

Если бы в этих запросах использовалось внешнее соединение, они бы не были синтаксически эквивалентны и имели бы существенно отличающуюся производительность.

## Вложенные запросы без корреляции с внешним запросом

Как пояснялось в *главе 4*, несвязанный вложенный запрос выполняется только один раз во время выполнения запроса и возвращает единственное значение. Эти запросы обычно сопряжены с минимальными затратами. Учтите, что вложенный запрос этого типа не может иметь ссылок (корреляции) на внешний запрос. В следующем примере используется вложенный запрос для возврата всех товаров, цена которых ниже средней цены товара. Вложенный запрос, вычисляющий среднюю цену товара, выполняется первым (только единожды) и затем значение, полученное вложенным запросом, используется во внешнем запросе как параметр.

```
USE AdventureWorks;

SELECT p.ProductID, p.Name, p.ListPrice
FROM Production.Product AS p
WHERE p.ListPrice > (SELECT AVG(p2.ListPrice)
 FROM Production.Product AS p2
);
```

## Связанные вложенные запросы

Связанные вложенные запросы содержат ссылку на внешний запрос. Обычно эта ссылка применяется для фильтрации связанного вложенного запроса. Вложенный запрос этого типа, если в нем для фильтрации внешнего запроса используется оператор `EXISTS`, как правило, с точки зрения производительности эквивалентен применению оператора `JOIN`. В следующем примере запроса для получения только проданных товаров используется оператор `EXISTS`.

```
USE AdventureWorks;

SELECT p.ProductID, p.Name
FROM Production.Product AS p
WHERE EXISTS (
 SELECT * FROM Sales.SalesOrderDetail AS sod
 WHERE sod.ProductID = p.ProductID
);
```

Несмотря на то, что вложенный запрос этого типа обычно представляет собой удачный вариант реализации, применение связанных вложенных запросов в предложении `SELECT` часто оказывает более негативное влияние на производительность, чем операторы `JOIN`. Конечно, это зависит от количества строк, возвращаемых внешним запросом. Если возвращается большое количество строк, каждый запрос в предложении `SELECT` будет выполняться для каждой строки, что означает большое число выполнений запроса. Следующий запрос возвращает 6224 строки и включает в себя два связанных вложенных запроса. Каждый из этих вложенных запросов выполняется по одному разу для каждой строки, приводя в результате к 12 448 выполнений вложенных запросов.

```
USE AdventureWorks;
```

```
SELECT soh.SalesOrderID, soh.OrderDate
 , (SELECT TOP(1) sod1.UnitPrice
 FROM Sales.SalesOrderDetail AS sod1
 WHERE sod1.SalesOrderID = soh.SalesOrderID
 ORDER BY sod1.OrderQty DESC
) AS UnitPrice
 , (SELECT TOP(1) sod2.OrderQty
 FROM Sales.SalesOrderDetail AS sod2
 WHERE sod2.SalesOrderID = soh.SalesOrderID
 ORDER BY sod2.OrderQty DESC
) AS OrderQty
FROM Sales.SalesOrderHeader AS soh
WHERE soh.TerritoryID = 4;
```

В этом запросе есть и потенциальная ошибка. Поскольку каждый вложенный запрос выполняется отдельно, все может закончиться применением разных индексов. Это означает, что данные запросы могут вернуть результаты не из одной и той же строки (которая в них, возможно, предусмотрена), если в каком-либо заказе на продажу в нескольких строках с подробными сведениями о заказе существует одно и то же значение поля `OrderQty` (Объем заказа).

Данный запрос можно переписать несколькими способами: быть может, самый распространенный в SQL Server 2008 — использование нового предложения `APPLY`. Как объяснялось в *главе 4*, если вложенный запрос применяется в предложениях `FROM`, `JOIN` или `APPLY`, на него можно ссылаться как на *производную таблицу*. Предложение `APPLY` по своей сути предоставляет возможность объединить два вложенных запроса в один, уменьшив на половину количество выполнений вложенных запросов. В новом варианте запроса для возврата тех же результатов, что и в предыдущем запросе, следует применить оператор `OUTER APPLY`. (`OUTER APPLY` действует подобно оператору левого внешнего соединения, а его дополнение, оператор `CROSS APPLY`, ведет себя как внутреннее соединение.) В нашем примере этот прием сработает, т. к. в предыдущем примере внешний запрос воз-

вращает строку, даже если вложенные запросы не вернут ничего. Новый запрос можно было бы записать следующим образом.

```
USE AdventureWorks;
```

```
SELECT soh.SalesOrderID, soh.OrderDate, a.*
FROM Sales.SalesOrderHeader AS soh
OUTER APPLY (SELECT TOP(1) sod.UnitPrice, sod.OrderQty
 FROM Sales.SalesOrderDetail AS sod
 WHERE sod.SalesOrderID = soh.SalesOrderID
 ORDER BY sod.OrderQty DESC
) AS a
WHERE soh.TerritoryID = 4;
```

Стоимость этого варианта запроса равна 76, в то время как стоимость выполнения первого варианта запроса была вдвое больше, около 151.

Другое решение проблемы этого типа — использование функции `ROW_NUMBER` вместо связанного вложенного запроса. Применив функцию `ROW_NUMBER`, вы сможете найти определенное количество необходимых вам строк фильтрацией номера строки вместо использования предложения `TOP`. Для того чтобы иметь возможность фильтровать на основе результата функции `ROW_NUMBER`, запрос следует поместить в производную таблицу или обобщенное табличное выражение (ОТВ). Чем больше результирующий набор, тем эффективнее действует этот вариант по сравнению с двумя предыдущими вариантами запроса. Стоимость следующего запроса падает с 76 до 3.6, огромное снижение.

-- Обобщенное табличное выражение.

```
WITH a AS (SELECT soh.SalesOrderID, soh.OrderDate, sod.UnitPrice
 , sod.OrderQty
 , ROW_NUMBER() OVER (PARTITION BY soh.SalesOrderID
 ORDER BY sod.OrderQty DESC
) AS RowNo
 FROM Sales.SalesOrderDetail AS sod
 INNER JOIN Sales.SalesOrderHeader AS soh
 ON sod.SalesOrderID = soh.SalesOrderID
 WHERE soh.TerritoryID = 4
)
SELECT a.SalesOrderID, a.OrderDate, a.UnitPrice, a.OrderQty
FROM a
WHERE a.RowNo = 1;
```

## Скалярные пользовательские функции

Скалярная пользовательская функция возвращает единственное значение (а не результирующий набор). Функции этого типа часто применяются в запросах и могут существенно снижать производительность. Причина кроется в том, что эти функции не обрабатываются оптимизатором в основном плане запроса; вместо

этого они вызываются в плане выполнения просто, без оптимизации, основанной на рабочей среде, в которой они вставляются в план. Это означает, что затраты на проделываемую внутри функции работу не включаются в оценки стоимости запроса, которые вы найдете в графическом плане выполнения запроса. Та же самая проблема возникает и при получении вывода инструкции `SET STATISTICS IO ON`, который не содержит ссылок, на то, что делалось внутри функции. В этом разделе в качестве примера сравнивается производительность запроса с применением функции, определенной пользователем, и запроса с использованием связанного вложенного запроса. Пользовательская функция и вложенный запрос выполняют одни и те же инструкции `SELECT`.

Поскольку затраты функции не отражаются в показателе стоимости запроса, более подходящая мера производительности для этих запросов — время выполнения, которое можно получить с помощью инструкции `SET STATISTICS TIME ON`.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### ***Время выполнения запроса***

При использовании времени выполнения запроса в качестве меры производительности, как правило, следует выполнить каждый запрос несколько раз и использовать как показатель производительности наименьшее время выполнения или медиану (срединное значение выборки). Также имейте в виду, что показатели, полученные в тестовой среде, могут неточно отражать производительность в рабочей среде. Это зависит от ряда факторов, включая активность взаимодействия пользователей с реальными данными.

В следующем запросе применяется функция, определенная пользователем. Сформированный для запроса план выполнения показан на рис. 6.3.

```
USE AdventureWorks;
GO

CREATE FUNCTION dbo.fnGetCustomerAccountNumber(@CustomerID INT)
RETURNS VARCHAR(10)
AS
BEGIN
 RETURN ISNULL(
 (SELECT AccountNumber
 FROM Sales.Customer
 WHERE CustomerID = @CustomerID), 'NOT FOUND');
END
GO

SET STATISTICS IO ON;
SET STATISTICS TIME ON;

SELECT soh.SalesOrderID, soh.OrderDate
 , dbo.fnGetCustomerAccountNumber(soh.CustomerID)
FROM Sales.SalesOrderHeader AS soh;
```

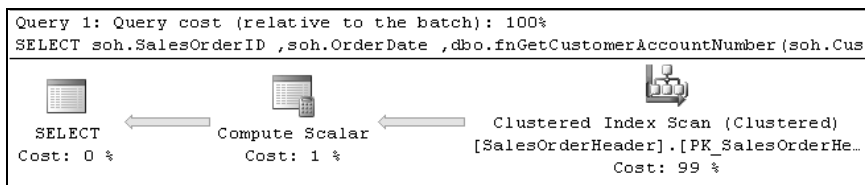


Рис. 6.3. Действующий план выполнения из среды SSMS для запроса с применением пользовательской функции

Стоимость этого запроса равна 0.56, и количество считываний страниц — 706 (ни один из показателей не точен и, следовательно, не может применяться для измерения производительности), время выполнения на тестовой машине, используемой в этом примере, — 25 секунд. Рассмотрите графический план выполнения и обратите внимание на то, что он не содержит ссылки на таблицу Sales.Customer (Клиент).

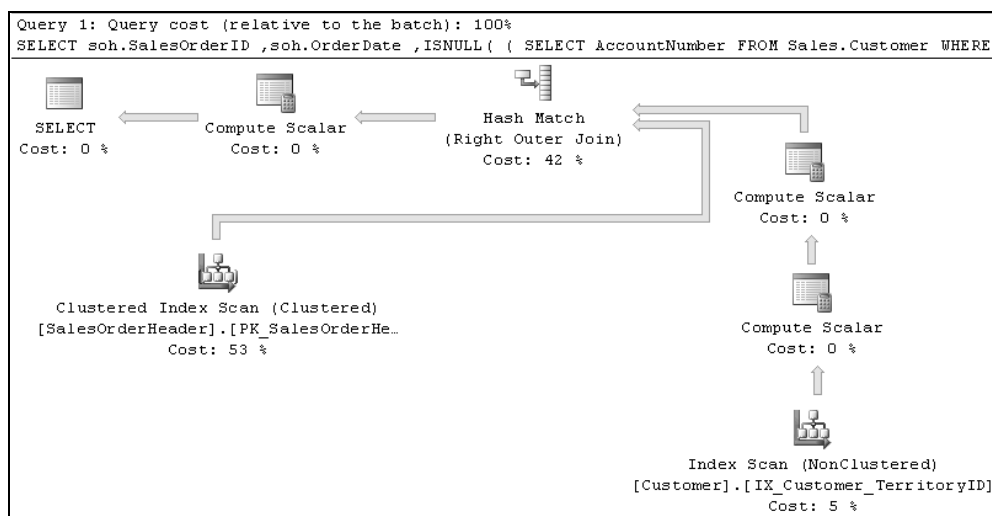


Рис. 6.4. Действующий план выполнения в среде SSMS для запроса, использующего связанный вложенный запрос

Далее приведен пример запроса, в котором на месте пользовательской функции применяется связанный вложенный запрос. План выполнения запроса показан на рис. 6.4.

```
USE AdventureWorks;
```

```
SET STATISTICS IO ON;
```

```
SET STATISTICS TIME ON;
```

```
SELECT soh.SalesOrderID, soh.OrderDate
 , ISNULL(
 (SELECT AccountNumber
 FROM Sales.Customer
 WHERE CustomerID = soh.CustomerID), 'NOT FOUND')
FROM Sales.SalesOrderHeader AS soh;
```

Стоимость запроса без пользовательской функции возрастает до 1.05 и количество считываний страниц до 742 (оба значения точные). При этом время выполнения падает до 1 секунды. Как видите, первый запрос из-за использования функции, определенной пользователем, в 25 раз медленнее последнего запроса.

Почему же встраиваемая табличная функция, определенная пользователем, намного быстрее? Применение встраиваемых табличных функций, определенных пользователем, и представлений не приводит к такому же снижению производительности, как использование скалярных пользовательских функций, потому что встраиваемые табличные функции, определенные пользователем, и представления оптимизируются (раскрываются) в плане запроса.

## Табличные функции, определенные пользователем

Существуют три типа пользовательских функций, возвращающих табличное значение. Два из них могут быть написаны на языке T-SQL, а третий — на языке среды CLR, например, C# или Microsoft Visual Basic .NET:

- ◆ встраиваемая табличная пользовательская функция на T-SQL;
- ◆ многооператорная табличная пользовательская функция на T-SQL;
- ◆ табличная пользовательская функция на языке среды CLR.

### ***Дополнительная информация о пользовательских функциях на T-SQL и CLR***

В *главе 5* более подробно обсуждаются пользовательские функции на языке T-SQL. В *главе 7* приводится дополнительная информация о пользовательских функциях среды CLR.

Функции этих трех типов ведут себя по-разному. Встраиваемая табличная пользовательская функция на языке T-SQL — это просто представление, которое может принимать параметры. Она оптимизируется так же, как представление или любая инструкция `SELECT`. Выигрыш от применения вместо представления встраиваемой табличной функции заключается в возможности потребовать от пользователя, применяющего функцию, предоставления параметров. В этом случае вы можете быть уверены, что в запросе внутри функции всегда применяется фильтр (на основе предоставленных параметров).

С другой стороны, многооператорная табличная функция на T-SQL может рассматриваться как хранимая процедура, заполняющая временную таблицу, ис-

пользуемую во внешнем запросе. Если включить в запрос многооператорную табличную пользовательскую функцию (например, в соединение), она должна выполняться полностью (т. е. завершить выполнение), прежде чем запрос сможет воспользоваться ее результатами. Это означает, что если многооператорной табличной функции нужно вернуть 1 000 000 строк, функция должна обработать все строки, прежде чем запрос сможет использовать ее результаты.

Функции третьего типа, т. е. табличные пользовательские функции на языке среды CLR, формируют потоковые результаты (stream). Это означает, что результаты табличной функции среды CLR становятся доступны вызывающему запросу во время ее выполнения. Это отличие повышает производительность, т. к. внешний запрос не должен ждать, пока станет доступным полный результат функции, для того чтобы начать обрабатывать возвращенные ею строки. Табличная функция CLR состоит из двух методов CLR: один метод управляет выполнением функции в целом, а второй вызывается для каждой строки, возвращаемой функцией. Метод, выполняемый для каждой строки, возвращаемой функцией, не запускается до тех пор, пока метод, управляющий выполнением функции в целом, не начнет выполнение команд `yield return`. Это важно помнить, т. к. любая обработка до запуска команд `yield return` должна быть завершена до того, как функция вернет какие бы то ни было строки. Табличные функции CLR обычно удобны для запросов к объектам, отличным от таблиц, например, строкам (с помощью регулярных выражений) или файловой системе. Имейте в виду, что обработка, выполняемая функцией CLR, неправильно отражается в показателях производительности: стоимости запроса и количестве считываний страниц.

## Курсоры

В основном следует избегать применения курсоров из-за их отрицательного влияния на производительность. Они обладают такой особенностью отчасти потому, что выполнение в цикле курсора каждой инструкции `FETCH` аналогично затратам на выполнение инструкции `SELECT`, возвращающей одну строку. Другая проблема состоит в том, что инструкция языка манипулирования данными (DML) оптимизируется как единый блок, а цикл курсора не может оптимизироваться аналогичным образом (если вообще может оптимизироваться). Вместо этого каждый элемент цикла оптимизируется и выполняется отдельно в каждом проходе (итерации) цикла.

Вы должны постараться преобразовать логику курсора в одну или несколько инструкций, основанных на наборах или множествах (`SELECT`, `INSERT`, `UPDATE`, `DELETE` или `MERGE`). Если невозможно преобразовать логику обработки с помощью курсора в инструкцию, основанную на наборе данных, рассмотрите реализацию логики с помощью хранимой процедуры CLR или табличной пользовательской функции (в зависимости от нужной вам функциональной возможности).



## Определение запросов, нуждающихся в настройке

Как вы, вероятно, уже поняли, для повышения производительности можно сделать многое. Следующая очевидная проблема — поиск запросов, нуждающихся в настройке. Для эффективного решения следует применить приложение SQL Server Profiler, предназначенное для отслеживания событий, происходящих в экземпляре SQL Server. Вы можете следить за появлением нескольких событий, но обычно для настройки используются события `SQL:BatchCompleted` и `RPC:Completed`. Помимо выбора отслеживаемых событий вам необходимо задать столбцы, которые будут извлекаться при возникновении события. Как правило, при определении, есть ли у события что-либо нуждающееся в настройке, полезны следующие столбцы:

- ◆ **Duration** — возвращает количество миллисекунд (или микросекунд при записи событий в файл или таблицу вместо применения графической утилиты для просмотра событий);
- ◆ **Reads** — возвращает общее количество 8-килобайтовых страниц, прочитанных во время выполнения;
- ◆ **Writes** — возвращает общее количество 8-килобайтовых страниц, записанных во время выполнения;
- ◆ **CPU** — возвращает общее время ЦП, затраченное на выполнение.

Значение, больше желаемого, в любом из этих столбцов должно заставить вас исследовать производительность запроса.

Поскольку SQL Server Profiler обычно, когда применяется на рабочем сервере, возвращает большое количество строк, можно рассмотреть возможность записи результата в файл или таблицу. Имейте также в виду, что вы можете запустить трассировку на сервере, не применяя графический интерфейс SQL Server Profiler. Для этого вы просто начинаете трассировку в SQL Server Profiler, а затем в меню **File** (Файл) выбираете последовательность команд **Export | Script Trace Definition | For SQL Server 2005-2008**. Трассировка сервера поддерживает только режим записи трассировки в файл. Если вы записали трассировку в файл, его можно запросить, передав имя файла трассировки как параметр в табличную пользовательскую функцию `fn_trace_gettable`.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

### Настройка производительности запроса

На этом практическом занятии вы проверите производительность трех разных версий одного запроса, формирующего один и тот же результирующий набор. Используемый вами запрос должен вернуть всех клиентов с определенной территории и последний полученный ими заказ. Клиент, не имеющий ни одного заказа, также включается в результирующий набор.

## Упражнение 1. Тестирование с помощью маленького результирующего набора.

В этом упражнении вы выполните три запроса, упомянутые во вводном абзаце к практическому занятию, и запишите стоимость каждого из них. В данном случае задаваемый во всех трех запросах параметр (*TerritoryID*) приводит к формированию маленького результирующего набора из 64 строк.

1. Откройте среду SSMS и установите соединение с подходящим экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных *TestDB*, схемы *Test* и двух таблиц, применяемых в этом упражнении.

```
CREATE DATABASE TestDB;
GO
```

```
USE TestDB;
GO
```

```
CREATE SCHEMA Test;
GO
```

```
SELECT * INTO Test.SalesOrderHeader
FROM AdventureWorks.Sales.SalesOrderHeader;
GO
```

```
SELECT * INTO Test.Customer
FROM AdventureWorks.Sales.Customer;
GO
```

```
ALTER TABLE Test.SalesOrderHeader
 ADD CONSTRAINT PKSalesOrderHeader
 PRIMARY KEY (SalesOrderID);
GO
```

```
ALTER TABLE Test.Customer
 ADD CONSTRAINT PKCustomer
 PRIMARY KEY (CustomerID);
```

3. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню **Query** (Запрос).
4. В имеющемся окне запроса для проверки производительности запроса, использующего два связанных вложенных запроса, введите, выделите и выполните запрос Query 1, приведенный далее.

Применение в этом запросе двух отдельных связанных вложенных запросов не гарантирует возврата этими вложенными запросами данных из одной и той же строки таблицы Test.Customer.

```
-- Query 1
SELECT c.CustomerID, c.AccountNumber
 , (SELECT TOP(1) soh.SalesOrderID
 FROM Test.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 ORDER BY OrderDate DESC) AS SalesOrderID
 , (SELECT TOP(1) soh.OrderDate
 FROM Test.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 ORDER BY OrderDate DESC) AS OrderDate
FROM Test.Customer AS c
WHERE c.TerritoryID = 2;
```

Запишите общую стоимость запроса Query 1.

(Вы можете найти ее величину на вкладке **Execution Plan** (План выполнения), перемещая курсор по инструкции SELECT до обнаружения значения, названного **Estimated Subtree Cost** (Предполагаемая стоимость поддерева).)

5. В имеющемся окне запроса для проверки производительности запроса, использующего оператор OUTER APPLY, введите, выделите и выполните запрос Query 2.

```
-- Query 2
SELECT c.CustomerID, c.AccountNumber, o.*
FROM Test.Customer AS c
OUTER APPLY (SELECT TOP(1) soh.SalesOrderID, soh.OrderDate
 FROM Test.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 ORDER BY OrderDate DESC) AS o
WHERE c.TerritoryID = 2;
```

Запишите общую стоимость запроса Query 2.

6. В имеющемся окне запроса для тестирования производительности запроса, применяющего функцию ROW\_NUMBER, введите, выделите и выполните запрос Query 3.

```
-- Query 3
WITH a AS (SELECT c.CustomerID, c.AccountNumber, c.TerritoryID
 , soh.SalesOrderID, soh.OrderDate
 , ROW_NUMBER() OVER (PARTITION BY c.CustomerID
 ORDER BY soh.OrderDate DESC) AS RowNo
 FROM Test.Customer AS c
 LEFT OUTER JOIN Test.SalesOrderHeader AS soh
 ON soh.CustomerID = c.CustomerID)
```

```
SELECT a.CustomerID, a.AccountNumber, a.SalesOrderID, a.OrderDate
FROM a
WHERE a.RowNo = 1 AND a.TerritoryID = 2;
```

Запишите общую стоимость запроса Query 3.

## Упражнение 2. Тестирование с помощью большого результирующего набора

В этом упражнении вы выполните три запроса, упомянутые во вводном абзаце практического занятия, и запишите стоимость каждого из них. В данном случае параметр, задаваемый во всех трех запросах (`TerritoryID`, выделенный жирным шрифтом в листинге) приводит к формированию большого результирующего набора, состоящего из 3433 строк (сравните с 64 строками из упражнения 1).

1. Если необходимо, откройте среду SSMS и установите соединение с подходящим экземпляром SQL Server 2008.
2. В окне создания запроса для подключения базы данных TestDB, созданной в упражнении 1, введите и выполните следующую инструкцию SQL.  
`USE TestDB;`
3. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню Query (Запрос).
4. В имеющемся окне запроса для проверки производительности запроса, использующего два связанных вложенных запроса и большой результирующий набор, введите, выделите и выполните запрос Query 1, приведенный далее.

Применение в этом запросе двух отдельных связанных вложенных запросов не гарантирует возврата этими вложенными запросами данных из одной и той же строки таблицы `Test.Customer`.

```
-- Query 1
SELECT c.CustomerID, c.AccountNumber
 , (SELECT TOP(1) soh.SalesOrderID
 FROM Test.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 ORDER BY OrderDate DESC) AS SalesOrderID
 , (SELECT TOP(1) soh.OrderDate
 FROM Test.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 ORDER BY OrderDate DESC) AS OrderDate
FROM Test.Customer AS c
WHERE c.TerritoryID = 1;
```

Запишите общую стоимость запроса Query 1.

(Вы можете найти ее величину на вкладке **Execution Plan** (План выполнения), перемещая курсор по инструкции **SELECT** до обнаружения значения, названного **Estimated Subtree Cost** (Предполагаемая стоимость поддерева).)

- В имеющемся окне запроса для тестирования запроса, применяющего оператор **OUTER APPLY** и большой результирующий набор, введите, выделите и выполните запрос Query 2.

```
-- Query 2
SELECT c.CustomerID, c.AccountNumber, o.*
FROM Test.Customer AS c
OUTER APPLY (
 SELECT TOP(1) soh.SalesOrderID, soh.OrderDate
 FROM Test.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 ORDER BY OrderDate DESC
) AS o
WHERE c.TerritoryID = 1;
```

Запишите общую стоимость запроса Query 2.

- В имеющемся окне запроса для тестирования запроса, применяющего функцию **ROW\_NUMBER** и большой результирующий набор, введите, выделите и выполните запрос Query 3.

```
-- Query 3
WITH a AS (SELECT c.CustomerID, c.AccountNumber, c.TerritoryID
 , soh.SalesOrderID, soh.OrderDate
 , ROW_NUMBER() OVER (PARTITION BY c.CustomerID
 ORDER BY soh.OrderDate DESC) AS RowNo
 FROM Test.Customer AS c
 LEFT OUTER JOIN Test.SalesOrderHeader AS soh
 ON soh.CustomerID = c.CustomerID)
SELECT a.CustomerID, a.AccountNumber, a.SalesOrderID, a.OrderDate
FROM a
WHERE a.RowNo = 1 AND a.TerritoryID = 1;
```

Какова общая стоимость запроса Query 3?

### Упражнение 3. Оптимизация запроса Query 3

В этом упражнении вы внесете в запрос Query 3 небольшие изменения для его оптимизации.

- Если необходимо, откройте среду SSMS и установите соединение с подходящим экземпляром SQL Server 2008.
- В окне создания запроса для подключения базы данных TestDB, созданной в упражнении 1, введите и выполните следующую инструкцию SQL.

```
USE TestDB;
```

3. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню Query (Запрос).
4. В имеющемся окне запроса введите, выделите и выполните новую версию запроса Query 3 с маленьким результирующим набором (TerritoryID = 2). Изменения, внесенные в предыдущую версию запроса, выделены жирным шрифтом.

```
WITH a AS (SELECT c.CustomerID, c.AccountNumber, c.TerritoryID
 , soh.SalesOrderID, soh.OrderDate
 , ROW_NUMBER() OVER (PARTITION BY c.CustomerID
 ORDER BY soh.OrderDate DESC) AS RowNo
FROM Test.Customer AS c
LEFT OUTER JOIN Test.SalesOrderHeader AS soh
ON soh.CustomerID = c.CustomerID
WHERE c.TerritoryID = 2)
SELECT a.CustomerID, a.AccountNumber, a.SalesOrderID, a.OrderDate
FROM a
WHERE a.RowNo = 1;
```

Запишите общую стоимость этой версии запроса Query 3.

5. В имеющемся окне запроса введите, выделите и выполните новую версию запроса Query 3 с большим результирующим набором (TerritoryID = 1). Отличия от предыдущей версии запроса снова выделены жирным шрифтом.

```
WITH a AS (SELECT c.CustomerID, c.AccountNumber, c.TerritoryID
 , soh.SalesOrderID, soh.OrderDate
 , ROW_NUMBER() OVER (PARTITION BY c.CustomerID
 ORDER BY soh.OrderDate DESC) AS RowNo
FROM Test.Customer AS c
LEFT OUTER JOIN Test.SalesOrderHeader AS soh
ON soh.CustomerID = c.CustomerID
WHERE c.TerritoryID = 1)
SELECT a.CustomerID, a.AccountNumber, a.SalesOrderID, a.OrderDate
FROM a
WHERE a.RowNo = 1;
```

Какова общая стоимость этой версии запроса Query 3 с большим результирующим набором?

6. Для чистки базы данных после этого практического занятия закройте все открытые в среде SSMS окна запроса и выполните следующие инструкции SQL.

```
USE master;
DROP DATABASE TestDB;
```

## Резюме урока

- ◆ Понимание того, как логически устроены запросы, очень важно для уверенности в правильности возвращаемых результатов.
- ◆ Представление о том, как логически устроены запросы, поможет вам понять, какие физические структуры (например, индексы) ускоряют выполнение запроса.
- ◆ При измерении производительности необходимо иметь четкое представление об используемых показателях производительности.

## УРОК 2. Создание индексов

SQL Server 2008 поддерживает два базовых типа индексов: кластеризованные и некластеризованные. Индексы обоих типов реализуются как сбалансированное дерево, в котором уровень листьев находится на нижнем уровне структуры. Разница между индексами двух типов состоит в том, что кластеризованный индекс представляет собой настоящую таблицу, т. е. нижний уровень кластеризованного индекса содержит реальные строки таблицы, включая все столбцы. Некластеризованный индекс содержит только столбцы, включенные в ключ индекса, и указатель на строку реальных данных. Если для таблицы не определен кластеризованный индекс, она называется *кучей* (heap) или неотсортированной таблицей. Можно сказать, что таблица может быть представлена в одной из двух форм: в виде кучи (неотсортированная) или в виде кластеризованного индекса (отсортированная).

---

### *После усвоения этого урока вы узнаете больше о том*

---

- Как выбрать тип индекса для реализации.
- Как создавать индексы.

Расчетное время урока — 40 мин.

## Повышение производительности с помощью индексов покрытия

Назначение индекса покрытия заключается в том, что SQL Server не нужно устанавливать соответствие между некластеризованным индексом и таблицей для возврата результатов запроса. Поскольку кластеризованный индекс — это настоящая таблица, кластеризованные индексы всегда покрывают запросы.

Рассмотрим индекс покрытия — он должен содержать все столбцы, на которые есть ссылки в запросе (в любом предложении SELECT, JOIN, WHERE, GROUP BY, HAVING и т. д.). Рассмотрим таблицу TEST.TABLEA, состоящую из столбцов Column1, Column2, Column3, и запрос.

```
SELECT Column1 FROM Test.TableA
WHERE Column2 = 1;
```

Для того чтобы индекс покрывал этот запрос, он должен содержать как минимум столбцы Column1 и Column2. Сделать это можно разными способами. Все следующие индексы покрывают данный запрос.

```
CREATE NONCLUSTERED INDEX TestIndex ON Test.TableA (Col1, Col2);
CREATE NONCLUSTERED INDEX TestIndex ON Test.TableA (Col2, Col1);
CREATE NONCLUSTERED INDEX TestIndex ON Test.TableA (Col1) INCLUDE (Col2);
CREATE NONCLUSTERED INDEX TestIndex ON Test.TableA (Col2) INCLUDE (Col1);
CREATE NONCLUSTERED INDEX TestIndex ON Test.TableA (Col1, Col2, Col3);
CREATE NONCLUSTERED INDEX TestIndex ON Test.TableA (Col3) INCLUDE (Col1,
Col2);
```

Как видите, достаточно обнаружить столбцы в индексе; их позиция — находятся они в ключе индекса или являются включенными столбцами (более подробно обсуждаются в разд. *"Применение включенных столбцов и снижение глубины индекса"* далее в этом уроке) — не имеет значения. Конечно, и план выполнения, и производительность у этих индексов могут сильно отличаться, но все они покрывают запрос.

Рост производительности, полученный в результате применения индекса с покрытием, значителен в запросах, возвращающих большое количество строк (неселективный запрос), и гораздо меньше в запросах, возвращающих несколько строк (селективный запрос). Имейте в виду, что небольшое количество строк — понятие относительное, оно может равняться 10 для таблицы с парой сотен строк и 1000 в случае таблицы, содержащей миллионы строк. В этом разделе проводится сравнение производительности четырех запросов. Таблица, к которой обращены эти запросы, имеет следующую схему и содержит 1 000 000 строк.

```
CREATE TABLE Test.CoveredIndexTest (
 Col1 INT NOT NULL
 , Col2 NVARCHAR(2047) NOT NULL
);
INSERT Test.CoveredIndexTest (Col1, Col2)
VALUES (0, 'A lonely row...');
INSERT Test.CoveredIndexTest (Col1, Col2)
SELECT TOP(999999) message_id, text FROM sys.messages AS sm
CROSS JOIN (SELECT TOP(15) 1 AS Col FROM sys.messages
) AS x;
```

На тестовой для этого примера машине размер приведенной таблицы равен 27 377 страницам (приблизительно 213 Мбайт). Кроме того, отметьте, что эта таблица представляет собой кучу, т. е. для нее не определен кластеризованный индекс. Определения из запросов и индексов, используемых в данном тесте, приведены далее. Показатели производительности запросов (измерена в считываниях страниц) приведены в табл. 6.4.



```
-- Непокрывающий индекс
CREATE NONCLUSTERED INDEX NonCovered ON Test.CoveredIndexTest (Col1);

-- Покрывающий индекс
CREATE NONCLUSTERED INDEX Covered ON Test.CoveredIndexTest (Col1) INCLUDE
 (Col2);
-- Query 1:
-- возвращает 1 строку
SELECT Col1, Col2 FROM Test.CoveredIndexTest
 WHERE Col1 = 0;

-- Query 2:
-- возвращает в грубом приближении 0.1% строк, найденных в таблице
-- (1,056 строк)
SELECT Col1, Col2 FROM Test.CoveredIndexTest
 WHERE Col1 BETWEEN 1205 AND 1225;

-- Query 3:
-- возвращает в грубом приближении 0.5% строк, найденных в таблице
-- (5,016 строк)
SELECT Col1, Col2 FROM Test.CoveredIndexTest
 WHERE Col1 BETWEEN 1205 AND 1426;

-- Query 4 (неселективный):
-- возвращает в грубом приближении 5% строк, найденных в таблице
-- (50,028 строк)
SELECT Col1, Col2 FROM Test.CoveredIndexTest
 WHERE Col1 BETWEEN 1205 AND 2298;
```

**Таблица 6.4.** Матрица производительности запросов в логических считываниях

|                         | Query 1<br>(1 строка),<br>страниц | Query 2<br>(селективный),<br>страниц | Query 3<br>(отчасти селек-<br>тивный), страниц | Query 4<br>(неселектив-<br>ный), страниц |
|-------------------------|-----------------------------------|--------------------------------------|------------------------------------------------|------------------------------------------|
| Без индекса             | 29 141                            | 29 141                               | 29 141                                         | 29 141                                   |
| Непокрывающий<br>индекс | 4                                 | 1703                                 | 5099                                           | 46 647                                   |
| Покрывающий<br>индекс   | 3                                 | 43                                   | 142                                            | 1346                                     |

Показатель производительности, использованный в табл. 6.4, — количество страниц данных, которые SQL Server обработал во время выполнения запроса (SET STATISTICS IO ON, logical reads).

Обратите внимание на то, что так называемый селективный запрос (Query 2) возвращает 0.01% строк таблицы. Для таблицы указанного размера это равно

1000 строк. Если вы говорите с кем-нибудь о количестве строк, на которые повлиял запрос, и он или она отмечают, что количество строк — "лишь малый процент всех строк таблицы", как правило, это означает значительное количество строк.

Далее приведены выводы, которые можно сделать на основе проведенного теста. (Они касаются только производительности считываний, о производительности записи речь пойдет позже в этом уроке.)

- ◆ Индекс с покрытием всегда действует лучше, чем индекс без покрытия.
- ◆ В запросах, возвращающих ограниченное число строк, индекс без покрытия также действует очень хорошо.
- ◆ В отчасти селективном запросе (Query 3) индекс без покрытия считывает в 34 раза больше страниц, чем индекс с покрытием. В этом случае оптимизатор рассматривает запрос как селективный, если он подобрал в грубом приближении менее 0.77% строк таблицы.

### **Применение включенных столбцов и снижение глубины индекса**

В версиях, предшествующих SQL Server 2005, создание некластеризованных индексов с покрытием часто было невозможно из-за того, что индекс мог содержать не более 16 столбцов или быть размером не более 900 байтов. Новое средство *Included Column* (включенный столбец) делает возможным добавление столбцов в индекс без вставки их в ключ индекса. *Включенные* столбцы не могут применяться для сортировки или фильтрации; их единственное достоинство — снижение количества считываний страниц с помощью полностью покрытых запросов и за счет исключения подстановок из таблицы (table lookups).

Индекс может иметь максимум 1023 столбца, а таблица — максимум 1024 столбца, что делает возможным создание некластеризованного индекса, который покрывает всю таблицу, что почти подобно наличию второго кластеризованного индекса! Кроме того, в индекс разрешено вставлять в виде включенного столбца столбцы с большими типами данных, такими как `VARCHAR(max)`, `NVARCHAR(max)`, `VARBINARY(max)`, `XML`, `TEXT`, `NTEXT` и `IMAGE`.

Частью ключа индекса должны быть только столбцы, применяемые для фильтрации, группировки или сортировки; все остальные столбцы, входящие в индекс, могут быть включенными столбцами. Помимо увеличения количества столбцов, допустимых в индексе, включенные столбцы обладают и другими достоинствами. В следующем сценарии SQL создается таблица из 1 000 000 строк с двумя индексами. У одного индекса все столбцы в ключе индекса, а у второго индекса только один столбец в ключе (тот, по которому будет выполняться фильтрация), а остальные столбцы включенные. Размер каждой строки индекса чуть больше 300 байтов. Такая строка индекса может показаться очень длинной, но приве-

денный размер строки встречается нередко. Кроме того, он компенсирует наличие лишь 1 млн строк в тестовой таблице; в таблицах большего размера, для того чтобы создать разницу в производительности, не понадобятся строки индекса такого внушительного размера. В следующем сценарии определяются (и заполняются) объекты и индексы, используемые в последующих примерах.

```
CREATE TABLE Test.IncludedColumnsTest (
 PKCol1 UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID()
 PRIMARY KEY CLUSTERED
 ,Col1 INT IDENTITY NOT NULL
 ,Col2 CHAR(20) NOT NULL
 ,Col3 CHAR(20) NOT NULL
 ,Col4 CHAR(20) NOT NULL
 ,Col5 CHAR(20) NOT NULL
 ,Col6 CHAR(20) NOT NULL
 ,Col7 CHAR(20) NOT NULL
 ,Col8 CHAR(20) NOT NULL
 ,Col9 CHAR(20) NOT NULL
 ,Col10 CHAR(20) NOT NULL
 ,Col11 CHAR(20) NOT NULL
 ,Col12 CHAR(20) NOT NULL
 ,Col13 CHAR(20) NOT NULL
 ,Col14 CHAR(20) NOT NULL
 ,Col15 CHAR(20) NOT NULL
 ,Col16 CHAR(20) NOT NULL
);
INSERT Test.IncludedColumnsTest (Col2, Col3, Col4, Col5, Col6, Col7,
 Col8, Col9, Col10, Col11, Col12, Col13, Col14, Col15, Col16)
SELECT TOP(1000000)
 CAST(message_id AS CHAR(20)) AS Col2
 ,CAST(message_id AS CHAR(20)) AS Col3
 ,CAST(message_id AS CHAR(20)) AS Col4
 ,CAST(message_id AS CHAR(20)) AS Col5
 ,CAST(message_id AS CHAR(20)) AS Col6
 ,CAST(message_id AS CHAR(20)) AS Col7
 ,CAST(message_id AS CHAR(20)) AS Col8
 ,CAST(message_id AS CHAR(20)) AS Col9
 ,CAST(message_id AS CHAR(20)) AS Col10
 ,CAST(message_id AS CHAR(20)) AS Col11
 ,CAST(message_id AS CHAR(20)) AS Col12
 ,CAST(message_id AS CHAR(20)) AS Col13
 ,CAST(message_id AS CHAR(20)) AS Col14
 ,CAST(message_id AS CHAR(20)) AS Col15
 ,CAST(message_id AS CHAR(20)) AS Col16
FROM sys.messages AS sm
CROSS JOIN (SELECT TOP(15) 1 AS Col FROM sys.messages) AS x;
```

```
CREATE NONCLUSTERED INDEX IncludedColumns ON Test.IncludedColumnsTest
 (Col1)
 INCLUDE (Col2, Col3, Col4, Col5, Col6, Col7, Col8, Col9, Col10,
 Col11, Col12, Col13, Col14, Col15, Col16);
```

```
CREATE NONCLUSTERED INDEX NoIncludedColumns ON Test.IncludedColumnsTest
 (Col1, Col2, Col3, Col4, Col5, Col6, Col7, Col8, Col9, Col10, Col11,
 Col12, Col13, Col14, Col15, Col16);
```

В табл. 6.5 показаны некоторые интересные различия между индексами с включенными столбцами и без них.

**Таблица 6.5. Матрица размеров индекса**

| Характеристика                                           | IncludedColumn                                                                     | NoIncludedColumn                                                                   |
|----------------------------------------------------------|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| Общий размер                                             | 40 147 страниц                                                                     | 41 743 страницы                                                                    |
| Размер не самого нижнего уровня (уровня листьев) индекса | 146 страниц                                                                        | 1743 страницы                                                                      |
| Глубина индекса                                          | Три уровня (корневая страница + один промежуточный уровень + один уровень листьев) | Пять уровней (корневая страница + три промежуточных уровня + один уровень листьев) |
| Средний размер строк на уровнях выше уровня листьев      | 27 байтов                                                                          | 327 байтов                                                                         |
| Средний размер строк на уровне листьев индекса           | 321 байт                                                                           | 321 байт                                                                           |

Вы можете получить эти данные из функции динамического администрирования `sys.dm_db_index_physical_stats`, выполнив следующий запрос.

```
SELECT *
FROM sys.dm_db_index_physical_stats(
 DB_ID(), OBJECT_ID('Test.IncludedColumnsTest')
 ,NULL, NULL, 'DETAILED') AS a;
```

Общий размер индекса уменьшается примерно на 4%, т. к. на уровне листьев оба индекса содержат одинаковые данные. Однако более высокие уровни дерева индекса с включенными столбцами содержат только один столбец, являющийся ключом индекса (плюс указатели на следующий уровень), в то время как у другого индекса все столбцы являются частью ключа индекса, что делает размер каждой строки на уровнях более высоких, чем уровень листьев, приблизительно равным размеру строки на уровне листьев. В табл. 6.6 приведена схема размещения каждого уровня индекса `NoIncludedColumns`.

Таблица 6.6. Уровни индекса *NoIncludedColumns*

| Уровень              | Содержимое                                                             |
|----------------------|------------------------------------------------------------------------|
| Корневой             | 1 страница с 4 строками, указывающими на следующий уровень             |
| Первый промежуточный | 4 страницы с 72 строками в целом, указывающими на следующий уровень    |
| Второй промежуточный | 70 страниц с 1668 строками в целом, указывающими на следующий уровень  |
| Третий промежуточный | 1668 страниц с 40 000 строк в целом, указывающими на следующий уровень |
| Уровень листьев      | 40 000 страниц, содержащие все 1 000 000 строк индекса                 |

В табл. 6.7 приведена схема размещения всех уровней индекса *IncludedColumns*.

Таблица 6.7. Уровни индекса *IncludedColumns*

| Уровень         | Содержимое                                                               |
|-----------------|--------------------------------------------------------------------------|
| Корневой        | 1 страница со 145 строками, указывающими на следующий уровень            |
| Промежуточный   | 145 страниц с 40 003 строками в целом, указывающими на следующий уровень |
| Уровень листьев | 40 003 страницы, содержащие все 1 000 000 строк индекса                  |

Поскольку строки на страницах корневого и промежуточных уровней индекса *NoIncludedColumns* существенно длиннее аналогичных строк индекса *IncludedColumns*, требуется больше страниц (а следовательно, и больше уровней) для создания сбалансированного дерева индекса. Так как индекс *NoIncludedColumns* на два уровня (т. е. на 40%) глубже индекса *IncludedColumns*, каждому поиску в индексе *NoIncludedColumns* требуется на 2 считывания страниц больше для того, чтобы достичь нижнего уровня индекса. Это может показаться не великой разницей, но если индекс применяется для многократного поиска, например для соединений или частых запросов, дополнительные уровни ведут к снижению производительности.

В табл. 6.8 приведены три примера запросов, соединяющих с помощью разных индексов таблицу *Test.OtherTable* (другая таблица) с таблицей *Test.IncludedColumnsTest* (тест включенных столбцов). Обратите внимание на то, что индексные подсказки (*index hints*) `[WITH (INDEX)]` применяются только, чтобы заставить SQL Server использовать заданный индекс вместо оптимального (которым был бы индекс *IncludedColumns*). Добавляется новый индекс *NotCovered*, чтобы показать производительность некластеризованного индекса, не покрывающего запрос. В следующем сценарии определены дополнительные объекты и индексы, необходимые для примера.

```
-- Создание индекса NotCovered
CREATE NONCLUSTERED INDEX NotCovered ON Test.IncludedColumnsTest (Col1);

-- Создание и заполнение таблицы Test.OtherTable
CREATE TABLE Test.OtherTable (
 PKCol INT IDENTITY NOT NULL PRIMARY KEY,
 Col1 INT NOT NULL
);

INSERT Test.OtherTable (Col1)
 SELECT Col1 FROM Test.IncludedColumnsTest;
```

**Таблица 6.8.** Матрица сравнения производительности

| Запрос                                                                                                 | Определение                                                                                                                                                                                                     | Считываний страниц     |
|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------|
| <b>Запрос Query 1,<br/>индекс<br/>IncludedColumns,<br/>план выполнения пока-<br/>зан на рис. 6.5</b>   | <pre>SELECT o.PKCol, i.Col2 pages FROM Test.OtherTable AS o INNER JOIN     Test.IncludedColumnsTest AS i     WITH(INDEX(IncludedColumns))     ON o.Col1 = i.Col1 WHERE o.PKCol BETWEEN 1     AND 10000;</pre>   | <b>32 726 страниц</b>  |
| <b>Запрос Query 2,<br/>индекс<br/>NoIncludedColumns,<br/>план выполнения пока-<br/>зан на рис. 6.6</b> | <pre>SELECT o.PKCol, i.Col2 pages FROM Test.OtherTable AS o INNER JOIN     Test.IncludedColumnsTest AS i     WITH(INDEX(NoIncludedColumns))     ON o.Col1 = i.Col1 WHERE o.PKCol BETWEEN 1     AND 10000;</pre> | <b>53 994 страницы</b> |
| <b>Запрос Query 3,<br/>индекс NotCovered,<br/>план выполнения пока-<br/>зан на рис. 6.7</b>            | <pre>SELECT o.PKCol, i.Col2 pages FROM Test.OtherTable AS o INNER JOIN     Test.IncludedColumnsTest AS i     WITH(INDEX(NotCovered))     ON o.Col1 = i.Col1 WHERE o.PKCol BETWEEN 1     AND 10000;</pre>        | <b>62 900 страниц</b>  |

Запрос Query 1 с индексом IncludedColumns — самый высокопроизводительный запрос с 32 726 считываниями страниц. Запрос Query 2 с индексом NoIncludedColumns использует 53 994 считывания страниц. Как видите, разница между двумя индексами в количестве считываемых страниц приблизительно такая же, как разница между индексными уровнями (40%). Запрос Query 3 — самый низкопроизводительный запрос с 62 900 считываниями страниц, т. к. для

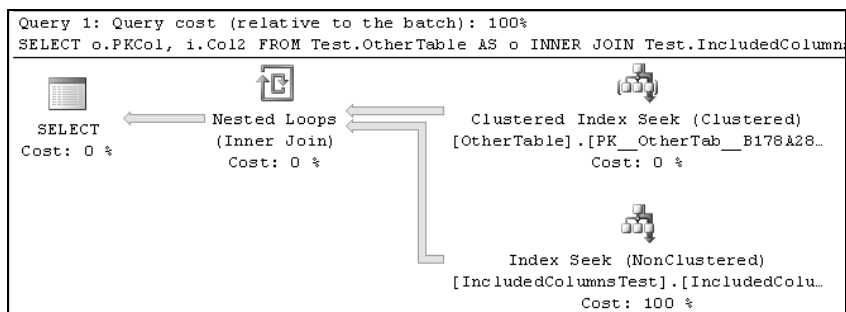


Рис. 6.5. Действующий план выполнения Query 1 в среде SSMS

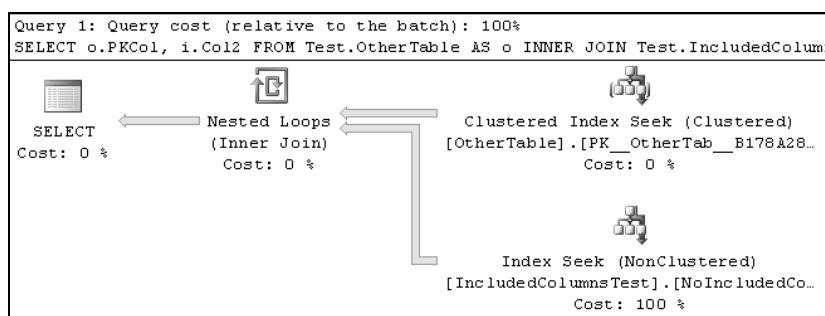


Рис. 6.6. Действующий план выполнения Query 2 в среде SSMS

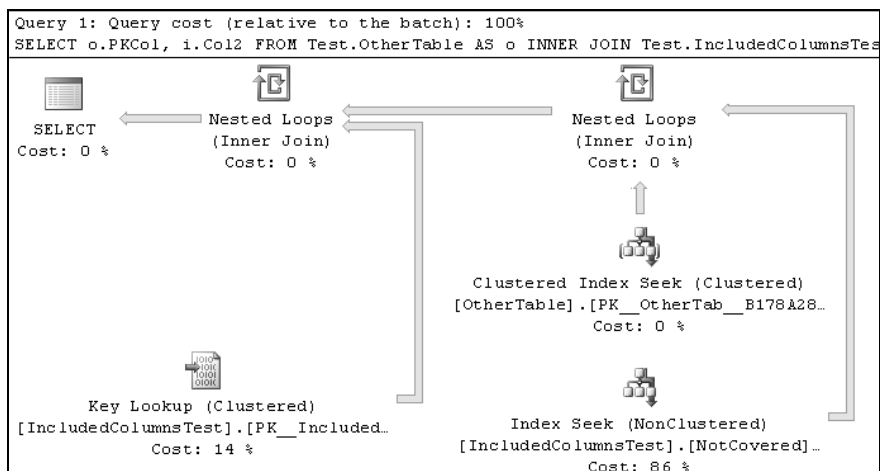


Рис. 6.7. Действующий план выполнения Query 3 в среде SSMS

извлечения из таблицы данных, которые не найдены в индексе таблицы, требуются дополнительные считывания. (Отметьте наличие дополнительного оператора **Nested Loops Join** в плане выполнения запроса Query 3.)

## Применение кластеризованных индексов

Поскольку кластеризованный индекс — это настоящая таблица, чтение из такого индекса никогда не приводит к подстановкам из базовой таблицы. Следовательно, кластеризованный индекс следует определять для столбцов, которые часто запрашиваются и, как правило, возвращают большой объем данных. Применение кластеризованного индекса устраняет проблему подстановок и поочередного извлечения большого количества строк. Хорошими кандидатами для кластеризованного индекса могут быть часто запрашиваемый столбец внешнего ключа таблицы (поиск по внешнему ключу обычно возвращает много строк) или наиболее часто участвующий в поиске столбец с датой. (Поиски по дате очень часто возвращают большое количество строк.)

При выборе столбца или столбцов, для которых создается кластеризованный индекс, важно иметь в виду, что размер ключа кластеризованного индекса должен быть минимальным из возможных. Если у таблицы есть кластеризованный индекс, все некластеризованные индексы этой таблицы применяют ключ кластеризованного индекса как указатель из строки некластеризованного индекса на таблицу. Если кластеризованного индекса нет, используется идентификатор строки, занимающий 8 байтов памяти в каждой строке некластеризованного индекса. В случае больших таблиц это может существенно увеличить размер индекса. Рассмотрим следующий сценарий.

- ◆ У вас есть таблица с 40 000 000 строк.
- ◆ У таблицы есть пять некластеризованных индексов.
- ◆ Размер ключа кластеризованного индекса — 60 байтов (это не так уж редко, если кластеризованные индексы объединяют несколько столбцов).

Общий размер всех указателей строк некластеризованных индексов данной таблицы (только указателей и ничего больше) будет равен

$$40\,000\,000 \times 5 \times 60 = 12\,000\,000\,000 \text{ байтов (приблизительно 12 Гбайт).}$$

Если ключ кластеризованного индекса заменить одним столбцом с меньшим типом данных, например, таким как `integer` для внешнего ключа, каждый указатель строки будет равен 4 байтам. Поскольку ко всем дубликатам ключа кластеризованного индекса для сохранения его внутренней уникальности добавляется 4 байта, реальный размер ключа кластеризованного индекса в этом случае был бы равен 8 байтам. Общий размер всех указателей строк стал бы следующим.

$$40\,000\,000 \times 5 \times 8 = 1\,600\,000\,000 \text{ байтов (приблизительно 1.5 Гбайт).}$$

Необходимая область хранения уменьшилась бы более чем на 10 Гбайт.



## Производительности считывания по сравнению с производительностью операции записи

Добавление индексов помогает поднять только производительность считывания. Производительность операции записи, как правило, снижается, т. к. должны храниться индексы, соответствующие текущему состоянию данных таблицы. Если у таблицы есть пять определенных для нее некластеризованных индексов, операция `INSERT` в такой таблице сводится в действительности к шести таким операциям; одна для таблицы и по одной для каждого индекса. То же самое происходит и с инструкциями `DELETE`. В случае инструкций `UPDATE` будут затронуты только те индексы, которые содержат столбцы, обновляемые данной инструкцией.

Если обновляются ключи индекса, строка индекса должна перемещаться на новое место в индексе (если обновление не касается только данных во включенных столбцах). В результате инструкция `UPDATE` делится на инструкцию `DELETE`, за которой следует инструкция `INSERT`. В зависимости от внутренней фрагментации страниц индекса, это также может привести к делению страниц.

Рассмотрим следующий простой тест производительности на таблице `Test.IndexInsertTest`, содержащей 1 000 000 строк. В каждом тесте вставляется 10 000 строк. Между тестами таблица пересоздается. Сначала инструкция `INSERT` выполняется в таблице без каких-либо некластеризованных индексов, затем она повторяется в таблице с одним некластеризованным индексом и в заключение инструкция повторяется в таблице с пятью некластеризованными индексами. Следующий программный код подготавливает тест.

```
CREATE TABLE Test.IndexInsertTest (
 PKCol UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID()
 PRIMARY KEY CLUSTERED
 , Col1 INT NOT NULL
);

INSERT Test.IndexInsertTest (Col1)
SELECT TOP(1000000)
 ROW_NUMBER() OVER (ORDER BY message_id) AS Col1
FROM sys.messages AS sm
CROSS JOIN (SELECT TOP(15) 1 AS Col FROM sys.messages) AS x;

-- Повторное создание кластеризованного индекса таблицы
ALTER INDEX ALL ON Test.OtherTable REBUILD;

-- Создание таблицы, содержащей строки, используемые для вставок.
CREATE TABLE Test.OtherTable (
 PKCol INT IDENTITY(100000,4) NOT NULL PRIMARY KEY
 , OtherCol INT NOT NULL
);
```

```
INSERT Test.OtherTable (OtherCol)
 SELECT Col1 FROM Test.IncludedColumnsTest
 WHERE Col1 BETWEEN 1 AND 10000;
```

Далее приведен первый тест без определения для таблицы некластеризованных индексов. План выполнения данной инструкции INSERT показан на рис. 6.8.

```
INSERT Test.IndexInsertTest (Col1)
 SELECT PKCol FROM Test.OtherTable;
```

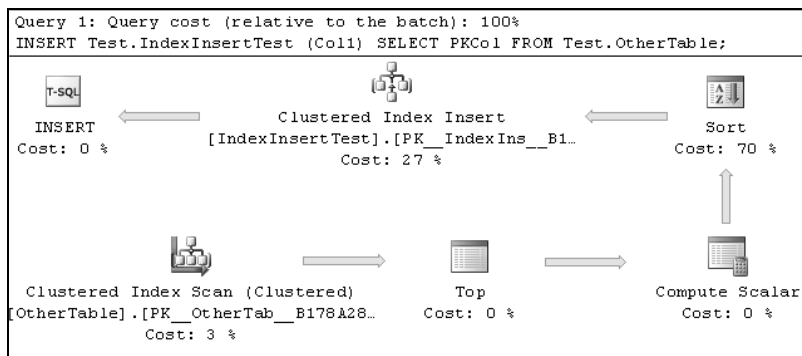


Рис. 6.8. В среде SSMS действующий план выполнения инструкции INSERT, примененной в первом тесте

Приблизительная стоимость запроса для инструкции INSERT из этого теста равна 0.88, и во время выполнения инструкций INSERT SQL Server повлиял на 32 085 страниц в таблице Test.IndexInsertTest.

Далее приведен второй тест с одним некластеризованным индексом, определенным для таблицы. План выполнения для этой инструкции INSERT показан на рис. 6.9.

```
-- 1. Удалите и создайте снова таблицу Test.IndexInsertTest.
-- 2. Добавьте один некластеризованный индекс.
CREATE NONCLUSTERED INDEX NCIdx1 ON Test.IndexInsertTest (Col1);

-- 3. Выполните инструкцию INSERT.
INSERT Test.IndexInsertTest (Col1)
 SELECT PKCol FROM Test.OtherTable;
```

Ориентировочная стоимость запроса для инструкции INSERT в этом тесте равна 1.58, и во время выполнения инструкций INSERT SQL Server воздействовал на 64 902 страницы в таблице Test.IndexInsertTest. Это в грубом приближении в два раза больше стоимости и считываний страниц, полученных в первом тесте.

Далее приведен третий тест с пятью некластеризованными индексами, определенными для таблицы. План выполнения инструкции INSERT показан на рис. 6.10.

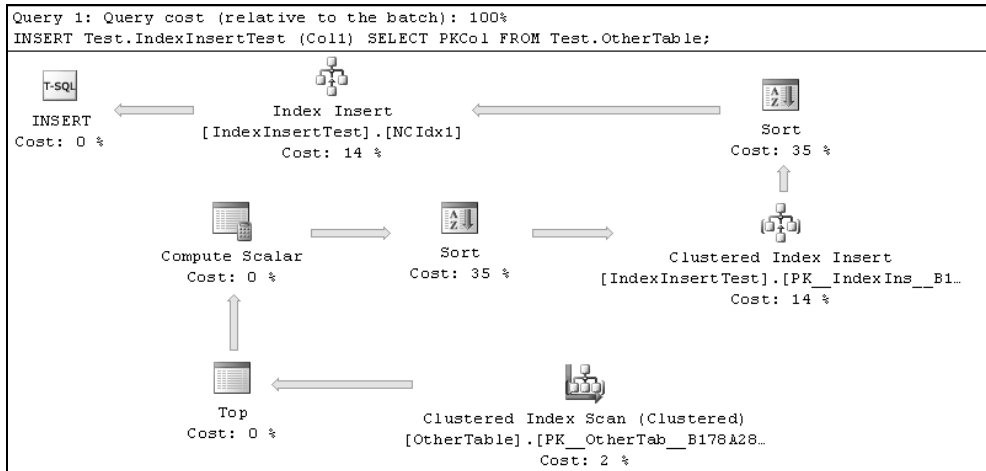


Рис. 6.9. В среде SSMS действующий план выполнения инструкции INSERT, примененной во втором тесте

```

-- 1. Удалите и создайте снова таблицу Test.IndexInsertTest.
-- 2. Добавьте пять некластеризованных индексов.
CREATE NONCLUSTERED INDEX NCIdx1 ON Test.IndexInsertTest (Col1);
CREATE NONCLUSTERED INDEX NCIdx2 ON Test.IndexInsertTest (Col1);
CREATE NONCLUSTERED INDEX NCIdx3 ON Test.IndexInsertTest (Col1);
CREATE NONCLUSTERED INDEX NCIdx4 ON Test.IndexInsertTest (Col1);
CREATE NONCLUSTERED INDEX NCIdx5 ON Test.IndexInsertTest (Col1);

-- 3. Выполните инструкцию INSERT.
INSERT Test.IndexInsertTest (Col1)
 SELECT PKCol FROM Test.OtherTable;

```

На этот раз приблизительная стоимость инструкции INSERT равна 5.04, и SQL Server обработал во время выполнения инструкций INSERT 196 170 страниц в таблице Test.IndexInsertTest. Как видите, стоимость выполнения инструкций INSERT приблизительно удваивается с каждым новым некластеризованным индексом. Однако в этом случае длина строк каждого некластеризованного индекса приблизительно равна ширине таблицы. В обычных таблицах некластеризованные индексы уже таблицы и не срезают производительность (в процентном отношении) до такой степени, как в данном тесте.

Поскольку соотношение между операциями чтения и записи сильно отличается в разных системах (и даже таблицах), как правило, полезно создать индексы для оптимизации производительности считываний, а затем протестировать влияние созданных индексов на производительность операций записи. И пока производительность записи приемлема (и у вас достаточно места на диске для управления созданными индексами), созданные индексы можно сохранить. Обычно реко-

мендуется выполнять подобный тест время от времени, чтобы убедиться в том, что соотношение операций чтения и записи для таблицы не изменилось.

Следует также отметить, что инструкции UPDATE и DELETE выигрывают от создания определенных индексов при поиске строк в таблице, которые они должны обновить или удалить.

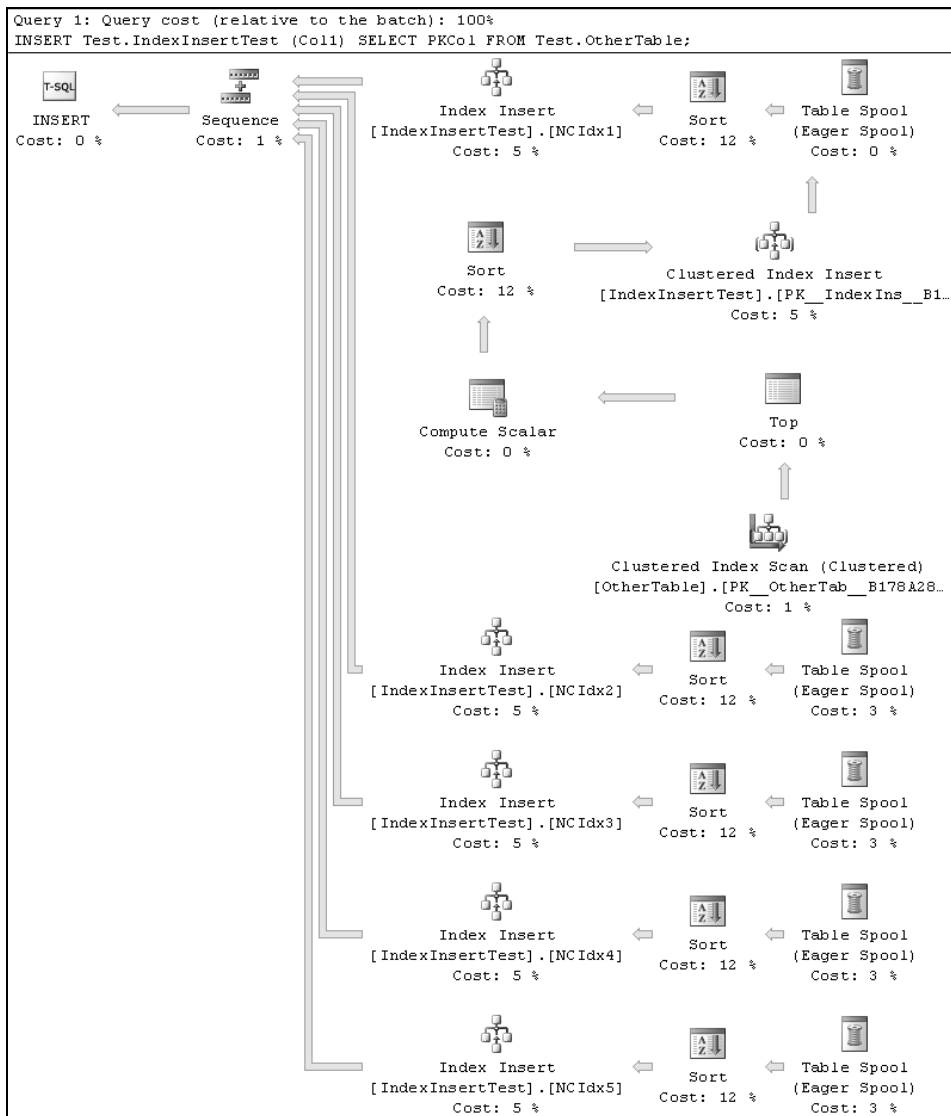


Рис. 6.10. В среде SSMS действующий план выполнения инструкции INSERT, примененной в третьем тесте

## Применение вычисляемых столбцов

*Вычисляемый столбец*, как правило, является производным от других столбцов этой же таблицы и может ссылаться в своем определении как на системные, так и на пользовательские функции. Для того чтобы можно было создать индекс для вычисляемого столбца, он должен удовлетворять некоторым требованиям, приведенным в разд. "Creating Indexes on Computed Columns" (Создание индексов вычисляемых столбцов) электронной документации по SQL Server на Web-странице <http://technet.microsoft.com/ru-ru/library/ms189292.aspx>.

Определение вычисляемого столбца и его индексирование позволяют создавать запросы, в которых в противном случае требовалось бы сканирование индекса или таблицы вместо операции поиска. Рассмотрим следующий запрос заказов на продажу из базы данных AdventureWorks. План выполнения запроса показан на рис. 6.11.

```
USE AdventureWorks;

-- Сначала для поддержки этого запроса создадим
-- индекс столбца OrderDate
CREATE NONCLUSTERED INDEX OrderDateIndex ON
 Sales.SalesOrderHeader (OrderDate);
GO

SET STATISTICS IO ON;

SELECT COUNT(*) FROM Sales.SalesOrderHeader
WHERE MONTH(OrderDate) = 5;
```

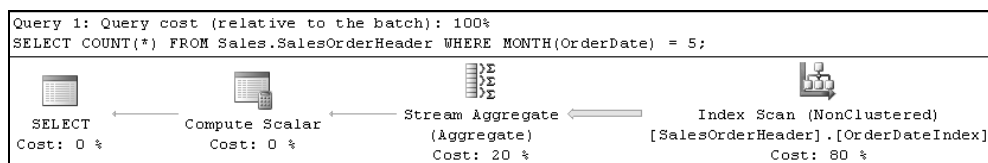


Рис. 6.11. План выполнения инструкции SELECT без SARG

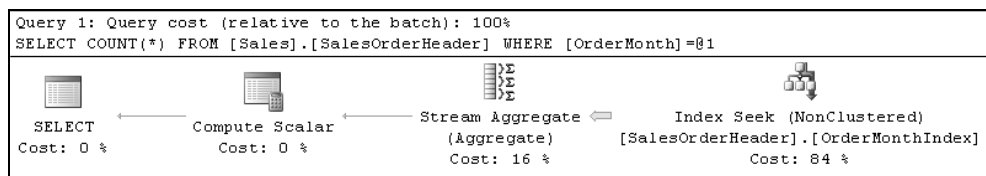
Поскольку в запросе не использовался корректный аргумент поиска (SARG) (столбец в предложении WHERE включен в выражение), индекс OrderDateIndex может применяться только для сканирования, а не для поиска. Для того чтобы иметь возможность поиска по индексу, SQL Server должен поддерживать индексирование результата вызова функции, в данном случае MONTH(OrderDate). Сделать это можно, добавив в таблицу вычисляемый столбец и проиндексировав его следующим образом (план выполнения показан на рис. 6.12).

```
-- Добавьте столбец
ALTER TABLE Sales.SalesOrderHeader
 ADD OrderMonth AS MONTH(OrderDate);

-- Создайте индекс вычисляемого столбца
CREATE NONCLUSTERED INDEX OrderMonthIndex
 ON Sales.SalesOrderHeader (OrderMonth);
GO

SET STATISTICS IO ON;

-- Выполните запрос и сошлитесь на новый столбец
SELECT COUNT(*) FROM Sales.SalesOrderHeader
WHERE OrderMonth = 5;
```



**Рис. 6.12.** Действующий план выполнения инструкции SELECT с вычисляемым столбцом в предложении WHERE

На этот раз запрос выполняет операцию поиска в индексе вычисляемого столбца, что в результате приводит к считываниям только восьми страниц. В зависимости от сложности вашего запроса и определения вычисляемого столбца оптимизатор автоматически использует индекс вычисляемого столбца даже при отсутствии в запросе ссылки на вычисляемый столбец. Следующий запрос, например, генерирует план выполнения, ранее показанный на рис. 6.12.

```
SET STATISTICS IO ON;

-- Выполните запрос, не ссылаясь на вычисляемый столбец
SELECT COUNT(*) FROM Sales.SalesOrderHeader
WHERE MONTH(OrderDate) = 5;
```

Как видите, SQL Server использовал индекс вычисляемого столбца, на который нет ссылки в запросе. Это прекрасное средство, т. к. оно делает возможным добавление вычисляемых столбцов и их индексирование и в дальнейшем применение нового индекса без корректировки запросов в приложениях или хранимых процедур.

Помимо использования индексированных вычисляемых столбцов с вызовами функций, вы можете применять индексированные вычисляемые столбцы для создания индексов для разных наборов обработки символьных параметров (collations). Предположим, что у вас есть таблица Test.Person со столбцом Name

(Имя), в котором используется набор `Latin1_General_CI_AI`. Вы хотите найти все строки, в которых имя начинается с символа `Ö`. В наборе `Latin1_General` точки над символом `O` считаются просто диакритическими знаками, но в других языках, таких как немецкий или шведский, `Ö` — это символ, отличающийся от символа `O`. Допустим, что таблицу запрашивают, как правило, англоговорящие клиенты, рассчитывающие в результате поиска с условием, например, `LIKE 'Ö%'`, получить оба символа (`O` и `Ö`), и иногда шведские клиенты, рассчитывающие получить в результате такого же поиска только `Ö`. Поскольку чаще таблицу запрашивают англоговорящие клиенты, есть смысл сохранять набор параметров символьной обработки `Latin1_General_CI_AI`, а при создании запросов шведскими клиентами применять ключевое слово `COLLATE` для явного задания набора `Finnish_Swedish_CI_AI`. Далее приведены сценарий и запросы. Планы выполнения для двух запросов из следующего сценария показаны на рис. 6.13 и 6.14.

```
-- Создание и заполнение таблицы
CREATE TABLE Test.ProductNames (
 Name NVARCHAR(50) COLLATE Latin1_General_CI_AI
);

INSERT Test.ProductNames (Name) VALUES ('Ol');
INSERT Test.ProductNames (Name) VALUES ('Olja');
INSERT Test.ProductNames (Name) VALUES ('Beer');
INSERT Test.ProductNames (Name) VALUES ('Oil');

CREATE CLUSTERED INDEX NameIndex ON Test.ProductNames (Name);
GO

-- Query 1
-- Запрос всех названий товаров, начинающихся с буквы O
-- с использованием стандартного набора параметров символьной обработки.
SELECT Name FROM Test.ProductNames
 WHERE Name LIKE 'O%';
```

Далее приведен результат запроса Query 1.

```
Name

Oil
Öl
Olja
```

Запрос Query 2 выглядит следующим образом.

```
-- Query 2
-- Запрос всех названий товаров, начинающихся с буквы Ö,
-- с использованием набора параметров Finnish_Swedish_CI_AI.
SELECT Name FROM Test.ProductNames
 WHERE Name LIKE 'Ö%' COLLATE Finnish_Swedish_CI_AI;
```

Далее приведен результат запроса Query 2.

Name

Ö1

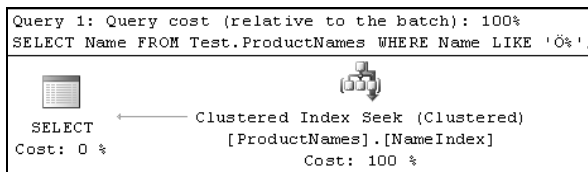


Рис. 6.13. В среде SSMS действующий план выполнения запроса Query 1

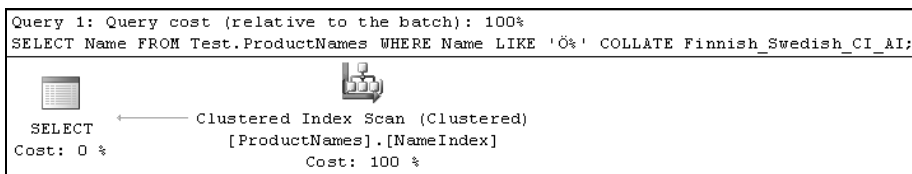


Рис. 6.14. В среде SSMS действующий план выполнения запроса Query 2

Сравнив планы выполнения запросов Query 1 (см. рис. 6.13) и Query 2 (см. рис. 6.14), вы увидите, что в Query 2 из-за необходимости при сравнении использовать набор параметров символьной обработки, отличающийся от заданного для столбца (а следовательно, и для индекса), вместо поиска в индексе, примененного в Query 1, используется сканирование кластеризованного индекса. Если в эту таблицу добавить индексированный вычисляемый столбец и задать для этого столбца набор параметров Finnish\_Swedish\_CI\_AI (как показано в следующем примере программного кода), SQL Server сможет автоматически использовать этот новый индекс. Обратите внимание на то, что сам запрос изменять не нужно, но предложенное решение жизнеспособно, только если вы используете относительно небольшое число наборов параметров символьной обработки, т. к. дополнительные индексы нужно хранить и поддерживать, как и все остальные. План выполнения запроса из следующего сценария показан на рис. 6.15.

-- Добавьте вычисляемый столбец с другим набором параметров.

```
ALTER TABLE Test.ProductNames
 ADD Name_Finnish_Swedish_CI_AI
 AS Name COLLATE Finnish_Swedish_CI_AI;
```

-- Создайте индекс для вычисляемого столбца.

```
CREATE NONCLUSTERED INDEX NameIndex2 ON Test.ProductNames
 (Name_Finnish_Swedish_CI_AI);
GO
```



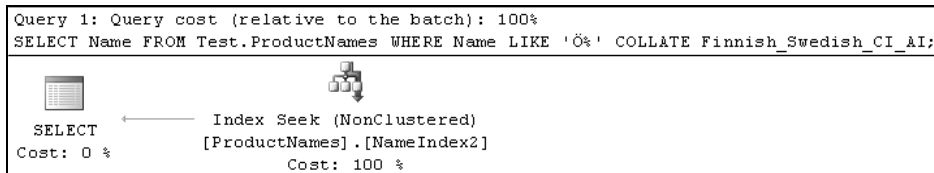
```
-- Запрос всех названий товаров, начинающихся с буквы Ö,
-- с применением набора параметров символьной обработки
-- Finnish_Swedish_CI_AI и без указания вычисляемого столбца.
SELECT Name FROM Test.ProductNames
 WHERE Name LIKE ' Ö%' COLLATE Finnish_Swedish_CI_AI;
```

Далее приведен результат запроса.

Name

-----

Ö1



**Рис. 6.15.** В среде SSMS действующий план выполнения запроса с применением индекса для альтернативного набора параметров символьной обработки

## Применение индексированных представлений

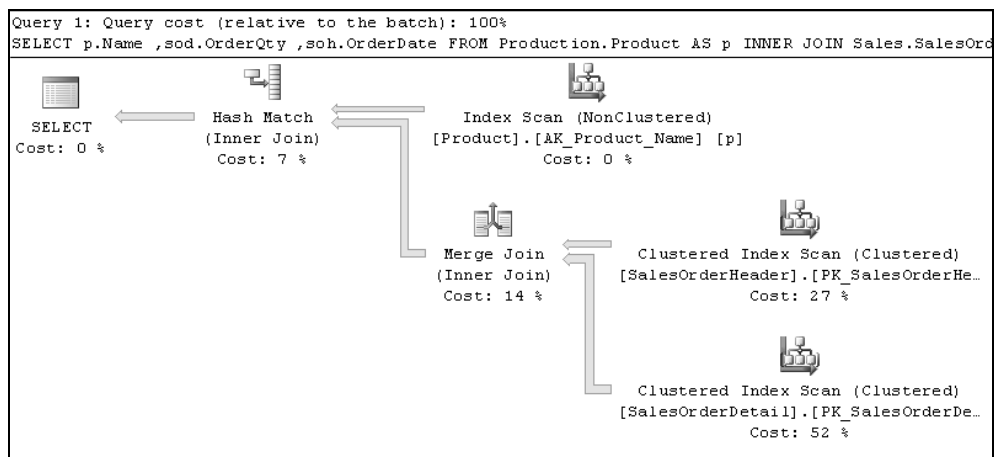
Обычное *представление базы данных* — это просто именованная инструкция SELECT, которая может использоваться в других инструкциях SELECT. Такие представления не оказывают влияния на производительность. Начиная с версии SQL Server 2000, появилась возможность создавать один или несколько индексов для представлений, удовлетворяющих определенным требованиям. Этих требований довольно много и их можно найти в разд. "Creating Indexed Views" (Создание индексированных представлений) электронной документации по SQL Server на Web-странице <http://msdn.microsoft.com/ru-ru/library/ms191432.aspx>. При создании индекса для представления оно материализуется. Это означает, что, оставаясь представлением с логической точки зрения, оно хранит данные, содержащиеся в представлении. (Подробное описание материализованных представлений см. в главе 5.) Если в таблицах, на которых основано представление, данные изменяются, представление автоматически обновляется для отражения этих изменений.

Создание индексированных представлений существенно повышает производительность операций чтения в запросе. Важная особенность индексированных представлений состоит в том, что оптимизатор (с учетом вашей версии SQL Server) может автоматически обнаружить и использовать индексированное представление, соответствующее конкретному запросу, даже если в запросе нет ссылки на это представление. Но это справедливо только для версий SQL Server 2008 Enterprise Edition и Developer Edition.

В следующем примере показан запрос и его план выполнения (рис. 6.16) без индексированного представления.

```
USE AdventureWorks;
```

```
SELECT p.Name, sod.OrderQty, soh.OrderDate
FROM Production.Product AS p
INNER JOIN Sales.SalesOrderDetail AS sod
 ON sod.ProductID = p.ProductID
INNER JOIN Sales.SalesOrderHeader AS soh
 ON soh.SalesOrderID = sod.SalesOrderID
WHERE soh.TerritoryID = 1;
```



**Рис. 6.16.** В среде SSMS действующий план выполнения запроса без индексированного представления

Стоимость приведенного запроса — 2.03. Далее для оптимизации запроса создается индексированное представление, и тот же самый запрос выполняется снова. План выполнения этого запроса показан на рис. 6.17. Первый индекс, созданный для представления, должен материализовать все представление, т. е. результирующий индекс должен быть кластеризованным. Первый индекс также должен быть уникальным (поэтому в пример с индексированным представлением добавлен столбец SalesOrderDetailID).

```
CREATE VIEW Sales.ProductsSoldVw
WITH SCHEMABINDING
AS
SELECT soh.TerritoryID, sod.SalesOrderDetailID, p.Name
 , sod.OrderQty, soh.OrderDate
FROM Production.Product AS p
INNER JOIN Sales.SalesOrderDetail AS sod
 ON sod.ProductID = p.ProductID
```

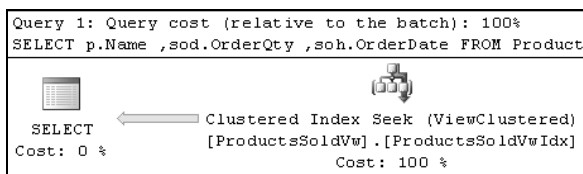
```

INNER JOIN Sales.SalesOrderHeader AS soh
 ON soh.SalesOrderID = sod.SalesOrderID
GO

CREATE UNIQUE CLUSTERED INDEX ProductsSoldVwIdx
 ON Sales.ProductsSoldVw (TerritoryID, SalesOrderDetailID);
GO

SELECT p.Name, sod.OrderQty, soh.OrderDate
FROM Production.Product AS p
INNER JOIN Sales.SalesOrderDetail AS sod
 ON sod.ProductID = p.ProductID
INNER JOIN Sales.SalesOrderHeader AS soh
 ON soh.SalesOrderID = sod.SalesOrderID
WHERE soh.TerritoryID = 1;

```



**Рис. 6.17.** В среде SSMS действующий план выполнения запроса с материализованным индексированным представлением, доступным в версии SQL Server 2008 Enterprise Edition или Developer Edition

На этот раз (если у вас версия SQL Server 2008 Enterprise Edition или Developer Edition) в запросе используется индексированное представление, даже несмотря на то, что на него нет ссылки, и стоимость запроса падает до 0.12.

После того как вы создали для представления уникальный кластеризованный индекс, можно создать дополнительные некластеризованные индексы того же самого представления.

### ***Версия SQL Server***

Если вы работаете в версии сервера, отличной от SQL Server 2008 Enterprise Edition или Developer Edition, ваш запрос должен явно ссылаться на представление, и вы должны вставить в запрос подсказку оптимизатора `WITH (NOEXPAND)`.

```

SELECT Name, OrderQty, OrderDate
FROM Sales.ProductsSoldVw WITH (NOEXPAND)
WHERE TerritoryID = 1;

```

## **Анализ применения индексов**

Поскольку индексы сопряжены с затратами (на область хранения и на поддержку их в актуальном состоянии при выполнении инструкций DML), очень важно от-

слеживать, какие индексы действительно используются в ваших приложениях. Если индекс никогда не применяется, вероятно, его можно удалить, как для экономии объема дискового пространства, так и для снижения стоимости операций записи. Но помните о том, что некоторые индексы создаются для конкретной задачи: например, для оптимизации отчетов о месячной зарплате. Следовательно, при удалении неиспользуемых или редко используемых индексов нужно соблюдать осторожность. Если вы удаляете редко используемый индекс, обязательно запишите ваши действия, чтобы удаленный индекс можно было при необходимости восстановить впоследствии.

Для получения информации о применении индексов можно запросить динамическое административное представление (dynamic management view, DMV) `sys.dm_db_index_usage_stats`. Особенно интересные столбцы этого представления приведены в табл. 6.9; сведения обо всей таблице см. в электронной документации на Web-странице <http://msdn.microsoft.com/ru-ru/library/ms188755.aspx>.

**Таблица 6.9.** Подмножество столбцов DMV `sys.dm_db_index_usage_stats`

| Название столбца              | Тип данных            | Описание                                                         |
|-------------------------------|-----------------------|------------------------------------------------------------------|
| <code>database_id</code>      | <code>smallint</code> | ID базы данных, в которой определены таблица или представление   |
| <code>object_id</code>        | <code>int</code>      | ID таблицы или представления, для которых определен индекс       |
| <code>index_id</code>         | <code>int</code>      | ID индекса                                                       |
| <code>user_seeks</code>       | <code>bigint</code>   | Количество поисков, инициированных пользовательскими запросами   |
| <code>user_scans</code>       | <code>bigint</code>   | Количество сканирований, выполненных пользовательскими запросами |
| <code>user_lookups</code>     | <code>bigint</code>   | Количество подстановок, выполненных пользовательскими запросами  |
| <code>user_updates</code>     | <code>bigint</code>   | Количество обновлений, выполненных пользовательскими запросами   |
| <code>last_user_seek</code>   | <code>datetime</code> | Время начала последнего пользовательского поиска                 |
| <code>last_user_scan</code>   | <code>datetime</code> | Время начала последнего пользовательского сканирования           |
| <code>last_user_lookup</code> | <code>datetime</code> | Время начала последней пользовательской подстановки              |

Обычно это представление следует запрашивать для получения данных об индексах, имеющих маленькие значения в столбцах `user_seeks` или `user_scans`. При любом перезапуске сервиса SQL Server все значения представления `sys.dm_db_index_usage_stats` переустанавливаются. Если база данных отсоединяется или закрывается, значения, относящиеся к ней, удаляются. Учтите, что ин-

дексы, не использовавшиеся после переустановки представления `sys.dm_db_index_usage_stats`, не включаются в него.

## Секционирование

Начиная с версии SQL Server 2005, у вас появилась возможность горизонтально (т. е. по строкам) секционировать таблицы и индексы, разделяя их на меньшие части. Эта функциональная возможность предназначена главным образом для повышения производительности импорта за счет уменьшения работы, выполняемой SQL Server при импорте данных. Почему же объем необходимой работы уменьшается? Как было показано ранее в этом уроке, у SQL Server возникает очень много работы по сопровождению индексов, когда строки вставляются, обновляются или удаляются. Это остается справедливым и при выполнении операций импорта. Если не применять секционирование, вы можете при любом импортировании новых данных позволить SQL Server автоматически обновлять индексы. Как вы уже видели, это приводит к существенной фрагментации. Если фрагментация значительна, вы, вероятно, захотите перестроить индексы после завершения импорта. Если подобное произошло, быстрее удалить все индексы, импортировать данные, а затем создать все индексы снова. В этом случае SQL Server не будет тратить ресурсы на поддержку индексов во время операции импорта. И здесь секционирование по-настоящему незаменимо. А если вместо повторного создания ваших индексов для целой таблицы сделать это только для вставляемых данных? Производительность импорта очень повысилась бы. С помощью секционирования можно было бы создать новую таблицу без индексов, импортировать данные в эту таблицу, создать индексы для этой новой таблицы и в заключение вставить ее как новую секцию в основную таблицу.

Секционирование может помочь и повышению производительности запроса, но лучший способ повышения производительности запроса — применение индексов, а не секционирование.

Для того чтобы секционировать таблицы и индексы, вы сначала должны создать два объекта: функцию секционирования (partition function) и схему секционирования (partition scheme). Функция секционирования просто определяет точки (вернее сказать, значения), в которых заканчивается каждая секция. Схема секционирования определяет, в какую файловую группу переходит каждая секция, вы можете определить одну файловую группу для хранения всех секций.

## Функции секционирования

Функции секционирования создаются инструкцией `CREATE PARTITION FUNCTION`. Функция секционирования — это просто список до 999 значений, определяющих разделители между секциями. Вы можете указать, что предложенные вами значения интерпретируются как "не превышающие" (`<=`) или как "меньшие" (`<`), определив функцию секционирования с ключевым словом `LEFT` или `RIGHT`.

В следующем программном коде создается функция секционирования, определенная как `LEFT`. Результирующие секции приведены в табл. 6.10.

```
CREATE PARTITION FUNCTION PF(INT)
AS RANGE LEFT
FOR VALUES (10, 20, 30);
```

**Таблица 6.10.** Секции, полученные с помощью функции секционирования `PF`, определенной как `LEFT`

| Номер секции | Диапазон секции             |
|--------------|-----------------------------|
| 1            | $\leq 10$                   |
| 2            | $> 10 \text{ AND } \leq 20$ |
| 3            | $> 20 \text{ AND } \leq 30$ |
| 4            | $> 30$                      |

Теперь рассмотрим следующий программный код, в котором создается фактически та же функция, но определенная как `RIGHT` вместо `LEFT`. Результирующие секции приведены в табл. 6.11.

```
CREATE PARTITION FUNCTION PF(INT)
AS RANGE RIGHT
FOR VALUES (10, 20, 30);
```

**Таблица 6.11.** Секции, полученные с помощью функции секционирования `PF`, определенной как `RIGHT`

| Номер секции | Диапазон секции             |
|--------------|-----------------------------|
| 1            | $< 10$                      |
| 2            | $\geq 10 \text{ AND } < 20$ |
| 3            | $\geq 20 \text{ AND } < 30$ |
| 4            | $\geq 30$                   |

## Схемы секционирования

Схема секционирования определяется с помощью инструкции `CREATE PARTITION SCHEME`. Схема секционирования — это просто карта распределения секций, каждой из них, по файловым группам. Использование разных файловых групп для разных секций обычно объясняется желанием хранить различные части таблицы на внешних устройствах разного типа: например, хранить более старые данные на более медленных, но более дешевых внешних устройствах, а новые данные на более быстрых, но дорогих устройствах.

В следующем примере создается схема секционирования, которая помещает каждую секцию, определенную в табл. 6.11, в свою файловую группу, как видите, для нескольких секций можно использовать одну и ту же файловую группу.

```
CREATE PARTITION SCHEME PS
AS PARTITION PF TO (FG1, FG2, FG1, FG2);
```

Далее приведен еще один пример, в котором создается схема секционирования, помещающая все секции в первичную файловую группу.

```
CREATE PARTITION SCHEME PS
AS PARTITION PF ALL TO ([PRIMARY]);
```

## Создание секционированной таблицы

После того как созданы функция секционирования и схема секционирования, можно создавать в схеме секционирования таблицы и индексы с помощью предложения `ON` инструкций `CREATE TABLE` и `CREATE INDEX`. Несмотря на то, что допускается иметь таблицу в одной схеме секционирования, а ее индексы в разных схемах секционирования (или один в схеме секционирования, а другой нет), рекомендуется создавать их все в одной схеме секционирования, для должной поддержки добавления и удаления секций без необходимости закрывать приложения. О таблице, у которой все индексы определены в одной схеме секционирования, говорят: секции с выравниванием. Неуникальные некластеризованные индексы при создании выравниваются автоматически в соответствии со схемой секционирования: т. е. вам даже не нужно задавать предложение `ON` в инструкции `CREATE INDEX`. В случае уникальных индексов для выравнивания их в соответствии с таблицей следует включить столбец секционирования в ключ индекса. Это, как правило, нарушает уникальность индекса. Например, если вы хотите добавить уникальный индекс для столбца `SSN` в таблице, секционированной по столбцу `ID`, для выравнивания вы должны создать составной индекс для столбцов `SSN` и `ID`. Обычно у вас должен быть один уникальный индекс, определенный без выравнивания, и если вам нужно добавить или удалить секцию, следует уничтожить невыровненный индекс, добавить или удалить секцию и затем снова создать индекс без выравнивания. Не нарушит ли все это секционирование? Нет, если у вас пять выровненных индексов и один невыровненный, вам необходимо создать заново только невыровненный индекс, в то время как при отсутствии секционирования вам пришлось бы создавать заново все шесть индексов.

Теперь давайте рассмотрим пример использования секционирования для повышения производительности импорта. В следующем пакете создается функция секционирования и схема секционирования, а также таблица и некластеризованный индекс, определяемый в схеме секционирования. После создания всех объектов применяется инструкция `INSERT` для заполнения таблицы 19 185 начальными строками. Несмотря на то, что в инструкции `CREATE INDEX` для задания схемы секционирования не применяется предложение `ON`, индекс формируется в схеме

секционирования. Последняя часть сценария — запрос к представлению каталога `sys.partitions`, который возвращает 8 секций, созданных сценарием: 4 для таблицы (`heap, index_id = 0`) и 4 для некластеризованного индекса (`index_id = 2`).

```
USE AdventureWorks;
```

```
CREATE PARTITION FUNCTION PFCustomerID (INT)
AS RANGE LEFT
FOR VALUES (5000, 10000, 15000);
```

```
CREATE PARTITION SCHEME PSCustomerID
AS PARTITION PFCustomerID ALL TO ([PRIMARY]);
```

```
CREATE TABLE Test.CustomersPartitioned (
 CustomerID INT IDENTITY NOT NULL
 , AccountNumber VARCHAR(50) NOT NULL
 , ModifiedDate DATETIME2 NOT NULL
) ON PSCustomerID (CustomerID);
```

```
CREATE NONCLUSTERED INDEX AccountNumberIdx
ON Test.CustomersPartitioned (AccountNumber);
```

```
INSERT Test.CustomersPartitioned (AccountNumber, ModifiedDate)
SELECT AccountNumber, ModifiedDate FROM Sales.Customer;
```

```
SELECT index_id, partition_number, rows FROM sys.partitions
WHERE object_id = OBJECT_ID('Test.CustomersPartitioned')
ORDER BY index_id, partition_number;
```

Далее приведены результаты запроса к представлению каталога `sys.partitions`.

| index_id | partition_number | rows |
|----------|------------------|------|
| 0        | 1                | 5000 |
| 0        | 2                | 5000 |
| 0        | 3                | 5000 |
| 0        | 4                | 4185 |
| 2        | 1                | 5000 |
| 2        | 2                | 5000 |
| 2        | 3                | 5000 |
| 2        | 4                | 4185 |

Для импорта данных в таблицу `Test.CustomersPartitioned` с помощью добавления новой секции следует определить новую секцию. Для этого вы должны определить следующее значение разделителя функции секционирования и вставить его в функцию. В следующем запросе определяется дополнительный разделитель для функции секционирования `PFCustomerID`.

```
SELECT MAX(CustomerID) AS MaxCustomerID FROM Test.CustomersPartitioned;
```



Далее приведен результат, содержащий нужное вам значение разделителя.

```
MaxCustomerID

19185
```

Теперь, имея значение 19 185, можно определить следующее значение разделителя как 19 185. Поскольку функция секционирования определена с ключевым словом `LEFT`, четвертая секция теперь содержит диапазон `CustomerID` 15 001—19 185, а новая секция включает в себя все `CustomerID`, большие 19 185. В следующем сценарии изменяется функция секционирования и приводится новый перечень секций таблицы.

```
ALTER PARTITION FUNCTION PFCustomerID()
 SPLIT RANGE (19185);

SELECT index_id, partition_number, rows FROM sys.partitions
 WHERE object_id = OBJECT_ID('Test.CustomersPartitioned')
 ORDER BY index_id, partition_number;
```

Далее приведены результаты запроса к представлению каталога `sys.partitions`, включающие новую секцию.

| index_id | partition_number | rows |
|----------|------------------|------|
| 0        | 1                | 5000 |
| 0        | 2                | 5000 |
| 0        | 3                | 5000 |
| 0        | 4                | 4185 |
| 0        | 5                | 0    |
| 2        | 1                | 5000 |
| 2        | 2                | 5000 |
| 2        | 3                | 5000 |
| 2        | 4                | 4185 |
| 2        | 5                | 0    |

Следующий шаг — импорт новых данных с помощью создания новой таблицы с точно такой же схемой, как и у основной таблицы, и последующего импортирования данных в нее. Имейте в виду, что если вы используете несколько файловых групп, эта новая таблица должна находиться в той же файловой группе, что и секция, в которую вы хотите ее превратить. В приведенном далее сценарии создается новая таблица, импортируются данные и создается некластеризованный индекс для таблицы. (Если у вас были какие-либо ограничения, например первичного ключа, проверочные или внешнего ключа, их также следует добавить перед тем, как вставлять новую таблицу в секционированную.) Одно предостережение, касающееся некластеризованного индекса: для того чтобы добавить новую таблицу в секционированную, вы должны включить столбец, по которому секционируется таблица, в определение некластеризованного индекса.

Это делается автоматически, если таблица уже секционирована, но не в случае новой таблицы, потому что она еще не разделена на секции, в этот момент она просто обычная таблица.

```
-- Новая пустая таблица.
CREATE TABLE Test.NewCustomers (
 CustomerID INT IDENTITY(19186, 1) NOT NULL
 , AccountNumber VARCHAR(50) NOT NULL
 , ModifiedDate DATETIME2 NOT NULL
);

-- Импорт в пустую таблицу, пока нет никаких индексов,
-- нуждающихся в поддержке!
INSERT Test.NewCustomers (AccountNumber, ModifiedDate)
SELECT TOP(3000) AccountNumber, ModifiedDate
FROM Sales.Customer;

-- Теперь создается индекс (и любые нужные ограничения).
CREATE NONCLUSTERED INDEX AccountNumberIdx
ON Test.NewCustomers (AccountNumber) INCLUDE (CustomerID);
```

Для того чтобы в заключение вставить таблицу Test.NewCustomers как секцию 5 в секционированную таблицу, следует определить для нее проверочное ограничение, гарантирующее ее соответствие значениям, допустимым в секции 5 (значения больше 19 185). В следующем сценарии добавляется проверочное ограничение и затем новая таблица с помощью текущей секции 5 (которая пуста) включается в секционированную таблицу, а текущая секция 5 становится пустой таблицей Test.NewCustomers, которую можно удалить.

```
-- Добавление ограничения check.
ALTER TABLE Test.NewCustomers ADD CHECK(CustomerID > 19185);

-- В секционированной таблице меняются местами новая таблица
-- и секция 5.
ALTER TABLE Test.NewCustomers SWITCH TO Test.CustomersPartitioned
PARTITION 5;

-- В заключение удаляется таблица Test.NewCustomers.
DROP TABLE Test.NewCustomers;

-- Выполнено:
SELECT index_id, partition_number, rows FROM sys.partitions
WHERE object_id = OBJECT_ID('Test.CustomersPartitioned')
ORDER BY index_id, partition_number;
```

Далее приведены результаты запроса к представлению каталога sys.partitions, показывающие, сколько строк было добавлено в новую секцию.

| index_id | partition_number | rows |
|----------|------------------|------|
| 0        | 1                | 5000 |
| 0        | 2                | 5000 |
| 0        | 3                | 5000 |
| 0        | 4                | 4185 |
| 0        | 5                | 3000 |
| 2        | 1                | 5000 |
| 2        | 2                | 5000 |
| 2        | 3                | 5000 |
| 2        | 4                | 4185 |
| 2        | 5                | 3000 |

Но почему же этот вариант импорта настолько лучше, чем импорт без секций? Во-первых, предположим, что во время операции нужно обрабатывать только импортированные строки — имеющиеся строки никогда не будут затронуты. Кроме того, облегчен доступ пользователей к секционированной таблице, она недоступна только во время выполнения инструкции `ALTER TABLE...SWITCH`. Поскольку эта инструкция изменяет только указатели в системном каталоге, она выполняется практически мгновенно, независимо от количества добавляемых строк.

Итак, вы можете значительно повысить производительность импорта, реализовав секционирование. Ну а что же с производительностью запроса? Да, секционирование может также повысить и производительность запроса, особенно если столбец секционирования — один из тех, которые вы часто применяете в запросах. Если так, SQL Server во время оптимизации может провести ограничение секций, т. к. ему необходимо выполнять поиск или сканирование только в определенных секциях, а не во всей таблице. Но если сравнить производительность запроса к секционированной таблице без подходящего индекса с запросом к несекционированной таблице, но с подходящим индексом, вы увидите, что индекс действует лучше, чем секционирование (без индекса). Следующий запрос выполняется в отношении схемы и строк (исходные 19 185 строк), определенных в предыдущем примере секционирования, и результаты показаны в табл. 6.12. Запрос сформирован так, что требуется сканирование только двух секций таблицы.

```
SET STATISTICS IO ON;
```

```
SELECT COUNT(*) FROM Test.CustomersPartitioned
WHERE CustomerID BETWEEN 1000 AND 10000;
```

Обратите внимание на то, что при отсутствии индекса запрос к секционированной таблице выполняется лучше всего. Но если есть индекс, наилучшим образом выполняется запрос к несекционированной таблице. Почему запрос к несекционированной таблице с индексом выполняется лучше, чем запрос к секционированной таблице с индексом? Потому что в секционированной таблице запрос выполняет операцию поиска по двум индексам (по одному на каждую секцию), а запрос к несекционированной таблице — поиск только по одному индексу.

Таблица 6.12. Сравнение производительности

| Описание                                              | Логических считываний (I/O), страниц | Стоимость |
|-------------------------------------------------------|--------------------------------------|-----------|
| Несекционированная таблица без индекса для CustomerID | 85                                   | 0.095     |
| Секционированная таблица без индекса для CustomerID   | 44                                   | 0.049     |
| Несекционированная таблица с индексом для CustomerID  | 20                                   | 0.030     |
| Секционированная таблица с индексом для CustomerID    | 23                                   | 0.031     |

## Автоматическая настройка индексов

Помимо ручной настройки индексов SQL Server предоставляет другие возможности, помогающие выбрать правильное индексирование в конкретном запросе. При просмотре графического плана выполнения вы можете встретить примечание о "Missing Index" (пропущенный индекс). В этом случае можно щелкнуть правой кнопкой мыши примечание, касающееся недостающего индекса, что позволит получить сценарий, необходимый для создания этого индекса. Сводку отсутствующих индексов, которые требовались оптимизатору, можно найти в административных представлениях базы данных (DMV) `sys.dm_db_missing_index_details`, `sys.dm_db_missing_index_groups` и `sys.dm_db_missing_index_group_stats`.

Вы также можете применить графическую утилиту Database Engine Tuning Advisor для получения сведений об индексах, индексированных представлениях и даже о секционированных решениях, которые могут помочь повысить производительность запроса. Database Engine Tuning Advisor можно также запустить для сценария, содержащего запросы, нуждающиеся в настройке, или для файла трассировки приложения SQL Server Profiler, содержащего рабочую нагрузку (workload) запросов, которым требуется настройка.

Учтите, что ни справка графического плана выполнения, касающаяся пропущенного индекса, вместе с DMV, содержащим сводку недостающих индексов, ни утилита Database Engine Tuning Advisor не могут заменить ручную настройку как запросов, так и индексов. Помните о том, что эти утилиты не могут знать, действительно ли вам нужно последнее соединение, извлекающее дополнительный столбец, чтобы улучшить внешний вид оперативного отчета — это решение можете принять только вы.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

### Индексирование для поддержки запросов

На этом практическом занятии вы примените в конкретном запросе два разных метода оптимизации. Запрос возвращает для всех клиентов, сделавших покупки,

идентификаторы клиентов и общий объем всех покупок в заданном регионе. На занятии рассматривается только производительность операций чтения, производительность операций записи не принимается во внимание.

Вы будете оптимизировать следующий запрос.

-- Запрос, который будет оптимизироваться:

```
USE AdventureWorks;
```

```
SELECT soh.CustomerID, SUM(sod.OrderQty * sod.UnitPrice) AS TotalPurchases
FROM Test.SalesOrderHeader AS soh
INNER JOIN Test.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.TerritoryID = 1
GROUP BY soh.CustomerID;
```

## Упражнение 1. Установка точки отсчета для производительности запроса

В этом упражнении вы создадите точку отсчета для запроса, который нуждается в оптимизации, выполнив его без добавления каких бы то ни было индексов.

1. Откройте среду SSMS и установите соединение с нужным экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных TestDB, схемы Test и двух таблиц, используемых в данном упражнении.

```
CREATE DATABASE TestDB;
```

```
GO
```

```
USE TestDB;
```

```
GO
```

```
CREATE SCHEMA Test;
```

```
GO
```

```
SELECT * INTO Test.SalesOrderHeader
FROM AdventureWorks.Sales.SalesOrderHeader;
```

```
GO
```

```
SELECT * INTO Test.SalesOrderDetail
FROM AdventureWorks.Sales.SalesOrderDetail;
```

```
GO
```

```
ALTER TABLE Test.SalesOrderHeader
 ADD CONSTRAINT PKSalesOrderHeader
 PRIMARY KEY(SalesOrderID);
```

```
GO
```

```
ALTER TABLE Test.SalesOrderDetail
 ADD CONSTRAINT PKSalesOrderDetail
 PRIMARY KEY(SalesOrderDetailID);
```

3. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню Query (Запрос).
4. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL для вывода сведений о считываниях страниц.

```
SET STATISTICS IO ON;
```

5. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL.

```
SELECT soh.CustomerID
 , SUM(sod.OrderQty * sod.UnitPrice) AS TotalPurchases
FROM Test.SalesOrderHeader AS soh
INNER JOIN Test.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.TerritoryID = 1
GROUP BY soh.CustomerID;
```

Запишите общую стоимость запроса. Таблицу, предназначенную для этого, можно найти в упражнении 5.

(Вы можете найти величину стоимости на вкладке **Execution Plan** (План выполнения), перемещая курсор по инструкции SELECT до обнаружения значения, названного **Estimated Subtree Cost** (Предполагаемая стоимость поддерева).)

Запишите общее количество считываний страниц в запросе. (Это значение можно найти, прокрутив до нижнего края вкладку **Messages** (Сообщения) и просуммировав значения логических считываний.)

6. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL для очистки базы данных после выполнения упражнения.

```
USE master;
DROP DATABASE TestDB;
```

## Упражнение 2. Настройка запроса с помощью кластеризованных индексов

В этом упражнении вы оптимизируете запрос, превратив ограничения первичного ключа в некластеризованные индексы, а затем создав подходящие кластеризованные индексы.

1. Откройте среду SSMS, если необходимо, и установите соединение с нужным экземпляром SQL Server 2008.

2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных TestDB, схемы Test и двух таблиц, используемых в данном упражнении.

```
CREATE DATABASE TestDB;
GO

USE TestDB;
GO

CREATE SCHEMA Test;
GO

SELECT * INTO Test.SalesOrderHeader
FROM AdventureWorks.Sales.SalesOrderHeader;
GO

SELECT * INTO Test.SalesOrderDetail
FROM AdventureWorks.Sales.SalesOrderDetail;
GO

ALTER TABLE Test.SalesOrderHeader
 ADD CONSTRAINT PKSalesOrderHeader
 PRIMARY KEY(SalesOrderID);
GO

ALTER TABLE Test.SalesOrderDetail
 ADD CONSTRAINT PKSalesOrderDetail
 PRIMARY KEY(SalesOrderDetailID);
```

3. В имеющемся окне запроса введите, выделите и выполните следующие инструкции SQL, преобразующие ограничение первичного ключа в таблице Test.SalesOrderHeader в некластеризованный индекс, а затем создайте для запроса подходящий кластеризованный индекс.

```
-- Превращение PK в некластеризованный индекс.
ALTER TABLE Test.SalesOrderHeader
 DROP CONSTRAINT PKSalesOrderHeader;

ALTER TABLE Test.SalesOrderHeader
 ADD CONSTRAINT PKSalesOrderHeader
 PRIMARY KEY NONCLUSTERED (SalesOrderID);

-- Создание кластеризованного индекса.
CREATE CLUSTERED INDEX CluIdx ON Test.SalesOrderHeader
 (TerritoryID, CustomerID);
```

4. В открытом окне запроса введите, выделите и выполните следующие инструкции SQL для преобразования ограничения первичного ключа таблицы

Test.SalesOrderDetail в некластеризованный индекс и затем создайте для запроса подходящий кластеризованный индекс.

```
-- Превращение PK в некластеризованный индекс.
ALTER TABLE Test.SalesOrderDetail
 DROP CONSTRAINT PKSalesOrderDetail;

ALTER TABLE Test.SalesOrderDetail
 ADD CONSTRAINT PKSalesOrderDetail
 PRIMARY KEY NONCLUSTERED (SalesOrderDetailID);

-- Создание кластеризованного индекса.
CREATE CLUSTERED INDEX CluIdx ON Test.SalesOrderDetail
 (SalesOrderID);
```

5. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню **Query** (Запрос).
6. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL для вывода сведений о считываниях страниц.

```
SET STATISTICS IO ON;
```

7. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL.

```
SELECT soh.CustomerID,
 SUM(sod.OrderQty * sod.UnitPrice) AS TotalPurchases
FROM Test.SalesOrderHeader AS soh
INNER JOIN Test.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.TerritoryID = 1
GROUP BY soh.CustomerID;
```

Запишите общую стоимость запроса.

Запишите общее количество считываний страниц в запросе.

8. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL для очистки базы данных после выполнения упражнения.

```
USE master;
DROP DATABASE TestDB;
```

### Упражнение 3. Настройка запроса с помощью покрывающих некластеризованных индексов

В этом упражнении вы оптимизируете запрос, создав покрывающие некластеризованные индексы.



1. Откройте среду SSMS, если необходимо, и установите соединение с нужным экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных TestDB, схемы Test и двух таблиц, используемых в данном упражнении.

```
CREATE DATABASE TestDB;
GO
```

```
USE TestDB;
GO
```

```
CREATE SCHEMA Test;
GO
```

```
SELECT * INTO Test.SalesOrderHeader
FROM AdventureWorks.Sales.SalesOrderHeader;
GO
```

```
SELECT * INTO Test.SalesOrderDetail
FROM AdventureWorks.Sales.SalesOrderDetail;
GO
```

```
ALTER TABLE Test.SalesOrderHeader
 ADD CONSTRAINT PKSalesOrderHeader
 PRIMARY KEY(SalesOrderID);
GO
```

```
ALTER TABLE Test.SalesOrderDetail
 ADD CONSTRAINT PKSalesOrderDetail
 PRIMARY KEY(SalesOrderDetailID);
```

3. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL для создания покрывающего некластеризованного индекса, который будет применяться запросом при доступе к таблице Test.SalesOrderHeader.

```
CREATE NONCLUSTERED INDEX TestIndex
 ON Test.SalesOrderHeader (TerritoryID, SalesOrderID)
 INCLUDE (CustomerID);
```

4. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL для создания покрывающего некластеризованного индекса, который будет использоваться запросом при доступе к таблице Test.SalesOrderDetail.

```
CREATE NONCLUSTERED INDEX TestIndex
 ON Test.SalesOrderDetail (SalesOrderID)
 INCLUDE (OrderQty, UnitPrice);
```

5. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню Query (Запрос).

6. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL для вывода сведений о считываниях страниц.

```
SET STATISTICS IO ON;
```

7. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL.

```
SELECT soh.CustomerID
 , SUM(sod.OrderQty * sod.UnitPrice) AS TotalPurchases
FROM Test.SalesOrderHeader AS soh
INNER JOIN Test.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.TerritoryID = 1
GROUP BY soh.CustomerID;
```

Запишите общую стоимость запроса.

Запишите общее количество считываний страниц в запросе.

8. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL для очистки базы данных после выполнения упражнения.

```
USE master;
DROP DATABASE TestDB;
```

## Упражнение 4. Настройка запроса с помощью применения индексируемого представления

В этом упражнении вы оптимизируете запрос, создав индексируемое представление, покрывающее запрос.

1. Откройте среду SSMS, если необходимо, и установите соединение с нужным экземпляром SQL Server 2008.
2. В окне создания запроса введите и выполните следующие инструкции SQL для создания базы данных TestDB, схемы Test и двух таблиц, используемых в данном упражнении.

```
CREATE DATABASE TestDB;
GO
```

```
USE TestDB;
GO
```

```
CREATE SCHEMA Test;
GO
```

```
SELECT * INTO Test.SalesOrderHeader
FROM AdventureWorks.Sales.SalesOrderHeader;
GO
```

```
SELECT * INTO Test.SalesOrderDetail
FROM AdventureWorks.Sales.SalesOrderDetail;
GO
```

```
ALTER TABLE Test.SalesOrderHeader
 ADD CONSTRAINT PKSalesOrderHeader
 PRIMARY KEY(SalesOrderID);
GO
```

```
ALTER TABLE Test.SalesOrderDetail
 ADD CONSTRAINT PKSalesOrderDetail
 PRIMARY KEY(SalesOrderDetailID);
```

3. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL для создания представления.

```
CREATE VIEW Test.SalesByCustomerVw
WITH SCHEMABINDING
AS
SELECT soh.TerritoryID, soh.CustomerID
 , SUM(sod.OrderQty * sod.UnitPrice) AS TotalPurchases
 , COUNT_BIG(*) AS NumberOfRows
FROM Test.SalesOrderHeader AS soh
 INNER JOIN Test.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
GROUP BY soh.TerritoryID, soh.CustomerID;
```

4. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL для создания индекса представления.

```
CREATE UNIQUE CLUSTERED INDEX SalesByCustomerVwIdx
 ON Test.SalesByCustomerVw (TerritoryID, CustomerID);
```

5. Включите в SSMS средство Actual Execution Plan (Действующий план выполнения), нажав комбинацию клавиш <Ctrl>+<M> или выбрав команду **Include Actual Execution Plan** (Включить действующий план выполнения) в меню **Query** (Запрос).

6. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL для вывода сведений о считываниях страниц.

```
SET STATISTICS IO ON;
```

7. В имеющемся окне запроса введите, выделите и выполните следующую инструкцию SQL.

```
SELECT soh.CustomerID
 , SUM(sod.OrderQty * sod.UnitPrice) AS TotalPurchases
```

```
FROM Test.SalesOrderHeader AS soh
INNER JOIN Test.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
WHERE soh.TerritoryID = 1
GROUP BY soh.CustomerID;
```

Изучив план выполнения запроса, убедитесь в том, что для выполнения запроса применяется индексированное представление. Если же оно не используется (это может быть, если вы применяете редакцию сервера, отличную от Developer Edition или Enterprise Edition SQL Server 2008), вместо предыдущего запроса выполните следующий запрос, чтобы заставить SQL Server использовать индексированное представление.

```
SELECT CustomerID, TotalPurchases
FROM Test.SalesByCustomerVw WITH (NOEXPAND)
WHERE TerritoryID = 1;
```

Запишите общую стоимость запроса.

Запишите общее количество считываний страниц в запросе.

8. В открытом окне запроса введите, выделите и выполните следующую инструкцию SQL для очистки базы данных после выполнения упражнения.

```
USE master;
DROP DATABASE TestDB;
```

## Упражнение 5. Сравнение результатов тестов

В этом упражнении вы сравните результаты, полученные в предыдущих упражнениях.

Введите в следующую таблицу стоимость и количество считываний страниц для каждого способа индексирования.

| Способ индексирования                  | Стоимость | Страницы |
|----------------------------------------|-----------|----------|
| Точка отсчета                          |           |          |
| Кластеризованные индексы               |           |          |
| Покрывающие некластеризованные индексы |           |          |
| Индексированное представление          |           |          |

Какой из вариантов обеспечивает наименьшую (лучшую) стоимость?

Какой из вариантов индексирования обеспечивает наименьшее (лучшее) количество считанных страниц?

Ваши результаты должны показать, что индексированное представление — вариант с наивысшей производительностью, за ним следует покрывающий некластеризованный индекс и в конце кластеризованный индекс.

## Резюме урока

- ◆ Индексы, как правило, повышают производительность операций чтения, но могут снизить производительность операций записи.
- ◆ Индексированные представления могут повысить производительность даже существенней, чем индексы, но у них много ограничений и обычно их нельзя создать для целого запроса.
- ◆ Решение о том, какие столбцы поместить в ключ индекса, а какие реализовать как включенные столбцы, очень важно.
- ◆ Анализируйте, какие индексы действительно используются, и удаляйте те, которые не применяются. Это сэкономит дисковое пространство и минимизирует ресурсы, затрачиваемые на поддержку индексов в операциях записи.

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить знания и навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ Всегда оценивайте разные варианты реализации дорогостоящих запросов.
- ◆ Поскольку индексы занимают дисковое пространство (если не отключены) и требуют сопровождения во время операций записи, старайтесь удалять неиспользуемые индексы.
- ◆ При определении производительности запроса всегда применяйте в качестве меры время выполнения запроса, не полагайтесь только на стоимость и количество считываний страниц.
- ◆ Оценивайте стоимость создания индексных представлений для полного или частичного покрытия запросов.

## Ключевые термины

Знаете ли вы, что означают следующие ключевые термины? Вы можете проверить свои ответы, заглянув в словарь терминов, приведенный в конце книги.

- ◆ Стоимость (Cost).
- ◆ Страница (Page).
- ◆ Индекс (Index).
- ◆ Куча (Heap).
- ◆ Кластеризованный индекс (Clustered index).
- ◆ Некластеризованный индекс (Nonclustered index).
- ◆ Функция секционирования (Partition function).
- ◆ Схема секционирования (Partition scheme).

## Сценарий

В следующем конкретном сценарии вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в *разд. "Ответы к сценариям"* в конце книги.

### Сценарий. Настройка производительности запроса

В этом сценарии вы — разработчик базы данных в вашей компании. Вам поручена задача оптимизации базы данных, используемой Web-сайтом компании. Поскольку активность пользователей на Web-сайте компании велика, существуют две проблемы, касающиеся производительности. Во-первых, время отклика пользователя на интерактивных чат-форумах крайне неудовлетворительное. Разработчики свели эту проблему к увеличению времени отклика из-за хранимых процедур экземпляра SQL Server, обслуживающего форумы. Вам необходимо оптимизировать эти хранимые процедуры. Во-вторых, сотрудники финансового отдела применяют для получения статистических показателей загруженности Web-сайта запросы к базе данных из приложения Microsoft Office Excel 2007. Выполнение этих запросов может занимать, в зависимости от извлекаемых статистических характеристик, до 10 минут. Финансовый директор заявил, что время выполнения подобных запросов должно быть снижено до пары секунд.

Ответьте на следующий вопрос вашего менеджера: "Какие шаги вам следует предпринять для решения этих проблем?"

## КОНТРОЛЬНОЕ УПРАЖНЕНИЕ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

## Создание и изменение индексов

**Упражнение 1.** Создайте простую таблицу без индексов, вставьте в нее 1 000 000 строк и запишите стоимость запроса и время выполнения вставки. Добавьте в таблицу один некластеризованный индекс, очистите таблицу, повторно вставьте в нее 1 000 000 строк и снова запишите стоимость запроса и время его выполнения. Добавьте еще несколько некластеризованных индексов и проверяйте после вставки каждого индекса разницу в производительности заполнения таблицы.

**Упражнение 2.** Создайте простую таблицу (с длиной строки как минимум 50 байт) без индексов и вставьте в нее 1 000 000 строк. Выполните запросы, извлекающие 10, 100, 1000, 10 000 и 100 000 строк (например, применяя фильтр BETWEEN к идентифицирующему столбцу таблицы) из таблицы без индексов, и запишите стоимость запроса и количество считываний страниц для каждого запроса. Создайте кластеризованный индекс для столбца, по которому ведется отбор, снова выполните запросы и запишите показатели их производительности еще раз. Прodelайте то же самое для покрывающего и непокрывающего некластеризованного индекса.

## Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подоби́я будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### **Дополнительная информация о тренировочных тестах**

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.

## ГЛАВА 7

# Расширение функциональных возможностей Microsoft SQL Server за счет XML, SQLCLR и *Filestream*

Microsoft SQL Server 2008 устанавливается с множеством функциональных возможностей, существенно расширяющих представление о типичной системе управления реляционной базой данных (СУРБД). В эту версию программного обеспечения было добавлено управление порциями двоичных данных большого объема без сохранения их в файлах базы данных, но при этом обеспечивающее транзакционную согласованность и согласованные операции резервного копирования и восстановления. Корпорация Microsoft также усовершенствовала поддержку в SQL Server данных, описываемых на расширяемом языке разметки (Extensible Markup Language, XML), и программного кода из общезыковой среды исполнения (Common Language Runtime, CLR) внутри базы данных. В этой главе вы узнаете, что необходимо для того, чтобы начать применять эти новые усовершенствованные средства.

### Экзаменационные темы в этой главе

- ◆ Создание и распространение объектов CLR.
- ◆ Извлечение реляционных данных на языке XML.
- ◆ Преобразование данных на XML в реляционные данные.
- ◆ Запросы данных XML.
- ◆ Управление данными XML.

### Уроки в этой главе

- ◆ Урок 1. Работа с XML.
- ◆ Урок 2. Применение SQLCLR и *Filestream*.



## Прежде чем начать

Для усвоения уроков этой главы необходимо иметь:

- ◆ базовое представление о языке Transact-SQL (T-SQL);
- ◆ базовое представление о языке XML;
- ◆ установленные версию Microsoft SQL Server 2008 Developer Edition, Enterprise Edition или Enterprise Evaluation Edition и учебную базу данных AdventureWorks;
- ◆ опыт разработки приложений на языках C# или Microsoft Visual Basic .NET также будет хорошим подспорьем.

### **РЕАЛЬНАЯ ИСТОРИЯ.**

#### ***Тобиас Тернстрем (Tobias Thernström)***

Однажды мне попался клиент, у которого были большие проблемы с хранимой процедурой, очень жестко связанной с бизнес-логикой. Этой процедуре требовалось передавать более 100 параметров, и ее вызов превращался в кошмар. Часть параметров представляла собой списки значений, которые, прежде чем использовать в хранимой процедуре, нужно было с помощью функции, определенной пользователем (ФОП), разделить на части и ввести в табличную переменную. Я предложил заменить все параметры одним с данными типа XML и прилагаемой к нему схемой XML. Сначала они запротестовали, но после внесения изменения в процедуру обнаружили, что ее стало гораздо легче вызывать из вызывающего приложения, т. к. она использовала только один параметр, и требовалось меньше "головомомных" изменений, поскольку все передавалось в параметре типа XML и в самой процедуре использовалось меньше программного кода. Кроме того, она выполнялась гораздо быстрее, т. к. подпрограммы, которые применялись в табличных переменных для разбиения на строки, были менее эффективны, чем метод `nodes` в типе данных XML.

## УРОК 1. Работа с XML

XML — это легко читаемый язык иерархической текстовой разметки, который обычно применяется для обмена данными в системах и между системами. Поскольку XML — это сама по себе обширная тема, его невозможно полностью описать в этой книге. Базовое представление о языке XML — необходимое предварительное условие для освоения данного урока.

### ***База данных***

Если не оговаривается особо, во всех примерах, приведенных в этой главе, применяется учебная база AdventureWorks для SQL Server 2008.

Далее в качестве образца приведен простой пример документа XML, содержащий двух клиентов (обозначенных тегом `<Customer>`) с несколькими заказами (обозначенными тегом `<Order>`) у каждого из них.

```

<?xml version="1.0"?>
<Customers>
 <Customer Id="1" AccountNumber="AW00000001" Type="S">
 <Orders>
 <Order Id="43860" OrderDate="2001-08-01T00:00:00"
 ShipDate="2001-08-08T00:00:00" />
 <Order Id="44501" OrderDate="2001-11-01T00:00:00"
 ShipDate="2001-11-08T00:00:00" />
 <Order Id="45283" OrderDate="2002-02-01T00:00:00"
 ShipDate="2002-02-08T00:00:00" />
 <Order Id="46042" OrderDate="2002-05-01T00:00:00"
 ShipDate="2002-05-08T00:00:00" />
 </Orders>
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002" Type="S">
 <Orders>
 <Order Id="46976" OrderDate="2002-08-01T00:00:00"
 ShipDate="2002-08-08T00:00:00" />
 <Order Id="47997" OrderDate="2002-11-01T00:00:00"
 ShipDate="2002-11-08T00:00:00" />
 <Order Id="49054" OrderDate="2003-02-01T00:00:00"
 ShipDate="2003-02-08T00:00:00" />
 <Order Id="50216" OrderDate="2003-05-01T00:00:00"
 ShipDate="2003-05-08T00:00:00" />
 <Order Id="51728" OrderDate="2003-08-01T00:00:00"
 ShipDate="2003-08-08T00:00:00" />
 <Order Id="57044" OrderDate="2003-11-01T00:00:00"
 ShipDate="2003-11-08T00:00:00" />
 <Order Id="63198" OrderDate="2004-02-01T00:00:00"
 ShipDate="2004-02-08T00:00:00" />
 <Order Id="69488" OrderDate="2004-05-01T00:00:00"
 ShipDate="2004-05-08T00:00:00" />
 </Orders>
 </Customer>
</Customers>

```

Как уже упоминалось, приведенный пример программного кода — это *документ XML*. Будут также обсуждаться *фрагменты XML*. Фрагменты XML подобны документам XML, но сами не являются документами XML. Звучит странно? Идея заключается в том, что фрагмент XML считается частью документа XML, т. е. он взят из контекста документа XML. Это значит, что у фрагмента XML отсутствует объявление XML (`<?xml...>`) и не должно быть корневого элемента (такого как тег `<Customers>` в предыдущем примере). Далее приведен пример фрагмента XML.

```

<Order Id="43860" OrderDate="2001-08-01T00:00:00"
 ShipDate="2001-08-08T00:00:00" />
<Order Id="44501" OrderDate="2001-11-01T00:00:00"
 ShipDate="2001-11-08T00:00:00" />

```

```
<Order Id="45283" OrderDate="2002-02-01T00:00:00"
 ShipDate="2002-02-08T00:00:00" />
```

Применение XML в системах управления реляционными базами данных — предмет частых дискуссий. Как и когда использовать большинство средств, зависит от задачи, которую вы пытаетесь решить. Язык XML можно применять в нескольких операциях, связанных с базой данных. Далее перечислены основные сферы его использования.

- ◆ *Извлечение реляционных данных в виде документа XML.* Вместо получения из базы данных табличного набора данных извлекается документ XML.
- ◆ *Передача данных в виде XML в базу данных.* Вместо передачи в базу данных скалярных значений с помощью выполнения множественных инструкций языка манипулирования данными (DML) или многократного выполнения хранимой процедуры можно передать прямо в базу данных документ или фрагмент XML.
- ◆ *Хранение или запрос действительного документа или фрагмента XML в базе данных.* Это одна из наиболее спорных тем. Почему вы должны хранить XML непосредственно в таблице? Для этого есть несколько причин. Которые будут обсуждаться позже в этом уроке.

---

#### ***После усвоения этого урока вы сможете***

---

- Применять предложение `FOR XML` для извлечения из SQL Server реляционных данных в виде XML.
- Использовать тип данных `XML` и методы XML для работы с данными XML в SQL Server.

Расчетное время урока — 60 мин.

## **Извлечение табличных данных в виде XML**

Для начала, зачем вам может понадобиться извлекать из базы данных документ или фрагмент XML вместо табличного результирующего набора? Одной из причин может оказаться просто желание человека, для которого вы извлекаете данные, получить их в виде XML. Другой причиной может быть то, что извлекаемые данные лучше поддаются описанию на языке XML, чем представлению в виде таблицы. Это справедливо для иерархических данных, как в первом примере данной главы с клиентами и их заказами. Если подумать, приложениям свойственно извлекать иерархию данных, подобную упомянутой. При невозможности использовать XML для форматирования данных SQL Server предлагает два других варианта.

Первый — выполнить две отдельные инструкции `SELECT` и объединить в клиентском приложении клиентов и соответствующие заказы. Учтите, что для объединения результатов в клиентском приложении необходимо включить в оба запроса поле `CustomerID`.

```
SELECT c.CustomerID, c.AccountNumber, c.CustomerType
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1, 2);
```

```
SELECT soh.CustomerID, soh.SalesOrderID, soh.OrderDate, soh.ShipDate
FROM Sales.SalesOrderHeader AS soh
WHERE soh.CustomerID IN (1, 2);
```

Далее приведены результаты.

| CustomerID | AccountNumber | CustomerType |
|------------|---------------|--------------|
| 1          | AW00000001    | S            |
| 2          | AW00000002    | S            |

| CustomerID | SalesOrderID | OrderDate               | ShipDate                |
|------------|--------------|-------------------------|-------------------------|
| 1          | 43860        | 2001-08-01 00:00:00.000 | 2001-08-08 00:00:00.000 |
| 1          | 44501        | 2001-11-01 00:00:00.000 | 2001-11-08 00:00:00.000 |
| 1          | 45283        | 2002-02-01 00:00:00.000 | 2002-02-08 00:00:00.000 |
| 1          | 46042        | 2002-05-01 00:00:00.000 | 2002-05-08 00:00:00.000 |
| 2          | 46976        | 2002-08-01 00:00:00.000 | 2002-08-08 00:00:00.000 |
| 2          | 47997        | 2002-11-01 00:00:00.000 | 2002-11-08 00:00:00.000 |
| 2          | 49054        | 2003-02-01 00:00:00.000 | 2003-02-08 00:00:00.000 |
| 2          | 50216        | 2003-05-01 00:00:00.000 | 2003-05-08 00:00:00.000 |
| 2          | 51728        | 2003-08-01 00:00:00.000 | 2003-08-08 00:00:00.000 |
| 2          | 57044        | 2003-11-01 00:00:00.000 | 2003-11-08 00:00:00.000 |
| 2          | 63198        | 2004-02-01 00:00:00.000 | 2004-02-08 00:00:00.000 |
| 2          | 69488        | 2004-05-01 00:00:00.000 | 2004-05-08 00:00:00.000 |

Второй вариант — выполнить одну инструкцию `SELECT` и извлечь нужные столбцы из каждой таблицы с помощью соединения. Этот вариант также полагается на клиентское приложение при определении, какие данные принадлежат клиенту, а какие — заказу. Конечно, все это можно сделать проще, применив подходящие псевдонимы столбцов, но вы, вероятно, согласитесь, что обработка таких результатов создаст некоторую путаницу в клиентском приложении. Два несущественных замечания относительно данного запроса. Во-первых, для объединения двух таблиц вы должны применить оператор `LEFT OUTER JOIN`, чтобы включить в результирующий набор клиентов без заказов. Во-вторых, для упрощения управления результатами в клиентском приложении полезно (но не обязательно) использовать предложение `ORDER BY`, т. к. оно гарантирует последовательный возврат всех заказов, относящихся к конкретному клиенту. Запрос с соединением выглядит следующим образом.

```
SELECT c.CustomerID AS Customer_CustomerID
 , c.AccountNumber AS Customer_AccountNumber
 , c.CustomerType AS Customer_CustomerType
 , soh.SalesOrderID AS Order_SalesOrderID
 , soh.OrderDate AS Order_OrderDate
 , soh.ShipDate AS Order_ShipDate
```

```

FROM Sales.Customer AS c
LEFT OUTER JOIN Sales.SalesOrderHeader AS soh
 ON soh.CustomerID = c.CustomerID
WHERE c.CustomerID IN (1, 2)
ORDER BY Customer_CustomerID;

```

Далее приведен результат (некоторые столбцы пропущены для экономии места).

| Customer_CustomerID | Customer_AccountNumber | Order_SalesOrderID |
|---------------------|------------------------|--------------------|
| 1                   | AW00000001             | 43860              |
| 1                   | AW00000001             | 44501              |
| 1                   | AW00000001             | 45283              |
| 1                   | AW00000001             | 46042              |
| 2                   | AW00000002             | 46976              |
| 2                   | AW00000002             | 47997              |
| 2                   | AW00000002             | 49054              |
| 2                   | AW00000002             | 50216              |
| 2                   | AW00000002             | 51728              |
| 2                   | AW00000002             | 57044              |
| 2                   | AW00000002             | 63198              |
| 2                   | AW00000002             | 69488              |

Оба предложенных табличных решения создают большой объем работы в клиентском приложении.

Почему же решение с применением XML предпочтительнее? Поскольку результат, который вы хотите получить из базы данных, — это иерархия (клиенты, имеющие заказы), следовательно, извлечение результата в виде XML, который иерархичен по определению, может упростить задачу для приложения.

Для примера рассмотрим показанную далее инструкцию `SELECT`, которая возвращает результат в виде XML. Чуть позже в этом уроке вы подробно ознакомитесь с этим запросом.

```

SELECT c.CustomerID AS "@Id"
 , c.AccountNumber AS "@AccountNumber"
 , c.CustomerType AS "@Type"
 , (SELECT soh.SalesOrderID AS "@Id"
 , soh.OrderDate AS "@OrderDate"
 , soh.ShipDate AS "@ShipDate"
 FROM Sales.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 FOR XML PATH('Order'), TYPE
) AS "Orders"
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1, 2)
FOR XML PATH('Customer'), ROOT('Customers');

```

Далее показан результат.

```
<Customers>
 <Customer Id="1" AccountNumber="AW00000001" Type="S">
 <Orders>
 <Order Id="43860" OrderDate="2001-08-01T00:00:00"
 ShipDate="2001-08-08T00:00:00" />
 <Order Id="44501" OrderDate="2001-11-01T00:00:00"
 ShipDate="2001-11-08T00:00:00" />
 <Order Id="45283" OrderDate="2002-02-01T00:00:00"
 ShipDate="2002-02-08T00:00:00" />
 <Order Id="46042" OrderDate="2002-05-01T00:00:00"
 ShipDate="2002-05-08T00:00:00" />
 </Orders>
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002" Type="S">
 <Orders>
 <Order Id="46976" OrderDate="2002-08-01T00:00:00"
 ShipDate="2002-08-08T00:00:00" />
 <Order Id="47997" OrderDate="2002-11-01T00:00:00"
 ShipDate="2002-11-08T00:00:00" />
 <Order Id="49054" OrderDate="2003-02-01T00:00:00"
 ShipDate="2003-02-08T00:00:00" />
 <Order Id="50216" OrderDate="2003-05-01T00:00:00"
 ShipDate="2003-05-08T00:00:00" />
 <Order Id="51728" OrderDate="2003-08-01T00:00:00"
 ShipDate="2003-08-08T00:00:00" />
 <Order Id="57044" OrderDate="2003-11-01T00:00:00" ShipDate="2003-11-
 08T00:00:00" />
 <Order Id="63198" OrderDate="2004-02-01T00:00:00" ShipDate="2004-02-
 08T00:00:00" />
 <Order Id="69488" OrderDate="2004-05-01T00:00:00"
 ShipDate="2004-05-08T00:00:00" />
 </Orders>
 </Customer>
</Customers>
```

Теперь давайте бегло рассмотрим, как выполнить этот запрос из приложения среды .NET, извлечь данные XML в память и легко обработать в цикле клиентов и соответствующие им заказы. Для простоты в примере используется консольное приложение.

' VB:

```
Imports System
Imports System.Xml
Imports System.Data.SqlClient
```

```
Module TK433
```

```
Sub Main()
```

```
 Using conn As SqlConnection = New SqlConnection(_
 "server=.;database=AdventureWorks;trusted_connection=yes")
```

```

Dim cmd As SqlCommand = conn.CreateCommand()
cmd.CommandText = _
 "SELECT " & vbCrLf & _
 " c.CustomerID AS ""@Id"" & vbCrLf & _
 " ,c.AccountNumber AS ""@AccountNumber"" & vbCrLf & _
 " ,c.CustomerType AS ""@Type"" & vbCrLf & _
 " ,(" & vbCrLf & _
 " SELECT" & vbCrLf & _
 " soh.SalesOrderID AS ""@Id"" & vbCrLf & _
 " ,soh.OrderDate AS ""@OrderDate"" & vbCrLf & _
 " ,soh.ShipDate AS ""@ShipDate"" & vbCrLf & _
 " FROM Sales.SalesOrderHeader AS soh" & vbCrLf & _
 " WHERE (soh.CustomerID = c.CustomerID)" & vbCrLf & _
 " FOR XML PATH('Order'), TYPE" & vbCrLf & _
 ") AS Orders" & vbCrLf & _
 " FROM Sales.Customer AS c" & vbCrLf & _
 "WHERE c.CustomerID IN (1, 2)" & vbCrLf & _
 "FOR XML PATH('Customer'), ROOT('Customers')"

conn.Open()

' Выполнение запроса с использованием XML Reader.
Dim reader As XmlReader = cmd.ExecuteXmlReader()
' Применение XML Reader для заполнения документа XML.
Dim doc As XmlDocument = New XmlDocument()
doc.Load(reader)

' Обработка в цикле клиентов.
For Each customer As XmlElement In
 doc.SelectNodes("/Customers/Customer")
 Console.WriteLine("Customer: {0}",
 customer.Attributes("Id").Value)
 For Each order As XmlElement In
 customer.SelectNodes("Orders/Order")
 Console.WriteLine(vbTab & "Order: {0}",
 order.Attributes("Id").Value)
 Next
Next
End Using
Console.WriteLine("Press [ENTER] to exit...")
Console.ReadLine()

End Sub
End Module

// C#:
using System;
using System.Xml;
using System.Data.SqlClient;

```

```
class TK433Demo
{
 static void Main()
 {
 using (SqlConnection conn = new SqlConnection(
 "server=.;database=AdventureWorks;trusted_connection=yes;"))
 {
 SqlCommand cmd = conn.CreateCommand();
 cmd.CommandText = @"
SELECT
 c.CustomerID AS ""@Id""
 ,c.AccountNumber AS ""@AccountNumber""
 ,c.CustomerType AS ""@Type""
 ,(
 SELECT
 soh.SalesOrderID AS ""@Id""
 ,soh.OrderDate AS ""@OrderDate""
 ,soh.ShipDate AS ""@ShipDate""
 FROM Sales.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 FOR XML PATH('Order'), TYPE
) AS Orders
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1, 2)
FOR XML PATH('Customer'), ROOT('Customers');";
 conn.Open();

 // Выполнение запроса с использованием XML Reader
 // для получения результатов.
 XmlReader reader = cmd.ExecuteXmlReader();

 // Применение XML Reader для заполнения документа XML.
 XmlDocument doc = new XmlDocument();
 doc.Load(reader);

 // Обработка в цикле клиентов.
 foreach (XmlElement customer in
 doc.SelectNodes("/Customers/Customer"))
 {
 Console.WriteLine("Customer: {0}",
 customer.Attributes["Id"].Value);
 foreach (XmlElement order in
 customer.SelectNodes("Orders/Order"))
 {
 Console.WriteLine("\tOrder: {0}",
 order.Attributes["Id"].Value);
 }
 }
 }
 }
}
```



```

 Console.WriteLine("Press [ENTER] to exit...");
 Console.ReadLine();
 }
}

```

Простоту можно увидеть в особенности в следующем программном коде, который обрабатывает в цикле клиентов и заказы.

```

' VB:
For Each customer As XElement In doc.SelectNodes("/Customers/Customer") _
 Console.WriteLine("Customer: {0}", customer.Attributes("Id").Value)
 For Each order As XElement In customer.SelectNodes("Orders/Order")
 Console.WriteLine(vbTab & "Order: {0}", _
 order.Attributes("Id").Value)
 Next
Next

// C#:
foreach (XXmlElement customer in doc.SelectNodes("/Customers/Customer"))
{
 Console.WriteLine("Customer: {0}", customer.Attributes["Id"].Value);
 foreach (XXmlElement order in customer.SelectNodes("Orders/Order"))
 {
 Console.WriteLine("\tOrder: {0}", order.Attributes["Id"].Value);
 }
}

```

Как видите, поскольку язык XML по природе иерархичен, вам не нужно отслеживать, какой клиент или заказ обрабатывается в цикле в настоящий момент. Вместо этого управление обеспечивается иерархией документа XML.

### **FOR XML <режим>**

В предыдущем примере для формирования результата на языке XML в конец инструкции `SELECT` вставлено дополнительное предложение `FOR XML`. В SQL Server 2008 существуют четыре варианта (или режима), применяемых в предложении `FOR XML`: `RAW`, `AUTO`, `EXPLICIT` и `PATH`. В предыдущем примере использовался режим `PATH`. Он и рекомендуемый, и самый мощный. Для полноты и потому, что вы можете работать с ними, в данной главе описаны и остальные три варианта. Начнем с режима `RAW`, поскольку он самый простой из всех.

**FOR XML RAW.** Инструкция `SELECT...FOR XML RAW` — простейшая реализация предложения `FOR XML`, но, тем не менее, она может быть полезна. Рассмотрим следующий запрос.

```

SELECT c.CustomerID, c.AccountNumber
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1, 2)
FOR XML RAW;

```

Как видно из приведенных далее результатов, в своем стандартном варианте применения (простое задание FOR XML RAW) запрос с предложением FOR XML RAW возвращает каждую строку как элемент XML, а каждый столбец как атрибут XML (<row Column1="..." Column2="..." />).

```
<row CustomerID="1" AccountNumber="AW00000001" />
<row CustomerID="2" AccountNumber="AW00000002" />
```

В версию SQL Server 2005 были внесены некоторые усовершенствования, позволяющие добавлять именованный корневой элемент, а также именовать элементы, создаваемые для каждой строки. Пример показан в следующем блоке программного кода, в котором задается директива ROOT, а также добавляется параметр режима RAW, задающий имя элемента (изменения выделены жирным шрифтом). Обратите внимание на то, что изменяя имя столбца с помощью псевдонима, вы также изменяете имя атрибута (это поддерживалось и в версии SQL Server 2000).

```
SELECT c.CustomerID AS Id, c.AccountNumber
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1,2)
FOR XML RAW ('Customer'), ROOT ('Customers');
```

Далее приведен результат.

```
<Customers>
 <Customer Id="1" AccountNumber="AW00000001" />
 <Customer Id="2" AccountNumber="AW00000002" />
</Customers>
```

Возможно, вас интересует, как обрабатываются в результатах XML значения NULL? По умолчанию атрибут со значением NULL просто удаляется. Для большинства приложений это подходит, но некоторые из них хотят отличать пропущенное значение от значения NULL. Как им сделать это? Рассмотрим следующий пример.

Элемент XML *Customer* описывает клиента из базы данных, сведения о котором необходимо обновить. У клиента 10 атрибутов, которые потенциально могут присутствовать в элементе XML, но в конкретном элементе XML есть только 2 атрибута. При передаче этого элемента XML в базу данных для выполнения обновления сведений о клиенте база данных может совершить одно из двух действий:

- ◆ обновить все 10 атрибутов, задав 8 из них как значения NULL, поскольку они отсутствуют, и присвоив двум атрибутам их новые значения;
- ◆ обновить только 2 атрибута, указанных в элементе XML, который передается в базу данных, и пропустить остальные 8, оставив их текущие значения без изменения.

Если вы выбираете второй вариант, то простое не включение в элемент XML атрибутов, действительно равных значению `NULL`, не принесет желаемого результата. Вы должны каким-то образом явно указать, что они равны `NULL` (или должны стать равны `NULL`). Сделать это можно с помощью элемента в языке XML, именуемого `NIL`. `NIL` в XML эквивалентен тому, что в базах данных называют `NULL`. Если значение должно быть определено как `NIL`, оно не может храниться в атрибуте. Оно должно сохраняться как отдельный элемент, потому что нуждается в атрибуте, определяющем это значение как `NIL` в случае необходимости. Посмотрите на следующий пример, в котором первый элемент представляет товар, имеющий цвет, заданный как `NULL`. Предыдущий запрос с предложением `FOR XML RAW` использовался как образец. Обратите внимание на то, что у первого товара атрибут `Color` (цвет) отсутствует.

```
SELECT p.ProductID AS Id, p.ListPrice, p.Color
FROM Production.Product AS p
WHERE p.ProductID IN (514, 707)
FOR XML RAW('Product'), ROOT('Products');
```

Запрос вернет следующий результат.

```
<Products>
 <Product Id="514" ListPrice="133.3400" />
 <Product Id="707" ListPrice="34.9900" Color="Red" />
</Products>
```

Теперь рассмотрим еще один пример, в который добавим директиву `ELEMENTS`, чтобы сделать каждый столбец элементом XML, а не атрибутом. При этом у первого товара элемент `Color` все еще отсутствует.

```
SELECT p.ProductID AS Id, p.ListPrice, p.Color
FROM Production.Product AS p
WHERE p.ProductID IN (514, 707)
FOR XML RAW('Product'), ROOT('Products'), ELEMENTS;
```

Далее показан результат.

```
<Products>
 <Product>
 <Id>514</Id>
 <ListPrice>133.3400</ListPrice>
 </Product>
 <Product>
 <Id>707</Id>
 <ListPrice>34.9900</ListPrice>
 <Color>Red</Color>
 </Product>
</Products>
```

В заключение добавим в директиву `ELEMENTS` директиву `XSINIL`, чтобы заставить SQL Server при обработке значений `NULL` сохранять элемент XML для отсутст-

вующего значения и устанавливать значение его атрибута `NIL`, равным `True`. В этом случае вы сможете увидеть, что элемент `Color` действительно равен `NULL` (или `NIL`), а не просто "пропущен".

```
SELECT p.ProductID AS Id, p.ListPrice, p.Color
FROM Production.Product AS p
WHERE p.ProductID IN (514, 707)
FOR XML RAW('Product'), ROOT('Products'), ELEMENTS XSINIL;
```

Далее приведен результат, в котором цвет со значением `NULL` выделен жирным шрифтом.

```
<Products xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
 <Product>
 <Id>514</Id>
 <ListPrice>133.3400</ListPrice>
 <Color xsi:nil="true" />
 </Product>
 <Product>
 <Id>707</Id>
 <ListPrice>34.9900</ListPrice>
 <Color>Red</Color>
 </Product>
</Products>
```

В последнем примере в корневой элемент добавлена ссылка на пространство имен XML, именуемое `xsi` и ссылающееся на Web-страницу <http://www.w3.org/2001/XMLSchema-instance>. Сделано это потому, что атрибут `NIL` определен в этом пространстве имен. Ссылка на пространство имен добавляется всегда, если применяется директива `XSINIL`. Прежде чем переходить к режиму `AUTO` предложения `FOR XML`, следует отметить, что директивы `ELEMENTS` и `XSINIL` есть и в режиме `AUTO`, но не в двух оставшихся режимах, `EXPLICIT` и `PATH`.

**FOR XML AUTO.** Режим `AUTO` отличается от режима `RAW` тем, что он естественным образом поддерживает иерархии. Однако иерархии должны быть простыми, т. к. режим `AUTO` не поддерживает разветвления (path of branches). Например, следующая иерархия поддерживается.

```
Customer
 Order
 Order row
```

А приведенная далее иерархия не поддерживается из-за наличия разветвления.

```
Customer
 Order
 Order row
 Contacts
```

В режиме `AUTO` каждая таблица, включенная в запрос, получает собственный элемент в иерархии, а имя элемента наследуется от псевдонима таблицы, ис-

пользуемого в запросе. Иерархия создается исходя из порядка столбцов, возвращенных запросом, а не из порядка столбцов в таблицах. Рассмотрим запрос в следующем примере, который возвращает клиентов с заказами, используя предложение `FOR XML AUTO`. Соединение выполняется от заказа к клиенту, но клиент остается в иерархии на более высоком уровне, чем заказ. Это происходит потому, что в списке столбцов инструкции `SELECT` столбцы, относящиеся к клиенту, перечисляются до столбцов заказа. Обратите внимание также на то, что элемент каждого клиента повторяется несколько раз в результате XML. Это объясняется тем, что иерархия также строится, исходя из порядка строк в результате. Следовательно, вы должны убедиться в том, что предложение `ORDER BY` применено корректно, т. е. группирует все заказы под соответствующими клиентами (в данном примере для получения сортировки в якобы случайном порядке строки сортируются по столбцу с уникальным идентификатором или GUID в таблице).

```
SELECT Customer.CustomerID AS Id, Customer.AccountNumber
 , "Order".SalesOrderID, "Order".rowguid AS RowGuid
FROM Sales.SalesOrderHeader AS "Order"
RIGHT OUTER JOIN Sales.Customer AS Customer
 ON Customer.CustomerID = "Order".CustomerID
WHERE Customer.CustomerID IN (1,2)
ORDER BY "Order".rowguid
FOR XML AUTO, ROOT('Customers');
```

Далее приведен результат на XML.

```
<Customers>
 <Customer Id="2" AccountNumber="AW00000002">
 <Order SalesOrderID="49054" RowGuid=
 "01C5EFDE-5C6E-47C9-B1AE-077937989297" />
 <Order SalesOrderID="51728" RowGuid=
 "C2B5D4CC-113E-4102-884B-22A6DACEEDE6" />
 </Customer>
 <Customer Id="1" AccountNumber="AW00000001">
 <Order SalesOrderID="46042" RowGuid=
 "62991BDA-C42D-494F-9EF1-2754BEC25FAE" />
 <Order SalesOrderID="43860" RowGuid=
 "D2745233-B05B-409C-93BB-4451569F4253" />
 <Order SalesOrderID="44501" RowGuid=
 "1A116F86-71E4-40A2-A32C-4938D8977D26" />
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002">
 <Order SalesOrderID="63198" RowGuid=
 "C6C5306D-F416-433C-92CC-4DB4747DC133" />
 <Order SalesOrderID="46976" RowGuid=
 "8A533BE6-0669-470A-B361-796DD1CD0ED4" />
 </Customer>
 <Customer Id="1" AccountNumber="AW00000001">
```

```

 <Order SalesOrderID="45283" RowGuid=
 "F57AB920-675E-4B1D-B43C-8EA091CF6F38" />
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002">
 <Order SalesOrderID="47997" RowGuid=
 "1FAAD98B-1DE0-4B80-A804-9FBBB6F289EB" />
 <Order SalesOrderID="69488" RowGuid=
 "50144563-A6B4-4857-9451-B229D21C7ED5" />
 <Order SalesOrderID="50216" RowGuid=
 "00A755D8-7BD9-4B12-AF58-E4EF22D39AA1" />
 <Order SalesOrderID="57044" RowGuid=
 "1841BF49-544C-4A8F-8ACE-F63DADA00314" />
 </Customer>
</Customers>

```

Далее показан табличный результат (созданный исключением из запроса части FOR XML).

Id	AccountNumber	SalesOrderID	RowGuid
2	AW00000002	49054	01C5EFDE-5C6E-47C9-B1AE-077937989297
2	AW00000002	51728	C2B5D4CC-113E-4102-884B-22A6DACEEDE6
1	AW00000001	46042	62991BDA-C42D-494F-9EF1-2754BEC25FAE
1	AW00000001	43860	D2745233-B05B-409C-93BB-4451569F4253
1	AW00000001	44501	1A116F86-71E4-40A2-A32C-4938D8977D26
2	AW00000002	63198	C6C5306D-F416-433C-92CC-4DB4747DC133
2	AW00000002	46976	8A533BE6-0669-470A-B361-796DD1CD0ED4
1	AW00000001	45283	F57AB920-675E-4B1D-B43C-8EA091CF6F38
2	AW00000002	47997	1FAAD98B-1DE0-4B80-A804-9FBBB6F289EB
2	AW00000002	69488	50144563-A6B4-4857-9451-B229D21C7ED5
2	AW00000002	50216	00A755D8-7BD9-4B12-AF58-E4EF22D39AA1
2	AW00000002	57044	1841BF49-544C-4A8F-8ACE-F63DADA00314

Для устранения проблемы, продемонстрированной в предыдущем примере, очевидно, что нужно просто правильно отсортировать результат. Звучит просто, но если не быть внимательным, в ваш код может закрасться коварная ошибка, которую легко пропустить. В следующем примере вы решите проблему сортировкой по полю CustomerID, чтобы сгруппировать вместе все заказы, принадлежащие конкретному клиенту. В примере есть маленькая хитрость — для создания элемента XML между элементами Customer и Order используется производная таблица, названная Orders.

```

SELECT Customer.CustomerID AS Id, Customer.AccountNumber, Orders.X
 , "Order".SalesOrderID, "Order".rowguid AS RowGuid
FROM Sales.SalesOrderHeader AS "Order"
RIGHT OUTER JOIN Sales.Customer AS Customer
 ON Customer.CustomerID = "Order".CustomerID
CROSS JOIN (SELECT NULL AS X) AS Orders
WHERE Customer.CustomerID IN (1, 2)

```

```
ORDER BY Customer.CustomerID
FOR XML AUTO, ROOT('Customers');
```

Далее приведен результат на языке XML.

```
<Customers>
 <Customer Id="1" AccountNumber="AW00000001">
 <Orders>
 <Order SalesOrderID="43860" RowGuid=
 "D2745233-B05B-409C-93BB-4451569F4253" />
 <Order SalesOrderID="44501" RowGuid=
 "1A116F86-71E4-40A2-A32C-4938D8977D26" />
 <Order SalesOrderID="45283" RowGuid=
 "F57AB920-675E-4B1D-B43C-8EA091CF6F38" />
 <Order SalesOrderID="46042" RowGuid=
 "62991BDA-C42D-494F-9EF1-2754BEC25FAE" />
 </Orders>
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002">
 <Orders>
 <Order SalesOrderID="46976" RowGuid=
 "8A533BE6-0669-470A-B361-796DD1CD0ED4" />
 <Order SalesOrderID="47997" RowGuid=
 "1FAAD98B-1DE0-4B80-A804-9FB8B6F289EB" />
 <Order SalesOrderID="49054" RowGuid=
 "01C5EFDE-5C6E-47C9-B1AE-077937989297" />
 <Order SalesOrderID="50216" RowGuid=
 "00A755D8-7BD9-4B12-AF58-E4EF22D39AA1" />
 <Order SalesOrderID="51728" RowGuid=
 "C2B5D4CC-113E-4102-884B-22A6DACEEDE6" />
 <Order SalesOrderID="57044" RowGuid=
 "1841BF49-544C-4A8F-8ACE-F63DADA00314" />
 <Order SalesOrderID="63198" RowGuid=
 "C6C5306D-F416-433C-92CC-4DB4747DC133" />
 <Order SalesOrderID="69488" RowGuid=
 "50144563-A6B4-4857-9451-B229D21C7ED5" />
 </Orders>
 </Customer>
</Customers>
```

Далее показан табличный результат (полученный исключением из запроса части FOR XML).

Id	AccountNumber X	SalesOrderID	RowGuid
1	AW00000001	NULL 43860	D2745233-B05B-409C-93BB-4451569F4253
1	AW00000001	NULL 44501	1A116F86-71E4-40A2-A32C-4938D8977D26
1	AW00000001	NULL 45283	F57AB920-675E-4B1D-B43C-8EA091CF6F38
1	AW00000001	NULL 46042	62991BDA-C42D-494F-9EF1-2754BEC25FAE

2	AW00000002	NULL	46976	8A533BE6-0669-470A-B361-796DD1CD0ED4
2	AW00000002	NULL	47997	1FAAD98B-1DE0-4B80-A804-9FBBB6F289EB
2	AW00000002	NULL	49054	01C5EFDE-5C6E-47C9-B1AE-077937989297
2	AW00000002	NULL	50216	00A755D8-7BD9-4B12-AF58-E4EF22D39AA1
2	AW00000002	NULL	51728	C2B5D4CC-113E-4102-884B-22A6DACEEDE6
2	AW00000002	NULL	57044	1841BF49-544C-4A8F-8ACE-F63DADA00314
2	AW00000002	NULL	63198	C6C5306D-F416-433C-92CC-4DB4747DC133
2	AW00000002	NULL	69488	50144563-A6B4-4857-9451-B229D21C7ED5

**FOR XML EXPLICIT.** Третий режим предложения `FOR XML` — `EXPLICIT` — трудно описывать и еще труднее сопровождать. Интересная особенность этого режима заключается в том, что с его помощью вы можете создать практически любую структуру XML, даже некоторые из тех, что не поддерживаются режимом `PATH`, о котором вы узнаете чуть позже в этом уроке. По существу для создания документа XML с помощью предложения `FOR XML EXPLICIT` нужно вернуть особый результирующий набор, т. е. следует особым образом назвать ваши столбцы. Кроме того, для получения желаемого результата вы должны отсортировать результаты должным образом, во многом так же, как в режиме `AUTO`. Результирующий набор в режиме `EXPLICIT` должен содержать два столбца с именами: `Tag` и `Parent`. В столбец `Tag` вставляется целочисленный идентификатор для каждого элемента XML, который вы хотите вернуть, а в столбце `Parent` задается идентификатор из столбца `Tag` элемента XML, который является родительским для данного элемента XML. Если у элемента нет предка, в столбце `Parent` задается `NULL`. Остальные столбцы в результирующем наборе применяются для определения, как имен, так и значений элементов и атрибутов, которые следует вернуть. В следующем примере показано, как можно создать запрос с клиентами и их заказами, используя режим `EXPLICIT`.

```

SELECT 1 AS Tag
 , NULL AS Parent
 , NULL AS "Customers!1!!element"
 , NULL AS "Customer!2!Id"
 , NULL AS "Customer!2!AccountNumber"
 , NULL AS "Order!3!Id"
 , NULL AS "Order!3!OrderDate"

UNION ALL

SELECT 2 AS Tag
 , 1 AS Parent
 , NULL AS "Customers!1!!element"
 , c.CustomerID AS "Customer!2!Id"
 , c.AccountNumber AS "Customer!2!AccountNumber"
 , NULL AS "Order!3!Id"
 , NULL AS "Order!3!OrderDate"

FROM Sales.Customer AS c
WHERE c.CustomerID IN (1,2)
UNION ALL

```



```

SELECT 3 AS Tag
 , 2 AS Parent
 , NULL AS "Customers!1!!element"
 , soh.CustomerID AS "Customer!2!Id"
 , NULL AS "Customer!2!AccountNumber"
 , soh.SalesOrderID AS "Order!3!Id"
 , soh.OrderDate AS "Order!3!OrderDate"
FROM Sales.SalesOrderHeader AS soh
WHERE soh.CustomerID IN (1,2)
ORDER BY "Customer!2!Id", Tag
FOR XML EXPLICIT;

```

Далее приведен результат.

```

<Customers>
 <Customer Id="1" AccountNumber="AW00000001">
 <Order Id="43860" OrderDate="2001-08-01T00:00:00" />
 <Order Id="44501" OrderDate="2001-11-01T00:00:00" />
 <Order Id="45283" OrderDate="2002-02-01T00:00:00" />
 <Order Id="46042" OrderDate="2002-05-01T00:00:00" />
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002">
 <Order Id="46976" OrderDate="2002-08-01T00:00:00" />
 <Order Id="47997" OrderDate="2002-11-01T00:00:00" />
 <Order Id="49054" OrderDate="2003-02-01T00:00:00" />
 <Order Id="50216" OrderDate="2003-05-01T00:00:00" />
 <Order Id="51728" OrderDate="2003-08-01T00:00:00" />
 <Order Id="57044" OrderDate="2003-11-01T00:00:00" />
 <Order Id="63198" OrderDate="2004-02-01T00:00:00" />
 <Order Id="69488" OrderDate="2004-05-01T00:00:00" />
 </Customer>
</Customers>

```

**FOR XML PATH.** Из всех режимов предложения `FOR XML` вариант `FOR XML PATH` — наилучший выбор для большинства решений. Режим `PATH` позволяет для облегчения создания различных структур XML просто при генерации результата в виде XML интерпретировать имена заданных столбцов с помощью выражения `XPath`. Рассмотрим следующий запрос.

```

SELECT c.CustomerID AS "@Id"
 , c.AccountNumber AS "@AccountNumber"
 , c.RowGuid AS "comment()"
 , CAST('<Test/>' AS XML) AS "node()"
 , c.CustomerType AS "AdditionalInfo/@Type"
 , c.ModifiedDate AS "AdditionalInfo/text()"
 , c.rowguid AS "node()"
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1, 2)
FOR XML PATH('Customer'), ROOT('Customers');

```

Далее показан результат.

```
<Customers>
 <Customer Id="1" AccountNumber="AW00000001">
 <!--3F5AE95E-B87D-4AED-95B4-C3797AFCB74F-->
 <Test />
 <AdditionalInfo Type="S">2004-10-13T11:15:07.263</AdditionalInfo>
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002">
 <!--E552F657-A9AF-4A7D-A645-C429D6E02491-->
 <Test />
 <AdditionalInfo Type="S">2004-10-13T11:15:07.263</AdditionalInfo>
 </Customer>
</Customers>
```

В результате XML вы видите следующее:

- ◆ столбец @Id в результате представлен атрибутом Id в элементе Customer;
- ◆ столбец @AccountNumber представлен атрибутом AccountNumber в элементе Customer;
- ◆ столбец comment() в результате дает значение столбца RowGuid, которое возвращается как комментарий XML;
- ◆ столбец node() представлен в запросе константой XML, помещаемой непосредственно в результат XML без заключения в подэлемент;
- ◆ столбец AdditionalInfo/@Type преобразуется в атрибут Type в подэлементе AdditionalInfo;
- ◆ столбец AdditionalInfo/text() преобразуется в текст в заданном подэлементе AdditionalInfo.

У данного режима гораздо больше возможностей, чем у режимов RAW, и AUTO, потому что он позволяет добавлять в результат как атрибуты, так и подэлементы (вложенные элементы), а также включать в него другие разновидности структур XML. Кроме того, гораздо проще применять и легче читать режим PATH, чем запрос с использованием режима EXPLICIT.

## Вложение запросов *FOR XML*

Для создания иерархии все запросы с предложением FOR XML <режим> можно вкладывать друг в друга. Это означает, что для создания законченного документа XML вы можете поместить один запрос FOR XML в другой запрос FOR XML.

Рассмотрим следующий запрос и результат.

```
SELECT c.CustomerID AS "@Id"
 , c.AccountNumber AS "@AccountNumber"
 , c.CustomerType AS "@Type"
 , (SELECT TOP(2) -- Включен для ограничения размера результата XML
```

```

 soh.SalesOrderID AS "@Id"
 , soh.OrderDate AS "@OrderDate"
 , soh.ShipDate AS "@ShipDate"
 , (SELECT TOP(2) -- Включен для ограничения размера
 -- результата XML
 sod.ProductID AS "@ProductId", sod.OrderQty AS "@Quantity"
 FROM Sales.SalesOrderDetail AS sod
 WHERE sod.SalesOrderID = soh.SalesOrderID
 FOR XML PATH('OrderDetail'), TYPE)
 FROM Sales.SalesOrderHeader AS soh
 WHERE soh.CustomerID = c.CustomerID
 FOR XML PATH('Order'), TYPE) AS "Orders"
FROM Sales.Customer AS c
WHERE c.CustomerID = 1
FOR XML PATH('Customer');

```

Далее представлен результат.

```

<Customer Id="1" AccountNumber="AW00000001" Type="S">
 <Orders>
 <Order Id="43860" OrderDate="2001-08-01T00:00:00"
 ShipDate="2001-08-08T00:00:00">
 <OrderDetail ProductId="761" Quantity="2" />
 <OrderDetail ProductId="770" Quantity="1" />
 </Order>
 <Order Id="44501" OrderDate="2001-11-01T00:00:00"
 ShipDate="2001-11-08T00:00:00">
 <OrderDetail ProductId="761" Quantity="1" />
 <OrderDetail ProductId="768" Quantity="3" />
 </Order>
 </Orders>
</Customer>

```

Обратите внимание на то, что параметр `TYPE` обязателен в каждом вложенном запросе. Без параметра `TYPE` SQL Server интерпретирует результат вложенного запроса как закодированную строку, в формате XML. Если в предыдущем примере удалить все параметры `TYPE`, будет возвращен следующий результат XML.

```

<Customer Id="1" AccountNumber="AW00000001" Type="S">
 <Orders>
 <Order Id="43860" OrderDate="2001-08-01T00:00:00" ShipDate="2001-08-08T00:00:00"><OrderDetail ProductId="761"
 Quantity="2"/><OrderDetail ProductId="770"
 Quantity="1"/></Order><Order Id="44501" OrderDate="2001-11-01T00:00:00" ShipDate="2001-11-08T00:00:00"><OrderDetail
 ProductId="761" Quantity="1"/><OrderDetail ProductId="768"
 Quantity="3"/></Order>
 </Orders>
</Customer>

```

И наконец, в результат можно вставить пространство имен XML, применив в запросе предложение `WITH XMLNAMESPACES`. В следующем примере пространство `http://www.contoso.com/CustomerSchema` включено как пространство имен по умолчанию (default) для документа XML, а пространство имен `http://www.contoso.com/CustomerSchemaV2` добавлено под псевдонимом `v2` (единственный элемент, в котором применяется псевдоним `v2`, — вложенный элемент `AdditionalInfo`).

```
WITH XMLNAMESPACES(DEFAULT 'http://www.contoso.com/CustomerSchema'
 , 'http://www.contoso.com/CustomerSchemaV2' AS v2)
SELECT c.CustomerID AS "@Id", c.AccountNumber AS "@AccountNumber"
 , c.CustomerType AS "@Type"
 , c.ModifiedDate AS "v2:AdditionalInfo/@ModifiedDate"
FROM Sales.Customer AS c
WHERE c.CustomerID IN (1, 2)
FOR XML PATH('Customer'), ROOT('Customers');
```

Далее приведен результат.

```
<Customers xmlns:v2="http://www.contoso.com/CustomerSchemaV2"
 xmlns="http://www.contoso.com/CustomerSchema">
 <Customer Id="1" AccountNumber="AW00000001" Type="S">
 <v2:AdditionalInfo ModifiedDate="2004-10-13T11:15:07.263" />
 </Customer>
 <Customer Id="2" AccountNumber="AW00000002" Type="S">
 <v2:AdditionalInfo ModifiedDate="2004-10-13T11:15:07.263" />
 </Customer>
</Customers>
```

## Применение типа данных XML

В SQL Server 2008 включен тип данных XML, который может использоваться для хранения фрагментов и документов XML. На внутреннем уровне тип данных XML хранится с помощью типа данных `varbinary(max)`, т. е. XML сохраняется не как текстовая строка, а как двоичное представление документа или фрагмента XML.

Тип данных XML может быть типизированным и нетипизированным. Типизированный означает, что типу для проверки его содержимого присваивается коллекция схем XML.

В следующем примере показано создание коллекции схем XML и таблица со столбцами, имеющими типизированный и нетипизированный типы XML. Обратите внимание на то, что в столбец с типизированным XML включен параметр `DOCUMENT` для поддержки в этом столбце только документов (но не фрагментов) XML.

```
CREATE XML SCHEMA COLLECTION BooksSchemaCollection
AS
N'<?xml version="1.0"?>
```

```

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
 targetNamespace="http://www.contoso.com/BooksSchema"
 xmlns="http://www.contoso.com/BooksSchema"
 elementFormDefault="qualified">
 <xs:element name="Book">
 <xs:complexType>
 <xs:attribute name="Title" type="xs:string"/>
 <xs:attribute name="Price" type="xs:decimal"/>
 </xs:complexType>
 </xs:element>
</xs:schema>';
GO

CREATE TABLE Test.Person
(
 PersonID INT IDENTITY PRIMARY KEY
 , Name NVARCHAR(50) NOT NULL
 , FavoriteBookUntypedXml XML NULL
 , FavoriteBookTypedXml XML(DOCUMENT BooksSchemaCollection) NULL
);

```

Теперь рассмотрим следующие инструкции DML. Учтите, что обновление типизированного столбца некорректными данными (в данном случае неверной ценой книги, равной *FortyFive*, а не 45) приводит к получению сообщения об ошибке, информирующего о некорректности или недопустимости XML, но те же самые данные считаются допустимыми для столбца с нетипизированным типом данных XML.

```

INSERT Test.Person (Name) VALUES ('Jane Dow');

-- Завершится успешно:
UPDATE Test.Person SET
 FavoriteBookUntypedXml =
 '<Book xmlns="http://www.contoso.com/BooksSchema"
 Title="The Best Book" Price="FortyFive"/>'
WHERE PersonID = 1;

-- Завершится аварийно:
UPDATE Test.Person SET
 FavoriteBookTypedXml =
 '<Book xmlns="http://www.contoso.com/BooksSchema"
 Title="The Best Book" Price="FortyFive"/>'
WHERE PersonID = 1;

-- Завершится успешно:
UPDATE Test.Person SET
 FavoriteBookTypedXml =
 '<Book xmlns="http://www.contoso.com/BooksSchema"
 Title="The BestBook" Price="45"/>'
WHERE PersonID = 1;

```

Далее приведены результаты.

```
(1 row(s) affected)
```

```
(1 row(s) affected)
```

```
Msg 6926, Level 16, State 1, Line 3
```

```
XML Validation: Invalid simple type value: 'FortyFive'. Location:
/*:Book[1]/*:Price
```

```
(1 row(s) affected)
```

Еще одно последнее важное замечание касается существующей проблемы применения столбцов с типизированным XML: очевидно, что структура данных может измениться. Если вам нужно изменять схему (например, вставлять дополнительные атрибуты в элемент), вы сначала должны изменить тип данных всех столбцов, использующих схему, на нетипизированный XML. Затем нужно удалить коллекцию схем, создать ее снова с добавленными атрибутами и в заключение вернуть обратно тип данных столбцов к использованию схемы. Проблема состоит не только в том, что для создания сценариев, выполняющих эти изменения, потребуется много работы (что само по себе досадно), большой объем работы появится и у SQL Server, потому что у типизированного и нетипизированного кода XML разная внутренняя структура (например, все данные в нетипизированном XML хранятся как строки, а в типизированном XML — с использованием реального типа данных, выбранного в схеме), и при каждом изменении схемы XML SQL Server должен будет преобразовывать все данные из типизированных в нетипизированные и обратно.

Далее показан пример, отсоединяющий схему от столбца и затем присоединяющий ее снова.

```
ALTER TABLE Test.Person
 ALTER COLUMN FavoriteBookTypedXml XML NULL;
GO
```

```
ALTER TABLE Test.Person
 ALTER COLUMN FavoriteBookTypedXml XML(DOCUMENT BooksSchemaCollection)
 NULL;
```

## Работа с данными XML, хранящимися в табличной переменной или столбце

Если у вас есть данные, хранящиеся с помощью типа данных XML, вы наверняка захотите их запрашивать и обновлять. Делается это с помощью нескольких методов, предоставляемых типом данных XML.

**Метод exist.** Возвращает битовое значение и применяется для проверки наличия выражения XPath в экземпляре XML. В следующем примере показан простой запрос к столбцу Demographics (Демографические сведения), применяющий ме-

тод `exist` для поиска всех частных покупателей с величиной покупки `TotalPurchaseYTD` более 5000. В нем также используется функция `xs:decimal` на языке XPath для преобразования значения элемента в десятичное значение. (Это преобразование не нужно, если используется типизированный XML, т. к. SQL Server черпает из схемы XML сведения о том, что элемент действительно десятичный.)

```
WITH XMLNAMESPACES (DEFAULT
 'http://schemas.microsoft.com/sqlserver/2004/07/
 adventure-works/IndividualSurvey')
SELECT COUNT(*)
FROM Sales.Individual
WHERE Demographics.exist(
 '/IndividualSurvey/TotalPurchaseYTD[xs:decimal(.) > 5000]') = 1;
```

**Метод `value`.** Применяется для выполнения запроса на языке XQuery к экземпляру XML для извлечения из него единственного скалярного значения. В данном примере используется статистическая функция `count` на языке XPath для подсчета количества элементов `IndividualSurvey` (Частные покупатели) в столбце XML `Demographics`, а затем с помощью статистической SQL-функции `SUM` суммируются количества, возвращенные методом `value`.

```
WITH XMLNAMESPACES (DEFAULT
 'http://schemas.microsoft.com/sqlserver/2004/07/adventure-
 works/IndividualSurvey')
SELECT
 SUM(Demographics.value('count(/IndividualSurvey)', 'INT')) AS
 NumberOfIndividualSurveys
FROM Sales.Individual;
```

Далее приведен результат.

```
NumberOfIndividualSurveys

18484
```

Второй аргумент метода `value` указывает, в какой тип данных SQL следует преобразовать в результате извлеченное из документа XML значение.

**Метод `nodes`.** Применяется для преобразования экземпляра XML в табличную форму. В следующем примере каждый найденный элемент `Order` возвращается в результирующем наборе как отдельная строка. Помимо метода `nodes` в программном коде используется метод `value` для извлечения конкретных значений из документа XML в столбцы результирующего набора. Имейте в виду, что этот метод можно применять с инструкциями `INSERT...SELECT` или `UPDATE` для передачи в SQL Server множественных значений в одном параметре.

```
DECLARE @Orders XML;
SET @Orders = N'
```

```
<Orders>
 <Order Product="Bike" Quantity="1"/>
 <Order Product="Bike" Quantity="2"/>
 <Order Product="Car" Quantity="4"/>
</Orders>';

SELECT tab.col.value('@Product', 'NVARCHAR(50)') AS Product
 , tab.col.value('@Quantity', 'INT') AS Quantity
 , tab.col.value('count(..Order)', 'INT') AS TotalNumberOfOrders
FROM @Orders.nodes('/Orders/Order[xs:integer(@Quantity) > 1]') AS
 tab(col);
```

Далее показан результат.

Product	Quantity	TotalNumberOfOrders
Bike	2	3
Car	4	3

Для создания столбца `TotalNumberOfOrders` (Общее количество заказов) применяется функция `count` и путь к родительскому элементу (`..`). Это очень удобно, но знайте, что применение родительских путей в методе `nodes`, при вызове метода `value`, может значительно понизить производительность.

**Метод `query`.** Используется для выполнения запроса на языке XQuery к экземпляру XML, извлекающего вместо скалярного или табличного значения фрагмент XML. В методе `query` для извлечения нужных вам частей документа XML и представления их в определенной форме применяются выражения FLWOR языка XQuery. Выражения XQuery FLWOR могут использоваться и в других методах типа данных XML, но обычно они применяются в методе `query`. Выражение FLWOR состоит из ключевых слов `for`, `let`, `return`, `order by` и `where`, где `for` в первом приближении эквивалентно предложению `FROM` в языке SQL, `let` — объявлению обобщенного табличного выражения (OTB), `return` — инструкции `SELECT` в SQL, а `order by` и `where` — своим тезкам в SQL.

В следующем примере выражение FLWOR применяется для возврата из документа XML всех заказов с объемом, не меньшим двух, отсортированных по убыванию, и представления полученных элементов XML на шведском языке.

```
DECLARE @Orders XML;
SET @Orders = N'
<Orders>
 <Order Product="Bike" Quantity="1"/>
 <Order Product="Bike" Quantity="2"/>
 <Order Product="Car" Quantity="4"/>
</Orders>';

SELECT @Orders.query('
<Beställningar>
```



```
{
 for $o in /Orders/Order
 where xs:decimal($o/@Quantity) >= 2
 order by xs:decimal($o/@Quantity) descending
 return <Beställning Produkt="{data($o/@Product)}"
 Antal="{data($o/@Quantity)}"/>
}
</Beställningar>
');
```

Далее показан результат.

```
<Beställningar>
 <Beställning Produkt="Car" Antal="4" />
 <Beställning Produkt="Bike" Antal="2" />
</Beställningar>
```

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Применение XML

На этом практическом занятии вы примените запрос с режимом `FOR XML PATH` для возврата из SQL Server результирующего набора в виде XML и комбинацию методов `nodes` и `value` для вставки строк в таблицу на основе параметра типа XML, передаваемого в хранимую процедуру.

### Упражнение 1. Использование *FOR XML PATH*

В этом упражнении для создания документа XML вы выполните запрос к таблицам `Production.Product`, `Sales.SalesOrderHeader` и `Sales.SalesOrderDetail`.

1. Откройте среду Microsoft SQL Server Management Studio (SSMS) и установите соединение с подходящим экземпляром SQL Server 2008.
2. В окне создания запроса выполните запрос на SQL к базе данных AdventureWorks, формирующий следующий документ XML для товара с `ProductID = 707`.

```
<Product Id="707">
 <Info Name="Sport-100 Helmet, Red" ListPrice="34.9900" />
 <Order Id="43665" Date="2001-07-01T00:00:00" CustomerId="146" />
 <Order Id="43668" Date="2001-07-01T00:00:00" CustomerId="514" />
 <Order Id="43673" Date="2001-07-01T00:00:00" CustomerId="618" />
 <Order Id="43677" Date="2001-07-01T00:00:00" CustomerId="679" />
 . . .
</Product>
```

Далее приведены корректные инструкции, но не смотрите на них, пока не попробуете создать запрос самостоятельно. Введите, выделите и выполните следующий программный код.

```
USE AdventureWorks;
GO
```

```

SELECT p.ProductID AS "@Id", p.Name AS "Info/@Name"
 , p.ListPrice AS "Info/@ListPrice"
 , (SELECT DISTINCT soh.SalesOrderID AS "@Id"
 , soh.OrderDate AS "@Date"
 , soh.CustomerID AS "@CustomerId"
 FROM Sales.SalesOrderHeader AS soh
 INNER JOIN Sales.SalesOrderDetail AS sod
 ON sod.SalesOrderID = soh.SalesOrderID
 WHERE sod.ProductID = p.ProductID
 FOR XML PATH('Order'), TYPE)
FROM Production.Product AS p
WHERE p.ProductID = 707
FOR XML PATH('Product');

```

## Упражнение 2. Применение методов *nodes* и *value*

В этом упражнении вы создадите хранимую процедуру, которая принимает параметр типа XML. Хранимая процедура преобразует документ XML в табличную форму с помощью методов *nodes* и *value* и вставляет результаты в таблицу.

1. Если необходимо, откройте среду SSMS и установите соединение с подходящим экземпляром SQL Server 2008.
2. В окне запроса введите, выделите и выполните следующие инструкции SQL для создания новой таблицы и схемы в базе данных AdventureWorks.

```

USE AdventureWorks;
GO

CREATE SCHEMA TestXml;
GO

CREATE TABLE TestXml.Messages
(MessageId INT IDENTITY PRIMARY KEY
, FromUser NVARCHAR(50) NOT NULL
, Message NVARCHAR(max) NOT NULL
, CreatedDateTime DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);

```

3. В имеющемся окне запроса введите, выделите и выполните следующие инструкции SQL, создающие хранимую процедуру, которая заполняет таблицу TestXml.Messages данными из документа XML.

```

CREATE PROCEDURE TestXml.spMessageInsertMultiple
@Messages XML
AS
BEGIN
 SET NOCOUNT ON;

 INSERT TestXml.Messages (FromUser, Message)
 SELECT tab.col.value('@FromUser', 'NVARCHAR(50)')
 , tab.col.value('text()[1]', 'NVARCHAR(max)')

```

```
FROM @Messages.nodes('/Messages/Message') AS tab(col);
END
```

4. В заключение в имеющемся окне запроса введите, выделите и выполните следующие инструкции SQL для выполнения хранимой процедуры (добавление двух сообщений в таблицу), а затем выполните запрос к таблице, чтобы увидеть результаты.

```
EXEC TestXml.spMessageInsertMultiple @Messages = N'
<Messages>
 <Message FromUser="Jeff Low">Hi, how are you?</Message>
 <Message FromUser="Jane Dow">Not bad, and yourself?</Message>
</Messages>';

SELECT * FROM TestXml.Messages;
```

Результат, возвращенный запросом, должен выглядеть следующим образом.

MessageId	FromUser	Message	CreatedDateTime
1	Jeff Low	Hi, how are you?	2008-11-05 12:40:56.14
2	Jane Dow	Not bad, and yourself?	2008-11-05 12:40:56.14

## Резюме урока

- ◆ Данные на языке XML можно сгенерировать с помощью инструкции SELECT в четырех разных режимах: FOR XML RAW, FOR XML AUTO, FOR XML PATH и FOR XML EXPLICIT.
- ◆ Для генерации данных XML предпочтительнее использовать режим FOR XML PATH.
- ◆ Тип данных XML может быть как типизированным (допустимость которого проверяется коллекцией схем XML), так и нетипизированным.
- ◆ В нетипизированном типе данных XML все значения всегда интерпретируются как строки.
- ◆ Для запроса и изменения экземпляров типа XML можно применять методы value, query, exist, nodes и modify.

## УРОК 2. Применение SQLCLR и *Filestream*

Давайте начнем этот урок с обсуждения того, что такое SQLCLR. SQLCLR обозначает функционирование среды CLR (Общезыковая среда исполнения) в компоненте SQL Server Database Engine. Среда CLR часто называют .NET Framework или просто .NET, а также по имени языка программирования, который может использоваться в ней, например, Visual Basic .NET или C#. Применение CLR в SQL Server — важное средство повышения гибкости SQL Server, расширяющее

возможности разработчиков по сравнению с простым применением языка T-SQL и его системных функций. Мы начнем этот урок с изучения основ применения среды CLR в SQL Server, а затем обсудим ситуации, требующие построения определенных объектов базы данных средствами CLR. Далее мы рассмотрим, как можно создавать с помощью CLR хранимые процедуры, триггеры, функции, типы и статистические функции или итоги. В конце урока мы поговорим о том, как создавать объекты *Filestream* для хранения больших двоичных объектов (binary large objects, BLOB) в SQL Server 2008.

---

**После усвоения этого урока вы сможете**

---

- Создавать хранимые процедуры, используя SQLCLR.
- Создавать скалярные функции, определенные пользователем (ФОП) с помощью SQLCLR.
- Создавать с помощью SQLCLR ФОП, возвращающие табличное значение.
- Создавать триггеры, используя SQLCLR.
- Создавать с помощью SQLCLR пользовательские типы данных.
- Формировать сводные или статистические функции, определенные пользователем, используя SQLCLR.
- Хранить в SQL Server объекты BLOB с помощью объекта *Filestream*.

Расчетное время урока — 60 мин.

## Основы применения SQLCLR

Для использования SQLCLR в базе данных следует выполнить следующие действия:

1. Установить в экземпляре SQL Server режим, позволяющий запускать программный код CLR.
2. Написать с помощью языка среды .NET (в эту главу мы включили для справки примеры программного кода на языках C# и Visual Basic .NET) программный код, который будет использоваться объектом.
3. Откомпилировать программный код, превратив его в сборку (assembly) (выполняемый CLR).
4. Загрузить сборку в SQL Server.
5. И в заключение создать средствами языка определения данных (Data Definition Language, DDL) объект базы данных и в нем указатель на сборку.

Теперь, следуя перечисленным действиям, создадим очень простую хранимую процедуру CLR, которая должна будет просто выполнить инструкцию SQL

```
SELECT * FROM Sales.Customer WHERE CustomerID = @CustomerID
```

Несмотря на то, что для данной хранимой процедуры вы никогда не будете применять среду CLR, это хороший пример, т. к. позволяет ясно увидеть разницу

между созданием хранимой процедуры для выполнения параметризованного запроса на языке T-SQL и использованием среды SQLCLR для той же самой цели. Далее для справки приведена версия хранимой процедуры на T-SQL.

```
CREATE PROCEDURE Sales.spCustomerGet
@CustomerID INT
AS
BEGIN
 SELECT * FROM Sales.Customer
 WHERE CustomerID = @CustomerID;
END
```

Для выполнения того же запроса средствами SQLCLR сначала мы разрешим выполнять в данном экземпляре SQL Server программный код SQLCLR с помощью приведенного далее программного кода на языке T-SQL. Имейте в виду, что системным типам и функциям, использующим SQLCLR (таким как пространственные типы *geography* и *geometry*), не требуется включение среды CLR, это нужно только для пользовательского программного кода.

```
EXEC sp_configure 'clr enabled', 1;
RECONFIGURE;
```

Первая строка кода изменяет установочный параметр 'CLR enabled' на 1 (истина), а вторая строка заставляет SQL Server начать применять любые измененные параметры конфигурации.

Теперь самое время написать программный код .NET (на языке C# или Visual Basic .NET) для хранимой процедуры. В этом примере мы создадим единственный файл, содержащий код .NET, и затем откомпилируем его с помощью командной строки.

### **СОВЕТ.**

#### ***Применение Visual Studio для создания файлов***

Если у вас есть под рукой платформа Microsoft Visual Studio, ее можно использовать для создания файлов. Для создания ваших CLR-объектов базы данных примените шаблон **SQL Server Project**, который можно найти под заголовком типа проекта **Database**. Visual Studio может даже помещать CLR-объекты базы данных непосредственно в SQL Server (т. е. Visual Studio выполнит все остальные действия, описываемые в этом разделе).

Создадим файл с именем CLRStoredProc.cs (если вы используете C#) или CLRStoredProc.vb (если вы применяете Visual Basic .NET) и добавим в него следующий программный код.

```
' VB:
Imports System
Imports System.Data.SqlTypes
Imports System.Data.SqlClient
Imports Microsoft.SqlServer.Server
```

```

Namespace TK433.Clr
Public Class Demo
Public Shared Sub CustomerGetProcedure(ByVal customerId As SqlInt32)
Using conn As SqlConnection = _
 New SqlConnection("context connection=true")
Dim cmd As SqlCommand = conn.CreateCommand()
cmd.CommandText = "SELECT * FROM Sales.Customer " & vbCrLf & _
 "WHERE CustomerID = @CustomerID;"
cmd.Parameters.AddWithValue("@CustomerID", customerId)
conn.Open()
' Выполнить команду и отправить результаты
' в вызывающее приложение
SqlContext.Pipe.ExecuteAndSend(cmd)
End Using
End Sub
End Class
End Namespace

```

```

// C#:
using System;
using System.Data.SqlTypes;
using System.Data.SqlClient;
using Microsoft.SqlServer.Server;

namespace TK433.Clr
{
 public class Demo
 {
 static public void CustomerGetProcedure(SqlInt32 customerId)
 {
 using (SqlConnection conn = new SqlConnection("context
 connection=true"))
 {
 SqlCommand cmd = conn.CreateCommand();
 cmd.CommandText = @"SELECT * FROM Sales.Customer
 WHERE CustomerID = @CustomerID";
 cmd.Parameters.AddWithValue("@CustomerID", customerId);
 conn.Open();
 // Выполнить команду и отправить результаты
 // в вызывающее приложение
 SqlContext.Pipe.ExecuteAndSend(cmd);
 }
 }
 }
}

```

Отметим некоторые особенности данного примера.

- ◆ Строка соединения "context connection=true" заставляет объект `SqlConnection` подключиться к экземпляру SQL Server, в котором вы уже работаете. Имейте

в виду, что вы все еще находитесь в том же сеансе, в котором были вне кода CLR. Это означает, что вы не блокируетесь никакими блокировками, поддерживаемыми сеансом, выполняющим данный код CLR.

- ◆ В примере используются следующие дополнительные пространства имен.
  - Пространство имен `System.Data.SqlClient` содержит классы `SqlConnection` и `SqlCommand`, применяемые в данном примере. Эти классы нужны для соединения с SQL Server и выполнения запроса. Учтите, что это те же самые классы, которые вы применяете для соединения из приложения.
  - Пространство имен `System.Data.SqlTypes` содержит тип данных `SqlInt32`, применяемый в данном примере. Этот тип имитирует поведение типа `INTEGER` в SQL Server. Для получения целочисленных значений CLR можно использовать свойство `Value` типа `SqlInt32`.
  - Пространство имен `Microsoft.SqlServer.Server` содержит классы `SqlContext` и `SqlPipe` (необходимые для вызова `SqlContext.Pipe`), применяемые в данном примере. Эти классы специфичны для реализации SQLCLR и используются для связи с сеансом SQL Server, в котором вы работаете в данный момент.
- ◆ Программный код очень похож на любой код, обращающийся к SQL Server из приложения.
- ◆ Приведенный пример кажется излишне громоздким для выполнения простой инструкции `SELECT`. Очевидно, что вам не захочется применять CLR для простых хранимых процедур, подобных приведенной в данном примере.

Следующий шаг — компиляция программного кода в сборку CLR. Делается это с помощью компилятора C# (`csc.exe`) или компилятора Visual Basic .NET (`vbc.exe`). Оба эти компилятора обычно можно найти в каталоге `C:\Windows\Microsoft.NET\Framework\v3.5`. Для компиляции кода из командной строки выполните следующую команду, находясь в каталоге, хранящем исходный программный код. (Имейте в виду, что вы должны установить свой путь, указывающий на каталог, содержащий компилятор.)

' VB:

```
vbc.exe /target:library /out:CLRStoredProc.dll CLRStoredProc.vb
```

// C#:

```
csc.exe /target:library /out:CLRStoredProc.dll CLRStoredProc.cs
```

Ключ `/target:library` сообщает компилятору о том, что вы компилируете сборку без начальной точки, т. е. сборка не может непосредственно выполняться. Это просто библиотека типов и, следовательно, она должна иметь расширение `dll`. Ключ `/out` сообщает компилятору, как назвать созданную сборку.

Теперь пришло время загрузить созданную сборку в вашу базу данных. Делается с помощью инструкции `CREATE ASSEMBLY`. Когда выполняется эта инструкция,

сборка физически копируется в базу данных, к которой вы подсоединены. Следовательно, после того как сборка будет загружена, вы можете удалить файл сборки с расширением dll, т. к. SQL Server он больше не нужен. Хранить сборку в базе данных очень удобно, потому что, когда вы перемещаете базу данных в другой экземпляр сервера, все сборки в базе данных перемещаются вместе с ней. Для просмотра сборок, имеющихся в базе данных, можно запросить представление каталога `sys.assemblies`.

Следующий программный код на языке T-SQL загружает сборку в базу данных AdventureWorks и выводит на экран ее свойства, запросив представление каталога `sys.assemblies`.

```
USE AdventureWorks;
```

```
CREATE ASSEMBLY TK433ClrDemo
FROM 'C:\TK433Clr\CLRStoredProc.dll';
```

```
SELECT assembly_id, name
FROM sys.assemblies
WHERE name = 'TK433ClrDemo';
```

Результат должен выглядеть подобно приведенному далее.

assembly_id	name
65544	TK433ClrDemo

Наконец перейдем к последнему шагу, созданию хранимой процедуры таким образом, чтобы ее могли выполнять пользователи. В следующем программном коде создается хранимая процедура с именем `Sales.spCustomerGetClr`, указывающая на метод CLR, именуемый `CustomerGetProcedure`, в классе `TK433.Clr.Demo` сборки `TK433ClrDemo`. После создания в следующем примере хранимой процедуры мы также запросим в представлении каталога `sys.assembly_modules` информацию, касающуюся хранимой процедуры CLR. Модули T-SQL можно найти в представлении каталога `sys.sql_modules`.

```
CREATE PROCEDURE Sales.spCustomerGetClr
@CustomerID INT
AS
EXTERNAL NAME TK433ClrDemo."TK433.Clr.Demo".CustomerGetProcedure;
GO
```

```
SELECT assembly_id, assembly_class, assembly_method
FROM sys.assembly_modules
WHERE object_id = OBJECT_ID('Sales.spCustomerGetClr');
```

Результат должен быть таким.

assembly_id	assembly_class	assembly_method
65544	TK433.Clr.Demo	CustomerGetProcedure



После того как хранимая процедура создана, ее можно выполнять. Выполнение хранимой процедуры выглядит точно так же, как выполнение хранимой процедуры на языке T-SQL.

```
EXEC Sales.spCustomerGetClr @CustomerID = 1;
```

Далее приведен результат (сокращенный для наглядности).

CustomerID	TerritoryID	AccountNumber	CustomerType...
1	1	AW00000001	S ...

Прежде чем мы продолжим и рассмотрим подробности создания объектов SQLCLR разных типов, следует отметить, что вы можете также хранить в базе данных исходный код CLR. SQL Server не использует исходный код, но его удобно хранить в базе данных, потому что позже этот программный код можно извлечь из базы данных (к тому времени вы можете потерять действующий исходный программный код). Для добавления в базу данных файла с исходным программным кодом выполните следующую инструкцию.

```
ALTER ASSEMBLY TK433ClrDemo
 ADD FILE FROM 'C:\TK433Clr\CLRStoredProc.cs'; -- Or .vb...
```

Для того чтобы увидеть, какие файлы были добавлены в сборку, можно запросить представление каталога `sys.assembly_files`. С помощью этого представления также можно найти реальную сборку (файл с расширением `dll`).

## Объекты, которые можно создать с помощью SQLCLR

Средствами SQLCLR можно создавать объекты следующих типов:

- ◆ хранимые процедуры (как показано в предыдущем примере);
- ◆ скалярные ФОП, возвращающие единственное значение;
- ◆ табличные ФОП, которые возвращают таблицу и могут вызываться из предложений `FROM`, `JOIN` или `APPLY`;
- ◆ триггеры (DML, DDL и триггеры входа);
- ◆ статистические функции или итоги, определенные пользователем;
- ◆ типы данных, определенные пользователем.

Между прочим, последние два объекта, итоги, определенные пользователем, и пользовательские типы могут создаваться только средствами SQLCLR; с помощью языка T-SQL их создать нельзя.

Поскольку мы уже обсудили, как создавать хранимую процедуру CLR, начнем с рассмотрения способов создания скалярной функции, определенной пользователем (ФОП).

## Создание скалярной ФОП

Как и хранимая процедура, скалярная функция содержит единственный метод. Разница состоит в том, что в ФОР метод должен вернуть значение, а в хранимой процедуре метод должен вернуть `void` (для C#) или быть `Sub` (для Visual Basic .NET). ФОР — действительно превосходное средство SQLCLR, поскольку их легко можно создать практически для чего угодно в среде .NET Framework. Как правило, вы будете применять ФОР для выполнения вычислений, а не для доступа к таблицам. Обычно ваши функции затем будут применяться в инструкциях SQL DML. В данном примере мы создадим ФОР, которая позволяет применять в SQL Server регулярные выражения. Использование регулярных выражений — часто требующееся средство, которого исходно нет в SQL Server, но которое вы легко можете "уташить" из .NET Framework с помощью SQLCLR. В следующих примерах мы не будем обсуждать компиляцию или загрузку сборки, потому что эти шаги уже были описаны в уроке.

Следующий код CLR используется для создания ФОР. Для краткости в пример включен только метод и импортируемые пространства имен.

[illegible]

```

 return (SqlBoolean)Regex.IsMatch(input.Value, pattern.Value);
}
. . .

```

Обратите внимание на атрибут `SqlFunction`, который применяется для определения функции. Он сообщает SQL Server о том, что функция точная (т. е. не использует вычисления с плавающей точкой, влияющие на возвращаемое значение) и детерминированная (т. е. при одних и тех же входных параметрах в случае многократных вызовов возвращает одно и то же значение), поэтому результат функции может сохраняться в вычисляемом столбце и индексироваться.

После компиляции и загрузки сборки в SQL Server для создания функции выполняется следующий программный код.

```

CREATE FUNCTION dbo.fnIsRegExMatch
(
 @Input NVARCHAR(MAX)
 , @Pattern NVARCHAR(100)
)
RETURNS BIT
AS
EXTERNAL NAME TK433ClrDemo."TK433.Clr.Demo".IsRegExMatch;

```

Теперь эту функцию можно использовать, когда вы захотите применить регулярное выражение в запросе. В следующем примере на T-SQL запрос возвращает количество строк в таблице `Sales.CreditCards` (Кредитные карты), в которых нет номера кредитной карты с 14 цифрами. Делается это с помощью регулярного выражения `^[0-9]{14}$`.

```

SELECT COUNT(*) AS InvalidCreditCardNumbers
FROM Sales.CreditCard
WHERE dbo.fnIsRegExMatch(CardNumber, N'^[0-9]{14}$') = 0;

```

Далее показан результат.

```

InvalidCreditCardNumbers

0

```

Другая интересная особенность ФОР CLR (и T-SQL) состоит в том, что они могут применяться в ограничениях. В следующем примере показано, как можно добавить в таблицу `Sales.CreditCards` проверочное ограничение, допускающее наличие только номеров кредитных карт, равных `NULL` или соответствующих регулярному выражению из предыдущего примера.

```

ALTER TABLE Sales.CreditCard
 ADD CONSTRAINT CKCreditCardNumber
 CHECK(dbo.fnIsRegExMatch(CardNumber, N'^[0-9]{14}$') = 1
 OR CardNumber IS NULL);

-- Проверьте ограничение, попытавшись вставить
-- некорректный номер карты
UPDATE Sales.CreditCard SET CardNumber = '1234' WHERE CreditCardID = 1;

```

Далее показана возникающая в результате ошибка.

Msg 547, Level 16, State 0, Line 1

Инструкция UPDATE противоречила проверочному ограничению "CKCreditCardNumber". Конфликт возник в базе данных AdventureWorks, таблице Sales.CreditCard, столбце CardNumber.

Поскольку в коде CLR мы поместили функцию как точную (IsPrecise) и детерминированную (IsDeterministic), можно сохранить и индексировать вычисляемый столбец, использующий функцию. Хотя в результате данного конкретного примера мало пользы от индексирования или сохранения, мы рассмотрим порядок действий, потому что это может пригодиться в других случаях. Для индексирования результата функции сначала ее нужно поместить в выражение вычисляемого столбца. В следующем примере на языке T-SQL в таблицу добавляется и помечается атрибутом PERSISTED использующий функцию вычисляемый столбец. Этот атрибут сообщает SQL Server о необходимости теперь вычислять результат функции, только если он записывается в основной столбец или столбцы, и сохранять результат функции физически в табличной строке. После добавления столбца мы создадим для него индекс. Учтите, что для индексирования столбца не нужно помечать вычисляемый столбец как сохраняемый (persisted). В запросе к новому столбцу, приведенному в конце примера, используется вновь созданный индекс.

```
ALTER TABLE Sales.CreditCard
 ADD IsValidCardNumber AS dbo.fnIsRegExMatch(CardNumber, N'^[0-9]{14}$')
 PERSISTED;
GO
```

```
CREATE NONCLUSTERED INDEX IsValidCardNumberIdx
 ON Sales.CreditCard (IsValidCardNumber);
GO
```

```
-- Этот запрос находит применение индексу IsValidCardNumberIdx и
-- выполняет операцию поиска в нем
SELECT COUNT(*)
FROM Sales.CreditCard
WHERE IsValidCardNumber = 1;
```

## Создание ФОП CLR, возвращающей табличное значение

Поскольку табличная (возвращающая табличное значение) ФОП возвращает множественные значения (или множественные строки), она немного сложнее скалярной ФОП. В то время как скалярная ФОП содержит один метод, табличная ФОП состоит из двух методов.

- ◆ Первый метод действует как итератор или конечный автомат (state machine), обрабатывая в цикле значения, которые должны быть возвращены как строки.

- ◆ Второй метод выполняется для каждой строки и заполняет реальную строку, возвращаемую механизму исполнения SQL Server.

Рассмотрим следующий код CLR, определяющий для табличной ФОРЭМ эти два метода. Данная функция может применяться для разбиения строк с запятыми в качестве разделителей.

### ***Почему нет примера на Visual Basic .NET***

В данном примере для создания итератора (управления циклом) используется ключевое слово `yield` языка C#. В языке Visual Basic .NET нет ключевого слова `yield`, поэтому пример программного кода на нем не представлен. С помощью Visual Basic .NET можно создавать табличные функции, но для этого требуется создать класс, действующий как итератор и обрабатывающий значения циклически (т. е. то, что делает для вас `yield` в C#).

```
// C#:
using System;
using System.Collections;
using System.Collections.Generic;
using System.Data.SqlTypes;
using Microsoft.SqlServer.Server;

...
[SqlFunction(FillRowMethodName = "SplitStringFillRow",
 TableDefinition = "ValueIndex INT, Value NVARCHAR(100)")]
static public IEnumerator SplitString(SqlString stringToSplit)
{
 // Выход, если строка для разбиения равна NULL
 if (stringToSplit.IsNull)
 yield break;

 int valueIndex = 0;
 foreach (string s in stringToSplit.Value.Split(new char[] { ',' },
 StringSplitOptions.RemoveEmptyEntries))
 {
 yield return new KeyValuePair<int, string>(valueIndex++, s.Trim());
 }
}

static public void SplitStringFillRow(object oKeyValuePair,
 out SqlInt32 valueIndex, out SqlString value)
{
 // Извлечение пары ключевых значений из первого параметра
 KeyValuePair<int, string> keyValuePair = (KeyValuePair<int,
 string>)oKeyValuePair;

 // Установка значений каждого выходного параметра
 valueIndex = keyValuePair.Key;
 value = keyValuePair.Value;
}

...
```

В предыдущем примере, как вы могли видеть, мы создали два метода `SplitString` и `SplitStringFillRow`. Когда ФОР выполняется из SQL Server, первым выполняется метод `SplitString`. Каждый оператор `yield return`, выполняющийся внутри метода, вызывает метод `SplitStringFillRow` (т. е. метод заполнения строки, заданный в атрибуте `SqlFunction`). Результирующий набор, возвращаемый функцией `SplitString`, определяется с помощью параметра `TableDefinition` атрибута `SqlFunction`.

Метод заполнения строки всегда принимает один параметр типа `System.Object`, содержащий ссылку на то, для чего вы вызвали `yield return`, а также один параметр `out` для каждого столбца, который должен возвращаться функцией в соответствии с параметром `TableDefinition`. В следующем примере вы увидите программный код на языке T-SQL, применяемый для создания и запрашивания табличной ФОР.

```
CREATE FUNCTION dbo.fnSplitString
(
 @StringToSplit NVARCHAR(max)
)
RETURNS TABLE (ValueIndex INT, Value NVARCHAR(100))
AS
EXTERNAL NAME TK433ClrDemo."TK433.Clr.Demo".SplitString;
GO
```

```
SELECT ValueIndex, Value
FROM dbo.fnSplitString('Hi,how,are,you?') AS a;
```

Далее показан результат.

ValueIndex	Value
0	Hi
1	how
2	are
3	you?

## Создание триггера CLR

Создание триггера CLR очень похоже на создание хранимой процедуры CLR. Программный код CLR состоит из единственного метода или `Sub`, которые выполняют нужные вам действия. У триггера CLR, так же как у триггера T-SQL, есть доступ к специальным таблицам `inserted` и `deleted`. В следующем примере триггера CLR выполняется откат транзакции, если инструкция, запускающая триггер, удаляет более одной строки.

```
' VB:
Imports System.Data.SqlClient
Imports System.Text.RegularExpressions
Imports Microsoft.SqlServer.Server
```

```

...
Public Shared Sub ClrTrigger()
 ' Если это не инструкция delete, просто выход.
 If SqlContext.TriggerContext.TriggerAction <> TriggerAction.Delete _
 Then
 Return
 End If

 Using conn As SqlConnection = New SqlConnection(_
 "context connection=true")
 Dim cmd As SqlCommand = conn.CreateCommand()
 Cmd.CommandText = "SELECT COUNT(*) FROM deleted"
 conn.Open()
 ' Проверяет количество строк, найденных в таблице deleted
 If (CType(cmd.ExecuteScalar(), Integer)) > 1 Then
 cmd.CommandText = _
 "RAISERROR('Too many rows deleted, rolling back " & _
 "transaction.', 16, 1);" & vbCrLf & _
 "ROLLBACK TRAN;"

 ' Этот блок try/catch нужен для того, чтобы пропустить ошибку,
 ' вызванную выполнением инструкции RAISERROR
 Try
 SqlContext.Pipe.ExecuteAndSend(cmd)
 Catch
 End Try
 End If
 End Using
End Sub
...

// C#:
using System;
using System.Data.SqlClient;
using Microsoft.SqlServer.Server;

...
static public void ClrTrigger()
{
 // Если это не инструкция delete, просто выход
 if (SqlContext.TriggerContext.TriggerAction != TriggerAction.Delete)
 return;
 using (SqlConnection conn = new SqlConnection("context
 connection=true"))
 {
 SqlCommand cmd = conn.CreateCommand();
 cmd.CommandText = "SELECT COUNT(*) FROM deleted;";
 conn.Open();
 }
}

```

```
// Проверяет количество строк, найденных в таблице deleted.
if (((int)cmd.ExecuteScalar()) > 1)
{
 cmd.CommandText = @"RAISERROR('Too many rows deleted, " +
 "rolling back transaction.', 16, 1);" +
 "ROLLBACK TRAN;";

 // Этот блок try/catch нужен для того, чтобы пропустить
 // ошибку, вызванную выполнением инструкции RAISERROR
 try
 {
 SqlContext.Pipe.ExecuteAndSend(cmd);
 }
 catch { }
}
}
```

Для создания триггера мог бы использоваться следующий программный код.

```
CREATE TRIGGER dbo.ClrTrigger
ON dbo.MyTable
AFTER DELETE
AS
EXTERNAL NAME TK433ClrDemo."TK433.Clr.Demo".ClrTrigger;
GO
```

Как и в случае хранимых процедур CLR, создание таким способом этого простого триггера выглядит очень громоздким по сравнению с созданием триггера на T-SQL, поскольку для выполнения любой инструкции SQL необходимо использовать объекты `SqlConnection` и `SqlCommand`. Как видите, у класса `SqlContext`, названного `TriggerContext` (типа `SqlTriggerContext`), есть свойство, которое может применяться для выполнения из кода CLR характерных для триггеров проверок.

Триггеры CLR следует применять, только если нужно решать задачи, которые нельзя выполнить с помощью обычных триггеров T-SQL, или если задачи требуют интенсивных вычислений и триггер CLR действует лучше, чем триггер T-SQL.

### **Формирование средствами CLR статистических функций или итогов, определенных пользователем**

Возможность создания пользовательских статистических функций может быть очень полезна для получения сводных данных (итогов), не предусмотренных в SQL Server (например, произведений), или для создания статистической функции, способной обрабатывать пользовательский тип данных CLR.

Формируя в среде CLR итоги, определенные пользователем, вы должны создать вместо одного или двух методов, как в предыдущих примерах объектов CLR,



целиком тип данных CLR (`class` или `struct`). Это необходимо, потому что этот тип должен уметь выполнять несколько задач, включая следующие.

- ◆ *Инициализация себя самого.* Она выполняется с помощью метода `Init` сводного типа данных CLR.
- ◆ *Добавление нового значения в вычисление.* Для каждого значения, которое нужно добавить в вычисление, вызывается метод `Accumulate`.
- ◆ *Комбинирование себя с другим экземпляром той же самой статистической функции.* Это делается вызовом метода `Merge`.
- ◆ *Возврат своего результата.* Это выполняется с помощью метода `Terminate`.

Прежде чем мы начнем подробно изучать получение итогов, определенных пользователем, потратим несколько минут на чтение следующего примера программного кода CLR, который определяет итог в виде арифметического произведения. Результат данного итога — перемноженные все входные параметры (например,  $5 \times 10 \times 15 = 750$ ). Для включения дополнительной функциональной возможности мы добавили второй параметр, сообщающий итогу о включении или не включении в результат нулей (поскольку включение нуля всегда даст в результате нулевое значение итога). Возможность применения нескольких параметров в определенных пользователями итогах появилась в версии SQL Server 2008. Далее приведено несколько замечаний, касающихся данного программного кода.

- ◆ В этом примере атрибут `SqlUserDefinedAggregate` сообщает SQL Server о том, что мы будем вручную управлять сериализацией итога (если это необходимо), задавая формат, который будет определен пользователем (`Format.UserDefined`). Атрибут также сообщает SQL Server о том, что ему придется вернуть `NULL`, если в подведение итога не включено никаких значений (`IsNullIfEmpty = True`), что итог следит за дубликатами (`IsInvariantToDuplicates = False`), не отслеживает значения `NULL` (`IsInvariantToNulls = True`) и не заботится о порядке следования входных параметров (`IsInvariantToOrder = True`). И, наконец, атрибут сообщает SQL Server о том, что максимальный объем памяти, используемый при сериализации этого итога равен 19 байтам (16 байт для десятичного значения и по 1 байту для трех логических значений).
- ◆ Атрибут `SqlFacet` применяется в программном коде несколько раз для задания разрядности и точности представления типа данных `SqlDecimal`.
- ◆ Поскольку при определении итога мы используем формат, определенный пользователем, должен быть реализован интерфейс `IBinarySerialize`. Вы также можете выбрать вариант использования собственного формата системы, но в данном примере он не сработает, потому что сериализация, реализованная в системе, не поддерживает применяемый десятичный тип данных.

- ◆ При необходимости для сериализации экземпляра во время выполнения SQL Server вызывает метод `IBinarySerialize.Write`.
- ◆ При необходимости для десериализации экземпляра во время выполнения SQL Server вызывает метод `IBinarySerialize.Read`.

' VB:

```
Imports System
Imports System.Data.SqlTypes
Imports System.Runtime.InteropServices
Imports Microsoft.SqlServer.Server

...
<SqlUserDefinedAggregate(Format.UserDefined, IsNullIfEmpty:=True, _
IsInvariantToDuplicates:=False, IsInvariantToNulls:=True, _
IsInvariantToOrder:=True, MaxByteSize:=19)> _
Public Structure DecimalProductAggregate
 Implements IBinarySerialize

 Private m_Value As Decimal ' 16 bytes storage
 Private m_ValueIsNull As Boolean ' 1 byte storage
 Private m_SkipZeros As Boolean ' 1 byte storage
 Private m_SkipZerosIsNull As Boolean ' 1 byte storage

 Public Sub Init()
 Me.m_ValueIsNull = True
 Me.m_SkipZerosIsNull = True
 End Sub

 Public Sub Accumulate(<SqlFacet(Precision:=38, Scale:=5)>ByVal value _
 As SqlDecimal, ByVal skipZeros As SqlBoolean)
 If skipZeros.IsNull Then
 Throw New InvalidOperationException(_
 "The @SkipZeros parameter cannot be null.")
 End If

 If value.IsNull Then
 Return
 End If

 ' Init пропускает флаг Zeros, если он не был установлен
 If (Me.m_SkipZerosIsNull) Then
 Me.m_SkipZeros = skipZeros.Value
 Me.m_SkipZerosIsNull = False
 ElseIf Me.m_SkipZeros <> skipZeros.Value Then
 ' Не разрешает изменять во время выполнения параметр SkipZeros
 Throw New InvalidOperationException(_
 "The @SkipZeros parameter cannot be changed.")
 End If
```

```

' Пропускает нулевые значения, если установочные параметры говорят
' об этом и текущее значение равно нулю.
If Me.m_SkipZeros And value.Value = 0 Then
 Return
End If

' Если это первое значение, просто присвойте его
If Me.m_ValueIsNull Then
 Me.m_Value = value.Value
 Me.m_ValueIsNull = False
 Return
End If

Me.m_Value *= value.Value
End Sub

Public Sub Merge(ByVal other As DecimalProductAggregate)
 Dim skipZeros As SqlBoolean
 If other.m_SkipZerosIsNull Then
 skipZeros = SqlBoolean.Null
 Else
 skipZeros = other.m_SkipZeros
 End If
 Dim otherValue As SqlDecimal = other.Terminate()

 Me.Accumulate(otherValue, skipZeros)
End Sub

Public Function Terminate() As <SqlFacet(Precision:=38, Scale:=5)> _
 SqlDecimal
 If Me.m_ValueIsNull Then
 Return SqlDecimal.Null
 End If
 Return Me.m_Value
End Function

' Чтение итога из SQL Server
Sub Read(ByVal r As System.IO.BinaryReader) Implements _
 IBinarySerialize.Read
 Me.m_ValueIsNull = r.ReadBoolean()
 Me.m_Value = r.ReadDecimal()
 Me.m_SkipZerosIsNull = r.ReadBoolean()
 Me.m_SkipZeros = r.ReadBoolean()
End Sub

' Запись итога в SQL Server
Sub Write(ByVal w As System.IO.BinaryWriter) Implements _
 IBinarySerialize.Write

```

```

 w.Write(Me.m_ValueIsNull)
 w.Write(Me.m_Value)
 w.Write(Me.m_SkipZerosIsNull)
 w.Write(Me.m_SkipZeros)
 End Sub
End Structure
...

// C#:
using System;
using System.Data.SqlTypes;
using System.Runtime.InteropServices;
using Microsoft.SqlServer.Server;

...
[SqlUserDefinedAggregate(Format.UserDefined, IsNullIfEmpty=true,
IsInvariantToDuplicates=false, IsInvariantToNulls=true,
IsInvariantToOrder=true, MaxByteSize=19)]
public struct DecimalProductAggregate : IBinarySerialize
{
 private decimal m_Value; // 16 bytes storage.
 private bool m_ValueIsNull; // 1 byte storage.
 private bool m_SkipZeros; // 1 byte storage.
 private bool m_SkipZerosIsNull; // 1 byte storage.

 public void Init()
 {
 this.m_ValueIsNull = true;
 this.m_SkipZerosIsNull = true;
 }

 public void Accumulate([SqlFacet(Precision = 38,
 Scale = 5)]SqlDecimal value,
 SqlBoolean skipZeros)
 {
 if (skipZeros.IsNull)
 {
 throw new InvalidOperationException(
 "The @SkipZeros parameter cannot be null.");
 }
 if (value.IsNull)
 return;

 // Пропускает флаг Zeros, если он не установлен
 if (this.m_SkipZerosIsNull)
 {
 this.m_SkipZeros = skipZeros.Value;

```

```

 this.m_SkipZerosIsNull = false;
 }
 // Не разрешает менять во время исполнения параметр SkipZeros.
 else if (this.m_SkipZeros != skipZeros.Value)
 {
 throw new InvalidOperationException(
 "The @SkipZeros parameter cannot be changed.");
 }

 // Пропускает нулевые значения. Если параметры говорят об этом
 // и текущее значение равно нулю
 if (this.m_SkipZeros && value.Value == 0M)
 {
 return;
 }

 // Если это первое значение, то просто присваивает его
 if (this.m_ValueIsNull)
 {
 this.m_Value = value.Value;
 this.m_ValueIsNull = false;
 return;
 }

 this.m_Value *= value.Value;
}

public void Merge(DecimalProductAggregate other)
{
 SqlBoolean skipZeros = other.m_SkipZerosIsNull ? SqlBoolean.Null
 : other.m_SkipZeros;
 SqlDecimal otherValue = other.Terminate();

 this.Accumulate(otherValue, skipZeros);
}

[return: SqlFacet(Precision = 38, Scale = 5)]
public SqlDecimal Terminate()
{
 if (this.m_ValueIsNull)
 {
 return SqlDecimal.Null;
 }
 return this.m_Value;
}

// Чтение итога из SQL Server
void IBinarySerialize.Read(System.IO.BinaryReader r)

```

```

 {
 this.m_ValueIsNull = r.ReadBoolean();
 this.m_Value = r.ReadDecimal();
 this.m_SkipZerosIsNull = r.ReadBoolean();
 this.m_SkipZeros = r.ReadBoolean();
 }

 // Запись итога в SQL Server
 void IBinarySerialize.Write(System.IO.BinaryWriter w)
 {
 w.Write(this.m_ValueIsNull);
 w.Write(this.m_Value);
 w.Write(this.m_SkipZerosIsNull);
 w.Write(this.m_SkipZeros);
 }
}
. . .

```

После загрузки сборки в SQL Server вы сможете выполнить следующий код на языке T-SQL, создающий итог, определенный пользователем.

```

CREATE AGGREGATE dbo.PRODUCT_DECIMAL_38_5
(@Input DECIMAL(38,5)
, @SkipZeros BIT
)
RETURNS DECIMAL(38,5)
EXTERNAL NAME TK433ClrDemo."TK433.Clr.DecimalProductAggregate";

```

В данном случае мы назвали итог PRODUCT\_DECIMAL\_38\_5, поскольку он отражает действительный тип, для которого может вычислить произведение. В следующем программном коде на T-SQL можно увидеть, как использовать итог, определенный пользователем, в запросе SQL.

```

WITH Numbers AS (SELECT * FROM (VALUES (0)
 , (5.5)
 , (10.5)
 , (15.5)) AS a(Number))
SELECT dbo.PRODUCT_DECIMAL_38_5(Number, 1) AS ProductExcludingZeros
 , dbo.PRODUCT_DECIMAL_38_5(Number, 0) AS ProductIncludingZeros
FROM Numbers;

```

Далее показан результат.

ProductExcludingZeros	ProductIncludingZeros
895.12500	0.00000

В этом примере запрос выполняется к ОТВ, но итог, определенный пользователем, можно применять и к таблицам.

## Создание типа данных CLR, определенного пользователем

Последний объект CLR, поддерживаемый SQL Server, — тип данных, определенный пользователем (ТОП). Создание ТОП аналогично созданию итогов, определенных пользователем. Тип состоит из типа CLR (`class` или `struct`), который использует атрибут `SqlUserDefinedType` для информирования SQL Server об особенностях своего поведения. В следующем примере вы увидите программный код CLR, который применяется для определения ТОП CLR. Этот тип, названный `CURRENCY_VALUE`, можно использовать для хранения денежных сумм (десятичных чисел) и кода валюты (строк). (Имейте в виду, что этот программный код не был строго оттестирован и его не следует применять в реальном рабочем программном коде.)

Как и в случае итогов, определенных пользователем, приведем несколько замечаний, касающихся этого кода.

- ◆ В этом примере атрибут `SqlUserDefinedType` сообщает SQL Server о том, что будет вручную управлять сериализацией типа, задавая формат, определенный пользователем (`Format.UserDefined`). Атрибут также информирует SQL Server о том, что должен выполняться метод, названный `Validate`, для проверки целостности типа при каждом преобразовании двоичного значения в данный тип (`ValidationMethodName`), и о том, что тип побайтно неупорядочен (`not byte-ordered`) (`IsByteOrdered = False`). Поскольку единственная сортировка, поддерживаемая SQL Server для ТОП, — это побайтная сортировка, вы должны быть очень внимательны при ее реализации. Помните о том, что при сравнении байтовых структур целочисленное значение `-1` больше чем `1`, т. к. самый старший бит в целочисленном значении равен `1`, если значение отрицательное, и `0`, если оно положительное. Два других задаваемых параметра — это `IsFixedLength`, сообщающий SQL Server о том, всегда ли этот тип использует одно и то же количество байтов для хранения, и `MaxByteSize`, который, как и в случае итогов, определенных пользователем, извещает SQL Server о максимальном количестве байтов, необходимых для хранения сериализованного экземпляра данного типа.
- ◆ Атрибут `SqlFacet` применяется в программном коде несколько раз для задания разрядности и точности типа `SqlDecimal`, а также максимальной длины типа `SqlString`.
- ◆ Атрибут `SqlMethod` применяется в двух местах для указания на то, что свойства `CurrencyCode` и `Value` детерминированы и точны; следовательно, свойство может индексироваться и сохраняться.
- ◆ Метод `Parse` применяется для преобразования строки в ТОП. Он автоматически вызывается SQL Server при явном и неявном преобразованиях строки в данный тип.

- ◆ Метод `ToString` используется SQL Server для преобразования ТОП в строку. Имейте в виду, что очень удобно в методах `Parse` и `ToString` применять одно и то же строковое представление типа.
- ◆ Поскольку при определении типа мы применили формат, определенный пользователем, необходимо использовать интерфейс `IBinarySerialize`. Вы можете выбрать использование собственного формата системы, но в данном примере он не сработает, потому что в реализованной в системе сериализации не поддерживаются десятичный и строковый типы из данного примера.
- ◆ Для сериализации типа SQL Server вызывает метод `IBinarySerialize.Write`.
- ◆ Для десериализации типа SQL Server вызывает метод `IBinarySerialize.Read`.

```
' VB:
Imports System
Imports System.Data.SqlTypes
Imports System.Runtime.InteropServices
Imports Microsoft.SqlServer.Server

...
<SqlUserDefinedType(Format.UserDefined, _
 ValidationMethodName:="Validate", _
 IsByteOrdered:=False, IsFixedLength:=True, MaxByteSize:=20)> _
Public Structure CurrencyValueType
 Implements IBinarySerialize, INullable

 Private m_Value As Decimal ' 16 bytes storage.
 Private m_CurrencyCode As String ' 4 bytes storage.
 Private m_IsNull As Boolean

 Public Sub New(ByVal value As Decimal, ByVal currencyCode As String)
 Me.m_Value = value
 Me.m_CurrencyCode = currencyCode.ToUpper()
 Me.m_IsNull = False
 End Sub

 ' Получение экземпляра null типа CurrencyValueType
 Public Shared ReadOnly Property Null() As CurrencyValueType
 Get
 Dim currValue As CurrencyValueType = New CurrencyValueType()
 currValue.m_IsNull = True
 Return currValue
 End Get
 End Property

 <SqlFacet(MaxSize:=3)> _
 Public Property CurrencyCode() As SqlString
```



```

 <SqlMethod(IsPrecise:=True, IsDeterministic:=True)> _
 Get
 If Me.m_IsNull Then
 Return SqlString.Null
 End If
 Return Me.m_CurrencyCode
 End Get
 Set(ByVal value As SqlString)
 Me.m_CurrencyCode = value.Value.ToUpper()
 If Me.Validate() = False Then
 Throw New InvalidOperationException(_
 "The currency code is invalid.")
 End If
 End Set
End Property

<SqlFacet(Precision:=38, Scale:=5)> _
Public Property Value() As SqlDecimal
 <SqlMethod(IsPrecise:=True, IsDeterministic:=True)> _
 Get
 If Me.m_IsNull Then
 Return SqlDecimal.Null
 End If
 Return Me.m_Value
 End Get
 Set(ByVal value As SqlDecimal)
 Me.m_Value = value.Value
 End Set
End Property

' Вызывается SQL Server для проверки денежной суммы
Public Function Validate() As Boolean
 Return System.Text.RegularExpressions.Regex.IsMatch(_
 Me.m_CurrencyCode, "^[A-Z]{3}$")
End Function

' Преобразует строку в денежную сумму
Public Shared Function Parse(ByVal input As SqlString) As _
 CurrencyValueType
 If input.IsNull Then
 Return CurrencyValueType.Null
 End If

 Dim space As Integer = input.Value.IndexOf(" ")
 If space <> 3 Then
 Throw New InvalidOperationException(_
 "The input string cannot be converted to a currency value.")
 End If

```

```

 Dim currencyCode As String = input.Value.Substring(0, 3)
 Dim value As Decimal = SqlDecimal.Parse(input.Value.Substring(_
 4, input.Value.Length - 4)).Value

 Return New CurrencyValueType(value, currencyCode)
End Function

' Преобразует денежную сумму в строку
Public Overrides Function ToString() As String
 If Me.m_IsNull Then
 Return Nothing
 End If
 Return String.Format("{0} {1}", Me.CurrencyCode.Value, _
 Me.Value.ToString())
End Function

' Считывает тип из SQL Server
Public Sub Read(ByVal r As System.IO.BinaryReader) _
 Implements IBinarySerialize.Read
 Me.m_Value = r.ReadDecimal()
 Me.m_CurrencyCode = r.ReadString()
 Me.m_IsNull = False
End Sub

' Записывает тип в SQL Server
Public Sub Write(ByVal w As System.IO.BinaryWriter) _
 Implements IBinarySerialize.Write
 w.Write(Me.m_Value)
 w.Write(Me.m_CurrencyCode)
End Sub

Public ReadOnly Property IsNull() As Boolean Implements _
 INullable.IsNull
 Get
 Return Me.m_IsNull
 End Get
End Property
End Structure
. . .

// C#:
using System;
using System.Data.SqlTypes;
using System.Runtime.InteropServices;
using Microsoft.SqlServer.Server;
. . .

```

```

[SqlUserDefinedType(Format.UserDefined, ValidationMethodName="Validate",
IsByteOrdered=false, IsFixedLength=true, MaxByteSize=20)]
public struct CurrencyValueType : IBinarySerialize, INullable
{
 private decimal m_Value; // 16 bytes storage.
 private string m_CurrencyCode; // 4 bytes storage.
 private bool m_IsNull; // Not stored...

 public CurrencyValueType(decimal value, string currencyCode)
 {
 this.m_Value = value;
 this.m_CurrencyCode = currencyCode.ToUpper();
 this.m_IsNull = false;
 }

 // Получает экземпляр null типа CurrencyValueType
 static public CurrencyValueType Null
 {
 get
 {
 return new CurrencyValueType() { m_IsNull = true };
 }
 }

 [SqlFacet(MaxSize = 3)]
 public SqlString CurrencyCode
 {
 [SqlMethod(IsPrecise = true, IsDeterministic = true)]
 get
 {
 if (this.m_IsNull) return SqlString.Null;
 return this.m_CurrencyCode;
 }
 set
 {
 this.m_CurrencyCode = value.Value.ToUpper();
 if (!this.Validate())
 {
 throw new InvalidOperationException("The currency code is
 invalid.");
 }
 }
 }
}

[SqlFacet(Precision = 38, Scale = 5)]
public SqlDecimal Value
{
 [SqlMethod(IsPrecise=true, IsDeterministic=true)]

```

```
get
{
 if (this.m_IsNull) return SqlDecimal.Null;
 return this.m_Value;
}
set
{
 this.m_Value = value.Value;
}
}

// Вызывается SQL Server для проверки денежной суммы
private bool Validate()
{
 return System.Text.RegularExpressions.Regex.IsMatch(
 this.m_CurrencyCode, "[A-Z]{3}$");
}

// Преобразует строку в денежную сумму.
static public CurrencyValueType Parse(SqlString input)
{
 if (input.IsNull) return CurrencyValueType.Null;
 int space = input.Value.IndexOf(' ');
 if (space != 3)
 throw new InvalidOperationException(
 "The input string cannot be converted to a currency value.");

 string currencyCode = input.Value.Substring(0, 3);
 decimal value = SqlDecimal.Parse(input.Value.Substring(
 4, input.Value.Length - 4)).Value;
 return new CurrencyValueType(value, currencyCode);
}

// Преобразует денежную сумму в строку
override public string ToString()
{
 if (this.m_IsNull) return null;
 return string.Format("{0} {1}", this.CurrencyCode.Value,
 this.Value.ToString());
}

// Считывает тип из SQL Server
void IBinarySerialize.Read(System.IO.BinaryReader r)
{
 this.m_Value = r.ReadDecimal();
 this.m_CurrencyCode = r.ReadString();
 this.m_IsNull = false;
}
```

```
// Записывает тип в SQL Server
void IBinarySerialize.Write(System.IO.BinaryWriter w)
{
 w.Write(this.m_Value);
 w.Write(this.m_CurrencyCode);
}
bool INullable.IsNull
{
 get
 {
 return this.m_IsNull;
 }
}
...

```

После загрузки сборки в SQL Server вы можете выполнить следующий программный код на T-SQL для создания ТОП.

```
CREATE TYPE dbo.CURRENCY_VALUE
EXTERNAL NAME TK433ClrDemo."TK433.Clr.CurrencyValueType";
```

В показанном далее примере на языке T-SQL создается новая таблица и заполняется строками из таблицы `Production.Product` (Товар). При вставке строк в новую таблицу мы преобразуем цены из таблицы `Product` в тип данных `CURRENCY_VALUE`. Также отметьте, что для чтения и записи данных можно применять общедоступные (`public`) свойства типа.

```
CREATE TABLE Production.TestProducts
(
 Name NVARCHAR(100) NULL
 , Price dbo.CURRENCY_VALUE NULL
);

INSERT Production.TestProducts (Name, Price)
SELECT Name
 , CAST('SEK ' + CAST(ListPrice AS NVARCHAR(100)) AS
 dbo.CURRENCY_VALUE)
FROM Production.Product;

UPDATE Production.TestProducts SET
 Price.CurrencyCode = 'USD'
WHERE Price.Value = 3578.27000;

SELECT Name, Price.CurrencyCode, Price.Value, Price
FROM Production.TestProducts
WHERE Price.CurrencyCode = 'USD';
```



Интересно отметить, что оптимизатор обнаруживает существование индекса `ComputedCurrencyCodeIdx` для выражения, использованного в последнем запросе, и применяет индекс, даже если мы не запрашиваем непосредственно вычисляемый столбец, как в следующем примере (изменения выделены жирным шрифтом).

```
SELECT COUNT(*)
FROM Production.TestProducts
WHERE Price.CurrencyCode = 'SEK';
-- SQL Server выполняет операцию поиска индекса, используя
-- индекс ComputedCurrencyCodeIdx
```

## Что разрешено делать моему коду CLR?

Программный код CLR, используемый в вашей базе данных может быть помещен в три разных набора разрешений (permission sets): `SAFE`, `EXTERNALACCESS` и `UNSAFE`. По умолчанию устанавливается набор разрешений `SAFE`. Этот набор разрешений использовался во всех примерах данного урока, и именно его вы должны стараться применять всегда, чтобы минимизировать проблемы безопасности и возможное воздействие ошибок. Далее приведено описание всех наборов разрешений.

- ◆ `SAFE`. Сборке `SAFE` не разрешен доступ к любым ресурсам вне базы данных, для которой она применяется; она может только выполнять вычисления и обращаться к локальной базе данных.
- ◆ `EXTERNALACCESS`. Сборке `EXTERNAL ACCESS` разрешен доступ к ресурсам вне локального экземпляра SQL Server, в котором она применяется, например, к ресурсам другого экземпляра SQL Server, файловой системе или даже сетевому ресурсу, такому как Web-сервис.
- ◆ `UNSAFE`. Сборке `UNSAFE` разрешено идти еще дальше, чем сборке `EXTERNAL ACCESS`. Ей разрешается выполнять код не CLR (или неуправляемый код), например Win32 API или компонент COM.

## Применение *Filestream*

Параметр `Filestream` может быть задан для столбцов с типом данных `varbinary(max)`. По существу он заставляет SQL Server сохранять данные из таких столбцов в отдельных файлах файловой системы, а не в реальных файлах базы данных. Применение `Filestream` может существенно повысить производительность чтения и записи данных указанного типа.

Применять параметр `Filestream` рекомендуется, если данные, которые вы храните в столбце, имеют размер не менее 1 Мбайт. `Filestream` может снизить производительность при частых вставках небольших порций данных типа BLOB.

Для того чтобы иметь возможность применять параметр `Filestream`, необходимо активизировать его в SQL Server, установив значение параметра конфигурации `sp_configure`, равным 1, 2 или 3. Параметр, равный 1, разрешает только доступ на языке T-SQL к данным `Filestream`, значение 2 допускает прямой файловый доступ к данным из файловой системы. И, наконец, параметр, равный 3, разрешает совместный файловый (сетевой) доступ к данным `Filestream`. Для применения параметра `Filestream` вы должны также сформировать файловую группу, содержащую файл базы данных `Filestream`. Файл базы данных `Filestream` на самом деле не файл, а каталог, в котором хранятся файлы данных `Filestream`. Файловая группа `Filestream` может содержать только один файл базы данных `Filestream`.

В следующем примере программного кода показано, как добавить файловую группу `Filestream` в базу данных `AdventureWorks` (строка `FILENAME` отформатирована в соответствии с размером печатной страницы).

```
ALTER DATABASE AdventureWorks
 ADD FILEGROUP FileStreamPhotosFG
 CONTAINS FILESTREAM;

ALTER DATABASE AdventureWorks
 ADD FILE
 (
 NAME = 'FileStreamPhotosDF'
 , FILENAME = 'C:\Program Files\Microsoft SQL Server\
 MSSQL10.MSSQLSERVER\MSSQL\DATA\FileStreamPhotosDF'
)
 TO FILEGROUP FileStreamPhotosFG;
```

После того как файловая группа добавлена, могут создаваться столбцы `Filestream` типа `varbinary(max)`. Для того чтобы таблица могла включать в себя столбцы `Filestream`, она должна иметь столбец `uniqueidentifier`, помеченный с помощью свойства `ROWGUIDCOL`, и ограничение `UNIQUE`, определенное для этого столбца. В следующем примере программного кода показано, как добавить свойство `ROWGUIDCOL` в таблицу `Production.ProductPhoto`.

```
ALTER TABLE Production.ProductPhoto
 ADD RowGuid UNIQUEIDENTIFIER NOT NULL
 ROWGUIDCOL
 CONSTRAINT DFProductPhotoRowGuid DEFAULT NEWSEQUENTIALID()
 CONSTRAINT UQProductPhotoRowGuid UNIQUE;
```

Теперь можно вставить столбец `Filestream` с типом данных `varbinary(max)` и скопировать в него данные обычными средствами T-SQL.

```
ALTER TABLE Production.ProductPhoto
 ADD ThumbNailPhotoAsFileStream VARBINARY(MAX) FILESTREAM NULL;
GO

UPDATE Production.ProductPhoto SET
 ThumbNailPhotoAsFileStream = ThumbNailPhoto;
```



Если вы просмотрите содержимое папки C:\Program Files\Microsoft SQL Server\MSSQL10.MSSQLSERVER\MSSQL\DATA\FileStreamPhotosDF, то найдете в ней следующие элементы:

- ◆ каталог \$FSLOG действует как журнал транзакций данных FileStream;
- ◆ файл FileStream.hdr хранит метаданные о файловой группе FileStream;
- ◆ все остальные каталоги с менами GUID, такими как 09A42544-450A-4932-B25F-5E33F117C179, хранят реальные данные.

Когда вы удаляете данные FileStream (с помощью инструкции UPDATE или DELETE), SQL Server не удаляет файл немедленно. Файлы удаляются, когда запускается процесс сбора мусора (garbage collection) FileStream. В свою очередь этот процесс запускается, когда выполняется процесс сохранения базы данных (database checkpoint).

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. SQLCLR

На этом практическом занятии вы создадите скалярную и возвращающую табличное значение функции CLR, определенные пользователем.

### Упражнение 1. Создание скалярной ФОР

В этом упражнении вы создадите скалярную ФОР, позволяющую преобразовать значение типа datetime2 в строку с применением строки format (например, уууу-MM-dd).

1. Создайте в корневом каталоге вашего жесткого диска каталог TK433SQLCLR (путь будет C:\TK433SQLCLR).
2. Создайте в каталоге C:\TK433SQLCLR файл ScalarUDF.cs (если хотите создавать функцию на языке C#) или ScalarUDF.vb (если хотите использовать язык Visual Basic .NET).
3. Введите в файл следующий блок программного кода, используя текстовый редактор, такой как Блокнот (Notepad.exe), сохраните и закройте файл.

```
' VB:
Imports System
Imports System.Data.SqlTypes

Namespace TK433.ClrLab
 Public Class ScalarUdf
 Public Shared Function DateTimeToString(_
 ByVal dateTime As SqlDateTime, ByVal format As SqlString) _
 As SqlString

 If dateTime.IsNull Or format.IsNull Then
 Return SqlString.Null
 End If
```

```

 Return dateTime.Value.ToString(format.Value)
 End Function
End Class
End Namespace

// C#:
using System;
using System.Data.SqlTypes;

namespace TK433.ClrLab
{
 public class ScalarUdf
 {
 static public SqlString DateTimeToString(SqlDateTime dateTime,
 SqlString format)
 {
 if(dateTime.IsNull || format.IsNull) return SqlString.Null;
 return dateTime.Value.ToString(format.Value);
 }
 }
}

```

4. Откройте новое окно командной строки и введите следующие команды для указания с помощью установки переменной окружения `PATH` каталога, содержащего компиляторы CLR, и изменения папки, в которой вы сохранили свой исходный программный код.

```

PATH "C:\Windows\Microsoft.NET\Framework\v3.5"
C:
CD \TK433SQLCLR

```

5. В том же окне командной строки введите следующую команду для компиляции сборки.

```

// C#:
csc /target:library /out:ScalarUDF.dll ScalarUDF.cs

' VB:
vbc /target:library /out:ScalarUDF.dll ScalarUDF.vb

```

6. Если необходимо, откройте SSMS и установите соединение с подходящим экземпляром SQL Server 2008.
7. В окне создания запроса введите и выполните следующие инструкции SQL, создающие сборку `ScalarUDF` в базе данных `AdventureWorks`.

```

USE AdventureWorks;
GO

CREATE ASSEMBLY ScalarUDF FROM 'C:\TK433SQLCLR\ScalarUDF.dll';

```

8. В имеющемся окне запроса введите, выделите и выполните следующие инструкции SQL для создания ФОП `fnDateTimeToString`.

```
CREATE FUNCTION dbo.fnDateTimeToString
(
 @DateTime DATETIME
 , @Format NVARCHAR(50)
)
RETURNS NVARCHAR(50)
AS
EXTERNAL NAME ScalarUDF."TK433.ClrLab.ScalarUdf".DateTimeToString;
```

9. В том же окне запроса введите, выделите и выполните следующую инструкцию SELECT, выполняющую ФОП `fnDateTimeToString`.

```
SELECT TOP(5) CustomerID,
 dbo.fnDateTimeToString(OrderDate, N'yyyy_MM_dd')
FROM Sales.SalesOrderHeader;
```

Результат должен выглядеть примерно так:

```
CustomerID

676 2001_07_01
117 2001_07_01
442 2001_07_01
227 2001_07_01
510 2001_07_01
```

## Упражнение 2. Создание ФОП, возвращающей табличное значение

В этом упражнении вы создадите табличную ФОП, таблицу чисел. Поскольку язык Visual Basic .NET потребует включения дополнительного программного кода, предлагается решение только на языке C#.

1. В каталоге `C:\TK433SQLCLR`, созданном в упражнении 1, создайте новый файл `TableValuedUDF.cs`.
2. Введите в файл с помощью текстового редактора, например, Блокнот (`Notepad.exe`) следующий блок программного кода, а затем сохраните и закройте файл.

```
// c#
using System;
using System.Collections;
using System.Data.SqlTypes;
using Microsoft.SqlServer.Server;

namespace TK433.ClrLab
{
 public class TableValuedUdf
```

```

{
 [SqlFunction(FillRowMethodName = "GetNumbersTableFillRow",
 TableDefinition = "Number INT")]
 static public IEnumerable GetNumbersTable(
 SqlInt32 startNumber, SqlInt32 endNumber)
 {
 for (int number = startNumber.Value; number <= endNumber.Value;
 ++number)
 {
 yield return number;
 }
 }

 static private void GetNumbersTableFillRow(
 object value, out SqlInt32 number)
 {
 number = (int)value;
 }
}

```

- Откройте окно командной строки и введите следующие команды для указания с помощью переменной окружения `PATH` каталога, содержащего компилятора CLR, и изменения папки, в которой вы сохранили исходный программный код.

```

PATH "C:\Windows\Microsoft.NET\Framework\v3.5"
C:
CD \TK433SQLCLR

```

- В том же окне командной строки введите следующую команду для компиляции сборки.

```
csc /target:library /out:TableValuedUDF.dll TableValuedUDF.cs
```

- Если необходимо, откройте SSMS и установите соединение с подходящим экземпляром SQL Server 2008.

- В окне создания запроса введите и выполните следующие инструкции SQL, создающие сборку TableValuedUDF в базе данных AdventureWorks.

```

USE AdventureWorks;
GO

CREATE ASSEMBLY TableValuedUDF
FROM 'C:\TK433SQLCLR\TableValuedUDF.dll';

```

- В имеющемся окне запроса введите, выделите и выполните следующие инструкции SQL для создания ФОП TableValuedUDF.

```
CREATE FUNCTION dbo.fnGetNumbersTable
(
 @StartNumber INT
 , @EndNumber INT
)
RETURNS TABLE (Number INT)
AS
EXTERNAL NAME
 TableValuedUDF."TK433.ClrLab.TableValuedUdf".GetNumbersTable;
```

8. Для выполнения ФОП TableValuedUDF в имеющемся окне запроса введите, выделите и выполните следующую инструкцию SELECT.

```
SELECT *
FROM dbo.fnGetNumbersTable(501, 505) AS n;
```

Результат должен выглядеть примерно так.

```
Number

501
502
503
504
505
```

## Резюме урока

- ◆ Для применения объектов среды SQLCLR ее необходимо активизировать в экземпляре SQL Server.
- ◆ Объекты, наиболее подходящие для разработки средствами SQLCLR, — ФОП и определенные пользователем итоги.
- ◆ Создавая ТОП средствами SQLCLR, убедитесь в том, что они прошли строгое тестирование.
- ◆ Рассматривайте возможность применения `FileStream`, если данные в основном включают потоки размером более 1 Мбайт.

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить знания и навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.

- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ Рассмотрение возможности применения XML в качестве транспортного протокола между вашим приложением (или частями вашего приложения) и SQL Server — подходящее решение ваших бизнес-проблем.
- ◆ Дважды подумайте, прежде чем сохранять в базе данных данные в виде XML, делайте это только в том случае, если сохранение этих значений в табличной форме невозможно или очень трудоемко.
- ◆ Рассмотрите все "за" и "против" применения коллекций схем XML для проверки ваших данных XML. Помните о том, проверка с помощью схемы не всегда выгодна.
- ◆ Прежде чем использовать среду SQLCLR, убедитесь в том, что она разрешена в экземпляре SQL Server, используемом вашим приложением.
- ◆ Дважды подумайте о применении SQLCLR. Как и в случае XML, используйте ее, только если это верный способ удовлетворить ваши бизнес-потребности; например, если производительность будет значительно выше по сравнению с T-SQL или если это облегчит написание и сопровождение вашего программного кода.
- ◆ Помните о том, что использование объектов SQLCLR сопряжено с дополнительной работой (установкой сборок и т. д.).

## Ключевые термины

- ◆ XML.
- ◆ Документ XML.
- ◆ Фрагмент XML.
- ◆ FOR XML.
- ◆ Тип данных XML.
- ◆ SQLCLR.
- ◆ Filestream.

## Сценарий

В следующем конкретном сценарии вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в разд. "Ответы к сценариям" в конце книги.

## Сценарий. Как следует хранить данные?

Вы — разработчик базы данных для корпорации Contoso News. Вам предоставлены полномочия по проектированию хранения новостных статей, написанных репортерами организации. У статьи всегда есть заголовок, но ее содержимое может состоять из нескольких абзацев, таблиц, подзаголовков и перечней. Статья всегда должна быть отнесена к одной из заранее заданных категорий.

Ответьте на следующий вопрос вашего менеджера: "Как вы будете проектировать структуру хранения этих данных? Иными словами, какие таблицы вам понадобятся, и какие столбцы необходимо включить в каждую таблицу?"

## КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

### Создайте и примените объекты на базе CLR

**Упражнение 1.** Создайте таблицы, предполагаемые в ответе предложенного вам сценария, и заполните их как минимум пятью статьями.

### Извлечение реляционных данных в виде XML

**Упражнение 2.** Создайте коллекцию схем XML, которая проверяет допустимость содержимого столбца XML из упражнения 1.

**Упражнение 3.** Примените коллекцию схем XML из упражнения 2 к содержимому столбца.

### Преобразование данных XML в реляционные данные

**Упражнение 4.** Измените схему, использовавшуюся коллекцией схем XML в упражнении 3, например, с помощью модификации имени одного из определенных элементов.

### Запрос данных XML

**Упражнение 5.** С помощью SQLCLR создайте ТОП, который может хранить информацию о клиенте, включая его фамилию, имя, отчество, номер телефона и адрес.

**Упражнение 6.** Используйте тип данных, созданный в упражнении 5, для создания столбца в таблице и затем заполните его данными нескольких клиентов.

### Управление данными XML

**Упражнение 7.** Измените определение ТОП для вашего клиента, создав в таблице столбец для хранения адреса. Переместите адрес из ТОП в новый столбец,

удалите свойство `address` из ТОП и заново включите ТОП в экземпляр SQL Server.

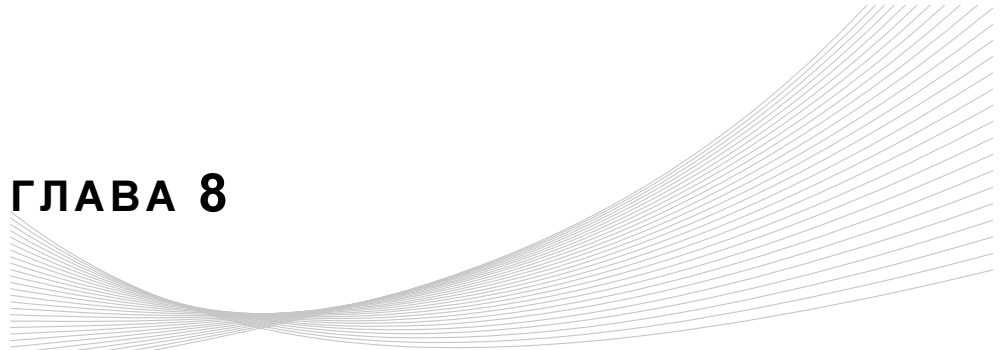
## Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобию будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### **Дополнительная информация о тренировочных тестах**

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.





## ГЛАВА 8

# Расширение функциональных возможностей Microsoft SQL Server с помощью пространственных типов, полнотекстового поиска и Service Broker

Благодаря новым пространственным типам данных, включенным в версию Microsoft SQL Server 2008, вы можете хранить данные о местоположении, которые позволяют работать с информацией, определяющей геометрическое и географическое местонахождение. Типы данных `geography` и `geometry` используются для учета обычных геометрических координат  $X$ ,  $Y$  и  $Z$  и сферических географических координат, таких как широта и долгота<sup>1</sup>. Компании могут применять эти данные для отслеживания регионов продаж, определения маршрута от склада до магазина, задания местоположения классной комнаты в школе и поиска расстояния до ближайшего эвакуационного выхода при пожаре.

Как разработчик базы данных, вы можете включать в свои базы типы данных особого назначения, например, `varchar(max)`, XML, `geometry` или `geography`. SQL Server 2008 предоставляет вам средства для эффективной работы с такими данными. Средства полнотекстового поиска позволяют выполнять сложный лингвистический анализ данных, оставляющий далеко позади скромные возможности предложения `LIKE`. Кроме того, такой поиск может проводиться в столбцах с типом данных, например, `varbinary(max)`, которые нельзя задать как ключевые в стандартных индексах.

---

<sup>1</sup> Из-за отклонения формы Земли (эллипсоида) от шара географические координаты точек несколько отличаются от сферических. — *Пер.*

Компонент SQL Server Service Broker предоставляет возможность построения приложений, эффективно использующих асинхронную доставку сообщений локальным и удаленным службам. Service Broker применяет комбинацию объектов, таких как очереди, диалоги, контракты, службы и сообщения разных типов для разработки решений на базе асинхронной доставки сообщений. Компания может использовать Service Broker для разработки ряда приложений, автоматизирующих погашения командировочных расходов своих сотрудников. После заполнения сотрудниками финансового отчета о командировке информация отправляется в очередь, в которой сохраняется до ночной пакетной обработки отчетов. Затем отчеты о расходах, удовлетворяющие всем требованиям, пересылаются в приложение для оплаты. Неудовлетворительные отчеты помечаются и затем через другую службу направляются в очередь ожидания ручной обработки менеджерам.

### Экзаменационные темы в этой главе

- ◆ Реализация типов данных.
- ◆ Выполнение полнотекстового поиска.
- ◆ Разработка решений Service Broker.

### Уроки в этой главе

- ◆ Урок 1. Реализация пространственных типов данных.
- ◆ Урок 2. Выполнение полнотекстового поиска.
- ◆ Урок 3. Разработка решений Service Broker.

## Прежде чем начать

Для усвоения уроков этой главы необходимо:

- ◆ иметь базовые знания о типах данных в SQL Server;
- ◆ иметь общее представление о типах данных общезыковой среды исполнения (CLR);
- ◆ разбираться в базовых конструкциях языка манипулирования данными (DML), таких как `INSERT`, `UPDATE` и `DELETE`;
- ◆ иметь общее представление о синтаксисе инструкции `SELECT`;
- ◆ знать основы описания в программах структур языка определения данных (DDL);
- ◆ знать о том, как открывать и выполнять запросы в среде SQL Server Management Studio (SSMS);
- ◆ установить на компьютере SQL Server 2008 версию Developer Edition, Enterprise Edition или Enterprise Evaluation Edition с учебной базой данных AdventureWorks2008.

**РЕАЛЬНАЯ ИСТОРИЯ.****Майк Хотек (Mike Hotek)**

У моего клиента, имеющего большой Web-сайт розничной торговли, возникла серьезная проблема при обработке кредитных карт. Его Web-сайт требовал, чтобы с кредитной карты клиента были сняты деньги до того, как завершилось оформление заказа. После того как заказ сделан, можно списывать заказанное количество из имеющегося запаса.

Обработка кредитной карты — очень сложный многокомпонентный процесс. Кредитные карты принимаются на Web-сайте, шифруются и по защищенным каналам направляются на шлюз приема платежей (payment Gateway). Интернет-шлюз перенаправляет запрос на свой процессор обработки платежей (merchant processor). Этот процессор может направить запрос сборщику платежей (payment aggregator) или банку-посреднику. Затем запрос пересылается компании, выдавшей кредитную карту. Компания подставляет счет, выполняет проверку подлинности и баланса счета и возвращает подтверждение или отказ.

Если на пути между Web-сайтом и компанией, выдавшей кредитную карту, возникли сбои, расчеты не могут быть произведены. У платежной системы есть также механизм учета времени, который пытается помешать несанкционированному проникновению и другим мошенничествам, автоматически отклоняя платежный запрос, если ответ не получен в течение определенного промежутка времени. Если какая-либо система между получателем платежей и компанией, выдавшей кредитную карту, переполнена запросами и не может ответить в течение должного времени, платежи не могут быть обработаны.

Одна или несколько систем между Web-сайтом и компанией, выдавшей кредитную карту, часто были недоступны, приводя тем самым к большому числу сбоев в работе. Несмотря на то, что все прерывания, как правило, длились менее часа, любые проблемы приводили к существенным потерям доходов компании. Решить проблему перебоев в работе можно было очень легко, обрабатывая платежи в отключенном от сети автономном режиме и разрешая завершать оформление заказов. К сожалению, мой клиент работал с очень ограниченными запасами товаров, и спрос почти всегда превышал предложение, отсюда и требование получить платеж прежде, чем разрешить выполнение заказа и уменьшение запаса.

Мы решили проблему сбоев в работе, стоивших компании миллионов долларов потерь, реализовав решение средствами Service Broker. Приложение Service Broker позволило при необходимости переводить обработку платежей в фоновый режим. Сначала делалась попытка обработать заказы в обычном режиме, но если из-за сбоев кредитная карта не могла быть обработана обычными методами, заказ помещался в очередь Service Broker, и клиенту разрешалось завершить свой заказ при условии подтверждения платежа. Затем приложение Service Broker повторно обращалось к системе до тех пор, пока не появлялась возможность обработать платеж. Для обеспечения целостности запасов после помещения заказа в очередь Service Broker все последующие заказы также помещались в очередь, гарантируя доступ к имеющимся запасам по принципу: первым поступил, первым обслужен. Только если очередь была пуста, обработка заказов возвращалась к обработке платежей в интерактивном режиме.

Реализация решения средствами Service Broker сэкономила компании сотни тысяч долларов, необходимых для специализированных разработок и приложений сторонних разработчиков. Решение на основе Service Broker также устранило все сбои, вызванные системой обработки платежей, тем самым обеспечив более 20 млн долларов чистой прибыли.

## УРОК 1. Реализация пространственных типов данных

В SQL Server 2008 включено множество новых типов данных, включая два специальных пространственных типа, позволяющих управлять географическими и геометрическими данными. Применяя указанные типы данных, вы можете хранить информацию о местоположении, такую как широта и долгота, которые можно использовать в сочетании с Microsoft Virtual Earth для визуального представления ваших геопространственных данных.

---

### ***После усвоения этого урока вы сможете***

- Описать функциональные возможности типов данных *geometry* и *geography*.
- Создавать экземпляры пространственных типов данных.

Расчетное время урока — 60 мин.

---

### ***Дополнительная информация о других типах данных***

Дополнительную информацию о типах данных SQL Server и о том, как они действуют, см. в главе 3.

## Общие представления о пространственных типах данных

В SQL Server 2008 включены два новых типа данных для работы с пространственными данными. Для этой поддержки предлагаются типы данных *geometry* и *geography*, реализованные в SQL Server 2008 как типы .NET CLR. Оба типа заранее определены и доступны во всех базах данных на вашем сервере как системные объекты, не требующие перед использованием никакой дополнительной настройки.

### ***РЕАЛЬНАЯ ИСТОРИЯ.***

#### ***Майк Хотек (Mike Hotek)***

Один из наших клиентов, большая инженерная фирма, специализирующаяся на нестандартном механическом оборудовании, нуждалась в упрощении цепочки поставщиков и расчета цены изделия для своих заказчиков. Новые заявки на оборудование сначала направлялись в конструкторский отдел, где команда инженеров создавала подробные спецификации для каждого компонента. После разработки спецификаций научно-исследовательская группа искала в сотнях электронных каталогов любые компоненты, которые можно купить готовыми. Ненайденные компоненты направлялись в виде заявок производителем, изготавливающим оборудование на заказ. Весь процесс расчета цены для заказчика мог занять до трех месяцев, и более 60% компонентов изготовлялось на заказ. Уменьшив количество компонентов, требующих специального изготовления, фирма могла бы снизить общие расходы своих клиентов и сократить время поставки.

Для усовершенствования предлагаемых заказчикам решений была разработана новая система, применяющая тип данных *geometry*. Конструкторский отдел создал

подробные спецификации для каждого компонента оборудования. После этого система автоматизированного проектирования (CAD) сгенерировала двумерные сечения трехмерных компонентов с шагом 1 мм по высоте. Каталоги всех их поставщиков были импортированы в базу данных SQL Server. Все компоненты также были смоделированы с помощью двумерных сечений, сделанных с шагом 1 мм по высоте компонента. После того как конструкторский отдел разработал все проектные спецификации, мог выполняться пространственный запрос, находящий все имеющиеся в запасе компоненты, подходящие по размерам. Запрос проектировался с 10-процентным показателем ошибки, позволяющим конструкторской группе решать, можно ли незначительно изменить компонент, чтобы он соответствовал имеющемуся в запасе компоненту, тем самым исключая необходимость изготовления компонента на заказ.

После реализации эта система повысила степень совпадений, и фирма смогла покупать более 70% необходимых компонентов в готовом виде. Время ответа заказчикам на их заявку сократилось в среднем до 6 недель. Но самое значительное достижение касалось снижения стоимости, фирма смогла снизить общую стоимость оборудования на 40% и прогнозировала сокращение времени сборки почти на 50%, тем самым сберегая заказчикам в среднем 6 млн долларов на единицу оборудования.

## **Терминология, относящаяся к пространственным типам данных**

При работе с пространственными данными необходимо знать следующие термины.

- ◆ *Тип данных geography*. Тип данных, применяемый для хранения сферических данных, таких как географические широта и долгота. Virtual Earth, многие картографические программы и результаты переписи обычно используют географические координаты.
- ◆ *Тип данных geometry*. Применяется для хранения двух или трехмерных координат. Например, можно использовать тип данных *geometry* для определения местонахождения различных товаров в хранилище.

### ***Дополнительная информация о сравнении типов пространственных данных***

Дополнительную информацию об отличиях двух типов пространственных данных, а также о влиянии на каждый тип единиц измерения и ориентации см. в разд. "Types of Spatial Data" (Типы пространственных данных) электронной документации по SQL Server.

- ◆ *Открытый геопространственный консорциум* (Open Geospatial Consortium, OGC). Международная некоммерческая организация, разрабатывающая стандарты для географических и основанных на местоположении сервисов.

### ***Дополнительная информация о стандартах организации OGC***

Более подробную информацию об Open Geospatial Consortium (OGC) и разработанных этой организацией стандартах см. на Web-сайте <http://www.opengeospatial.org/>.

- ◆ *Well-known text (WKT)*. Стандарт, созданный OGC, применяется для текстового отображения географических объектов.
- ◆ *Well-known binary (WKB)*. Двоичный эквивалент WKT, иногда применяемый для преобразования и хранения географических данных.
- ◆ *Методы*. Действия, которые способен выполнять объект.
- ◆ *OGC и расширенные методы*. Методы, разработанные для работы с типами данных `geometry` и `geography`.
- ◆ *Объект*. Коллекция свойств и методов, обеспечивающая определенные функциональные возможности.
- ◆ *Создание экземпляра*. Процесс формирования конкретного экземпляра объекта, основанный на свойствах и методах объекта.
- ◆ *Идентификатор пространственной ссылки (Spatial Reference Identifier, SRID)*. Идентификатор ссылки, связанный с конкретной моделью Земли. Эти идентификаторы ссылаются на стандарт системы идентификации Европейской группы нефтепоисковых исследований (European Petroleum Survey Group, EPSG) Например, SRID 4326, стандартный географический SRID, соответствует стандарту WGS 84. Поскольку типы данных `geometry` могут существовать на неопределенной плоскости, стандартный SRID типа `geometry` равен 0 и представляет неопределенную плоскость.

Типы данных `geometry` и `geography` поддерживают перечисленные типы пространственных экземпляров (объектов данных). В ваших базах данных SQL Server можно создавать экземпляры типов `Point`, `LineString`, `Polygon`, `MultiPolygon`, `MultiLineString`, `MultiPoint` и `GeomCollection`, помеченных на рис. 8.1 серым цветом.

### Дополнительная информация о WGS 84

Более подробную информацию о системе WGS 84 и дополнительных геодезических стандартах см. на Web-странице <http://www.ngs.noaa.gov/faq.shtml#WGS84>.

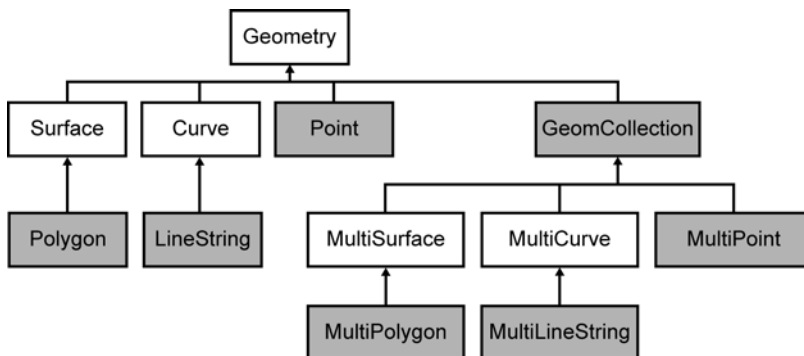


Рис. 8.1. Иерархия класса типов пространственных экземпляров

- ◆ *Типы экземпляров (или объекты пространственных данных).* Группа из 11 объектов пространственных данных, разработанная для работы с геометрическими и географическими данными. Только из семи из них можно создать экземпляры в базе данных. Типы пространственных экземпляров — часть `GeometryCollection` и встроены в иерархию. Каждый объект получает свойства от родительского объекта в иерархии класса, показанной на рис. 8.1. Белые прямоугольники на рис. 8.1 обозначают типы пространственных экземпляров, применяемых для определения общих свойств, которые наследуются объектами типов, расположенных на более низком уровне иерархии, но сами эти типы экземпляров не достаточно полные и не могут создавать собственные экземпляры.

## Ограничения

### при использовании типа данных *geography*

При использовании типа данных `geography` каждый его экземпляр должен размещаться в одном полушарии (полусфере). Вы не можете хранить объекты, которые больше полушария. Обычно мы представляем себе полушария как северное, южное, восточное и западное, но в случае типа данных `geography` это не так. В данном типе полушарие представляет собой просто полусферу. Кроме того, если вы применяете тип данных `geography`, которому на входе требуется два или несколько экземпляров `geography` и результаты методов не умещаются в одном полушарии, на выходе возвращается значение `NULL`. И наконец, если вы применяете представление в формате WKT или WKB, результаты должны уместиться в одном полушарии, иначе система генерирует исключение `ArgumentException`.

## Создание экземпляров пространственных типов данных

Прежде чем создавать экземпляры пространственных типов данных, необходимо создать таблицу, включающую столбец, определенный с типом данных `geometry` или `geography`. В примерах этой главы мы используем тип `geography`.

В следующем примере программного кода создается таблица `Museum` (Музей). Столбец `Location` (Местонахождение) содержит географические координаты местоположения музея.

```
CREATE TABLE Museum (MuseumID int IDENTITY PRIMARY KEY,
 MuseumName nchar(50),
 MuseumAddress nvarchar(200),
 Location geography);
```

После того как таблица со столбцом типа `geography` создана, можно создавать экземпляр пространственного типа, вставив в таблицу картографические данные, такие как широта и долгота.

Существует множество методов для ввода в столбец `geography` данных в разных форматах. Расширенный статический метод `Point` создает экземпляр, представляющий точку, которая содержит сведения о широте и долготе и SRID.

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### **Порядок следования аргументов**

Важно помнить о том, что географические данные часто задают как широту и следующую за ней долготу. Если вы применяете выпуск Community Technology Preview (CTP) SQL Server 2008, предшествующий CTP 6, порядок команд будет таким же: широта и следом за ней долгота. Начиная с выпуска CTP 6 и в заключительном выпуске SQL Server 2008, корпорация Microsoft в ответ на пользовательский ввод помещает долготу, за которой следует широта. В зависимости от используемой вами версии интерактивного справочного руководства значения долготы и широты в синтаксических записях пространственных методов могут располагаться в обратном порядке.

Расширенный статический метод `Parse` типа данных `geography` возвращает экземпляр этого типа, если входной параметр выражен в формате OGC WKT. У метода `Parse` единственный входной параметр, определяющий представление WKT экземпляра типа `geography`, которое должно быть возвращено.

#### **Дополнительная информация о пространственных методах**

Более подробную информацию о пространственных методах, включенных в версию SQL Server 2008, см. в разд. "geography Data Type Method Reference" ("Справочник по методам типа данных geography") и "geometry Data Type Method Reference" ("Справочник по методам типа данных geometry") электронной документации по SQL Server.

В следующем программном коде в таблицу `Museum` вставляется строка с музеем COSI в г. Колумбус, штат Огайо. Для создания экземпляра точки на основе представления в формате WKT применяются методы `Parse` и `Point`. Используется SRID по умолчанию, равный 4326.

```
INSERT INTO Museum (MuseumName, MuseumAddress, Location)
VALUES ('COSI Columbus',
 '333 West Broad Street, Columbus, OH 43215',
 geography::Parse('POINT(-83.0086 39.95954)'));
```

#### **Методы PARSE и STGEOMFROMTEXT**

Методы `PARSE` и `STGEOMFROMTEXT` возвращают экземпляр типа `geometry` на основе представления в формате OGC WKT. Разница между ними состоит в том, что в методе `PARSE` предполагается параметр SRID, равный 0, а метод `STGEOMFROMTEXT` включает аргумент для задания SRID.

На рис. 8.2 показана команда `SELECT` и результирующий набор, отображающий текстовый вывод предварительно введенной строки с музеем COSI.

Функция `ToString` отображает значение типа `geography` как пригодный для чтения текст в том же формате WKT, который вы использовали при вставке строки.



На рис. 8.3 показана информация о местоположении без ее преобразования в строку для облегчения чтения.

Обратите внимание на то, что на рис. 8.3 появляется дополнительная вкладка **Spatial results** (Пространственные результаты), следующая за вкладкой **Results** (Результаты). Эту вкладку можно использовать для просмотра графического представления ваших пространственных данных.

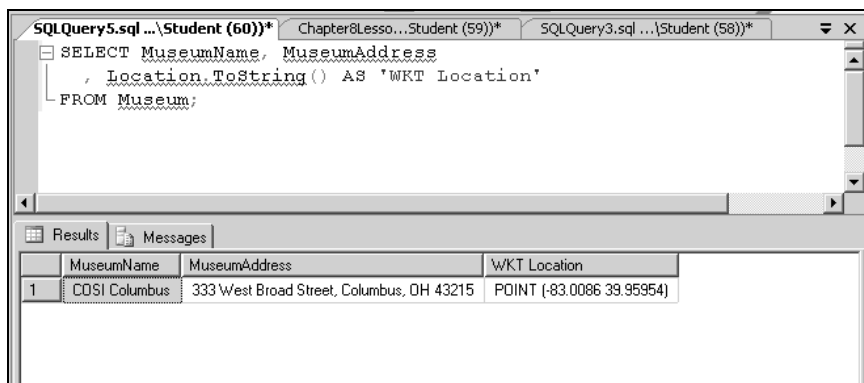


Рис. 8.2. Точка типа geography, отображенная как текст

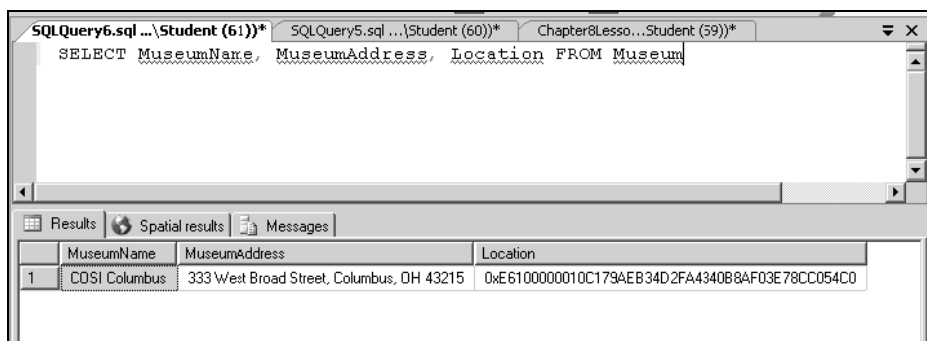


Рис. 8.3. Точка типа geography, возвращенная как трудночитаемое шестнадцатеричное значение

Перед формированием рис. 8.4 в таблицу добавлена строка с художественным музеем г. Колумбус. Следующий программный код можно использовать для вставки второй строки в таблицу Museum.

```
INSERT INTO Museum (MuseumName, MuseumAddress, Location)
VALUES ('Columbus Art Museum',
 '480 East Broad Street, Columbus, OH 43215',
 geography::Parse('POINT (-82.98775 39.963775)'));
```

На рис. 8.4 представлены две точки данных из таблицы `Museum`. К сожалению, трудно увидеть эти отдельные точки на полученном снимке экрана. Если навести указатель мыши на точку, на экране появляется ее описание (рис. 8.4).

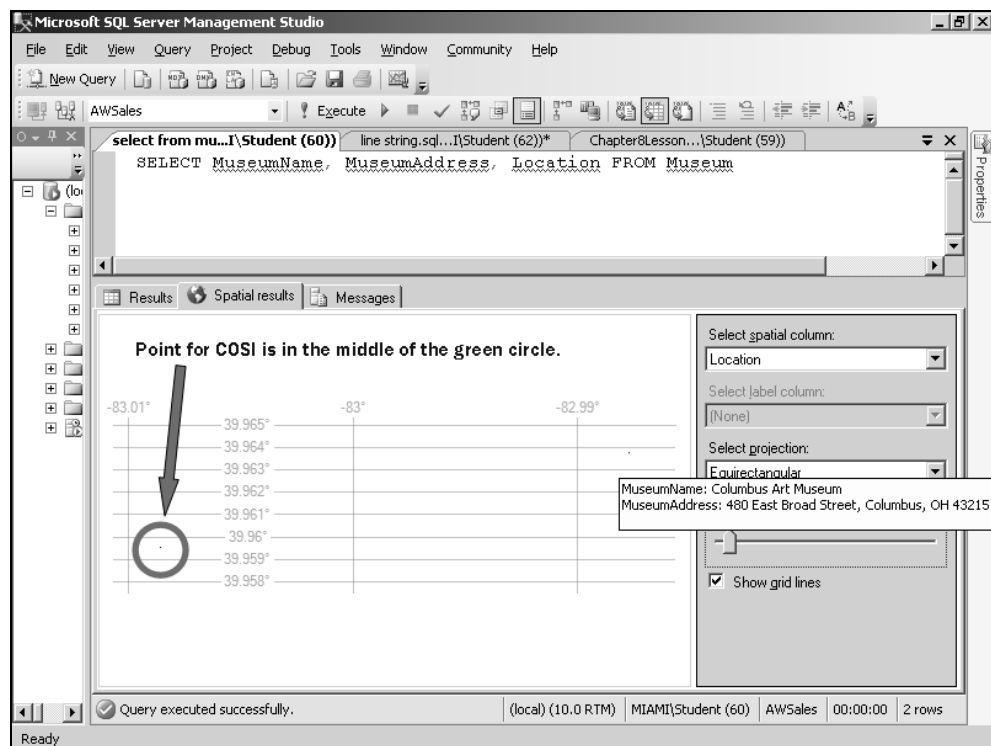


Рис. 8.4. Пример с двумя точками на вкладке **Spatial results**

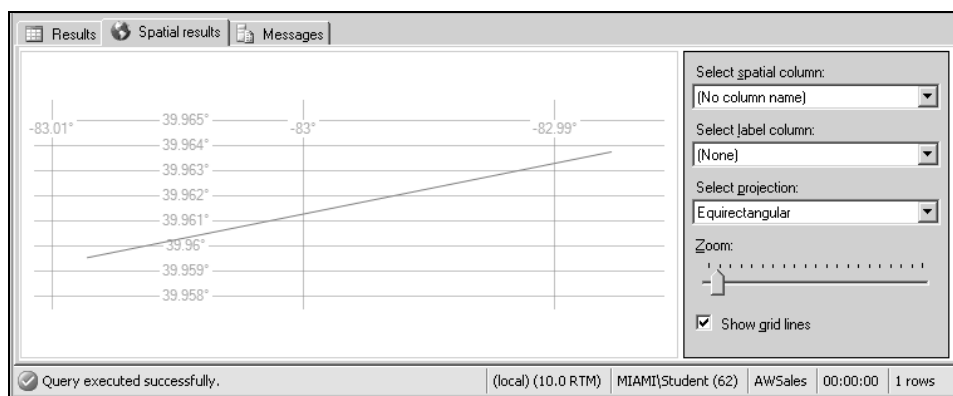


Рис. 8.5. Пример использования метода `LINESTRING`

Для того чтобы лучше показать вкладку **Spatial results** (Пространственные результаты), можно создать линию с помощью методов `STLineFromText` и `LINESTRING`. В следующем примере была определена переменная типа `geography` и использована для соединения двух точек отрезком прямой. На рис. 8.5 показана линия, отображенная на вкладке **Spatial results**.

#### ***Дополнительная информация о пространственных данных***

Дополнительные сведения о работе с пространственными данными в SQL Server 2008 см. в разд. "Spatial Ed" (Пространственный Эд) на Web-странице <http://blogs.msdn.com/edkatibah/>.

## **ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Создание экземпляров пространственных типов данных**

На этом практическом занятии вы создадите новую таблицу **Airports** (Аэропорты) для хранения данных о местоположении аэропортов, используемых клиентами вашего бюро путешествий. Затем вы примените несколько методов для вставки координат каждого местоположения. Вы также создадите таблицу **Sales** (Продажи) для отслеживания регионов продаж, в которых вашим агентам назначены клиенты. Для просмотра региона продаж вы примените вкладку **Spatial results** (Пространственные результаты) в среде SSMS.

### **Упражнение 1. Работа с точками**

В этом упражнении вы создадите точки пространственных данных и примените вкладку **Spatial results** для просмотра введенных вами точек.

1. Запустите SSMS (если она еще не запущена), установите соединение с нужным экземпляром SQL Server и откройте окно создания запроса.
2. В окне создания запроса введите и выполните следующую команду для создания новой базы данных для вашего бюро путешествий.

```
CREATE DATABASE Travel;
GO
```

3. Под имеющимся текстом введите, выделите и выполните следующий программный код, создающий таблицы **Airports** и **Sales**.

```
USE Travel;
```

```
CREATE TABLE Airports (AirportID int IDENTITY PRIMARY KEY
 , AirportName nchar(50)
 , AirportCode nchar(3)
 , Location geography);
```

```
CREATE TABLE Sales (SalesPersonID int IDENTITY PRIMARY KEY
 , FirstName nchar(50)
```

```
, LastName nchar(50)
, SalesRegionName Nchar(50)
, SalesRegionDesc nvarchar(200)
, SalesRegion geography);
```

4. Под имеющимся текстом введите, выделите и выполните следующий код для вставки в таблицу `Airports` строк с данными о международном аэропорте в Лос-Анджелесе (Los Angeles International Airport, LAX) и лондонском аэропорте Хитроу (London Heathrow Airport, LHR).

```
INSERT INTO Airports (AirportName, AirportCode, Location)
VALUES ('Los Angeles International Airport'
, 'LAX'
, geography::STGeomFromText('POINT(-118.4071611 33.9425222)',
4326));

INSERT INTO Airports (AirportName, AirportCode, Location)
VALUES ('London Heathrow Airport'
, 'LHR'
, geography::Parse('POINT(-0.45277777 51.47138888)'));
```

5. Попрактикуйтесь, вставляя разными методами строки с аэропортами, оставшимися в файле `Airport.xls`, который вы сможете найти в папке `Chapter08\Lesson 1` среди примеров, установленных с сопроводительного компакт-диска.
6. Под имеющимся текстом введите, выделите и выполните следующую команду для просмотра данных, которые вы ввели в таблицу `Airports`.
- ```
SELECT * FROM Airports;
```
7. Перейдите на вкладку **Spatial results**. Наведите указатель мыши на каждую точку, представляющую аэропорт, чтобы увидеть их свойства. Если необходимо, используйте предложения `WHERE` для ограничения результирующего набора и облегчения его чтения.
8. Оставьте среду SSMS открытой для следующего упражнения.

Упражнение 2. Работа с многоугольником

В этом упражнении вы создадите экземпляр пространственных данных в виде многоугольника и примените вкладку **Spatial results** для его просмотра.

1. Откройте окно создания запроса.
2. В окне запроса введите и выполните следующий программный код, добавляющий в таблицу `Sales` (Продажи) Kim Abercrombie.

```
USE Travel;

INSERT INTO Sales
(FirstName, LastName, SalesRegionName, SalesRegionDesc, SalesRegion)
```

```
VALUES ('Kim', 'Abercrombie', 'Southeast US',
        'Florida, Georgia, Alabama',
        geography::STGeomFromText('POLYGON(( -88.34609167 30.286825,
        -86.28281667 30.43045278,
        -85.10322222 29.63877222,
        -84.04541389 30.06674722,
        -82.78871389 29.10671389,
        -82.80399722 28.12101667,
        -82.63304722 27.47348611,
        -82.12766111 26.46439722,
        -81.65219167 25.88982778,
        -81.06721944 25.18627222,
        -80.41878333 25.15618611,
        -80.01395278 26.75898889,
        -80.55964722 28.29220833,
        -80.53146667 28.59193056,
        -81.41381944 30.69794444,
        -80.90161944 31.97196944,
        -83.19839167 34.92200556,
        -88.13663611 34.96236944,
        -88.34609167 30.286825))', 4326));
```

3. Под имеющимся текстом введите и выполните следующий текст для просмотра строки, которую вы ввели в таблицу Sales.

```
SELECT * FROM Sales;
```

4. В окне запроса просмотрите результаты запроса, щелкните кнопкой мыши вкладку **Spatial results** для просмотра контура региона Southeast US (юго-восток США).
5. Сохраните ваш сценарий и закройте SSMS.

Резюме урока

- ◆ Типы данных `geography` и `geometry` предоставляют возможность работать с пространственными данными с помощью типов, определенных системой, и избавляют от необходимости определять собственные типы данных среды CLR.
- ◆ Вы можете создавать экземпляры пространственных типов, применяя любые пространственные методы, включенные в состав SQL Server 2008.

УРОК 2. Выполнение полнотекстового поиска

В версию SQL Server 2008 включен полностью интегрированный механизм полнотекстового поиска. Полнотекстовый поиск (full-text search) позволяет формировать сложные запросы, функционально значительно превосходящие традиционную команду `SELECT` с аргументом `LIKE`. Применяя предикаты `CONTAINS` и

`FREETEXT`, а также функции `CONTAINSTABLE` и `FREETEXTTABLE`, вы сможете писать запросы, возвращающие следующие результаты:

- ◆ грамматические формы введенного вами глагола;
- ◆ результаты совпадений в данных, хранящихся как файл PDF в столбце типа `varbinary(max)`;
- ◆ синонимы искомого термина, найденные с помощью поиска в тезаурусе.

В этом уроке приведен обзор архитектуры полнотекстового поиска и способов его настройки в SQL Server 2008. Вы также познакомитесь с практическими приемами написания запросов полнотекстового поиска.

После усвоения этого урока вы сможете

- Описать функциональные возможности полнотекстового поиска в SQL Server 2008.
- Настроить полнотекстовые индексы.
- Писать полнотекстовые запросы с помощью `CONTAINS`, `CONTAINSTABLE`, `FREETEXT` и `FREETEXTTABLE`.

Расчетное время урока — 60 мин.

Обзор полнотекстового поиска

Средства полнотекстового поиска появились в версии SQL Server 7.0 и продолжают развиваться и улучшаться в каждом новом выпуске. В SQL Server 2008 сервис полнотекстового поиска полностью интегрирован в механизм управления Database Engine и больше не нуждается во внешнем поисковом сервисе. Кроме того, полнотекстовые индексы теперь могут существовать как часть структуры базы данных, а не как отдельный файл в файловой системе.

Организация полнотекстового поиска

Для того чтобы уметь находить ошибки в неверно функционирующих запросах полнотекстового поиска, вы должны знать общую структуру и конфигурацию полнотекстового поиска.

Механизм полнотекстового поиска формируют следующие процессы.

- ◆ *Процесс SQL Server* (`Sqlservr.exe`) содержит обработчик полнотекстового поиска, который управляет полнотекстовым индексированием и запросами. Благодаря полной интеграции полнотекстового поиска в механизм управления Database Engine оптимизатор распознает запросы полнотекстового поиска и повышает их производительность.

Сервис MSFTESQL больше не используется

В версии SQL Server 2008 больше нет сервиса полнотекстового поиска MSFTESQL.

- ◆ *Процесс узла управляющей программы фильтрации* (Filter daemon host process, Fdhost.exe) запускается как изолированный процесс, управляющий сторонними компонентами и защищающий таким образом процесс SQL Server от них.
- ◆ *Служба запуска FDHOST* (SQL Full-Text Filter Daemon Launcher, Fdlauncher.exe) запускает при необходимости процессы Fdhost.exe.

Дополнительная информация об изменениях в поведении полнотекстового поиска

Более подробную информацию об изменениях в поведении полнотекстового поиска, внесенных в версию SQL Server 2008, см. на Web-странице <http://technet.microsoft.com/ru-ru/library/ms143272.aspx>.

Терминология полнотекстового поиска

Знакомясь с полнотекстовым поиском, вы, возможно, столкнетесь с большим количеством новых понятий и терминов. К ним относятся следующие.

- ◆ *Элемент* (term) или выражение — слово, фраза или символьная строка, включенные как входные данные в запрос полнотекстового поиска.
- ◆ *Полнотекстовый каталог* (full-text catalog) — виртуальный объект, представляющий группу полнотекстовых индексов. Если полнотекстовый каталог создается в среде SQL Server 2008, он не должен принадлежать никакой файловой группе.
- ◆ *Полнотекстовый индекс* (full-text index) — объект, содержащий значимые слова и их местоположение в столбцах, включенных в индекс. Когда определяется полнотекстовый индекс, указываются: уникальный индекс, идентифицирующий строки в таблице; столбцы, которые следует включить в индекс; каталог, которому принадлежит индекс; файловая группа, в которой создается индекс; и дополнительные параметры.
- ◆ *Средство разбиения по словам* (word breaker) — процесс, находящий границы слов (т. е. размечающий текст) на основе лингвистических правил заданного языка данных. Вы можете задать язык данных столбца при создании полнотекстового индекса таблицы.
- ◆ *Маркер* (token) — слово или символьная строка, определенные средством разбиения слов.
- ◆ *Модуль парадигм* (stemmer) — процесс, спрягающий глаголы на основе лингвистических правил определенного языка данных.
- ◆ *Тезаурус* (thesaurus) — файлы на языке Extensible Markup Language (XML), которые задают синонимы элемента или выражения в конкретном языке. Прежде чем запросы полнотекстового поиска смогут найти синонимы в за-

данном языке, следует определить отображения тезауруса для этого языка. По умолчанию файлы тезауруса пусты.

- ◆ *Стоп-слово* или пропускаемое слово (stopword) — часто используемое слово, не имеющее дополнительного смыслового значения для поиска, например, артикли в английском языке. Стоп-слова в SQL Server 2008 играют ту же роль, что пропускаемые слова (noise words) в SQL Server 2005.
- ◆ *Список стоп-слов* (stoplist) — объект базы данных, применяемый для управления стоп-словами. Существует определяемый системой список стоп-слов, но вы также можете создавать собственные списки. Список стоп-слов может быть связан с полнотекстовым индексом.
- ◆ *Фильтр* (filter) — компонент, обрабатывающий документ для извлечения текстовой информации из документов, хранящихся в столбцах `varbinary(max)` или `image`, и затем отправляющий ее средству разбиения по словам. У файла каждого типа (например, doc, xls или pdf) должен быть собственный фильтр. Если вы определяете полнотекстовый индекс для столбца типа `varbinary(max)` или `image`, содержащего документы, необходимо также определить столбец типа, содержащий расширение файла, связанное с типом файла, хранящимся в этой же строке.
- ◆ *Заполнение* (population или crawl) — процесс добавления данных в (заполняемый) индекс либо во время его создания, либо во время перестройки. Он может инициироваться автоматически или вручную.
- ◆ *Обработчик полнотекстового поиска* (full-text engine) — встроенный компонент процесса SQL Server Service, управляющий административными задачами полнотекстового поиска, выполнением полнотекстовых запросов и процессом узла управляющей программы фильтрации (filter daemon host process).
- ◆ *Процесс узла управляющей программы фильтрации* (Filter daemon host process, Fdhost.exe) — процесс, управляющий сторонними компонентами, такими как средства разбиения по словам и модули парадигм (stemmers), отделенными от процесса SQL Server.
- ◆ *Служба запуска FDHOST* (SQL Full-Text Filter Daemon Launcher, Fdlauncher.exe) — служба, запускающая процессы Fdhost.exe, если они нужны обработчику полнотекстового поиска. Это единственная функция данной службы.

Дополнительная информация о списках стоп-слов и файлах тезауруса

Дополнительные сведения о списках стоп-слов и файлах тезауруса, включая способы их создания и редактирования, см. в разд. "CREATE FULL-TEXT STOPLIST (Transact-SQL)", "Thesaurus Configuration" (Конфигурация тезауруса) и "How to: Edit a Thesaurus File (Full-Text Search)" (Как изменить файл тезауруса (компонент Full-Text Search)) электронной документации по SQL Server. Другой источник — глава 5, "Managing Full Text Indexes" (Управление полнотекстовыми индексами) в книге Но-

tek M. MCTS Self-Paced Training Kit (Exam 70-432): SQL Server 2008 — Implementation and Maintenance. — Microsoft Press, 2009 (Хотек М. Обучающий курс для получения MCTS (экзамен 70-432): SQL Server 2008 — реализация и поддержка).

Настройка полнотекстового поиска

Несмотря на то, что, как правило, настраивает полнотекстовые индексы и управляет ими администратор базы данных, вы как разработчик базы данных должны иметь базовое представление о том, как активизировать и настроить полнотекстовый поиск. Все необходимые действия можно выполнить с помощью программного кода на языке Transact-SQL (T-SQL) или в среде SSMS. В этом разделе применяется программный код на T-SQL.

Дополнительная информация о настройке полнотекстового поиска средствами SSMS

Подробные сведения о том, как настраивать полнотекстовые индексы и каталоги с помощью SSMS, см. в разд. "Full-Text Catalog and Index How-to Topics (Full-Text Search)" (Инструкции по полнотекстовым каталогам и индексам (полнотекстовый поиск)) электронной документации по SQL Server.

По умолчанию полнотекстовый поиск включен в установку Database Engine версии SQL Server 2008. Если при инсталляции SQL Server вы отказались от установки компонентов полнотекстового поиска, можно использовать Центр установки SQL Server (SQL Server Installation Center) для добавления этого средства. Кроме того, в SQL Server 2008 все базы данных настроены на автоматическую поддержку полнотекстового поиска. Найти Центр установки SQL Server можно, выполнив последовательность команд **Start | All Programs | Microsoft SQL Server 2008 | Configuration Tools** (Пуск | Все программы | Microsoft SQL Server 2008 | Configuration Tools).

Для активизации функциональных возможностей полнотекстового поиска следует создать полнотекстовый каталог и полнотекстовый индекс, как поясняется в следующих разделах.

Создание полнотекстовых каталогов

Первый шаг в настройке полнотекстового индексирования в базе данных — создание полнотекстового каталога для базы данных, из которой вы хотите запрашивать данные, применяя средства полнотекстового поиска. Вы не можете создавать полнотекстовые каталоги в базах данных master, model или tempdb. Для формирования полнотекстового каталога можно воспользоваться командой `CREATE FULLTEXT CATALOG`.

Эта команда включает в себя следующие аргументы.

- ◆ `Catalog_name` — имя, которое будет дано каталогу.
- ◆ `ON FILEGROUP filegroup` — включен для обратной совместимости и в версии SQL Server 2008 не оказывает никакого влияния.

- ◆ `IN PATH 'rootpath'` — включен для обратной совместимости и в версии SQL Server 2008 не оказывает никакого влияния.
- ◆ `ACCENT_SENSITIVITY ON/OFF` — задает выполнение полнотекстового поиска с учетом диакритических знаков или без их учета. Если этот параметр не задан, применяется отношение к диакритическим знакам, заданное в наборе параметров символьной обработки (collation) базы данных.
- ◆ `AS DEFAULT` — определяет создаваемый вами каталог как каталог по умолчанию для базы данных. (Если в этой базе данных вы создадите полнотекстовый индекс, не указав при этом каталог, будет применяться каталог по умолчанию.)
- ◆ `AUTHORIZATION owner_name` — определяет владельца полнотекстового каталога.

В следующем программном коде создается каталог `ftCatalog`, используемый как каталог по умолчанию в базе данных `AdventureWorks2008`.

```
USE AdventureWorks2008;  
CREATE FULLTEXT CATALOG ftCatalog AS DEFAULT
```

Создание полнотекстовых индексов

Прежде чем создавать полнотекстовый индекс с помощью команды `CREATE FULLTEXT INDEX`, необходимо выполнить следующие требования.

- ◆ В таблице не может уже существовать полнотекстовый индекс. В каждой таблице можно создать только один полнотекстовый индекс.
- ◆ В таблице должен существовать индекс уникального ключа. Этот индекс ключа должен быть основан на столбце с одним ключом, однозначно идентифицирующим строки и не допускающим значений `NULL`.
- ◆ В соответствующей базе данных должен присутствовать полнотекстовый каталог. Если в базе данных используемой таблицы нет каталога по умолчанию, его имя следует задать в вашей команде `CREATE FULLTEXT INDEX`.

Применение Мастера построения полнотекстового индекса

Если в среде SSMS вы примените **Full-Text Indexing Wizard** (Мастер построения полнотекстового индекса), новый полнотекстовый каталог можно будет создать в процессе определения полнотекстового индекса.

Команда `CREATE FULLTEXT INDEX` включает в себя следующие параметры.

- ◆ `Table_name` — имя таблицы или индексированного представления, для которых создается полнотекстовый индекс.
- ◆ `Column_name` — имена столбцов, включаемых в полнотекстовый индекс. Если не задать никаких столбцов, команда успешно создаст полнотекстовый индекс без единого включенного в него столбца. Прежде чем вы сможете заполнить или запросить такой индекс, придется добавить в него столбцы.

- ◆ `TYPE COLUMN type_column_name` — задается имя столбца, содержащего расширение, представляющее тип файла данных, хранящегося в столбце типа `varbinary`, `varbinary(max)` или `image`. Если вы зададите этот параметр и не включите в полнотекстовый индекс двоичный столбец с данными, будет сгенерирована ошибка.
- ◆ `LANGUAGE language_term` — задает строковое, целочисленное или шестнадцатеричное значение, представляющее идентификатор локализации (`locale identifier`). Если язык не задан, используется язык по умолчанию экземпляра SQL Server.
- ◆ `KEY INDEX index_name` — определяет имя уникального индекса, необходимого полнотекстовому индексу для идентификации каждой строки в таблице. Это обязательное значение.
- ◆ `fulltext_catalog_name` — имя логического полнотекстового каталога, в котором создается индекс. Если не существует каталога по умолчанию, следует задать корректное имя полнотекстового каталога в базе данных, к которой относится таблица.
- ◆ `FILEGROUP filegroup_name` — задает корректное имя существующей файловой группы, в которой будет храниться полнотекстовый индекс. Если не указать файловую группу, полнотекстовый индекс сохраняется в той же файловой группе, что и заданные таблица или представление. Если таблица секционирована, полнотекстовый индекс сохраняется в первичной (`primary`) файловой группе для секционированной таблицы.
- ◆ `CHANGE_TRACKING` — определяет, как изменения в таблице переносятся в полнотекстовый индекс. У этого аргумента следующие возможные варианты.
 - `AUTO` — перенос изменений производится автоматически. Тем не менее, изменения не обязательно немедленно отражаются в полнотекстовом индексе. Это вариант, принятый по умолчанию.
 - `MANUAL` — для переноса изменений в полнотекстовый индекс следует выполнить инструкцию `ALTER FULLTEXT INDEX...START UPDATE POPULATION` вручную или с помощью задания службы SQL Server Agent.
 - `OFF` — SQL Server не отслеживает изменения в таблице. Для переноса любых изменений в полнотекстовый индекс необходимо выполнить полное заполнение. Если не задан вариант `NO POPULATION`, начальное заполнение полнотекстового индекса будет происходить автоматически, после создания этого индекса.
 - `OFF, NO POPULATION` — после создания полнотекстового индекса не происходит его автоматическое начальное заполнение. Вариант `NO POPULATION` допустим только вместе с вариантом отслеживания изменений `OFF`.

Модификации с помощью команд `WRITETEXT` и `UPDATETEXT`

Модификации, выполненные командами `WRITETEXT` и `UPDATETEXT`, не отражаются в полнотекстовом индексе и не отслеживаются с помощью средств отслеживания изменений.

- ◆ **STOPLIST** — задает список стоп-слов, связанный с полнотекстовым индексом. Если этот параметр не задан, применяется системный полнотекстовый список стоп-слов, устанавливаемый по умолчанию. У этого параметра следующие возможные значения.
- **OFF** — с этим полнотекстовым индексом не связан никакой список стоп-слов.
 - **SYSTEM** — с этим полнотекстовым индексом связан системный список стоп-слов, устанавливаемый по умолчанию.
 - **Stoplist_name** — корректное имя имеющегося списка стоп-слов, который будет связан с данным полнотекстовым индексом.

В следующем программном коде создается полнотекстовый индекс для столбца **Description** (Описание) в таблице **Production.ProductDescription** (Описание товара). Полнотекстовый индекс использует каталог **AW2008FullTextCatalog** и системный список стоп-слов.

```
CREATE FULLTEXT INDEX ON Production.ProductDescription (Description)
KEY INDEX PK_ProductDescription_ProductDescriptionID
ON AW2008FullTextCatalog
WITH STOPLIST = SYSTEM;
```

Создание полнотекстовых запросов

Когда вы пишете полнотекстовые запросы, можно выбирать между предикатами **CONTAINS** и **FREETEXT** и функциями **CONTAINSTABLE** и **FREETEXTTABLE**. Названные команды предоставляют множество вариантов запросов, позволяющих вернуть разные формы данных. В этом разделе вы узнаете о функциональных возможностях данных предикатов и функций и познакомитесь с примерами запросов всех типов.

Выявление ошибок полнотекстового поиска

По мере изучения полнотекстовых запросов и знакомства с их примерами вы поймете, какие в вашем распоряжении есть средства, способные помочь находить ошибки в возвращаемых результатах запросов и анализировать полученные результаты.

РЕАЛЬНАЯ ИСТОРИЯ.

Анна Вебер (Ann Weber)

У меня был опыт построения в версиях SQL Server 2000 и SQL Server 2005 полнотекстовых запросов, содержащих в условиях поиска специальные символы. Напри-

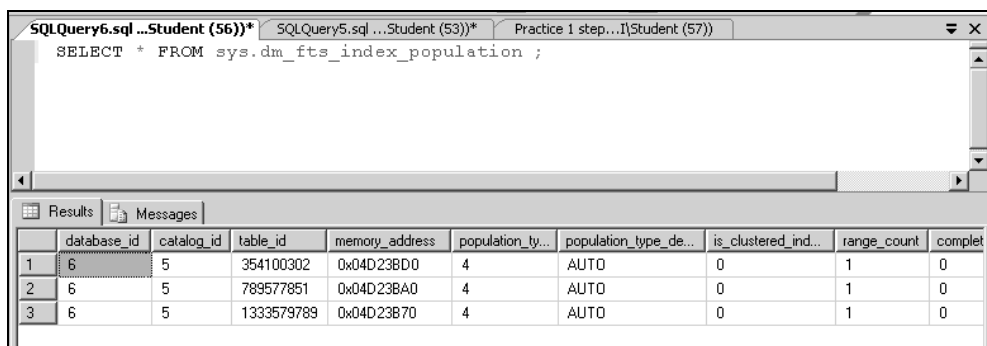
мер, для обнаружения п/а и С# мы должны были выполнить пространные тестовые запросы к различным образцам данных, чтобы определить, что же вернут эти разнообразные запросы. Не существовало легкого способа увидеть, как обработчик полнотекстового поиска будет интерпретировать конкретный запрос. Теперь в версии SQL Server 2008 функция динамического администрирования (dynamic management function, DMF) `sys.dm_fts_parser` покажет вам результаты работы средства разбиения по словам для разных входных выражений в зависимости от входных данных и операторов в выражениях.

Если вы выполните полнотекстовый запрос до завершения полнотекстового заполнения, запрос может вернуть только часть совпадающих строк. Для того чтобы определить состояние операции заполнения полнотекстового каталога, можно применить функцию `FULLTEXTCATALOGPROPERTY`. Если заполнение в действии, обычно возвращается 1.

Для проверки состояния заполнения каталога `AdvWksDocFTCat` можно выполнить следующую команду.

```
SELECT FULLTEXTCATALOGPROPERTY('AdvWksDocFTCat', 'PopulateStatus');
```

Кроме того, текущее состояние заполнения возвращает объект динамического администрирования `sys.dm_fts_index_population`. На рис. 8.6 показана простейшая форма функции `sys.dm_fts_index_population`.



| | database_id | catalog_id | table_id | memory_address | population_ty... | population_type_de... | is_clustered_ind... | range_count | complet |
|---|-------------|------------|------------|----------------|------------------|-----------------------|---------------------|-------------|---------|
| 1 | 6 | 5 | 354100302 | 0x04D23BD0 | 4 | AUTO | 0 | 1 | 0 |
| 2 | 6 | 5 | 789577851 | 0x04D23BA0 | 4 | AUTO | 0 | 1 | 0 |
| 3 | 6 | 5 | 1333579789 | 0x04D23B70 | 4 | AUTO | 0 | 1 | 0 |

Рис. 8.6. Базовая синтаксическая запись функции `sys.dm_fts_index_population` и пример результатов

Для большей наглядности результатов можно создать хранимые процедуры, которые вернут результат в более понятной пользователю форме. В приведенном далее программном коде используются псевдонимы и системные функции для облегчения чтения содержимого динамического административного представления `sys.dm_fts_index_population`. На рис. 8.7 показаны результаты этого запроса.

```
SELECT DB_NAME(database_id) AS 'Database Name'
      , database_id AS 'DB_ID'
      , OBJECT_NAME(table_id) AS 'Table Name'
      , table_id
```

```

, population_type_description AS 'Population Desc.'
, status_description AS 'Status Desc.'
, completion_type_description AS 'Completion Desc.'
, start_time
FROM sys.dm_fts_index_population;

```

| | Database Name | DB_ID | Table Name | table_id | Population Desc. | Status Desc. | Completion Desc. | start_time |
|---|--------------------|-------|---------------|------------|------------------|--------------|------------------|-------------------------|
| 1 | AdventureWorks2008 | 6 | ProductReview | 354100302 | AUTO | Starting | NONE | 2008-11-24 05:03:07.330 |
| 2 | AdventureWorks2008 | 6 | Document | 789577851 | AUTO | Starting | NONE | 2008-11-24 05:03:08.610 |
| 3 | AdventureWorks2008 | 6 | JobCandidate | 1333579789 | AUTO | Starting | NONE | 2008-11-24 05:03:08.610 |

Рис. 8.7. Пример более наглядного вывода из DMV `sys.dm_fts_index_population`

Для отображения столбцов, включенных в полнотекстовый индекс, можно применять представление каталога `sys.fulltext_index_columns`, как показано в следующей команде.

```

SELECT OBJECT_NAME (object_id) AS TableName
, object_id
, COL_NAME(object_id, column_id) AS ColumnName
, column_id
, COL_NAME(object_id, type_column_id) AS TypeColumn
, language_id
FROM sys.fulltext_index_columns;

```

На рис. 8.8 приведены имена таблиц и столбцов, для которых определен полнотекстовый индекс.

| | TableName | object_id | ColumnName | column_id | TypeColumn | language_id |
|---|---------------|------------|-----------------|-----------|---------------|-------------|
| 1 | ProductReview | 354100302 | Comments | 7 | NULL | 1033 |
| 2 | Document | 789577851 | DocumentSummary | 11 | NULL | 1033 |
| 3 | Document | 789577851 | Document | 12 | FileExtension | 1033 |
| 4 | JobCandidate | 1333579789 | Resume | 3 | NULL | 1033 |

Рис. 8.8. Пример синтаксической записи и вывода представления каталога `sys.fulltext_index_columns`

Если запрос полнотекстового поиска не возвращает ожидаемый результирующий набор, можно воспользоваться функцией динамического администрирования (DMF) `sys.dm_fts_parser` для отображения окончательного результата разметки данных из запроса. Окончательный результат разметки основан на входном элементе поиска и параметрах, таких как средство разбиения по словам, тезаурус и используемые списки стоп-слов.

У функции `sys.dm_fts_parser` следующая синтаксическая запись:

```
sys.dm_fts_parser('query_string', lcid, stoplist_id, accent_sensitivity)
```

В ней применяются следующие аргументы:

- ◆ *query_string* — строка, для которой вы хотите увидеть результат работы средства разбиения по словам. Эта строка может включать любые допустимые параметры из предиката `CONTAINS`, включая `INFLECTIONAL`, `THESAURUS` и логические операторы;
- ◆ *lcid* — идентификатор локализации (location identifier), задающий применяемое средство разбиения по словам;

Отображение имеющихся языков

Вы можете просмотреть все языки, доступные для конкретного экземпляра SQL Server, выполнив следующий запрос:

```
SELECT * FROM sys.full-text_languages ORDER BY lcid
```

- ◆ *stoplist_id* — числовой идентификатор списка стоп-слов, связанного с запросом. Значение 0 указывает на использование системного списка стоп-слов, а значение `NULL` — на отказ от использования любого списка;
- ◆ *accent_sensitivity* — отношение к диакритическим знакам. Для запросов с учетом диакритических знаков задавайте этот параметр равным 1, а для запросов без учета таких знаков — 0. Если язык, например французский, содержит знак ударения (диакритический знак), этот параметр определяет, будут ли считаться совпадающими слова `ou` и `оў`. Если данный параметр равен 1, приведенные слова не будут рассматриваться как совпадающие.

На рис. 8.9 показан пример использования функции динамического администрирования `sys.dm_fts_parser`. Результирующий набор содержит найденные совпадения для грамматических форм глагола *read* (читать).

Если ваши данные для поиска содержат специальные символы, такие как амперсанд или прямой слэш, вы можете получить неожиданный результат. У этих и некоторых других символов в механизме полнотекстового поиска особое назначение. На рис. 8.10 показаны результаты следующих запросов поиска элемента `n/a`.

```
SELECT * FROM sys.dm_fts_parser ('n/a', 1033, 0, 0);  
SELECT * FROM sys.dm_fts_parser ('n/a', 1033, NULL, 0);  
SELECT * FROM sys.dm_fts_parser ('"n/a"', 1033, 0, 0);
```

В первом запросе применяется системный список пропускаемых слов. Анализатор отображает входной параметр (*query_string*) как два отдельных элемента: `n` и `a`. Одиночные буквы помечаются как пропускаемые слова, и в фактическом запросе к вашим данным не будет найдено ни одной строки, потому что пропускаемые (не значимые) слова игнорируются в результирующем наборе.

SQLQuery9.sql ...Student (53))* dm_fts_parser ...{Student (57))

```

SELECT * FROM sys.dm_fts_parser
L ('FORMSOF(INFLECTIONAL, read) ', 1033, 0,0)|

```

| | keyword | group_id | phrase_id | occurrence | special_term | display_term | expansion_type | source_term |
|---|--------------------------------|----------|-----------|------------|--------------|--------------|----------------|-------------|
| 1 | 0x00720065006100640069006E0067 | 1 | 0 | 1 | Exact Match | reading | 2 | read |
| 2 | 0x00720065006100640073 | 1 | 0 | 1 | Exact Match | reads | 2 | read |
| 3 | 0x0072006500610064 | 1 | 0 | 1 | Exact Match | read | 0 | read |

Рис. 8.9. Синтаксическая запись поиска грамматических форм глагола *read* и результат

| | keyword | group... | phrase... | occurren... | special te... | display te... | expansion ty... | source te... |
|---|---------|----------|-----------|-------------|---------------|---------------|-----------------|--------------|
| 1 | 0x006E | 1 | 0 | 1 | Noise Word | n | 0 | n/a |
| 2 | 0x0061 | 1 | 0 | 2 | Noise Word | a | 0 | n/a |

| | keyword | group... | phrase... | occurren... | special_term | display_term | expansion_ty... | source_te... |
|---|---------|----------|-----------|-------------|--------------|--------------|-----------------|--------------|
| 1 | 0x006E | 1 | 0 | 1 | Exact Match | n | 0 | n/a |
| 2 | 0x0061 | 1 | 0 | 2 | Exact Match | a | 0 | n/a |

| | keyword | group... | phrase... | occurren... | special te... | display te... | expansion ty... | source te... |
|---|---------|----------|-----------|-------------|---------------|---------------|-----------------|--------------|
| 1 | 0x006E | 1 | 0 | 1 | Noise Word | n | 0 | n/a |
| 2 | 0x0061 | 1 | 0 | 2 | Noise Word | a | 0 | n/a |

Рис. 8.10. Пример синтаксической записи и результатов нескольких вариантов поиска элемента *n/a*

Второй запрос использует значение `NULL`, чтобы сообщить анализатору о том, что все списки стоп-слов игнорируются. По этой причине фактический запрос к вашим данным, применяющий этот вариант поиска, вернет все строки с единственной буквой *n* и все строки с единственной буквой *a*.

В последнем запросе элемент заключен в двойные кавычки, демонстрируя, что даже теперь, когда ввод выглядит как *n/a*, анализатор воспринимает его как *n* и *a*.

Дополнительная информация о полнотекстовом поиске и специальных символах

Дополнительные сведения о полнотекстовом поиске и его интерпретации специальных символов, включая влияние специальных символов на файлы тезауруса, см. на Web-странице <http://support.microsoft.com/default.aspx/kb/923317> и в разд. "sys.dm_fts_parser (Transact SQL)" электронной документации по SQL Server.

В заключительном примере показано применение двойных кавычек для определения не отдельного слова, а фразы. Во втором запросе также использованы логические операторы.

```
SELECT * FROM sys.dm_fts_parser
('"Backyard Playground"', 1033,0,0)
SELECT * FROM sys.dm_fts_parser
('"Backyard Playground" OR "Swing Set" ', 1033,0,0)
```

| Results | | Messages | | | | | | |
|---------|----------------------------|----------|-----------|------------|--------------|--------------|----------------|---------------------|
| | keyword | group_id | phrase_id | occurrence | special_term | display_term | expansion_type | source_term |
| 1 | 0x006200610063006800790... | 1 | 0 | 1 | Exact Match | backyard | 0 | Backyard Playground |
| 2 | 0x0070006C0061007900670... | 1 | 0 | 2 | Exact Match | playground | 0 | Backyard Playground |
| | | | | | | | | |
| | keyword | group_id | phrase_id | occurrence | special_term | display_term | expansion_type | source_term |
| 1 | 0x006200610063006800790... | 1 | 0 | 1 | Exact Match | backyard | 0 | Backyard Playground |
| 2 | 0x0070006C0061007900670... | 1 | 0 | 2 | Exact Match | playground | 0 | Backyard Playground |
| 3 | 0x007300770069006E0067 | 2 | 0 | 1 | Exact Match | swing | 0 | Swing Set |
| 4 | 0x007300650074 | 2 | 0 | 2 | Exact Match | set | 0 | Swing Set |

Рис. 8.11. Пример синтаксической записи запросов с двойными кавычками и логическими операторами и их результаты

CONTAINS и CONTAINSTABLE

Предикат `CONTAINS` позволяет писать запросы, возвращающие точные совпадения с вашими входными параметрами и нечеткие (менее точные) (fuzzy) совпадения с входным элементом. У функции `CONTAINSTABLE` те же условия поиска, что и у предиката `CONTAINS`, но она также позволяет возвращать в результирующем наборе степень соответствия (relevance value) (RANK) и полнотекстовый ключ (KEY) для каждой строки. С помощью этих команд вы можете искать следующие элементы.

- ◆ Точные совпадения со словом или фразой.
- ◆ Синоним (совпадение в тезаурусе) для слова или фразы. Например, можно создать пользовательский тезаурус, который возвращает "teacher" (учитель), "educator" (воспитатель) и "professor" (преподаватель) при запросе элемента "teacher".
- ◆ Родственные грамматические формы глагола. Например, если вы запросите элемент write (писать), запрос, учитывающий словоформы, вернет такие совпадения, как "wrote", "written" и "writes".
- ◆ Последовательность символов, появляющихся в начале слова или в начале любого слова в фразе. Например, можно определить начальный элемент в ви-

де фразы "local school". Полнотекстовый поиск такого элемента вернет как совпадения "local schools" и "locally schooled".

- ◆ Слово, расположенное рядом с другим словом.

У предиката `CONTAINS` два аргумента — `IncludedColumns` и `SearchCondition`. Аргумент `IncludedColumns` может содержать любой из следующих компонентов:

- ◆ `Column_name` — имя столбца, если хотите искать только в одном столбце;
- ◆ `Column_list` — имена столбцов, включаемых в поиск, если хотите искать в нескольких столбцах;
- ◆ `*` (звездочка) — означает, что столбцы таблицы в предложении `FROM`, допускающие полнотекстовый поиск, должны быть включены в него. Если в предложении `FROM` перечислено несколько таблиц, вы должны указать имя таблицы (например, `production.productreview.*`);
- ◆ `Language` — задавайте с помощью этого параметра язык запроса. Его значение может быть одним из названий языков из таблицы `syslanguages` (заключенное в одинарные кавычки), целочисленным идентификатором локализации (LCID) или шестнадцатеричным значением LCID.

Аргумент `SearchCondition` может содержать следующие компоненты.

- ◆ `Simple Term` (простое выражение) — задавайте в этом компоненте слово (строку символов без знаков препинания или пробелов) или фразу (несколько слов, как правило, с пробелом между ними). Фразы следует заключать в двойные кавычки ("").
- ◆ `Prefix Term` (префиксное выражение) — в этом компоненте задавайте слово или фразу, если хотите найти слова или фразы, начинающиеся с вашего префиксного выражения. За последним словом префиксного выражения должна следовать звездочка, и все префиксное выражение, включая звездочку, нужно заключить в двойные кавычки. Если применить одинарные кавычки, SQL Server решит, что в запросе простое выражение, а не префиксное. В префиксном выражении звездочка интерпретируется как 0 или больше символов. Например, если в вашем условии поиска записано `"text*"`, будут возвращены как строки со значениями `"text"`, так и строки со значениями `"textbook"`.
- ◆ `Generation Term` (производное выражение) — задайте ключевое слово `INFLECTIONAL` или `THESAURUS` для определения зависящей от языка подстановки. Выражение с ключевым словом `INFLECTIONAL` использует модуль парадигм (stemmer) заданного языка для поиска форм глаголов и существительных. Выражение с ключевым словом `THESAURUS` применяет тезаурус на соответствующем языке для сопоставления с файлом тезауруса самого длинного образца или образцов, взятых из слова или фразы, которые представлены в ар-

гументе `contains_Search_Condition`. Если в этом файле не найдено совпадений, параметр `Generation Term` игнорируется и выполняется поиск простого элемента.

- ◆ **Proximity Term** (выражение с учетом расположения) — задавайте его оператором близости `NEAR` или тильда (`~`), чтобы указать, что элемент слева от оператора близости (`proximity operator`) должен быть расположен недалеко от элемента справа. Если вы связываете с помощью операторов близости более двух элементов, все элементы должны располагаться близко друг к другу. Ключевое слово `NEAR` и тильда (`~`) действуют идентично.
- ◆ **Weighted Term** (взвешенное выражение) — задавайте его с ключевым словом `ISABOUT` для обозначения применения взвешенного выражения. Кроме того, в нем используется ключевое слово `WEIGHT` и число в диапазоне от 0.0 до 1.0 для указания относительного веса каждого компонента во взвешенном элементе. Этот параметр имеет смысл только совместно с функцией `CONTAINSTABLE`.
- ◆ **Логические операторы** — к ним относятся `AND` (или `&`), `AND NOT` (или `&!`) и `OR` (или `|`). Ключевые слова и символы взаимозаменяемы. `AND` означает, что оба условия должны быть соблюдены, а `AND NOT`, что первое условие должно быть истинным (`true`), а второе — ложным (`false`). Оператор `OR` означает, что должны соблюдаться одно или оба условия.

Порядок операций

Если включено несколько логических операторов, первыми вычисляются группы, заключенные в скобки, за которыми следуют операторы `NOT`, далее `AND` и затем `OR`. Оператор `NOT` должен сопровождать оператор `AND`, как в случае `AND NOT`. Порядок следования при многократном применении одного и того же оператора не важен (например, `1 OR 2 OR 3` дает в результате то же, что `2 OR 3 OR 1`), поскольку логические операторы ассоциативны.

У функции `CONTAINSTABLE` есть следующие дополнительные аргументы:

- ◆ **Table** — имя таблицы, для которой включен полнотекстовый поиск;
- ◆ **Top_n_by_rank** — задает возврат только указанного числа строк с наибольшими рангами. Ограничив с помощью этого параметра результаты только самыми значимыми строками, вы сможете повысить производительность запроса.

Примеры использования `CONTAINS` и `CONTAINSTABLE`

Следующий запрос возвращает строки из таблицы `ProductReview` (Обзор товаров), в которых слово *quality* (качество) находится рядом со словом *comfort* (комфорт). Для отображения названия товара вместо его идентификатора в пример включено соединение с таблицей `Product` (Товар). Поскольку предикат `CONTAINS` может запрашивать только один полнотекстовый индекс, следует указать в предложении `WHERE`, из какой таблицы вы хотите запросить столбцы, как показано жирным шрифтом в приведенном далее примере.

```

SELECT ProductReviewID, Production.Product.Name AS 'Product Name'
      , Rating, Comments, Production.ProductReview.ModifiedDate
FROM Production.Product
JOIN Production.ProductReview
      ON Production.Product.ProductID = Production.ProductReview.ProductID
WHERE CONTAINS(Production.ProductReview.*, 'Quality NEAR comfort')

```

Следующий запрос возвращает строки из таблицы `ProductReview`, в которых содержатся грамматические формы слова *bike* (велосипед), например, *biking* (езда на велосипеде).

```

SELECT ProductReviewID, Production.Product.Name AS 'Product Name'
      , Rating, Comments, Production.ProductReview.ModifiedDate
FROM Production.Product
JOIN Production.ProductReview
      ON Production.Product.ProductID = Production.ProductReview.ProductID
WHERE CONTAINS(Production.ProductReview.*, 'FORMSOF(INFLECTIONAL , bike)')

```

Повышение производительности

Если для задания простого элемента поиска в запросе применяется переменная, используйте для нее тип данных `nvarchar`, а не тип данных `varchar`. Если применяется тип данных `varchar`, SQL Server вынужден выполнять неявное преобразование, а преобразования ограничивают средства оптимизации запроса.

Следующий программный код возвращает строки таблицы `ProductDescription` (Описание товара), включающие слова *safety* (безопасность), *performance* (эффективность) или *comfort* (удобство). Каждому слову присвоен относительный вес, и результаты упорядочены по убыванию в соответствии с рангом соответствия.

```

USE AdventureWorks2008
GO
SELECT FT_Table.ProductDescriptionID, FT_Table.[Description]
      , KEY_TBL.RANK
FROM Production.ProductDescription AS FT_Table
INNER JOIN CONTAINSTABLE(production.ProductDescription, Description,
      'ISABOUT (comfort weight (.8)
      , Safety weight (.5)
      , Performance weight(.2) )' ) AS KEY_TBL
      ON FT_Table.ProductDescriptionID = KEY_TBL.[KEY]
ORDER BY KEY_TBL.RANK DESC;

```

FREETEXT и FREETEXTTABLE

Предикат `FREETEXT` позволяет писать запросы, которые возвращают значения, соответствующие по смыслу условию поиска, а не просто совпадающие слова или синонимы элемента поиска.

Когда выполняется полнотекстовый запрос с предикатом `FREETEXT`, обработчик полнотекстовых запросов выполняет следующие задачи:

- ◆ применяет средство разбиения по словам для разделения на отдельные слова строки, предназначенной для поиска;
- ◆ использует модуль парадигм для создания грамматических форм слов, сформированных средством разбиения по словам;
- ◆ применяет тезаурус для поиска дополнительных расширений или замен искомым элементам.

У предиката `FREETEXT` могут быть следующие аргументы.

- ◆ `Column` — имя отдельного столбца, список имен столбцов, разделенный запятыми и заключенный в скобки, или звездочка (*) для обозначения всех полнотекстовых столбцов.
- ◆ `Free-text string` — строка для поиска в перечисленных столбцах. Если заключить строку в двойные кавычки, выполняется поиск фразы. В случае поиска фразы модуль парадигм и тезаурус не используются.
- ◆ `Language` — язык, применяемый в средстве разбиения по словам, модуле парадигм, списках стоп-слов и подстановках из тезауруса.

В функции `FREETEXTTABLE` применяется комбинация аргументов из предиката `FREETEXT` и функции `CONTAINSTABLE`. Функция `FREETEXTTABLE` включает следующие ранее определенные аргументы: `Table`, `Column`, `Free-text string`, `Language` и `Top-n-by-rank`.

Примеры использования *FREETEXT* и *FREETEXTTABLE*

Следующая команда использует предикат `FREETEXT` для возврата строк с описаниями, совпадающих по смыслу с фразой "provides a light stiff ride" (трудную езду делает легкой). Результирующий набор в этом примере возвращает 34 из 762 строк таблицы `ProductDescription` (Описание товара) при наличии полнотекстового индекса, построенного для столбца описания.

```
SELECT * FROM Production.ProductDescription
WHERE FREETEXT(*,'provides a light stiff ride')
```

Если же вы примените следующую команду, то будет возвращена только одна строка.

```
SELECT * FROM Production.ProductDescription
WHERE CONTAINS(*,'"provides a light stiff ride"')
```

В приведенном далее примере для возврата строк, близких по смыслу входному тексту, используется функция `FREETEXTTABLE`. Обработчик полнотекстового поиска присваивает ранг на основе близости найденного соответствия с входной фразой. В результате первыми возвращаются строки с наибольшими рангами.

```
SELECT FT_TBL.ProductDescriptionID, FT_TBL.Description, KEY_TBL.RANK
FROM Production.ProductDescription AS FT_TBL
INNER JOIN FREETEXTTABLE (Production.ProductDescription, description,
    'light stiff ride') AS KEY_TBL
ON FT_TBL.ProductDescriptionID = KEY_TBL.[KEY]
ORDER BY KEY_TBL.RANK DESC
```

Быстрая проверка

ВОПРОС:

Вам необходимо написать запрос, возвращающий все времена глагола *write* (писать). Какое ключевое слово следует включить в ваш запрос?

ОТВЕТ: INFLECTIONAL.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Написание полнотекстовых запросов

На этом практическом занятии вы выполните запросы к таблицам и столбцам базы данных AdventureWorks2008, в которую включено полнотекстовое индексирование.

Упражнение 1. Просмотр таблиц с полнотекстовыми индексами

В этом упражнении вы просмотрите существующие в базе данных AdventureWorks2008 полнотекстовые индексы.

1. Запустите среду SSMS (если она еще не запущена), установите соединение с вашим экземпляром SQL Server и откройте окно создания запроса.
2. В окне создания запроса введите и выполните следующую команду для просмотра таблиц, снабженных полнотекстовыми индексами.

```
USE AdventureWorks2008;
GO
```

```
SELECT db_name(database_id) AS 'Database Name'
, OBJECT_NAME(table_id) AS 'Table Name'
FROM sys.dm_fts_index_population;
```

3. Просмотрите результирующий набор и отметьте, что полнотекстовое индексирование включено в таблицы ProductReview, Document и JobCandidate.

Индекс таблицы ProductDescription

Если вы запускали примеры из данной главы, то знаете, что полнотекстовый индекс есть и в таблице ProductDescription (Описание товара).

4. Если необходимо, откройте Object Explorer (Обозреватель объектов), выбрав строку **Object Explorer** (Обозреватель объектов) в меню **View** (Вид).

5. В Object Explorer раскройте узел **Databases** (Базы данных), раскройте строку **AdventureWorks2008**, затем строку **Tables** (Таблицы), щелкните правой кнопкой мыши таблицу `Production.ProductReview`, выберите команду **Full-Text Index** (Полнотекстовый индекс) и затем строку **Properties** (Свойства).
6. В окне **Full-Text Index Properties** (Свойства полнотекстового индекса) на панели **Select A Page** (Выбор страницы) щелкните кнопкой мыши **Columns** (Столбцы).
7. На странице **Columns** (Столбцы) найдите столбцы, включенные в полнотекстовый индекс.
8. Повторите пункты 5—7 для таблиц `Production.Document` и `HumanResources.JobCandidate`. Обратите внимание на то, что у столбца `Document` таблицы `Production.Document` есть значение `Type Column` в столбце `FileExtension`. Это объясняется тем, что у столбца `Document` тип данных `varbinary(max)`, и файлы должны обрабатываться соответствующим фильтром, который определяется расширением файла, включенным в столбец `FileExtension`. У всех файлов в примере расширение `doc`, и они будут обрабатываться фильтром для приложения Microsoft Office Word.
9. Оставьте открытой среду SSMS для следующего упражнения.

Упражнение 2. Применение предиката **CONTAINS**

В этом упражнении вы используете предикат `CONTAINS` для запроса к таблице `Production.Document`. Вы также примените функцию `sys.dm_fts_parser`, чтобы увидеть, как средство разбиения по словам интерпретирует входной элемент.

1. Откройте окно создания запроса, введите и выполните следующую команду для извлечения из таблицы `Production.Document` строк, содержащих любую глагольную форму слова *lubricate* (мазать) в любом из столбцов с полнотекстовым индексом.

```
USE AdventureWorks2008;  
GO
```

```
SELECT Title, Filename, FileExtension, DocumentSummary  
FROM Production.Document  
WHERE CONTAINS(*, 'FORMSOF(INFLECTIONAL, lubricate)');
```

2. Обратите внимание на то, что в столбец `DocumentSummary` включено слово "lubricating". Поскольку столбец `Document` возвращается с двоичными данными, вы не сможете прочесть в нем слова, встречающиеся в файле, но вы можете убедиться в том, что документ действительно содержит глагольную форму этого слова, если выполните поиск только в столбце `Document`, а не в столбцах `Document` и `DocumentSummary`, как в предыдущей команде. Для этого в текущем окне запроса скопируйте и вставьте под имеющийся запрос инст-

рукцию `SELECT` из п. 1. В только что вставленном запросе замените `*` на `Document`, выделите этот новый запрос и выполните его. Результирующий набор будет тем же самым, доказывая, что формы слова "lubricate" есть в обоих столбцах, `Document` и `DocumentSummary`.

3. Откройте окно создания запроса. В этом окне введите и выполните следующую команду для просмотра вывода, генерируемого параметром `FORMSOF`, который обработчик полнотекстового поиска применяет для сопоставления при поиске соответствующих строк. Данная команда применяет средство разбиения по словам для английского языка с идентификатором 1033, системный список стоп-слов и не учитывает диакритические знаки.

```
SELECT * FROM sys.dm_fts_parser  
( 'FORMSOF(INFLECTIONAL, lubricate) ', 1033, 0,0)
```

4. Просмотрите столбец `display_term` в результирующем наборе и обратите внимание на существительное "lubrication", которое не было возвращено как одна из форм глагола "lubricate".
5. Оставьте открытой среду SSMS для следующего упражнения.

Упражнение 3. Применение функции *FREETEXTTABLE* для ранжирования результатов

В этом упражнении вы примените функцию `FREETEXTTABLE` для ранжирования результатов вашего запроса. Строки, наиболее соответствующие элементу `quality bike` (достойный велосипед), будут возвращены в начале результирующего набора.

1. Откройте окно создания запроса, введите и выполните следующую команду, возвращающую совпадающие строки и их относительный ранг.

```
SELECT FT_TBL.ProductReviewID, FT_TBL.COMMENTS, KEY_TBL.RANK  
FROM Production.ProductReview AS FT_TBL  
INNER JOIN FREETEXTTABLE (Production.ProductReview, comments,  
    'quality bike') AS KEY_TBL  
    ON FT_TBL.ProductReviewID = KEY_TBL.[KEY]  
ORDER BY KEY_TBL.RANK DESC
```

2. Просмотрите результирующий набор.
3. Сохраните свои сценарии и закройте среду SSMS.

Резюме урока

- ◆ SQL Server 2008 предоставляет полностью интегрированные возможности полнотекстового поиска.
- ◆ В базе данных создаются и сопровождаются полнотекстовые индексы, организованные в виде виртуального полнотекстового каталога.

- ◆ Предикаты `CONTAINS` и `FREETEXT`, а также функции `CONTAINSTABLE` и `FREETEXTTABLE` позволяют запрашивать полностью текст, данные XML и определенные виды двоичных данных.

УРОК 3. Разработка решений Service Broker

Компонент Service Broker появился в версии SQL Server 2005 для предоставления надежной и масштабируемой системы организации очередей асинхронных сообщений для локальных и распределенных приложений. Решения Service Broker простираются от простых приложений, существующих в единственной базе данных, до сложных приложений, доступ к которым осуществляется через несколько удаленных экземпляров SQL Server.

Этот урок предлагает обзор компонентов, с которыми вам придется работать для создания решения Service Broker. Вы также узнаете о том, как настроить простое приложение Service Broker в одном экземпляре SQL Server.

После усвоения этого урока вы сможете

- Описать компоненты Service Broker.
- Определить метод активации.
- Настроить Service Broker.
- Начать диалог и отправлять и получать сообщения.

Расчетное время урока — 60 мин.

Обзор Service Broker

Для того чтобы уметь работать с Service Broker, прежде всего следует понять, как действующие совместно компоненты обеспечивают решение Service Broker. Как и в версии SQL Server 2005, решение Service Broker формируется из очередей, сообщений, типов сообщений и контрактов.

Необходимые условия функционирования Service Broker

Перед активацией Service Broker в базе данных для нее необходимо создать главный ключ (master key) базы данных. Если этого не сделать, возникнет иллюзия действующих процессов, но сообщения никогда не доберутся до очереди.

Новые функциональные возможности в SQL Server 2008

Помимо перечисленных ранее компонентов в версии SQL Server 2008 у Service Broker появились следующие новые свойства.

- ◆ *Приоритеты Service Broker* позволяют дать одному диалогу более высокий приоритет по сравнению с другим.

- ◆ *Утилита Ssbdiagnose* применяется для анализа диалогов и служб Service Broker.
- ◆ *Объект системного монитора и счетчики* обеспечивают дополнительные аналитические возможности с помощью объекта Broker TO Statistics и пяти новых счетчиков, добавленных в объект Broker Statistics.

Дополнительная информация о сборе статистики Service Broker

Подробные сведения о статистических показателях Service Broker см. в разд. "SQL Server, Broker Statistics Object" (SQL Server, объект Broker Statistics) и "SQL Server, Broker TO Statistics Object" (SQL Server, объект Broker TO Statistics) электронной документации по SQL Server.

Компоненты Service Broker

Компоненты Service Broker в версии SQL Server 2008 можно разделить на следующие три основные категории.

- ◆ *Компоненты диалога.* Эти компоненты создаются во время выполнения и действуют в соответствии с правилами, определенными с помощью компонентов определения служб, сетевых и безопасности. *Диалог* существует между инициатором и целью. Диалоги долговременны, асинхронны и надежны. Формируются диалоги из сообщений. *Сообщение* принадлежит одному и только одному диалогу и формируется на основе определенного типа сообщения. Диалог устанавливается между двумя конкретными службами Service Broker. Диалоги обеспечивают доставку сообщения once-in-order (EOIO), управляя потоком сообщений между службами. Каждый диалог принадлежит *группе сообщений* (conversation group) и следует правилам, заданным в контракте. *Приоритеты диалогов* (conversation priorities) устанавливают относительное старшинство диалогов.
- ◆ *Компоненты определения служб.* Эти компоненты вы задаете, проектируя свое решение средствами Service Broker.
 - *Очереди.* Для хранения необработанных сообщений существуют таблицы. Каждая строка в такой таблице представляет сообщение. Когда в очередь добавляется новое сообщение, в конец таблицы вставляется строка. При получении сообщений, если не задан параметр RETENTION, они удаляются из начала таблицы.
 - *Службы.* Так называют группу задач, требующих отправки сообщений. Когда создается служба, очередь, содержащая входные сообщения, задается как часть определения службы. Для управления поступающими диалогами со службами связывают контракты. С целевой службой может быть связано несколько контрактов. Если создается служба, инициирующая диалоги, но никогда не являющаяся целевой службой для любых новых

сообщений, в определение службы не нужно включать контракт. Если служба может получать сообщения на основе контракта `DEFAULT`, при определении службы следует указать контракт `DEFAULT`.

- *Контракты.* Это соглашение между двумя службами, которое определяет типы сообщений, отправляемых службами для выполнения определенных задач. В контракте также определяется, какие участники могут отправлять сообщения каких типов. Контракт находится в базе данных, в которой он создан. Если вы разрабатываете решение, включающее несколько баз данных, следует создать идентичные контракты в каждой базе данных, участвующей в решении. Контракт `DEFAULT` содержит тип сообщения `DEFAULT`. Если вы иницилируете диалог и не указываете контракт, применяется контракт `DEFAULT`.
- *Типы сообщений.* Это объект, который определяет имя и содержимое сообщения. В каждой базе данных есть тип сообщения `DEFAULT`, в котором применяется проверка допустимости `NONE`. Будьте внимательны: не путайте сообщение типа `DEFAULT` с системным контрактом `DEFAULT`. Кроме того, вы должны знать, что `DEFAULT` — это идентифицирующее имя объекта, а не ключевое слово, применяемое для определения контракта или типа сообщения.
- ◆ *Сетевые компоненты и компоненты обеспечения безопасности.* Эти компоненты применяются для определения инфраструктуры, которая позволяет доставлять сообщения. К ним относятся следующие.
 - *Маршруты.* Они используются для задания места доставки сообщений. По умолчанию в каждой базе данных есть маршрут, по которому в текущем экземпляре SQL Server следует доставлять все сообщения без заданного определения маршрута. Этот маршрут называется `AutoCreatedLocal` и соответствует любому имени службы (и экземпляру брокера). При определении маршрута задается имя службы, связанной с маршрутом, идентификатор экземпляра Service Broker, определяющий конкретную базу данных, куда следует отправлять сообщения, и сетевой адрес, содержащий адрес реального компьютера или ключевое слово, идентифицирующее машину, на которой размещена служба.
 - *Привязки удаленных служб (Remote service bindings).* Они применяются для обеспечения безопасности диалогов с удаленными базами данных.

Дополнительная информация о безопасности диалогов

Безопасность диалогов позволяет вашему решению Service Broker применять авторизацию, аутентификацию и шифрование. Некоторыми из этих средств может управлять инструкция `BEGIN DIALOG CONVERSATION`. Дополнительными характеристиками управляют с помощью привязок удаленных служб. Дополнительную информацию о безопасности диалогов и шифровании см. в разд. "Service Broker Dialog Security" (Обеспечение диалогов компонента Service Broker) электронной документации по SQL Server.

Дополнительная информация о создании привязок удаленных служб

Дополнительные сведения о привязках удаленных служб см. в разд. "CREATE REMOTE SERVICE BINDING (Transact-SQL)" электронной документации по SQL Server.

- *Конечные точки Service Broker.* Они применяются для настройки в SQL Server отправки и приема сообщений с помощью соединений по протоколу TCP/IP. Конечные точки можно использовать для контроля соединений с конечной точкой и обеспечения транспортной безопасности. По умолчанию Service Broker не может взаимодействовать через сеть, т. к. конечных точек Service Broker нет, пока вы их не настроили.

Создание приложений Service Broker

Для выполнения решений на базе Service Broker действуют несколько приложений. Для запуска диалога и отправки сообщений приложение (инициирующая служба) выполняет инструкцию `BEGIN DIALOG`, которая включает в себя всю необходимую информацию об иницилирующей и целевой службах, контрактах и т. д. После того как сообщения отправлены в очередь, должно запускаться целевое или принимающее приложение для приема и обработки сообщений. В вашем решении можно настроить от одного до четырех стартовых параметров приложений Service Broker.

Приложения с непрерывным потоком сообщений и приложения, требующие большого объема ресурсов для запуска, можно настроить так, чтобы они запускались при старте SQL Server как часть группы запуска Microsoft Windows или как служба. Поскольку эти приложения функционируют всегда, они постоянно удерживают нужные им ресурсы, но при этом они могут немедленно обработать сообщения без задержки, необходимой для запуска нового приложения.

Второй параметр запуска, который можно настраивать, использует агента SQL Server для формирования расписания выполнения приложения в заданные моменты времени. Применение задач формирования расписания для запуска приложений характерно для целевых приложений, которые могут выполняться в пакете. Например, сотрудники могут предоставлять формы финансовых отчетов о командировках в течение всего дня. К вечеру принимающее приложение получает все сообщения и либо формирует чек для возмещения расходов, либо помещает сообщение, как нуждающееся в обработке менеджером, и применяет другое приложение, которое отправляет формы в еще одну очередь для ожидания обработки другим целевым приложением. На рис. 8.12 показан информационный поток для примера с возмещением командировочных расходов.

В сценарии возмещения расходов на командировки могут пройти недели без единого сообщения, отправленного в очередь для просмотра менеджером. В такой ситуации было бы неэффективно обрабатывать эту очередь по расписанию.

Кроме того, поскольку компании важно обрабатывать оплату расходов быстро и эффективно, вам понадобится метод активации для удовлетворения этих нужд. Для этого лучше всего подходит внутренняя и основанная на событиях активация обработки.

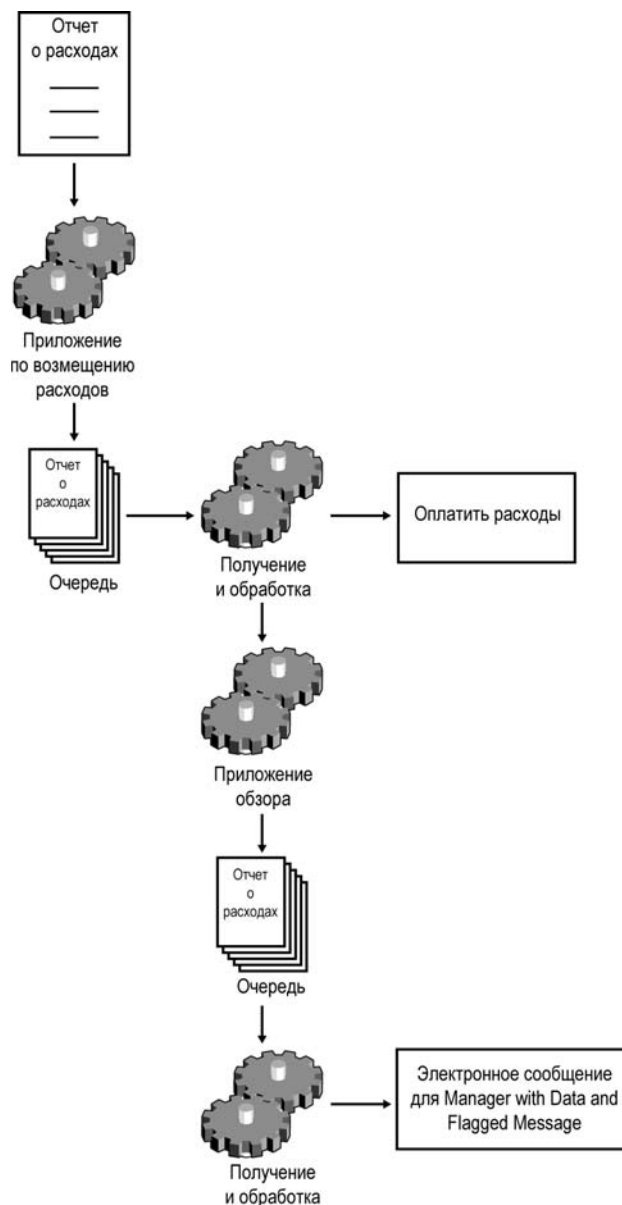


Рис. 8.12. Пример решения Service Broker

Активация Service Broker

Вы можете настроить активацию Service Broker, что позволит вашим решениям запускаться автоматически, когда есть работа для программы. Существуют два метода автоматической активации — внутренняя активация и активация, основанная на событиях.

Внутренняя активация управляется с помощью хранимых процедур и требует, чтобы приложение было написано в виде хранимых процедур. *Активация, основанная на событиях*, управляется внешним приложением и активируется событием активации очереди SQL Server, отправленным внешнему приложению.

Эти виды активации Service Broker идеальны для приложений, которые могут запускаться быстро, и для приложений, в которых частота и количество сообщений меняются во времени. Для запуска приложений, в которых сообщения получаются нечасто, можно также применять активацию Service Broker, которая позволяет приложениям автоматически запускать процессы чтения очереди по мере накопления в ней сообщений.

Дополнительная информация о настройке активации Service Broker

Дополнительные сведения о выборе метода запуска ваших приложений Service Broker, а также информацию об активации Service Broker и способах ее реализации см. следующие разделы электронной документации по SQL Server:

- "Choosing a Startup Strategy" (Выбор стратегии запуска);
- "Service Broker Activation" (Активация компонента Service Broker);
- "Implementing Internal Activation" (Реализация внутренней активации);
- "Understanding When Activation Occurs" (Основные сведения об условиях возникновения активации).

Включение компонента Service Broker

В версии SQL Server 2008 по умолчанию Service Broker включен во всех пользовательских базах данных и в базе данных msdb. С помощью Object Explorer (Обозреватель объектов) в среде SSMS вы можете просматривать и изменять свойства, относящиеся к компоненту Service Broker, на странице **Options** (Параметры) окна **Properties** (Свойства) конкретной базы данных (рис. 8.13).

Следующий запрос, отображающий свойства Service Broker для базы данных AdventureWorks2008, демонстрирует другой вариант.

```
SELECT name, service_broker_guid
      , is_broker_enabled, is_honor_broker_priority_on
FROM sys.databases
WHERE name = 'AdventureWorks2008'
```

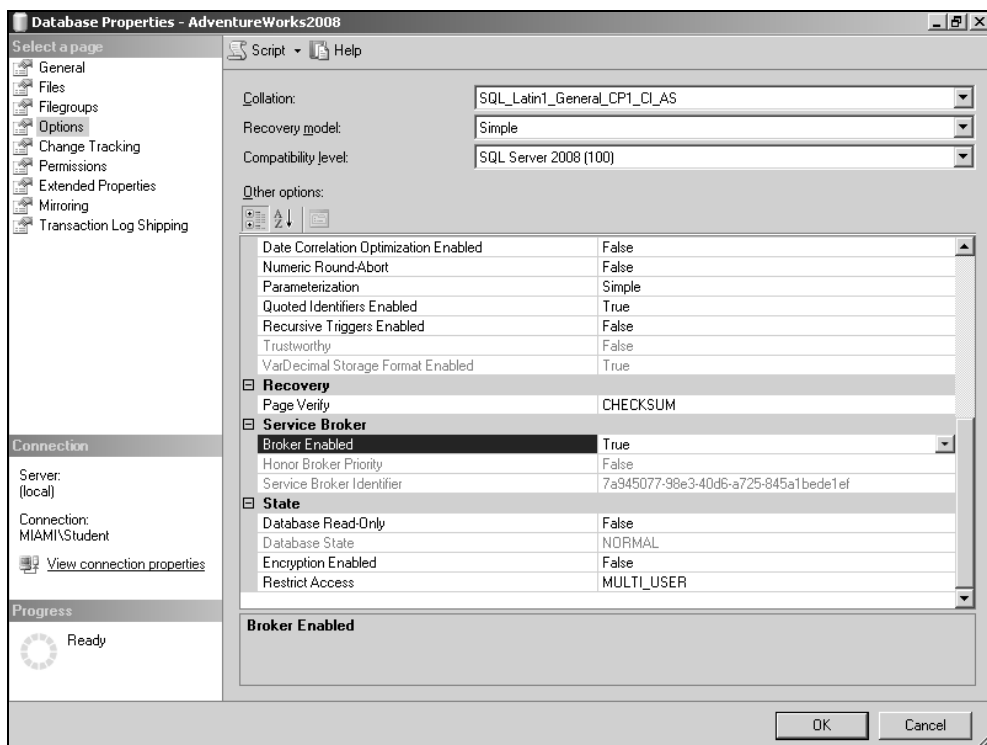


Рис. 8.13. Параметры базы данных, относящиеся к компоненту Service Broker

Для изменения параметров базы данных, относящихся к Service Broker, вы также можете применять команду `ALTER DATABASE`.

В этом случае используется следующая синтаксическая запись команды `ALTER DATABASE`.

```
ALTER DATABASE database_name
SET { ENABLE_BROKER
      | DISABLE_BROKER
      | NEW_BROKER
      | ERROR_BROKER_CONVERSATIONS
      | HONOR_BROKER_PRIORITY { ON | OFF }
}
```

Параметр `DISABLE_BROKER` отключает Service Broker в текущей базе данных и останавливает доставку сообщений, но сохраняет текущий идентификатор Service Broker. Когда для повторного включения Service Broker вы применяете параметр `ENABLE_BROKER`, текущий идентификатор Service Broker также сохраняется. Параметр `NEW_BROKER` создает новый идентификатор Service Broker и немедленно удаляет все существующие диалоги без их чистого окончания или отправки сообщений о завершении диалога. Все диалоги, начатые со старым идентификатором

Service Broker, для восстановления корректности должны быть перезапущены. Параметр `ERROR_BROKER_CONVERSATIONS` можно применять, если две базы данных, участвующие в диалоге, теряют синхронизацию из-за ошибки или сбоя в одной из них. Когда используется этот параметр, Service Broker остается включенным и сохраняется его идентификатор, но все диалоги завершаются с сообщением об ошибке. Этот параметр можно применять для того, чтобы заставить приложение выполнять регулярную очистку имеющихся диалогов. И последний параметр `HONOR_BROKER_PRIORITY` устанавливается в состояние `ON` или `OFF` для того, чтобы сообщить системе о том, нужно ли отдавать предпочтение сообщениям, пришедшим из диалогов с более высоким приоритетом.

Время реакции на изменение параметра `HONOR_BROKER_PRIORITY`

Изменения параметра `HONOR_BROKER_PRIORITY` не оказывают влияния на диалоги с имеющимися сообщениями до тех пор, пока текущие сообщения не будут отправлены. Это означает, что возможна существенная задержка между установкой состояния параметра и началом использования диалогами этого нового состояния.

Настройка компонентов Service Broker

Прежде чем вы создадите приложения для вашего решения средствами Service Broker, необходимо иметь возможность создать и настроить отдельные компоненты. К настраиваемым вами компонентам относятся очереди, службы, типы сообщений, контракты и приоритеты диалогов.

Создание типов сообщений

При создании типа сообщения вы задаете имя сообщения. Вы также определяете, выполняет ли Service Broker проверку допустимости сообщений с этим именем. Перед запуском диалога необходимо определить одинаковые типы сообщений для обеих сторон диалога.

Далее приведена полная синтаксическая запись команды `CREATE MESSAGE TYPE`.

```
CREATE MESSAGE TYPE message_type_name
[ AUTHORIZATION owner_name ]
[ VALIDATION = { NONE
                  | EMPTY
                  | WELL_FORMED_XML
                  | VALID_XML WITH SCHEMA COLLECTION
                    schema_collection_name
                  } ]
[ ; ]
```

Команда `CREATE MESSAGE TYPE` содержит следующие аргументы:

- ◆ *message_type_name* — имя типа сообщения, на которое вы ссылаетесь, когда создаете контракт (контракты), использующий этот тип сообщения. Тип со-

общения создается в текущей базе данных, и следует указывать одночастное имя (не включающее сервер, базу данных или схему);

- ◆ **AUTHORIZATION** — определяет владельца этого типа сообщения. Если при создании типа сообщения вы зарегистрированы с учетной записью, не имеющей разрешения `sa`, `dbo`, необходимо определить вашу собственную учетную запись, роль, к которой вы принадлежите, или учетную запись пользователя, для которой у вас есть разрешение `impersonate`. Если этот аргумент не задан, по умолчанию ему присваивается ваша учетная запись;
- ◆ **VALIDATION** — задается `NONE`, `EMPTY`, `WELL_FORMED_XML` или `VALID_XML WITH SCHEMA COLLECTION schema_collection_name` для определения типа проверки допустимости, которую должен выполнять Service Broker для сообщений данного типа. Установка по умолчанию, равная `NONE`, задает отсутствие проверки допустимости. Вариант `EMPTY` задает равенство тела сообщения значению `NULL`. Значение `WELL_FORMED_XML` задает тело сообщения в виде документа XML правильного формата. И последний вариант `VALID_XML WITH SCHEMA COLLECTION` задает имя существующей коллекции схем XML, которой должно соответствовать тело сообщения.

В следующем программном коде создается тип сообщения, названный `[/Adventure-Works.com/SampleType]`.

```
CREATE MESSAGE TYPE [//Adventure-Works.com/SampleType]
    AUTHORIZATION dbo
    VALIDATION = WELL_FORMED_XML
```

Создание очередей

При создании решения Service Broker следует сформировать как минимум одну очередь. Service Broker применяет очереди для хранения сообщений. Вы связываете очереди со службами, когда создаете службу.

Далее приведена полная синтаксическая запись команды `CREATE QUEUE`.

```
CREATE QUEUE <object>
[ WITH
    [ STATUS = { ON | OFF } [ , ] ]
    [ RETENTION = { ON | OFF } [ , ] ]
    [ ACTIVATION (
        [ STATUS = { ON | OFF } , ]
        PROCEDURE_NAME = <procedure>,
        MAX_QUEUE_READERS = max_readers,
        EXECUTE AS { SELF | 'user_name' | OWNER }
    ) ]
]
```

```

    [ ON { filegroup | [ DEFAULT ] } ]
[ ; ]

<object> ::=
{
    [ database_name. [ schema_name ] . | schema_name. ]
    queue_name
}

<procedure> ::=
{
    [ database_name. [ schema_name ] . | schema_name. ]
    stored_procedure_name
}

```

Единственный обязательный параметр — имя очереди. Приведенная далее команда создает очередь TestQueue:

```
CREATE QUEUE TestQueue
```

Команда CREATE QUEUE содержит следующие аргументы:

- ◆ *object* — присвойте этому аргументу имя очереди, которую нужно создать. Имя объекта выражается как полное трехчастное имя (database_name.schema_name.queue_name) или как неполное имя. Если не задать имя базы данных, применяется текущая база данных. Если не указывать схему, используется схема по умолчанию для пользователя, выполняющего инструкцию CREATE QUEUE;
- ◆ STATUS — задайте этот параметр, равным ON, если очередь доступна, и равным OFF, если она недоступна и никакие сообщения не могут быть добавлены в очередь или удалены из нее. Установка по умолчанию — ON;
- ◆ RETENTION — установите значение ON, если хотите отправлять и получать сообщения из диалогов, использующих эту очередь, оставаясь в очереди до тех пор, пока связанный с ней диалог не будет завершен. Установка по умолчанию — значение OFF — удаляет сообщения из очереди, когда для сообщения выполняется команда RECEIVE;
- ◆ ACTIVATION — задает информацию о хранимой процедуре, запускающей обработку сообщений в очереди, и содержит следующие аргументы:
 - STATUS — установите равным ON, если Service Broker должен запускать хранимую процедуру;
 - PROCEDURE_NAME — имя требуемой хранимой процедуры. Как и в случае имени очереди, имя процедуры может быть полным или неполным трехчастным именем в формате database_name.schema_name.queue_name;
 - MAX_QUEUE_READERS — максимальное количество экземпляров хранимой процедуры, запускаемых одновременно;

- EXECUTE AS — учетная запись пользователя, применяемая для выполнения хранимой процедуры. Это значение может быть равно SELF, OWNER или user_name базы данных;
- ◆ ON — файловая группа, в которой следует создать очередь.

ПРАКТИЧЕСКИЙ СОВЕТ.

Размещение очередей

Для повышения производительности и облегчения восстановления рекомендуется создавать очереди в специально выделенной файловой группе, что позволит выполнять частичные восстановления и минимизирует конфликты ввода/вывода (I/O) с операциями чтения и записи другой базы данных.

Создание контрактов

Контракты задают типы сообщений, которые могут применяться в конкретном диалоге. Контракт также определяет кем (инициатором или целевой службой или обеими службами) каждый конкретный тип сообщения может отправляться.

Далее приведена полная синтаксическая запись команды CREATE CONTRACT.

```
CREATE CONTRACT contract_name
    [ AUTHORIZATION owner_name ]
    ( { { message_type_name | [ DEFAULT ] }
      SENT BY { INITIATOR | TARGET | ANY }
    } [ , ...n ] )
[ ; ]
```

Команда CREATE CONTRACT содержит следующие аргументы:

- ◆ contract_name — имя контракта, на которое вы ссылаетесь при запуске диалога. Контракт всегда создается в текущей базе данных. Вы не можете в определении задать имена сервера, базы данных или схемы;
- ◆ AUTHORIZATION — определяет владельца контракта. Если при создании типа сообщения вы зарегистрированы с учетной записью, не имеющей разрешения sa, dbo, необходимо определить вашу собственную учетную запись, роль, к которой вы принадлежите, или учетную запись пользователя, для которой у вас есть разрешение impersonate. Если этот аргумент не задан, по умолчанию ему присваивается ваша учетная запись;
- ◆ message_type_name — имена типов сообщений, включенных в диалоги, задаваемые этим контрактом. В контракт можно включить несколько типов сообщений.
- ◆ SENT BY — задайте значение INITIATOR, TARGET или ANY для указания служб, которые могут отправлять сообщения типа, заданного в соответствующем па-

парамetre *message_type_name*. ANY означает, что и иницирующая, и целевая службы могут использовать соответствующий тип сообщений.

В каждой базе данных есть контракт с именем DEFAULT, который задает тип сообщения DEFAULT, использующий параметр VALIDATION со значением NONE.

```
CREATE CONTRACT SampleContract
AUTHORIZATION dbo
  (SampleType1 SENT BY INITIATOR,
   SampleType2 SENT BY ANY)
```

Создание служб

После создания очереди следующий шаг — создание служб, которые определяют очередь и контракт, с которыми связана служба. Служба представляет бизнес-задачу или группу задач, запрашиваемых приложением. Несмотря на то, что на одну очередь могут указывать несколько служб, каждая служба, как правило, использует свою очередь для облегчения получения и обработки сообщений.

Далее приведена полная синтаксическая запись команды CREATE SERVICE.

```
CREATE SERVICE service_name
  [ AUTHORIZATION owner_name ]
  ON QUEUE [ schema_name. ]queue_name
  [ ( contract_name | [DEFAULT] [ ,...n ] ) ]
[ ; ]
```

В отличие от очередей, позволяющих задать базу данных и схему, в которых они должны создаваться, службы всегда создаются в текущей базе данных, и вы не можете задать для них схему.

Параметр AUTHORIZATION определяет владельца службы. В параметре ON QUEUE вы должны указать имя очереди, в которой будут храниться сообщения для данной службы.

ПРАКТИЧЕСКИЙ СОВЕТ.

Задание имени схемы

Несмотря на то, что необязательно задавать имя схемы как часть имени очереди, если очередь размещается в схеме, назначенной по умолчанию пользователю, выполняющему команду, включение в ваши сценарии имени схемы облегчает их чтение и устраняет возможные проблемы в дальнейшем при выполнении сценария в рабочей среде другого пользователя.

Если в команде CREATE SERVICE не задан контракт, служба может лишь иницировать диалоги, но не может быть целевой службой. Если указан контракт DEFAULT, служба может быть целевой (или приемной) в диалогах, использующих контракт DEFAULT. В данном случае слово DEFAULT применяется как идентифицирующее имя контракта, а не в качестве ключевого слова, как в некоторых других командах, например в CREATE TABLE.

Настройка приоритетов диалогов

Вы можете применять команду `CREATE BROKER PRIORITY` для установки величины приоритета диалогов, связанных с конкретными контрактами и службами. При задании приоритета вы указываете имя для приоритета диалога, имя контракта, имена локальной и удаленной служб и величину приоритета, задаваемую для комбинации указанных контракта и служб. Уровень приоритета можно задать как любое значение из диапазона 0—10. Значение по умолчанию, равное 5, присваивается, если параметр `PRIORITY_LEVEL` не задан или равен `DEFAULT`.

Далее приведена полная синтаксическая запись команды `CREATE BROKER PRIORITY`.

```
CREATE BROKER PRIORITY ConversationPriorityName
FOR CONVERSATION
[ SET ( [ CONTRACT_NAME = {ContractName | ANY } ]
        [ [ , ] LOCAL_SERVICE_NAME = {LocalServiceName | ANY } ]
        [ [ , ] REMOTE_SERVICE_NAME = { 'RemoteServiceName' | ANY } ]
        [ [ , ] PRIORITY_LEVEL = {PriorityValue | DEFAULT } ]
      )
]
[;]
```

Дополнительная информация об уровнях приоритета диалогов

Дополнительные сведения о критериях, которыми пользуется Service Broker при назначении диалогу величины приоритета, см. в разд. "CREATE BROKER PRIORITY (Transact-SQL)" электронной документации по SQL Server.

Отправка и получение сообщений

После того как вы настроили компоненты определения служб, сетевые компоненты и компоненты обеспечения безопасности, необходимо настроить ваши приложения для инициации диалогов, отправки сообщений и обработки получаемых сообщений.

Формирование диалога

Для инициации диалога одной службы с другой можно воспользоваться инструкцией `BEGIN DIALOG`. Диалог существует между двумя службами и обеспечивает доставку сообщений, гарантируя, что каждое сообщение будет получено только один раз и в должном порядке.

Далее приведена полная синтаксическая запись команды `BEGIN DIALOG`.

```
BEGIN DIALOG [ CONVERSATION ] @dialog_handle
FROM SERVICE initiator_service_name
TO SERVICE 'target_service_name'
[ , { 'service_broker_guid' | 'CURRENT DATABASE' } ]
```

```
[ ON CONTRACT contract_name ]  
[ WITH  
  [ { RELATED_CONVERSATION = related_conversation_handle  
    | RELATED_CONVERSATION_GROUP = related_conversation_group_id } ]  
  [ [ , ] LIFETIME = dialog_lifetime ]  
  [ [ , ] ENCRYPTION = { ON | OFF } ] ]  
[ ; ]
```

Перед выполнением команды `BEGIN DIALOG` необходимо определить переменную `@dialog_handle` как `uniqueidentifier`. После определения переменной `@dialog_handle` следует ввести следующие аргументы как часть команды `BEGIN DIALOG`:

- ◆ `FROM SERVICE` — имя службы, инициирующей диалог. Эта служба должна существовать в текущей базе данных. Очередь, связанная с этой службой, применяется для хранения сообщений, возвращаемых целевой службой;
- ◆ `TO SERVICE` — имя целевой службы, которой посылаются сообщения. Это имя службы учитывает регистр клавиш, даже если база данных использует набор параметров символьной обработки (collation) без учета регистра.

Как часть команды `BEGIN DIALOG` можно задать следующие необязательные аргументы:

- ◆ `service_broker_guid` — глобальный уникальный идентификатор (GUID) Service Broker в конкретной базе данных. Этот аргумент может применяться, если ваша целевая служба размещена в нескольких базах данных и вы хотите настроить диалог для экземпляра Service Broker конкретной базы данных;
- ◆ `'CURRENT DATABASE'` — задает использование Service Broker ID текущей базы данных;
- ◆ `ON CONTRACT` — контракт, определенный для данного диалога. Если контракт не задан, для диалога применяется контракт с именем `DEFAULT`;
- ◆ `RELATED_CONVERSATION` или `RELATED_CONVERSATION_GROUP` — задает `related_conversation_handle` для указания одного связанного диалога или `related_conversation_groupid` для указания группы сообщений, к которым следует добавить этот новый диалог;
- ◆ `LIFETIME` — максимальный период времени, в течение которого диалог может оставаться открытым;
- ◆ `ENCRYPTION` — принимает значения `OFF` или `ON` для определения состояния шифрования сообщений, включенных в диалог. По умолчанию установлено состояние `ON`, требующее шифрования сообщений, пересылаемых между службами в разных экземплярах SQL Server, при этом сообщения, пересылаемые между службами, находящимися в одном экземпляре SQL Server, никогда не шифруются. Но если инициирующая и целевая службы находятся в

разных базах данных даже в одном и том же экземпляре SQL Server, необходима настройка главного ключа (master key) базы данных и соответствующих сертификатов. Это облегчит в дальнейшем перемещение базы данных в другой экземпляр сервера.

Далее приведен пример программного кода простейшего варианта команды BEGIN DIALOG.

```
DECLARE @dialog_handle uniqueidentifier
BEGIN DIALOG @dialog_handle
FROM SERVICE AW_Initiate
TO SERVICE AW_Target
```

Отправка сообщений

После создания диалога воспользуйтесь командой SEND для отправки сообщений в диалог, созданный командой BEGIN DIALOG.

Команда SEND очень проста и понятна и имеет следующую синтаксическую запись:

```
SEND
    ON CONVERSATION conversation_handle
    [ MESSAGE TYPE message_type_name ]
    [ ( message_body_expression ) ]
[ ; ]
```

После того как инициатор отправил сообщение, оно сохраняется в очереди, связанной с целевой или приемной службой. Параметр типа сообщения необязательный и обычно в команду SEND не включается. Вместо этого он задается в определении контракта. Несмотря на то, что параметр *message_body_expression* тоже не обязателен, если его не задать, тело сообщения будет пустым. Обычно тело сообщения предоставляет информацию, имеющую отношение к целевой службе.

Получение сообщений

Когда сообщения переданы в очередь, для просмотра включенных в очередь сообщений можно применить простую инструкцию SELECT. Для обработки сообщений используется команда RECEIVE, извлекающая из очереди одно или несколько сообщений. Чтение сообщений начинается с начала очереди. Если параметр очереди RETENTION равен OFF, сообщения удаляются из очереди после извлечения их командой RECEIVE. Далее приведена синтаксическая запись команды RECEIVE.

```
[ WAITFOR ( ]
    RECEIVE [ TOP ( n ) ]
        <column_specifier> [ ,...n ]
    FROM <queue>
    [ INTO table_variable ]
```

```

    [ WHERE { conversation_handle = conversation_handle
              | conversation_group_id = conversation_group_id } ]
[ ) ] [ , TIMEOUT timeout ]
[ ; ]

```

<column_specifier> ::=

```

{
    *
    | { column_name | [ ] expression } [ [ AS ] column_alias ]
    | column_alias = expression
}
[ , ...n ]

```

<queue> ::=

```

{
    [ database_name . [ schema_name ] . | schema_name . ]
    queue_name
}

```

Команда `RECEIVE` принимает следующие аргументы:

- ◆ `WAITFOR` — задает ожидание командой `RECEIVE` нового сообщения, которое следует извлечь. Этот аргумент применяется, только если в текущий момент очередь пуста;
- ◆ `TOP` — максимальное количество сообщений, которые нужно извлечь из очереди. Если в команду не включен параметр `TOP`, получаются все сообщения, удовлетворяющие критерию, определенному в инструкции `RECEIVE`;
- ◆ `column_specifier` — имя столбца, псевдоним, выражение с именами столбцов;
- ◆ `FROM` — имя очереди, из которой вы хотите извлечь сообщения;
- ◆ `INTO` — задает (необязательно) имя табличной переменной, в которую возвращаются результаты для последующей обработки. В противном случае результирующий набор может быть обработан напрямую. В большинстве рабочих сред применяется параметр `INTO`;
- ◆ `WHERE` — ограничивает количество извлекаемых строк с помощью задания диалога или группы сообщений (`conversation group`), которым должны принадлежать принимаемые сообщения;
- ◆ `TIMEOUT` — ограничивает период времени, в течение которого команда `RECEIVE` будет ждать новое сообщение, если также задан параметр `WAITFOR`. Период времени по умолчанию, равный `-1`, задает ожидание командой `RECEIVE` нового сообщения, которое нужно извлечь, в течение неограниченного периода времени.

ВАЖНАЯ ИНФОРМАЦИЯ.

Признак конца команды

Инструкция в пакете, предшествующая команде `RECEIVE`, обязательно должна завершаться точкой с запятой (;). Кстати, завершение каждой инструкции T-SQL точкой с запятой считается признаком хорошего стиля программирования.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

Реализация простого решения Service Broker

На этом практическом занятии вы создадите компоненты, необходимые для простого решения Service Broker. Вы также создадите диалог, отправите сообщения в диалог и получите сообщения из очереди.

Упражнение 1. Создание компонентов определения служб

В этом упражнении вы создадите компоненты определения служб, необходимые для инициации диалога Service Broker.

1. Откройте среду SSMS (если она еще не открыта), установите соединение с вашим экземпляром SQL Server и откройте окно создания запроса.
2. В окне создания запроса введите и выполните следующую команду для создания главного ключа базы данных AdventureWorks2008:

```
USE AdventureWorks2008;  
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'dyfnds65;sdf%h457!';
```

3. Откройте окно создания запроса, введите и выполните следующий сценарий для создания типов сообщений и контракта, которые должны использоваться иницирующей и целевой службами.

```
CREATE Message Type  
    AWWNewNotice  
    VALIDATION = NONE;
```

```
CREATE Message Type  
    AWAck  
    VALIDATION = NONE;
```

```
CREATE CONTRACT  
    NewNoticeContract  
    AUTHORIZATION dbo  
    (AWNewNotice SENT BY ANY, AWAck SENT BY ANY );
```

4. В текущем окне запроса под имеющимся текстом введите, выделите и выполните следующий сценарий, создающий очереди и службы для вашего решения.

```
CREATE QUEUE AWWNewNoticeQueue;
```

```
CREATE QUEUE AWAckQueue;
```

```
CREATE SERVICE AWWNewNoticeService  
ON QUEUE AWWNewNoticeQueue;
```

```
CREATE SERVICE AWAckService  
ON QUEUE AWAckQueue;
```

5. Оставьте открытой среду SSMS для следующего упражнения.

Упражнение 2. Отправка и получение сообщений

В этом упражнении вы инициируете диалог и отправите и получите сообщения в диалоге.

1. Откройте окно создания запроса. Введите, но не выполняйте следующие команды, определяющие диалог между AWNewNoticeService и AWAckService.

```
DECLARE @dialog_handle uniqueidentifier, @XMLdata XML;  
  
SET @XMLdata = (SELECT * FROM sys.tables FOR XML AUTO);  
  
BEGIN DIALOG @dialog_handle  
FROM SERVICE AWNewNoticeService  
TO SERVICE 'AWAckService'  
ON CONTRACT NewNoticeContract;
```

2. В текущем окне запроса под имеющимся текстом введите следующую команду и выполните сценарий целиком для запуска диалога и отправки сообщения в диалог.

```
SEND ON CONVERSATION @dialog_handle  
MESSAGE TYPE AWNewNotice;
```

3. Откройте окно создания запроса, введите и выполните следующую команду для просмотра очереди AWNewNoticeQueue.

```
SELECT * FROM AWNewNoticeQueue;
```

4. В текущем окне запроса, под имеющимся текстом введите, выделите и выполните следующую команду, получающую сообщения и просматривающую данные XML.

```
SELECT * FROM AWNewNoticeQueue;  
  
DECLARE @dialog_handle UNIQUEIDENTIFIER, @XMLdata XML;  
  
SET @XMLdata = (SELECT * FROM sys.tables FOR XML AUTO);  
  
RECEIVE TOP (1) @dialog_handle = conversation_handle  
FROM AWNewNoticeQueue;  
  
END CONVERSATION @dialog_handle;  
  
SELECT @XMLdata;
```

5. Выполните инструкцию `SELECT * FROM AWNewNoticeQueue` еще раз, чтобы убедиться в том, что строка уже не в очереди.
6. Сохраните свои сценарии и закройте среду SSMS.

Резюме урока

- ◆ Компонент Service Broker предоставляет вашему экземпляру SQL Server надежный асинхронный обмен сообщениями.
- ◆ Выполнение своих решений потребует настройки компонентов Service Broker. К ним могут относиться типы сообщений, контракты, службы, очереди, диалоги и приоритеты последних.
- ◆ Для управления отдельными диалогами между двумя службами применяются команды `BEGIN DIALOG`, `SEND` и `RECEIVE`.

Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

Резюме

- ◆ Версия SQL Server 2008 предоставляет два новых пространственных типа данных. Типы данных `geometry` и `geography` могут применяться для управления пространственными данными, такими как координаты местоположения классной комнаты в школе или долгота и широта местоположения школы.
- ◆ Полностью интегрированный механизм полнотекстового поиска позволяет задавать расширенный лингвистический поиск для обнаружения совпадающих строк, содержащих различные типы данных.
- ◆ Компонент Service Broker предоставляет надежную систему асинхронного обмена сообщениями, которая может обеспечить решения как для одной базы данных, так и для нескольких экземпляров серверов.

Ключевые термины

- ◆ Тип данных *geography*.
- ◆ Тип данных *geometry*.
- ◆ Открытый геопространственный консорциум (Open Geospatial Consortium, OGC).
- ◆ Well-known text (WKT).
- ◆ Well-known binary (WKB).
- ◆ Методы.
- ◆ Расширенные методы OGC.
- ◆ Создание экземпляра (*instantiation*).
- ◆ Идентификатор пространственной ссылки (*Spatial reference identifier, SRID*).
- ◆ Полнотекстовый обработчик (*Full-Text Engine*).
- ◆ Процесс узла управляющей программы фильтрации (*Filter daemon host process, Fdhost.exe*).
- ◆ Служба запуска FDHOST (*SQL Full-Text Filter Daemon Launcher, Fdlauncher.exe*).
- ◆ Заполнение (*Population, crawl*).
- ◆ Элемент или выражение (*Term*).
- ◆ Фильтр.
- ◆ Список стоп-слов.
- ◆ Стоп-слово.
- ◆ Тезаурус.
- ◆ Модуль парадигм (*Stemmer*).
- ◆ Маркер (*Token*).
- ◆ Средство разбиения по словам (*Word breaker*).
- ◆ Полнотекстовый индекс.
- ◆ Полнотекстовый диалог.
- ◆ Диалог.
- ◆ *Conversation group*.
- ◆ Приоритет диалога.
- ◆ Сообщение.
- ◆ Очередь.
- ◆ Служба.

- ◆ Контракт.
- ◆ Тип сообщения.
- ◆ Маршрут.
- ◆ Remote service binding.
- ◆ Конечная точка Service Broker (endpoint).

Сценарии

В следующих конкретных сценариях вы примените все, чему научились в этой главе. Ответы на предложенные вопросы можно найти в разд. "Ответы к сценариям" в конце книги.

Сценарий 1. Создание экземпляров пространственных данных

Вы — разработчик базы данных для компании Wide World Importers. Недавно ваша компания установила новую версию SQL Server 2008, и руководство хотело бы получить выигрыш от использования новых пространственных типов данных для улучшения управления импортом.

Руководству представляется необходимой реализация двух разных функций. Первая должна обеспечить разработку двумерной модели складских помещений компании. Руководство хотело бы отслеживать местонахождение конкретных категорий товаров в хранилище. Местонахождение следует задавать координатами X и Y и единицей измерения, в которой заданы координаты.

Вторая функция включает в себя отслеживание физического местоположения каждого склада, а также региона, который снабжается этим складом. Вы хотите применять картографическую программу для отображения местоположений складов и закрашивания регионов, обслуживаемых этим складом.

Ответьте на следующий вопрос вашего менеджера: "Какой тип данных и метод создания экземпляра следует применить в каждой из требуемых функций?"

Сценарий 2. Запросы полнотекстового индекса

Вы — разработчик базы данных для компании Litware, Inc. Litware изучила множество документов и теперь хранит их в столбцах типа `varbinary(max)` базы данных LitwareDoc. Это документы Microsoft Office Word и Microsoft Office Excel. Администратор базы данных настроил полнотекстовые индексы в соответствующих таблицах и включил в каждую из них надлежащие столбцы. Вам необходимо определить запросы, которые извлекают имя и местонахождение документов, содержащих общие фразы. Вы должны иметь возможность запросить слово, такое как *author* (автор), и также извлекать строки, содержащие слова *writer* (писатель), *contributor* (автор статьи). Вам необходимо сопоставлять строки, близкие по смыслу, а не только содержащие одни и те же выражения. Вы не

хотите искать совпадения для слова *Litware*, даже если оно введено в выражении для поиска; вы хотите его игнорировать, поскольку оно встречается почти в каждом документе. Вам также требуется вернуть самые подходящие строки в начале результирующего набора.

Ответьте на следующие вопросы вашего менеджера:

1. "Нужно ли провести дополнительную настройку, чтобы ваши запросы выполнялись корректно?"
2. "Какая команда обеспечит нужные вам функциональные возможности?"

КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

Реализация типов данных

Упражнение 1. Добавьте в тестовую базу данных типы данных *geometry* и *geography*. Попрактикуйтесь в создании экземпляров пространственных данных разными пространственными методами.

Выполнение полнотекстового поиска

Упражнение 2. Напишите и выполните различные команды с предикатами и функциями *CONTAINS*, *CONTAINSTABLE*, *FREETEXT* и *FREETEXTTABLE*. Внимательно изучите результирующие наборы.

Упражнение 3. Измените файл тезауруса и создайте пользовательский список стоп-слов. Выполните еще раз команды, написанные вами в упражнении 2, и посмотрите, как эти изменения повлияли на ваши данные. Кроме того, попытайтесь выполнить команду с часто встречающимся словом, например *to* (*в*), как с использованием списка стоп-слов, так и без него.

Реализация решений Service Broker

Упражнение 4. Создайте компоненты, необходимые для запуска диалога Service Broker, запустите диалог, отправьте и получите сообщения.

Упражнение 5. Добавьте в решение, реализованное в *уроке 3*, отправку сообщения с подтверждением получения из службы *AWAckService* в службу *AWNewNoticeService*.

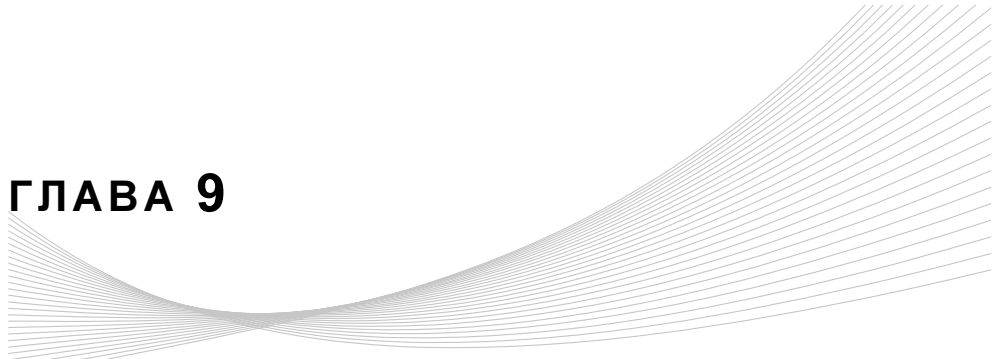
Тренировочный тест

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя

по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобия будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

Дополнительная информация о тренировочных тестах

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.



ГЛАВА 9

Знакомство со средствами, облегчающими управление Microsoft SQL Server

Вам как разработчику баз данных может потребоваться расширить решения для ваших баз данных, перейдя от простого извлечения запрашиваемых данных к приложению с предварительной обработкой данных (front-end application). Например, вам может понадобиться пересылать по электронной почте отчеты пользователям или запрашивать метаданные из системы. В этой главе обсуждается несколько средств в Microsoft SQL Server 2008, позволяющих эффективно распространять данные вашей базы данных и управлять ими. К этим средствам относятся Database Mail, Windows PowerShell и отслеживание изменений данных. Компонент Database Mail появился в версии SQL Server 2005, и в SQL Server 2008 продолжена поддержка этого средства, позволяющего серверам с запущенным SQL Server обмениваться сообщениями по электронной почте. В SQL Server 2008 корпорация Microsoft включила оболочку Windows PowerShell для управления технологиями, поддерживаемыми SQL Server Management Objects (SMO). Кроме того, у разработчиков баз данных появились новые средства, облегчающие отслеживание изменений данных.

Экзаменационные темы в этой главе

- ◆ Интеграция Database Mail.
- ◆ Реализация сценариев средствами Windows PowerShell и SQL Server Management Objects (SMO).
- ◆ Отслеживание изменений данных.

Уроки в этой главе

- ◆ Урок 1. Интеграция Database Mail.

- ◆ Урок 2. Реализация сценариев средствами Windows PowerShell.
- ◆ Урок 3. Отслеживание изменений данных.

Прежде чем начать

Для усвоения уроков этой главы необходимо:

- ◆ знать базовые конструкции языка манипулирования данными (DML), такие как INSERT, UPDATE и DELETE;
- ◆ иметь базовые знания о представлениях, хранимых процедурах и функциях, применяемых в SQL Server;
- ◆ уметь открывать и выполнять запросы в среде SQL Server Management Studio (SSMS);
- ◆ установить на вашем компьютере версию SQL Server 2008 Developer Edition, Enterprise Edition или Enterprise Evaluation Edition с учебными базами данных AdventureWorks2008 и AdventureWorksDW2008;
- ◆ иметь сервер SMTP (Simple Mail Transfer Protocol, протокол электронной почты Интернета) с двумя действующими адресами электронной почты, один для учетной записи службы SQL Server Service, а другой для получения тестовых сообщений электронной почты.

УРОК 1. Интеграция Database Mail

В версию SQL Server 2005 корпорация Microsoft включила компонент Database Mail, позволяющий службе SQL Server отправлять сообщения электронной почты. Применяя Database Mail, можно разрабатывать приложения баз данных, которые посылают пользователям результаты запросов и файлы-вложения. В этом уроке дается краткий обзор и пояснения, касающиеся настройки Database Mail, за которыми следует подробное описание вариантов применения системной хранимой процедуры `sp_send_dbmail`.

После усвоения этого урока вы сможете

- Разобраться в процессе настройки Database Mail.
- Отправлять сообщения электронной почты с помощью `sp_send_dbmail`.
- Составить представление об основах управления компонентом Database Mail.

Расчетное время урока — 45 мин.

РЕАЛЬНАЯ ИСТОРИЯ.

Анна Вебер (Ann Weber)

Недавно я обсуждала с клиентом варианты автоматической отправки пользователям его базы данных очень простых недельных отчетов. Отчеты не требовали спе-

циального форматирования, но должны были создаваться и доставляться пользователям до начала рабочего дня. Для удовлетворения нужд клиента мы смогли использовать службу SQL Server Agent с компонентом Database Mail.

Для каждого набора отчетов, нуждавшихся в рассылке, было создано задание. Затем мы разработали этап задания, применяющий системную хранимую процедуру `sp_send_dbmail` для выполнения запроса к базе данных и отправки результатов заданной группе. Поскольку запросы были уже готовы и использовали для извлечения текущей даты функцию типа `datetime`, настройка Database Mail и SQL Server Agent была быстрой и безболезненной.

Обзор компонента Database Mail

Database Mail позволяет администраторам и разработчикам отправлять сообщения электронной почты, сгенерированные службой SQL Server. В версию SQL Server 2005 компонент Database Mail был включен для замены компонента SQL Mail. В версии SQL Server 2008 SQL Mail сохранен только для обратной совместимости. Одно из главных достоинств Database Mail — умение обмениваться сообщениями с помощью протокола SMTP и отсутствие потребности в приложении электронной почты, совместимом с протоколом Extended MAPI, например Microsoft Office Outlook, на сервере с действующим SQL Server. Кроме того, Database Mail обеспечивает высокую отказоустойчивость за счет поддержки множественных SMTP-серверов и множественных пользовательских учетных записей SMTP и профилей пользователя.

Настройка Database Mail

Несмотря на то, что традиционно настройка Database Mail — это задача администратора базы данных, далее для справки приводится ее краткий обзор. Для запуска мастера настройки Database Mail Configuration Wizard можно использовать среду SSMS.

Включение Database Mail

Для минимизации контактной зоны (security footprint) вашего сервера компонент Database Mail по умолчанию отключен. Если вы задаете Database Mail, используя диспетчер настройки (Database Mail Configuration Manager), включение Database Mail становится частью процесса настройки. Кроме того, для включения Database Mail на одном или нескольких серверах можно применить средство настройки контактной зоны механизма управления на основе политики (facet Surface Area Configuration Policy-Based Management) или системную хранимую процедуру `sp_configure`.

Для настройки компонента Database Mail необходимо выполнить следующие действия.

1. Откройте среду SSMS и установите соединение с экземпляром SQL Server, который хотите настроить.
2. В Object Explorer (Обозреватель объектов) раскройте папку **Management** (Управление), щелкните правой кнопкой мыши компонент **Database Mail** и

выберите команду **Configure Database Mail** (Настроить Database Mail), как показано на рис. 9.1.

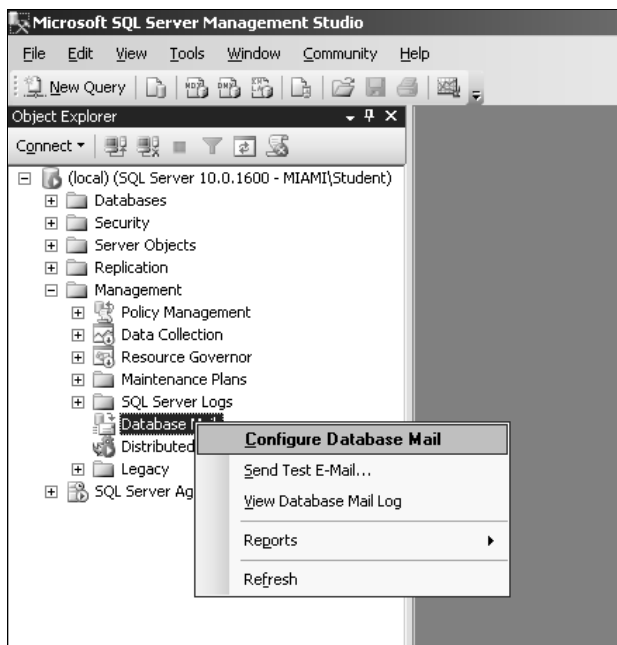


Рис. 9.1. Вариант настройки Database Mail

3. Просмотрите страницу приветствия мастера настройки **Mail Configuration Wizard Welcome** и щелкните мышью кнопку **Next** (Далее).
4. На странице **Select Configuration Task** (Выбор задачи конфигурации) убедитесь, что выбран вариант **Set up Database Mail by performing the Following Tasks** (Установка компонента Database Mail, при помощи выполнения следующих задач), и щелкните мышью кнопку **Next**.

Управление Database Mail

После завершения настройки Database Mail мастер настройки Database Mail Configuration Wizard можно применять для управления параметрами настройки.

5. Если предупреждение среды SSMS уведомит вас о том, что компонент Database Mail не включен, щелкните мышью кнопку **Yes**, чтобы его включить.

Потребность в Service Broker

Если в базе данных msdb был отключен компонент Service Broker, перед включением Database Mail следует остановить службу SQL Server Agent (если она запущена) и включить Service Broker в базе данных msdb.

6. На странице **New Profile** (Создание профиля) введите в соответствующие поля информативное имя профиля и его описание.
7. В области **SMTP Accounts** (Учетные записи SMTP) щелкните мышью кнопку **Add** (Добавить) для создания новой учетной записи Database Mail.
8. В окне **New Database Mail Account** (Создание учетной записи Database Mail) введите данные, соответствующие учетной записи SMTP, которую будет применять служба SQL Server, и затем щелкните мышью кнопку **OK**. На рис. 9.2 показан простой пример. Замените эти данные информацией об учетной записи, подходящей для вашего сервера.

The screenshot shows the 'New Database Mail Account' dialog box. It has a title bar with a close button. The main area contains the following fields and options:

- Specify name, description, and attributes for your SMTP account.**
- Account name:** SQLService
- Description:** Main SQL Service Account
- Outgoing Mail Server (SMTP)**
- E-mail address:** sqlservice@adventure-works.com
- Display name:** SQLService
- Reply e-mail:** (empty field)
- Server name:** localhost
- Port number:** 25
- ☐ This server requires a secure connection (SSL)
- SMTP Authentication**
- ☐ Windows Authentication using Database Engine service credentials
- ☐ Basic authentication
 - User name:** (empty field)
 - Password:** (empty field)
 - Confirm password:** (empty field)
- ☒ Anonymous authentication
- Buttons:** OK, Cancel, Help

Рис. 9.2. Пример вводимой информации на странице **New Database Mail Account**

9. На странице **New Profile** (Создание профиля) проверьте вашу конфигурационную информацию и щелкните мышью кнопку **Next**.
10. На странице **Manage Profile Security** (Управление безопасностью профиля) настройте подходящие открытые и личные профили и щелкните мышью кнопку **Next**.

Дополнительная информация о параметрах безопасности профиля

Дополнительные сведения об определении подходящих параметров обеспечения безопасности профиля см. в разд. "Manage Profile Security, Public Tab (Database Mail)" ("Управление безопасностью профиля, вкладка "Открытый" (компонент Database Mail)") и "Manage Profile Security, Private Tab (Database Mail)" (Управление профилем безопасности, вкладка "Личное" (компонент Database Mail)) в электронной документации по SQL Server.

11. На странице **Configure System Parameters** (Настройка параметров системы) просмотрите системные установки по умолчанию, выполните все необходимые изменения и щелкните мышью кнопку **Next**.
12. На странице **Complete the Wizard** (Завершение работы мастера) щелкните мышью кнопку **Finish** (Готово).
13. Убедитесь в том, что настройка удачна, и щелкните мышью кнопку **Close** (Закреть).

Дополнительная информация о настройке Database Mail

Более подробные сведения о включении и настройке компонента Database Mail см. в разд. "Database Mail Configuration Wizard" (Мастер настройки компонента Database Mail) электронной документации по SQL Server.

Отправка сообщений Database Mail

Вы можете применять Database Mail для встраивания в ваши приложения сообщений электронной почты. Например, можно вставить системную хранимую процедуру `sp_send_dbmail` в конец хранимой процедуры, которую ваше приложение вызывает для добавления нового клиента в базу данных кредитов, и послать по электронной почте сообщение с информацией, добавленной для нового клиента, управляющему кредитной политикой.

Системная хранимая процедура `sp_send_dbmail` используется для отправки сообщений из службы SQL Server получателям электронной почты. У этой команды есть параметры, позволяющие задать стандартные поля сообщения электронной почты, такие как **To** (Кому), **CC** (Копия), **BCC** (Скрытая) и **Subject** (Тема), а также дополнительные параметры, относящиеся к SQL Server (например, запрос, который нужно выполнить) и обеспечивающие гибкость при разработке приложений SQL Server, нуждающихся в рассылке сообщений электронной почты. Системная хранимая процедура `sp_send_dbmail` включает в себя следующие аргументы.

- ◆ `@profile_name` — имя профиля электронной почты, из которого посылается сообщение. Если не существует профиля электронной почты по умолчанию, вы должны задать аргумент `@profile_name`. Если не указано никакого имени профиля, система сначала пытается использовать личный профиль текущего

пользователя, применяемый по умолчанию. Если такого профиля у текущего пользователя нет, система использует открытый профиль, заданный по умолчанию для базы данных msdb.

- ◆ @recipients — адреса получателей сообщения электронной почты. Весь список адресов электронной почты следует заключить в апострофы и разделить в нем адреса точками с запятой (;). Это поле необязательное, но если поля @recipients, @copy_recipients и @blind_copy_recipients остаются пустыми, команда завершается аварийно.
- ◆ @copy_recipients — адреса получателей электронной почты в поле сообщения **СС** (Копия). Весь список адресов электронной почты следует заключить в апострофы и разделить в нем адреса точками с запятой (;). Это поле необязательное, но если поля @recipients, @copy_recipients и @blind_copy_recipients остаются пустыми, команда завершается аварийно.
- ◆ @blind_copy_recipients — адреса получателей электронной почты в поле сообщения **ВСС** (Скрытая). Весь список адресов электронной почты следует заключить в апострофы и разделить в нем адреса точками с запятой (;). Это поле необязательное, но если поля @recipients, @copy_recipients и @blind_copy_recipients остаются пустыми, команда завершается аварийно.
- ◆ @subject — содержимое, которое должно быть внесено в поле **Subject** (Тема), заголовок сообщения электронной почты. Если тема не задана, это поле будет содержать 'SQL Server Message'. Содержимое поля **Subject** следует заключать в одинарные кавычки.
- ◆ @body — включает в себя содержимое (тело) сообщения; аргумент следует заключать в апострофы.
- ◆ @body_format — тип форматирования тела сообщения: 'TEXT' или 'HTML'. По умолчанию тип форматирования — 'TEXT'.
- ◆ @importance — степень важности сообщения: 'Low' (низкая), 'Normal' (обычная) или 'High' (высокая). Если не указана, подразумевается 'Normal'.
- ◆ @sensitivity — степень конфиденциальности сообщения: 'Normal' (обычное), 'Personal' (личное), 'Private' (не подлежащее огласке) или 'Confidential' (секретное). Значение по умолчанию — 'Normal'.
- ◆ @file_attachments — разделенный точками с запятой список имен файлов, которые вы хотите вложить в сообщение. Весь список следует заключить в апострофы.
- ◆ @query — запрос, выполняемый системой. Вы можете включить результаты запроса в тело сообщения электронной почты или как файл-вложение. Весь запрос заключается в апострофы.
- ◆ @execute_query_database — база данных для выполняемого запроса. Этот аргумент игнорируется, если не задан аргумент @query.

- ◆ `@attach_query_result_as_file` — способ вставки результатов запроса: в тело сообщения или в виде вложения. Значение 1 задает применение файла вложения. Значение 0 указывает на то, что результаты запроса должны следовать за содержимым, заданным в аргументе `@body`. Данный аргумент игнорируется, если не задан аргумент `@query`. Если этот аргумент не указан, результаты включаются в тело сообщения электронной почты.
- ◆ `@query_attachment_filename` — имя файла вложения с результатами запроса. Этот аргумент следует заключать в одинарные кавычки, если имя файла содержит расширение (например, `txt`) или другие запрещенные символы. Если аргумент `@attach_query_result_as_file` равен 1 и данный параметр не задан, Database Mail задает имя файла по умолчанию. Параметр `@query_attachment_filename` игнорируется, если аргумент `@attach_query_result_as_file` не задан или равен 0, или если не определен аргумент `@query`.
- ◆ `@query_result_header` — возможные значения 1 или 0, задающие включение или исключение заголовков столбцов в результирующий набор. По умолчанию равен 1, что означает включение заголовков столбцов. Данный аргумент игнорируется, если не задан аргумент `@query`.
- ◆ `@query_result_width` — количество символов, которые должны вставляться в одну строку при форматировании результирующего набора. По умолчанию равен 256 символам, но вы можете задать любое число из диапазона 10—32 767. Этот аргумент игнорируется, если не задан `@query`.
- ◆ `@query_result_separator` — символ, который следует применять для разделения столбцов в результирующем наборе. Значение по умолчанию — пробел.
- ◆ `@exclude_query_output` — устанавливается равным 0 или 1 для задания отображения или неотображения на мониторе сообщений об ошибках, подобных приведенному на рис. 9.3. Значение 0 включает вывод на монитор сообщений об ошибках, а значение 1 означает вывод на монитор только сообщения об успешном завершении команды, даже если запрос в хранимой процедуре завершился аварийно.
- ◆ `@append_query_error` — указывает, посылается ли сообщение в случае возникновения ошибки в запросе. Значение по умолчанию 0 предполагает, что Database Mail не отправляет сообщение в случае аварийного завершения запроса. Если значение аргумента равно 1, Database Mail отправляет сообщение и добавляет сообщение об ошибке в сообщение электронной почты (рис. 9.4). Этот параметр также влияет на поведение аргумента `@exclude_query_output`.
- ◆ `@query_no_truncate` — задается равным 0 или 1, определяет, усекаются ли в результирующем наборе столбцы большой переменной длины (*large variable-length*). Значение по умолчанию 0 усекает столбцы до 256 символов. Если параметр равен 1, столбцы с большой переменной длиной не усекаются, заголовки столбцов не включаются и для выполнения запроса требуются допол-

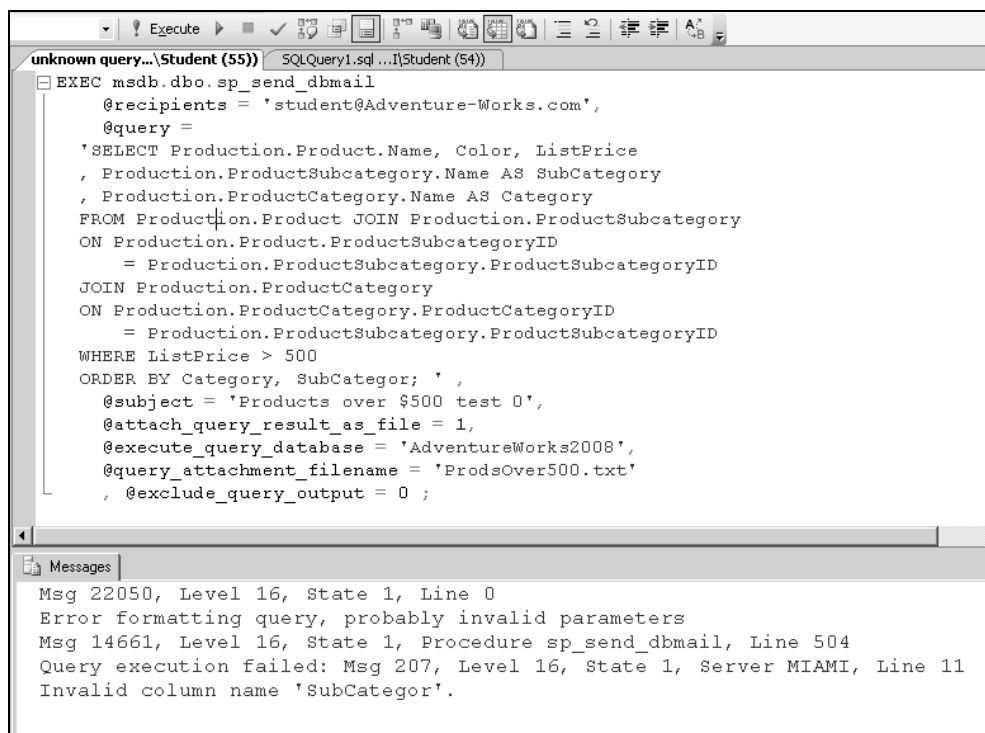


Рис. 9.3. Пример сообщения об ошибке в запросе

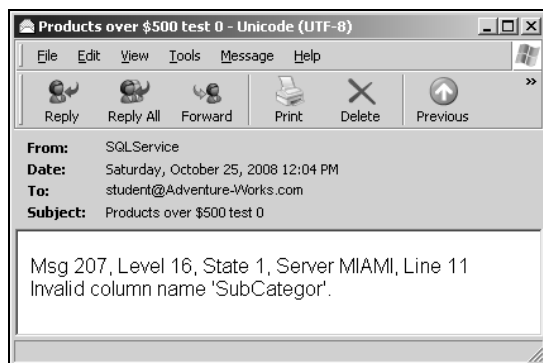


Рис. 9.4. Пример добавления сообщения об ошибке

нительные ресурсы. На рис. 9.5 и 9.6 показана разница в длине строки, если столбец Resume (Резюме) определен, как данные типа XML, и посылается в виде вложенного текстового файла. На рис. 9.5 показан результат выполнения с аргументом @query_no_truncate, равным 0, а на рис. 9.6 — с аргументом @query_no_truncate, равным 1.

- ◆ @mailitem_id[OUTPUT] — задает необязательную выходную переменную для возврата mailitem_id сообщения.

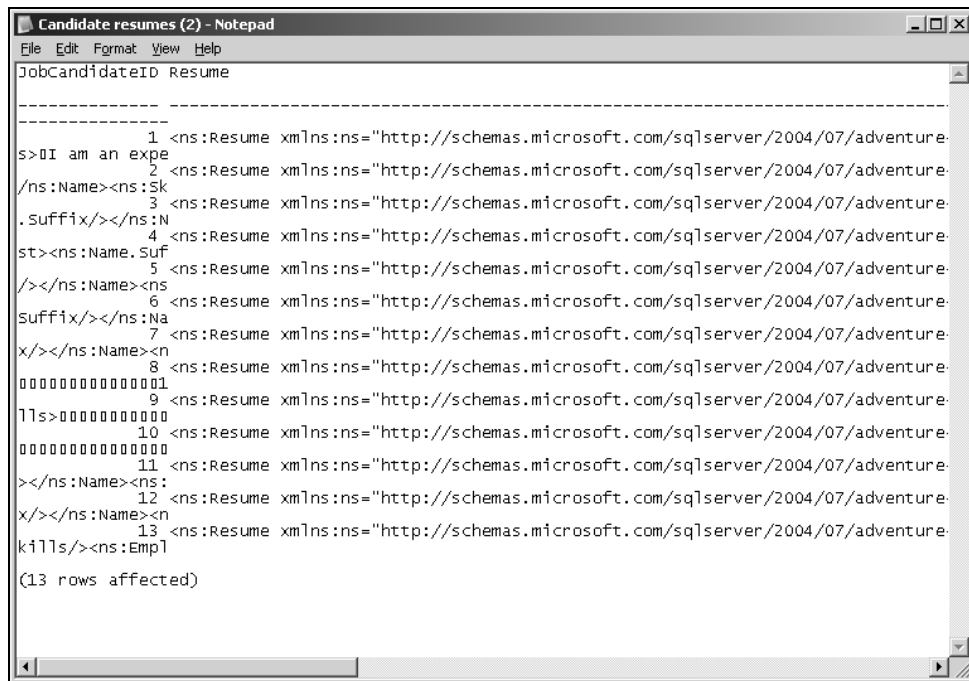


Рис. 9.5. Усеченный столбец Resume

Следующий запрос посылает по адресу **student@Adventure-Works.com** сообщение электронной почты с темой "Job Candidate Resumes" (Резюме соискателей работы). Сообщение содержит файл вложения **Candidate_resumes.txt**. В это вложение включены результаты запроса, возвращающего столбцы **JobCandidateID** (ID соискателя работы) и **Resume** (Резюме) из таблицы **HumanResources.JobCandidate** базы данных **AdventureWorks2008**. Данные типа **XML**, извлекаемые из столбца **Resume**, не обрезаются. Генерируемый текстовый файл см. на рис. 9.6.

```
EXEC msdb.dbo.sp_send_dbmail
    @recipients = 'student@Adventure-Works.com',
    @query =
    'SELECT JobCandidateID,
    Resume from HumanResources.JobCandidate',
    @subject = 'Job Candidate Resumes',
    @execute_query_database = 'AdventureWorks2008',
    @attach_query_result_as_file = 1,
    @query_attachment_filename = 'Candidate_resumes.txt',
    @query_no_truncate = 1;
```


фикации и устранения конфликтов конфигурационных параметров, влияющих на сообщения, отправляемые из ваших приложений.

- ◆ `sysmail_configure_sp` — настраивает параметры, такие как максимальный допустимый размер файла-вложения, запрещенные расширения имен файлов, которые нельзя посылать как вложения, и параметры повторных попыток отправки сообщения.
- ◆ `sysmail_help_configure_sp` — отображает текущие установочные параметры Database Mail.
- ◆ `sysmail_help_queue_sp` — выводит информацию о состоянии и очередях электронной почты. Эту хранимую процедуру можно использовать для выявления неотправленных сообщений.
- ◆ `sysmail_delete_mailitems_sp` — окончательно удаляет почтовые сообщения из таблиц Database Mail базы данных msdb. Эти сообщения могут быть удалены, исходя из их состояния или даты их отправки. Вложения удаляемых сообщений также удаляются, но связанные с ними регистрируемые события (associated logged events) должны удаляться независимо с помощью процедуры `sysmail_delete_log_sp`.
- ◆ `sysmail_delete_log_sp` — удаляет насовсем записи из журналов Database Mail. Эти записи могут быть удалены на основе их состояния или даты отправки связанного с ними сообщения. Сообщения электронной почты, связанные с удаляемыми записями журналов, должны удаляться независимо с помощью процедуры `sysmail_delete_mailitems_sp`.
- ◆ `sysmail_start_sp` — запускает компонент Database Mail с помощью запуска связанных объектов Service Broker.
- ◆ `sysmail_stop_sp` — останавливает Database Mail с помощью остановки связанных объектов Service Broker. Эту хранимую процедуру можно применять для поиска неисправностей с помощью временной приостановки обработки сообщений в очередях Database Mail. Хранимая процедура все еще продолжает работать, несмотря на то, что объекты Service Broker остановлены.

Дополнительная информация о хранимых процедурах

Подробную информацию обо всех хранимых процедурах компонента Database Mail см. в разд. "Database Mail and SQL Mail Stored Procedures (Transact-SQL)" (Хранимые процедуры служб Database Mail и SQL Mail (Transact-SQL)) электронной документации по SQL Server.

Быстрая проверка

ВОПРОС:

Ваша компания применяет Database Mail в SQL Server 2008. Вам необходимо послать менеджеру по продажам почтовое сообщение, содержащее результаты запроса. Какую команду следует применить для генерации и отправки сообщения электронной почты?

ОТВЕТ: следует использовать хранимую процедуру `sp_send_dbmail`.

ПРАКТИЧЕСКОЕ ЗАНЯТИЕ

На этом практическом занятии вы настроите компонент Database Mail в вашем экземпляре SQL Server. Затем вы настроите параметры электронной почты и отправите сообщение Database Mail из SQL Server.

Упражнение 1. Настройка Database Mail

Если в вашей системе компонент Database Mail не настроен, выполните следующие действия для его настройки.

1. Откройте среду SSMS.
2. В Object Explorer (Обозреватель объектов) раскройте папку **Management** (Управление), щелкните правой кнопкой мыши строку **Database Mail**, а затем щелкните кнопкой мыши команду **Configure Database Mail** (Настройка Database Mail).
3. Просмотрите информацию на странице приветствия мастера настройки **Mail Configuration Wizard Welcome** и щелкните мышью кнопку **Next**.
4. На странице **Select Configuration Task** (Выбор задачи конфигурации) убедитесь, что выбран вариант **Set up Database Mail by Performing the Following Tasks** (Установка компонента Database Mail при помощи выполнения следующих задач), и щелкните мышью кнопку **Next**.
5. Если предупреждающее сообщение среды SSMS уведомит вас о том, что компонент Database Mail не включен, щелкните мышью кнопку **Yes** для того, чтобы включить его.
6. На странице **New Profile** (Создание профиля) введите в соответствующие поля информативное имя профиля и его описание.
7. В разделе **SMTP Accounts** (Учетные записи SMTP) щелкните мышью кнопку **Add** (Добавить) для создания новой учетной записи Database Mail.
8. В окне **New Database Mail Account** (Создание учетной записи Database Mail) введите данные, соответствующие учетной записи SMTP, которую будет использовать служба SQL Server, и щелкните мышью кнопку **OK**. Для того чтобы учетная запись стала действующей, следует задать имя учетной записи, адрес электронной почты, корректное имя сервера и номер порта для отправляемых вами почтовых сообщений.
9. На странице **New Profile** проверьте ваши конфигурационные данные и затем щелкните мышью кнопку **Next**.
10. На странице **Manage Profile Security** (Управление безопасностью профиля), на вкладке **Public Profiles** (Открытые профили), установите флажок **Public** (Открытый), находящийся рядом с именем вашего профиля, выберите вари-

ант **Yes** (Да) в раскрывающемся списке в столбце **Default Profile** (Профиль по умолчанию) и щелкните мышью кнопку **Next**.

Снижение безопасности

Использование профиля SQLService в качестве открытого профиля по умолчанию — не лучший выбор с точки зрения обеспечения безопасности. Мы применили его здесь, чтобы упростить процесс установки.

11. На странице **Configure System Parameters** (Настройка параметров системы) просмотрите информацию и щелкните мышью кнопку **Next**.
12. На странице **Complete the Wizard** (Завершение работы мастера) щелкните мышью кнопку **Finish** (Готово).
13. Убедитесь в том, что настройка удачна, и щелкните мышью кнопку **Close** (Заккрыть).
14. Оставьте среду SSMS открытой для следующего упражнения.

Упражнение 2. Проверка настройки Database Mail

В этом упражнении вы проверите настройку Database Mail, отправив по электронной почте тестовое сообщение.

1. В Object Explorer (Обозреватель объектов) раскройте папку **Management** (Управление) щелкните правой кнопкой мыши строку **Database Mail** и выберите команду **Send Test E-Mail** (Отправка тестового сообщения).
2. В окне **Send Test E-Mail From <server name>** (Отправка тестового сообщения от <имя сервера>) выберите из списка ваш профиль **Database Mail Profile**, введите в поле **To** (Кому) адрес электронной почты, по которому хотите отправить тестовое сообщение, просмотрите информацию в полях **Subject** (Тема) и **Body** (Тело сообщения) и щелкните мышью кнопку **Send Test E-mail** (Отправка тестового сообщения). На экране появится диалоговое окно **Database Mail Test E-mail** (Тестовое сообщение компонента Database Mail) с подтверждением того, что ваше сообщение помещено в очередь.
3. Подождите несколько минут и затем проверьте почту. Если вы получите сообщение электронной почты, щелкните мышью кнопку **OK** для закрытия диалогового окна или **Troubleshoot** (Диагностика...) для открытия электронной документации по SQL Server с советами по выявлению и устранению неисправностей.
4. Оставьте среду SSMS открытой для следующего упражнения.

Упражнение 3. Написание сообщений Database Mail

В этом упражнении вы отправите различные почтовые сообщения с помощью разных параметров команды `sp_send_dbmail`.

ВАЖНАЯ ИНФОРМАЦИЯ.

Конфигурационные параметры

Если у вас нет профиля по умолчанию, настроенного для Database Mail, необходимо добавлять параметр @profile_name в каждый вызов процедуры sp_send_dbmail, приведенный в упражнении 3.

1. Для отправки тестового сообщения на вашу учетную запись электронной почты с помощью команды sp_send_dbmail замените параметр @recipients вашим адресом электронной почты и выполните следующий программный код в окне создания запроса.

```
EXEC msdb.dbo.sp_send_dbmail
    @recipients = 'student@Adventure-Works.com'
    , @body = 'The command completed successfully.'
    , @subject = 'Test Automated Success Message' ;
```

Зачем вам использовать это сообщение?

Вы можете использовать простое сообщение, подобное созданному вами в п. 1, как часть более объемной процедуры с множеством шагов. После завершения других шагов можно убедиться в успехе и затем послать простое сообщение, подобное этому, чтобы уведомить менеджера или другого пользователя об успешном завершении процедуры.

2. Откройте окно создания запроса.
3. В окне создания запроса введите и выполните следующую команду для отправки на вашу учетную запись электронной почты почтового сообщения с результатами запроса, включенными в него в виде вложения. Не забудьте заменить параметр @recipients на ваш адрес электронной почты.

```
EXEC msdb.dbo.sp_send_dbmail
    @recipients = 'student@Adventure-Works.com'
    , @query =
    'USE AdventureWorks2008;
    Go
    SELECT Production.Product.Name, Color, ListPrice
      , Production.ProductSubcategory.Name AS SubCategory
      , Production.ProductCategory.Name AS Category
    FROM Production.Product JOIN Production.ProductSubcategory
    ON Production.Product.ProductSubcategoryID =
      Production.ProductSubcategory.ProductSubcategoryID
    JOIN Production.ProductCategory
    ON Production.ProductCategory.ProductCategoryID =
      Production.ProductSubcategory.ProductSubcategoryID
    WHERE ListPrice > 500
    ORDER BY Category, SubCategory; '
    , @subject = 'Products over $500'
    , @attach_query_result_as_file = 1 ;
```

4. Перейдите в вашу программу электронной почты и убедитесь, что получено сообщение с вложением. Откройте и просмотрите файл-вложение. Обратите внимание на информацию в результирующем наборе, касающуюся изменения среды выполнения (контекста) базы данных. Для исключения этой информации вы должны использовать аргумент `@execute_query_database` для задания базы данных AdventureWorks2008.
5. Перейдите в среду SSMS и измените запрос из п. 3, установив значение аргумента `@attach_query_result_as_file` равным 0 и тем самым включив результаты запроса непосредственно в тело сообщения вместо применения файла-вложения. Также задайте в аргументе `@execute_query_database` 'AdventureWorks2008' и удалите из запроса команды `USE database` и `go` для удаления сведений об изменении контекста базы данных из результатов запроса. Выполняемый вами запрос должен выглядеть подобно приведенному далее программному коду.

```
EXEC msdb.dbo.sp_send_dbmail
    @recipients = 'student@Adventure-Works.com'
    , @query =
        'SELECT Production.Product.Name, Color, ListPrice
        , Production.ProductSubcategory.Name AS SubCategory
        , Production.ProductCategory.Name AS Category
        FROM Production.Product JOIN Production.ProductSubcategory
        ON Production.Product.ProductSubcategoryID =
            Production.ProductSubcategory.ProductSubcategoryID
        JOIN Production.ProductCategory
        ON Production.ProductCategory.ProductCategoryID =
            Production.ProductSubcategory.ProductSubcategoryID
        WHERE ListPrice > 500
        ORDER BY Category, SubCategory; '
    , @subject = 'Products over $500'
    , @attach_query_result_as_file = 0
    , @execute_query_database = 'AdventureWorks2008';
```

6. Перейдите в вашу программу электронной почты и убедитесь в том, что сообщение получено и содержит результирующий набор запроса.
 7. Вернитесь в среду SSMS, измените и выполните запрос из п. 3 и 5 так, чтобы посылался файл-вложение с именем файла ProdsOver500.txt. Для этого параметр `@attach_query_result_as_file` задайте равным 1 и вставьте следующий программный код после последней строки набранного кода, но перед точкой с запятой.
- ```
 , @query_attachment_filename = 'ProdsOver500.txt'
```
8. Перейдите в вашу программу электронной почты и убедитесь в том, что сообщение получено и содержит результирующий набор запроса.

## Резюме урока

- ◆ Компонент Database Mail был введен в версии SQL Server 2005 и должен применяться вместо службы SQL Mail.
- ◆ Компонент Database Mail по умолчанию выключен для минимизации контактной зоны сервера.
- ◆ Для интеграции ваших приложений с Database Mail следует применять системную хранимую процедуру `sp_send_dbmail`.
- ◆ Широкий набор аргументов процедуры позволяет настраивать почтовые сообщения и вложения, посылаемые с сервера баз данных.

## УРОК 2. Реализация сценариев средствами Windows PowerShell

Как разработчик вы можете применять в SQL Server 2008 оболочку PowerShell для автоматизации процесса развертывания ваших приложений. Вы также можете использовать ее для автоматизации перечислений объектов базы данных и свойств объекта. Кроме того, из оболочки SQL Server PowerShell можно запускать программу `Sqlcmd`, позволяющую выполнять любые корректные команды языка SQL из рабочей среды Windows PowerShell.

---

### ***После усвоения этого урока вы сможете***

---

- Составить представление о функциональных возможностях SQL Server PowerShell.
- Перемещаться по иерархии SQL Server PowerShell.
- Применять SQL Server PowerShell для перечисления объектов.

Расчетное время урока — 45 мин.

## Что такое Windows PowerShell?

Windows PowerShell — это оболочка командной строки и среда создания сценариев, позволяющая с помощью создания надежных сценариев автоматизировать административные задачи и задачи разработки. Поскольку Windows PowerShell функционирует во многих приложениях Microsoft, освоив этот распространенный язык написания сценариев, вы сможете применять его для управления многочисленными серверами и другими программными продуктами.

Во время установки SQL Server 2008 программа устанавливает Windows PowerShell 1.0 (если она не установлена), поставщика SQL Server PowerShell, набор командлетов PowerShell и утилиту `sqlps`, обеспечивая таким образом SQL Server возможностью функционировать в рабочей среде PowerShell. С помощью SQL Server PowerShell вы сможете писать сценарии, которые затем можно запус-



катель по расписанию как задания SQL Server Agent, используя команду **Start PowerShell** в среде SSMS или выполняя приложение рабочей среды SQL Server PowerShell, такое как утилита sqlps или пользовательское приложение.

### ***Поддержка Windows PowerShell***

В SQL Server 2008 поддержка Windows PowerShell ограничена объектами SMO, относящимися к компонентам Database Engine и Service Broker. Поставщик Windows PowerShell для аналитических служб SQL Server Analysis Services (SSAS) доступен на Web-сайте <http://www.codeplex.com/powerSSAS>.

### ***ПРЕДУПРЕЖДЕНИЕ.***

#### ***Требования к памяти***

Когда вы создаете многошаговые задания Windows PowerShell с помощью определения вашего агента SQL Server Agent, на каждом шаге задания запускается отдельный процесс sqlps. Каждому из этих процессов требуется примерно 20 Мбайт памяти. Если вы запускаете одновременно большое количество шагов задания SQL Server PowerShell, следует протестировать ресурсы и влияние на производительность.

## **Перемещение по иерархии SQL Server PowerShell**

Для представления взаимосвязей объектов на сервере SQL Server использует иерархию. Например, таблица существует в схеме, которая находится в базе данных в экземпляре SQL Server, размещенном на сервере. Оболочка SQL Server PowerShell применяет для представления этой иерархии формат "диск-и-путь" (drive-and-path) Структура вида "диск-и-путь", используемая SQL Server PowerShell, формируется с помощью терминологии и команд, аналогичных тем, которые вы применяете в файловой системе.

Корневой узел для SQL Server — диск SQLSERVER:. На диске SQLSERVER: поставщик SQL Server PowerShell создает три следующие папки:

- ◆ SQLSERVER:\SQL — содержит объекты баз данных, такие как таблицы, представления и хранимые процедуры;
- ◆ SQLSERVER:\SQLPolicy — содержит управляющие объекты на основе политик, такие как политики и аспекты (facets);
- ◆ SQLSERVER:\SQLRegistration — содержит зарегистрированные объекты серверов, такие как группы серверов и зарегистрированные серверы.

В каждом обработчике и папке можно задать путь к объекту, который вы хотите создать, просмотреть или изменить. Папки, вложенные папки и объекты, к которым есть доступ, определяются моделью SMO, входящей в состав версии SQL Server 2008.

### ***Дополнительная информация о справке по SMO***

Полный список объектов SMO можно найти в разд. "SMO Object Model Diagram" (Схема модели объектов SMO) электронной документации по SQL Server.

Если для запуска сеанса SQL Server PowerShell вы применяете Object Explorer (Обозреватель объектов) в среде SSMS, устанавливается путь к объекту, с которого вы начали сеанс (рис. 9.7 и 9.8).

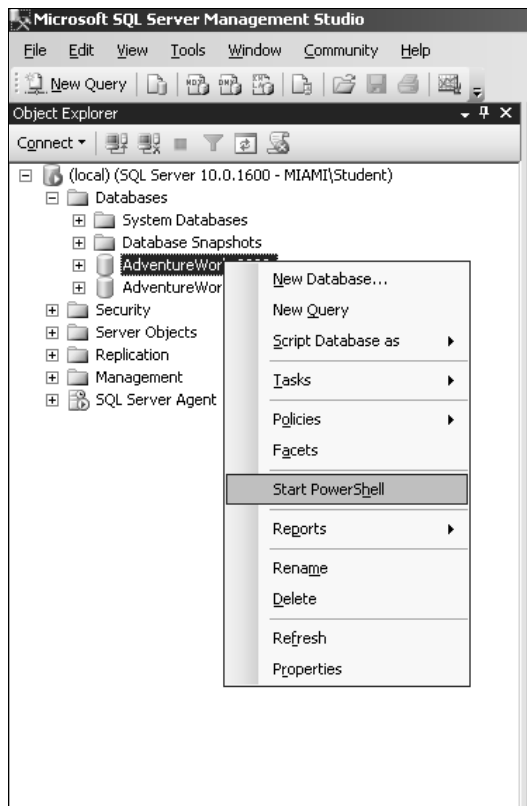


Рис. 9.7. Применение Object Explorer для запуска сеанса SQL Server PowerShell

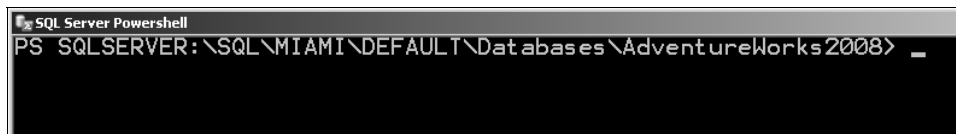


Рис. 9.8. Путь, указывающий местоположение объекта, с которого был запущен сеанс SQL Server PowerShell

Путь SQL Server PowerShell начинается с диска, за которым следует одна из трех поддерживаемых папок. В случае папки SQL, за именем папки следует имя сервера и имя экземпляра. Если вы ссылаетесь на экземпляр по умолчанию, следует задать слово DEFAULT. После имени экземпляра в пути чередуются тип объекта и имя объекта, на который вы ссылаетесь.

Например, путь

SQLSERVER:\SQL\MIAMI\DEFAULT\Databases\AdventureWorks2008\Tables\Person.Address

ссылается на таблицу Address (Адрес) в схеме Person (Частное лицо) в базе данных AdventureWorks2008, в экземпляре по умолчанию, названном MIAMI.

---

### **Быстрая проверка**

#### **ВОПРОС:**

Какую папку следует задать, если вы хотите узнать, какие объекты находятся в конкретной схеме?

**ОТВЕТ:** следует задать папку SQLSERVER:\SQL.

## **Применение SQL Server PowerShell для перечисления объектов**

После того как путь к нужному объекту установлен, можно получить список подчиненных (дочерних) элементов, переместить и переименовать элементы и выполнить множество других действий. На командлеты PowerShell можно ссылаться с помощью их полных имен или любого из их псевдонимов. Далее приведен перечень командлетов, их функций и псевдонимов, назначенных им в версии SQL Server 2008.

- ◆ Get-Location — возвращает имя текущего узла. Псевдонимы: gl, pwd.
- ◆ Set-Location — изменяет текущий узел. Псевдонимы: sl, cd, chdir.
- ◆ Get-ChildItem — перечисляет объекты, хранящиеся в текущем узле. Псевдонимы: gci, dir, ls.
- ◆ Get-Item — возвращает свойства текущего элемента. Псевдоним: gu.
- ◆ Move-Item — перемещает элемент. Псевдонимы: mi, move, mv.
- ◆ Rename-Item — переименовывает объект. Псевдонимы: rni, rn, ren.
- ◆ Remove-Item — удаляет объект. Псевдонимы: ri, del, rd, rm, rmdir.

### **ПРАКТИЧЕСКИЙ СОВЕТ.**

#### **Применение псевдонимов**

---

Несмотря на то, что в интерактивном режиме легче работать с псевдонимами, применение полного имени командлета облегчит чтение и последующее сопровождение сохраняемых сценариев.

При интерактивной работе в сеансе SQL Server PowerShell можно воспользоваться следующими рекомендациями.

- ◆ Используйте клавиши <↑> и <↓> для перемещения по уже выполненным командам.

- ◆ Используйте клавиши <→> и <←> для перемещения по команде и редактирования команды, к которой вы вернулись с помощью клавиш <↑> и <↓> или которую только что ввели.
- ◆ Для минимизации ввода применяйте псевдонимы.
- ◆ Применяйте полный или относительный путь в зависимости от текущего и желаемого пути, как показано в следующих примерах.
  - Если текущий путь  
`SQLSERVER:\SQL\MIAMI\DEFAULT\Databases\AdventureWorks2008\Tables\Person.Address`  
 и вы хотите перейти к вложенной папке Tables, находящейся на один уровень выше текущего пути, можно ввести команду `cd ..` для перехода на один уровень выше.
  - Если текущий путь  
`SQLSERVER:\SQL\MIAMI\DEFAULT\Databases\AdventureWorks2008\Tables\Person.Address`  
 и вы хотите перейти к папке SQLPolicy, быть может, самый легкий вариант ввести `cd \SQLPolicy`.
- ◆ Применяйте команду `cls` для очистки экрана и облегчения чтения вашего результирующего набора.
- ◆ Используйте параметр `-force` для отображения системных объектов, например, схемы `sys` и объектов в ней.
- ◆ Применяйте завершение с помощью клавиши <Tab>, позволяющее ввести часть пути или имя командлета и нажать клавишу <Tab> для получения списка объектов с именами, совпадающими с вашим вводом.

### ***Дополнительная информация об управлении завершением с помощью <Tab>***

Более подробные сведения об использовании завершения с помощью клавиши <Tab> см. в статье "Managing Tab-Completion" (Работа с завершением по клавише TAB) разд. "Using the SQL Server PowerShell Provider" (Использование поставщика SQL Server для PowerShell) электронной документации по SQL Server.

## **Дополнительные командлеты SQL Server PowerShell**

Помимо уже перечисленных командлетов поставщик SQL Server PowerShell включает в себя другие встроенные командлеты для обеспечения дополнительных функциональных возможностей. Разработчикам могут оказаться полезными следующие командлеты.

- ◆ `Get-Help` — предоставляет справочную информацию по каждому командлету. Параметр `-Full` обеспечивает вывод полной технической справки, включая

примеры. На рис. 9.9 показаны результаты справочного экрана для командлета `Invoke-Sqlcmd`.

```

SQL Server Powershell
PS SQLSERVER:\SQL\MIAMI\DEFAULT\Databases> Get-Help Invoke-Sqlcmd

NAME
 Invoke-Sqlcmd

SYNOPSIS
 Runs a script containing statements from the languages (Transact-SQL and XQuery) and commands supported by the SQL Server sqlcmd utility.

SYNTAX
 Invoke-Sqlcmd [-ServerInstance <PSObject>] [-Database <String>] [-EncryptConnection] [-Username <String>] [-Password <String>] [-Query <String>] [-QueryTimeout <Int32>] [-ConnectionTimeout <Int32>] [-ErrorLevel <Int32>] [-SeverityLevel <Int32>] [-MaxCharLength <Int32>] [-MaxBinaryLength <Int32>] [-AbortOnError] [-DedicatedAdministratorConnection] [-DisableVariables] [-DisableCommands] [-HostName <String>] [-NewPassword <String>] [-Variable <String[]>] [-InputFile <String>] [-OutputSqlErrors] [-SuppressProviderContextWarning] [-IgnoreProviderContext] [<CommonParameters>]

DETAILED DESCRIPTION
 Runs a script containing the languages and commands supported by the SQL Server sqlcmd utility. The languages supported are Transact-SQL and the XQuery syntax supported by the Database Engine. Invoke-Sqlcmd also accepts many of the commands supported by sqlcmd, such as GO and QUIT. Invoke-Sqlcmd accepts the sqlcmd scripting variables, such as SQLCMDUSER. Invoke-Sqlcmd does not set sqlcmd scripting variables by default.
 Invoke-Sqlcmd does not support the sqlcmd commands primarily related to interactive script editing. The commands not supported include :!, :connect, :error, :out, :ed, :list, :listvar, :reset, :perftrace, and :serverlist. The first result set the script returns is displayed as a formatted table. Result sets after the first are not displayed if their column list is different from the column list of the first result set. If result sets after the first set have the same column list, their rows are appended to the formatted table that contains the rows that were returned by the first result set.
 Invoke-Sqlcmd does not return message output, such as the output of PRINT statements, unless you use the PowerShell -Verbose parameter.

```

Рис. 9.9. Пример справочного экрана

### ***Установка свойств окна SQL Server Powershell***

Возможно, вам понадобится увеличить размер буфера экрана для вывода всей справочной информации, когда применяется параметр `-Full`. Вы можете использовать свойства окна **SQL Server Powershell** для изменения размера буфера команды, цветов и других устанавливаемых по умолчанию параметров. Для перенастройки буфера экрана можно выполнить следующие действия.

1. Открыть сеанс SQL Server PowerShell.
2. В окне **SQL Server Powershell** щелкнуть правой кнопкой мыши полосу заголовка и выбрать команду **Properties** (Свойства).
3. В окне свойств щелкнуть кнопкой мыши вкладку **Layout** (Макет).

4. На странице макета увеличьте параметр **Screen Buffer Size Height** (Высота в размере буфера экрана) до 500 и щелкните мышью кнопку **OK**.
  5. Появится окно **Apply Properties** (Применить свойства), выберите, хотите ли вы применить свойства только в текущем окне или во всех будущих окнах с тем же заголовком, и щелкните мышью кнопку **OK**.
  6. Если требуется вернуть значения параметров по умолчанию, в окне **SQL Server Powershell** щелкните правой кнопкой мыши полосу заголовка и щелкните мышью команду **Defaults** (настройки по умолчанию).
- ◆ `Invoke-Sqlcmd` — запускает сценарий на языке Transact-SQL (T-SQL) или XQuery.
  - ◆ `Encode-SqlName` — шифрует идентификатор SQL Server (имя объекта) для перереформатирования любых символов, не поддерживаемых языком оболочки SQL Server PowerShell.
  - ◆ `Decode-SqlName` — возвращает исходный идентификатор SQL, если предоставлен зашифрованный идентификатор SQL.
  - ◆ `Convert-UrnToPath` — преобразует универсальное имя ресурса SMO (Uniform Resource Name, URN) в структуру пути, используемую SQL Server PowerShell. И путь, и URN содержат одинаковую информацию, но формат представления разный.

Например, вы можете использовать следующую команду для получения текущих даты и времени (рис. 9.10).

```
Invoke-Sqlcmd -Query "SELECT GETDATE() AS 'Date';"
```

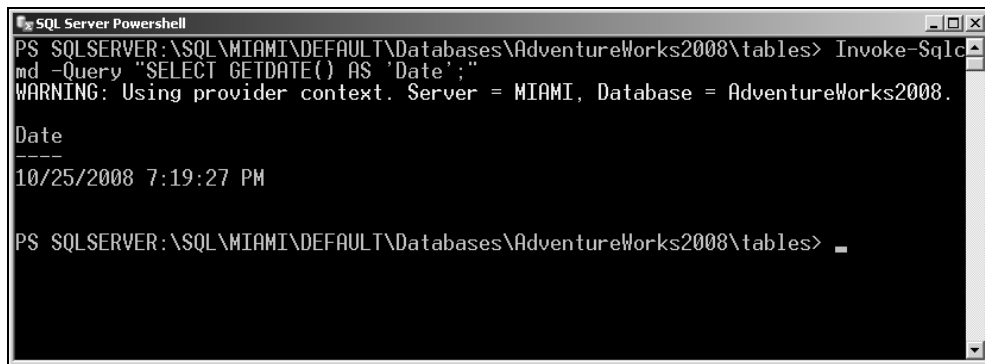


Рис. 9.10. Пример командлета `Invoke-Sqlcmd`

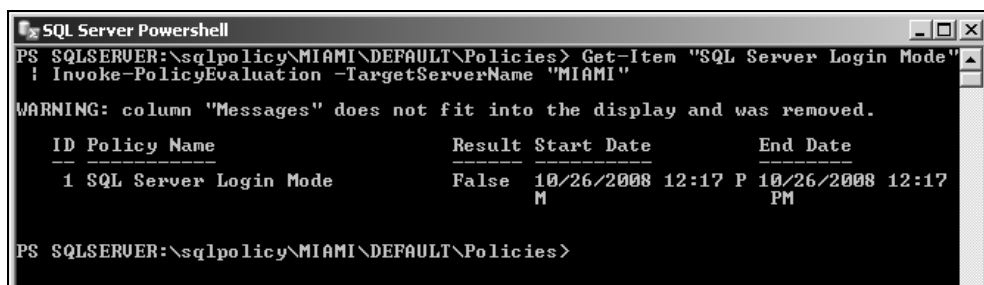
### Дополнительная информация о расширениях SQL PowerShell

Более подробные сведения о расширениях SQL PowerShell см. на Web-страницах <http://www.codeplex.com/SQLPSX> и <http://www.sqlservercentral.com/articles/powershell/64326/>.

## Примеры использования SQL Server PowerShell

Далее представлены примеры, которые помогут познакомиться поближе с несколькими командами и параметрами, доступными при использовании SQL Server PowerShell в версии SQL Server 2008.

В первом примере используется командлет `Get-Item` и определяется и отображается текущее состояние политики SQL Server Login Mode на сервере с именем MIAMI. Это одна из включенных в SQL Server 2008 политик Best Practices, которые вы можете импортировать в ваши серверные политики. Если на сервере установлена только встроенная (Integrated-only) проверка подлинности (аутентификация), будет возвращено `True`. Если параметры обеспечения безопасности разрешают проверки подлинности, встроенные как в SQL, так и в Windows, будет возвращен результат, равный `False` (рис. 9.11).



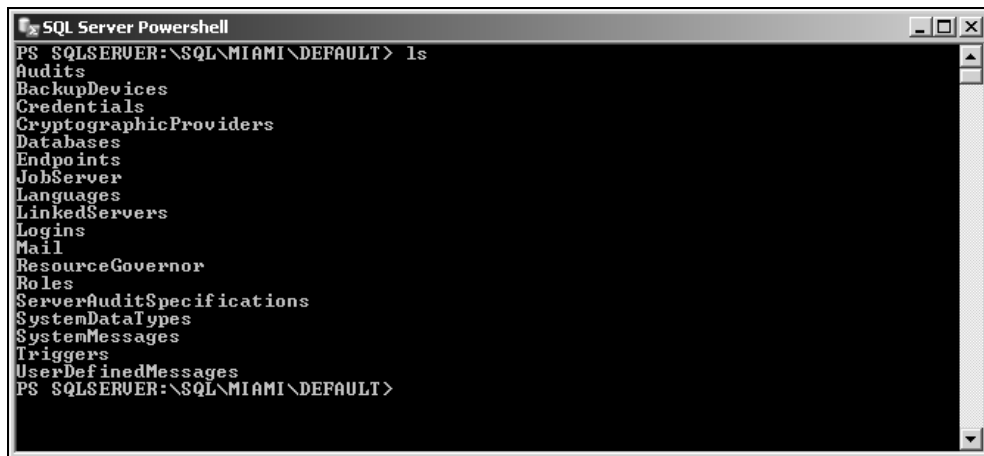
```
PS SQLSERVER:\sqlpolicy\MIAMI\DEFAULT\Policies> Get-Item "SQL Server Login Mode"
! Invoke-PolicyEvaluation -TargetServerName "MIAMI"

WARNING: column "Messages" does not fit into the display and was removed.

 ID Policy Name Result Start Date End Date
 --
 1 SQL Server Login Mode False 10/26/2008 12:17 P 10/26/2008 12:17
 M PM

PS SQLSERVER:\sqlpolicy\MIAMI\DEFAULT\Policies>
```

Рис. 9.11. Пример командлета `Get-Item`



```
PS SQLSERVER:\SQL\MIAMI\DEFAULT> ls
Audits
BackupDevices
Credentials
CryptographicProviders
Databases
Endpoints
JobServer
Languages
LinkedServers
Logins
Mail
ResourceGovernor
Roles
ServerAuditSpecifications
SystemDataTypes
SystemMessages
Triggers
UserDefinedMessages
PS SQLSERVER:\SQL\MIAMI\DEFAULT>
```

Рис. 9.12. Пример использования командлета `Get-ChildItem` для вывода перечня вложенных папок, доступных в экземпляре DEFAULT

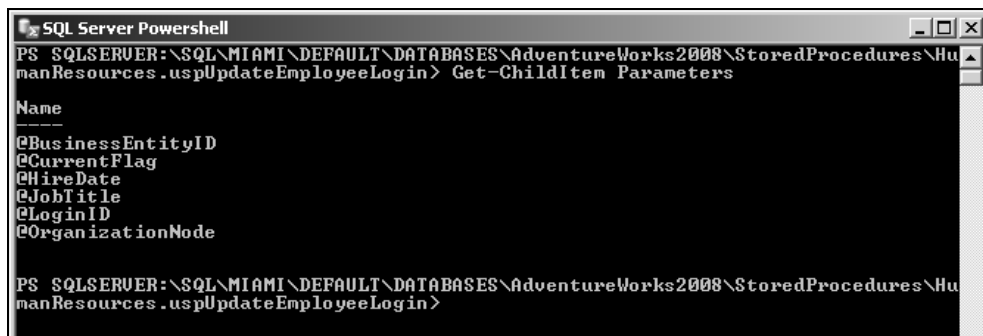


Рис. 9.13. Пример использования командлета `Get-ChildItem` для перечисления параметров, определенных в хранимой процедуре

### Командные требования

Прежде чем выполнять пример, приведенный на рис. 9.11, необходимо заменить путь подходящим и убедиться в том, что включен сервер RPC (сервер удаленного вызова процедур).

Во втором примере на рис. 9.12 показан список вложенных папок, доступных в экземпляре DEFAULT на сервере MIAMI в папке SQL. Обратите внимание на то, что в примере вместо ввода полного имени командлета `Get-ChildItem` применен псевдоним.

В последнем примере на рис. 9.13 показано использование командлета `Get-ChildItem` для вывода списка параметров, определенных в пользовательской хранимой процедуре `HumanResources.uspUpdateEmployeeLogin` в базе данных AdventureWorks2008 в экземпляре DEFAULT сервера MIAMI.

## ПРАКТИЧЕСКОЕ ЗАНЯТИЕ.

### Применение поставщика SQL Server PowerShell

На этом практическом занятии вы примените оболочку SQL Server PowerShell для просмотра информации на диске SQLSERVER.

#### Упражнение. Просмотр иерархии SQL Server PowerShell

В этом упражнении вы просмотрите различные пути и объекты с помощью утилиты SQL Server PowerShell.

1. Откройте среду SSMS, если она еще не открыта.
2. В Object Explorer (Обозреватель объектов) раскройте папку **Databases** (Базы данных), щелкните правой кнопкой мыши имя базы AdventureWorks2008 и выберите команду **Start PowerShell** (Запустить PowerShell).



3. В окне SQL Server PowerShell отметьте, что активный путь подобен приведенному далее:  
`PS SQLSERVER:\SQL\MIAMI\DEFAULT\Databases\AdventureWorks2008.`
4. Щелкните правой кнопкой мыши в окне **SQL Server Powershell** и выберите команду **Mark** (Пометить). Воспользуйтесь клавишами со стрелками для перемещения курсора на три позиции вправо, к первой букве вашего пути. Не выделяйте подсказку `PS` перед путем или символ `>` в конце пути. Прежде чем нажимать другие клавиши, нажмите клавишу `<Enter>`. Ваш курсор должен переместиться в конец текущей командной строки.
5. В текущей командной строке введите `Set-Location` и затем пробел, далее щелкните правой кнопкой мыши в окне **SQL Server Powershell** и выберите команду **Paste** (Вставить). Сразу же после вставленного пути введите `\Tables` и нажмите клавишу `<Enter>`. Убедитесь в том, что путь теперь ведет к объектам уровня **Tables** (Таблицы).
6. В окне **SQL Server Powershell** в текущей командной строке введите `Get-ChildItem` и нажмите клавишу `<Enter>`. Просмотрите полученные данные. Убедитесь, что для каждой таблицы, определенной пользователем в текущей базе данных, выводятся имена схемы и таблицы и дата создания.
7. В окне **SQL Server Powershell** в текущей командной строке введите `Set-Location ..`, чтобы перейти на уровень выше при указании пути.
8. В окне **SQL Server Powershell** в текущей командной строке введите `cd Views` для перехода на уровень объектов **Views** (Представления). Примените команду `Get-ChildItem` из п. 6 для вывода представлений, определенных пользователем в базе данных AdventureWorks2008. Просмотрите список полученных представлений.
9. Пока текущий путь указывает на вложенную папку **Views**, введите команду `Get-ChildItem -force` для вывода всех представлений, включая системные. Обратите внимание на отличия от результатов, полученных в п. 8.
10. В окне **SQL Server Powershell** в текущей командной строке введите `Set-Location HumanResources.vEmployee` и нажмите клавишу `<Enter>`.
11. Для просмотра столбцов, определенных в представлении `HumanResources.vEmployee`, введите `Get-ChildItem Columns` и нажмите клавишу `<Enter>`. Просмотрите столбцы, определенные в представлении `HumanResources.vEmployee`.

## Резюме урока

- ◆ SQL Server PowerShell — это оболочка с режимом командной строки и среда создания сценариев, основанная на Windows PowerShell.

- ◆ SQL Server PowerShell применяет иерархию для представления взаимосвязей объектов.
- ◆ В поставщике SQL Server PowerShell есть три папки: SQLSERVER:\SQL, SQLSERVER:\SQLPolicy и SQLSERVER\SQLRegistration.
- ◆ Просмотреть иерархию объектов можно, используя имена командлетов или их псевдонимы.

## УРОК 3. Отслеживание изменений данных

Если вы работали в предыдущих версиях SQL Server, вам, возможно, приходилось разрабатывать сложные решения, включающие вставку столбцов с метками времени, добавление триггеров и даже настройку репликации для отслеживания изменений в вашей базе данных. В версию SQL Server 2008 корпорация Microsoft ввела средство отслеживания изменений и систему отслеживания измененных данных (CDC), чтобы помочь ответить на вопросы о данных, которые изменены в базе данных. Кроме того, версия SQL Server 2008 Enterprise содержит объект SQL Server Audit, который обеспечивает автоматический контроль экземпляра SQL Server. Помимо контроля событий уровня сервера и базы данных SQL Server Audit предоставляет средство протоколирования не только в момент модификации таблицы (инструкциями `INSERT`, `UPDATE`, `DELETE`), но и во время чтения данных из таблицы (инструкцией `SELECT`).

### *После усвоения этого урока вы сможете*

- Описать и сравнить средство отслеживания изменений и систему CDC и разницу между ними.
- Включать отслеживание изменений в базах данных и таблицах.
- При применении отслеживания изменений запрашивать сведения об измененных данных.
- Включать CDC в базах данных и таблицах.
- При использовании CDC запрашивать сведения об измененных данных.

Расчетное время урока — 60 мин.

## Сравнение отслеживания изменений и CDC

При разработке метода отслеживания изменений в вашей базе данных необходимо выбирать между методами средства отслеживания изменений и методами системы отслеживания измененных данных, CDC. Все эти методы позволяют определить, были ли изменены данные. В следующих разделах сравниваются достоинства методов и описывается функционирование каждого из них.

## Отслеживание изменений

У *отслеживания изменений* следующие функциональные возможности:

- ◆ обеспечение функционирования с помощью инструкций DML;
- ◆ предоставление ответов на вопросы, подобные следующим:
  - Какие строки в таблице были изменены?
  - Какие столбцы были изменены?
  - Была ли конкретная строка обновлена?
  - Выполнялись ли инструкции INSERT, UPDATE или DELETE?
- ◆ синхронное функционирование, обеспечивающее немедленное предоставление сведений об изменениях;
- ◆ меньшее потребление памяти по сравнению с CDC;
- ◆ наличие встроенного механизма очистки;
- ◆ использование времени фиксации транзакции для определения последовательности изменений;
- ◆ работа, не требующая изменений в схеме таблицы или дополнительных триггеров;
- ◆ обязательное включение на уровне базы данных с помощью инструкции ALTER DATABASE.

## Система отслеживания измененных данных

У *системы отслеживания измененных данных* следующие функциональные возможности:

- ◆ обеспечение функционирования с помощью инструкций DML;
- ◆ предоставление ответов на те же вопросы, что и в случае отслеживания изменений, плюс следующие:
  - Какие промежуточные изменения были сделаны после последней синхронизации?
  - Сколько раз обновлялась строка с момента после последней синхронизации?
- ◆ применение *таблиц изменений* (change table) для записи данных измененного столбца и метаданных, которые необходимы для применения изменений в целевой рабочей среде (target environment) (структура столбцов зеркально отображает структуру исходной таблицы);
- ◆ использование записей журнала транзакций как входных данных для добавления сведений в таблицы изменений;

- ◆ асинхронное функционирование и доступность изменений только после завершения инструкции DML;
- ◆ предоставление функций, возвращающих табличные значения, для получения доступа к данным в таблицах изменений;
- ◆ необходимость включения в базе данных с помощью процедуры `sys.sp_cdc_enable_db`;
- ◆ наличие версии SQL Server 2008 Enterprise, Developer или Evaluation Edition.

### **РЕАЛЬНАЯ ИСТОРИЯ.**

#### ***Анна Вебер (Ann Weber)***

Несколько недель назад я работала с большой компанией розничной торговли в районе Чикаго. Ее сотрудники в этот момент применяли базы данных OLTP версии SQL Server 2005 наряду со службами Analysis Services, а торговые менеджеры меняли районы на регулярной основе. Когда происходило изменение в промежуточной базе данных, предназначенной для переноса информации в хранилище данных OLAP, необходимо было выполнить специальную программу, прежде чем обрабатывать многомерные кубы данных. Мы обсуждали разные способы выполнения этой задачи в SQL Server 2005, и хотя ее можно было решить, требовалось вставлять в таблицы дополнительные столбцы и триггеры, что могло замедлить ввод в упомянутую промежуточную базу данных и усложнить решение.

Мы также обсуждали версию SQL Server 2008 и средство отслеживания изменений. Клиентам нужно было знать только, обновлялась ли строка и каковы текущие данные в таблице, поэтому меньшие накладные расходы, характерные для отслеживания изменений, идеальны для них. Мы также говорили и о дополнительных функциональных возможностях CDC, но на данном этапе клиенты в них не нуждались.

## **Настройка отслеживания изменений**

Прежде чем вы сможете настраивать отслеживание изменений в ваших таблицах, необходимо включить его в базе данных. Для этого можно использовать среду SSMS или инструкцию `ALTER DATABASE`.

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### ***Уровень совместимости базы данных***

Прежде чем настраивать отслеживание изменений в заданной базе данных, вы должны установить для нее уровень совместимости, равный SQL Server 2005 или выше.

## **Включение отслеживания изменений в базе данных**

При включении отслеживания изменений в базе данных можно настраивать следующие параметры:

- ◆ `Change Tracking` — задается равным `True` или `False` (по умолчанию) для включения или отключения отслеживания изменений;

- ◆ **Retention Period** — задается числовым значением, определяющим минимальный промежуток времени, в течение которого изменения сохраняются. По умолчанию период сохранения равен 2;
- ◆ **Retention Period Units** — задается как **Days** (Дни) (по умолчанию), **Hours** (Часы) или **Minutes** (Минуты);
- ◆ **Auto CleanUp** — задается равным **ON** (по умолчанию) или **OFF** для включения или отключения процесса, удаляющего устаревшую информацию об отслеживании изменений.

### **ПРЕДУПРЕЖДЕНИЕ.**

**Включение режима *Auto CleanUp* восстанавливает значения по умолчанию**

Каждая установка параметра **Auto CleanUp** в состояние **ON** приводит к повторной установке значений по умолчанию параметров **Retention Period** и **Retention Period Units**, т. е. 2 и 2 Days соответственно.

Для включения отслеживания изменений с помощью среды SSMS необходимо выполнить следующие действия.

1. В **Object Explorer** (Обозреватель объектов) откройте папку **Databases** (Базы данных), щелкните правой кнопкой мыши базу данных, в которой вы хотите применить отслеживание данных, и щелкните кнопкой мыши команду **Properties** (Свойства).
2. На панели выбора страницы в окне **Database Properties** (Свойства базы данных) щелкните кнопкой мыши строку **Change Tracking** (Отслеживание изменений).
3. На странице **Change Tracking** настройте соответствующие параметры и щелкните мышью кнопку **OK**.

На рис. 9.14 показаны установочные параметры отслеживания изменений для базы данных **AdventureWorks2008**. Период сохранения задан равным 7 дням. Если промежуточная база данных службы **Analysis Services** синхронизируется с базой данных **AdventureWorks2008** один раз каждые три дня, заданный период сохранения позволяет пропускать синхронизацию и восстанавливать ее до того, как данные удаляются из таблиц изменений. В данном сценарии таблицам изменений требуется больше дискового пространства, чем в случае более короткого периода сохранения.

Вы можете включать отслеживание изменений и с помощью команды **ALTER DATABASE**. В приведенный далее программный код включена полная синтаксическая запись команды **ALTER DATABASE**, содержащая параметры, относящиеся к отслеживанию данных.

```
ALTER DATABASE database_name
SET
```

```

{
<change_tracking_option> ::=
{ CHANGE_TRACKING { = ON [<change_tracking_option_list >] |
 <change_tracking_option_list> | = OFF }
}
<change_tracking_option_list> ::=
{ (<change_tracking_option> |
 <change_tracking_option_list>, <change_tracking_option>)
}
<change_tracking_option> ::=
{ AUTO_CLEANUP = { ON | OFF }
 | CHANGE_RETENTION = { retention_period { DAYS | HOURS | MINUTES } }
}
}

```

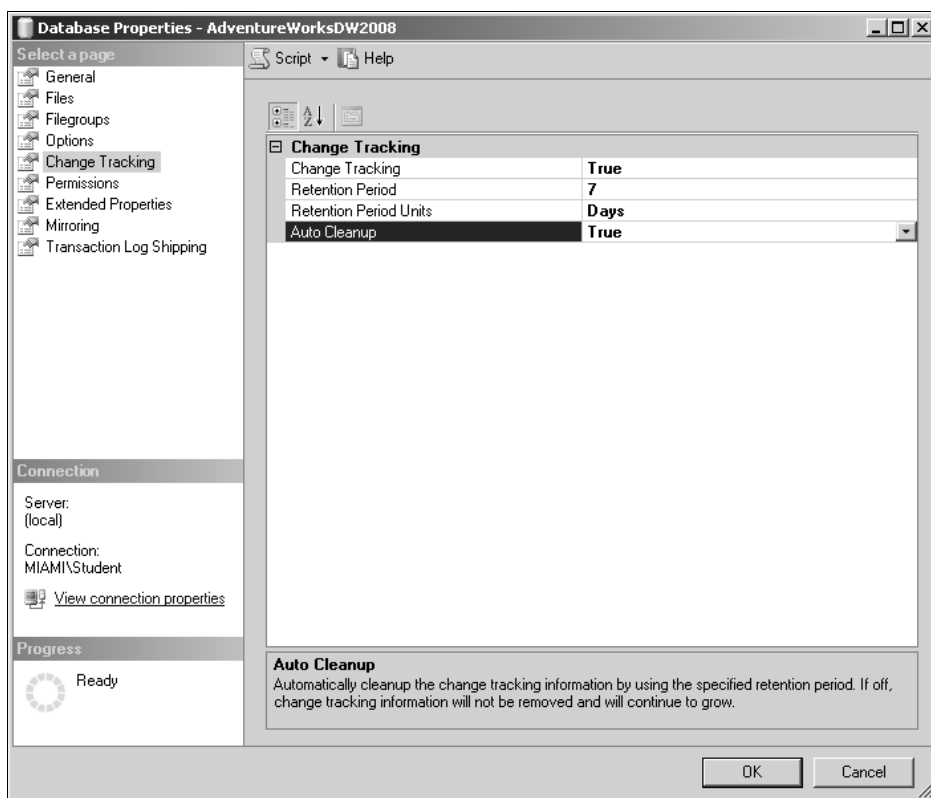


Рис. 9.14. Параметры отслеживания изменений базы данных

Для включения отслеживания изменений в базе данных AdventureWorksDW2008 можно выполнить следующую команду. Изменения сохраняются как минимум

семь дней, и автоматический процесс очистки удаляет сведения об отслеживании изменений, которые были созданы более семи дней назад.

```
ALTER DATABASE AdventureWorksDW2008
SET CHANGE_TRACKING = ON
(CHANGE_RETENTION = 7 DAYS, AUTO_CLEANUP = ON)
```

## Включение отслеживания изменений в таблице

После включения отслеживания изменений в базе данных применяется команда `ALTER TABLE` для включения отслеживания изменений в отдельных таблицах, за изменениями в которых вы хотите следить. У команды `ALTER TABLE` есть только два аргумента, влияющих на отслеживание изменений:

- ◆ `Change_Tracking` — для обозначения состояния отслеживания изменений в таблице задается равным `ENABLE` (по умолчанию) или `DISABLE`;
- ◆ `Track_Columns_Updated` — устанавливается равным `ON` или `OFF` (по умолчанию), обозначая ваше желание сохранять или не сохранять список столбцов, измененных во время выполнения команды `UPDATE` для этой таблицы.

Для настройки отслеживания изменений средствами SSMS выберите страницу **Change Tracking** (Отслеживание изменений) в окне **Table Properties** (Свойства таблицы) для таблицы, изменения в которой вы хотите отслеживать.

Вы можете выполнить следующий программный код для включения отслеживания изменений и получения информации об изменениях в отдельных столбцах таблицы `DimEmployee` (Сотрудники) в базе данных `AdventureWorksDW2008`.

```
ALTER TABLE DimEmployee
ENABLE CHANGE_TRACKING
WITH (TRACK_COLUMNS_UPDATED = ON);
```

## Отключение отслеживания изменений

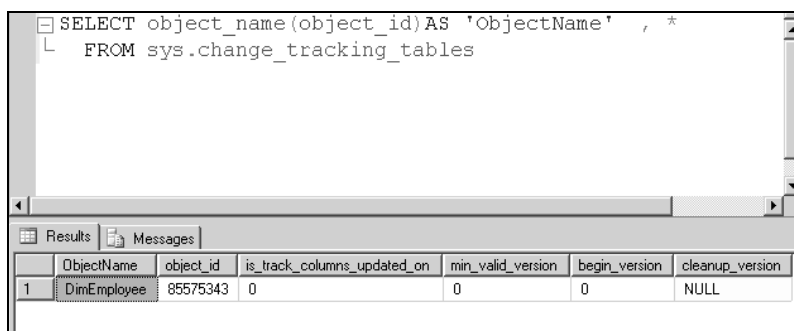
Прежде чем отключать отслеживание изменений в базе данных, следует с помощью среды SSMS или команды `ALTER TABLE` отключить отслеживание изменений в каждой таблице.

Для того чтобы узнать, в каких таблицах базы данных `AdventureWorksDW2008` включено отслеживание изменений, можно запросить представление каталога `sys.change_tracking_tables` (рис. 9.15).

Для отключения отслеживания изменений в таблице `FactInternetSales` (Интернет-продажи) выполните следующий программный код.

```
ALTER TABLE FactInternetSales
DISABLE CHANGE_TRACKING;
```

После отключения отслеживания изменений во всех таблицах базы данных можно отключить отслеживание изменений в базе данных с помощью среды SSMS или команды `ALTER DATABASE`.



```
SELECT object_name(object_id) AS 'ObjectName' , *
FROM sys.change_tracking_tables
```

	ObjectName	object_id	is_track_columns_updated_on	min_valid_version	begin_version	cleanup_version
1	DimEmployee	85575343	0	0	0	NULL

Рис. 9.15. Отображение таблиц с включенным отслеживанием изменений

Приведенная далее команда отключает отслеживание изменений в базе данных AdventureWorksDW2008.

```
ALTER DATABASE AdventureWorksDW2008
SET CHANGE_TRACKING = OFF
```

## Побочные эффекты отслеживания изменений

При включенном отслеживании изменений следующие команды могут изменить свое поведение.

- ◆ **TRUNCATE TABLE** — завершается успешно, но удаленные строки не отслеживаются, и обновляется минимальная допустимая версия, требующая от приложений повторной инициализации, прежде чем они смогут возобновить синхронизацию.
- ◆ **DROP INDEX** или **ALTER INDEX DISABLE** — завершается аварийно, только если ссылается на индекс, связанный с ограничением **PRIMARY KEY**.
- ◆ **DROP TABLE** — выполняется успешно и удаляет всю информацию отслеживания изменений, относящуюся к удаляемой таблице.
- ◆ **ALTER TABLE DROP CONSTRAINT** — завершается аварийно, если вы пытаетесь удалить ограничение **PRIMARY KEY**. Прежде чем вы сможете удалить ограничение таблицы **PRIMARY KEY**, следует отключить отслеживание изменений.
- ◆ **ALTER TABLE DROP COLUMN** — выполняется успешно, пока столбец не является частью ограничения **PRIMARY KEY**. Если в приложении не запрограммирована обработка удаляемого столбца, данные из него можно все же получить как часть информации отслеживания изменений.
- ◆ **ALTER TABLE ADD COLUMN** — выполняется успешно и начинает отслеживание изменений, выполняемых в новом столбце.
- ◆ **ALTER TABLE ALTER COLUMN** — выполняется успешно, но изменения типа данных в столбцах, не входящих в первичный ключ, не отслеживаются.



- ◆ `ALTER TABLE SWITCH` — завершается аварийно, если в одной или обеих таблицах включено отслеживание изменений.

## Работа с отслеживанием изменений

При разработке приложений для синхронизации данных с помощью отслеживания изменений можно применять несколько команд на языке T-SQL и представлений каталогов, предоставляемых версией SQL Server 2008.

Следующие функции позволяют запрашивать сведения об изменениях и управлять параметрами отслеживания изменений.

- ◆ `CHANGETABLE` — применяется для получения информации обо всех изменениях в таблице с помощью параметра `CHANGES`, или информации об изменениях конкретной строки с помощью параметра `VERSION`.
- ◆ `CHANGE_TRACKING_MIN_VALID_VERSION` — возвращает минимальную версию, которую допустимо использовать для отслеживания изменений при выполнении функции `CHANGETABLE`. Вы должны задать в качестве аргумента команды идентификатор табличного объекта для отслеживаемой таблицы.
- ◆ `CHANGE_TRACKING_CURRENT_VERSION` — получает версию, связанную с последней зафиксированной транзакцией. Данные этой версии можно использовать при выполнении функции `CHANGETABLE`.
- ◆ `CHANGE_TRACKING_IS_COLUMN_IN_MASK` — интерпретирует значение `SYS_CHANGE_COLUMNS`, возвращенное функцией `CHANGETABLE (CHANGES...)`, позволяя вашему приложению определить, включен ли столбец, заданный как аргумент, в значения, возвращаемые для `SYS_CHANGE_COLUMNS`.
- ◆ `WITH CHANGE_TRACKING_CONTEXT` — позволяет при выполнении обновлений установить контекстные данные в поле типа `varbinary(128)`, входящем в информацию отслеживания изменений. Это в свою очередь позволяет определить, какое приложение инициировало обновление.

### ***Дополнительная информация о синтаксисе функций отслеживания изменений***

Подробную информацию о синтаксических записях функций отслеживания изменений см. в разд. "Change Tracking Functions (Transact-SQL)" (Функции отслеживания изменений) электронной документации по SQL Server.

## Вывод функции `CHANGETABLE` и примеры

Функция `CHANGETABLE (CHANGES)` при необходимости выводит на экран следующие столбцы.

- ◆ `SYS_CHANGE_VERSION` — отображает для строки версию с самыми последними изменениями.

- ◆ `SYS_CHANGE_CREATION_VERSION` — версия со значениями последней операции INSERT.
- ◆ `SYS_CHANGE_OPERATION` — выводит U для операции UPDATE, D для DELETE или I для INSERT.
- ◆ `SYS_CHANGE_COLUMNS` — столбцы, которые изменились по сравнению с исходной версией. Этот столбец содержит значение NULL в следующих случаях:
  - аргумент `Track_Columns_Updated` равен OFF;
  - операция — INSERT или DELETE;
  - все столбцы, не входящие в первичный ключ, были обновлены в одной операции.
- ◆ `SYS_CHANGE_CONTEXT` — контекст обновления, если он задан. Контекст настраивается как часть инструкции INSERT, UPDATE или DELETE с помощью предложения WITH.
- ◆ `<primary key column value>` — значение первичного ключа для строки.

В следующих примерах запросов включено отслеживание изменений в базе данных AdventureWorksDW2008 и таблице DimEmployee (Сотрудники) этой базы данных. Для начала аргумент `Track_Columns_Updated` установлен равным OFF.

В таблицу DimEmployee вставляется новая строка с сотрудником Jonathan Haas. Его табельный номер — 299. На рис. 9.16 с помощью функции CHANGETABLE отображены изменения. Обратите внимание на то, что столбец `SYS_CHANGE_VERSION` равен 1 и столбец `SYS_CHANGE_OPERATION` равен I для обозначения операции INSERT.

```

DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(85575343);

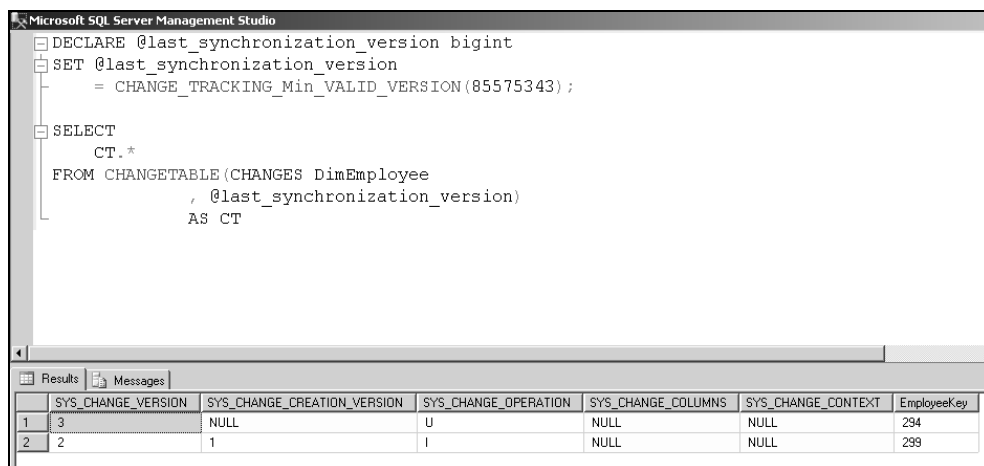
SELECT
 CT.*
FROM CHANGETABLE(CHANGES DimEmployee
 , @last_synchronization_version)
 AS CT

```

	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	EmployeeKey
1	1	1	I	NULL	NULL	299

Рис. 9.16. Пример вывода для таблицы изменений запроса с аргументом `Track_Columns_Updated`, равным OFF

В следующем примере обновляются две строки в таблице DimEmployee. Сотрудник 299, Jonathan Haas, становится менеджером по продажам (столбец SalesPersonFlag был задан равным 1). Сотрудник 294, Syed Abbas, больше не менеджер по продажам, значение в столбце SalesPersonFlag было задано равным 0. На рис. 9.17 показаны результаты функции CHANGETABLE (CHANGES), отображающие все обновления с начала отслеживания и до настоящего момента. Значение столбца SYS\_CHANGE\_VERSION для EmployeeKey 299 увеличилось до 2, а операция все еще I. Строка не была синхронизирована после первоначальной инструкции INSERT, поэтому INSERT все еще должна выполняться при синхронизации таблицы, но в синхронизацию уже включены новые текущие данные с 1 в столбце SalesPersonFlag. Для сотрудника с номером 294 значение столбца SYS\_CHANGE\_VERSION установлено равным 3 (каждое обновление таблицы увеличивает этот счетчик на 1), значение столбца SYS\_CHANGE\_OPERATION равно U для UPDATE, а столбец SYS\_CHANGE\_CREATION\_VERSION равен NULL, т. к. эта строка была вставлена до включения отслеживания изменений.



```

DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(85575343);

SELECT
 CT.*
FROM CHANGETABLE (CHANGES DimEmployee
 , @last_synchronization_version)
 AS CT

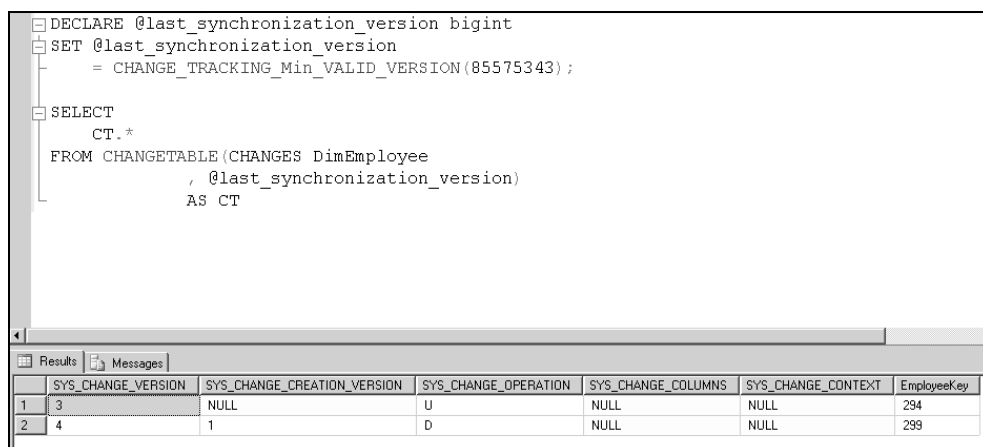
```

	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	EmployeeKey
1	3	NULL	U	NULL	NULL	294
2	2	1	I	NULL	NULL	299

Рис. 9.17. Результат после обновления строки из примера 1 и дополнительной строки

Заключительный пример этого сценария содержит удаление нового сотрудника Jonathan Haas. На рис. 9.18 для строки сотрудника с EmployeeKey 299 значение столбца SYS\_CHANGE\_VERSION возросло до 4, а столбец SYS\_CHANGE\_OPERATION теперь стал равен D, обозначая операцию DELETE.

В следующей серии моментальных снимков экрана применяется очищенный сервер и для базы данных снова включено отслеживание изменений. При включении отслеживания изменений в таблице DimEmployee аргумент TRACK\_COLUMNS\_UPDATED задан равным ON. В базу данных еще раз вставляется строка с сотрудником Jonathan Haas, его табельный номер EmployeeKey на этот раз 297, и затем выполняется запрос, показанный на рис. 9.19. Убедитесь, что результаты такие же, как в первой серии запросов.



```

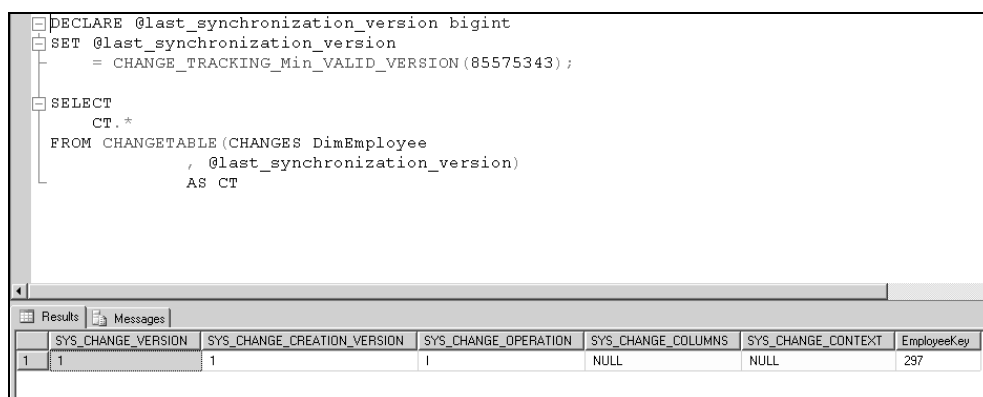
DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(85575343);

SELECT
 CT.*
FROM CHANGETABLE(CHANGES DimEmployee
 , @last_synchronization_version)
 AS CT

```

	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	EmployeeKey
1	3	NULL	U	NULL	NULL	294
2	4	1	D	NULL	NULL	299

Рис. 9.18. Результат после удаления новой строки



```

DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(85575343);

SELECT
 CT.*
FROM CHANGETABLE(CHANGES DimEmployee
 , @last_synchronization_version)
 AS CT

```

	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	EmployeeKey
1	1	1	I	NULL	NULL	297

Рис. 9.19. Результат после первой вставки с аргументом TRACK\_COLUMNS\_UPDATED, равным ON

В следующем запросе снова обновляются две строки в таблице DimEmployee. Jonathan Haas, с EmployeeKey 297 становится менеджером по продажам (значение столбца SalesPersonFlag равно 1). Syed Abbas, с Employee 294, больше не работает менеджером по продажам, следовательно, значение SalesPersonFlag было приравнено 0. На рис. 9.20 приведены результаты функции CHANGETABLE (CHANGES), показывающей все обновления от первого до последнего.

На этот раз столбец SYS\_CHANGE\_COLUMNS содержит двоичное значение, представляющее измененный столбец. Для интерпретации этой информации следует применить функцию CHANGE\_TRACKING\_IS\_COLUMN\_IN\_MASK.

В заключительном примере на рис. 9.21 вы видите, что аргумент TRACK\_COLUMNS\_UPDATED не оказывает влияния на инструкцию DELETE.

```

DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(85575343);

SELECT
 CT.*
FROM CHANGETABLE(CHANGES DimEmployee
 , @last_synchronization_version)
 AS CT

```

	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	EmployeeKey
1	3	NULL	U	0x000000001A000000	NULL	294
2	2	1	I	NULL	NULL	297

Рис. 9.20. Результат после обновления столбцов

```

DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(85575343);

SELECT
 CT.*
FROM CHANGETABLE(CHANGES DimEmployee
 , @last_synchronization_version)
 AS CT

```

	SYS_CHANGE_VERSION	SYS_CHANGE_CREATION_VERSION	SYS_CHANGE_OPERATION	SYS_CHANGE_COLUMNS	SYS_CHANGE_CONTEXT	EmployeeKey
1	3	NULL	U	0x000000001A000000	NULL	294
2	4	1	D	NULL	NULL	297

Рис. 9.21. Результат после удаления вновь добавленной строки

Функция `CHANGETABLE(VERSION)` при необходимости выводит на экран следующие столбцы:

- ◆ `SYS_CHANGE_VERSION` — версия строки с самыми последними изменениями;
- ◆ `SYS_CHANGE_CONTEXT` — контекст обновления, если он задан. Контекст настраивается как часть инструкции `INSERT`, `UPDATE` или `DELETE` с помощью предложения `WITH`;
- ◆ `<primary key column value>` — значение первичного ключа для строки.

## Управление отслеживанием изменений

Следующие представления каталогов отображают сведения о настройке отслеживания изменений.

◆ `sys.change_tracking_databases` — отображает следующие данные:

- `Database_id` — целочисленное поле, представляющее для баз данных с включенным отслеживанием изменений уникальный идентификатор базы данных в экземпляре SQL Server;
- `Is_auto_cleanup_on` — битовое поле, равное 0 для состояния `off` и 1 для состояния `on`;
- `Retention_period` — целочисленное поле, представляющее минимальный период времени, в течение которого сохраняются отслеженные изменения, прежде чем процесс автоочистки удалит их;
- `Retention_period_units_desc` — поле типа `nvarchar`, задающее Minutes (Минуты), Hours (Часы), Days (Дни);
- `Retention_period_unit` — поле типа `tinyint` со значением 1 для минут, 2 для часов и 3 для дней.

◆ `sys.change_tracking_tables` — отображает следующие данные:

- `Object_id` — целочисленное поле, представляющее уникальный идентификатор таблицы для таблиц в базе данных, имеющих журнал изменений (`change journal`). Таблица с отключенным в данный момент отслеживанием изменений также может иметь такой журнал;
- `Is_track_columns_updated_on` — битовое поле, равное 0 для состояния `off` и 1 для `on`;
- `Begin_version` — поле типа `bigint`, содержащее номер версии базы данных, в момент начала отслеживания изменений таблицы;
- `Cleanup_version` — поле типа `bigint`, содержащее номер версии, предшествующей данной версии, в которой данные могли быть удалены процессом автоматической очистки;
- `Min_valid_version` — поле типа `bigint`, содержащее минимальный номер корректной версии данных об отслеживании изменений в каждой таблице.

### **Дополнительная информация о представлениях каталогов отслеживания изменений**

Более подробные сведения о представлениях каталогов отслеживания изменений см. в разд. "Change Tracking Catalog Views (Transact-SQL)" ("Представления каталога отслеживания изменений (Transact-SQL)") электронной документации по SQL Server.

## Настройка CDC

Настройка системы отслеживания измененных данных, CDC, — это многошаговый процесс. Прежде чем вы сможете включить CDC для ваших таблиц, необходимо включить CDC для базы данных, содержащей таблицы, которые вы хотите отслеживать.

### ***ВАЖНАЯ ИНФОРМАЦИЯ.*** ***Требования CDC***

---

Для настройки и успешного применения CDC следует убедиться в том, что запущена служба SQL Server Agent.

## Включение CDC для вашей базы данных

Для включения CDC в вашей базе данных выполните системную хранимую процедуру `sys.sp_cdc_enable_db`.

Следующая команда включает CDC в базе данных AdventureWorksDW2008.

```
USE AdventureWorksDW2008;
GO
EXECUTE sys.sp_cdc_enable_db;
GO
```

Когда вы включаете CDC для базы данных, система делает равным 1 столбец `is_cdc_enabled` в представлении каталога `sys.databases` и создает системные объекты, необходимые CDC. К ним относятся схема `cdc`, учетная запись базы данных `cdc` и таблицы, задания, хранимые процедуры и функции, которые требуются процессу CDC.

### ***Ограничения CDC***

---

Вы не можете включать CDC для системных баз данных.

## Включение CDC в ваших таблицах

После того как CDC включена в вашей базе данных, можно включить систему отслеживания измененных данных и для таблиц в этой базе данных. Когда для таблицы с помощью системной хранимой процедуры `sys.sp_cdc_enable_table` включается CDC, для исходной таблицы создается экземпляр CDC (capture instance).

Системная хранимая процедура `sys.sp_cdc_enable_table` содержит следующие аргументы.

- ◆ `source_schema` — имя схемы, которой принадлежит таблица.
- ◆ `source_name` — имя исходной таблицы. Это имя должно существовать в текущей базе данных и его не должно быть в схеме `cdc`.

- ◆ *role\_name* — имя роли базы данных, которая применяется для обеспечения доступа к собранному данным. Если роль не существует, SQL Server попытается ее создать. Если у пользователя, выполняющего команду, нет достаточных разрешений для создания роли, вся хранимая процедура завершается аварийно.

### **Необходимые аргументы**

При выполнении процедуры `sys.sp_cdc_enable_table` вы должны задать аргументы *source\_schema*, *source\_name* и *role\_name*. Аргумент *role\_name* может задать равным `NULL`. Дополнительную информацию см. в разд. "Представление о разрешениях CDC" далее в этой главе.

- ◆ *capture\_instance* — имя данное экземпляру, применяемому для именования связанных с экземпляром объектов. У исходной таблицы может быть до двух заданных экземпляров. Если аргумент *capture\_instance* не указан, имя по умолчанию — `schemaname_sourcename`. Имя экземпляра не должно превышать 100 символов.
- ◆ *supports\_net\_changes* — задается равным 1 или 0 и указывает, есть ли поддержка запросов суммарных изменений. Значение по умолчанию — 0, позволяющее функциям запрашивать только все изменения. Значение 1 позволяет запросить суммарные (итоговые) изменения.

### **Дополнительные требования для поддержки суммарных изменений**

Если вы включаете поддержку отслеживания суммарных изменений и в исходной таблице нет поля первичного ключа, следует задать в аргументе *index\_name* допустимый уникальный индекс для исходной таблицы.

- ◆ *index\_name* — имя допустимого уникального индекса исходной таблицы.
- ◆ *captured\_column\_list* — имена исходных столбцов, которые будут отслеживаться. Если этот аргумент равен `NULL`, в таблицу изменений включаются все столбцы.

### **Дополнительные требования к столбцам**

К отслеживаемым столбцам предъявляются следующие требования.

- Вы должны указать в списке столбцы, включенные в поле первичного ключа или уникального индекса, заданного в аргументе *index\_name*.
- Для разделения имен столбцов следует использовать запятую.
- Если в имена входит запятая или вы хотите вставить для имен столбцов идентификаторы, заключенные в кавычки, можно заключить такие имена в апострофы или квадратные скобки.
- Нельзя включать в список следующие зарезервированные имена столбцов: `__start_lsn`, `__end_lsn`, `__$seqval`, `__$operation` и `__$update_mask`.
- Нельзя включать в список столбцы, переопределенные с помощью новых типов данных, введенных в версии SQL Server 2008.



- ◆ *filegroup\_name* — имя файловой группы, в которой будет создана таблица изменений. Если это значение равно NULL, таблица изменений создается в файловой группе по умолчанию. Если параметр *filegroup\_name* задан, имя файловой группы должно существовать в текущей базе данных.
- ◆ *partition\_switch* — задается равным TRUE или FALSE (по умолчанию) для обозначения возможности или невозможности выполнения в отношении исходной таблицы команды SWITCH PARTITION инструкции ALTER DATABASE при включенной CDC.

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### **Как взаимодействуют SWITCH PARTITION и CDC**

Изменения данных, вызванные командой SWITCH PARTITION, не фиксируются в таблице изменений. Например, если секция переключается с исходной таблицы на другую, операция DELETE не отслеживается.

Для включения CDC в таблице FactInternetSales (Интернет-продажи) базы данных AdventureWorksDW2008 можно использовать следующий программный код. Этот пример позволит запросить суммарные изменения данных.

```
USE AdventureWorksDW2008;
GO
EXEC sys.sp_cdc_enable_table
 @source_schema = N'dbo'
 , @source_name = N'FactInternetSales'
 , @role_name = N'cdc_admin'
 , @capture_instance = N'InternetSales'
 , @supports_net_changes = 1
GO
```

Для проверки конфигурационных параметров CDC можно применить функцию cdc.fn\_cdc\_get\_net\_changes.

## **Представление о разрешениях CDC**

По умолчанию у обладателей ролей sysadmin и db\_owner есть полный доступ к данным в таблицах изменений.

Если параметр *role\_name* равен NULL, CDC включается без *шлюзовой роли* (gating role). Шлюзовая роль определяет доступ к данным в таблице изменений. При подобной настройке CDC любой пользователь с разрешением на выполнение инструкции SELECT в исходной таблице может получить доступ к данным в таблице изменений.

Если в аргументе *role\_name* задается конкретная встроенная или определенная пользователем роль, тем самым определяется шлюзовая роль для таблицы изменений, требующая от пользователей, нуждающихся в доступе к измененным

данным, наличия разрешения `SELECT` для измененных столбцов исходной таблицы, а также членства в шлюзовой роли для данного экземпляра системы отслеживания.

## Запросы измененных данных

Для запроса данных в таблице изменений применяется функция `cdc.fn_cdc_get_all_changes_<capture_instance>` или функция `cdc.fn_cdc_get_net_changes_<capture_instance>`. Если вы включаете CDC и устанавливаете аргумент `supports_net_changes`, равным 1, в заданном экземпляре CDC доступны обе функции. Если аргумент `supports_net_changes` задан равным 0 или вы не включили этот аргумент, доступна только функция `cdc.fn_cdc_get_all_changes_<capture_instance>`.

При включении суммарных изменений, если вы хотите написать запрос, возвращающий только одну строку в результирующем наборе для каждой измененной строки из диапазона регистрационных номеров транзакций (Log Sequence Number, LSN) и включающий только окончательное содержимое без учета возможных промежуточных изменений, следует применять функцию `cdc.fn_cdc_get_net_changes_<capture_instance>`.

Функция `cdc.fn_cdc_get_net_changes_<capture_instance>` включает в себя следующие аргументы.

- ◆ *from\_lsn* — начальный номер LSN для создания диапазона возвращаемых строк. Все строки в таблице изменений с величиной LSN, равной или большей значения *from\_lsn*, включаются в результирующий набор. Этот параметр задает *нижнюю границу* (lower bound) диапазона запроса.
- ◆ *to\_lsn* — конечный номер LSN диапазона возвращаемых строк. Все строки в таблице изменений с величиной LSN, меньшей или равной *to\_lsn*, включаются в результирующий набор. Этот параметр определяет *верхнюю границу* (upper bound) диапазона запроса.
- ◆ *row\_filter\_option* — управляет выбором возвращаемых строк и информацией, отображаемой в столбцах метаданных результирующего набора. Данный аргумент может иметь следующие значения:
  - *all* — в столбце `_start_lsn` функция возвращает номер транзакции, LSN, в столбце `_operation` — обозначение выполненной операции, в столбце `_update_mask` — NULL;
  - *all with mask* — в столбце `_start_lsn` функция возвращает номер LSN и в столбце `_operation` — обозначение выполненной операции. Если операция UPDATE возвращает 4, биты в столбце `_update_mask`, связанные с обновленными столбцами, устанавливаются равными 1;

- `all with merge` — в столбце `_$start_lsn` функция возвращает номер LSN и в столбце `_$operation` — 1, если строка была удалена, и 5, если требуется операция `INSERT` или `UPDATE` для внесения изменений. Столбец `_$update_mask` всегда равен `NULL`.

#### **Применение значения `all with merge`**

Этот вариант разработан для повышения производительности, т. к. не требует разграничения инструкций `INSERT` и `UPDATE`. Применяя это значение, вы должны считать, что применяете операцию `MERGE`, доступную в версии SQL Server 2008.

#### **Дополнительная информация о команде `MERGE`**

Более подробные сведения о команде `MERGE` см. в уроке 2 главы 2.

Таблица, возвращаемая при запросе функции `cdc.fn_cdc_get_net_changes_<capture_instance>`, включает в себя следующие столбцы.

- ◆ `_$start_lsn` — отображает номер LSN, связанный с действием зафиксированной транзакции, выполняющим изменения.
- ◆ `_$seqval` — отображает последовательное значение для упорядочивания изменений строк, происходящих в транзакции. У всех строк в транзакции одно и то же значение `_$start_lsn`.
- ◆ `_$operation` — отображает целочисленное значение, основанное на выполненной операции и значении параметра `row_filter_option`. Если вы задали параметр `row_filter_option`, равным `all` или `all with mask`, в этом столбце отображаются следующие значения:
  - 1 для операций `DELETE`;
  - 2 для операций `INSERT`;
  - 4 для операций `UPDATE`.

Когда параметр `row_filter_option` задан равным `all with merge`, в столбце `_$operation` отображаются следующие значения:

- 1 для операций `DELETE`;
- 5 для операций, как `INSERT`, так и `UPDATE`.
- ◆ `_$update_mask` — отображает битовую маску, представляющую отслеженные столбцы исходной таблицы. 1 в маске обозначает столбец, который был изменен. Для инструкций `INSERT` и `DELETE` все биты равны 1.
- ◆ `<captured source table columns>` — возвращаются все отслеженные столбцы исходной таблицы.

В следующем запросе изменяются строки в таблице `FactInternetSales` (Интернет-продажи). На рис. 9.22 показан результирующий набор функции `cdc.fn_cdc_get_net_changes_InternetSales` после выполнения обновлений.

```

USE AdventureWorksDW2008;
GO
DECLARE @begin_time datetime, @end_time datetime, @from_lsn binary(10)
 , @to_lsn binary(10);
-- Для фиксации изменений устанавливается начало диапазона
-- на день раньше.
SET @begin_time = GETDATE() -1;
-- Инструкции DML для внесения изменений.

UPDATE dbo.FactInternetSales
SET OrderQuantity = 5
WHERE SalesOrderNumber = 'SO43697' AND SalesOrderLineNumber = 1;

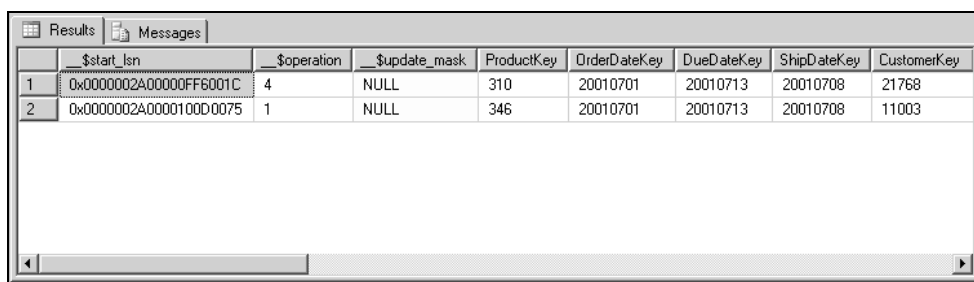
UPDATE dbo.FactInternetSales
SET OrderQuantity = 2
WHERE SalesOrderNumber = 'SO43697' AND SalesOrderLineNumber = 1;

DELETE FROM dbo.FactInternetSales
WHERE SalesOrderNumber = 'SO43701' AND SalesOrderLineNumber = 1;

-- Устанавливается конец диапазона после внесения изменений.
SET @end_time = GETDATE();
-- Временной интервал отображается в диапазон отслеживания
-- измененных данных.
SET @from_lsn = sys.fn_cdc_map_time_to_lsn(
 'smallest greater than or equal', @begin_time);
SET @to_lsn = sys.fn_cdc_map_time_to_lsn(
 'largest less than or equal', @end_time);

-- Возвращает окончательные изменения, происшедшие в окне запроса.
SELECT * FROM cdc.fn_cdc_get_net_changes_InternetSales(
 @from_lsn, @to_lsn, 'all');

```



	\$start_lsn	\$operation	\$update_mask	ProductKey	OrderDateKey	DueDateKey	ShipDateKey	CustomerKey
1	0x0000002A00000FF6001C	4	NULL	310	20010701	20010713	20010708	21768
2	0x0000002A0000100D0075	1	NULL	346	20010701	20010713	20010708	11003

Рис. 9.22. Частичный вывод функции cdc.fn\_cdc\_get\_net\_changes\_InternetSales

Если вы хотите написать запрос, который возвращает строки со всеми изменениями, внесенными в течение заданного периода времени, следует применить

функцию `cdc.fn_cdc_get_all_changes_capture_instance`. Если в конкретной строке исходной таблицы было несколько изменений, данная функция вернет для нее несколько строк.

Функция `cdc.fn_cdc_get_all_changes_capture_instance` принимает следующие аргументы.

- ◆ *from\_lsn* — начальный номер LSN для создания диапазона, из которого будут возвращены строки. Все строки в таблице изменений с номерами LSN, равными или большими, чем номер *from\_lsn*, включаются в результирующий набор. Этот параметр называют *нижней границей диапазона запроса*.
- ◆ *to\_lsn* — конечный номер LSN для создания диапазона возвращаемых строк. Все строки в таблице изменений с номерами LSN, меньшими или равными *to\_lsn*, будут включены в результирующий набор. Этот параметр называют *верхней границей диапазона запроса*.
- ◆ *row\_filter\_option* — может принимать следующие значения:
  - *all* — возвращаются все изменения, внесенные в диапазоне, определенном параметрами *from\_lsn* и *to\_lsn*. Если строки были изменены с помощью инструкции `UPDATE`, возвращается только строка, содержащая новые значения;
  - *all update old* — возвращаются все изменения, внесенные в диапазоне, определенном параметрами *from\_lsn* и *to\_lsn*. Если строки были изменены с помощью инструкции `UPDATE`, возвращаются строка, содержащая значения до обновления, и строка со значениями после обновления.

Таблица, возвращаемая функцией `cdc.fn_cdc_get_all_changes_capture_instance`, включает в себя те же столбцы, что и таблица, возвращаемая функцией `cdc.fn_cdc_get_net_changes_capture_instance`.

На рис. 9.23 показаны результаты функции `cdc.fn_cdc_get_all_changes_InternetSales` с изменениями, такими же, как в результирующем наборе, показанном на рис. 9.22. Вы можете сравнить эти два рисунка, чтобы увидеть отличия между отображением суммарных и всех изменений.

	<i>_start_lsn</i>	<i>_seqval</i>	<i>_operation</i>	<i>_update_mask</i>	<i>ProductKey</i>	<i>OrderQuantity</i>	<i>OrderDateKey</i>
1	0x0000002A00000FF6001C	0x0000002A00000FF6001A	4	0x000800	310	5	20010701
2	0x0000002A0000100D0075	0x0000002A0000100D0073	1	0x7FFFFFFF	346	1	20010701
3	0x0000002A000010840004	0x0000002A000010840002	4	0x000800	310	2	20010701

Рис. 9.23. Пример результатов функции `cdc.fn_cdc_get_all_changes_InternetSales`

Возможно, при построении приложения, запрашивающего таблицу изменений, вы захотите задать возвращаемые строки, основываясь на времени внесения из-

менений, а не на номерах LSN. Для определения номеров LSN, которые применяются в аргументах `from_lsn` и `to_lsn` функций `cdc.fn_cdc_get_all_changes_<capture_instance>` или `cdc.fn_cdc_get_net_changes_<capture_instance>`, можно использовать функцию `sys.fn_cdc_map_time_to_lsn`.

В запросах к измененным данным можно применять следующие функции CDC.

- ◆ `sys.fn_cdc_has_column_changed` — возвращает 1 или 0 для обозначения в любой связанной строке изменений наличие или отсутствие обновлений в столбце, указанном с помощью предоставляемой маски. Входные параметры этой функции — экземпляр системы отслеживания измененных данных, имя столбца и значения маски обновлений.
- ◆ `sys.fn_cdc_increment_lsn` — возвращает следующий номер LSN в серии, основанный на входном значении LSN. Если вы знаете значение верхней границы из предыдущего запроса, можно использовать эту функцию для определения новой нижней границы диапазона нового запроса. Эту логику можно применить, чтобы не нужно было выяснять время последней выполненной синхронизации. Вам нужно просто сохранять значения LSN из диапазона предыдущей синхронизации.
- ◆ `sys.fn_cdc_decrement_lsn` — возвращает предыдущий номер LSN, основанный на входном значении LSN. Эту функцию можно использовать для определения верхней границы диапазона LSN, определенного в запросе.
- ◆ `sys.fn_cdc_is_bit_set` — использует порядковый номер в битовой маске (значение `$_update_mask`) для заданного столбца и битовую маску как входные параметры для возврата бита, определяющего, был ли обновлен столбец. Вы можете применять эту функцию для добавления столбца в ваш вывод, предоставляющий состояние обновления заданного столбца исходной таблицы. Например, вы запрашиваете таблицу изменений и хотите знать, был ли обновлен столбец `GroupName` (Название группы) в каждой строке, возвращенной из таблицы `HR_Department` (Отдел). Можно использовать функции `cdc.fn_cdc_get_all_changes_HR_Department`, `sys.fn_cdc_get_column_ordinal` и `sys.fn_cdc_is_bit_set` для добавления в результирующий набор столбца с псевдонимом `'Group Name Updated'`, в котором отображается 1, если `True`, и 0, если `False`.
- ◆ `sys.fn_cdc_get_column_ordinal` — использует экземпляр системы отслеживания измененных данных и имя столбца как входные параметры и возвращает целое число, представляющее порядковый номер столбца в битовой маске.
- ◆ `sys.fn_cdc_map_lsn_to_time` — использует LSN как входной параметр и возвращает значение типа `DATETIME` времени фиксации из столбца `tran_end_time` системной таблицы `cdc.lsn_time_mapping`.
- ◆ `sys.fn_cdc_get_max_lsn` — возвращает максимальный номер LSN, имеющийся в столбце `start_lsn` системной таблицы `cdc.lsn_time_mapping`. Это значение

представляет последнее зафиксированное изменение, переданное в таблицу изменений в текущей базе данных. Это значение не зависит от экземпляра системы отслеживания.

- ◆ `sys.fn_cdc_get_min_lsn` — возвращает минимальный номер LSN, имеющийся в столбце `start_lsn` системной таблицы `cdc.lsn_time_mapping`. Это значение может меняться после выполнения процесса очистки. Данную функцию можно использовать для определения нижнего предельного значения LSN, чтобы убедиться в том, что ваш диапазон лежит в пределах временных рамок действия CDC.

Помимо функций, позволяющих запрашивать данные в таблицах изменений, SQL Server содержит хранимые процедуры, которыми можно воспользоваться для управления CDC.

### ***Дополнительная информация об управлении CDC***

Дополнительные сведения о применении встроенных хранимых процедур CDC см. в разд. "Change Data Capture Stored Procedures (Transact-SQL)" (Хранимые процедуры системы отслеживания измененных данных (Transact-SQL)) электронной документации по SQL Server.

## **SQL Server Audit**

Подсистема аудита SQL Server Audit предоставляет возможность записывать в журнал информацию о событиях и изменениях, происходящих на вашем сервере с запущенной программой SQL Server. Для применения SQL Server Audit следует создать и настроить как минимум один объект SQL Server Audit для каждого экземпляра, в котором ведется аудит. Затем на основе объекта SQL Server Audit создаются спецификации для сервера Server Audit, для базы данных Database Audit или для обоих, чтобы управлять аудитом определенных событий и объектов.

### ***Дополнительная информация о настройке SQL Server Audit***

Обсуждение SQL Server Audit этого урока сводится к способам отслеживания доступа к данным и модификаций в вашем экземпляре SQL Server. Полное описание SQL Server Audit см. в главе 11 книги Hotek M. MCTS Self-Paced Training Kit (Exam 70-432): Microsoft SQL Server 2008 — Implementation and Maintenance. — Microsoft Press, 2009 (Хотек М. Обучающий курс для получения MCTS (экзамен 70-432): SQL Server 2008 — реализация и поддержка) и главе 18 книги Hotek M. Microsoft SQL Server 2008 Step by Step. — Microsoft Press, 2008 (Хотек М. Microsoft SQL Server 2008 шаг за шагом).

## **Создание объекта SQL Server Audit**

Перед определением спецификаций аудита вы должны создать объект SQL Server Audit. Сделать это можно в среде SSMS или с помощью инструкции `CREATE SERVER AUDIT`.

В среде SSMS можно использовать Object Explorer (Обозреватель объектов) и просмотреть папку **Audits** (Аудиты), вложенную в папку **Security** (Безопасность). Затем вы можете щелкнуть правой кнопкой мыши папку **Audits** и выбрать команду **New Audit** (Создать аудит) для создания нового объекта SQL Server Audit. В определении аудита можно настроить следующие параметры.

- ◆ **Audit Name** (Имя аудита) — имя, присвоенное объекту Audit.
- ◆ **Queue Delay** (Задержка очереди) — промежуток времени в миллисекундах, который может пройти до того, как сервер начнет выполнять действие аудита. По умолчанию это значение равно 1 секунде или 1000 миллисекундам.
- ◆ **Shut Down Server On Audit Log Failure** (Остановить сервер при возникновении ошибки аудита) — определяет, завершает ли работу SQL Server при аварийном завершении аудита. Вы можете активизировать этот параметр, если политика аудита вашей организации или соответствующие нормативные документы, такие как Health Insurance Portability and Accountability Act (HIPAA, закон об ответственности и праве перевода данных о страховании здоровья граждан), акт Payment Card Industry (PCI, стандарт защиты данных платежных карточек) и Federal Information Security Management Act (FISMA, Федеральный закон США об управлении информационной безопасностью) требуют от вас аудита каждого заданного события без аварийного завершения.
- ◆ **Audit Destination** (Тип назначения аудита) — определяет, куда записываются данные аудита. К возможным вариантам относятся файл, журнал событий безопасности Windows Security Log или журнал событий приложений Windows Application Log. Если в качестве места назначения данных аудита вы выбираете File, также следует настроить следующие параметры.
  - **File Path** (Путь к файлу) — каталог, в котором хранится файл. Компонент Database Engine автоматически генерирует имя файла, основанное на имени объекта Audit и глобальном уникальном идентификаторе (GUID) аудита.
  - **Maximum Rollover** (Максимальное число файлов продолжения) — максимальное число файлов продолжения (rollover), которые могут быть созданы, если не выбран вариант **Unlimited**. Если для данного параметра выбран вариант **Unlimited**, количество файлов продолжения не ограничено.
  - **Maximum Filesize** (Максимальный размер файла) — максимальный размер файла до создания файла продолжения. Вы можете задать целочисленное значение и единицу измерения для него: мегабайты, гигабайты или терабайты. Если выбран вариант **Unlimited**, файл увеличивается до заполнения всего доступного пространства на жестком диске. Если файл займет все имеющееся пространство на диске, запись прекращается и сервер завершает работу, если установлен параметр **Shut Down Server On Audit**



**Log Failure.** Минимально допустимое значение данного параметра — 2 Мбайт.

- **Reserve Disk Space** (Резервирование места на диске) — задает немедленное создание файла максимального размера. В противном случае файл увеличивается по мере вставки в него подробностей аудита. Вы не можете выбрать этот параметр, если максимальный размер файла задан как **Unlimited**.

В следующем программном коде представлена полная синтаксическая запись инструкции `CREATE SERVER AUDIT`. Имеющиеся параметры зеркально отражают параметры настройки, описанные ранее в варианте создания объекта аудита средствами SSMS.

```
CREATE SERVER AUDIT audit_name
 TO { [FILE (<file_options> [, ...n])] | APPLICATION_LOG |
 SECURITY_LOG }
 [WITH (<audit_options> [, ...n])]
}
[;]
<file_options>::=
{
 FILEPATH = 'os_file_path'
 [, MAXSIZE = { max_size { MB | GB | TB } | UNLIMITED }]
 [, MAX_ROLLOVER_FILES = integer]
 [, RESERVE_DISK_SPACE = { ON | OFF }]
}

<audit_options>::=
{
 [QUEUE_DELAY = integer]
 [, ON_FAILURE = { CONTINUE | SHUTDOWN }]
 [, AUDIT_GUID = uniqueidentifier]
}
```

После создания объекта SQL Server Audit вы должны включить его. Делается это с помощью либо SSMS, либо инструкции `ALTER SERVER AUDIT`.

## Создание спецификации аудита уровня базы данных

После создания объекта SQL Server Audit можно выбирать создание спецификаций аудита на уровне сервера или на уровне базы данных. Для отслеживания доступа к данным и их модификации создается Database Audit Specification (спецификация аудита на уровне базы данных).

Создать эту спецификацию можно с помощью инструкции `CREATE DATABASE AUDIT SPECIFICATION` или Object Explorer (Обозреватель объектов) в среде SSMS. Папка Database Audit Specifications находится в папке [instance name] \[Databases]\[database name]\Security.

Для определения спецификации Database Audit Specification следует задать имя спецификации аудита, объект Audit, в который будут записываться события и каждое регистрируемое действие аудита.

Существует множество типов действий Audit Action Types, которые можно настраивать в каждой спецификации Database Audit Specification. Например, можно задать INSERT, UPDATE, DELETE или SELECT и определить, в каких объектах вы хотите отслеживать эти команды. Кроме того, можно указать, для каких пользователей и ролей вам хотелось бы регистрировать события.

Помимо задания отдельных регистрируемых событий вы можете указать целую группу событий, предназначенных для регистрации. Группа SCHEMA\_OBJECT\_ACCESS\_GROUP, а также эквивалент класса событий Audit Schema Object Access возвращает информацию, когда применяется разрешение на объект INSERT, UPDATE, DELETE, SELECT, EXECUTE или REFERENCES. Информация, предоставляемая этим классом событий, сообщает о синтаксисе запроса, который выполнен, но не сохраняет значения измененных данных до и после обновления или удаления. Полный перечень столбцов и данные, предоставляемые в записях журналов, приведены в разд. "Audit Schema Object Access Event Class" ("Класс событий Audit Schema Object Access") электронной документации по SQL Server.

---

**ВАЖНАЯ ИНФОРМАЦИЯ.*****Производительность и аудит***

---

При задании аудита следует учитывать влияние на производительность протоколирования событий. Существует тенденция пытаться регистрировать каждое событие, что может привести к существенному снижению производительности. Тщательно выбирайте объекты, для которых требуется аудит, а также события, нуждающиеся в отслеживании.

Прежде чем спецификация Database Audit Specification начнет сбор информации, ее необходимо включить (сделать активной) с помощью SSMS либо в инструкциях CREATE DATABASE AUDIT SPECIFICATION или ALTER DATABASE AUDIT SPECIFICATION.

---

**ВАЖНАЯ ИНФОРМАЦИЯ.*****Изменение спецификаций аудита баз данных***

---

Прежде чем вносить изменения в спецификацию Database Audit Specification, ее следует отключить.

## **ПРАКТИЧЕСКОЕ ЗАНЯТИЕ. Настройка SQL Server для отслеживания изменений**

На этом практическом занятии вы будете следить за изменениями с помощью средств отслеживания изменений и системы отслеживания измененных данных (CDC).

## Упражнение 1.

### Настройка и применение отслеживания изменений

В этом упражнении вы настроите отслеживание изменений в таблице *ProspectiveBuyer* (Потенциальный покупатель). Приложению нужно знать только, когда выполняются в таблице операции *INSERT* и *DELETE*. Сведения об обновленных столбцах вас не интересуют.

#### **ВАЖНАЯ ИНФОРМАЦИЯ.**

##### **Изменения данных**

В упражнениях этого практического занятия вы удаляете и обновляете строки в базе данных *AdventureWorksDW2008*. Вы должны сделать копию ваших таблиц и работать с копиями или после завершения практических занятий этой главы восстановить исходную базу данных *AdventureWorksDW2008*.

1. Откройте среду SSMS, если она еще не открыта.
2. На панели инструментов щелкните мышью кнопку **New Query** (Создать запрос) для открытия окна создания запроса.
3. В окне создания запроса введите и выполните следующую команду, включающую в базу данных *AdventureWorksDW2008* отслеживание изменений с включенной автоочисткой и периодом сохранения изменений, равным 5 дням.

```
ALTER DATABASE AdventureWorksDW2008
SET CHANGE_TRACKING = ON
(CHANGE_RETENTION = 5 DAYS, AUTO_CLEANUP = ON)
```

4. В открытом окне запроса под имеющимся программным кодом введите, выделите и выполните следующий код, включающий отслеживание изменений в таблице *ProspectiveBuyer* (Потенциальный покупатель) базы данных *AdventureWorksDW2008*.

```
USE AdventureWorksDW2008;
GO
ALTER TABLE ProspectiveBuyer
ENABLE CHANGE_TRACKING
WITH (TRACK_COLUMNS_UPDATED = OFF);
```

5. В имеющемся окне запроса под введенным программным кодом введите, выделите и выполните следующий код, вставляющий в таблицу *ProspectiveBuyer* две строки.

```
INSERT INTO ProspectiveBuyer
(FirstName, LastName, MaritalStatus, Gender)
VALUES ('Terry', 'Adams', 'M', 'M');

INSERT INTO ProspectiveBuyer
(FirstName, LastName, MaritalStatus, Gender)
VALUES ('Wilson', 'Pais', 'S', 'M');
```

6. В имеющемся окне запроса под программным кодом введите, выделите и выполните следующий код для определения `object_id` и данных версии для таблицы `ProspectiveBuyer`. Запишите `object_id`.

```
SELECT object_name(object_id) AS 'ObjectName' , *
FROM sys.change_tracking_tables
```

7. В открытом окне запроса под имеющимся кодом введите, выделите и выполните следующий программный код для просмотра информации в таблице изменений. Если нужно, отредактируйте `object_id` в аргументе `CHANGE_TRACKING_Min_VALID_VERSION`.

```
DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(389576426);

SELECT CT.*
FROM CHANGETABLE (CHANGES ProspectiveBuyer
 , @last_synchronization_version) AS CT
```

Просмотрите результирующий набор.

8. В имеющемся окне запроса под существующим кодом введите, выделите и выполните следующий программный код для просмотра строки с данными о Terry Adams, добавленной в п. 5. Запишите значение столбца `ProspectiveBuyerKey` в строке о Terry Adams для использования в следующем пункте.

```
SELECT * FROM ProspectiveBuyer
WHERE LastName = 'Adams' and FirstName = 'Terry'
```

9. В имеющемся окне запроса под существующим кодом введите, выделите и выполните следующий программный код, удаляющий строку о Terry Adams и строку с `ProspectiveBuyerKey`, равным 308. При необходимости откорректируйте значение `ProspectiveBuyerKey` в соответствии с ключом, полученным в п. 8.

```
DELETE FROM ProspectiveBuyer
WHERE ProspectiveBuyerKey = 2060;
```

```
DELETE FROM ProspectiveBuyer
WHERE ProspectiveBuyerKey = 308;
```

10. В имеющемся окне запроса под существующим кодом введите, выделите и выполните следующий программный код для просмотра информации в таблице изменений. При необходимости отредактируйте `object_id` для аргумента `CHANGE_TRACKING_Min_VALID_VERSION`.

```
DECLARE @last_synchronization_version bigint
SET @last_synchronization_version
 = CHANGE_TRACKING_Min_VALID_VERSION(389576426);
```

```
SELECT CT.*
FROM CHANGETABLE(CHANGES ProspectiveBuyer
 , @last_synchronization_version) AS CT
```

11. Просмотрите результирующий набор. Оставьте открытой среду SSMS для следующего упражнения.

## Упражнение 2. Настройка и применение CDC

В этом упражнении вы настроите CDC в таблицах `FactInternetSales` (Интернет-продажи) и `FactResellerSales` (Продажи через посредников), так чтобы в таблице `FactInternetSales` разрешить доступ к суммарным изменениям, а в таблице `FactResellerSales` — ко всем изменениям.

### **ВАЖНАЯ ИНФОРМАЦИЯ.** **Изменения данных**

В упражнениях этого практического занятия вы удаляете и обновляете строки в базе данных `AdventureWorksDW2008`. Вы должны сделать копию ваших таблиц и работать с копиями или после завершения практических занятий этой главы восстановить исходную базу данных `AdventureWorksDW2008`.

1. В среде SSMS щелкните мышью на панели инструментов кнопку **New Query** (Создать запрос) для открытия окна создания запроса.
2. В окне создания запроса введите и выполните следующую команду, включающую CDC в базе данных `AdventureWorksDW2008`.

```
USE AdventureWorksDW2008;
GO
EXECUTE sys.sp_cdc_enable_db;
GO
```

3. В открытом окне запроса под существующим кодом введите, выделите и выполните следующий программный код для включения CDC в таблицах `FactInternetSales` и `FactResellerSales`.

```
USE AdventureWorksDW2008;
GO
EXEC sys.sp_cdc_enable_table
 @source_schema = N'dbo',
 @source_name = N'FactInternetSales',
 @role_name = N'cdc_admin',
 @capture_instance = N'InternetSales',
 @supports_net_changes = 1
GO

USE AdventureWorksDW2008;
GO
EXEC sys.sp_cdc_enable_table
```

```
@source_schema = N'dbo',
@source_name = N'FactResellerSales',
@role_name = N'cdc_admin',
@capture_instance = N'ResellerSales',
@supports_net_changes = 0
```

GO

4. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код для проверки настроек CDC.

```
sys.sp_cdc_help_change_data_capture;
```

### **ВАЖНАЯ ИНФОРМАЦИЯ.**

#### **Требования CDC**

Для успешной настройки и использования CDC необходимо убедиться в том, что служба SQL Server Agent запущена.

5. В имеющемся окне запроса под существующим программным кодом введите, выделите и выполните следующий код для обновления таблицы FactResellerSales и просмотра изменений с помощью функции cdc.fn\_cdc\_get\_all\_changes\_ResellerSales.

```
USE AdventureWorksDW2008;
```

GO

```
DECLARE @from_lsn binary(10), @to_lsn binary(10);
```

```
-- Инструкции DML для выполнения изменений.
```

```
UPDATE dbo.FactResellerSales
```

```
SET OrderQuantity = 6
```

```
WHERE SalesOrderNumber = 'SO44771' AND SalesOrderLineNumber = 12;
```

```
DELETE FROM dbo.FactResellerSales
```

```
WHERE SalesOrderNumber = 'SO44771' AND SalesOrderLineNumber = 31;
```

```
UPDATE dbo.FactResellerSales
```

```
SET OrderQuantity = 4
```

```
WHERE SalesOrderNumber = 'SO44771' AND SalesOrderLineNumber = 12;
```

```
-- Дождитесь запуска процесса системы отслеживания для обработки
-- изменений данных.
```

```
WAITFOR DELAY '00:00:10'
```

```
-- Задайте начальное и конечное значения lsn, равными минимальному и
-- максимальному допустимым значениям.
```

```
SET @from_lsn = sys.fn_cdc_get_min_lsn('ResellerSales')
```

```
SET @to_lsn = sys.fn_cdc_get_max_lsn()
```

```
-- Верните все изменения, произведенные в окне запроса.
```

```
SELECT * FROM cdc.fn_cdc_get_all_changes_ResellerSales(
@from_lsn, @to_lsn, 'all');
```

6. Просмотрите результаты и отметьте, что были напечатаны две строки для обновлений в строке со значениями в столбце SalesOrderNumber — SO44771 и в столбце SalesOrderLineNumber — 12.
7. В имеющемся окне запроса под существующим кодом введите, выделите и выполните следующий программный код, обновляющий таблицу FactInternetSales и отображающий суммарные изменения с помощью функции cdc.fn\_cdc\_get\_net\_changes\_InternetSales.

```
USE AdventureWorksDW2008;
GO
DECLARE @from_lsn binary(10), @to_lsn binary(10);

-- Инструкции DML, выполняющие изменения.
UPDATE dbo.FactInternetSales
SET OrderQuantity = 4
WHERE SalesOrderNumber = 'SO43698' AND SalesOrderLineNumber = 1;

DELETE FROM dbo.FactInternetSales
WHERE SalesOrderNumber = 'SO43717' AND SalesOrderLineNumber = 1 ;

UPDATE dbo.FactInternetSales
SET OrderQuantity = 2
WHERE SalesOrderNumber = 'SO43698' AND SalesOrderLineNumber = 1;

-- Дождитесь запуска процесса системы отслеживания для обработки
-- изменений данных.
WAITFOR DELAY '00:00:10'
-- Задайте начальное и конечное значения lsn, равными минимальному и
-- максимальному допустимым значениям.
SET @from_lsn = sys.fn_cdc_get_min_lsn('ResellerSales')
SET @to_lsn = sys.fn_cdc_get_max_lsn()

-- Верните суммарные изменения, произведенные в окне запроса.
SELECT * FROM cdc.fn_cdc_get_net_changes_InternetSales(
 @from_lsn, @to_lsn, 'all');
```

8. Просмотрите результаты и отметьте, что для каждой строки исходной таблицы возвращена только одна строка, а не по одной строке для каждого промежуточного изменения.

## Резюме урока

- ◆ Отслеживание изменений включается сначала на уровне базы данных, а затем на уровне таблицы.
- ◆ Отслеживание изменений может сообщить вам, какие строки были изменены, и предоставляет вам окончательные результирующие данные.

- ◆ Отслеживанию изменений требуется меньше системных ресурсов, чем CDC.
- ◆ CDC может сообщить вам, какие строки были изменены, и предоставить как окончательные данные, так и промежуточные состояния данных.
- ◆ SQL Server Audit позволяет протоколировать доступ к таблицам, представлениям и другим объектам.

## Обзор главы

Для того чтобы спустя какое-то время попрактиковаться и освежить навыки, полученные в результате изучения данной главы, можно выполнить следующее.

- ◆ Прочитать резюме главы.
- ◆ Просмотреть список ключевых терминов, введенных в этой главе.
- ◆ Найти способ решения проблемы в предлагаемых для тренировки конкретных сценариях. Эти сценарии описывают реальную ситуацию, включающую темы, рассмотренные в этой главе, и вам предлагается разработать решения.
- ◆ Выполнить предложенные практические задания.
- ◆ Выполнить тренировочный тест.

## Резюме

- ◆ Для встраивания сообщений электронной почты в ваши приложения баз данных можно использовать хранимую процедуру `sp_send_dbmail`.
- ◆ Поставщик SQL Server PowerShell позволяет создавать сценарии для автоматизации выполнения команд SQL и перечисления объектов на сервере баз данных.
- ◆ Отслеживание изменений и CDC предоставляют разные способы отслеживания изменений данных и обеспечивают синхронизацию с другими базами данных.

## Ключевые термины

- ◆ Командлеты (Cmdlets).
- ◆ Таблицы изменений (Change tables).
- ◆ Шлюзовая роль (Gating role).
- ◆ Нижняя граница (Lower bound).
- ◆ Верхняя граница (Upper bound).
- ◆ Регистрационный номер транзакции (Log Sequence Number, LSN).



## Сценарии

В следующих конкретных сценариях вы примените все, что узнали об интеграции Database Mail. Ответы на предложенные вопросы можно найти в *разд. "Ответы к сценариям"* в конце книги.

### Сценарий 1. Интеграция Windows PowerShell и Database Mail

Вы — разработчик базы данных для компании Litware, Inc. Ваш менеджер попросил вас создать сценарий, перечисляющий все созданные пользователями таблицы и представления в базе данных LitwareSales. Этот сценарий должен генерироваться автоматически в 17:00 каждую пятницу.

Администратор баз данных настроил компонент Database Mail на вашем сервере с запущенной программой SQL Server. В базе данных msdb нет открытого профиля по умолчанию. У пользовательской среды, в которой вы выполняете этот пункт задания, нет личного профиля по умолчанию, связанного с ней.

Ответьте на следующий вопрос вашего менеджера: "Какие шаги должны быть включены в разработку решения для вашего менеджера?"

### Сценарий 2. Отслеживание изменений

Вы — разработчик базы данных для компании Litware, Inc. Вы сопровождаете базу данных оперативной обработки транзакций (OLTP), названную LitwareSales, и реляционное хранилище данных (RDW), названное LitwareDW. Вы синхронизируете данные в LitwareSales и LitwareDW каждый день. Бизнес-правила запрещают обновления данных после добавления строки в базу данных LitwareDW.

Вас попросили определить наилучший метод отслеживания изменений в базе данных LitwareSales для оптимизации процесса синхронизации. Ресурсы ограничены, и вы хотите минимизировать дисковое пространство, загрузку процессора и объем памяти, необходимые для реализации решения.

Ответьте на следующие вопросы вашего менеджера:

- ◆ "Какое решение наиболее соответствует требованиям?"
- ◆ "Какие параметры следует задать для минимизации потребления ресурсов?"

## КОНТРОЛЬНЫЕ УПРАЖНЕНИЯ

Для того чтобы освоить полностью экзаменационные темы, обсуждавшиеся в этой главе, выполните все следующие упражнения.

### Интеграция Database Mail

**Упражнение 1.** Создайте и протестируйте ряд почтовых сообщений, формируемых с помощью разных значений параметров системной хранимой процедуры `sp_send_dbmail`.

## **Выполнение сценариев с помощью Windows PowerShell и модели объектов SQL Server Management Objects (SMO)**

**Упражнение 2.** Попрактикуйтесь, просматривая иерархию SQL Server и выполняя разные командлеты в сеансе SQL Server PowerShell.

**Упражнение 3.** Создайте сценарий SQL Server PowerShell для перечисления столбцов конкретной таблицы и выполните его с помощью задания службы SQL Server Agent.

## **Отслеживание изменений данных**

**Упражнение 4.** Включите отслеживание изменений в базе данных и в таблице. Измените данные и просмотрите информацию в таблицах изменений, чтобы понять, как разные значения конфигурационных параметров и различные функции позволяют увидеть разные результаты.

**Упражнение 5.** Включите CDC в базе данных и в таблице. Измените данные и просмотрите информацию в таблицах изменений, чтобы понять, как разные значения конфигурационных параметров и различные функции позволяют увидеть разные результаты.

## **Тренировочный тест**

Тренировочные тесты на сопроводительном компакт-диске, прилагаемом к книге, предлагают множество возможностей. Например, вы можете проверить себя по одной экзаменационной теме или по всему содержимому сертификационного экзамена 70-433. Вы можете настроить текст так, что он с большой долей подобия будет воспроизводить сдачу экзамена на получение сертификата, или выполнить тест в режиме обучения, который позволит увидеть правильные ответы и пояснения после вашего ответа на каждый вопрос.

### **Дополнительная информация о тренировочных тестах**

Дополнительную информацию обо всех режимах работы тренировочных тестов см. в разд. "Варианты тренировочных тестов" Введения.

# Ответы к сценариям

## Глава 1

### Сценарий 1. Извлечение данных

- ◆ Включите пример с предложением `ORDER BY`, которое ссылается либо на дату помещения заказа, либо на дату выполнения заказа.
- ◆ Включите несколько запросов и примеров результатов, которые используют функцию `CONVERT` для отображения дат в различных форматах. Включите в результирующий набор имена стилей форматирования, чтобы помочь разработчику приложения принять верное решение, касающееся форматирования дат.
- ◆ Включите запрос, применяющий в предложениях `SELECT` и `ORDER BY` функцию `DATEDIFF`, которая определяет количество дней между датой помещения заказа и датой его выполнения.
- ◆ Включите запрос, в котором функция `DATEDIFF` применяется для поиска заказов, которые были помещены в течение 1 месяца (`MONTH`), 1 квартала (`QUARTER`) и 1 года (`YEAR`), начиная с текущей даты.
- ◆ Предоставьте информацию о любых операциях `JOIN`, которые могут потребоваться из-за имеющегося уровня нормализации базы данных.

### Сценарий 2. Группировка данных

- ◆ Включите пример инструкции `SELECT`, использующей статистическую функцию `SUM` для получения общего количества проданных товаров. Предоставьте примеры предложений `WHERE`, которые могут быть добавлены для получения информации, основанной на разных критериях.
- ◆ Включите пример предложения `GROUP BY`, содержащего статистическую функцию `SUM` оператора `ROLLUP`.

- ◆ Включите пример предложения `GROUP BY`, содержащего статистическую функцию `SUM` оператора `CUBE`.

## Глава 2

### Сценарий 1. Изменение данных

Добавить данные электронной почты и заменить номер факса значением `NULL` может инструкция `UPDATE`. Кроме того, сохранить необходимую информацию в таблице `ImporterPropertiesAudit` поможет предложение `OUTPUT`.

### Сценарий 2. Применение транзакций

Вам следует принять во внимание следующие соображения, но не ограничиваться ими.

- ◆ Каков уровень изоляции текущей транзакции?
- ◆ Какие бизнес-правила затронет изменение уровня изоляции транзакции?
- ◆ Какова длительность каждой транзакции? Есть ли способы сокращения времени выполнения каждой транзакции или разделения транзакции на несколько?
- ◆ Все ли данные получаются и проверяются до начала транзакции, чтобы исключить взаимовлияние пользователей после запуска транзакции?
- ◆ Обеспечивается ли во всех транзакциях одинаковый порядок обращения к ресурсам?
- ◆ Сколько пакетов входит в одну транзакцию? Есть ли способ минимизировать их количество без нарушения функционирования?
- ◆ Применяются ли в запросах какие-либо подсказки блокировок?

#### ***Дополнительная информация о подсказках блокировок***

Подсказки блокировок и их влияние на блокирование и блокировки не рассматриваются в этой книге. Обсуждение подсказок блокировок можно найти в книге Kalen Delaney et al. *Microsoft SQL Server 2008 Internals*. — Microsoft Press, 2009 (Кэлен Делани и др. *Microsoft SQL Server 2008 изнутри*).

## Глава 3

### Сценарий. Ограничения и типы данных

1. Поскольку в таблице уже есть данные, не соответствующие ограничениям, которые необходимо добавить, вы должны вставить ограничения с помощью параметра `WITH NOCHECK` (`ALTER TABLE...ADD CONSTRAINT...WITHNOCHECK`). Это

заставит новые ограничения проверять любые вводимые данные (включая обновления имеющихся данных) и при этом позволит сохранить уже имеющиеся в таблице данные.

2. Самый большой тип данных, поддерживаемый свойством `IDENTITY`, — `decimal(38,0)`.

## Глава 4

### Сценарий. Повышение производительности запроса

- ◆ Отчет о загрузках данных хранилища данных можно легко получить с помощью вложенных запросов, ОТВ или комбинации ОТВ с ранжирующими функциями. Поскольку применение ОТВ с ранжирующей функцией должно обеспечить наилучшую производительность, вам следует применить именно этот способ.
- ◆ Циклическая обработка результирующего набора может быть выполнена с помощью ОТВ, что будет гораздо эффективнее решения на базе курсора. В зависимости от конкретной бизнес-задачи вы можете применять один и тот же метод для поиска пропусков в загружаемых в хранилище данных и последующего решения задач обнаружения пропущенных данных или заполнения пустых мест. Вы могли бы переписать курсоры, вычисляющие суммы продаж, применив вложенные запросы, которые могут предоставить итоги с накоплением и сводные данные в группах.
- ◆ Исключить все промежуточные временные таблицы можно с помощью производных таблиц, что позволит получить выигрыш за счет использования имеющейся на компьютере оперативной памяти вместо физических считываний с диска и операций записи на диск.

## Глава 5

### Сценарий. Повышение производительности приложения

1. Большинство проблем Fabrikam кроется в плохо спроектированных приложениях. Приложения не должны генерировать специализированные запросы, предъявляемые SQL Server. Вместо этого они должны следовать передовым методам инкапсуляции программного кода, применяемым в промышленности. Весь программный код базы данных следует удалить из приложения и перенести в хранимые процедуры. Затем нужно переписать приложения, включив в них обращения к хранимым процедурам. Как только будет получена стабильная версия приложения, использующего хранимые процедуры,

вы сможете начать анализ и настройку действий, необходимых Fabrikam. Хранимые процедуры можно будет настраивать изолированно от приложений, так что любые изменения, которые необходимо включить, затронут только хранимые процедуры и не потребуют создания новой версии приложения.

2. Помешать случайному удалению объектов на производственном сервере можно с помощью создания триггера DDL для отката любого события `DROP`. Но вам следует принимать во внимание снижение уровня безопасности и лишить административного доступа всех, кто не отвечает за управление сервером с работающей программой SQL Server.
3. Сложный программный код, касающийся базы данных, следует удалить из приложения и перенести в хранимые процедуры, функции и представления соответственно. Это уменьшит сложность приложений и преобразует программный код баз данных в формат, который легче сопровождать и которым легче управлять.

## Глава 6

### Сценарий. Настройка производительности запроса

Для оптимизации плохо работающих хранимых процедур выполните их в среде SSMS, проверьте их графические планы выполнения и рассмотрите возможность переделки их запросов и индексов в запрашиваемых таблицах. Для уменьшения времени выполнения запросов для финансового отдела с 10 минут до пары секунд вы, вероятно, должны ввести дополнительные индексы. После добавления индексов протестируйте запросы, чтобы убедиться в том, что вы не снизили производительность ваших приложений.

## Глава 7

### Сценарий. Как следует хранить данные?

Во-первых, лучшего ответа на этот вопрос, возможно, не существует. Скорее всего, вам придется разработать опытный образец, чтобы убедиться в его правильности. Но в данном сценарии особенно важно принять во внимание следующие аспекты.

- ◆ Категории хранятся в таблице.
- ◆ Статьи сохраняются в таблице, которая ссылается на таблицу категорий.
- ◆ При условии, что у каждой статьи всегда есть один заголовок, его, возможно, следует хранить непосредственно в столбце таблицы статей.

- ◆ Поскольку статья может состоять из различных частей и эти части могут быть вложены одна в другую (например, перечень в ячейку таблицы) и учитывая то, что порядок следования частей важен, содержимое статьи, вероятно, следует сохранять в единственном столбце типа XML.
- ◆ Имея в виду, что все допустимые данные в столбце содержимого должны быть очень строго определены, вероятно, следует применить коллекцию схем XML для задания типа столбца с содержимым статей.

## Глава 8

### Сценарий 1. Создание экземпляров пространственных данных

Для местоположений стеллажей на складе следует применить тип данных `geometry`. Вы должны создать экземпляр пространственных данных с помощью одного из методов `POINT`. Поскольку вы задаете конкретные координаты  $X$ ,  $Y$ , лучше всего использовать метод `POINT`. Для определения региона, обслуживаемого складом, следует применить тип данных `geometry`. Необходимо создать экземпляр пространственных данных с помощью одного из методов `POLYGON`. Поскольку вы хотите задать регион, метод `POLYGON` позволит определить координаты внешнего контура региона.

### Сценарий 2. Запросы полнотекстового индекса

1. Следует создать пользовательский список стоп-слов и связать его с соответствующим каталогом. Кроме того, необходимо отредактировать файлы тезауруса для соответствующего языка и включить в них все синонимы слов, определенных бизнес-требованиями.
2. Функция `FREETEXTTABLE` предоставит вам все необходимые функциональные средства. Вам следует упорядочить результирующий набор по убыванию ранга, чтобы вывести наиболее важные (подходящие) строки первыми в результирующем наборе.

## Глава 9

### Сценарий 1. Интеграция Windows PowerShell и Database Mail

Далее перечислена последовательность возможных действий.

1. Следует определить путь и командлеты, необходимые для сценария SQL Server PowerShell, и затем написать и протестировать сценарий.

2. Создайте задание службы SQL Server Agent с соответствующим расписанием. При определении шага задания, который должен выполняться, используйте тип PowerShell. Откройте или скопируйте в шаг задания протестированный сценарий. На вкладке **Advanced** (Дополнительно) окна **Job Step** (Шаг задания) создайте файл вывода результатов для сценария SQL Server PowerShell.

### ***Дополнительная информация о заданиях SQL Server Agent***

Полное описание заданий SQL Server Agent см. в книге Hotek M. MCTS Self-Paced Training Kit (Exam 70-432): SQL Server 2008 — Implementation and Maintenance. — Microsoft Press, 2009 (Хотек М. Обучающий курс для получения MCTS (экзамен 70-432): SQL Server 2008 — реализация и поддержка) и главу 18 в книге Hotek M. Microsoft SQL Server 2008 Step by Step. — Microsoft Press, 2008 (Хотек М. Microsoft SQL Server 2008 шаг за шагом).

3. Добавьте еще один шаг задания для отправки электронного почтового сообщения с присоединенным файлом результатов. Убедитесь, что с помощью параметра @profile\_name определен подходящий профиль.

## **Сценарий 2. Отслеживание изменений**

1. Средства отслеживания изменений потребляют меньше ресурсов и требуют меньше накладных расходов, чем CDC. Поскольку в отслеживаемых таблицах не выполняются команды UPDATE, нет необходимости в просмотре промежуточных изменений, а значит, и во включении системы CDC.
2. Два параметра отслеживания изменений на уровне базы данных, оказывающие влияние на требуемое дисковое пространство, — автоочистка (auto-cleanup) и период сохранения изменений (retention period). В зависимости от дневного расписания синхронизаций и ваших потребностей относительно возможности восстановления можно выбрать период сохранения между 26 и 50 часами.



# Словарь терминов

## F

**Filestream.** Свойство, которое может применяться к типу данных `varbinary(max)` и позволяет размещать реальные данные в структуре каталогов файловой системы вместо хранения их в файле или файлах базы данных.

**Filter daemon host process (Fdhost.exe).** Процесс, управляющий сторонними компонентами, такими как фильтры, средства разбиения по словам, модули парадигм, отделенными от процесса SQL Server.

**FOR XML.** Способ заставить SQL Server вернуть поток, формирующий документ или фрагмент XML.

## I

**INNER JOIN.** Соединение, включающее только строки, общие для всех таблиц, упомянутых в предложении `FROM` (см. также `JOIN` и `OUTER JOIN`).

## J

**JOIN.** Оператор языка T-SQL, который для получения результирующего набора соединяет несколько связанных таблиц. В синтаксической записи `JOIN` задается столбец или столбцы, общие для таблиц, используемых для связывания данных. (См. также `INNER JOIN` и `OUTER JOIN`).

## O

**OUTER JOIN.** Соединение, включающее все строки из внешней таблицы и любые строки из внутренней таблицы, которые соответствуют критерию соединения.

**S**

**SQLCLR.** Исполняемая среда внутри SQL Server, позволяющая создавать некоторые объекты с помощью языков платформы .NET, таких как C# или Visual Basic.NET.

**SQL Full-Text Filter Daemon Launcher (Fdlauncher.exe).** Служба, запускающая процессы Fdhost.exe, когда они нужны механизму полнотекстового поиска.

**U**

**UNION.** Оператор языка T-SQL, соединяющий результирующие наборы из множественных запросов.

**W**

**Well-known binary (WKB).** Двоичный эквивалент WKT, который иногда применяется для передачи и хранения пространственных данных.

**Well-known text (WKT).** Стандарт, созданный OGC и применяемый для представления описаний на текстовой основе геопространственных объектов.

**X**

**XML.** Спецификация консорциума World Wide Web Consortium для передачи и хранения слабоструктурированных данных.

---

**A**

**Атомарность (atomicity).** Если в транзакцию включено два или несколько фрагментов данных, фиксируются либо все, либо не одного.

**B**

**Верхняя граница (upper bound).** Задаёт конечный регистрационный номер транзакции (LSN), применяемый для определения корректного диапазона строк в таблице изменений.

**Выражение (term).** Слово, фраза или символьная строка, включенные как входной параметр в запрос полнотекстового поиска.

**Г**

**Группа событий (event group).** Набор событий DDL в иерархической структуре событий DDL.

**Группа сообщений (conversation group).** Коллекция связанных сообщений.

## Д

**Детерминированная функция** (deterministic function). Функция, возвращающая одно и то же значение при наличии одних и тех же входных параметров.

**Диалог** (dialog). Обмен сообщениями между двумя конкретными службами Service Broker, обеспечивающий строго однократную упорядоченную доставку сообщений.

**Длительность или долговечность** (durability). После фиксации транзакции окончательное состояние данных сохраняется, даже если сервер аварийно завершил работу или был перезагружен.

**Документ XML** (XML document). Правильно построенные данные XML, т. е. имеющие корневой элемент.

## З

**Заполнение** (population, crawl). Процесс добавления данных в (заполняемый) полнотекстовый индекс либо во время создания, либо когда полнотекстовый индекс построен.

## И

**Идентификатор пространственной ссылки** (Spatial reference identifier, SRID).

Идентификатор ссылки, связанный с конкретной моделью Земли. Эти идентификаторы ссылаются на стандарт системы идентификации Европейской группы нефтепоисковых исследований (European Petroleum Survey Group, EPSG).

**Идентификатор строки** (row identifier, RID). Числовое значение, представляющее физическое местоположение в куче и на странице.

**Идентифицирующий столбец** (Identity). Один столбец в таблице может быть определен как идентифицирующий. Свойство `IDENTITY` при вставке строк автоматически генерирует для столбца новый увеличенный или уменьшенный на единицу номер.

**Изоляция** (isolation). Во время выполнения транзакции (до ее фиксации или отката) данные должны оставаться изолированными и недоступными другим транзакциям. В SQL Server уровень изоляции может настраиваться для каждой транзакции.

**Индекс** (index). Физическая структура, реализованная в виде сбалансированного дерева и применяемая для ускорения выполнения запросов.

**Индексированное представление** (indexed view). Детерминированное представление, имеющее созданный для него кластеризованный индекс.

**К**

**Кластеризованный индекс** (clustered index). Индекс, вводящий физический порядок следования строк в таблице. В таблице может быть только один кластеризованный индекс.

**Командлеты** (cmdlets). Приложения .NET Framework, обеспечивающие специализированные функции в оболочке SQL Server PowerShell.

**Конечная точка Service Broker** (Service Broker endpoint). Применяется для настройки в SQL Server отправки и получения сообщений по протоколу TCP/IP.

**Контекст выполнения** (execution context). Регистрационные учетные данные (имя пользователя и пароль), под которыми выполняется модуль программного кода.

**Контракт** (contract). Соглашение между двумя службами, определяющее типы сообщений, которые службы посылают для выполнения определенных задач. Он также определяет, какие участники могут посылать, какие типы сообщений.

**Курсор** (cursor). Серверный объект, позволяющий извлекать и обрабатывать набор данных последовательно строка за строкой.

**Куча** (heap). Таблица без кластеризованного индекса. Куча может иметь определенные в ней некластеризованные индексы.

**М**

**Маркер** (token). Слово или символьная строка, определенные средством разбиения по словам.

**Маршрут** (route). Определяет, куда доставлять сообщения Service Broker. Если не задан, используется маршрут AutoCreatedLocal.

**Материализованное представление** (materialized view). Другое название индексированного представления.

**Методы** (methods). Действия, которые может совершать объект.

**Механизм полнотекстового поиска** (Full-Text Engine). Часть процесса SQL Server, управляющая административными задачами полнотекстового поиска, обеспечивает выполнение запросов полнотекстового поиска и управляет процессом.

**Модуль** (module). Набор строк программного кода, хранящийся в базе данных. К типам модулей, поддерживаемых SQL Server, относятся триггеры, функции, представления и хранимые процедуры.

**Модуль парадигм** (stemmer). Процесс, предоставляющий словоформы глаголов и существительных, основываясь на лингвистических правилах заданного языка данных.

## Н

**Недетерминированная функция** (nondeterministic function). Функция, которая может возвращать разные значения при одних и тех же заданных входных параметрах.

**Некластеризованный индекс** (nonclustered index). Индекс, который содержит один или несколько столбцов таблицы, применяемых для повышения производительности запроса. Если у таблицы есть кластеризованный индекс, корневой уровень некластеризованного индекса указывает на ключ кластеризации, в противном случае корневой уровень указывает на строку в таблице.

**Непротиворечивость** (consistency). В конце транзакции либо существует новый корректный вариант данных, либо данные возвращаются в исходное состояние. Возврат данных в исходное состояние выполняется операцией отката, обеспечиваемой транзакциями SQL Server.

**Несвязанный вложенный запрос** (noncorrelated subquery). Запрос, встроенный во внешний запрос, но не ссылающийся ни на какие столбцы из внешнего запроса.

**Нижняя граница** (lower bound). Определяет начальный регистрационный номер транзакции (LSN) для задания корректного диапазона строк в таблице изменений.

## О

**Объект** (object). Коллекция свойств и методов, обеспечивающая определенные функциональные возможности.

**Ограничение**. Правило, применяемое к столбцу (столбцам) в таблице для проверки допустимости изменений и гарантии того, что данные в таблице не нарушают бизнес-правила.

**Ограничение CHECK**. Ограничение, применяющее логическое выражение к добавляемым или обновляемым строкам в таблицах для проверки допустимости обновления.

**Ограничение FOREIGN KEY**. Ограничение, применяемое для обеспечения ссылочной целостности между таблицами (или даже в пределах одной таблицы).

**Ограничение PRIMARY KEY**. Ограничение, задающее один столбец или комбинацию столбцов в таблице, которые однозначно идентифицируют строку

таблицы. По определению первичный ключ не может содержать значения NULL.

**Ограничение UNIQUE.** Ограничение, устанавливаемое для одного или нескольких столбцов таблицы и гарантирующее, что данные в каждой строке в пределах таблицы уникальны в этом столбце или этих столбцах. Ограничение UNIQUE допускает значения NULL и, следовательно, не может применяться для однозначной идентификации строки, если не во всех столбцах, заданных в уникальном индексе, не допускаются значения NULL.

**Олицетворение (impersonation).** Процесс присваивания пользователем учетных данных другого пользователя.

**Открытый геопространственный консорциум (Open Geospatial Consortium, OGC).** Международная некоммерческая организация, разрабатывающая стандарты для географических и основанных на местоположении сервисов.

**Очередь (queue).** Скрытая таблица, в которой сообщения Service Broker хранятся до тех пор, пока не будут обработаны.

## П

**Параметр (parameter).** Переменная, передаваемая в хранимую процедуру или функцию.

**Переменная (variable).** Объект, применяемый для хранения скалярного значения для использования в функции, триггере или хранимой процедуре.

**Полнотекстовый индекс (full-text index).** Объект, содержащий значимые слова и их местонахождение в столбцах, включенных в индекс.

**Полнотекстовый каталог (full-text catalog).** Виртуальный объект, представляющий группу полнотекстовых индексов.

**Привязка удаленных служб (Remote service binding).** Обеспечивает безопасность диалогов, подключающихся к удаленным базам данных.

**Приоритет диалога (conversation priority).** Задаёт относительное старшинство диалогов компонента Service Broker.

## Р

**Распределенное секционированное представление (distributed partitioned view).** Особый вариант секционированного представления, в котором таблицы-участники хранятся в нескольких экземплярах SQL Server.

**Расширенные методы OGC (OGC Extended Methods).** Методы, разработанные для работы с типами данных *geometry* и *geography*.

**Регистрационный номер транзакции** (Log Sequence Number, LSN). Номер, присваиваемый записям журнала транзакций для облегчения операции восстановления.

**Рекурсивное ОТВ** (recursive CTE). Специальный тип обобщенного табличного выражения, у которого есть опорный запрос (anchor query), источник рекурсии, наряду с инструкцией `UNION ALL` и вторым запросом, который выполняет рекурсию опорного запроса, и внешний запрос, который ссылается на ОТВ и задает максимальный уровень рекурсии.

## С

**Связанный вложенный запрос** (correlated subquery). Запрос, встроенный во внешний запрос и ссылающийся на столбцы из внешнего запроса.

**Связь на уровне схемы** (schema binding). Доступный для представлений и функций параметр, гарантирующий, что никакие зависимые объекты не могут быть удалены, пока сначала не будут удалены представление или функция.

**Секционированное представление** (partitioned view). Представление, которое применяет предложение `UNION ALL` для объединения множественных таблиц-участников одинаковой структуры в единый результирующий набор.

**Системный идентификатор процесса** (system process ID, SPID). Числовое значение, присваиваемое каждому соединению с SQL Server.

**Скалярная функция** (scalar function). Функция, на которую ссылаются в предложении `SELECT`, принимает выражение и возвращает единственное значение заданного типа данных.

**Служба** (service). Имя, данное группе задач в решении Service Broker, требующем отправки сообщений.

**Создание экземпляра** (instantiation). Процесс создания конкретного экземпляра объекта, основанного на свойствах и методах объекта.

**Сообщение** (message). Содержит информацию, которая пересылается в диалоге Service Broker.

**Список стоп-слов** (stoplist). Объект базы данных, применяемый для управления стоп-словами. Существует системный список стоп-слов, и вы также можете создавать собственные списки стоп-слов.

**Средство разбиения по словам** (word breaker). Процесс, который находит границы слова (помечает слова) на основе лингвистических правил заданного языка данных. Язык данных в столбце можно задать во время создания полнотекстового индекса в таблице.

**Статистическая функция** (aggregate function). Встроенная или пользовательская функция, выполняющая вычисление на множестве строк или значений.

**Стоимость** (cost). Внутренняя мера, используемая SQL Server для определения объема ресурсов обработки, которые понадобятся данному плану выполнения запроса.

**Стоп-слово** (stopword). Часто используемое слово, не несущее дополнительного смысла в условии поиска, например артикли в английском языке. Стоп-слова в SQL Server 2008 играют ту же роль, что пропускаемые слова в SQL Server 2005.

**Страница** (page). Мельчайший объект хранения, поддерживаемый SQL Server; точный размер страницы равен 8 Кбайт.

**Схема секционирования** (partition scheme). Определяет файловую группу, в которой следует сохранять каждую секцию, заданную в функции секционирования.

## Т

**Таблицы изменений** (change tables). Таблицы в базе данных, сохраняющие измененные строки из базы данных с включенной CDC.

**Тезаурус** (thesaurus). Файлы XML, определяющие синонимы выражений на конкретном языке.

**Тип данных geography**. Применяется для хранения сферических (географических) координат, широты и долготы.

**Тип данных geometry**. Применяется для хранения плоскостных данных Эвклидова пространства, таких как точки, линии и многоугольники.

**Тип данных XML**. Тип данных, определенный в SQL Server и способный хранить фрагменты и документы XML, а также нетипизированные и типизированные (проверяемые на соответствие схеме) данные XML.

**Тип сообщения** (message type). Определяет имя и содержание сообщения.

**Триггер DDL** (DDL trigger). Триггер, который выполняется в ответ на событие DDL.

**Триггер DML** (DML trigger). Триггер, который выполняется в ответ на событие DML.

## Ф

**Фильтр** (filter). Компонент, обрабатывающий документ и извлекающий текстовую информацию из документов, хранящихся в столбце типа `varbinary(max)`



или `image`, и затем отправляющий эту информацию в средство разбиения по словам (`word breaker`).

**Фрагмент XML** (XML fragment). Данные XML, не являющиеся правильно построенным документом XML, т. е. не имеющие корневого элемента.

**Функция** (function). Именованный программный объект из инструкций на языке SQL, который может вызываться в предложении `SELECT` или `FROM` и возвращает единичное (скалярное) значение или значение табличного типа, но не имеет права модифицировать состояние базы данных или экземпляра сервера.

**Функция, возвращающая табличное значение** (table-valued function). Функция, возвращающая результаты табличного типа данных.

**Функция, определенная пользователем** (user-defined function). Функция на языке T-SQL или CLR-коде. Пользовательские функции на T-SQL могут быть скалярными, многооператорными табличными и встраиваемыми табличными функциями.

**Функция секционирования** (partition function). Функция, определяющая значения, которые делят таблицу или индекс на несколько секций.

## Х

**Хранимая процедура** (stored procedure). Модуль программного кода, который хранится в базе данных и предоставляет прикладной интерфейс (API), который могут использовать приложения для отделения структуры базы данных от программного кода приложения.

## Ш

**Шлюзовая роль** (gating role). Роль, задаваемая при включении CDC и применяемая для ограничения доступа к измененным данным.

# Системные требования

Для выполнения упражнений, приведенных в этой книге, рекомендуется использовать компьютер, не являющийся вашей основной рабочей станцией, поскольку вы будете вносить изменения в настройки операционной системы и Microsoft SQL Server.

## Аппаратные требования

Для выполнения большинства практических заданий из этой книги необходима одна машина. Ваши компьютеры или виртуальные машины должны соответствовать следующим минимальным аппаратным спецификациям:

- ◆ процессор Pentium III или более быстрый с тактовой частотой 1.0 ГГц для 32-разрядной и 1.6 ГГц для 64-разрядной версии;
- ◆ 512 Мбайт RAM;
- ◆ 20 Гбайт свободного пространства на жестком диске;
- ◆ дисковод DVD-ROM;
- ◆ монитор Super VGA с разрешением 800×600 или выше;
- ◆ клавиатура и мышь Microsoft или совместимый манипулятор.

## Программные требования

Для выполнения практических упражнений требуется следующее программное обеспечение:

- ◆ платформа Microsoft .NET Framework 3.5;
- ◆ любая из следующих операционных систем:
  - Windows XP Professional Home, Tablet, Media Center, or Professional with SP2;

- Windows Vista SP1;
- Windows Server 2003 Standard or Enterprise with SP2;
- Windows Server 2008 Standard or Enterprise;
- ◆ интерфейс доступа к данным Microsoft Data Access Components (MDAC) 2.8 SP1 или более поздняя версия;
- ◆ Shared Memory, Named Pipes или Transmission Control Protocol/Internet Protocol (TCP/IP);
- ◆ сетевая поддержка;
- ◆ Web-обозреватель Microsoft Internet Explorer 6.0 SP1 или Windows Internet Explorer 7.0 или более поздняя версия.

# ПРИЛОЖЕНИЕ

## Описание компакт-диска

### Как использовать этот компакт-диск

#### Меню AutoRun CD

Для использования компакт-диска вставьте его в дисковод CD-ROM. На экране появится меню. Если на вашем компьютере не включен автозапуск, для вывода на экран стартового меню запустите файл StartCD.exe в корневом каталоге компакт-диска. Меню содержит ссылки на все ресурсы, имеющиеся на этом компакт-диске, а также на Web-сайт технической поддержки издательства Microsoft Press.

#### Электронные обозреватели файлов

Электронная версия книги и некоторая другая информация на сопроводительном компакт-диске представлена в формате Portable Document Format (PDF). Для просмотра этих файлов вам необходима программа Adobe Acrobat или Adobe Reader. Дополнительную информацию см. на Web-сайте компании Adobe <http://www.adobe.com>.

### Что на этом компакт-диске?

Данный сопроводительный компакт-диск содержит:

- ◆ электронную версию книги "MCTS Self-Paced Training Kit (Exam 70-433): Microsoft SQL Server 2008 Database Development";
- ◆ тренировочные тесты;
- ◆ файлы программного кода;
- ◆ базы данных;
- ◆ дополнительные главы.

## Электронная версия книги "MCTS Self-Paced Training Kit (Exam 70-433): Microsoft SQL Server 2008 Database Development"

Полный текст книги помещен на этот компакт-диск в доступном для поиска формате PDF. Для просмотра электронной версии книги откройте PDF-файл в папке \eBook.

### **Примечание**

Электронная книга представлена в формате Portable Document Format (PDF). Для просмотра этого файла вам нужна программа Adobe Acrobat или Adobe Reader. За дополнительной информацией об этих программных продуктах или за файлами для загрузки обращайтесь на Web-сайт компании Adobe по адресу <http://www.adobe.com>.

## Тренировочные тесты

Модуль подготовки к экзамену обучающего курса Microsoft Press, включающий тренировочные тесты к экзамену 70-433, находится на этом компакт-диске в папке \Practice Tests. Для его установки на ваш жесткий диск запустите файл Setup.exe в папке \Practice Tests и следуйте инструкциям, появляющимся на экране.

Для удаления модуля подготовки к экзамену Training Kit выберите соответствующую программу в списке программ апплета **Установка и удаление программ** в Панели управления.

## Файлы программного кода

Для копирования примеров программного кода на ваш компьютер запустите инсталлятор и следуйте инструкциям, появляющимся на экране.

Для удаления примеров программного кода выберите нужную строку в списке программ апплета **Установка и удаление программ** в Панели управления.

## Базы данных

Файлы баз данных Northwind, AdventureWorks2008 и AdventureWorksDW2008, необходимые для выполнения упражнений из этой книги, находятся на данном компакт-диске в папке \Databases.

## Дополнительные главы

На компакт-диск помещены дополнительные главы из разных книг издательства Microsoft Press, посвященных SQL. Для их просмотра откройте PDF-файлы в папке \Bonus Material\Sample Chapters.

**Примечание**

Дополнительные главы представлены в формате Portable Document Format (PDF). Для просмотра этих файлов вам нужна программа Adobe Acrobat или Adobe Reader. За дополнительной информацией об этих программных продуктах или за файлами для загрузки обращайтесь на Web-сайт компании Adobe по адресу <http://www.adobe.com>.

## **Информация о технической поддержке**

### **Microsoft Learning Technical Support**

Для устранения ошибок и опечаток из книги и содержимого данного компакт-диска были предприняты все возможные усилия. По мере накопления корректировки и изменения будут помещены в раздел базы знаний Microsoft Knowledge Base.

Microsoft Press оказывает техническую поддержку книг и сопроводительных компакт-дисков на следующем Web-сайте: <http://www.microsoft.com/learning/support/books/>.

Если у вас есть замечания, вопросы или предложения, касающиеся книги или этого компакт-диска, или вопросы, на которые вы не нашли ответов на указанном Web-сайте, пожалуйста, отправляйте их издательству Microsoft Press по электронной почте на адрес: [TKinput@microsoft.com](mailto:TKinput@microsoft.com).

Имейте в виду, что по указанным адресам не оказывается техническая поддержка программных продуктов Microsoft.

### **Заявление об отсутствии гарантий: стороннее ПО или ссылки на сторонние Web-сайты**

Для удобства пользователей на этом компакт-диске может содержаться программное обеспечение или ссылки на сайты сторонних организаций.

Имейте в виду, что эти продукты не контролируются корпорацией Microsoft, и, следовательно, Microsoft не несет ответственности за их содержимое, и их включение в содержимое данного компакт-диска не следует считать свидетельством технической поддержки (endorsement) этого программного продукта. Самые последние версии стороннего программного обеспечения проверяйте на Web-сайтах соответствующих организаций.

# Предметный указатель

## D

Database Engine Tuning Advisor 265

DMF:

◇ sys.dm\_fts\_index\_population 362

◇ sys.dm\_fts\_parser 363

DML 60

## F

Filter daemon host process (Fdhost.exe)  
356, 357

## O

OLTP 83

## S

Service Broker 374, 400

◇ активация:

▫ внутренняя 379

▫ основанная на событиях 379

SQL Full-Text Filter Daemon Launcher  
(Fdlauncher.exe) 356, 357

SQL Server Audit 444

SQL Server process (Sqlservr.exe) 355

SQL Server Profiler 230

SQL-инъекция 176

## W

Windows PowerShell 413

## A

Аргумент поиска 221

Атомарность 83

Атрибут:

◇ PERSISTED 313

◇ SqlFacet 324

◇ SqlFunction 312

◇ SqlUserDefinedType 324

## Б

База данных:

◇ master 186

◇ mssqlsystemresource 186

Блок:

◇ BEGIN...END 188

◇ TRY...CATCH 171

Блокировка:

◇ диапазона ключей 88

◇ массового обновления 88

◇ монопольная 88

◇ обновления 88

◇ с намерением 88

◇ совмещаемая 87

◇ схемы 88

## В

Взаимоблокировка 89

Включенные столбцы 239

Выражения FLWOR 301

Выходной параметр 166

**Г**

Глобальная переменная @@trancount 85

Группа событий 196

**Д**

Диалог 375, 386

Диапазон 18

Динамическое административное  
представление 257

Динамическое выполнение 175

Директива:

◇ ELEMENTS 288

◇ ROOT 287

◇ XSINIL 288

Диск SQLSERVER 414

Длительность 83

Документ XML 279

**З**

Запрос:

◇ время выполнения 217

◇ настройка 215

◇ несвязанный вложенный 145

◇ оптимизация 220

◇ связанный вложенный 145, 148

◇ стоимость выполнения 216

Значение NULL 16, 112

**И**

Идентификатор пространственной  
ссылки 347

Изоляция 83

Индекс:

◇ кластеризованный 236, 245

◇ некластеризованный 236, 246

◇ покрытия 236

Инструкция:

◇ ALTER DATABASE 195, 380, 426

◇ ALTER INDEX DISABLE 429

◇ ALTER PROCEDURE 162

◇ ALTER SERVER AUDIT 446

◇ ALTER TABLE 428

◇ ALTER TABLE ADD COLUMN 429

◇ ALTER TABLE ALTER COLUMN  
429

◇ ALTER TABLE DROP COLUMN 429

◇ ALTER TABLE DROP CONSTRAINT  
429

◇ ALTER TABLE SWITCH 430

◇ BEGIN DIALOG 377, 386

◇ BEGIN TRANSACTION 84

◇ BREAK 168

◇ CLOSE 177

◇ COMMIT TRANSACTION 84

◇ CONTINUE 168

◇ CREATE ASSEMBLY 308

◇ CREATE BROKER PRIORITY 386

◇ CREATE CONTRACT 384

◇ CREATE DATABASE AUDIT  
SPECIFICATION 446

◇ CREATE FULLTEXT CATALOG 358

◇ CREATE FULLTEXT INDEX 359

◇ CREATE INDEX 260

◇ CREATE MESSAGE TYPE 381

◇ CREATE PARTITION FUNCTION  
258

◇ CREATE PARTITION SCHEME 259

◇ CREATE QUEUE 382

◇ CREATE RULE 121

◇ CREATE SCHEMA 109

◇ CREATE SERVER AUDIT 444

◇ CREATE SERVICE 385

◇ CREATE TABLE 109, 111

◇ DBCC LOG 92

◇ DEALLOCATE 177

◇ DECLARE 164, 177

◇ DELETE 67, 179

◇ DROP INDEX 429

◇ DROP TABLE 429

◇ EXEC 174

◇ FETCH 177, 180

◇ GOTO 169

◇ INSERT 63

◇ MERGE 73, 76

◇ OPEN 177

◇ RAISERROR 170

◇ RECEIVE 388



- ◇ RETURN 166, 188
- ◇ ROLLBACK 84
- ◇ ROLLBACK TRANSACTION 84
- ◇ SAVE TRANSACTION 85
- ◇ SELECT 14, 15, 164, 166, 177, 188, 217, 254
- ◇ SELECT INTO 65, 109
- ◇ SEND 388
- ◇ SET STATISTICS TIME ON 217, 226
- ◇ SET TRANSACTION ISOLATION LEVEL 93
- ◇ TRUNCATE TABLE 69, 429
- ◇ UPDATE 65, 179
- ◇ WAITFOR 168
- ◇ WHILE 168

## К

Ключ:

- ◇ внешний 122
- ◇ естественный 113
- ◇ первичный 113

Ключевое слово:

- ◇ COLLATE 252
- ◇ GO 163
- ◇ OUT 175
- ◇ OUTPUT 175

Код возврата 166

Количество считываний страниц 216

Команда yield return 229

Командлет:

- ◇ Convert-UrnToPath 419
- ◇ Decode-SqlName 419
- ◇ Encode-SqlName 419
- ◇ Get-ChildItem 416
- ◇ Get-Help 417
- ◇ Get-Item 416
- ◇ Get-Location 416
- ◇ Invoke-Sqlcmd 419
- ◇ Move-Item 416
- ◇ Remove-Item 416
- ◇ Rename-Item 416
- ◇ Set-Location 416

Комментарий 162

Конечные точки 377

Контракт 375, 376, 384

Курсор 177, 229

- ◇ DYNAMIC 179
  - ◇ FAST\_FORWARD 179
  - ◇ KEYSET 179
  - ◇ STATIC 179
- Куча 92, 236

## Л

Логический оператор:

- ◇ BETWEEN 20
- ◇ EXISTS 20
- ◇ LIKE 18

Логическое считывание 216

## М

Маршрут 376

- ◇ AutoCreatedLocal 376

Метод:

- ◇ LINESTRING 352
- ◇ Parse 349
- ◇ Point 349
- ◇ STGEOMFROMTEXT 349
- ◇ STLineFromText 352
- ◇ value() 196

## Н

Набор:

- ◇ параметров символьной обработки 102, 252
- ◇ разрешений 332

## О

Обобщенное табличное выражение (ОТВ) 28, 142

- ◇ рекурсивное 142

Объект:

- ◇ sys.dm\_tran\_active\_transactions 87
- ◇ sys.dm\_tran\_current\_transaction 86

Ограничение:

- ◇ CHECK 125, 190
- ◇ FOREIGN KEY 122
- ◇ PRIMARY KEY 121
- ◇ UNIQUE 121

## Оператор:

- ◇ APPLY 46
- ◇ CROSS APPLY 26
- ◇ CROSS JOIN 27
- ◇ EXCEPT 45
- ◇ EXISTS 223
- ◇ FULL OUTER JOIN 26
- ◇ GROUPING SETS 37
- ◇ INNER JOIN 24
- ◇ INTERSECT 45
- ◇ JOIN 24
- ◇ LEFT OUTER JOIN 25, 26
- ◇ OUTER APPLY 46
- ◇ OUTER JOIN 25
- ◇ RIGHT OUTER JOIN 26
- ◇ UNION 43, 44, 219
- ◇ WITH CUBE 35
- ◇ WITH ROLLUP 34

## Определение таблицы 62

## Отслеживание изменений 424

## Очередь 375

## Ошибка:

- ◇ номер 169
- ◇ уровень серьезности 169

## П

## Параметр 165

- ◇ ANSI\_NULLS 16, 205
- ◇ ANSI\_PADDING 204
- ◇ CALLED ON NULL INPUT 188
- ◇ 'CLR enabled' 306
- ◇ ENCRYPTION 162, 188
- ◇ Filestream 332
- ◇ FORWARD\_ONLY 179
- ◇ NOT FOR REPLICATION 194
- ◇ ON ALL SERVER 195
- ◇ ON DATABASE 195
- ◇ RECOMPILE 181
- ◇ RETENTION 375
- ◇ RETURNS NULL ON NULL INPUT 188
- ◇ SCHEMABINDING 188
- ◇ SCROLL 180
- ◇ sp\_configure 333
- ◇ VIEW\_METADATA 202

- ◇ WITH CHECK 133

- ◇ XACT\_ABORT 172, 174

## Параметр XML:

- ◇ DOCUMENT 297
- ◇ TYPE 296

## Переменная 163

- ◇ глобальная 163
  - @@ERROR 171
  - @@FETCH\_STATUS 178
  - @@IDENTITY 164
  - @@TRANCOUNT 174
- ◇ локальная 163

## План выполнения 131, 181

- ◇ графический 220

## Подсказка оптимизатора

- WITH(NOEXPAND) 256

## Полнотекстовый поиск 354

## Предикат:

- ◇ CONTAINS 366
- ◇ FREETEXT 369

## Предложение:

- ◇ APPLY 224
- ◇ EXECUTE AS 176
- ◇ FOR XML 286
- ◇ GROUP BY 31, 33
- ◇ HAVING 40, 148
- ◇ ON 222, 260
- ◇ OPTION 143
- ◇ ORDER BY 21, 44
- ◇ OUTPUT 73, 74
- ◇ PARTITION BY 151
- ◇ WHERE 16, 49, 203, 222
- ◇ WHERE CURRENT OF 179
- ◇ WITH 143
- ◇ WITH CHECK OPTION 203
- ◇ WITH XMLNAMESPACES 297

## Представление 201, 202

- ◇ индексированное 205
- ◇ распределенное секционированное 204
- ◇ секционированное 203

## Представление каталога:

- ◇ sys.assemblies 309
- ◇ sys.assembly\_files 310
- ◇ sys.assembly\_modules 309
- ◇ sys.change\_tracking\_tables 428

- ◇ sys.databases 436
- ◇ sys.fulltext\_index\_columns 363
- ◇ sys.messages 170
- ◇ sys.sql\_modules 309

## С

- Самосоединение 28
- Свойство:
  - ◇ IDENTITY 62, 114
  - ◇ соединения
  - SET\_IMPLICIT\_TRANSACTIONS 63
- Секционирование 258
- Сжатие на уровне строки 115
- Система отслеживания измененных данных 424
- Служба 375, 385
  - ◇ иницилирующая 375
  - ◇ целевая 375
- Событие:
  - ◇ RPC:Completed 230
  - ◇ SQL:BatchCompleted 230
- Согласованность 83
- Соединение:
  - ◇ внешнее 25
  - ◇ внутреннее 25
  - ◇ перекрестное 27
  - ◇ самосоединение 28
- Сообщение 375, 386, 388
- Столбец, вычисляемый 250
- Схема 109
  - ◇ секционирования 259
  - ◇ таблицы 62

## Т

- Таблица:
  - ◇ deleted 195
  - ◇ inserted 195
  - ◇ виртуальная 147
  - ◇ внешняя 25
  - ◇ изменений 424
  - ◇ производная 146
- Тип данных:
  - ◇ bigint 32, 103
  - ◇ char 102

- ◇ date 105, 107
- ◇ datetime 105
- ◇ datetime2 105, 108
- ◇ datetimeoffset 105, 108
- ◇ decimal 104
- ◇ float 104
- ◇ geography 346
- ◇ geometry 346
- ◇ int 103
- ◇ nchar 103
- ◇ ntext 102
- ◇ numeric 104
- ◇ nvarchar 103
- ◇ real 105
- ◇ smalldatetime 105
- ◇ smallint 103
- ◇ text 102
- ◇ time 105, 107
- ◇ tinyint 103
- ◇ varchar 102
- ◇ XML 297
- ◇ нерекомендуемый 164
- Тип сообщения 376
- Транзакция 60, 83
- Триггер 193
  - ◇ AFTER 194
  - ◇ DDL 195
  - ◇ DML 194
  - ◇ INSTEAD OF 194
  - ◇ входа 197

## Ф

- Фрагмент XML 279
- Функция 185
  - ◇ @@ERROR 53
  - ◇ @@TRANCOUNT 53
  - ◇ AVG 31
  - ◇ CAST 130
  - ◇ cdc.fn\_cdc\_get\_net\_changes\_<capture\_instance> 439
  - ◇ CHANGE\_TRACKING\_CURRENT\_VERSION 430
  - ◇ CHANGE\_TRACKING\_IS\_COLUMN\_IN\_MASK 430

Функция (*прод.*):

- ◇ CHANGE\_TRACKING\_MIN\_VALID\_VERSION 430
- ◇ CHANGETABLE 430
- ◇ CHARINDEX 54
- ◇ CHECKSUM\_AGG 31
- ◇ CONTAINSTABLE 366
- ◇ CONVERT 52
- ◇ COUNT 32
- ◇ COUNT\_BIG 32
- ◇ DATALENGTH 54
- ◇ DATEADD 51
- ◇ DATEDIFF 51
- ◇ DATENAME 51
- ◇ DATEPART 51
- ◇ db\_id 49
- ◇ DENSE\_RANK 152
- ◇ ERROR\_LINE 173
- ◇ ERROR\_MESSAGE 173
- ◇ ERROR\_NUMBER 173
- ◇ ERROR\_PROCEDURE 173
- ◇ ERROR\_SEVERITY 173
- ◇ ERROR\_STATE 173
- ◇ EVENTDATA 196
- ◇ fn\_helpcollations 102
- ◇ fn\_trace\_gettable 230
- ◇ FREETEXTTABLE 370
- ◇ FULLTEXTCATALOGPROPERTY 362
- ◇ GROUPING 32, 36
- ◇ ISNULL 50
- ◇ MAX 32
- ◇ MIN 32
- ◇ NTILE 152
- ◇ PATINDEX 54
- ◇ RANK 152
- ◇ ROW\_NUMBER 151, 225
- ◇ SCOPE\_IDENTITY() 164
- ◇ STDEV 32
- ◇ STDEVP 32
- ◇ SUM 32
- ◇ sys.dm\_db\_index\_physical\_stats 241
- ◇ sys.dm\_fts\_parser 362
- ◇ sys.fn\_cdc\_decrement\_lsn 443
- ◇ sys.fn\_cdc\_get\_column\_ordinal 443
- ◇ sys.fn\_cdc\_get\_max\_lsn 443
- ◇ sys.fn\_cdc\_get\_min\_lsn 444
- ◇ sys.fn\_cdc\_has\_column\_changed 443
- ◇ sys.fn\_cdc\_increment\_lsn 443
- ◇ sys.fn\_cdc\_is\_bit\_set 443
- ◇ sys.fn\_cdc\_map\_lsn\_to\_time 443
- ◇ sys.fn\_cdc\_map\_time\_to\_lsn 443
- ◇ ToString 349
- ◇ VAR 32
- ◇ VARP 32
- ◇ WITH CHANGE\_TRACKING\_CONTEXT 430
- ◇ XACT\_STATE 174
- ◇ встраиваемая табличная 187, 228
- ◇ встроенная 49
- ◇ детерминированная 206
- ◇ многооператорная 187
  - табличная 228
- ◇ недетерминированная 206
- ◇ пользовательская 126, 186
- ◇ секционирования 258
- ◇ системная 185
- ◇ скалярная пользовательская 225
- ◇ строковая 53

**X**

## Хранимая процедура 161

- ◇ configure 399
- ◇ sp\_sysmail\_help\_queue\_sp 408
- ◇ sp\_addmessage 169
- ◇ sp\_altermessage 170
- ◇ sp\_dropmessage 170
- ◇ sp\_executesql 176
- ◇ sp\_send\_dbmail 398, 402
- ◇ sp\_settriggerorder 194
- ◇ sys.sp\_cdc\_enable\_db 436
- ◇ sys.sp\_cdc\_enable\_table 436
- ◇ sysmail\_configure\_sp 408
- ◇ sysmail\_delete\_log\_sp 408
- ◇ sysmail\_delete\_mailitems\_sp 408
- ◇ sysmail\_help\_configure\_sp 408
- ◇ sysmail\_start\_sp 408
- ◇ sysmail\_stop\_sp 408

**Ц**

Целостность данных:

- ◇ декларативная 121
- ◇ процедурная 121

**Ш**

Шлюзовая роль 438

**Э**

Элемент NIL 288

**Я**

Явная транзакция 84

Язык:

- ◇ XML 278
- ◇ манипулирования данными 60

Microsoft®

# SQL Server® 2008

## Разработка баз данных

### MCTS

**Учебный  
курс**  
Microsoft®

Экзамен **70-433**

### Официальное пособие для самоподготовки

Освоив материалы этого курса и опробовав полученные знания на реальных сценариях, вы успешно подготовитесь к сдаче экзамена № 70-433 по программе сертификации MCTS.

#### Вы научитесь:

- создавать объекты базы данных и управлять ими;
- запрашивать и изменять данные;
- формировать вложенные запросы;
- оптимизировать структуры таблиц и целостность данных;
- создавать хранимые процедуры, функции и триггеры;
- управлять транзакциями, обрабатывать ошибки и отслеживать изменения;
- настраивать производительность запросов;
- реализовывать электронную почту базы данных, полнотекстовый поиск, решения средствами Service Broker, сценарии;
- работать с XML и SQLCLR.

#### Авторы:

**Тобиаш Тернстрем**, специалист с 13-летним опытом работы с SQL Server, менеджер проекта в подразделении SQL Engine Platform Team корпорации Microsoft, разработчик программ сертификации Microsoft для SQL Server.

**Энн Вебер**, MCITP, MCSE, инструктор, консультант и автор с более чем 12-летним опытом работы с SQL Server.

**Майк Хотек**, эксперт SQL Server с 30-летним опытом работы в качестве консультанта и разработчика, автор нескольких книг.



#### На прилагаемом компакт-диске:

- электронная версия книги на английском языке в формате PDF;
- тренировочные тесты;
- файлы программного кода;
- базы данных;
- дополнительные главы из других книг на английском языке издательства Microsoft Press, посвященные SQL Server 2008, в формате PDF.

ISBN 978-5-7502-0394-9



9 785750 203949



Microsoft®  
**SQL Server® 2008**

Издательство

**Русская Редакция**

Москва

Шелепихинская наб., 32

E-mail: [info@rusedit.com](mailto:info@rusedit.com)

Internet: [www.rusedit.com](http://www.rusedit.com)

Тел./факс: (499) 256-7145

 РУССКАЯ РЕДАКЦИЯ